

Scenario Library — 800 Round-2/3 Questions

Java Interview Prep — 2026 Edition · Learn & Excel

This is the round-2/3 scenario file of the pack. 7 scenario categories — debug, design, what-will-output, performance, concurrency bugs, refactor, real-world tradeoffs. Each scenario follows the same format: tags · question · 30-second answer · step-by-step thought process · edge cases · what NOT to say · common follow-up.

Table of Categories

- Category 1 — Debug This Code
 - Category 2 — Design This
 - Category 3 — What Will This Output
 - Category 4 — Memory & Performance Scenarios
 - Category 5 — Concurrency Bugs
 - Category 6 — Refactor This
 - Category 7 — Real-World Tradeoffs
-

Category 1 — Debug This Code

150 Java code snippets that look harmless and aren't. Each scenario is a real bug type that ships to production: race conditions, broken equals contracts, autoboxing traps, stream side-effects, leaked resources, swallowed exceptions, and the rest of the lineup. The job is to spot the bug, name the failure mode, and propose the fix the way you would in an interview room.

Scenario 1 · The counter that lies under load

Asked at: Razorpay · Swiggy · PhonePe · **Difficulty:** Easy · **Topic:** Concurrency race

The code:

```
public class PaymentCounter {
    private int successCount = 0;

    public void recordSuccess() {
        successCount++;
    }

    public int getSuccessCount() {
        return successCount;
    }
}
```

The interviewer's prompt: "We deployed this to track successful payments. The dashboard shows 9,847 successes for the day but Kafka has 10,000 success events. Where did 153 go?"

Thinking out loud (what to say while you stare at the screen): 1. "Counter going up — single-threaded this is fine. But the question mentions Kafka events, which means many consumers. Multi-threaded." 2. " `successCount++` looks atomic but isn't — it's read, add one, write back. Three bytecodes." 3. "Two threads can both read 100, both write 101. One increment is lost. Over 10,000 events you lose hundreds. Matches the gap."

The actual bug: `successCount++` is not atomic. Under concurrent calls, lost updates accumulate because the read-modify-write sequence is interleaved. There's also a visibility issue — `getSuccessCount()` may return a stale value because `successCount` isn't `volatile`.

The fix (1-3 options if applicable): - **Option A:** Use `AtomicInteger` and call `successCount.incrementAndGet()`. CAS-based, lock-free, perfect for a single counter. - **Option B:** Use `LongAdder` if the counter is hot (many writers, infrequent reads). LongAdder shards the count across cells to reduce CAS contention. - **Option C:** `synchronized` on the method works but is the heaviest. Fine if you have other invariants to protect; overkill for a counter.

What NOT to say: - "Mark it `volatile` and it's fixed" — `volatile` fixes visibility, not atomicity. `volatile int` still loses updates on `++`. - "Wrap it in a `try-catch`" — wrong category of bug entirely.

Common follow-up: "What's the difference between `AtomicInteger` and `LongAdder` under heavy contention?"

Scenario 2 · The order that never expires

Asked at: Meesho · Flipkart · **Difficulty:** Medium · **Topic:** equals/hashCode contract

The code:

```
public class OrderId {
    private final String value;
    public OrderId(String value) { this.value = value; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof OrderId)) return false;
        return value.equals(((OrderId) o).value);
    }
}

Map<OrderId, Order> pendingOrders = new HashMap<>();
pendingOrders.put(new OrderId("ORD-42"), order);
Order found = pendingOrders.get(new OrderId("ORD-42")); // null
```

The interviewer's prompt: "We use `OrderId` as a `HashMap` key for pending orders. Lookups return null even when we just inserted the order. What's going on?"

Thinking out loud: 1. "They overrode `equals` but I don't see `hashCode`. That's the contract: equal objects must have equal hash codes." 2. "Without `hashCode` override, it inherits `Object.hashCode` — based on identity. Two `new OrderId("ORD-42")` instances hash to different buckets." 3. "HashMap looks in the wrong bucket, never finds the entry. `get()` returns null."

The actual bug: `equals` was overridden but `hashCode` was not. `HashMap/HashSet` bucket entries by `hashCode` first, then `equals` within the bucket. Different hashes mean the lookup never reaches the `equals` check.

The fix (1-3 options if applicable): - **Option A:** Add

`@Override public int hashCode() { return value.hashCode(); }`. Simple and matches `equals`. - **Option B:** Use `Objects.hash(value)` if `equals` uses multiple fields — it auto-handles nulls. - **Option C:** Make `OrderId` a `record` — records auto-generate `equals` and `hashCode` that match exactly.

What NOT to say: - "HashMap is broken in this case" — no, the class is. - "Use `TreeMap`, it's safer" — `TreeMap` uses `Comparator/Comparable`, not `equals/hashCode`. Different problem.

Common follow-up: "What changes if you mutate `value` after putting the key into the map?"

Scenario 3 · The cache that never shrinks

Asked at: Cred · JP Morgan · **Difficulty:** Medium · **Topic:** Resource leak / Memory

The code:

```
public class CustomerCache {
    private final Map<Long, Customer> cache = new HashMap<>();

    public Customer get(Long customerId) {
        return cache.computeIfAbsent(customerId, this::loadFromDb);
    }

    private Customer loadFromDb(Long id) { /* fetch */ }
}
```

The interviewer's prompt: *"This cache is in our checkout service. Memory grows steadily for two days then we OOM. What's missing?"*

Thinking out loud: 1. "A plain HashMap with no eviction. Every customer that hits the service stays in memory forever." 2. "Two days of unique customers in a busy service — millions of entries. Each Customer object holds onto its address list, order history etc. Heap grows linearly until OOM." 3. "Also — HashMap is not thread-safe; under concurrent access this can spin or corrupt entries."

The actual bug: Unbounded cache with no eviction policy. Every lookup adds an entry and nothing removes them. The cache is also non-thread-safe.

The fix (1-3 options if applicable): - **Option A:** Use Caffeine —

`Caffeine.newBuilder().maximumSize(10_000).expireAfterWrite(Duration.ofMinutes(15)).build()`. Bounded, thread-safe, battle-tested. - **Option B:** LinkedHashMap with access-order + override `removeEldestEntry`. Works but you also need synchronization wrapping. - **Option C:** Guava's `CacheBuilder` — same idea as Caffeine; Caffeine is the modern successor.

What NOT to say: - "Increase heap to 16 GB and it's fine" — kicks the OOM down the road; eventually still leaks. - "Use `WeakHashMap`, JVM will clean it" — weak keys are based on identity (`new Long(id)` would be a new key); rarely behaves the way people expect.

Common follow-up: *"How does Caffeine evict — TinyLFU vs LRU?"*

Scenario 4 · The list that won't add a string

Asked at: TCS · Infosys · **Difficulty:** Easy · **Topic:** Arrays.asList vs List.of

The code:

```
List<String> skus = Arrays.asList("SKU-1", "SKU-2", "SKU-3");
skus.add("SKU-4"); // UnsupportedOperationException
```

The interviewer's prompt: *"Why does this throw? Both list types should support add, right?"*

Thinking out loud: 1. " `Arrays.asList` doesn't return a `java.util.ArrayList` . It returns a fixed-size list backed by the array." 2. "Fixed size means you can `set` an index but not add or remove. `add()` throws `UnsupportedOperationException` ." 3. "Probably copy-pasted from a tutorial; very common confusion."

The actual bug: `Arrays.asList` returns an inner class `Arrays$ArrayList` that does not implement `add/remove`. It mirrors the underlying array, which has a fixed length.

The fix (1-3 options if applicable): - **Option A:** `new ArrayList<>(Arrays.asList(...))` — wraps it in a real, growable `ArrayList`. - **Option B:** `new ArrayList<>(List.of(...))` if Java 9+. `List.of` returns a truly immutable list; the copy gives you mutability. - **Option C:** If you don't need to mutate, prefer `List.of(...)` directly — clearer intent and immutable by design.

What NOT to say: - "It's a JVM bug" — it's documented behaviour. - "Just use raw types" — won't fix it and introduces unchecked-warning hell.

Common follow-up: "What's the difference between `Arrays.asList` , `List.of` , and `Collections.unmodifiableList` ?"

Scenario 5 · The price that compares unequal

Asked at: Zerodha · Groww · **Difficulty:** Medium · **Topic:** `BigDecimal` equals vs `compareTo`

The code:

```
BigDecimal expected = new BigDecimal("100.00");
BigDecimal actual   = new BigDecimal("100");

if (expected.equals(actual)) {
    // never enters
    confirmOrder();
}
```

The interviewer's prompt: "Our trade reconciliation says prices don't match. Both are 100 rupees. Why does `equals` return false?"

Thinking out loud: 1. "`BigDecimal` carries scale as part of identity. `100.00` has scale 2; `100` has scale 0." 2. " `equals` considers both value AND scale equal. So `100.00` and `100` are not equal even though they're numerically the same." 3. "The fix is `compareTo == 0` , which compares numerical value only."

The actual bug: `BigDecimal.equals` requires the same scale. Use `compareTo` for numerical equality.

The fix (1-3 options if applicable): - **Option A:** `expected.compareTo(actual) == 0` — numerical comparison, scale-agnostic. - **Option B:** Normalize first with `stripTrailingZeros()` then equals — fragile because `100` stripped becomes `1E2` and you may not want that representation. - **Option C:** If you always know the scale (currency), `setScale(2, RoundingMode.HALF_EVEN)` on both before equals.

What NOT to say: - "BigDecimal is buggy" — it's by design; documented in the Javadoc. - "Use `==`" — never. Reference equality is meaningless for BigDecimal.

Common follow-up: "What rounding mode would you pick for an Indian stock exchange settlement?"

Scenario 6 · The for-loop that throws ConcurrentModificationException

Asked at: Wipro · Accenture · Cognizant · **Difficulty:** Easy · **Topic:** ConcurrentModificationException

The code:

```
List<Order> orders = new ArrayList<>(fetchOrders());
for (Order order : orders) {
    if (order.isCancelled()) {
        orders.remove(order);
    }
}
```

The interviewer's prompt: "This was working in dev. In prod with real data it throws ConcurrentModificationException. Same single thread. Why?"

Thinking out loud: 1. "Single thread but still CME — that's the classic fail-fast iterator behaviour." 2. "Enhanced for-loop uses an Iterator under the hood. The iterator records a `modCount` snapshot when created. Any modification through the list (not the iterator) bumps modCount." 3. "Next call to `iterator.next()` checks modCount vs expected and throws CME. In dev maybe no orders were cancelled; in prod they were."

The actual bug: Modifying the collection through `list.remove` while iterating it triggers fail-fast detection.

The fix (1-3 options if applicable): - **Option A:** Use the iterator's own remove:

`Iterator<Order> it = orders.iterator(); while (it.hasNext()) { if (it.next().isCancelled()) it.remove(); }` . The iterator updates its expected modCount. -

Option B: `orders.removeIf(Order::isCancelled)` — one-liner, idiomatic, safe. - **Option C:**

Stream filter: `orders.stream().filter(o -> !o.isCancelled()).collect(toList())` .

Returns a new list rather than mutating.

What NOT to say: - "Switch to CopyOnWriteArrayList" — solves the symptom but wastes memory on copy-per-write; not the right tool here. - "Wrap in try-catch and ignore CME" — masks correctness bug; you skip elements.

Common follow-up: "How does removeSelf avoid this if the underlying list is the same?"

Scenario 7 · The Integer comparison that fails after 127

Asked at: PhonePe · **Cred:** · **Difficulty:** Easy · **Topic:** Autoboxing / Integer cache

The code:

```
Integer count1 = 200;
Integer count2 = 200;
if (count1 == count2) {
    log.info("Counts match");
}

Integer small1 = 50;
Integer small2 = 50;
if (small1 == small2) {
    log.info("Smalls match");
}
```

The interviewer's prompt: "Why does the second log fire but the first doesn't? Both look identical."

Thinking out loud: 1. "Two Integer references compared with `==`. That's reference comparison." 2. "Java caches autoboxed Integers in the range -128 to 127 by default. Same boxed value within that range yields the same reference." 3. "50 is in the cache so both refs are the same object. 200 is outside; two fresh Integer objects, different references."

The actual bug: `==` on Integers compares references. Autoboxing returns the cached instance only for -128..127.

The fix (1-3 options if applicable): - **Option A:** `count1.equals(count2)` — value comparison, always correct. - **Option B:** `Objects.equals(count1, count2)` — null-safe variant of the above. - **Option C:** Convert to primitive: `count1.intValue() == count2.intValue()` — works but defeats the boxing.

What NOT to say: - "Increase the Integer cache size" — possible via `-XX:AutoBoxCacheMax` but band-aiding the real bug. - "Use Long for safety" — same problem (Long has its own -128..127 cache).

Common follow-up: "What's the JVM flag that changes the upper bound of the Integer cache?"

Scenario 8 · The connection pool that bleeds

Asked at: JP Morgan · Walmart Labs · **Difficulty:** Medium · **Topic:** Resource leak

The code:

```
public Order fetchOrder(long orderId) throws SQLException {
    Connection conn = dataSource.getConnection();
    PreparedStatement ps = conn.prepareStatement("SELECT * FROM orders WHERE id = ?");
    ps.setLong(1, orderId);
    ResultSet rs = ps.executeQuery();
    if (rs.next()) {
        return mapRow(rs);
    }
    return null;
}
```

The interviewer's prompt: "HikariCP starts logging 'connection pool exhausted' after about 30 minutes under load. Service hangs. What's wrong?"

Thinking out loud: 1. "No try-finally and no try-with-resources. If anything throws — or even on the happy path — the Connection, PreparedStatement and ResultSet are never closed." 2. "HikariCP pool size is typically 10. Once 10 connections are checked out and never returned, the next caller blocks forever." 3. "Even on success this leaks. The garbage collector won't help — sockets are OS resources, not Java memory."

The actual bug: Resources are never closed. Connections leak back into a pool that has no more to give.

The fix (1-3 options if applicable): - **Option A:** Wrap everything in try-with-resources: `try (Connection c = ds.getConnection(); PreparedStatement ps = c.prepareStatement(sql); ResultSet rs = ps.executeQuery()) { ... }`. Auto-close in reverse order. - **Option B:** Use Spring's JdbcTemplate or JPA — they handle connection lifecycle for you. - **Option C:** Manual try-finally — works but verbose and easy to forget on one branch.

What NOT to say: - "Increase pool size to 100" — postpones the failure by a few hours. - "Connections close themselves when GC runs" — they don't; this is a common myth.

Common follow-up: "What's HikariCP's leak detection threshold and how do you turn it on?"

Scenario 9 · The stream that double-counts

Asked at: Swiggy · Meesho · **Difficulty:** Medium · **Topic:** Stream side-effects

The code:


```
List<BigDecimal> amounts = payments.stream()
    .filter(Payment::isSuccessful)
    .peek(p -> totalProcessed.add(p.getAmount()))
    .map(Payment::getAmount)
    .collect(toList());
```

The interviewer's prompt: "Our `totalProcessed` metric is double the actual revenue. The list `amounts` is correct. What's happening?"

Thinking out loud: 1. "`peek` is intended for debugging. The contract is no guarantee it runs once per element across pipeline operations or across executions." 2. "If anything later in the chain re-evaluates the source — or if a downstream operation triggers two passes — `peek` runs twice." 3. "Also `totalProcessed.add(...)` — if `totalProcessed` is a `BigDecimal`, `.add` returns a new value; the result is thrown away. Either way, `peek` is the wrong tool for side effects."

The actual bug: Two compounding issues. (1) `peek` is for observation, not state mutation — its execution count is unreliable. (2) `BigDecimal.add` returns a new value and is being discarded — but the doubling suggests the metric is an `AtomicReference` or similar where the side effect does land, just twice.

The fix (1-3 options if applicable): - **Option A:** Compute the sum properly inside the pipeline: `BigDecimal total = amounts.stream().reduce(BigDecimal.ZERO, BigDecimal::add);` . No side effect. - **Option B:** Iterate twice if metrics and filtering are separate concerns — once for the metric, once for the list. Premature-optimization not to. - **Option C:** If you really must side-effect, use `forEach` after `collect` — at least the contract is well-defined.

What NOT to say: - "Move it to `forEach` inside the stream" — `forEach` on a parallel stream has no defined order; on a sequential stream it's at least defined but still mixes concerns. - "BigDecimal is mutable" — it is not; that's part of the bug.

Common follow-up: "Where does `peek` run differently in JDK 9+ if downstream operations don't need every element?"

Scenario 10 · The string that doesn't match itself

Asked at: Infosys · TCS · **Difficulty:** Easy · **Topic:** String pool / equals

The code:

```
String configValue = loadFromYaml("payment.mode"); // returns "UPI"
String expected = "UPI";

if (configValue == expected) {
    enableUpi();
}
```

The interviewer's prompt: "This worked when both strings were inline literals. After moving the value to YAML, the if-block never enters. Why?"

Thinking out loud: 1. "`==` on Strings compares references. Two String literals in source point to the same pool entry, hence the earlier behaviour." 2. "A string loaded from YAML (or any source — DB, file, network) is a fresh String object on the heap, not interned by default." 3. "References differ; `==` returns false. Always use `equals` for content."

The actual bug: Reference comparison on Strings. Source literals got interned; runtime-loaded strings did not.

The fix (1-3 options if applicable): - **Option A:** `"UPI".equals(configValue)` — null-safe (literal on the left avoids NPE if configValue is null). - **Option B:**

`Objects.equals(configValue, expected)` — also null-safe and reads cleanly. - **Option C:** For enum-like config values, parse to an enum: `PaymentMode mode = PaymentMode.valueOf(configValue);` then compare with `==` on enums (which is fine).

What NOT to say: - "Call `configValue.intern()` and use `==`" — works but a misuse of intern; brittle and a maintenance trap. - "Strings are interned automatically" — only string literals are.

Common follow-up: "When is `intern()` actually a good idea?"

Scenario 11 · The switch that does everything

Asked at: TCS · Accenture · **Difficulty:** Easy · **Topic:** Switch fall-through

The code:

```
public BigDecimal calculateShipping(ShippingTier tier) {
    BigDecimal cost = BigDecimal.ZERO;
    switch (tier) {
        case STANDARD:
            cost = new BigDecimal("49");
        case EXPRESS:
            cost = new BigDecimal("99");
        case SAME_DAY:
            cost = new BigDecimal("199");
    }
    return cost;
}
```

The interviewer's prompt: "Every order ships at Rs 199 regardless of tier. The logic looks right. What did I miss?"

Thinking out loud: 1. "Classic switch fall-through. No `break` statements." 2. "STANDARD assigns 49, then falls through to EXPRESS (99), then SAME_DAY (199). All three cases execute in order." 3. "Final value is always 199, regardless of which case matched first."

The actual bug: Missing `break` (or `return`) in each case. Control falls through to subsequent cases.

The fix (1-3 options if applicable): - **Option A:** Add `break;` after each assignment. Traditional fix. - **Option B:** Use the Java 14+ arrow form: `case STANDARD -> new BigDecimal("49");` — no fall-through possible, and it's an expression. - **Option C:** A Map lookup — cleaner if the table is data, not logic.

What NOT to say: - "Java switch always falls through" — only the classic colon-form does; the arrow form does not. - "Add `default` — it'll fix it" — `default` doesn't change fall-through behaviour.

Common follow-up: "What's the difference between switch expression and switch statement in Java 17?"

Scenario 12 · The optional that's never empty

Asked at: Razorpay · Swiggy · **Difficulty:** Easy · **Topic:** Optional misuse

The code:

```
public Optional<Customer> findCustomer(long id) {
    Customer c = customerRepo.findById(id);
    return Optional.of(c);
}
```

The interviewer's prompt: "Callers complain they keep getting `NullPointerException` from `Optional.of`. Isn't `Optional` supposed to prevent that?"

Thinking out loud: 1. "`Optional.of(value)` throws NPE if value is null. It's the strict variant." 2. "`Optional.ofNullable(value)` is what you want — returns empty if null." 3. "The repo presumably returns null on miss; need to handle that path."

The actual bug: `Optional.of` is non-null only; the repository's null return triggers NPE before the caller ever sees an `Optional`.

The fix (1-3 options if applicable): - **Option A:** `Optional.ofNullable(c)` — empty when null. - **Option B:** Change the repo to return `Optional<Customer>` natively (Spring Data JPA's `findById` already does this). - **Option C:** `Optional.ofNullable(customerRepo.findById(id))` inline — same idea.

What NOT to say: - "Add null check then call `Optional.of`" — works but defeats the point of `Optional`. - "Catch NPE and return `Optional.empty`" — exception-driven control flow, awful pattern.

Common follow-up: "When is `Optional.of` actually the right choice over `ofNullable`?"

Scenario 13 · The session map that fails after midnight

Asked at: PhonePe · Walmart Labs · **Difficulty:** Medium · **Topic:** `HashMap` thread safety

The code:

```
public class SessionStore {
    private final Map<String, Session> sessions = new HashMap<>();

    public void put(String token, Session s) { sessions.put(token, s); }
    public Session get(String token)         { return sessions.get(token); }
}
```

The interviewer's prompt: "This works fine 99% of the time. Occasionally a single thread enters an infinite loop using 100% CPU. Thread dump shows it stuck in `HashMap.getNode`. Diagnose."

Thinking out loud: 1. "`HashMap` is not thread-safe. Multiple writers can corrupt the internal linked list / tree structure." 2. "In Java 7 this caused the infamous infinite loop on resize. Java 8+ uses trees and the loop is less common but data corruption and lost entries still happen." 3. "Symptom — one thread pegged at 100% inside `getNode` — points at a corrupted bucket structure from a concurrent put."

The actual bug: Concurrent access to a plain `HashMap` can corrupt internal structure, producing infinite loops or lost data.

The fix (1-3 options if applicable): - **Option A:** `ConcurrentHashMap` — thread-safe, lock-stripped, high-throughput. The default answer for concurrent map use. - **Option B:**

`Collections.synchronizedMap(new HashMap<>())` — single global lock; works but doesn't scale. - **Option C:** For session storage specifically, use Redis or Hazelcast — sessions usually need cross-instance sharing anyway.

What NOT to say: - "HashMap is fine if reads outnumber writes" — concurrent writes during any read can corrupt; risk doesn't depend on ratio. - "Mark the map `volatile`" — volatile on the reference doesn't protect the internal state.

Common follow-up: "How does `ConcurrentHashMap` achieve thread safety without one big lock?"

Scenario 14 · The exception that vanished

Asked at: Cognizant · Infosys · **Difficulty:** Easy · **Topic:** Exception swallowing

The code:

```
public void processPayment(Payment p) {
    try {
        gateway.charge(p);
        notifyCustomer(p);
        recordToLedger(p);
    } catch (Exception e) {
        // TODO: handle later
    }
}
```

The interviewer's prompt: "Customers report missing ledger entries. Logs are clean — no errors. Yet payments are charged and notifications go out. What's broken?"

Thinking out loud: 1. "Empty catch. Classic swallowed exception." 2. "If `recordToLedger` throws, the exception disappears. The customer sees a charge and a notification but the ledger row never gets written." 3. "No log of the failure either — silent data loss. The 'TODO' is doing real damage."

The actual bug: Catch-all with empty body. Exceptions are silently discarded, masking real failures.

The fix (1-3 options if applicable): - **Option A:** At minimum, log the exception with stack trace:

`log.error("Payment processing failed for {}", p.getId(), e);` — even if you don't rethrow. - **Option B:** Let it propagate. Add the throws clause and let the caller decide. - **Option C:** Split the try blocks — charge separately from ledger; on ledger failure, raise an alert/retry rather than silently move on.

What NOT to say: - "Use `Throwable` instead of `Exception` to catch more" — makes it worse. - "Add `e.printStackTrace()`" — better than nothing but no log routing, no level, hard to monitor.

Common follow-up: "How do you make sure exceptions in async code (`CompletableFuture`, `@Async`) don't disappear?"

Scenario 15 · The Set that contains the same thing twice

Asked at: JP Morgan · Walmart Labs · **Difficulty:** Medium · **Topic:** equals/hashCode + mutability

The code:

```
record CustomerKey(String email) {}
class Customer {
    String email;
    int loyaltyTier;
    Customer(String e, int t) { email = e; loyaltyTier = t; }
    @Override public boolean equals(Object o) { return o instanceof Customer c && c.email.equals(email); }
    @Override public int hashCode() { return email.hashCode(); }
}

Set<Customer> set = new HashSet<>();
Customer c = new Customer("a@x.com", 1);
set.add(c);
c.email = "b@x.com";
set.add(new Customer("a@x.com", 2));
// set.size() == 2, but contains(new Customer("a@x.com")) == ?
```

The interviewer's prompt: "After mutation, the Set's contains() check returns false for the email even though an object with that email is in the set. Why?"

Thinking out loud: 1. "HashSet computes the bucket from hashCode at insertion time. Mutating email after insertion changes the hashCode but not the bucket." 2. "contains() recomputes hash and looks in the new bucket. The mutated entry sits in the old bucket — invisible." 3. "The fundamental issue: hash-bucketed collections cannot tolerate keys whose equals/hashCode-relevant fields change after insertion."

The actual bug: Mutable equals-relevant field on a HashSet member. Once mutated, the entry is effectively lost.

The fix (1-3 options if applicable): - **Option A:** Make equals/hashCode-relevant fields final. Best practice. - **Option B:** Use the record CustomerKey(String email) as the actual map/set key, and a separate Customer value object that can mutate. - **Option C:** Remove from the set before mutating, then re-add. Brittle.

What NOT to say: - "Call set.rehash()" — Java sets don't expose a rehash; even if they did, this is solving the symptom. - "Use TreeSet" — TreeSet compares with Comparator; same problem if you mutate the compared field.

Common follow-up: "What's the difference between == and equals for records?"

Scenario 16 · The thread pool that processes one task at a time

Asked at: Cred · Razorpay · **Difficulty:** Medium · **Topic:** ThreadPoolExecutor sizing

The code:

```
ThreadPoolExecutor pool = new ThreadPoolExecutor(
    2, 100, 60, TimeUnit.SECONDS,
    new LinkedBlockingQueue<>()
);
```

The interviewer's prompt: "We sized this with `corePoolSize=2` and `maxPoolSize=100`, expecting it to scale up to 100 threads under load. Under load it only ever uses 2 threads. Why?"

Thinking out loud: 1. "ThreadPoolExecutor's growth logic: if a task arrives and core is full, queue it. Only when the queue is full does it spawn beyond `corePoolSize`." 2. " `LinkedBlockingQueue` with no capacity is unbounded. The queue never fills, so the pool never grows past 2." 3. "This is a classic confusion. People assume max kicks in based on load; really it kicks in based on queue saturation."

The actual bug: Unbounded queue means the executor never spawns more than `corePoolSize` threads.

The fix (1-3 options if applicable): - **Option A:** Use a bounded queue: `new ArrayBlockingQueue<>(50)`. Now the pool grows to max when the queue fills. - **Option B:** Use a `SynchronousQueue` if you want each task handed off immediately — pool grows to max under any sustained load. Pair with a `RejectedExecutionHandler` for back-pressure. - **Option C:** Match core to max (e.g. both 100) if you genuinely want 100 threads sitting around.

What NOT to say: - "Call `allowCoreThreadTimeOut(true)`" — that controls core thread termination, not growth. - "Increase `corePoolSize`" — yes, but you should understand *why* max never engaged.

Common follow-up: "What `RejectedExecutionHandler` would you pick for a payment-processing pool?"

Scenario 17 · The generic that loses its type

Asked at: Swiggy · Meesho · **Difficulty:** Medium · **Topic:** Generic type erasure

The code:

```
public class Cache<T> {
    private T[] items;

    public Cache(int size) {
        items = (T[]) new Object[size];
    }

    public T[] getItems() {
        return items;
    }
}

Cache<String> c = new Cache<>(10);
String[] arr = c.getItems(); // ClassCastException at runtime
```

The interviewer's prompt: "The constructor compiles with a warning. The `getItems()` call throws `ClassCastException`. What's going on?"

Thinking out loud: 1. "Generics are erased at runtime. `T[]` is actually `Object[]` after compilation." 2. "The cast `(T[]) new Object[size]` doesn't actually convert anything — it just suppresses the compiler warning. The runtime array is still `Object[]`." 3. "At the call site, the compiler inserts an implicit cast to `String[]`. `Object[]` is not a `String[]`. `ClassCastException`."

The actual bug: Creating a generic array via unchecked cast. The runtime type stays as `Object[]`, which fails the implicit cast at the use site.

The fix (1-3 options if applicable): - **Option A:** Use `List<T>` (e.g. `new ArrayList<>(size)`). Lists don't suffer from this because generics in lists are erased to `Object`, and there's no array-typed return. - **Option B:** Pass the class at construction and use `Array.newInstance(clazz, size)`. The runtime array has the correct component type. - **Option C:** Return `Object[]` honestly and let the caller cast individual elements. Ugly but explicit.

What NOT to say: - "Add `@SuppressWarnings` to the cast" — silences the symptom; bug remains. - "Use raw types" — same problem, more warnings.

Common follow-up: "Why are generic arrays forbidden in Java but not in C# or Kotlin?"

Scenario 18 · The transaction that commits nothing

Asked at: JP Morgan · **Cred:** · **Difficulty:** Hard · **Topic:** Spring @Transactional self-invocation

The code:


```
@Service
public class OrderService {
    @Autowired private OrderRepo repo;

    public void placeOrder(Order o) {
        saveOrderInternal(o);
    }

    @Transactional
    public void saveOrderInternal(Order o) {
        repo.save(o);
        chargePayment(o);
        repo.save(o.withStatus("CHARGED"));
    }
}
```

The interviewer's prompt: "If `chargePayment` throws, we expect the first save to roll back. It doesn't. Rows stay. Why?"

Thinking out loud: 1. "`@Transactional` works via a Spring AOP proxy. The proxy wraps external calls into the bean." 2. "`placeOrder` calls `saveOrderInternal` directly via `this.` — that bypasses the proxy. No transactional advice runs." 3. "So both `repo.save` calls run in auto-commit mode (or whatever the default is). First save persists; exception in `chargePayment` doesn't roll back anything."

The actual bug: Self-invocation skips the Spring proxy, so `@Transactional` is never applied.

The fix (1-3 options if applicable): - **Option A:** Move `@Transactional` to `placeOrder`. The external caller hits the proxy. - **Option B:** Inject the bean into itself (or use `ApplicationContext.getBean`) and call through the proxy. Works but smells. - **Option C:** Use AspectJ load-time weaving — transactional advice is woven into the class, not added via a proxy, so internal calls work too. Heavy ceremony though.

What NOT to say: - "Add `@Transactional` to both methods" — doesn't help; self-invocation still bypasses the proxy. - "Use `PROPAGATION_REQUIRES_NEW`" — propagation doesn't matter if the advice isn't running at all.

Common follow-up: "How does Spring's CGLIB proxy differ from JDK dynamic proxy for `@Transactional`?"

Scenario 19 · The list iterator that skips elements

Asked at: TCS · Wipro · **Difficulty:** Easy · **Topic:** Iterator + indexed loop bug

The code:

```
List<Order> orders = new ArrayList<>(fetchOrders());
for (int i = 0; i < orders.size(); i++) {
    if (orders.get(i).isCancelled()) {
        orders.remove(i);
    }
}
```

The interviewer's prompt: "This removes cancelled orders. We notice some cancelled orders survive. Why?"

Thinking out loud: 1. "Two cancelled orders in a row. Remove index 3. Element at 4 shifts to 3. Loop increments *i* to 4 — skipping the shifted element." 2. "So every second consecutive cancellation survives the pass." 3. "Need to decrement *i* on remove, iterate backwards, or use `removeIf`."

The actual bug: Removing by index in a forward loop skips the element that shifts into the just-removed slot.

The fix (1-3 options if applicable): - **Option A:** `orders.removeIf(Order::isCancelled)` — one line, correct, idiomatic. - **Option B:** Iterate backwards: `for (int i = orders.size() - 1; i >= 0; i--) { ... }`. Shifting happens above the cursor, safe. - **Option C:** Decrement *i* after remove: `orders.remove(i--);`. Works but obscure.

What NOT to say: - "Use enhanced for-loop" — that throws CME on remove. Different bug. - "Use a parallel stream" — over-engineered and introduces ordering risk.

Common follow-up: "What's the complexity of `removeIf` on an `ArrayList` vs a `LinkedList`?"

Scenario 20 · The Comparator that throws `IllegalArgumentException`

Asked at: Walmart Labs · JP Morgan · **Difficulty:** Medium · **Topic:** Comparator contract violation

The code:

```
List<Trade> trades = ...;
trades.sort((a, b) -> (int)(a.getPnl() - b.getPnl()));
```

The interviewer's prompt: "This sorts trades by P&L. Works on small samples. On the full dataset we sometimes get `IllegalArgumentException: Comparison method violates its general contract`. Why?"

Thinking out loud: 1. "PnL is probably a long or `BigDecimal`. Cast to `int` — subtraction overflows." 2. "If `a.getPnl() = Long.MAX_VALUE` and `b.getPnl() = -1`, the subtraction wraps and the `(int)` cast lies about the sign." 3. "TimSort detects when transitivity is broken ($a < b$, $b < c$, $c < a$) and throws this exact exception."

The actual bug: Integer subtraction for comparison overflows. Cast to int compounds the lie. Transitivity is violated; TimSort catches it.

The fix (1-3 options if applicable): - **Option A:** `Comparator.comparingLong(Trade::getPnl)` — uses `Long.compare` under the hood, no subtraction. - **Option B:** `Long.compare(a.getPnl(), b.getPnl())` if writing it by hand. - **Option C:** For `BigDecimal`: `Comparator.comparing(Trade::getPnl)` which delegates to `BigDecimal.compareTo`.

What NOT to say: - "Add `(long)` cast before subtraction" — still wraps for extreme values; `Long.compare` is safer. - "Catch the exception and retry" — doesn't fix the comparator.

Common follow-up: "What's the difference between `TimSort`'s behaviour on a broken `Comparator` vs the old `MergeSort`'s?"

Scenario 21 · The double-checked singleton that isn't

Asked at: Cred · Razorpay · **Difficulty:** Hard · **Topic:** Memory model / DCL

The code:

```
public class ConfigManager {
    private static ConfigManager instance;

    public static ConfigManager getInstance() {
        if (instance == null) {
            synchronized (ConfigManager.class) {
                if (instance == null) {
                    instance = new ConfigManager();
                }
            }
        }
        return instance;
    }
}
```

The interviewer's prompt: "Looks like textbook double-checked locking. We get the occasional `NullPointerException` when reading fields on the returned instance. Diagnose."

Thinking out loud: 1. "`new ConfigManager()` is not atomic — allocate memory, run constructor, publish reference. The JVM is allowed to reorder." 2. "Without happens-before guarantees, another thread can see the assigned `instance` reference before the constructor's writes are visible — partially constructed object." 3. "The fix is `volatile` on `instance` — it establishes the happens-before that prevents the unsafe publication."

The actual bug: Double-checked locking without `volatile` is broken on the JMM. A thread can see a non-null `instance` whose fields are still defaults.

The fix (1-3 options if applicable): - **Option A:** Mark the field `private static volatile ConfigManager instance;`. Cheapest fix. - **Option B:** Use the holder-class idiom: `private static class Holder { static final ConfigManager INSTANCE = new ConfigManager(); }`. Lazy, thread-safe, no volatile. - **Option C:** Use an `enum` singleton. Thread-safe, serializes safely, can't be cloned.

What NOT to say: - "Java guarantees constructor completes before assignment" — it does not, prior to publication. - "Just synchronize the entire method" — works but slow on every call.

Common follow-up: "What changed in Java 5 that made DCL safe with volatile?"

Scenario 22 · The for-loop that never terminates

Asked at: Zerodha · Groww · **Difficulty:** Easy · **Topic:** Integer overflow / loop bound

The code:

```
public void processTrades(int totalTrades) {
    for (int i = 0; i <= totalTrades; i++) {
        process(trades[i]);
    }
}

processTrades(Integer.MAX_VALUE);
```

The interviewer's prompt: "A backfill job runs forever on heavy volume. Single-threaded loop. CPU pegged. Diagnose."

Thinking out loud: 1. "`i <= Integer.MAX_VALUE` — when `i` hits `MAX_VALUE`, the next `i++` wraps to `Integer.MIN_VALUE`." 2. "`MIN_VALUE` is still `<= MAX_VALUE`, so the loop continues. It'll go back up through zero, `MAX_VALUE`, wrap again — infinite loop." 3. "Classic int overflow. The fix is `<` not `<=`, or use long, or use Iterator/forEach."

The actual bug: `i <= Integer.MAX_VALUE` is always true after overflow. Loop never exits.

The fix (1-3 options if applicable): - **Option A:** `for (int i = 0; i < totalTrades; i++)`. The `<` comparison avoids the edge case. - **Option B:** Use `long i` so the upper bound is far away — but only if you genuinely have > 2 billion trades. - **Option C:** Iterate over the collection directly: `for (Trade t : trades) { process(t); }`. Sidesteps the bound entirely.

What NOT to say: - "Use a do-while" — same bug. - "Increase int to long for `i`" — works but doesn't address the off-by-one.

Common follow-up: "What's `Math.addExact` and when would you use it?"

Scenario 23 · The parallel stream that corrupts the result

Asked at: Cred · PhonePe · **Difficulty:** Hard · **Topic:** Parallel stream + shared state

The code:

```
List<Long> ids = new ArrayList<>();
payments.parallelStream().forEach(p -> ids.add(p.getId()));
```

The interviewer's prompt: "After processing 100k payments we expect 100k IDs. We sometimes get 98,403 or an `ArrayIndexOutOfBoundsException`. Why?"

Thinking out loud: 1. "ArrayList is not thread-safe. parallelStream forks across the common ForkJoinPool — multiple threads calling add concurrently." 2. "Lost adds (the 98k case) and corrupted internal arrays (the AIOOBE) both happen." 3. "The right approach is to collect, not side-effect."

The actual bug: Mutating a non-thread-safe collection from a parallel stream.

The fix (1-3 options if applicable): - **Option A:** `List<Long> ids = payments.parallelStream().map(Payment::getId).collect(toList());` — proper functional pipeline; Collectors handle parallel merging. - **Option B:** Wrap in `Collections.synchronizedList` — works but defeats parallelism (every add takes the lock). - **Option C:** Use `Collectors.toConcurrentMap` or `groupingByConcurrent` if you need a concurrent terminal collector.

What NOT to say: - "ArrayList is thread-safe in Java 11" — it never has been. - "Use `forEachOrdered`" — keeps insertion order but doesn't fix the data race.

Common follow-up: "When does `Collectors.toList()` produce a thread-safe result for parallel streams?"

Scenario 24 · The date that's off by 53 weeks

Asked at: JP Morgan · Walmart Labs · **Difficulty:** Medium · **Topic:** Date formatting / week year

The code:

```
SimpleDateFormat fmt = new SimpleDateFormat("YYYY-MM-dd");
System.out.println(fmt.format(new Date(125, 11, 30))); // Dec 30, 2025
// prints "2026-12-30"
```

The interviewer's prompt: "We log invoices with this format. December invoices are showing year 2026 even though it's still 2025. Why?"

Thinking out loud: 1. " `YYYY` (capital Y) is week-based year. `yyyy` (lowercase) is calendar year." 2. "For dates near year boundaries that fall in week 1 of next year, week-based year leaps ahead." 3. "Classic `SimpleDateFormat` trap — always use lowercase `yyyy` for calendar year."

The actual bug: `YYYY` is week-year, not calendar year. Around year boundaries, it differs from the calendar year.

The fix (1-3 options if applicable): - **Option A:** `"yyyy-MM-dd"` — lowercase `y` is calendar year. The right answer 99% of the time. - **Option B:** Switch to `DateTimeFormatter.ofPattern("yyyy-MM-dd")` and `LocalDate` — modern API, fewer surprises, and `SimpleDateFormat` is not thread-safe. - **Option C:** Use `DateTimeFormatter.ISO_LOCAL_DATE` — built-in, fast, idiomatic.

What NOT to say: - "SimpleDateFormat is wrong about the year" — it's a documented format-letter difference. - "Use Joda-Time" — Joda is deprecated; `java.time` is the successor.

Common follow-up: "Why is `SimpleDateFormat` not thread-safe and how would that surface in production?"

Scenario 25 · The lock that the wrong thread holds

Asked at: Cred · PhonePe · **Difficulty:** Hard · **Topic:** Lock acquired in one thread, released in another

The code:

```
Lock lock = new ReentrantLock();

public void startWork() {
    lock.lock();
    executor.submit(() -> {
        try {
            doWork();
        } finally {
            lock.unlock();
        }
    });
}
```

The interviewer's prompt: "Production logs show `IllegalMonitorStateException` from this code. Lock is acquired in `startWork()`. Why does `unlock` fail?"

Thinking out loud: 1. "ReentrantLock is owned by the thread that acquired it. Only that thread can release it." 2. "Here the main thread locks, the executor thread tries to unlock. `IllegalMonitorStateException`." 3. "Smells like the intent is mutex across submissions. Need a different primitive — maybe Semaphore, or move the lock entirely into the worker."

The actual bug: ReentrantLock has thread-ownership semantics. Acquiring on one thread and releasing on another throws.

The fix (1-3 options if applicable): - **Option A:** Move lock acquisition INSIDE the runnable so the same thread acquires and releases. - **Option B:** Use a `Semaphore(1)` — semaphores don't track ownership; any thread can release. Loses reentrancy. - **Option C:** Rethink the design — if you need cross-thread coordination, `BlockingQueue` or `CompletableFuture` is usually a better fit.

What NOT to say: - "Switch to synchronized" — synchronized also requires the same thread; same problem and worse. - "Use `lockInterruptibly`" — irrelevant; ownership semantics are the same.

Common follow-up: "What's the difference between `ReentrantLock` and `synchronized` in terms of fairness?"

Scenario 26 · The empty list from `toMap`

Asked at: Swiggy · Meesho · **Difficulty:** Medium · **Topic:** `Collectors.toMap` duplicate keys

The code:

```
Map<String, Order> byCity = orders.stream()
    .collect(Collectors.toMap(Order::getCity, o -> o));
```

The interviewer's prompt: "This sometimes throws `IllegalStateException` at runtime. Sometimes works. What's wrong?"

Thinking out loud: 1. "`toMap` with two args has no merge function. If two elements produce the same key, it throws." 2. "`getCity` returns the same string for many orders. Duplicates are guaranteed in any realistic order dataset." 3. "Need a merge function or a `groupingBy`."

The actual bug: Two-arg `toMap` throws on duplicate keys. With real-world data, duplicates are inevitable.

The fix (1-3 options if applicable): - **Option A:** Decide what 'value for a duplicate key' means. If you want a list, use `Collectors.groupingBy(Order::getCity)`. - **Option B:** If you want first-wins or last-wins: `toMap(Order::getCity, o -> o, (a, b) -> a)` or `(a, b) -> b`. - **Option C:** If duplicates would be a real bug, throw with a helpful message: `(a, b) -> { throw new IllegalStateException("Dup city: " + a.getCity()); }`.

What NOT to say: - "Collectors.toMap is broken" — strict-by-design. - "Distinct first" — `.distinct()` uses equals on the whole Order, not the city.

Common follow-up: "What's the difference between `toMap` and `groupingBy` in terms of intent?"

Scenario 27 · The reader that loses data

Asked at: TCS · Infosys · **Difficulty:** Easy · **Topic:** Resource leak / Buffered flush

The code:

```
BufferedWriter w = new BufferedWriter(new FileWriter("audit.log", true));
for (Audit a : audits) {
    w.write(a.toCsv());
    w.newLine();
}
// w never closed
```

The interviewer's prompt: "Audit file is missing the last several entries when the JVM is killed. We do see most rows. What happened?"

Thinking out loud: 1. "BufferedWriter buffers in memory. Writes flush only when the buffer fills or close/flush is called." 2. "No close, no explicit flush. On graceful shutdown maybe a shutdown hook flushes; on a hard kill the in-memory buffer is lost." 3. "Also a file-descriptor leak — every call leaks an FD."

The actual bug: Missing close (and implicit flush). Buffered writes sit in memory; FD leaks accumulate.

The fix (1-3 options if applicable): - **Option A:** `try (BufferedWriter w = new BufferedWriter(new FileWriter("audit.log", true))) { ... }` . Closes and flushes deterministically. - **Option B:** For append-only audit, consider `Files.writeString(path, line, APPEND)` per row — auto-closed, but slower because no buffer. - **Option C:** Use a logging framework with a file appender designed for this — handles flush, rotation, async-write.

What NOT to say: - "GC will close it eventually" — no guarantee; FDs are OS resources. - "Call `System.gc()` on shutdown" — won't reliably trigger close; bad practice.

Common follow-up: "What's the difference between `FileWriter` and `Files.newBufferedWriter`?"

Scenario 28 · The map iteration that explodes

Asked at: Walmart Labs · JP Morgan · **Difficulty:** Easy · **Topic:** ConcurrentModificationException + Map

The code:

```
Map<String, Integer> stockLevels = new HashMap<>(getStock());
for (String sku : stockLevels.keySet()) {
    if (stockLevels.get(sku) == 0) {
        stockLevels.remove(sku);
    }
}
```


The interviewer's prompt: "Throws `ConcurrentModificationException` on the first removal. Single-threaded. Why?"

Thinking out loud: 1. "`keySet()` returns a view backed by the map. Modifying the map invalidates the iterator from the view." 2. "Same fail-fast detection as `ArrayList` — `next` `hasNext` or `next` throws CME." 3. "Standard fixes: `iterator.remove`, `removeIf` on `entrySet`, or a second pass."

The actual bug: Modifying the map while iterating its `keySet` view triggers fail-fast detection.

The fix (1-3 options if applicable): - **Option A:** `stockLevels.entrySet().removeIf(e -> e.getValue() == 0);` — one line, safe. - **Option B:** Use an explicit iterator and call `iterator.remove()`. - **Option C:** Collect keys to remove first, then remove in a second pass: `var toRemove = stockLevels.entrySet().stream().filter(e -> e.getValue() == 0).map(Map.Entry::getKey).toList(); toRemove.forEach(stockLevels::remove);`.

What NOT to say: - "ConcurrentHashMap fixes this" — its iterators are weakly-consistent (won't throw CME), but plain modify-while-iterate is still poor style. - "Iterate over a copy of keys" — works (`new HashSet<>(stockLevels.keySet())`) but allocates and is less idiomatic than `removeIf`.

Common follow-up: "How does `ConcurrentHashMap`'s iterator differ in terms of consistency?"

Scenario 29 · The varargs that gets a single null

Asked at: Cognizant · TCS · **Difficulty:** Medium · **Topic:** Varargs + null

The code:

```
public static void log(String tag, Object... params) {
    System.out.println(tag + ": " + params.length);
}

log("checkout", null);
```

The interviewer's prompt: "This throws `NullPointerException` at runtime. We expected to see `checkout: 1`. What's going on?"

Thinking out loud: 1. "`log(\"checkout\", null)` is ambiguous — `null` could be a single `Object` element or the entire `Object[]` varargs array." 2. "Java resolves this by treating `null` as the array itself. So `params == null`." 3. "`params.length` then dereferences `null`. NPE."

The actual bug: Passing a single `null` to a varargs parameter binds it as the array, not as a single element. `params` is null, not an array of one null.

The fix (1-3 options if applicable): - **Option A:** Caller side: `log("checkout", (Object) null)` — the cast forces the single-element interpretation. - **Option B:** Callee side: defensive `if (params ==`

`null) params = new Object[0];` — keeps the API forgiving. - **Option C:** Avoid varargs in APIs where this matters; take `List<Object>` or `Object[]` explicitly.

What NOT to say: - "Varargs always wraps to an array" — only for non-null arguments; null is special-cased. - "Use `Optional<Object[]>` parameter" — Optional on parameters is an antipattern.

Common follow-up: "What's the compile-time warning Java emits for this pattern and how do you suppress it?"

Scenario 30 · The closed stream

Asked at: Razorpay · Cred · **Difficulty:** Medium · **Topic:** Stream reuse

The code:

```
Stream<Order> ordersStream = orders.stream().filter(Order::isPaid);

long count = ordersStream.count();
List<Order> list = ordersStream.collect(toList()); // IllegalStateException
```

The interviewer's prompt: "We compute a count then a list from the same filtered stream. The second call throws `stream has already been operated upon or closed`. Why?"

Thinking out loud: 1. "Streams are one-shot — a terminal operation consumes the stream." 2. "`count()` is a terminal op. After that the stream is consumed; calling collect throws." 3. "Need to either materialize once and reuse, or re-create the stream."

The actual bug: Streams are single-use. After a terminal op, the stream is closed.

The fix (1-3 options if applicable): - **Option A:** Materialize once: `List<Order> list = orders.stream().filter(Order::isPaid).toList(); long count = list.size();` . -

Option B: Re-create the stream: `long count = orders.stream().filter(Order::isPaid).count(); List<Order> list = orders.stream().filter(Order::isPaid).collect(toList());` . Filters run twice. - **Option C:** Use a `Supplier<Stream<Order>>` if the predicate is expensive and reuse is frequent: `Supplier<Stream<Order>> paid = () -> orders.stream().filter(Order::isPaid);` .

What NOT to say: - "Streams are infinite by default" — they aren't; bounded by source. - "Reset the stream with `.parallel()`" — that's a configuration call, not a reset.

Common follow-up: "Is there a way to make a stream multi-shot like an Iterable?"

Scenario 31 · The ThreadLocal that leaks across requests

Asked at: PhonePe · Walmart Labs · **Difficulty:** Hard · **Topic:** ThreadLocal + thread pool leak

The code:

```
public class UserContext {
    private static final ThreadLocal<User> CURRENT = new ThreadLocal<>();
    public static void set(User u) { CURRENT.set(u); }
    public static User get() { return CURRENT.get(); }
}

@RestController
class CartController {
    @PostMapping("/cart")
    public void add(@RequestBody Item i) {
        UserContext.set(authService.lookup(i));
        cartService.add(i);
    }
}
```

The interviewer's prompt: "Customers complain they sometimes see another user's items in their cart. We use ThreadLocal for user context. Diagnose."

Thinking out loud: 1. "Tomcat reuses threads from its pool. ThreadLocal values stick to the thread." 2. "Request A sets user=Alice on thread T. Request B reuses thread T, never calls set, calls get — gets Alice." 3. "The set() must be paired with remove() in a finally block, ideally at a request filter level."

The actual bug: ThreadLocal values are not cleared after the request. Thread pool reuse leaks state across users.

The fix (1-3 options if applicable): - **Option A:** Add a Servlet/Spring filter: `try { UserContext.set(u); chain.doFilter(req, res); } finally { UserContext.remove(); }` - **Option B:** Java 21+: use `ScopedValue.runWhere(USER, u, () -> ...)` — bound for scope only, no leak risk. - **Option C:** Pass user as a parameter through the call chain. Verbose but explicit; no thread-state dependency.

What NOT to say: - "ThreadLocal is per-thread so it's safe" — yes, but threads in pools are recycled. - "Use synchronized on the ThreadLocal" — irrelevant; this isn't a concurrency race.

Common follow-up: "What's the memory leak risk of ThreadLocal in an app server's classloader?"

Scenario 32 · The HashMap that "loses" a value

Asked at: TCS · Wipro · **Difficulty:** Easy · **Topic:** Boxed null + Map

The code:

```
Map<String, Integer> retries = new HashMap<>();
retries.put("payment-svc", 3);

int count = retries.get("billing-svc"); // NullPointerException
```

The interviewer's prompt: *"This blows up when querying a key we haven't inserted. Why does NPE come from a primitive int?"*

Thinking out loud: 1. " `Map.get` returns `Integer` (the boxed type). For a missing key, it returns `null`." 2. "Assigning to `int count` auto-unboxes — calls `.intValue()` on `null`. NPE." 3. "Use `getOrDefault`, or check `containsKey`, or keep the result boxed."

The actual bug: Auto-unboxing a null `Integer` from a missed `Map` lookup triggers NPE.

The fix (1-3 options if applicable): - **Option A:** `int count = retries.getOrDefault("billing-svc", 0);` — explicit default. - **Option B:** `Integer count = retries.get("billing-svc");` then check `null`. Verbose. - **Option C:** Use a `Map<String, Integer>` semantic where 0 is meaningful only if explicitly stored — but `getOrDefault` is fine when 0 is the natural zero-state.

What NOT to say: - "Use `Optional` from the map" — `Map` has no direct `Optional` API; you'd box-and-wrap, which is heavier than `getOrDefault`. - "Null-check then auto-unbox" — fine but verbose for a one-liner.

Common follow-up: *"When would `computeIfAbsent` be cleaner than `getOrDefault`?"*

Scenario 33 · The `String.split` that drops empty trailing values

Asked at: Infosys · Cognizant · **Difficulty:** Easy · **Topic:** `String.split` limit

The code:

```
String line = "100,200,,";
String[] parts = line.split(",");
System.out.println(parts.length); // 2, not 4
```

The interviewer's prompt: *"CSV parsing. We pad rows with trailing commas to represent missing values. `split` gives back the wrong count. Why?"*

Thinking out loud: 1. "Default `split` discards trailing empty strings." 2. "To keep them, use the 2-arg form with a negative limit: `split(\\",\\", -1)` ." 3. "Documented behaviour — surprising the first time you hit it."

The actual bug: `String.split(regex)` removes trailing empty strings unless a negative limit is supplied.

The fix (1-3 options if applicable): - **Option A:** `line.split(",", -1)` — keeps all trailing empties. - **Option B:** Use a proper CSV library (Apache Commons CSV, Univocity). Real CSV has quoted fields, escaped commas — `split` is wrong as soon as data has commas inside quotes. - **Option C:** Use `Pattern.compile(",").split(line, -1)` if you want pattern reuse for performance.

What NOT to say: - "Split is broken" — it's documented; default of 0 discards trailing empties. - "Pad input with a sentinel" — fragile and obscures intent.

Common follow-up: "Why would you use a real CSV parser over `split` even for 'simple' files?"

Scenario 34 · The volatile flag that doesn't stop the loop

Asked at: JP Morgan · **Cred:** · **Difficulty:** Medium · **Topic:** volatile + visibility

The code:

```
class Worker implements Runnable {
    private boolean running = true;

    public void run() {
        while (running) {
            doWork();
        }
    }

    public void stop() { running = false; }
}
```

The interviewer's prompt: "We call `worker.stop()` from the main thread but the worker thread never exits. Eventually the JVM has to be killed. Why?"

Thinking out loud: 1. "`running` is read in a tight loop. The JIT can hoist it into a register — never reread from memory." 2. "Main thread's write to `running` is never observed by the worker thread without a happens-before." 3. "Mark `volatile`, or use `AtomicBoolean`."

The actual bug: Without `volatile`, the JIT can cache `running` in a register; the worker never sees the update.

The fix (1-3 options if applicable): - **Option A:** `private volatile boolean running = true;` — establishes visibility. - **Option B:** `private final AtomicBoolean running = new AtomicBoolean(true);` — also adds atomic compare-and-set if needed. - **Option C:** Interrupt-based: check `Thread.currentThread().isInterrupted()` and call `worker.interrupt()` to stop. Standard pattern.

What NOT to say: - "Synchronize `doWork`" — doesn't fix loop visibility. - "Sleep in the loop" — coincidentally helps because sleep is a memory barrier on some JVMs, but it's accidental.

Common follow-up: "What's the canonical way to signal cancellation to a worker thread in modern Java?"

Scenario 35 · The map merge that doubles values

Asked at: Swiggy · Meesho · **Difficulty:** Medium · **Topic:** Map.merge semantics

The code:

```
Map<String, Integer> revenue = new HashMap<>();
for (Order o : orders) {
    revenue.merge(o.getCity(), o.getAmount(), (a, b) -> a + b + b);
}
```

The interviewer's prompt: "Revenue numbers per city look way too high. We're using Map.merge. What's the bug?"

Thinking out loud: 1. "merge(key, value, remappingFn) : if key absent, puts value; if present, calls fn(oldValue, value)." 2. "Here fn is (a, b) -> a + b + b. That's old + new + new — double-counting every merge." 3. "Should be (a, b) -> a + b or Integer::sum."

The actual bug: The remapping function adds the new value twice.

The fix (1-3 options if applicable): - **Option A:** revenue.merge(city, amount, Integer::sum);
— idiomatic. - **Option B:** groupingBy(Order::getCity, summingInt(Order::getAmount))
— single-pass stream form. - **Option C:** Use long-based sum if amounts can overflow int.

What NOT to say: - "merge does the addition for you" — only with a sum BiFunction; it doesn't auto-sum. - "BiFunction has implicit accumulator" — no, the function is exactly what you write.

Common follow-up: "What's the difference between Map.merge and Map.compute?"

Scenario 36 · The reflection-based setter that ignores final

Asked at: Cred · JP Morgan · **Difficulty:** Hard · **Topic:** Reflection + final + inlining

The code:

```
class Config { static final int MAX = 100; }

Field f = Config.class.getDeclaredField("MAX");
f.setAccessible(true);
f.setInt(null, 999);

System.out.println(Config.MAX); // 100
System.out.println(getMaxViaReflection()); // 999
```

The interviewer's prompt: "We change a static final field via reflection. The reflective read returns the new value; direct field access returns the old. Why?"

Thinking out loud: 1. "Compile-time constants — static final primitives initialized with constant expressions — are inlined at every call site." 2. "`Config.MAX` is replaced with `100` in callers' bytecode. The actual field can be mutated, but the inlined constants are frozen at compile time." 3. "Reflective read pulls the current field value. Direct access shows the inlined constant."

The actual bug: Constant-folding by the compiler. `static final` primitives become baked-in literals in any class compiled against them.

The fix (1-3 options if applicable): - **Option A:** Don't mutate static final fields via reflection. That violates the language guarantee. - **Option B:** If you legitimately need a mutable config value, drop `final` — let the JIT decide what to do. - **Option C:** Recompile any class that references `Config.MAX` if the constant changes. Often impractical at runtime.

What NOT to say: - "Reflection doesn't bypass `final`" — it can mutate the field; the issue is consumers' inlined copies. - "Mark it `volatile`" — volatile doesn't matter when the compiler inlined the literal.

Common follow-up: "In Java 17+, what restrictions does the module system add to setting final fields via reflection?"

Scenario 37 · The retry loop that never retries

Asked at: Razorpay · PhonePe · **Difficulty:** Medium · **Topic:** Exception catching too broad

The code:

```
for (int attempt = 0; attempt < 3; attempt++) {  
    try {  
        return gateway.charge(payment);  
    } catch (Exception e) {  
        log.warn("attempt {} failed: {}", attempt, e.getMessage());  
    }  
}  
throw new RuntimeException("All attempts failed");
```

The interviewer's prompt: *"This is meant to retry transient failures. In prod we sometimes hit 'All attempts failed' even on permanent errors like InvalidCardException. We should fail fast on those."*

Thinking out loud: 1. "Catching `Exception` lumps transient (timeout, 5xx) with permanent (4xx, validation)." 2. "On a permanent error, retrying wastes time and may double-charge if idempotency is broken." 3. "Need to narrow the catch to retryable exceptions and rethrow others immediately."

The actual bug: Catching `Exception` retries every failure including permanent ones — wasted time and risk of duplicate side-effects.

The fix (1-3 options if applicable): - **Option A:** Catch only retryable types:

`catch (TimeoutException | TransientGatewayException e) { ... }`. Let others propagate. - **Option B:** Use a library like Resilience4j — its Retry policy supports predicates for retry decisions. - **Option C:** Categorize exceptions: `if (isTransient(e)) continue; throw e;`. Explicit and easy to audit.

What NOT to say: - "Always retry; idempotency keys handle duplicates" — assumes a contract you may not have. - "Sleep between retries" — useful for backoff but doesn't fix the broad-catch bug.

Common follow-up: *"What backoff strategy would you pick for charging a payment gateway?"*

Scenario 38 · The CompletableFuture exception that disappears

Asked at: Cred · Swiggy · **Difficulty:** Hard · **Topic:** CompletableFuture error handling

The code:

```
CompletableFuture  
    .supplyAsync(() -> riskyCall())  
    .thenApply(this::transform);  
  
// later — no get(), no exceptionally
```

The interviewer's prompt: *"We launch this async job and never block on it. If it fails, the error vanishes silently — nothing in logs. Why?"*

Thinking out loud: 1. "CompletableFuture exceptions only surface when you observe the future (`get` , `join` , `exceptionally` , `handle` , `whenComplete`)." 2. "Without observation, the exception sits in the future and dies with the GC. No logging." 3. "This is the async equivalent of the empty catch."

The actual bug: Unobserved CompletableFuture exceptions are silently swallowed. There's no UncaughtExceptionHandler safety net.

The fix (1-3 options if applicable): - **Option A:** Always end the chain with `.exceptionally(t -> { log.error("async failed", t); return null; })` or `.whenComplete((v, t) -> if (t != null) log.error(...))` . - **Option B:** Track the future and assert success on a periodic basis — fine for long-running services. - **Option C:** Java 21+: use structured concurrency. Child task exceptions propagate to the scope.

What NOT to say: - "CompletableFuture logs to stderr by default" — it does not. - "The thread pool's UncaughtExceptionHandler catches it" — not for CompletableFuture; only direct thread exceptions.

Common follow-up: "What's the difference between `exceptionally` and `handle` ?"

Scenario 39 · The hot reload that doubles handlers

Asked at: Walmart Labs · **Cred:** · **Difficulty:** Medium · **Topic:** Listener leak

The code:

```
@PostConstruct
public void init() {
    eventBus.subscribe(this::onPaymentEvent);
}
```

The interviewer's prompt: "In our dev environment we hot-reload Spring beans. After three reloads, every payment event is processed 3 times. Production logs show similar duplication after some pod restarts. Why?"

Thinking out loud: 1. " `subscribe` adds a handler. There's no `unsubscribe` on bean destruction." 2. "On reload, a new bean instance subscribes — old subscription stays. After 3 reloads, 3 active handlers." 3. "Same can happen if the bus survives JVM restart in a shared process (rare) — or, more commonly, the bus is in static state."

The actual bug: Subscription without an unsubscription. Old subscribers leak across bean lifecycle events.

The fix (1-3 options if applicable): - **Option A:** Add `@PreDestroy public void shutdown() { eventBus.unsubscribe(this::onPaymentEvent); }` . Method refs may not equal each other, so store the subscription token. - **Option B:** Subscribe via Spring's `@EventListener` annotation —

Spring manages the lifecycle automatically. - **Option C:** Use a weak-reference subscription mechanism if the bus is shared across instances.

What NOT to say: - "Hot reload is unreliable, ignore it" — same pattern bites you with Spring's dev tools and pod restarts. - "Method references `this::onPaymentEvent` are cached, so unsubscribe by reference works" — they're not interned; two `this::method` are not equal.

Common follow-up: "Why doesn't `this::method == this::method` return true?"

Scenario 40 · The Long primitive vs Long wrapper trap

Asked at: TCS · Infosys · **Difficulty:** Medium · **Topic:** Wrapper comparison + collections

The code:

```
List<Long> ids = List.of(1L, 2L, 3L);
ids.remove(1);
System.out.println(ids); // [1, 3]
```

The interviewer's prompt: "We want to remove ID `1` from the list but get back `[1, 3]`. It removed `2`. Why?"

Thinking out loud: 1. "`List<E>` has two remove methods: `remove(int index)` and `remove(Object)`." 2. "`remove(1)` is an int literal — binds to `remove(int index)`. Removes the element at index 1 (which is 2L)." 3. "To remove by value, pass a boxed Long: `ids.remove(Long.valueOf(1))` or `ids.remove((Long) 1L)`."

The actual bug: Method overload resolution. `int` argument selects `remove(int index)`, not `remove(Object)`.

The fix (1-3 options if applicable): - **Option A:** `ids.remove(Long.valueOf(1L))` — explicit boxing. - **Option B:** `ids.removeIf(id -> id == 1L)` — clearer intent. - **Option C:** Cast: `ids.remove((Object) 1L)` — forces the Object overload. Slightly cryptic.

What NOT to say: - "Use a TreeSet to avoid this" — different collection, but the API is the same. - "Convert to int first" — won't help; the issue is overload resolution, not value.

Common follow-up: "Why does `List.of(1L, 2L, 3L)` not throw when you call `remove` on it?" (Trick: `List.of` is immutable, so it actually throws `UnsupportedOperationException`. The scenario uses a mutable list elsewhere.)

Scenario 41 · The stream that doubles down on the database

Asked at: JP Morgan · Walmart Labs · **Difficulty:** Hard · **Topic:** Stream + lazy eval + side effects

The code:

```
Stream<Order> orders = repo.findAll().stream()
    .peek(o -> auditLog.write(o));

long highValueCount = orders.filter(o -> o.amount() > 10_000).count();
```

The interviewer's prompt: "Each request writes to audit log twice as much data as expected. We thought peek runs once per element. Why doesn't it?"

Thinking out loud: 1. "`count()` after `filter()` with no other stateful ops is a candidate for an optimization where filter is run but peek may be reordered or even short-circuited." 2. "In JDK 9+, the stream pipeline can decide not to invoke peek for elements not needed downstream. In some configurations it can also be invoked redundantly for partial pulls (parallel streams)." 3. "Bottom line: peek's invocation count is not part of the contract. Mutating audit log inside peek is unsafe."

The actual bug: Stream pipelines do not guarantee per-element invocation of intermediate ops with side effects. Side-effecting in `peek` is unsafe by spec.

The fix (1-3 options if applicable): - **Option A:** Audit explicitly before or after the pipeline:

`orders.forEach(auditLog::write); long count = orders.stream()...` (or, better, iterate the list directly for audit). - **Option B:** Replace peek with map+forEach if you need ordering: keep audit out of the lazy chain entirely. - **Option C:** If the pipeline must be fused, use `forEachOrdered` as the terminal op and write audit there — but lose the count short-circuiting.

What NOT to say: - "peek is reliable for side effects" — explicitly the opposite per the API doc. - "Mark the stream sequential" — doesn't change the guarantee.

Common follow-up: "Which intermediate stream operations are guaranteed to traverse every element?"

Scenario 42 · The static counter shared across tests

Asked at: Cred · Razorpay · **Difficulty:** Medium · **Topic:** Static state in tests

The code:

```
public class IdGenerator {
    private static long counter = 0;
    public static long next() { return ++counter; }
}
```

The interviewer's prompt: "Each test class works individually. When we run the full suite via gradle, some tests fail with duplicate IDs or unexpected sequences. Why?"

Thinking out loud: 1. " `counter` is static and lives for the JVM lifetime. Multiple tests in the same JVM share it." 2. " `++counter` is also not thread-safe — parallel test execution can race." 3. "Both atomicity and shared-state-across-tests are problems."

The actual bug: Static mutable state shared across tests. Also not thread-safe.

The fix (1-3 options if applicable): - **Option A:** `private static final AtomicLong counter = new AtomicLong(); public static long next() { return counter.incrementAndGet(); }` . Fixes atomicity. - **Option B:** Reset in `@BeforeEach` for tests that depend on sequence values: `IdGenerator.reset()` . - **Option C:** Don't expose a static singleton. Inject an `IdGenerator` instance per test. Forces explicit lifecycle.

What NOT to say: - "Add `volatile` to counter" — fixes visibility but not atomicity. - "Use `UUID.randomUUID().getMostSignificantBits()`" — distributions different; may not match test expectations.

Common follow-up: "How would you write `next()` to be both monotonically increasing and unique across multiple JVM instances?"

Scenario 43 · The Scanner that exits after the integer

Asked at: TCS · Wipro · **Difficulty:** Easy · **Topic:** Scanner `nextInt` + `nextLine`

The code:

```
Scanner sc = new Scanner(System.in);
int age = sc.nextInt();
String name = sc.nextLine();
System.out.println("Hello " + name + ", age " + age);
```

The interviewer's prompt: "User enters 30 then 'Bharat'. Output shows `Hello , age 30` — name is empty. Why?"

Thinking out loud: 1. " `nextInt()` consumes '30' but not the trailing newline." 2. " `nextLine()` reads from the current position to the next newline — which is empty (just the leftover `\n`)." 3. "Classic Scanner quirk; needs an extra `nextLine` after `nextInt`."

The actual bug: `nextInt()` doesn't consume the line separator. The follow-up `nextLine()` reads the empty remainder of the integer's line.

The fix (1-3 options if applicable): - **Option A:** Add a dummy `sc.nextLine();` after `nextInt()` to consume the newline. - **Option B:** Use `Integer.parseInt(sc.nextLine())` for the int — read

whole line, parse manually. - **Option C:** For real input handling use `BufferedReader` with `readLine()` and parse — cleaner mental model.

What NOT to say: - "Scanner is broken" — documented behaviour. - "Use `System.in.read` directly" — character-level and worse.

Common follow-up: "What changes if you `useDelimiter` differently?"

Scenario 44 · The map that uses arrays as keys

Asked at: Walmart Labs · **Cred:** · **Difficulty:** Medium · **Topic:** Array equals + Map

The code:

```
Map<int[], String> regionByCoords = new HashMap<>();
regionByCoords.put(new int[]{28, 77}, "Delhi");

String region = regionByCoords.get(new int[]{28, 77}); // null
```

The interviewer's prompt: "We use `int` arrays as Map keys for coordinates. Lookups always return null. What's wrong?"

Thinking out loud: 1. "Arrays inherit `Object.equals` and `Object.hashCode` — based on identity." 2. "Two `new int[]{28, 77}` are different objects. Different hash, different bucket, miss." 3. "Use a record, a List, or wrap in a custom class with proper `equals/hashCode`."

The actual bug: Java arrays do not override `equals/hashCode`. Identity comparison is the default; two separate arrays never match as keys.

The fix (1-3 options if applicable): - **Option A:** Use a `record Coord(int x, int y) {}` as the key — auto-generated `equals/hashCode` on values. - **Option B:** Use `List.of(28, 77)` — Lists implement value-based `equals/hashCode`. - **Option C:** Encode as a string or long key: `28L * 1000 + 77` — works only if components fit safely.

What NOT to say: - "Use `Arrays.deepEquals` as the `equals` method" — Map calls `equals` on the key object; you can't change array's `equals`. - "Use `IdentityHashMap`" — that's identity-based, the opposite of what's wanted.

Common follow-up: "Why doesn't Java provide value-based `equals` for arrays?"

Scenario 45 · The wait without while

Asked at: JP Morgan · **Cred:** · **Difficulty:** Hard · **Topic:** wait/notify + spurious wakeups

The code:

```
synchronized (lock) {  
    if (!ready) {  
        lock.wait();  
    }  
    process();  
}
```

The interviewer's prompt: "Sometimes `process()` is called before `ready` is actually true. We use `wait/notify` exactly as the textbook shows. What's wrong?"

Thinking out loud: 1. "Java permits spurious wakeups — `wait()` can return without `notify()` ever being called." 2. " `if (!ready)` checks once. After a spurious wakeup, the code falls through and calls `process` even if `ready` is still false." 3. "Standard pattern: `while (!ready) lock.wait();`. Re-check after every wakeup."

The actual bug: Using `if` instead of `while` around `wait()`. A spurious wakeup or a wakeup for a different condition slips through.

The fix (1-3 options if applicable): - **Option A:** Replace `if` with `while`. Standard wait-loop idiom. - **Option B:** Use `Condition` from `java.util.concurrent.locks` — same rule applies (use `while`), but the API encourages clearer signalling. - **Option C:** Use `BlockingQueue` or `CompletableFuture` — higher-level primitives don't require this discipline.

What NOT to say: - "Java doesn't have spurious wakeups" — it explicitly allows them. - "Notify everyone via `notifyAll` and the bug goes away" — doesn't fix the missing recheck.

Common follow-up: "What's the difference between `notify` and `notifyAll`, and when to pick each?"

Scenario 46 · The synchronized that blocks itself

Asked at: Cred · Walmart Labs · **Difficulty:** Medium · **Topic:** synchronized on different objects

The code:

```
class Counter {  
    private int n = 0;  
    public synchronized void inc() { n++; }  
    public int get() { return n; }    // not synchronized  
}
```

The interviewer's prompt: " `inc()` is synchronized so writes are safe. We don't synchronize `get()` because reads are atomic for `int`. Reviewers say there's still a bug. Why?"

Thinking out loud: 1. "Atomicity is fine for primitive int reads. But visibility is not." 2. "`get()` may return a stale value cached in a register, never observing the write made by another thread inside `inc()`." 3. "Need either `volatile`, `synchronized` on get, or `AtomicInteger`."

The actual bug: Synchronized writes but unsynchronized reads. No happens-before relationship; reads may see stale data.

The fix (1-3 options if applicable): - **Option A:** `public synchronized int get() { return n; }` — establishes the same monitor relationship. - **Option B:** Make `n` volatile and drop `synchronized` on `inc` — but then `inc()` is non-atomic again, so switch to `AtomicInteger`. - **Option C:** Use `AtomicInteger` and remove `synchronized` entirely — atomic and visible by spec.

What NOT to say: - "Reading int is atomic so we're safe" — true for atomicity, false for visibility. - "Reads are always consistent on x86" — JMM, not the hardware, defines the contract.

Common follow-up: "What's the happens-before relationship between synchronized blocks on the same monitor?"

Scenario 47 · The for-each on a null collection

Asked at: TCS · Wipro · **Difficulty:** Easy · **Topic:** Null collection iteration

The code:

```
public BigDecimal sum(List<Order> orders) {
    BigDecimal total = BigDecimal.ZERO;
    for (Order o : orders) {
        total = total.add(o.getAmount());
    }
    return total;
}
```

The interviewer's prompt: "For some inputs we get NPE on the for-each line. What's the cleanest fix?"

Thinking out loud: 1. "`for (X x : collection)` requires the collection to be non-null. Null throws NPE on the `iterator()` call." 2. "Either guard, default to empty, or change the API contract." 3. "Prefer returning empty over null from the source method; many bugs disappear."

The actual bug: For-each on a null collection throws NPE. The method accepts null without documenting it.

The fix (1-3 options if applicable): - **Option A:** Defensive: `if (orders == null) return BigDecimal.ZERO;` . Cheapest at the call site. - **Option B:** Helper: `for (Order o : Optional.ofNullable(orders).orElse(List.of())) { ... }` . Clearer intent. - **Option C:** Change the source — never return null; return an empty list. Removes the bug class.

What NOT to say: - "Java should auto-handle null collections in for-each" — it doesn't; this is by spec. - "Catch NPE and ignore" — masks all NPEs in the body, not just the collection one.

Common follow-up: "What's `Collections.emptyList()` vs `List.of()` in terms of guarantees?"

Scenario 48 · The race on `isShutdown`

Asked at: JP Morgan · **Cred:** · **Difficulty:** Hard · **Topic:** `ExecutorService` shutdown

The code:

```
executor.shutdown();
if (executor.isTerminated()) {
    cleanup();
}
```

The interviewer's prompt: "`shutdown()` is called but `isTerminated()` returns false right after, even though we have no tasks. `cleanup` never runs. What's wrong?"

Thinking out loud: 1. "`shutdown()` initiates an orderly shutdown — it returns immediately. Doesn't wait for tasks." 2. "`isTerminated()` is true only once all currently-running tasks complete." 3. "Need `awaitTermination`, or wait until `isTerminated` in a loop, or use `shutdownNow` if you don't need graceful."

The actual bug: `shutdown()` is non-blocking. `isTerminated()` immediately after will almost always be false.

The fix (1-3 options if applicable): - **Option A:** `executor.shutdown(); if (!executor.awaitTermination(30, TimeUnit.SECONDS)) { executor.shutdownNow(); } cleanup();`. Block up to 30s, force after. - **Option B:** `shutdownNow()` if you don't care about graceful drain — returns the list of unprocessed tasks. - **Option C:** Hook into JVM shutdown: `Runtime.getRuntime().addShutdownHook(new Thread(() -> executor.shutdown()))`. Only if you really mean process-wide.

What NOT to say: - "Loop with sleep until `isTerminated`" — works but reinventing `awaitTermination`. - "shutdown is synchronous" — it is not.

Common follow-up: "What's the difference between `shutdown` and `shutdownNow` in terms of in-flight tasks?"

Scenario 49 · The clone that shares the inner list

Asked at: TCS · Cognizant · **Difficulty:** Medium · **Topic:** Shallow clone

The code:

```
class Cart implements Cloneable {
    List<Item> items = new ArrayList<>();
    public Cart clone() {
        try { return (Cart) super.clone(); }
        catch (CloneNotSupportedException e) { throw new AssertionError(); }
    }
}

Cart a = new Cart(); a.items.add(new Item("X"));
Cart b = a.clone();
b.items.add(new Item("Y"));
System.out.println(a.items.size()); // 2, expected 1
```

The interviewer's prompt: "We clone the cart and modify the clone. The original is also modified. Why?"

Thinking out loud: 1. " `Object.clone()` does a shallow copy — primitive fields are copied, object references are shared." 2. "a.items and b.items point to the SAME ArrayList. Modifying either modifies both." 3. "Need a deep copy of mutable referenced state."

The actual bug: Default `Object.clone()` is shallow. Inner mutable collections are shared between original and clone.

The fix (1-3 options if applicable): - **Option A:** Override clone to deep-copy: `Cart c = (Cart) super.clone(); c.items = new ArrayList<>(items); return c;` - **Option B:** Drop Cloneable entirely — implement a copy constructor `public Cart(Cart other) { this.items = new ArrayList<>(other.items); }`. Idiomatic and explicit. - **Option C:** Make Cart a record with an immutable List — copies become trivial via `with` -style helpers.

What NOT to say: - "Cloneable is fine, just call it twice" — calling clone again still copies references. - "Add a copy constructor and use it inside clone()" — fine, but Cloneable buys you nothing then.

Common follow-up: "Why is Cloneable considered a broken interface?"

Scenario 50 · The HashMap loaded from JSON

Asked at: Razorpay · Cred · **Difficulty:** Medium · **Topic:** Jackson + map types

The code:

```
Map<Long, BigDecimal> balances = mapper.readValue(json, Map.class);
BigDecimal bal = balances.get(123L); // null even when key is in JSON
```

The interviewer's prompt: "Jackson reads a Map from JSON. Lookup by Long key returns null. The JSON definitely has '123' as a key."

Thinking out loud: 1. "Erasure on the call site — `Map.class` discards the type parameters. Jackson defaults to String keys and Object values (typically String, Integer, Double, etc.)." 2. "The actual map at runtime is `Map<String, Object>`. Key `"123"` is a String, not a Long. `get(123L)` misses." 3. "Need a `TypeReference` for parameterized deserialization."

The actual bug: Erasure at the call site. `Map.class` doesn't carry type parameters; Jackson deserializes to a String-keyed map.

The fix (1-3 options if applicable): - **Option A:** `mapper.readValue(json, new TypeReference<Map<Long, BigDecimal>>() {})`. The anonymous subclass preserves the generic info. - **Option B:** `mapper.getTypeFactory().constructMapType(Map.class, Long.class, BigDecimal.class)` — slightly more verbose. - **Option C:** For JSON-native string keys, change the field type to `Map<String, BigDecimal>` and parse keys to Long at use sites. Honest about the wire format.

What NOT to say: - "Generics work at runtime in Java" — they don't. - "Use Gson, it handles this" — Gson has the same issue; you'd use `TypeToken`.

Common follow-up: "How does `TypeReference` capture the type info despite erasure?"

Scenario 51 · The Comparable that throws on null

Asked at: Walmart Labs · JP Morgan · **Difficulty:** Medium · **Topic:** Comparable null handling

The code:

```
class Order implements Comparable<Order> {
    String city;
    public int compareTo(Order o) {
        return city.compareTo(o.city);
    }
}

orders.sort(null);
```

The interviewer's prompt: "Sorting throws NPE when city is null for some orders. Catalog of orders allows null city (international ones). What's the right approach?"

Thinking out loud: 1. "compareTo dereferences `city` directly. If `this.city` or `o.city` is null, NPE." 2. "Sort.natural() doesn't tolerate nulls. Need explicit null handling." 3. "Use `Comparator.nullsFirst(naturalOrder())` on the field, not naive compareTo."

The actual bug: `compareTo` doesn't handle null fields. Sorting blows up the moment one element has null `city`.

The fix (1-3 options if applicable): - **Option A:** `Comparator.comparing(Order::getCity, Comparator.nullsFirst(naturalOrder()))`. Nulls go first; sort is total. - **Option B:** In `compareTo`: `if (city == null) return o.city == null ? 0 : -1; if (o.city == null) return 1; return city.compareTo(o.city);`. Verbose but explicit. - **Option C:** Reject null at construction — `Objects.requireNonNull(city)`. Solve at the source if domain allows.

What NOT to say: - "Catch NPE in `compareTo`" — masks broken contract; sort can throw anywhere. - "Sort with parallel stream — it skips nulls" — false; nulls still crash.

Common follow-up: "What's the difference between `nullsFirst` and `nullsLast` for sort stability?"

Scenario 52 · The Long autobox that exceeds int

Asked at: Zerodha · Groww · **Difficulty:** Medium · **Topic:** Numeric promotion + overflow

The code:

```
int seconds = 60 * 60 * 24 * 365;
long ms = seconds * 1000;
```

The interviewer's prompt: "Computing milliseconds in a year. `ms` comes out wildly wrong — actually negative. Why?"

Thinking out loud: 1. "`60 * 60 * 24 * 365 = 31,536,000` — fits in `int`." 2.

"`seconds * 1000` — both operands `int`. Multiplication done in `int`, overflows to ~ -1.4 billion. THEN promoted to `long`." 3. "Need to promote before multiplication: `(long) seconds * 1000` or use a `long` literal `1000L`."

The actual bug: `int` multiplication overflows before the assignment-time widening to `long`.

The fix (1-3 options if applicable): - **Option A:** `long ms = (long) seconds * 1000;`. The cast promotes the left operand; multiplication is done in `long`. - **Option B:** `long ms = seconds * 1000L;`. The `L` literal forces long arithmetic. - **Option C:** Use `TimeUnit.SECONDS.toMillis(seconds)`. Most readable.

What NOT to say: - "Java auto-promotes for assignment" — only after the right-hand expression is fully evaluated. - "Use double" — loses precision for big values.

Common follow-up: "How would `Math.multiplyExact` change this code?"

Scenario 53 · The lazy initialization that races

Asked at: Cred · Razorpay · **Difficulty:** Hard · **Topic:** Race on lazy init

The code:

```
public class GeoLookup {
    private Map<String, City> cache;

    public City lookup(String pin) {
        if (cache == null) {
            cache = loadFromCsv();
        }
        return cache.get(pin);
    }
}
```

The interviewer's prompt: "Service occasionally returns null for valid pin codes. CSV is fine; cache is loaded once at startup. Under load. Why intermittent?"

Thinking out loud: 1. "Two threads race on `cache == null`. Both call `loadFromCsv`. Both reassign `cache`." 2. "While `loadFromCsv` is in progress, another thread sees a partially-populated map (depending on impl) — or a fresh empty map, or the right one." 3. "Visibility issue too: without `volatile`, other threads may keep using a stale null view."

The actual bug: Unsynchronized lazy initialization. Race condition + visibility issue.

The fix (1-3 options if applicable): - **Option A:** Eagerly load in constructor or static initializer — fastest, simplest. Removes laziness. - **Option B:** Double-checked locking with `private volatile Map<String, City> cache;` and the standard DCL pattern. - **Option C:** Holder idiom: `private static class Holder { static final Map<String, City> CACHE = loadFromCsv(); }`. Thread-safe lazy init for free.

What NOT to say: - "Synchronize the method" — works but slow on every call. - "Use `ConcurrentHashMap` as `cache`" — doesn't fix the race on the reference assignment.

Common follow-up: "What's the holder-class idiom and why is it safe?"

Scenario 54 · The finally that hides the throw

Asked at: TCS · Wipro · **Difficulty:** Medium · **Topic:** Finally swallow

The code:

```
public Order load() {
    try {
        return repo.fetch();
    } finally {
        return cachedFallback();
    }
}
```

The interviewer's prompt: "This always returns the fallback even when fetch succeeds. Why?"

Thinking out loud: 1. " `return` in a finally block overrides any prior return or exception." 2. "Try's return value is computed but discarded. Finally returns its own thing." 3. "Worse — if fetch throws, finally's return SWALLOWS the exception silently."

The actual bug: `return` in `finally` overrides the try return and any pending exception.

The fix (1-3 options if applicable): - **Option A:** Use try-catch for fallback intent: `try { return repo.fetch(); } catch (Exception e) { log.warn(...); return cachedFallback(); }` . - **Option B:** Never put return in finally. Finally is for cleanup only — close resources, restore flags. - **Option C:** Use try-with-resources for cleanup; keep returns inside try/catch.

What NOT to say: - "Finally only runs on exception" — it runs on every exit including normal return. - "Use System.exit(0) instead of return" — terrible.

Common follow-up: "What happens if finally also throws an exception?"

Scenario 55 · The catch with the wrong order

Asked at: Infosys · Cognizant · **Difficulty:** Easy · **Topic:** Catch ordering

The code:

```
try {
    parse(input);
} catch (Exception e) {
    log.error("unknown", e);
} catch (NumberFormatException e) {
    log.warn("bad number", e);
}
```

The interviewer's prompt: "This won't compile. Reorder."

Thinking out loud: 1. "NumberFormatException is a subclass of Exception. Catching the superclass first means the second clause is unreachable." 2. "Javac flags this at compile time: `exception NumberFormatException has already been caught` ." 3. "Reorder — most-specific first."

The actual bug: Catch clauses are checked in order. A superclass catch first makes subclass catches unreachable.

The fix (1-3 options if applicable): - **Option A:** Reorder: `NumberFormatException` first, `Exception` second. - **Option B:** Multicatch if logic is identical: `catch (NumberFormatException | DateTimeParseException e) { ... }`. - **Option C:** Avoid catching `Exception` at all — too broad. Catch named types you actually expect.

What NOT to say: - "Java auto-orders catches" — it doesn't. - "Rename the variable" — won't fix anything.

Common follow-up: "When is catching `Throwable` ever justified?"

Scenario 56 · The `ConcurrentHashMap` that still races

Asked at: PhonePe · JP Morgan · **Difficulty:** Hard · **Topic:** `ConcurrentHashMap` + compound ops

The code:

```
ConcurrentHashMap<String, Integer> counters = new ConcurrentHashMap<>();

public void increment(String key) {
    Integer current = counters.get(key);
    counters.put(key, current == null ? 1 : current + 1);
}
```

The interviewer's prompt: "We switched from `HashMap` to `ConcurrentHashMap` to fix races. Counters are still off. What now?"

Thinking out loud: 1. "`ConcurrentHashMap` makes individual operations atomic. Compound operations (read-modify-write) are not." 2. "Two threads can both read 5, both write 6. Same lost-update problem as before." 3. "Use `computeIfAbsent` or `merge` — the map provides atomic compound primitives."

The actual bug: Compound read-modify-write isn't atomic across `get` + `put` on `ConcurrentHashMap`.

The fix (1-3 options if applicable): - **Option A:** `counters.merge(key, 1, Integer::sum);` — atomic increment by 1. - **Option B:** `counters.compute(key, (k, v) -> v == null ? 1 : v + 1);` — atomic per-key computation. - **Option C:** `ConcurrentHashMap<String, LongAdder> counters;` `counters.computeIfAbsent(key, k -> new LongAdder()).increment();`. Best for hot keys.

What NOT to say: - "`ConcurrentHashMap` is broken" — it's not; you're using it wrong. - "Wrap in `synchronized`" — defeats the purpose.

Common follow-up: "Why is `merge` implemented atomically in `ConcurrentHashMap`?"

Scenario 57 · The currency stored as double

Asked at: Zerodha · Razorpay · Cred · **Difficulty:** Easy · **Topic:** Floating-point precision

The code:

```
double balance = 0.0;
for (int i = 0; i < 10; i++) {
    balance += 0.1;
}
System.out.println(balance == 1.0); // false
```

The interviewer's prompt: "Reconciliation report says 1.0 doesn't equal 1.0. What's going on with this loop?"

Thinking out loud: 1. "0.1 is not exactly representable in binary floating point." 2. "Sum of ten approximations of 0.1 is something like 0.9999999999999999." 3. "Never use double for money. Use `BigDecimal` or long-paise."

The actual bug: Floating-point representation. 0.1 in binary is a repeating fraction; sum drifts.

The fix (1-3 options if applicable): - **Option A:** Use `BigDecimal`: `BigDecimal balance = BigDecimal.ZERO; balance = balance.add(new BigDecimal("0.1"));` . Exact decimal arithmetic. - **Option B:** Use long for paise (smallest currency unit). Add as integers, divide by 100 at display. - **Option C:** For comparison only: `Math.abs(balance - 1.0) < 1e-9` . Works for tolerance comparison but not for storage.

What NOT to say: - "Round at every step with `Math.round`" — loses precision differently. - "Use float, it's smaller" — same problem, worse precision.

Common follow-up: "What's the difference between `BigDecimal` scale and precision?"

Scenario 58 · The instanceof that misses subclasses

Asked at: TCS · Walmart Labs · **Difficulty:** Easy · **Topic:** instanceof / pattern matching

The code:

```
if (animal.getClass() == Dog.class) {
    bark((Dog) animal);
}
```

The interviewer's prompt: "This works for plain Dog objects but not for our subclass `Labrador` extends `Dog` . We need to bark for both. "

Thinking out loud: 1. "`getClass() == Dog.class` is strict identity. Subclasses' `getClass` returns `Labrador.class`, not `Dog.class`." 2. "`instanceof` matches the class and any subclass." 3. "Use `animal instanceof Dog d` (Java 16+ pattern) for cleanest form."

The actual bug: Strict class equality misses subclass instances.

The fix (1-3 options if applicable): - **Option A:** `if (animal instanceof Dog d) bark(d);`. Pattern matching for `instanceof`; clean cast-free. - **Option B:** Pre-Java-16: `if (animal instanceof Dog) { bark((Dog) animal); }`. - **Option C:** Polymorphism: define `bark()` on `Animal`; subclasses override. No `instanceof` at all.

What NOT to say: - "Use `getClass().isAssignableFrom(Dog.class)`" — backwards; that checks whether `Dog` is a superclass of `animal`'s class. - "Use `Class.equals`" — same identity issue.

Common follow-up: "Why is `getClass() ==` faster than `instanceof`?"

Scenario 59 · The Spring bean that survives nulls

Asked at: Walmart Labs · JP Morgan · **Difficulty:** Medium · **Topic:** Spring constructor injection

The code:

```
@Service
class OrderService {
    @Autowired
    private PaymentService paymentService;

    public void place(Order o) {
        paymentService.charge(o);
    }
}
```

The interviewer's prompt: "In some configurations `PaymentService` is null and `place()` throws NPE. We expected the context to fail to start. Why doesn't it?"

Thinking out loud: 1. "Field injection. Spring will try to wire but won't fail if no candidate is found in some configurations." 2. "Constructor injection makes dependencies mandatory — context fails on startup if missing." 3. "Also field injection is hard to test (have to use reflection to set)."

The actual bug: Field injection allows the bean to instantiate without the dependency. The error appears at first call, not at startup.

The fix (1-3 options if applicable): - **Option A:** Constructor injection: `private final PaymentService paymentService; public OrderService(PaymentService ps) { this.paymentService = ps; }`. Spring auto-wires; missing dep = context failure at startup. - **Option B:** Mark field `@Autowired(required = true)` explicitly — defaults to true, but be explicit. -

Option C: Lombok: `@RequiredArgsConstructor` on a class with `private final` fields — auto-generates the constructor.

What NOT to say: - "Add `@PostConstruct` to verify" — works but late; fail at wiring time, not init. - "Spring will always fail loud" — depends on `required` setting and bean profiles.

Common follow-up: "What's the difference between `@Autowired` on field, setter, and constructor?"

Scenario 60 · The text block that leaks indentation

Asked at: Razorpay · Cred · **Difficulty:** Easy · **Topic:** Text blocks + incidental whitespace

The code:

```
String sql = ""
    SELECT id, amount
      FROM payments
    WHERE customer_id = ?
"";
log.info(sql);
```

The interviewer's prompt: "The SQL is logged with weird leading spaces. We thought text blocks strip indentation. What's the rule?"

Thinking out loud: 1. "Text blocks strip incidental whitespace based on the position of the closing `\"\"\"\"`." 2. "Here the closing `\"\"\"\"` is at column 0. So no indentation is stripped — every line keeps its leading spaces." 3. "Move the closing delimiter to the indent level you want as the baseline."

The actual bug: Closing delimiter is at column 0; no whitespace is treated as incidental.

The fix (1-3 options if applicable): - **Option A:** Move closing delimiter to match the indentation you want stripped:

```
String sql = ""
    SELECT id, amount
      FROM payments
    WHERE customer_id = ?
    "";
```

- **Option B:** Call `.stripIndent()` explicitly if you can't control the closing position.
- **Option C:** Use `String.indent(int n)` to add/remove indentation programmatically.

What NOT to say: - "Text blocks always strip all indentation" — they don't; they strip to the min common indent including the closing delimiter. - "Use `\s` to remove whitespace" — `\s` produces a single space, not strip incidental whitespace.

Common follow-up: "What's the difference between `\\n` and `\\s` in a text block?"

Scenario 61 · The scheduled task that stops silently

Asked at: Walmart Labs · **Cred:** · **Difficulty:** Medium · **Topic:** ScheduledExecutorService exceptions

The code:

```
ScheduledExecutorService scheduler = Executors.newSingleThreadScheduledExecutor();
scheduler.scheduleAtFixedRate(() -> {
    cleanupExpiredSessions();
}, 0, 1, TimeUnit.MINUTES);
```

The interviewer's prompt: "Cleanup runs every minute for hours, then stops. No error log. Why?"

Thinking out loud: 1. "scheduleAtFixedRate cancels the task if the runnable ever throws — and the exception goes to the thread's UncaughtExceptionHandler, which by default is silent." 2.

"cleanupExpiredSessions threw something transient (network blip, DB lock). The whole schedule died." 3.

"Always wrap the body in try-catch inside the runnable."

The actual bug: An uncaught exception in a scheduled task cancels all future runs silently.

The fix (1-3 options if applicable): - **Option A:** Wrap the body: `try`

`{ cleanupExpiredSessions(); } catch (Throwable t) { log.error("cleanup failed", t); }`. Schedule survives. - **Option B:** Add a custom ThreadFactory with an

UncaughtExceptionHandler that logs — covers other code paths too. - **Option C:** Use Spring's

`@Scheduled` with an `@EventListener` for ApplicationContextEvent — Spring logs by default.

What NOT to say: - "Restart the scheduler in a finally" — no finally fires if you never return. - "Use scheduleWithFixedDelay, it's more reliable" — same cancellation-on-exception behaviour.

Common follow-up: "What's the difference between scheduleAtFixedRate and scheduleWithFixedDelay?"

Scenario 62 · The enum compared with equals

Asked at: TCS · Cognizant · **Difficulty:** Easy · **Topic:** Enum identity

The code:

```
enum Status { ACTIVE, INACTIVE, PENDING }

if (order.getStatus().equals(Status.ACTIVE)) {
    process(order);
}
```

The interviewer's prompt: "This works but throws NPE if getStatus() returns null. Cleaner pattern?"

Thinking out loud: 1. "equals on enum is fine but dereferences the receiver. NPE if status is null." 2. "For enums, == is safe and correct — enum instances are singletons." 3. "And Status.ACTIVE.equals(order.getStatus()) flips to a null-safe form."

The actual bug: equals on a possibly-null receiver. Enums are singletons; reference equality is correct AND null-safe.

The fix (1-3 options if applicable): - **Option A:** `if (order.getStatus() == Status.ACTIVE)` — short, fast, null-safe (null == ACTIVE is false, no NPE). - **Option B:** `if (Status.ACTIVE.equals(order.getStatus()))` — also null-safe, idiomatic for Java < 5 style. - **Option C:** Switch expression: `return switch (order.getStatus()) { case ACTIVE -> ...; default -> ...; }.`

What NOT to say: - "Use equals always for safety" — wrong for enums; == is faster and null-safe. - "Null-check first then equals" — verbose; == already handles null.

Common follow-up: "Why is == safe for enums but not for Integer in general?"

Scenario 63 · The toString that recurses

Asked at: TCS · Wipro · **Difficulty:** Easy · **Topic:** Cyclic toString → StackOverflowError

The code:

```
class Department {
    String name; List<Employee> employees;
    public String toString() { return name + " " + employees; }
}
class Employee {
    String name; Department dept;
    public String toString() { return name + " of " + dept; }
}
```

The interviewer's prompt: "Printing a Department crashes with StackOverflowError. Cycle in the data model. Fix?"

Thinking out loud: 1. "Department.toString prints its employees. Each employee's toString prints its department. Infinite recursion." 2. "Common with JPA bidirectional associations + Lombok @ToString." 3. "Break the cycle in toString — print just IDs, or exclude one side."

The actual bug: Mutual references trigger infinite toString recursion.

The fix (1-3 options if applicable): - **Option A:** Make one side's toString omit the back-reference:

Employee.toString() { return name + " (id=" + dept.id + ")"; } . - **Option B:** Lombok: @ToString(exclude = "dept") or @ToString.Exclude on the field. - **Option C:** Use a JSON serializer with bidirectional annotation (Jackson's @JsonManagedReference / @JsonBackReference) for serialization paths.

What NOT to say: - "Catch StackOverflowError" — masks the design issue and isn't reliable to recover from. - "Use a Set to detect cycles" — overkill; the right fix is to not recurse.

Common follow-up: "What other Object methods are commonly broken by bidirectional refs?"

Scenario 64 · The list returned from Arrays.asList holding primitives

Asked at: Infosys · TCS · **Difficulty:** Easy · **Topic:** Arrays.asList on primitive array

The code:

```
int[] nums = {1, 2, 3};
List<int[]> l = Arrays.asList(nums);
System.out.println(l.size()); // 1
```

The interviewer's prompt: "We expected a List of size 3. Got a List of one element. Why?"

Thinking out loud: 1. "Arrays.asList has signature <T> List<T> asList(T... elements) . T must be reference type — autoboxing doesn't promote int[] to Integer[]." 2. "int[] is itself an Object. asList wraps it as a single element of type int[]." 3. "For Integer[] it works; for int[] you need to box manually or use streams."

The actual bug: Arrays.asList on a primitive array treats the array as a single element of type int[] .

The fix (1-3 options if applicable): - **Option A:**

Arrays.stream(nums).boxed().collect(toList()) . Stream auto-boxes ints. - **Option B:** IntStream.of(nums).boxed().toList() — cleaner with Java 16+. - **Option C:** Use Integer[] from the start if you want a real list.

What NOT to say: - "Arrays.asList autoboxes" — only references; not primitive arrays. - "Use Collections.singletonList(nums)" — explicit but same effect.

Common follow-up: "What if you call `Arrays.asList(1, 2, 3)` directly with int literals?"

Scenario 65 · The HashMap initial capacity

Asked at: JP Morgan · **Cred:** · **Difficulty:** Medium · **Topic:** HashMap sizing / load factor

The code:

```
Map<String, Customer> cache = new HashMap<>(1000);  
// load exactly 1000 customers
```

The interviewer's prompt: "We pre-size the map for 1000 entries. Profiler still shows several rehashes during loading. Why?"

Thinking out loud: 1. "HashMap's default load factor is 0.75. Rehash triggers when $\text{size} > \text{capacity} * \text{load factor}$." 2. "Capacity 1000, load factor 0.75 → rehash at 750 entries. Multiple rehashes follow as more are added and the bucket array grows." 3. "Size hint should be `expected / loadFactor` rounded up, or use the static helper `HashMap.newHashMap(expectedSize)` (Java 19+)."

The actual bug: Initial capacity = expected size, not accounting for load factor. Rehash kicks in at 75%.

The fix (1-3 options if applicable): - **Option A:** Java 19+: `HashMap.newHashMap(1000)` — handles the math for you. - **Option B:** Manual: `new HashMap<>((int) (1000 / 0.75f) + 1)` — sized to avoid rehash for the expected load. - **Option C:** Bump initial capacity well past expected — wastes memory but avoids rehash. Bad trade-off usually.

What NOT to say: - "Pre-sizing doesn't matter" — it does; rehashes are $O(n)$ operations. - "Decrease load factor to 1.0 to avoid rehash" — increases collision rate; worse lookup performance.

Common follow-up: "What's the default load factor and why was 0.75 chosen?"

Scenario 66 · The string concatenation in a loop

Asked at: TCS · Cognizant · **Difficulty:** Easy · **Topic:** String + in loop

The code:

```
String csv = "";  
for (Order o : millionOrders) {  
    csv += o.toCsv() + "\n";  
}
```

The interviewer's prompt: "This works for hundreds of orders. At a million it takes 4 minutes and uses massive heap. Why?"

Thinking out loud: 1. " `+=` on String creates a new String each iteration. Quadratic time, quadratic allocation." 2. "At a million entries, you allocate hundreds of GB of temporary Strings." 3. "Use `StringBuilder` or `Stream.collect(joining)`."

The actual bug: String is immutable. `+=` builds a fresh String each iteration, $O(n^2)$ time and allocation.

The fix (1-3 options if applicable): - **Option A:**

`StringBuilder sb = new StringBuilder(); for (Order o : orders) { sb.append(o.toCsv()).append('\n'); } String csv = sb.toString();` . Amortized linear. - **Option B:**

`orders.stream().map(Order::toCsv).collect(Collectors.joining("\n"));` . Same idea, idiomatic. - **Option C:** Stream directly to a Writer — never hold the full CSV in memory.

What NOT to say: - "Java optimizes string concatenation in loops" — only for single-statement concatenation (`a + b + c`), not loops. - "Use `String.format`" — even slower.

Common follow-up: "What does `javac` do with `String s = a + b + c;` outside a loop?"

Scenario 67 · The lock that protects nothing

Asked at: Walmart Labs · JP Morgan · **Difficulty:** Hard · **Topic:** Wrong lock object

The code:

```
class TransferService {
    private final Object lock = new Object();

    public void transfer(Account from, Account to, BigDecimal amount) {
        synchronized (lock) {
            from.withdraw(amount);
            to.deposit(amount);
        }
    }
}

// many TransferService instances created
```

The interviewer's prompt: "We lock to prevent concurrent transfers. Each `TransferService` has its own lock. Different instances racing on the same accounts still corrupt balances. Why?"

Thinking out loud: 1. "Lock is per-instance. Two `TransferService` instances → two locks → no mutual exclusion." 2. "Locking should be on a shared object — the `Account` itself, or a static lock keyed by account ID." 3. "In a clustered deployment you'd need a distributed lock (`Redisson`, `ZooKeeper`)."

The actual bug: Lock is scoped to the wrong object. Multiple service instances don't synchronize.

The fix (1-3 options if applicable): - **Option A:** Lock on the Account objects in canonical order to avoid deadlock: `synchronized (id1 < id2 ? from : to) { synchronized (id1 < id2 ? to : from) { ... } }`. - **Option B:** Make the service a singleton (Spring default) and the lock per-account via a `ConcurrentHashMap<Long, ReentrantLock>`. - **Option C:** For correctness at scale, run transfers through a DB transaction with row-level locks (`SELECT ... FOR UPDATE`).

What NOT to say: - "Static lock on the class" — works for single JVM but serializes all transfers; throughput cliff. - "Use ConcurrentHashMap for the accounts" — doesn't help; locking is about the operation, not the storage.

Common follow-up: "How do you avoid deadlock when locking two accounts at once?"

Scenario 68 · The Iterator that mutates the wrong list

Asked at: Cognizant · TCS · **Difficulty:** Medium · **Topic:** Iterator + sublist aliasing

The code:

```
List<Order> orders = new ArrayList<>(fetchAll());
List<Order> highValue = orders.subList(0, 5);
highValue.clear();
System.out.println(orders.size()); // 5 less
```

The interviewer's prompt: "We took a subList and cleared it. The original list lost those rows too. Is that expected?"

Thinking out loud: 1. " `subList` returns a VIEW backed by the original list, not a copy." 2. "Modifications via the view modify the underlying list." 3. "Documented; common surprise. To copy, wrap: `new ArrayList<>(orders.subList(0, 5))` ."

The actual bug: `subList` returns a live view. Mutations propagate to the backing list.

The fix (1-3 options if applicable): - **Option A:** Copy explicitly: `List<Order> highValue = new ArrayList<>(orders.subList(0, 5));`. Independent copy. - **Option B:** If you need a view but want immutability, wrap: `Collections.unmodifiableList(orders.subList(0, 5))`. - **Option C:** Use a stream: `orders.stream().limit(5).toList()` — independent immutable list.

What NOT to say: - "Use Collections.copy()" — does an in-place copy; not relevant here. - "subList is broken" — it's a documented view.

Common follow-up: "What happens to the subList view if you structurally modify the original list?"

Scenario 69 · The InputStream that doesn't read fully

Asked at: Razorpay · Walmart Labs · **Difficulty:** Medium · **Topic:** InputStream.read return value

The code:

```
byte[] buf = new byte[1024];
int n = inputStream.read(buf);
String body = new String(buf, 0, n);
```

The interviewer's prompt: "We read a POST body. Small requests work. Larger ones occasionally come back truncated. Why?"

Thinking out loud: 1. "`read(buf)` returns the number of bytes actually read in this call — may be less than `buf.length`, even if more is available." 2. "For streams from network, partial reads are routine. The contract requires a loop until `-1`." 3. "Or use a helper: `InputStream.readAllBytes()` (Java 9+) or `IOUtils.toByteArray`."

The actual bug: `read(byte[])` does not guarantee filling the buffer. A single call may return fewer bytes; truncation results.

The fix (1-3 options if applicable): - **Option A:** Use `InputStream.readAllBytes()` — Java 9+ helper, reads to EOF. - **Option B:** Loop manually: `while ((n = in.read(buf, offset, buf.length - offset)) > 0) offset += n;` with grow logic. - **Option C:** For HTTP requests, use the framework's built-in body parsing (`HttpServletRequest.getReader`, Spring's `@RequestBody`).

What NOT to say: - "read always fills the buffer" — it does not; only `InputStream.readNBytes` does (Java 9+). - "Add a sleep before read" — race-prone hack.

Common follow-up: "What's the difference between `read`, `readFully`, and `readAllBytes`?"

Scenario 70 · The clone of a record

Asked at: Cred · Razorpay · **Difficulty:** Medium · **Topic:** Records + cloning

The code:

```
record Cart(List<Item> items) {}

Cart a = new Cart(List.of(new Item("X")));
Cart b = new Cart(a.items());
b.items().add(new Item("Y")); // UnsupportedOperationException
```

The interviewer's prompt: "We pass a record's list field to a new record. Mutating the new one throws `UnsupportedOperationException`. Why?"

Thinking out loud: 1. "`List.of(...)` returns an immutable list. Mutating throws." 2. "Records don't auto-copy components. Both records reference the SAME immutable list." 3. "Either accept immutability (preferred) or make a defensive copy in a compact constructor."

The actual bug: Records share their component values by reference. The shared list is immutable.

The fix (1-3 options if applicable): - **Option A:** Lean into immutability: `record Cart(List<Item> items) { public Cart { items = List.copyOf(items); } }` — defensive copy + reassignment to immutable. - **Option B:** If mutation is needed, don't store a list field in a record. Records are designed for immutable data. - **Option C:** Provide a method that returns a new Cart with the added item: `public Cart with(Item i) { var copy = new ArrayList<>(items); copy.add(i); return new Cart(copy); } .`

What NOT to say: - "Records auto-deep-copy their fields" — they don't. - "Use `final` instead of record" — same problem; the list reference is final but still mutable.

Common follow-up: "What's a compact constructor and when do you use it?"

Scenario 71 · The SimpleDateFormat shared across threads

Asked at: JP Morgan · TCS · **Difficulty:** Hard · **Topic:** SimpleDateFormat thread safety

The code:

```
public class DateUtils {
    private static final SimpleDateFormat FMT = new SimpleDateFormat("yyyy-MM-dd");
    public static String format(Date d) { return FMT.format(d); }
}
```

The interviewer's prompt: "This sometimes produces nonsense dates like '0027-15-30' under load. Single instance, deterministic input. Why?"

Thinking out loud: 1. "SimpleDateFormat is NOT thread-safe. Internal Calendar field is mutated during format/parse." 2. "Concurrent threads stomp on the shared calendar — garbage output." 3. "Either use ThreadLocal, or migrate to java.time DateTimeFormatter (thread-safe)."

The actual bug: SimpleDateFormat is stateful and not thread-safe. Sharing it across threads corrupts output.

The fix (1-3 options if applicable): - **Option A:** Use `DateTimeFormatter.ofPattern("yyyy-MM-dd")` and `LocalDate.format(formatter)` . DateTimeFormatter is immutable and thread-safe. - **Option B:** Wrap in `ThreadLocal<SimpleDateFormat>` — each thread gets its own. Beware ThreadLocal leak risk in pools. - **Option C:** Create a fresh SimpleDateFormat per call. Cheap enough for most workloads.

What NOT to say: - "Synchronize the format call" — works but serializes a hot path. - "Use Joda-Time" — Joda is deprecated; java.time supersedes it.

Common follow-up: "Why is `DateTimeFormatter` thread-safe but `SimpleDateFormat` is not?"

Scenario 72 · The getter that exposes mutable state

Asked at: Walmart Labs · Cred · **Difficulty:** Medium · **Topic:** Defensive copy

The code:

```
public class Cart {
    private final List<Item> items = new ArrayList<>();
    public List<Item> getItems() { return items; }
    public void add(Item i) { items.add(i); }
}

Cart c = new Cart();
c.getItems().add(new Item("X")); // bypasses validation
```

The interviewer's prompt: "Cart has an `add()` method with validation. But callers add items via the getter, bypassing checks. What's the pattern to lock this down?"

Thinking out loud: 1. "Returning the internal list directly exposes mutability." 2. "Defensive copy on return: `new ArrayList<>(items)` — caller mutations have no effect on the cart." 3. "Or `Collections.unmodifiableList(items)` — read-only view, throws on modification attempts."

The actual bug: Getter exposes the live internal list. External code can mutate cart state without going through the validating API.

The fix (1-3 options if applicable): - **Option A:** `return Collections.unmodifiableList(items);` — cheap, signals immutability via runtime exceptions. - **Option B:** `return List.copyOf(items);` (Java 10+) — immutable copy, true protection. - **Option C:** Return a stream: `public Stream<Item> items() { return items.stream(); }`. Callers must collect to mutate.

What NOT to say: - "Make the list `final`" — it already is; final means the reference can't change, not the contents. - "Document that callers shouldn't mutate" — runtime checks beat documentation.

Common follow-up: "What's the performance cost of `Collections.unmodifiableList` vs `List.copyOf`?"

Scenario 73 · The infinite stream that never terminates

Asked at: Cred · Razorpay · **Difficulty:** Medium · **Topic:** Stream + infinite source

The code:

```
long primes = Stream.iterate(2, n -> n + 1)
    .filter(this::isPrime)
    .count();
```

The interviewer's prompt: "Counting primes hangs and pegs CPU. Why?"

Thinking out loud: 1. " `Stream.iterate(2, n -> n + 1)` is infinite." 2. " `count()` is a terminal op that traverses everything. On an infinite stream, it never returns." 3. "Need `limit(n)` to bound it."

The actual bug: Terminal `count()` on an infinite stream. No upper bound; runs forever.

The fix (1-3 options if applicable): - **Option A:** `Stream.iterate(2, n -> n + 1).filter(this::isPrime).limit(100).count();` — count first 100. - **Option B:** `Stream.iterate(2, n -> n <= 1000, n -> n + 1)` (Java 9+) — built-in predicate-bounded iterate. - **Option C:** Switch to `IntStream.rangeClosed(2, 1000).filter(this::isPrime).count();` — clearer bounded form.

What NOT to say: - "count() will eventually return on a finite machine" — no, the source itself is infinite. - "Use parallel" — `parallelStream` on infinite is even worse.

Common follow-up: "What's the difference between `Stream.iterate` and `Stream.generate`?"

Scenario 74 · The thread that doesn't start

Asked at: TCS · Wipro · **Difficulty:** Easy · **Topic:** Thread.run vs start

The code:

```
Thread t = new Thread(() -> processBatch());
t.run();
log.info("started");
```

The interviewer's prompt: "processBatch is slow but runs sequentially before 'started' is logged. We expected concurrency."

Thinking out loud: 1. " `run()` invokes the runnable directly on the current thread. No new thread is created." 2. " `start()` schedules the runnable on a NEW thread." 3. "Classic copy-paste mistake from a method called 'run'."

The actual bug: `Thread.run()` doesn't spawn a thread — it just calls the runnable inline.

The fix (1-3 options if applicable): - **Option A:** `t.start();` — spawns the thread. - **Option B:** Use an `ExecutorService`: `executor.submit(this::processBatch)` . Modern, returns a `Future`. - **Option C:** Java 21+: `Thread.ofVirtual().start(this::processBatch)` for a virtual thread.

What NOT to say: - "Thread.run is async too" — it is not. - "Increase the JVM thread count" — irrelevant; no new thread was requested.

Common follow-up: "Why is `Thread` itself the `Runnable` in some APIs?"

Scenario 75 · The serialization that drops a field

Asked at: JP Morgan · Walmart Labs · **Difficulty:** Medium · **Topic:** Jackson + missing setter / getter

The code:

```
class Order {
    private String id;
    private BigDecimal amount;

    public String getId() { return id; }
    public void setId(String id) { this.id = id; }
    // no getter/setter for amount
}

String json = mapper.writeValueAsString(new Order(...)); // {"id":"X"}
```

The interviewer's prompt: "Jackson doesn't include `amount` in the JSON output. Why?"

Thinking out loud: 1. "Jackson by default uses getter/setter visibility. Private fields without a getter are ignored." 2. "Add a getter for amount, or configure ObjectMapper to use fields directly." 3. "Or use a constructor annotation if amount is part of construction."

The actual bug: Jackson defaults to bean accessor visibility. Field without a getter is skipped.

The fix (1-3 options if applicable): - **Option A:** Add a public getter `public BigDecimal getAmount() { return amount; }` . Standard fix. - **Option B:** Configure: `mapper.setVisibility(PropertyAccessor.FIELD, Visibility.ANY);` . Uses fields directly. - **Option C:** Annotate field: `@JsonProperty("amount") private BigDecimal amount;` .

What NOT to say: - "Jackson reads private fields by default" — it does not; only public. - "Use Gson, it just works" — Gson defaults to fields but has its own gotchas.

Common follow-up: "How does Jackson handle records out of the box?"

Scenario 76 · The static field initialized in the wrong order

Asked at: Cred · JP Morgan · **Difficulty:** Hard · **Topic:** Static initialization order

The code:

```
class Config {
    public static final String HOST = System.getProperty("host");
    public static final URL ENDPOINT = buildEndpoint();

    static URL buildEndpoint() {
        try { return new URL("https://" + HOST + "/api"); }
        catch (MalformedURLException e) { throw new RuntimeException(e); }
    }
}
```

The interviewer's prompt: "In one deployment ENDPOINT is null. -Dhost=... is set. Why?"

Thinking out loud: 1. "Static initializers run top-to-bottom. HOST is read from system properties first." 2. "If -D is not set, HOST is null. buildEndpoint produces 'https://null/api' — not actually null." 3. "But if there's an exception or a circular dependency with another class, static init can leave a field at default."

The actual bug: Order of static initialization matters. If `System.getProperty` returns null (property not set), HOST is null and the URL becomes malformed garbage.

The fix (1-3 options if applicable): - **Option A:** Validate: `public static final String HOST = Objects.requireNonNull(System.getProperty("host"), "host property not set");`. Fail loud at class load. - **Option B:** Move to constructor injection (Spring's `@Value("${host}")`) — fails at app startup, not class load. - **Option C:** Lazy init: don't compute ENDPOINT until first use; allows the caller to set host later in tests.

What NOT to say: - "Static fields default to non-null" — they default per type; reference types default to null. - "Catch the exception in static init" — works but hides the config issue.

Common follow-up: "What happens if a static initializer throws an exception?"

Scenario 77 · The CompletableFuture that runs on the calling thread

Asked at: Razorpay · Cred · **Difficulty:** Hard · **Topic:** CompletableFuture default executor

The code:

```
CompletableFuture
    .supplyAsync(() -> heavyDbCall())
    .thenApply(this::transform)
    .thenAccept(this::publish);
```

The interviewer's prompt: *"This is supposed to be async. Profiler shows transform and publish running on the same thread as supplyAsync — sometimes the request thread. Why?"*

Thinking out loud: 1. " `thenApply` runs on whichever thread completes the prior stage. If `supplyAsync`'s task is already done when `thenApply` is wired, the calling thread runs it." 2. "For predictable async behaviour, use `thenApplyAsync` and pass an `Executor`." 3. "Default executor is the common `ForkJoinPool` — IO-bound work in it can starve other consumers."

The actual bug: `thenApply` (no Async) runs on whatever thread completes the prior stage — possibly the calling thread.

The fix (1-3 options if applicable): - **Option A:** Use `thenApplyAsync(this::transform, executor)` — guarantees async on the named executor. - **Option B:** Use `Executors.newVirtualThreadPerTaskExecutor()` (Java 21+) for IO-bound chains. - **Option C:** If you don't need true async between stages, accept the current behaviour — it's an optimization.

What NOT to say: - "thenApply always runs on the common pool" — it does not. - "Use `parallelStream` instead" — different abstraction; doesn't fix the issue.

Common follow-up: *"What's the risk of using the common `ForkJoinPool` for blocking calls?"*

Scenario 78 · The map with a comparator that contradicts equals

Asked at: JP Morgan · Walmart Labs · **Difficulty:** Hard · **Topic:** TreeMap comparator vs equals

The code:

```
TreeMap<String, Integer> caseInsensitive = new TreeMap<>(String.CASE_INSENSITIVE_ORDER);
caseInsensitive.put("Order-1", 100);
caseInsensitive.put("order-1", 200);
System.out.println(caseInsensitive.size()); // 1
```

The interviewer's prompt: *"We expect 2 entries. TreeMap collapses them to 1. Why? Equals on those strings is false."*

Thinking out loud: 1. "TreeMap orders and de-duplicates based on the Comparator, not equals." 2. "CASE_INSENSITIVE_ORDER treats 'Order-1' and 'order-1' as equal." 3. "This is by spec: TreeMap is inconsistent with equals when the comparator is."

The actual bug: TreeMap's notion of equality is `compare == 0`, not equals. Case-insensitive comparator collapses keys that are case-different.

The fix (1-3 options if applicable): - **Option A:** Use a `Comparator.comparing(String::toLowerCase).thenComparing(naturalOrder())` — distinguishes equal-lowercase entries. - **Option B:** Use a `HashMap` or `LinkedHashMap` if you want

equals-based semantics. - **Option C:** Pre-normalize keys (`toLowerCase`) before insert — explicit at the API.

What NOT to say: - "TreeMap is broken" — it's documented; the contract uses compare, not equals. - "Use a TreeSet of entries" — same problem; TreeSet uses compare.

Common follow-up: "When is it OK to have a comparator inconsistent with equals?"

Scenario 79 · The `Collectors.toList` that's immutable

Asked at: TCS · Razorpay · **Difficulty:** Easy · **Topic:** Stream `toList` vs `Collectors.toList`

The code:

```
List<Order> filtered = orders.stream()
    .filter(Order::isActive)
    .toList();

filtered.add(extra); // UnsupportedOperationException
```

The interviewer's prompt: " `.toList()` returns a list we can't add to. Used to work with `.collect(toList())` . What changed?"

Thinking out loud: 1. " `Stream.toList()` (Java 16+) returns an unmodifiable list." 2. " `Collectors.toList()` returns a mutable list (typically `ArrayList`) — by convention but not contract." 3. "If you need mutability, use `.collect(Collectors.toCollection(ArrayList::new))` or wrap the result."

The actual bug: `Stream.toList()` is unmodifiable. `Collectors.toList()` is mutable by current implementation.

The fix (1-3 options if applicable): - **Option A:** `.collect(toCollection(ArrayList::new))` — explicit mutable container. - **Option B:** Wrap: `new ArrayList<>(stream.toList())` — clean and mutable. - **Option C:** Don't mutate the collected list. Stream-derived results often shouldn't be mutated.

What NOT to say: - "Stream.toList is broken" — it's by design. - "Add the elements before collecting" — possible but defeats the point if extras aren't in the source.

Common follow-up: "What guarantees does `Collectors.toList` make about the returned type?"

Scenario 80 · The `String.format` that ignores arguments

Asked at: TCS · Wipro · **Difficulty:** Easy · **Topic:** printf format specifiers

The code:

```
log.info(String.format("Processing %s orders for customer %s",
    orders.size(), customer.getId()));
```

The interviewer's prompt: "Logs sometimes show 'Processing 142 orders for customer null'. customer is definitely non-null. Why?"

Thinking out loud: 1. "Wait — `customer.getId()` may return null even if customer isn't. `%s` on null prints 'null'." 2. "More serious: if there's an exception evaluating `customer.getId`, `String.format` throws — but that's not the case here." 3. "Real fix is logging: never pass a string with side effects; use parameterized logging (`log.info(\"... {} ...\", a, b);`)."

The actual bug: The customer's id is null. `%s` happily prints "null". Logger constructs the string even if log level filters it out.

The fix (1-3 options if applicable): - **Option A:** Use parameterized logging: `log.info("Processing {} orders for customer {}", orders.size(), customer.getId());`. SLF4J defers formatting. - **Option B:** Defensive null: `customer.getId() == null ? "<unknown>" : customer.getId()` — masks the underlying data issue. - **Option C:** Trace why `customer.getId()` returns null upstream — the log line is correct; the data is broken.

What NOT to say: - "String.format throws on null" — `%s` doesn't; other specifiers like `%d` might. - "Use println for logging" — never; you lose levels, structure, async.

Common follow-up: "What's the perf impact of `String.format` vs parameterized SLF4J?"

Scenario 81 · The InputStream that loops forever

Asked at: Walmart Labs · **Cred:** · **Difficulty:** Medium · **Topic:** InputStream read loop

The code:

```
int b;
while ((b = in.read()) != 0) {
    out.write(b);
}
```

The interviewer's prompt: "Copy never finishes. Pegs CPU. Input contains a 0 byte in the middle. Why does it not terminate?"

Thinking out loud: 1. "`InputStream.read()` returns -1 at EOF, not 0. 0 is a legitimate byte value." 2. "Loop terminates only when read returns 0 — never, because 0 means 'byte value zero'." 3. "At EOF, read returns -1, loop body keeps running with -1 as byte, runaway."

The actual bug: Wrong terminator. EOF is `-1`, not `0`. The loop never breaks.

The fix (1-3 options if applicable): - **Option A:** `while ((b = in.read()) != -1) out.write(b);` — standard form. - **Option B:** Buffered copy with byte array: `byte[] buf = new byte[8192]; int n; while ((n = in.read(buf)) != -1) out.write(buf, 0, n);` — much faster. - **Option C:** Java 9+: `in.transferTo(out);` — single call, JVM-optimized.

What NOT to say: - "Use 0 to detect end of file" — wrong; -1 is the convention. - "Add a try-catch on read" — masks the loop bug.

Common follow-up: "What's the difference between `read()` and `readAllBytes()`?"

Scenario 82 · The shared mutable record component

Asked at: Razorpay · Cred · **Difficulty:** Medium · **Topic:** Record immutability surface

The code:

```
record Profile(String name, Date dob) {}

Date d = new Date();
Profile p = new Profile("Asha", d);
d.setTime(0); // mutate the original Date
// p.dob() now also reflects the mutation
```

The interviewer's prompt: "Records are supposed to be immutable. Why does mutating the original date change the record's view?"

Thinking out loud: 1. "Records have shallow immutability. The reference is final; the referenced object may still be mutable." 2. "Date is mutable. Both `d` and `p.dob()` point to the same object." 3. "Defensive copy in compact constructor: `public Profile { dob = new Date(dob.getTime()); }`. Or migrate to `LocalDate` (immutable)."

The actual bug: Records hold reference fields by reference. A mutable component is mutable externally.

The fix (1-3 options if applicable): - **Option A:** Use immutable types — `LocalDate` instead of `Date`, `List.copyOf` for lists. Best long-term. - **Option B:** Compact constructor defensive copy + accessor copy: `public Profile { dob = new Date(dob.getTime()); } public Date dob() { return new Date(dob.getTime()); }`. Both directions. - **Option C:** Make the wrapping API surface that takes `Date` refuse `new Date()` shared instances — practically impossible.

What NOT to say: - "Records deep-copy" — they do not. - "Mark the field volatile" — irrelevant.

Common follow-up: "Why is `Date` still in the JDK if it's mutable and unsafe?"

Scenario 83 · The exception chain that loses cause

Asked at: Wipro · Infosys · **Difficulty:** Easy · **Topic:** Exception chaining

The code:

```
try {
    repo.fetch();
} catch (SQLException e) {
    throw new DataAccessException("fetch failed");
}
```

The interviewer's prompt: "Logs show `DataAccessException` with our message but no SQL cause. Hard to debug. What's missing?"

Thinking out loud: 1. "Constructor takes a message but no cause. The original `SQLException` is dropped." 2. "Use the (message, Throwable) constructor: `new DataAccessException(\"fetch failed\", e);`" 3. "Or `throw new DataAccessException(\"...\").initCause(e);`. Less idiomatic."

The actual bug: Original exception is not chained. Stack trace of the cause is lost.

The fix (1-3 options if applicable): - **Option A:** Pass the cause: `throw new DataAccessException("fetch failed", e);`. Standard exception-chaining ctor. - **Option B:** If `DataAccessException` doesn't have such a ctor, use `initCause`: `throw (DataAccessException) new DataAccessException("...").initCause(e);`. Ugly; usually a sign to add the ctor. - **Option C:** Use Spring's existing `DataAccessException` hierarchy — already chains JDBC.

What NOT to say: - "Log the original then throw" — separates info in time; harder to correlate. - "Just throw the `SQLException` as-is" — leaks JDBC types from the data layer.

Common follow-up: "What's the difference between `getCause` and `getStackTrace`?"

Scenario 84 · The `CountDownLatch` reused

Asked at: Cred · JP Morgan · **Difficulty:** Medium · **Topic:** `CountDownLatch` reuse

The code:

```
CountDownLatch latch = new CountDownLatch(workers);

for (int i = 0; i < runs; i++) {
    submitWork(latch);
    latch.await();
}
```

The interviewer's prompt: "First batch works. Second iteration's await returns immediately, even though workers haven't run. Why?"

Thinking out loud: 1. "CountDownLatch is one-shot. Once count reaches 0, it stays 0." 2. "Second await returns immediately because count is still 0 from the first run." 3. "For reusable barriers use CyclicBarrier or Phaser."

The actual bug: CountDownLatch is single-use. After reaching zero, it never re-arms.

The fix (1-3 options if applicable): - **Option A:** Create a new latch each iteration: `latch = new CountDownLatch(workers);` inside the loop, pass to submitWork. - **Option B:** Use `CyclicBarrier(workers)` — auto-resets after each cycle. Same workers, multiple barriers. - **Option C:** Use `Phaser` for more flexible barrier behaviour with dynamic party count.

What NOT to say: - "Call latch.reset()" — CountDownLatch has no reset. - "Increase the initial count to handle all runs" — defeats the per-batch synchronization.

Common follow-up: "What's the difference between CountDownLatch and CyclicBarrier?"

Scenario 85 · The deep recursion that StackOverflows

Asked at: TCS · Wipro · **Difficulty:** Easy · **Topic:** Tail recursion

The code:

```
public BigInteger factorial(int n) {
    if (n == 0) return BigInteger.ONE;
    return BigInteger.valueOf(n).multiply(factorial(n - 1));
}

factorial(100_000); // StackOverflowError
```

The interviewer's prompt: "Factorial works for small inputs. At 100k we get StackOverflowError. Java doesn't tail-recurse?"

Thinking out loud: 1. "JVM doesn't optimize tail calls. Each recursive call adds a stack frame." 2. "Default stack is ~512 KB to 1 MB; 100k frames overflow." 3. "Rewrite iteratively, or use a loop with a stack-based accumulator."

The actual bug: JVM doesn't do tail-call optimization. Deep recursion overflows the call stack.

The fix (1-3 options if applicable): - **Option A:** Iterative: `BigInteger r = BigInteger.ONE; for (int i = 2; i <= n; i++) r = r.multiply(BigInteger.valueOf(i)); return r;` . - **Option B:** Increase stack size: `-Xss4m` . Postpones, doesn't solve. - **Option C:** For very large n, use a divide-and-conquer multiplication tree to reduce depth and improve multiply performance.

What NOT to say: - "Java has TCO since Java 8" — it does not. - "Use Stream.range and reduce" — works at small n; reduce isn't tail-call optimized either, but it's iterative under the hood.

Common follow-up: "Why doesn't the JVM optimize tail calls?"

Scenario 86 · The volatile array

Asked at: Cred · JP Morgan · **Difficulty:** Hard · **Topic:** volatile + array elements

The code:

```
private volatile int[] buffer = new int[1024];

public void update(int idx, int val) {
    buffer[idx] = val;
}
```

The interviewer's prompt: "buffer is volatile, so writes are visible across threads. Other threads don't see updates. Why?"

Thinking out loud: 1. " `volatile` on a reference field guarantees that reads of the reference see the latest assigned reference." 2. "It does NOT make writes to ARRAY ELEMENTS volatile. Reads of `buffer[idx]` are not necessarily fresh." 3. "Use `AtomicIntegerArray`, or `VarHandle` for per-element acquire/release semantics."

The actual bug: `volatile` on an array reference doesn't make element accesses volatile. Visibility of individual writes is not guaranteed.

The fix (1-3 options if applicable): - **Option A:** `AtomicIntegerArray` — per-element atomicity and visibility. - **Option B:** `VarHandle` (Java 9+): `VarHandle vh = MethodHandles.arrayElementVarHandle(int[].class); vh.setVolatile(buffer, idx, val);` . Fine-grained. - **Option C:** Replace the entire array each update (copy-on-write) — the reference reassignment is volatile-visible. Slow for large arrays.

What NOT to say: - "Mark each element volatile" — Java doesn't allow `volatile` on array elements. - "Use `synchronized` on the array" — locks on the reference; doesn't propagate to elements.

Common follow-up: "What does volatile guarantee for object reference fields vs array contents?"

Scenario 87 · The Boolean compared with == on autoboxing

Asked at: TCS · Cognizant · **Difficulty:** Easy · **Topic:** Boolean caching

The code:

```
Boolean a = new Boolean("true");
Boolean b = Boolean.TRUE;
if (a == b) {
    enable();
}
```

The interviewer's prompt: " `new Boolean(\"true\")` and `Boolean.TRUE` — why is reference comparison false here?"

Thinking out loud: 1. " `new Boolean(...)` always creates a fresh object." 2. " `Boolean.TRUE` is the cached singleton." 3. "Different objects, `==` is false. Use equals or `.booleanValue()`."

The actual bug: `new Boolean(...)` bypasses the cache. Reference comparison with `Boolean.TRUE` fails.

The fix (1-3 options if applicable): - **Option A:** `Boolean.TRUE.equals(a)` — null-safe and value-based. - **Option B:** Avoid `new Boolean(...)` entirely (deprecated for removal in Java 16+); use `Boolean.valueOf(s)` which returns the cached instance. - **Option C:** Unbox to primitive: `if (a)` — auto-unboxes, throws NPE on null but compares values.

What NOT to say: - "Use Boolean.TRUE everywhere" — fine but doesn't fix `new Boolean(...)`. - "Add `intern()`" — doesn't exist on Boolean.

Common follow-up: "Why was `new Boolean(String)` deprecated in Java 9?"

Scenario 88 · The HashMap loaded under load

Asked at: Walmart Labs · Cred · **Difficulty:** Hard · **Topic:** HashMap resize under contention

The code:

```
class GeoRegistry {
    private HashMap<String, Region> cache = new HashMap<>();

    public void init() {
        // multiple threads pre-populate from different sources
        IntStream.range(0, 10).parallel().forEach(i -> loadShard(cache, i));
    }
}
```

The interviewer's prompt: "During startup we pre-populate from parallel sources. Cache ends up with fewer entries than the sources, or sometimes a thread spins forever in put. Why?"

Thinking out loud: 1. "HashMap is not thread-safe. Parallel puts race on resize and bucket chains." 2. "Lost entries (race) and infinite loop (corrupted internal structure) are both possible." 3. "Use ConcurrentHashMap or pre-size + load sequentially then publish."

The actual bug: Parallel writes to HashMap. Races corrupt the internal structure.

The fix (1-3 options if applicable): - **Option A:** Use ConcurrentHashMap for concurrent inserts. - **Option B:** Load each shard into a local HashMap, then merge sequentially into the registry. - **Option C:** Pre-size and load in a single thread. Publish via volatile reference.

What NOT to say: - "It works most of the time" — race conditions; eventually corrupts. - "Synchronize on the map" — works but loses parallelism.

Common follow-up: "How does ConcurrentHashMap's resize differ from HashMap's?"

Scenario 89 · The Class.forName that doesn't run static init

Asked at: JP Morgan · **Cred:** · **Difficulty:** Hard · **Topic:** Class loading

The code:

```
Class.forName("com.example.DriverImpl", false, loader);
// expected static block to register the driver
```

The interviewer's prompt: "We used to do `Class.forName(\"com.mysql.cj.jdbc.Driver\")` to register the driver. The new form with the 3-arg overload doesn't register. Why?"

Thinking out loud: 1. "3-arg `Class.forName(name, initialize, loader)` — second arg controls whether static initializers run." 2. "Passing `false` skips initialization. The static block that calls `DriverManager.registerDriver` never runs." 3. "Single-arg `Class.forName(name)` defaults to `initialize=true`."

The actual bug: The 3-arg form with `initialize=false` skips static initializers. The driver registration code never runs.

The fix (1-3 options if applicable): - **Option A:** `Class.forName(name, true, loader)` — initialize explicitly. - **Option B:** Use the 1-arg form: `Class.forName(name)` — defaults to `initialize=true`. - **Option C:** Modern JDBC drivers (4.0+) self-register via the ServiceLoader mechanism. No `Class.forName` needed.

What NOT to say: - "Static blocks always run on class load" — not when `initialize=false`. - "Use `Class.getDeclaredConstructor.newInstance`" — separate issue; doesn't run static init by itself either.

Common follow-up: "What's the difference between class loading and class initialization?"

Scenario 90 · The Stream.distinct that doesn't dedupe

Asked at: TCS · Razorpay · **Difficulty:** Easy · **Topic:** distinct uses equals

The code:

```
class Person { String name; int age;
    Person(String n, int a) { name = n; age = a; }
}

List<Person> deduped = people.stream().distinct().toList();
// duplicates persist
```

The interviewer's prompt: "distinct doesn't remove visibly identical entries. Why?"

Thinking out loud: 1. "distinct uses equals + hashCode. Person doesn't override either — defaults to identity." 2. "Each `new Person(...)` is unique by identity, so distinct sees no duplicates." 3. "Either override equals/hashCode, use a record, or distinct by a field."

The actual bug: `distinct()` relies on equals/hashCode. Person uses Object identity — no two instances are equal.

The fix (1-3 options if applicable): - **Option A:** Make Person a record — auto equals/hashCode by all components. - **Option B:** Override equals/hashCode on Person manually. - **Option C:** Distinct by a key: `people.stream().collect(toMap(Person::name, p -> p, (a, b) -> a)).values()` — last-wins per name.

What NOT to say: - "distinct uses reference equality" — it does not. - "Use `distinct().sorted()`" — sort doesn't help dedupe; same issue.

Common follow-up: "How would you `distinctBy(Person::name)` since the Stream API doesn't have one?"

Scenario 91 · The List.copyOf surprise

Asked at: Cred · Razorpay · **Difficulty:** Medium · **Topic:** Immutable list null elements

The code:

```
List<String> with = Arrays.asList("a", null, "c");  
List<String> copy = List.copyOf(with); // NullPointerException
```

The interviewer's prompt: "List.copyOf throws NPE on a list that contains null. Why does it care?"

Thinking out loud: 1. "List.of and List.copyOf (Java 9+) reject null elements by spec." 2. "Justification: clearer semantics and small footprint of the underlying immutable list." 3. "For lists that may contain nulls, use Collections.unmodifiableList(new ArrayList<>(with))."

The actual bug: List.copyOf does not permit null elements.

The fix (1-3 options if applicable): - **Option A:** Collections.unmodifiableList(new ArrayList<>(with)) — allows nulls, immutable view. - **Option B:** Filter nulls first if they're not meaningful: with.stream().filter(Objects::nonNull).toList() . - **Option C:** Replace nulls with a sentinel (" " or a special value) — only if domain allows.

What NOT to say: - "List.copyOf silently drops nulls" — it does not; it throws. - "Add a try-catch wrapper" — doesn't help; throws unconditionally on null.

Common follow-up: "Why do List.of and Map.of reject nulls?"

Scenario 92 · The Iterator from removelf during forEach

Asked at: TCS · Walmart Labs · **Difficulty:** Medium · **Topic:** ConcurrentModificationException in forEach

The code:

```
List<Order> orders = new ArrayList<>(fetchOrders());  
orders.forEach(o -> {  
    if (o.isCancelled()) orders.remove(o);  
});
```

The interviewer's prompt: "Same fail-fast CME bug, this time in forEach. We thought forEach was safe."

Thinking out loud: 1. "List.forEach uses the iterator under the hood. Mutating the list mid-iteration triggers CME on the next read." 2. "removelf is the right primitive for this." 3. "Same as the enhanced-for case; forEach changes syntax, not semantics."

The actual bug: `forEach` is not magic — it iterates via `Iterator`. Mutating the list inside `forEach` throws `CME`.

The fix (1-3 options if applicable): - **Option A:** `orders.removeIf(Order::isCancelled);` — one-liner. - **Option B:** Iterate a copy: `new ArrayList<>(orders).forEach(o -> { if (o.isCancelled()) orders.remove(o); });` . Allocates. - **Option C:** Build a removal set in `forEach`, then bulk remove: `Set<Order> toRemove = new HashSet<>(); orders.forEach(o -> { if (o.isCancelled()) toRemove.add(o); }); orders.removeAll(toRemove);` .

What NOT to say: - "forEach is exception-safe" — about exceptions in the body, not modification. - "Wrap in a synchronized block" — doesn't change fail-fast behaviour in a single thread.

Common follow-up: "What's the difference between `Iterable.forEach` and `Stream.forEach`?"

Scenario 93 · The `URLConnection` that leaks

Asked at: Walmart Labs · JP Morgan · **Difficulty:** Medium · **Topic:** `URLConnection` resource leak

The code:

```
URL u = new URL("https://gateway/charge");
URLConnection c = (URLConnection) u.openConnection();
c.setDoOutput(true);
c.getOutputStream().write(payload);
int code = c.getResponseCode();
```

The interviewer's prompt: "After a few hundred calls we hit 'too many open files'. Each call closes nothing explicitly. Why does this leak?"

Thinking out loud: 1. "`URLConnection` holds an underlying socket. Even if you only read code, the input stream must be consumed and closed." 2. "Without consuming the body, the JVM may keep the connection in the keep-alive cache holding the FD." 3. "Best practice: `getInputStream()` then close, `getErrorStream()` on failure then close, or migrate to `java.net.http.HttpClient` (Java 11+)."

The actual bug: Input/error streams not consumed and closed. Connections (and sockets) remain pinned.

The fix (1-3 options if applicable): - **Option A:** Always read+close the input/error stream. Wrap in `try-finally`. - **Option B:** Move to Java 11+ `HttpClient` — handles connection lifecycle internally and has a cleaner API. - **Option C:** Use Apache `HttpClient` or `OkHttp` with `try-with-resources`; well-tested for connection management.

What NOT to say: - "GC will close them" — not reliable for sockets. - "Increase the ulimit" — postpones; eventually leaks.

Common follow-up: "Why is `java.net.http.HttpClient` preferred over `HttpURLConnection` in new code?"

Scenario 94 · The hashCode that hits zero on every record

Asked at: Cred · JP Morgan · **Difficulty:** Hard · **Topic:** Custom hashCode quality

The code:

```
class Point {
    int x, y;
    @Override public int hashCode() { return x ^ y; }
    @Override public boolean equals(Object o) {
        return o instanceof Point p && p.x == x && p.y == y;
    }
}

// many points with (1,1), (2,2), (3,3) – all hash to 0
```

The interviewer's prompt: "HashMap of Point gets pathologically slow on certain inputs. Our hashCode looks fine — XOR of two ints. What's wrong?"

Thinking out loud: 1. "XOR collapses (a, b) and (b, a) to the same hash. (n, n) always hashes to 0." 2. "For inputs on the diagonal, every Point hashes to 0 — single bucket, O(n) lookups." 3. "Use Objects.hash(x, y) which combines via a prime multiplier."

The actual bug: XOR is a poor hash combiner. Patterns in the input collide heavily.

The fix (1-3 options if applicable): - **Option A:** `Objects.hash(x, y)` — uses $31 * a + b$ style; good distribution. - **Option B:** Convert to record `record Point(int x, int y) {}` — auto-generated hashCode is high-quality. - **Option C:** Manual: `int h = 31 * Integer.hashCode(x) + Integer.hashCode(y);` — same idea without the boxing.

What NOT to say: - "Add a random salt" — breaks the contract; equal objects must have equal hash. - "Use Long.hashCode((long)x << 32 | y)" — works but boxing-heavy.

Common follow-up: "What's HashMap's mitigation for poor hash functions in Java 8+?"

Scenario 95 · The synchronized on Integer

Asked at: TCS · Cognizant · **Difficulty:** Hard · **Topic:** Synchronizing on boxed types

The code:

```
public void inc(Integer lockKey, Order o) {
    synchronized (lockKey) {
        process(o);
    }
}
```

The interviewer's prompt: "Different lockKey values block each other. We see contention where there shouldn't be any. Why?"

Thinking out loud: 1. "Integer instances in -128..127 are cached. Two calls with lockKey=42 from different callers share the same Integer instance." 2. "Worse: any unrelated code that happens to synchronize on `Integer.valueOf(42)` enters mutual exclusion with you." 3. "Never synchronize on cached/interned objects."

The actual bug: Integer caching means callers from different code paths can accidentally lock on the same instance.

The fix (1-3 options if applicable): - **Option A:** Use a `ConcurrentHashMap<Integer, ReentrantLock>` and `computeIfAbsent` to provide a per-key lock you own. - **Option B:** Use Striped lock pattern (Guava's `Striped.lock(stripes)`). - **Option C:** Lock on a dedicated wrapper object: `class LockKey { final int v; LockKey(int v){this.v=v;} }` — your own type, no caching.

What NOT to say: - "Integer cache only affects == " — affects synchronized too; same object identity. - "Use String for lockKey" — strings can also be interned; same hazard.

Common follow-up: "What other JDK types should you never synchronize on?"

Scenario 96 · The mocked clock that never advances

Asked at: Cred · Razorpay · **Difficulty:** Medium · **Topic:** Time-coupled tests

The code:

```
class OtpService {
    public Otp generate() {
        return new Otp(randomDigits(), Instant.now().plus(Duration.ofMinutes(5)));
    }
}
```

The interviewer's prompt: "OTP expiry tests pass locally, fail in CI. We can't predict the time. How would you fix the design?"

Thinking out loud: 1. "Tight coupling to `Instant.now()`. Tests can't control time." 2. "Inject a `Clock` and use `Instant.now(clock)`. Tests pass a fixed `Clock`; prod gets system clock." 3. "Standard `java.time` pattern; underused."

The actual bug: Direct use of `Instant.now()` couples logic to system time. Untestable; flaky in CI.

The fix (1-3 options if applicable): - **Option A:** Inject `Clock`: `class OtpService { private final Clock clock; ... new Otp(..., Instant.now(clock).plus(...)); }`. Tests pass `Clock.fixed(...)`. - **Option B:** Wrap in a `TimeProvider` interface. More indirection; usually unnecessary when `Clock` exists. - **Option C:** Test with a sleep tolerance — flaky; avoid.

What NOT to say: - "Use `PowerMock` to mock static" — bytecode hackery; smell. - "Compare with `assertEquals(otp.expiresAt(), Instant.now().plus(...))`" — exact time match impossible.

Common follow-up: "How do you test code that uses `LocalDate.now()`?"

Scenario 97 · The Boolean parsed leniently

Asked at: TCS · Wipro · **Difficulty:** Easy · **Topic:** `Boolean.parseBoolean` leniency

The code:

```
String s = "yes";
boolean enabled = Boolean.parseBoolean(s);
log.info("enabled={}", enabled);
```

The interviewer's prompt: "Config has 'yes' for an enable flag. `parseBoolean` returns false. Why no error?"

Thinking out loud: 1. "`Boolean.parseBoolean(s)` returns true only if s equals `\"true\"` (case-insensitive). Everything else — including 'yes', 'on', '1' — returns false." 2. "No error; silent false." 3. "Need explicit normalization or a stricter parser."

The actual bug: `parseBoolean` is permissive — any non-true returns false silently. 'yes' is silently treated as false.

The fix (1-3 options if applicable): - **Option A:** Normalize at parse: `Set<String> trues = Set.of("true", "yes", "on", "1"); boolean enabled = trues.contains(s.toLowerCase());`. - **Option B:** Use a config framework (Spring `@Value`, `Typesafe Config`) that handles common boolean forms. - **Option C:** Strict parse: throw on anything but "true"/"false". Catches config typos.

What NOT to say: - "parseBoolean throws on invalid" — it doesn't. - "Use `Boolean.valueOf`" — same behaviour.

Common follow-up: "What does YAML's parser do with 'yes', 'no', 'on', 'off'?"

Scenario 98 · The ZonedDateTime that crosses DST

Asked at: JP Morgan · Walmart Labs · **Difficulty:** Hard · **Topic:** DST + addition

The code:

```
ZonedDateTime start = ZonedDateTime.of(2026, 3, 13, 1, 0, 0, 0, ZoneId.of("America/New_York"));
ZonedDateTime end = start.plus(Duration.ofHours(2));
// expected 3 AM; actual 4 AM
```

The interviewer's prompt: "On the DST-transition night, adding 2 hours lands on 4 AM instead of 3 AM. Why?"

Thinking out loud: 1. "DST jumps 2 AM → 3 AM. There is no 2 AM in local time on that day." 2. "`Duration.ofHours(2)` is wall-clock duration in physical hours. 1 AM + 2 physical hours = 4 AM local (because we skipped an hour)." 3. "For 'add 2 wall-clock hours' use `Period` semantics or `plusHours(2)` on the local part; `ZonedDateTime` exposes both."

The actual bug: `Duration` is physical seconds. Across a DST transition, two physical hours later isn't always two wall-clock hours later.

The fix (1-3 options if applicable): - **Option A:** Use `start.plusHours(2)` — operates on the local time, preserving wall-clock arithmetic. - **Option B:** Decide which semantic you want — physical elapsed (`Duration`) vs wall-clock (`plusHours`). Document. - **Option C:** For scheduling, work in UTC and convert at the boundary.

What NOT to say: - "DST doesn't affect millis" — it does affect local time mapping. - "Use `OffsetDateTime` to skip the rule" — fixes by removing the zone; may not match domain intent.

Common follow-up: "Why does 2 AM not exist on a DST spring-forward night?"

Scenario 99 · The ImmutableMap that mutates

Asked at: Cred · TCS · **Difficulty:** Medium · **Topic:** Unmodifiable vs immutable

The code:

```
Map<String, Integer> backing = new HashMap<>();
backing.put("a", 1);

Map<String, Integer> view = Collections.unmodifiableMap(backing);

backing.put("b", 2);
System.out.println(view.size()); // 2
```

The interviewer's prompt: "We wrap a map in unmodifiableMap and pass it to clients. We then mutate the backing. Clients see the new entries. Did we just leak our internal map?"

Thinking out loud: 1. " `unmodifiableMap` is a VIEW. Mutations on the backing are visible through it." 2. "For true immutability, copy first: `Map.copyOf(backing)` (Java 10+) or `Map.copyOf(new HashMap<>(backing))` ." 3. "Common mental model mistake. Unmodifiable = clients can't modify via the wrapper. Backing changes still leak."

The actual bug: `unmodifiableMap` is a wrapper that prevents writes through the view but reflects backing changes.

The fix (1-3 options if applicable): - **Option A:** `Map.copyOf(backing)` — Java 10+ snapshot, truly immutable. - **Option B:** `Collections.unmodifiableMap(new HashMap<>(backing))` — copy then wrap. Pre-Java-10 style. - **Option C:** Use Guava's `ImmutableMap.copyOf(backing)` — also a snapshot.

What NOT to say: - "unmodifiableMap is the same as immutable" — no, it's a view. - "Just don't mutate the backing" — relies on convention; copying is safer.

Common follow-up: "What's the heap cost of copyOf vs unmodifiable wrapper?"

Scenario 100 · The catch InterruptedException without restoring

Asked at: Cred · JP Morgan · **Difficulty:** Hard · **Topic:** Interruption protocol

The code:

```
public void waitForFlag() {
    try {
        latch.await();
    } catch (InterruptedException e) {
        log.warn("interrupted");
    }
}
```

The interviewer's prompt: "We catch InterruptedException and log. Higher-level shutdown logic doesn't recognize the interruption. Why?"

Thinking out loud: 1. "When the JVM throws `InterruptedException`, it clears the thread's interrupted flag." 2. "If we just log and continue, callers up the stack can't detect the interruption." 3. "Either restore the flag (`Thread.currentThread().interrupt()`) or rethrow."

The actual bug: `InterruptedException` was caught and swallowed. The interruption signal is lost.

The fix (1-3 options if applicable): - **Option A:** Restore the flag: `catch (InterruptedException e) { Thread.currentThread().interrupt(); log.warn("interrupted"); return; } .` -

Option B: Rethrow wrapped: `throw new RuntimeException(e);` after restoring — useful in lambdas where the checked exception is awkward. - **Option C:** Don't catch if you can declare the throws — push the decision up.

What NOT to say: - "Interruption is a request, ignore it" — breaks shutdown protocols. - "Sleep then continue" — masks the signal.

Common follow-up: "What's the interruption protocol for `ExecutorService` shutdown?"

Scenario 101 · The Scanner that doesn't close

Asked at: TCS · Wipro · **Difficulty:** Easy · **Topic:** Resource leak / Scanner

The code:

```
public List<String> readLines(Path p) throws IOException {
    Scanner sc = new Scanner(p);
    List<String> lines = new ArrayList<>();
    while (sc.hasNext()) lines.add(sc.nextLine());
    return lines;
}
```

The interviewer's prompt: "Reads work. After many calls we get 'too many open files'. We always finish the loop. Why leak?"

Thinking out loud: 1. "Scanner is `Closeable` but we never close. The underlying file descriptor leaks." 2. "Wrap in `try-with-resources`." 3. "Or use `Files.readAllLines(p)` — handles open/close for you."

The actual bug: Scanner is never closed; FDs leak.

The fix (1-3 options if applicable): - **Option A:** `try (Scanner sc = new Scanner(p)) { ... }` — auto-closes. - **Option B:** `return Files.readAllLines(p);` — JDK helper, handles all cleanup. - **Option C:** `try (Stream<String> s = Files.lines(p)) { return s.toList(); }` — streaming, also closes.

What NOT to say: - "GC will close it eventually" — FDs aren't memory; not guaranteed. - "Close in a finalizer" — finalizers are deprecated and unreliable.

Common follow-up: "What's the difference between `Files.lines` and `Files.readAllLines` for big files?"

Scenario 102 · The lambda capture of mutable state

Asked at: Cred · Razorpay · **Difficulty:** Medium · **Topic:** Effectively final + capture

The code:

```
int counter = 0;
List<Runnable> tasks = new ArrayList<>();
for (Order o : orders) {
    tasks.add(() -> System.out.println(counter + ": " + o));
    counter++;
}
```

The interviewer's prompt: "This won't compile: 'local variable counter defined in an enclosing scope must be final or effectively final'. We need a counter."

Thinking out loud: 1. "Lambdas can only capture effectively final locals. `counter++` makes it non-final." 2. "Either use a 1-element array (`int[] counter = {0};`) — ugly but works." 3. "Better: use stream + index, or refactor to a stateful approach."

The actual bug: Lambdas can't capture mutating locals. The compiler rejects.

The fix (1-3 options if applicable): - **Option A:**

`IntStream.range(0, orders.size()).forEach(i -> tasks.add(() -> System.out.println(i + ": " + orders.get(i))));` — stream-based indexing. - **Option B:**

Hack: `int[] counter = {0};` and `counter[0]++` — captured reference, mutable contents. -

Option C: Use `AtomicInteger counter = new AtomicInteger();` and `counter.getAndIncrement()` — also thread-safe.

What NOT to say: - "Mark counter `final`" — then `counter++` won't compile. - "Move counter to instance field" — works but changes scoping.

Common follow-up: "Why does Java enforce effectively-final on captured locals?"

Scenario 103 · The `String.matches` that re-compiles every call

Asked at: Walmart Labs · Cred · **Difficulty:** Medium · **Topic:** Regex compile cost

The code:


```
public boolean isPanFormat(String s) {
    return s.matches("[A-Z]{5}[0-9]{4}[A-Z]");
}
// called millions of times
```

The interviewer's prompt: "PAN validation is in a hot loop. Profiler shows 70% time in regex compile. What changed when we moved validation here?"

Thinking out loud: 1. "`String.matches(regex)` compiles the pattern on every call." 2. "For a hot path, hoist the Pattern to a constant: `private static final Pattern PAN = Pattern.compile(\"...\\");` then `PAN.matcher(s).matches()` ." 3. "Same compile cost on `Pattern.matches` static method — re-compiles each call."

The actual bug: `String.matches` re-compiles the regex on every invocation. Expensive in a hot loop.

The fix (1-3 options if applicable): - **Option A:** `private static final Pattern PAN = Pattern.compile(\"[A-Z]{5}[0-9]{4}[A-Z]\"); ... PAN.matcher(s).matches();` .
Cached. - **Option B:** For simple checks, write a manual char-by-char validator. Fastest. - **Option C:** Use Apache Commons or a library validator that caches internally.

What NOT to say: - "Regex is always slow" — once compiled, it's reasonable. - "Use String.contains" — different semantics.

Common follow-up: "How does Pattern's internal NFA differ from a hand-written validator?"

Scenario 104 · The Optional inside Optional

Asked at: TCS · Cognizant · **Difficulty:** Medium · **Topic:** Optional.map vs flatMap

The code:

```
Optional<Customer> c = repo.find(id);
Optional<Optional<Address>> a = c.map(Customer::getAddress);
```

The interviewer's prompt: "getAddress returns Optional

. After map we get Optional<Optional>. Awkward. What to use?"

Thinking out loud: 1. "`map` wraps the function's return in another Optional. If the function already returns Optional, you get nesting." 2. "`flatMap` is the unwrap-then-wrap version. Returns Optional ." 3. "Same as Stream.map vs flatMap — same pattern."

The actual bug: `map` on a function that returns Optional produces nested Optionals. Use `flatMap` .

The fix (1-3 options if applicable): - **Option A:** `c.flatMap(Customer::getAddress)` — flattens to `Optional<Address>`. - **Option B:** Change `getAddress` to return `Address` (or null) and use `map + ofNullable`. Less idiomatic. - **Option C:** Avoid the chain — `if (c.isPresent()) c.get().getAddress()...`. Verbose but explicit.

What NOT to say: - "Use `get().get()`" — both throw `NoSuchElementException` on empty; bad style. - "Map twice" — won't unnest.

Common follow-up: "What's the difference between `flatMap` on `Optional` and `flatMap` on `Stream`?"

Scenario 105 · The Java 7 switch on null

Asked at: TCS · Infosys · **Difficulty:** Easy · **Topic:** switch on null

The code:

```
String s = null;
switch (s) {
    case "A": handleA(); break;
    case "B": handleB(); break;
    default: handleDefault();
}
```

The interviewer's prompt: "Switch on null. NPE on the switch line. Why doesn't default catch it?"

Thinking out loud: 1. "Pre-Java-21, switch on null throws NPE before matching anything." 2. "default doesn't match null; it matches values that don't match any case label." 3. "Java 21 pattern matching for switch supports `case null` explicitly."

The actual bug: Switch on String/enum throws NPE on null receiver in pre-Java-21.

The fix (1-3 options if applicable): - **Option A:** Null-check before switch: `if (s == null) { handleDefault(); return; }`. - **Option B:** Java 21+: `switch (s) { case "A" -> ...; case null -> handleNull(); default -> ...; }`. Explicit null case. - **Option C:** Use `Optional.ofNullable(s).ifPresentOrElse(...)` for a different control flow.

What NOT to say: - "Catch NPE on the switch" — works but ugly. - "default handles null" — it does not.

Common follow-up: "How does Java 21's switch pattern matching handle null?"

Scenario 106 · The LocalDate equality vs ChronoLocalDate

Asked at: JP Morgan · Walmart Labs · **Difficulty:** Hard · **Topic:** `ChronoLocalDate` equals

The code:

```

LocalDate iso = LocalDate.of(2026, 5, 26);
ChronoLocalDate hijri = HijrahDate.from(iso);
System.out.println(iso.equals(hijri)); // false
System.out.println(iso.isEqual(hijri)); // true

```

The interviewer's prompt: "Same calendar day, equals false, isEqual true. Why two methods?"

Thinking out loud: 1. "`equals` compares both date and chronology. ISO vs Hijri — different chronologies — never equal." 2. "`isEqual` compares the underlying instant. Same calendar day regardless of chronology." 3. "Use `isEqual` for 'is this the same day on the timeline'; `equals` for full identity."

The actual bug: This isn't a bug; it's a feature easy to misuse. Cross-chronology comparison needs `isEqual`.

The fix (1-3 options if applicable): - **Option A:** Decide intent — `equals` for full identity, `isEqual` for instant-level. - **Option B:** Convert to a common chronology before comparing:

`iso.equals(hijri.toEpochDay() == iso.toEpochDay())`. - **Option C:** For most domain code, stay in ISO unless you genuinely need other calendars.

What NOT to say: - "`LocalDate.equals` is broken" — it's by design and documented. - "Use `compareTo == 0`" — works if both are `LocalDate`; cross-chronology it's also by chronology.

Common follow-up: "What other `ChronoLocalDate` implementations exist in the JDK?"

Scenario 107 · The Iterator that yields stale values

Asked at: Cred · JP Morgan · **Difficulty:** Hard · **Topic:** `CopyOnWriteArrayList` iterator semantics

The code:

```

CopyOnWriteArrayList<Listener> listeners = new CopyOnWriteArrayList<>();
listeners.add(a);

Iterator<Listener> it = listeners.iterator();
listeners.add(b);

while (it.hasNext()) it.next().onEvent(); // only fires a, not b

```

The interviewer's prompt: "Adding a listener after starting iteration doesn't include the new listener. Is that a bug?"

Thinking out loud: 1. "CopyOnWriteArrayList's iterator is a snapshot at creation time." 2. "Subsequent writes go to a new copy; the iterator keeps the old reference." 3. "By design — trades freshness for safety. If you need fresh, re-create the iterator after writes."

The actual bug: No bug — by design. CopyOnWriteArrayList iterators are snapshot iterators.

The fix (1-3 options if applicable): - **Option A:** If you want the new listener to fire, re-iterate after add: collect work into a queue and iterate fresh per cycle. - **Option B:** Use a ConcurrentLinkedQueue or similar with weakly-consistent iterators that pick up additions. - **Option C:** Document the snapshot semantics in the API — most listener registries are fine with it.

What NOT to say: - "CopyOnWrite is broken" — by spec. - "Synchronize the iteration" — defeats the purpose; snapshot doesn't need a lock.

Common follow-up: "When would you choose CopyOnWriteArrayList over ConcurrentLinkedQueue?"

Scenario 108 · The static method that throws ExceptionInInitializerError

Asked at: Cred · JP Morgan · **Difficulty:** Medium · **Topic:** Static init failure

The code:

```
public class CountryCodes {
    private static final Map<String, String> CODES;
    static {
        CODES = parseCsv("/country-codes.csv");
    }
}
```

The interviewer's prompt: "In some test profiles we get ExceptionInInitializerError when loading CountryCodes. Restart doesn't help. Why?"

Thinking out loud: 1. "Static initializer threw (likely FileNotFoundException). Wrapped as ExceptionInInitializerError." 2. "Once a class's static init fails, the class is in an error state. All subsequent uses throw NoClassDefFoundError without re-running init." 3. "Fix the underlying parseCsv failure (resource missing/wrong path/wrong classloader)."

The actual bug: Static initializer threw on first access. The class is now permanently failed for this classloader; subsequent accesses NCDFE.

The fix (1-3 options if applicable): - **Option A:** Make CSV loading defensive: try-catch in the static block; degrade gracefully. - **Option B:** Lazy-init with a Supplier: `static volatile Map<String,String> CODES;` and load on first call with error handling. - **Option C:** Move CSV loading out of class init — load from a Spring bean or service at startup; report errors normally.

What NOT to say: - "Reload the class" — JVM doesn't re-init failed classes. - "Catch and log; rest will work" — works only if you guard against the missing map at use sites.

Common follow-up: "What's the difference between `NoClassDefFoundError` and `ClassNotFoundException`?"

Scenario 109 · The boolean expression that always evaluates both sides

Asked at: TCS · Wipro · **Difficulty:** Easy · **Topic:** & vs && short-circuit

The code:

```
if (customer != null & customer.isActive()) {  
    process(customer);  
}
```

The interviewer's prompt: "NPE on `customer.isActive()` when customer is null. We thought the null check guards it. What?"

Thinking out loud: 1. "`&` is bitwise AND. Both operands evaluated. `&&` is short-circuit logical AND." 2. "Single `&` forces evaluation of `customer.isActive()` even when customer is null." 3. "Type mismatch easy to miss visually."

The actual bug: `&` doesn't short-circuit. Right operand evaluated even when left is false.

The fix (1-3 options if applicable): - **Option A:** Use `&&` — short-circuits when the left is false. - **Option B:** Nested if: `if (customer != null) if (customer.isActive()) process(customer);`.

Verbose but unambiguous. - **Option C:** Defensive:

`Optional.ofNullable(customer).filter(Customer::isActive).ifPresent(this::process);`

What NOT to say: - "& and && are the same on booleans" — same result if no exception; different evaluation semantics. - "Add parens to fix precedence" — won't help; not a precedence bug.

Common follow-up: "When would you ever use `&` instead of `&&`?"

Scenario 110 · The Java 8 Collectors.toMap with null values

Asked at: Razorpay · Cred · **Difficulty:** Medium · **Topic:** `Collectors.toMap` NPE on null

The code:

```
Map<Long, String> byId = customers.stream()
    .collect(toMap(Customer::getId, Customer::getEmail));
// NullPointerException
```

The interviewer's prompt: "toMap throws NPE for customers with null email. Why does the value matter?"

Thinking out loud: 1. "Collectors.toMap calls HashMap.merge under the hood. merge requires non-null value." 2. "Allowing null in a map is fine; toMap's implementation specifically disallows it." 3. "Use a manual collector or filter out nulls first."

The actual bug: `Collectors.toMap` disallows null values, despite HashMap permitting them.

The fix (1-3 options if applicable): - **Option A:** Filter out null emails first if they're meaningless:

```
.filter(c -> c.getEmail() != null) . - Option B: Use a manual collector:
.collect(HashMap::new, (m, c) -> m.put(c.getId(), c.getEmail()),
HashMap::putAll); . Permits nulls. - Option C: Provide a sentinel:
Optional.ofNullable(c.getEmail()).orElse("") .
```

What NOT to say: - "HashMap doesn't allow nulls" — it does. - "Wrap each value in Optional" — type changes; awkward downstream.

Common follow-up: "What's the underlying call that toMap makes that fails on null?"

Scenario 111 · The HashMap that returns the wrong type after generics warning

Asked at: TCS · Cognizant · **Difficulty:** Medium · **Topic:** Raw type pollution

The code:

```
Map raw = new HashMap();
raw.put("count", "ten");

Map<String, Integer> typed = raw;
int n = typed.get("count"); // ClassCastException
```

The interviewer's prompt: "Compiler warns about raw types but compiles. Runtime throws ClassCastException on retrieval. Why?"

Thinking out loud: 1. "Raw type lets you put anything. The unchecked assignment moves the bad data into a typed view." 2. "At retrieval, the generated cast to Integer fails — the value is String 'ten'." 3. "Raw types defeat generics. Treat warnings as errors in CI."

The actual bug: Mixing raw and parameterized types poisons the collection.

The fix (1-3 options if applicable): - **Option A:** Use the parameterized type throughout: `Map<String, Integer> raw = new HashMap<>(); raw.put("count", 10);` . Compile-time enforcement. - **Option B:** Compile with `-Xlint:unchecked -Werror` to make these errors. - **Option C:** If you really must mix, use a defensive cast: `Integer n = (Integer) raw.get("count");` — at least the cast is in your face.

What NOT to say: - "Raw types are convenient for tests" — they're a footgun anywhere. - "Add `@SuppressWarnings`" — silences the warning, the bug remains.

Common follow-up: "What's heap pollution?"

Scenario 112 · The JPA entity with eager fetch

Asked at: JP Morgan · Walmart Labs · **Difficulty:** Hard · **Topic:** Hibernate N+1 / fetch type

The code:

```
@Entity
class Order {
    @Id Long id;
    @OneToMany(fetch = FetchType.EAGER)
    List<LineItem> items;
}

List<Order> orders = repo.findAll();
```

The interviewer's prompt: "findAll on 1000 orders runs 1001 SQL queries. We expected 1. Why?"

Thinking out loud: 1. "EAGER fetch on @OneToMany triggers a separate select per parent." 2. "Classic N+1: 1 query for parents, N queries for children." 3. "Use LAZY by default; use JOIN FETCH or @EntityGraph when you need eager loading per query."

The actual bug: EAGER fetch on a OneToMany creates the N+1 problem.

The fix (1-3 options if applicable): - **Option A:** Mark as LAZY (default for collections): `@OneToMany(fetch = FetchType.LAZY)` . Load on demand. - **Option B:** Use a JPQL JOIN FETCH for the specific query that needs items: `SELECT o FROM Order o JOIN FETCH o.items` . - **Option C:** `@EntityGraph` named entity graphs for declarative fetch overrides.

What NOT to say: - "EAGER is the safe default" — it's the slow default. - "Increase the DB connection pool" — doesn't help; problem is round-trip count.

Common follow-up: "What's the difference between JOIN FETCH and a JPQL JOIN?"

Scenario 113 · The new Thread that won't finish

Asked at: Cred · JP Morgan · **Difficulty:** Medium · **Topic:** Daemon vs non-daemon thread

The code:

```
public static void main(String[] args) {
    Thread t = new Thread(() -> { while (true) heartbeat(); });
    t.setDaemon(false);
    t.start();
    log.info("main done");
}
```

The interviewer's prompt: "Main returns but the JVM keeps running. We expected it to exit. What controls this?"

Thinking out loud: 1. "JVM exits when all non-daemon threads finish. The heartbeat thread is non-daemon and runs forever." 2. "Either set daemon=true so it doesn't keep the JVM alive, or shut it down explicitly." 3. "Common with monitoring or scheduled threads accidentally kept non-daemon."

The actual bug: Non-daemon thread runs an infinite loop. JVM waits for it before exiting.

The fix (1-3 options if applicable): - **Option A:** `t.setDaemon(true);` if heartbeat should die with the app. - **Option B:** Add a shutdown hook: `Runtime.getRuntime().addShutdownHook(new Thread(() -> running.set(false)));` to signal the loop to exit. - **Option C:** Use a `ScheduledExecutorService` with daemon thread factory; call shutdown at app exit.

What NOT to say: - "Call `System.exit` on the thread" — drastic; bypasses shutdown hooks. - "Interrupt the thread" — doesn't terminate `while(true)` unless the loop checks the flag.

Common follow-up: "What's a daemon thread and when do you use it?"

Scenario 114 · The mockito test that passes accidentally

Asked at: Cred · Razorpay · **Difficulty:** Medium · **Topic:** Mockito stub vs verify

The code:

```
@Test void chargesCustomer() {
    when(gateway.charge(any())).thenReturn(true);
    service.placeOrder(o);
    verify(gateway).charge(any());
}
```

The interviewer's prompt: "Test passes. We deleted the `placeOrder` body and it still passes. Why?"

Thinking out loud: 1. "Wait — if placeOrder is empty, gateway.charge isn't called, so verify should fail." 2. "Unless `verify(gateway).charge(any())` is being misinterpreted. Actually that DOES assert call count = 1 by default." 3. "Maybe the test setup runs charge elsewhere — e.g. a previous test left state, or an @Before fixture calls it." 4. "Real failure mode: any() can match null, hiding type-mismatch bugs; or test reuse of mocks across @Test methods without reset."

The actual bug: Tests may pass for the wrong reason. Common cause: shared mock state across tests, or stubbing that hides argument mismatches.

The fix (1-3 options if applicable): - **Option A:** Use `@ExtendWith(MockitoExtension.class)` (JUnit 5) or `MockitoAnnotations.openMocks(this)` per-test to get fresh mocks. - **Option B:** Use argument captors instead of any: `ArgumentCaptor<Payment> cap = ArgumentCaptor.forClass(Payment.class); verify(gateway).charge(cap.capture()); assertEquals(...);`. Stricter assertions. - **Option C:** Cover negative cases too: a test that the method does NOT call charge when input is invalid.

What NOT to say: - "Mockito always passes if not strict" — there are strict and lenient modes; use STRICT_STUBS. - "Add Thread.sleep to fix timing" — wrong category.

Common follow-up: "What's the difference between Mockito's STRICT_STUBS and LENIENT?"

Scenario 115 · The Spring @Async that doesn't return

Asked at: Walmart Labs · JP Morgan · **Difficulty:** Hard · **Topic:** Spring @Async + proxy

The code:

```
@Service
class NotifierService {
    @Async
    public void send>Email e) { mailer.deliver(e); }

    public void onEvent>Email e){
        send(e); // expected async; runs sync
    }
}
```

The interviewer's prompt: "@Async should run in background. The call from onEvent runs synchronously. Why?"

Thinking out loud: 1. "Same self-invocation issue as @Transactional. @Async works via proxy." 2. "Internal `this.send(e)` bypasses the proxy. No async dispatch." 3. "Move the async method to a different bean, or invoke through the proxy."

The actual bug: Self-invocation skips the Spring proxy. @Async never runs async.

The fix (1-3 options if applicable): - **Option A:** Inject NotifierService into itself or into a separate caller that invokes through the proxy. - **Option B:** Split into two beans: NotifierService (with @Async send) and EventHandler (calls notifierService.send). - **Option C:** Use AspectJ load-time weaving — internal calls work.

What NOT to say: - "Spring's @Async magically wraps every call" — only through proxies. - "Add @Async to onEvent too" — onEvent isn't called externally either.

Common follow-up: "What ThreadPoolTaskExecutor configuration should you set for @Async in production?"

Scenario 116 · The Stream that processes elements twice in parallel

Asked at: Cred · Razorpay · **Difficulty:** Hard · **Topic:** Parallel stream + sorted

The code:

```
List<Long> ids = LongStream.range(0, 100)
    .parallel()
    .sorted()
    .boxed()
    .collect(toList());
```

The interviewer's prompt: "Parallel + sorted on a small stream sometimes runs the boxed step on more elements than we have. Tracing shows duplicate work. Why?"

Thinking out loud: 1. "Parallel streams split work across the common ForkJoinPool. Stateful intermediate ops like `sorted` require buffering." 2. "`sorted` forces a barrier — collects all elements, sorts, then continues. Splits get processed independently before the merge." 3. "Side effects in map/filter under parallel + stateful ops can run inconsistently; non-side-effect work is fine but recovery from a split can re-do work."

The actual bug: Reasoning about parallel pipelines with stateful intermediate ops is tricky. Side effects break.

The fix (1-3 options if applicable): - **Option A:** For small/medium data, drop `parallel()` — overhead of fork/join exceeds gains under 10k elements typically. - **Option B:** Avoid side effects in pipeline functions; rely on collectors only. - **Option C:** If you genuinely need sorted parallel processing, materialize first, sort sequentially, then process in parallel:

```
LongStream.range(...).boxed().sorted().toList().parallelStream()...
```

What NOT to say: - "Parallel streams always speed up the work" — they often don't. - "Use `forEachOrdered` to dedupe" — addresses ordering, not the side-effect concern.

Common follow-up: "When is `parallelStream` faster than sequential?"

Scenario 117 · The mutable static map and a memory leak

Asked at: Walmart Labs · **Cred:** · **Difficulty:** Hard · **Topic:** Classloader leak via static state

The code:

```
public class GlobalRegistry {  
    public static final Map<Object, byte[]> registry = new HashMap<>();  
}
```

The interviewer's prompt: "In Tomcat hot redeploy, the old app's heap stays around — leaked classloader. Eventually OOM. We don't keep references in the app. Trace shows our GlobalRegistry. Why?"

Thinking out loud: 1. "`public static final`" means the map lives for the lifetime of the classloader that loaded GlobalRegistry." 2. "Tomcat's parent classloader (shared) loads common classes; if any of those reference your app's classes through this map, app classloader can't unload." 3. "Classic deployment leak. Avoid static collections holding app-specific data."

The actual bug: Static state pins the classloader. Hot deploys leak the old app's heap.

The fix (1-3 options if applicable): - **Option A:** Avoid static mutable collections. Use a Spring-managed bean scoped to the app. - **Option B:** Clear the registry in a `@PreDestroy` / `ServletContextListener.contextDestroyed`. - **Option C:** Use `WeakHashMap` or weak values if the entries should be reclaimable.

What NOT to say: - "GC will collect the map on redeploy" — not if anything in the parent classloader references it. - "Increase Metaspace" — postpones the OOM, doesn't fix.

Common follow-up: "What's the typical leak pattern with `ThreadLocal` in Tomcat?"

Scenario 118 · The double release on `AutoCloseable`

Asked at: Cred · JP Morgan · **Difficulty:** Medium · **Topic:** try-with-resources + manual close

The code:

```
try (Connection c = ds.getConnection()) {  
    use(c);  
    c.close(); // explicit close, then try-with-resources also closes  
}
```

The interviewer's prompt: "This sometimes throws on the second close. What's idempotent and what isn't?"

Thinking out loud: 1. "AutoCloseable doesn't require idempotent close. JDBC Connection.close on an already-closed conn may throw 'connection closed'." 2. "try-with-resources unconditionally calls close on exit. The earlier explicit close makes it double-close." 3. "Remove the explicit close; let try-with-resources do it."

The actual bug: Double-close on a non-idempotent resource. Try-with-resources already handles closing.

The fix (1-3 options if applicable): - **Option A:** Remove the explicit `c.close()`. The try-with-resources handles it. - **Option B:** If you must close early, set the variable to null/wrap in null-check — but design smell. - **Option C:** Wrap in a Closeable that's idempotent: check internal flag, skip second close.

What NOT to say: - "AutoCloseable is idempotent" — the interface doesn't require it. - "Use finally instead" — same issue if both finally and try-with-resources close.

Common follow-up: "How does try-with-resources handle exceptions from close?"

Scenario 119 · The toString from a record vs Object

Asked at: TCS · **Cred:** · **Difficulty:** Easy · **Topic:** Record vs class default toString

The code:

```
class CartV1 { String id; int items; CartV1(String i, int n){id=i; items=n;} }
record CartV2(String id, int items) {}

System.out.println(new CartV1("C1", 3));
System.out.println(new CartV2("C2", 3));
```

The interviewer's prompt: "V1 prints garbage like `CartV1@1b6d3586`. V2 prints `CartV2[id=C2, items=3]`. Why the difference and what to do for V1?"

Thinking out loud: 1. "Object's default toString prints class@hashCode. Useless for diagnostics." 2. "Records auto-generate toString listing components." 3. "For classes, override toString manually or use Lombok @ToString. Records get it for free."

The actual bug: The class doesn't override toString. Default Object.toString is unhelpful.

The fix (1-3 options if applicable): - **Option A:** Convert to record if applicable — auto toString, equals, hashCode. - **Option B:** Override toString: `public String toString() { return "CartV1[id=" + id + ", items=" + items + "]; }`. - **Option C:** Lombok `@ToString` — auto-generates similar output.

What NOT to say: - "Object.toString prints the values" — it doesn't. - "Convert to JSON for printing" — overkill for log lines.

Common follow-up: "What does the default Object.toString format mean?"

Scenario 120 · The HashMap key mutation after insert

Asked at: Cred · Walmart Labs · **Difficulty:** Medium · **Topic:** Mutable key

The code:

```
class Key { Long id; Key(Long i){id=i;} public int hashCode(){return id.hashCode();}
    public boolean equals(Object o){return o instanceof Key k && k.id.equals(id);} }

Map<Key, Order> map = new HashMap<>();
Key k = new Key(1L);
map.put(k, o);
k.id = 2L;
System.out.println(map.get(k)); // null
System.out.println(map.get(new Key(1L))); // null
System.out.println(map.size()); // 1
```

The interviewer's prompt: "Insert with key id=1, mutate to id=2. Map has one entry but no key gets it. Why?"

Thinking out loud: 1. "Hash at insertion was hash(1). Bucket chosen accordingly." 2. "After mutation, hashCode is hash(2). get(k) hashes to new bucket — wrong bucket." 3. "get(new Key(1L)) hashes to old bucket but equals matches via id==2 vs id==1 → false. Still null." 4. "Entry is orphaned: can't be found via any key."

The actual bug: Mutation of a hash-relevant field after insertion orphans the entry.

The fix (1-3 options if applicable): - **Option A:** Make hash-relevant fields final. Best practice. - **Option B:** Use immutable key types (records, autoboxed primitives, etc.). - **Option C:** If you must mutate, remove + re-add: `map.remove(k); k.id = 2L; map.put(k, o);`.

What NOT to say: - "Rehash the map" — Java map doesn't expose rehash. - "Use IdentityHashMap" — doesn't help; problem is the bucket, not identity.

Common follow-up: "How does this differ for TreeMap with a mutating Comparable field?"

Scenario 121 · The Optional field on a JPA entity

Asked at: JP Morgan · Walmart Labs · **Difficulty:** Medium · **Topic:** Optional as field

The code:

```
@Entity
class Customer {
    @Id Long id;
    private Optional<String> phone;
}
```

The interviewer's prompt: "This entity won't map cleanly. Hibernate complains. Why is Optional a bad field type?"

Thinking out loud: 1. "Optional is meant for return types — explicit absence in method APIs." 2. "As a field, Optional doesn't serialize cleanly, doesn't map to JPA columns, and adds a wrapper allocation per row." 3. "Use a nullable String field; expose getPhone returning Optional."

The actual bug: Optional used as a JPA field. Hibernate has no out-of-box mapping; pollutes the persistence model.

The fix (1-3 options if applicable): - **Option A:** `private String phone;` and `public Optional<String> getPhone() { return Optional.ofNullable(phone); }` . Standard. - **Option B:** Use a custom Hibernate UserType — complex; rarely worth it. - **Option C:** For non-JPA, similar story for Jackson serialization — Optional fields need a module.

What NOT to say: - "Optional is the future, use it everywhere" — designed for return types only. - "Mark Optional @Transient" — defeats the field's purpose.

Common follow-up: "What's the canonical use case for Optional?"

Scenario 122 · The volatile that's not enough for compound check

Asked at: Cred · JP Morgan · **Difficulty:** Hard · **Topic:** volatile + check-then-act

The code:

```
private volatile int seats = 100;

public void book() {
    if (seats > 0) {
        seats--;
    }
}
```

The interviewer's prompt: "Volatile makes seats visible. We still oversell. Why?"

Thinking out loud: 1. "Volatile gives visibility, not atomicity of compound ops." 2. "Two threads both see 1, both decrement, both succeed. Sold 2 of last 1." 3. "Need atomic compare-and-set or a lock."

The actual bug: Check-then-act is not atomic. Volatile doesn't help with compound operations.

The fix (1-3 options if applicable): - **Option A:** `AtomicInteger` + CAS loop: `int s; do { s = seats.get(); if (s == 0) return; } while (!seats.compareAndSet(s, s - 1));` . - **Option B:** synchronized block around the if-decrement. - **Option C:** Use a `Semaphore(100)` — `if (semaphore.tryAcquire()) bookOne();` . Models capacity directly.

What NOT to say: - "Volatile makes it atomic" — only for reads/writes of a single value, not compound checks. - "Add a sleep before the check" — race-prone hack.

Common follow-up: "What's a CAS loop and when does it spin too much?"

Scenario 123 · The instance method reference to a null

Asked at: TCS · **Cred:** · **Difficulty:** Easy · **Topic:** Method reference + null

The code:

```
Customer c = null;
Supplier<String> s = c::getName;
s.get(); // NPE
```

The interviewer's prompt: "Creating the method reference seems to succeed. Calling it throws NPE on null. Why?"

Thinking out loud: 1. "`c::getName` captures `c` at creation. If `c` is null then, you've captured null." 2. "Invocation later dereferences `c`. NPE." 3. "`Class::instanceMethod` form takes the instance as the first arg — different binding."

The actual bug: Bound method reference captures the receiver at creation. A null receiver fails on invocation, not creation.

The fix (1-3 options if applicable): - **Option A:** Check for null before creating the reference. - **Option B:** Use the unbound form: `Function<Customer, String> f = Customer::getName;` `f.apply(c);` — explicit null behaviour. - **Option C:** Wrap: `Supplier<String> s = () -> c == null ? "<unknown>" : c.getName();` . Defensive.

What NOT to say: - "Method references are lazy" — the receiver is captured eagerly. - "Use `Optional.of(c).map(...)`" — `Optional.of` throws on null too.

Common follow-up: "What's the difference between `instance::method` , `Class::staticMethod` , and `Class::instanceMethod` ?"

Scenario 124 · The catch that hides the OutOfMemoryError

Asked at: Walmart Labs · **Cred:** · **Difficulty:** Hard · **Topic:** Catching Throwable

The code:

```
try {
    runBatch();
} catch (Throwable t) {
    log.error("batch failed", t);
}
```

The interviewer's prompt: "We catch Throwable so the app keeps running. After OOM, behaviour gets weird — wrong results, threads die randomly. Why?"

Thinking out loud: 1. "Catching Throwable swallows Error too — OOM, StackOverflow, etc." 2. "OOM means the JVM was unable to allocate. Other threads may already have died mid-operation. Recovery is undefined." 3. "Don't catch Error. At most catch Exception."

The actual bug: Catching Throwable lets the JVM proceed after Error. Errors are not safely recoverable in general.

The fix (1-3 options if applicable): - **Option A:** `catch (Exception e)` — leaves Errors to propagate and JVM-default handling. - **Option B:** If you must catch Error for logging, rethrow: `catch (Throwable t) { log.error("...", t); if (t instanceof Error) throw t; }`. - **Option C:** Configure heap dump on OOM (`-XX:+HeapDumpOnOutOfMemoryError`) — gives forensic data even when the process exits.

What NOT to say: - "Errors are recoverable with enough memory" — OOM may leave the JVM in an unstable state. - "Restart the affected thread" — fragile; not the right answer.

Common follow-up: "What's the difference between Exception and Error in the hierarchy?"

Scenario 125 · The ScheduledThreadPool that drifts

Asked at: Cred · JP Morgan · **Difficulty:** Medium · **Topic:** `scheduleWithFixedDelay` vs `scheduleAtFixedRate`

The code:

```
scheduler.scheduleAtFixedRate(this::syncData, 0, 60, TimeUnit.SECONDS);
```


The interviewer's prompt: "syncData takes 90 seconds during peak load. We expected the next run 60 seconds later. Logs show back-to-back execution. Why?"

Thinking out loud: 1. "scheduleAtFixedRate schedules invocations at fixed wall-clock intervals — start times every 60s regardless of duration." 2. "If a run takes 90s, the next is overdue by 30s and fires immediately on the same thread (since single-threaded pool serializes)." 3. "For 'wait 60s after the previous run ends', use scheduleWithFixedDelay."

The actual bug: Misuse of scheduleAtFixedRate when scheduleWithFixedDelay was intended.

The fix (1-3 options if applicable): - **Option A:** `scheduleWithFixedDelay(this::syncData, 0, 60, SECONDS);` — 60s between completion and next start. - **Option B:** Keep AtFixedRate but increase the interval to exceed worst-case duration. - **Option C:** Use a different pool size if you want parallel execution despite long runs.

What NOT to say: - "They behave the same if duration < interval" — true; the bug surfaces when duration > interval. - "Throw on long runs" — doesn't fix scheduling semantics.

Common follow-up: "What happens if a task throws under scheduleAtFixedRate?"

Scenario 126 · The Comparator that depends on external state

Asked at: JP Morgan · Walmart Labs · **Difficulty:** Hard · **Topic:** Stateful comparator

The code:

```
Comparator<Order> byCity = (a, b) -> {
    int aRank = cityRanks.getOrDefault(a.getCity(), 0);
    int bRank = cityRanks.getOrDefault(b.getCity(), 0);
    return Integer.compare(aRank, bRank);
};
orders.sort(byCity);
// cityRanks updated by another thread during sort
```

The interviewer's prompt: "Sort throws IllegalArgumentException about contract violation, but only sometimes. The comparator looks deterministic."

Thinking out loud: 1. "The map cityRanks is mutated concurrently. compare(a, b) early in the sort may use rank=1; later compare(a, c) may use rank=2 for the same a." 2. "Transitivity is violated. TimSort detects and throws." 3. "Comparators must be deterministic for the duration of the sort."

The actual bug: Comparator depends on mutable shared state. Concurrent modification of `cityRanks` breaks transitivity.

The fix (1-3 options if applicable): - **Option A:** Snapshot `cityRanks` before sorting: `Map<String, Integer> ranks = new HashMap<>(cityRanks); Comparator<Order> byCity = (a, b)`

-> `Integer.compare(ranks.get(...), ranks.get(...));` . - **Option B**: Synchronize access to `cityRanks` for the sort duration. - **Option C**: Pre-compute ranks per order into the list first, then sort by the pre-computed values.

What NOT to say: - "Sort always works on a snapshot" — it does not; it calls `compare` directly on the list. - "Use parallel sort" — same problem, worse.

Common follow-up: "What's the formal contract of `Comparator.compare`?"

Scenario 127 · The `forEach` in `Map.Entry` replacing value

Asked at: TCS · Wipro · **Difficulty**: Easy · **Topic**: Map entry mutation

The code:

```
Map<String, Integer> stock = new HashMap<>(loadStock());
for (Map.Entry<String, Integer> e : stock.entrySet()) {
    if (e.getValue() < 10) {
        e.setValue(0); // mark stockout
    }
}
```

The interviewer's prompt: "This works on `HashMap`. On a sub-map view from `ConcurrentSkipListMap.subMap`, `setValue` throws. Why?"

Thinking out loud: 1. "`Entry.setValue` is optional. Standard `HashMap` supports it; some sub-map views or unmodifiable views may not." 2. "For `ConcurrentSkipListMap` submap, depending on bounds, `setValue` may throw." 3. "Best to call `map.put(key, newValue)` instead of relying on `setValue`."

The actual bug: `Entry.setValue` is an optional operation. Some `Map` implementations or views don't support it.

The fix (1-3 options if applicable): - **Option A**: Use `map.put(key, newValue)` — universal, doesn't rely on `Entry` semantics. - **Option B**: Use `map.replaceAll((k, v) -> v < 10 ? 0 : v);` — bulk replace with a function. - **Option C**: Use `entrySet().stream().filter(...).forEach(e -> map.put(e.getKey(), 0));` — explicit `put`.

What NOT to say: - "`Entry.setValue` always works" — optional in the `Map.Entry` contract. - "Wrap in a try-catch" — fine but masks where the bug is.

Common follow-up: "What other operations on `Map` are 'optional' per the `Collection` framework spec?"

Scenario 128 · The synchronized that protects the wrong scope

Asked at: Cred · JP Morgan · **Difficulty:** Hard · **Topic:** synchronized scope too narrow

The code:

```
class Wallet {
    BigDecimal balance = BigDecimal.ZERO;

    public boolean tryDebit(BigDecimal amount) {
        if (balance.compareTo(amount) < 0) return false;
        synchronized (this) {
            balance = balance.subtract(amount);
        }
        return true;
    }
}
```

The interviewer's prompt: "Wallet can go negative under load. The synchronized block is right there. What's wrong?"

Thinking out loud: 1. "Check (compareTo) is OUTSIDE the synchronized block. Two threads can both pass the check before either acquires the lock." 2. "Both then enter, subtract, balance goes negative." 3. "Move the check inside the lock."

The actual bug: Check-then-act split across sync boundary. Race window between check and decrement.

The fix (1-3 options if applicable): - **Option A:** Move check inside synchronized: `synchronized (this) { if (balance.compareTo(amount) < 0) return false; balance = balance.subtract(amount); } return true;` . - **Option B:** Use a single AtomicReference with compare-and-set in a loop. - **Option C:** Use a Lock + Condition for more complex wallet operations.

What NOT to say: - "Synchronized always covers the whole method" — only when applied to the method declaration. - "Mark `balance` volatile" — doesn't help with compound operation.

Common follow-up: "What's the correct order: lock-then-check or check-then-lock?"

Scenario 129 · The IntStream that boxes more than expected

Asked at: Razorpay · Cred · **Difficulty:** Medium · **Topic:** Stream primitive vs object

The code:

```
int total = orders.stream()
    .map(Order::getAmount)
    .reduce(0, Integer::sum);
```

The interviewer's prompt: "This works but profiling shows a lot of Integer boxing. We want to sum ints, not Integers. What changes?"

Thinking out loud: 1. "`stream().map(...)` produces a Stream. Reduce with `Integer::sum` boxes and unboxes per step." 2. "Use `mapToInt` to get an `IntStream` — primitive-specialized, no boxing." 3. "Then `.sum()` is a single primitive op."

The actual bug: Using object Stream where `IntStream` would avoid boxing.

The fix (1-3 options if applicable): - **Option A:**

`orders.stream().mapToInt(Order::getAmount).sum();` . No boxing, faster. - **Option B:** For long values: `mapToLong` . For double: `mapToDouble` . - **Option C:** For `BigDecimal` sums, no primitive specialization; stay with object stream and `reduce(BigDecimal.ZERO, BigDecimal::add)` .

What NOT to say: - "Boxing is free on modern JVMs" — not free; predictable overhead. - "Use parallel stream" — fixes nothing; same boxing.

Common follow-up: "What's the JVM cost of boxing an int?"

Scenario 130 · The Thread.sleep that loses precision

Asked at: Cred · JP Morgan · **Difficulty:** Medium · **Topic:** Thread.sleep accuracy + interruption

The code:

```
public void rateLimit() {
    try {
        Thread.sleep(100);
    } catch (InterruptedException e) {
        // ignored
    }
}
```

The interviewer's prompt: "We rate-limit at 100ms per call. Under load it sometimes sleeps 200ms or more. Also, shutdown takes forever because interruption is swallowed. Two bugs?"

Thinking out loud: 1. "Sleep precision is OS-dependent. On busy systems, sleeps overshoot by tens to hundreds of ms." 2. "Empty catch on `InterruptedException` — restore the flag or rethrow." 3. "For tighter timing, use `ScheduledExecutorService` or higher-resolution waits."

The actual bug: Two issues — sleep precision under load isn't tight, and `InterruptedException` is swallowed, breaking shutdown propagation.

The fix (1-3 options if applicable): - **Option A:** Use `RateLimiter` (Guava) or `Semaphore` with permit refill — designed for this; not based on sleep. - **Option B:** At minimum, restore interrupt: `catch (InterruptedException e) { Thread.currentThread().interrupt(); return; } .` - **Option C:** For tighter timing, `LockSupport.parkNanos(100_000_000L)` — also subject to OS scheduling but better wakeup guarantee.

What NOT to say: - "Sleep is exact" — it's a minimum, not a guarantee. - "Catch and ignore is fine" — breaks interruption protocol.

Common follow-up: "How would you build a rate limiter for 100 ops/sec?"

Scenario 131 · The recursive toString via Lombok

Asked at: Cred · TCS · **Difficulty:** Medium · **Topic:** Lombok @Data + bidirectional refs

The code:

```
@Data
class Department { String name; List<Employee> employees; }

@Data
class Employee { String name; Department dept; }
```

The interviewer's prompt: "Stackoverflow when we log a Department. Both classes use Lombok @Data. We didn't write toString. What gives?"

Thinking out loud: 1. "@Data generates toString, equals, hashCode for ALL fields including the back-reference." 2. "Cycle: department.toString → employees → employee.toString → dept → ..." 3. "@ToString.Exclude one side, or use @Data(exclude=...)."

The actual bug: Lombok-generated toString recurses through bidirectional reference.

The fix (1-3 options if applicable): - **Option A:** Annotate `@ToString.Exclude` on `Employee.dept` . Breaks the cycle. - **Option B:** Replace @Data with @Getter @Setter @EqualsAndHashCode(exclude=...). Finer control. - **Option C:** Use DTOs for logging — never log the entity directly.

What NOT to say: - "Lombok is buggy" — it does what you ask. - "Catch StackOverflowError in toString" — masks the design.

Common follow-up: "Why is equals/hashCode also dangerous for bidirectional refs?"

Scenario 132 · The classpath-bound resource that's null

Asked at: Walmart Labs · JP Morgan · **Difficulty:** Medium · **Topic:** `getResourceAsStream` path

The code:

```
InputStream in = MyService.class.getResourceAsStream("/data/cities.csv");
if (in == null) throw new IllegalStateException("cities.csv not on classpath");
```

The interviewer's prompt: "File is in `src/main/resources/data/cities.csv`. Works locally. Throws on the Docker build. What changed?"

Thinking out loud: 1. "`getResourceAsStream`" looks via the class loader. Path with leading slash is absolute from classpath root." 2. "Locally maybe IDE serves `src/main/resources` directly. In Docker the jar may not include the file (build excluded it, wrong path, packaging issue)." 3. "Verify the jar contents: `jar tf app.jar | grep cities` or check Spring Boot's `BOOT-INF/classes` layout."

The actual bug: The resource isn't in the deployment artifact. IDE classpath differs from packaged classpath.

The fix (1-3 options if applicable): - **Option A:** Add the resource to your build: check pom/gradle includes, verify with `jar tf`. - **Option B:** For Spring Boot, use `ClassPathResource("data/cities.csv")` — handles the `BOOT-INF` prefix. - **Option C:** Move to an external mount (configmap, S3) if the file is environment-specific.

What NOT to say: - "Classpath always includes resources" — not if build excludes them. - "Drop the leading slash" — different semantics (relative to class package).

Common follow-up: "What's the difference between `Class.getResource` and `ClassLoader.getResource`?"

Scenario 133 · The TreeSet with custom Comparator and contains

Asked at: JP Morgan · Cred · **Difficulty:** Medium · **Topic:** `TreeSet` equals via compare

The code:

```
class Order { String id; BigDecimal amount; }

TreeSet<Order> set = new TreeSet<>(Comparator.comparing(Order::getAmount));
set.add(new Order("A", new BigDecimal("100")));
System.out.println(set.contains(new Order("B", new BigDecimal("100")))); // true
```

The interviewer's prompt: "Two orders with different IDs but same amount. `TreeSet` says it contains the second. We never inserted it. Why?"

Thinking out loud: 1. "TreeSet uses the Comparator for equality. compare == 0 means 'equal' to the set." 2. "Both orders have amount 100. compare returns 0. Set considers them equal." 3. "For TreeSet, comparator must be consistent with equals (or you must accept this semantics)."

The actual bug: TreeSet treats compare==0 as equality. Comparator on amount alone collides across different orders.

The fix (1-3 options if applicable): - **Option A:** Multi-field comparator:

`Comparator.comparing(Order::getAmount).thenComparing(Order::getId)` .

Distinguishes. - **Option B:** Use HashSet if equals/hashCode on Order are correct — uses equals, not compare. - **Option C:** Wrap in a sorted view: HashSet for membership, TreeSet for ordering.

What NOT to say: - "TreeSet uses equals" — it uses compare. - "Add a unique ID to the comparator after a sleep" — meaningless.

Common follow-up: "What's the difference between a Set's equals semantics with HashSet vs TreeSet?"

Scenario 134 · The static factory that returns the same instance

Asked at: Cred · Razorpay · **Difficulty:** Easy · **Topic:** Static factory cached instance

The code:

```
public static List<String> emptyTags() {
    return Collections.emptyList();
}

emptyTags().add("x"); // UnsupportedOperationException
```

The interviewer's prompt: "We return Collections.emptyList() as a default. Callers add to it sometimes. Throws. We expected a fresh mutable list."

Thinking out loud: 1. "Collections.emptyList() returns a singleton — immutable, shared." 2. "Mutating throws. Callers expected `new ArrayList<>()` semantics." 3. "If you want a mutable empty, return new ArrayList<>() explicitly."

The actual bug: `Collections.emptyList` is a shared immutable singleton. Callers assume mutability.

The fix (1-3 options if applicable): - **Option A:** `return new ArrayList<>();` — fresh, mutable, every call. - **Option B:** Document that the returned list is immutable; callers must copy before mutating. - **Option C:** Return `List.of()` and force callers to copy — at least the immutability is named.

What NOT to say: - "Collections.emptyList returns a fresh list" — it returns a singleton. - "Subclass emptyList" — can't; it's package-private.

Common follow-up: "What's the singleton pattern in JDK's emptyList/emptyMap/emptySet?"

Scenario 135 · The Stream that processes lazily off the end

Asked at: Walmart Labs · JP Morgan · **Difficulty:** Hard · **Topic:** Stream + closed resource

The code:

```
Stream<String> lines() throws IOException {
    return Files.lines(Path.of("data.csv"));
}

lines().forEach(this::process); // works
List<String> all = lines().collect(toList()); // works
// later
Stream<String> s = lines();
return s; // returned to caller; caller iterates
```

The interviewer's prompt: " `Files.lines` returns a Stream backed by a file. If we return it to the caller, the file is never closed properly. What's the pattern?"

Thinking out loud: 1. "Files.lines returns a Stream that holds an open file handle. Closing the stream closes the file." 2. "If we return the stream and caller forgets to close, FD leak." 3. "Either consume inside the method (try-with-resources around Files.lines), or document that callers must close, or return List after collecting."

The actual bug: Resource-backed stream returned without ownership transfer. Caller may leak FDs.

The fix (1-3 options if applicable): - **Option A:** Consume internally: `try (Stream<String> s = Files.lines(path)) { return s.collect(toList()); }` . Caller gets a List. - **Option B:** Document and require the caller to use try-with-resources: `try (Stream<String> s = service.lines()) { ... }` . - **Option C:** For very large files, use `BufferedReader.lines()` similarly with try-with-resources.

What NOT to say: - "Streams don't hold resources" — Files.lines does. - "GC will close it" — FDs aren't memory; not reliable.

Common follow-up: "When does a stream need to be in try-with-resources?"

Scenario 136 · The lambda inside a loop that captures the iteration variable

Asked at: TCS · Wipro · **Difficulty:** Medium · **Topic:** Capture in loop

The code:


```
List<Runnable> tasks = new ArrayList<>();
for (int i = 0; i < 3; i++) {
    final int idx = i;
    tasks.add(() -> System.out.println(idx));
}
tasks.forEach(Runnable::run);
// prints 0 1 2 – works
```

The interviewer's prompt: "This works. But colleagues' old Java 7 code with anonymous inner classes had the same loop without the `final int idx = i;` trick and printed 3 3 3. What changed?"

Thinking out loud: 1. "Java 7 needed `final` for capture. The for-loop counter `i` isn't final, so they had to alias to a final." 2. "Java 8+ allows effectively-final capture. Inside a for-loop body, each iteration creates a fresh `idx` if you declare it inside; for the loop variable `i` itself, it's reassigned, so not effectively final." 3. "The pattern of `final int idx = i;` still works; without it, lambda using `i` won't compile in modern Java either."

The actual bug: Not a bug here; the snippet uses the workaround correctly. The trap is the old version that printed 3 3 3.

The fix (1-3 options if applicable): - **Option A:** Keep the alias pattern (`final int idx = i;`) — modern Java enforces this via "effectively final". - **Option B:** Convert to streams:

`IntStream.range(0, 3).forEach(i -> tasks.add(() -> System.out.println(i)));`
— `i` is effectively final per call. - **Option C:** Use enhanced for-loop where possible — loop variable is effectively final.

What NOT to say: - "Lambdas capture by reference" — they capture by value, but the value must be effectively final. - "Use `var i`" — same rule applies.

Common follow-up: "In Java 7 anonymous inner classes, why did `i` print 3 3 3?"

Scenario 137 · The Set whose contains is O(n)

Asked at: Walmart Labs · **Cred:** · **Difficulty:** Medium · **Topic:** Wrong Set + hashCode collision

The code:

```
class Cell { int row, col;
    Cell(int r, int c) { row = r; col = c; }
    public int hashCode() { return row; } // bad: only uses row
    public boolean equals(Object o) { return o instanceof Cell c && c.row == row && c.col == col; }
}

Set<Cell> grid = new HashSet<>();
for (int r = 0; r < 1000; r++) for (int c = 0; c < 1000; c++) grid.add(new Cell(r, c));
grid.contains(new Cell(500, 500)); // very slow
```

The interviewer's prompt: "contains is slow even though HashSet should be O(1). Profiler shows linear scan in a bucket. Why?"

Thinking out loud: 1. "hashCode = row only. All 1000 cells with the same row hash to the same bucket — chain of length 1000." 2. "contains walks the chain — O(n) within the bucket." 3. "Bad hash distribution. Use both fields."

The actual bug: hashCode uses only one field; massive collisions ruin HashSet performance.

The fix (1-3 options if applicable): - **Option A:** `Objects.hash(row, col)` — combines properly. - **Option B:** Make Cell a record — auto hashCode uses all components. - **Option C:** Use a 2D array if cells are dense — no hashing needed.

What NOT to say: - "HashSet is slow on big data" — it's not, with a good hash. - "Use LinkedHashMap" — doesn't change distribution.

Common follow-up: "What does Java 8 do when a bucket grows large?"

Scenario 138 · The Stream.collect that returns wrong order

Asked at: TCS · **Cred:** · **Difficulty:** Medium · **Topic:** parallel + ordered + unordered

The code:

```
List<String> result = items.parallelStream()
    .map(Item::getName)
    .collect(toList());
```

The interviewer's prompt: "Sometimes the result is in a different order than the input. We thought streams preserve order."

Thinking out loud: 1. " `parallelStream` + ordered source + `collect(toList())` preserves encounter order in the result by spec." 2. "If you see reorder, the source may be unordered (e.g. HashSet), or you used `forEach` (no ordering guarantee in parallel)." 3. "For sequential streams with HashSet source: iteration order is HashMap-internal — not insertion."

The actual bug: Either the source is unordered (HashSet, ConcurrentHashMap keys, etc.) or `forEach` was used instead of `forEachOrdered`.

The fix (1-3 options if applicable): - **Option A:** Use an ordered source (List, LinkedHashSet) if order matters. - **Option B:** For parallel + side-effect ordering, use `forEachOrdered` instead of `forEach`. - **Option C:** Sort explicitly after collection if order semantics need a clear definition.

What NOT to say: - "Parallel streams reorder by spec" — only with `forEach` or unordered sources. - "Use `.sequential()`" — works but throws away parallelism.

Common follow-up: "What's the difference between encounter order and iteration order?"

Scenario 139 · The HashMap with NaN key

Asked at: Zerodha · JP Morgan · **Difficulty:** Hard · **Topic:** Double.NaN equals

The code:

```
Map<Double, String> map = new HashMap<>();
map.put(Double.NaN, "missing");
System.out.println(map.get(Double.NaN)); // null
```

The interviewer's prompt: "NaN as a key — `put` succeeds, `get` returns null. Why?"

Thinking out loud: 1. "`Double.NaN != NaN`". NaN is not equal to itself by IEEE 754." 2. "But `Double.equals` (wrapper) treats NaN as equal to NaN (consistent with hash)." 3. "Trick: HashMap uses `==` for the key compare alongside `equals` — wait, no, only `equals`." 4. "Actually HashMap.get works with NaN. The bug here might be that one side is primitive double and the other is Double wrapper." 5. "Re-check: `map.get(Double.NaN)` autoboxes — yes, Double. `equals` on Double considers `NaN==NaN`. Should work." 6. "So actually, `contains` and `get` should return the value. The 'null' result suggests something else — maybe NaN encoding differs (different NaN bit patterns)."

The actual bug: Multiple NaN bit patterns exist. If you put one and lookup with another, `Double.equals` considers them equal (it normalizes); but if the implementation uses `==` anywhere on the boxed value's internal field, it can miss. In practice, Java's `Double.equals` is consistent for NaN, so the bug is usually that the user expects primitive `==` semantics and is surprised that lookups by value work.

The fix (1-3 options if applicable): - **Option A:** Don't use floating-point as a map key. Use a discrete type (long, String). - **Option B:** If you must, use `Double.valueOf(Double.NaN)` consistently — same canonical wrapper. - **Option C:** For numeric keys with potential NaN, normalize NaN to a sentinel before insert/get.

What NOT to say: - "NaN equals NaN" — primitive `==` says false; wrapper `equals` says true. - "Use TreeMap with NaN" — even worse; NaN ordering is undefined.

Common follow-up: "Why does `Double.NaN == Double.NaN` return false?"

Scenario 140 · The class cast on List

Asked at: TCS · Cognizant · **Difficulty:** Medium · **Topic:** Generic cast erasure

The code:

```
public <T> List<T> emptyTyped() { return Collections.emptyList(); }

List<Integer> ints = emptyTyped();
ints.add(1); // throws? compiles?
Object o = ints;
List<String> strs = (List<String>) o;
strs.add("hi");
System.out.println(ints.get(0)); // ClassCastException at use site
```

The interviewer's prompt: "Heap pollution example. Explain when the ClassCastException happens."

Thinking out loud: 1. "Generics are erased. Underlying list is just List." 2. "`ints.add(1)` throws because `Collections.emptyList` is immutable — `UnsupportedOperationException`, not heap pollution." 3. "With a mutable List cast to List, you can add a String. Later `.get` on the Integer-view triggers the implicit cast to Integer, throwing `ClassCastException`." 4. "The CCE fires at the USE site, not the add site."

The actual bug: Erasure allows the unchecked cast at compile time. The error appears at the use site when the implicit cast to the declared type fails.

The fix (1-3 options if applicable): - **Option A:** Never cast across generic types via Object. Refactor or duplicate logic. - **Option B:** Use `Class<T>` token to check element types at runtime. - **Option C:** Use `Collections.checkedList` — adds runtime type checks on add.

What NOT to say: - "Java enforces generic types at runtime" — it does not. - "CCE on add" — actually on get/iteration.

Common follow-up: "What is `Collections.checkedList` and when do you use it?"

Scenario 141 · The Spring bean leaking after exception

Asked at: Walmart Labs · JP Morgan · **Difficulty:** Medium · **Topic:** @Transactional rollback on checked exception

The code:

```
@Service
class OrderService {
    @Transactional
    public void placeOrder(Order o) throws PaymentException {
        repo.save(o);
        paymentClient.charge(o); // throws PaymentException (checked)
    }
}
```

The interviewer's prompt: "PaymentException is thrown. The save was supposed to roll back. Row stays in DB. Why?"

Thinking out loud: 1. "Spring's default rollback rule: rolls back on RuntimeException and Error, NOT on checked Exception." 2. "PaymentException is checked. No rollback." 3. "Configure `@Transactional(rollbackFor = PaymentException.class)` or `rollbackFor = Exception.class`."

The actual bug: Spring doesn't roll back checked exceptions by default.

The fix (1-3 options if applicable): - **Option A:** `@Transactional(rollbackFor = PaymentException.class)` . Explicit. - **Option B:** Change PaymentException to extend RuntimeException — auto-rollback. - **Option C:** `@Transactional(rollbackFor = Exception.class)` — catch-all; clear intent.

What NOT to say: - "Spring rolls back any exception" — not by default. - "Catch and rethrow as RuntimeException" — workaround; better to configure.

Common follow-up: "Why doesn't Spring roll back checked exceptions by default?"

Scenario 142 · The Stream.skip on parallel that's slow

Asked at: Cred · Razorpay · **Difficulty:** Hard · **Topic:** Stateful op + parallel

The code:

```
List<Order> page = orders.parallelStream()
    .skip(10_000)
    .limit(100)
    .toList();
```

The interviewer's prompt: "Pagination via skip+limit on parallel stream is SLOWER than sequential on a million-element list. Why?"

Thinking out loud: 1. "skip and limit are stateful, ordering-dependent. In parallel, ordering must be maintained — requires extra coordination." 2. "Parallel adds overhead without speedup for these ops." 3. "For pagination over large data, don't use streams skip/limit; index into a List directly or use DB-level pagination."

The actual bug: skip/limit are stateful intermediate ops. Parallel + stateful ops give little or negative speedup.

The fix (1-3 options if applicable): - **Option A:** Use `orders.subList(10_000, 10_100)` if it's an ArrayList — O(1) view. - **Option B:** For DB-backed data, paginate at the SQL level: LIMIT/OFFSET. - **Option C:** Sequential stream for skip/limit, or pre-bucket the data.

What NOT to say: - "Parallel is always faster" — depends on workload. - "Increase the parallelism level" — won't help stateful ops.

Common follow-up: "What's a stateful intermediate operation in the Stream API?"

Scenario 143 · The HashMap put after computeIfAbsent

Asked at: TCS · Razorpay · **Difficulty:** Medium · **Topic:** ConcurrentModificationException from computeIfAbsent

The code:

```
ConcurrentHashMap<String, List<Order>> byCity = new ConcurrentHashMap<>();
byCity.computeIfAbsent("Bengaluru", k -> {
    byCity.put("Pune", new ArrayList<>()); // forbidden
    return new ArrayList<>();
});
```

The interviewer's prompt: "computeIfAbsent throws IllegalStateException. We modify the map inside the function. Why is that prohibited?"

Thinking out loud: 1. "ConcurrentHashMap's compute methods are atomic on the key. Modifying other keys inside the lambda can lead to recursion or deadlock." 2. "CHM detects re-entrant modification on the same map and throws." 3. "Compute the side effect outside the lambda."

The actual bug: ConcurrentHashMap forbids modifications to itself inside computeIfAbsent/compute/merge functions.

The fix (1-3 options if applicable): - **Option A:** Compute the secondary insert outside:
`byCity.computeIfAbsent("Bengaluru", k -> new ArrayList<>());`

`byCity.putIfAbsent("Pune", new ArrayList<>());` . - **Option B:** Use a separate Map for the side effects. - **Option C:** Use plain HashMap if you don't need concurrency — but then you lose safety.

What NOT to say: - "ConcurrentHashMap supports any modification" — it doesn't, inside compute. - "Wrap in synchronized" — doesn't fix the re-entrance check.

Common follow-up: "Why does ConcurrentHashMap explicitly forbid re-entrance in compute?"

Scenario 144 · The Pattern.compile that's not thread-safe

Asked at: Cred · Walmart Labs · **Difficulty:** Medium · **Topic:** Pattern vs Matcher concurrency

The code:

```
public class PanValidator {
    private static final Pattern PATTERN = Pattern.compile("[A-Z]{5}[0-9]{4}[A-Z]");
    private static final Matcher MATCHER = PATTERN.matcher(""); // shared

    public static boolean isValid(String s) {
        MATCHER.reset(s);
        return MATCHER.matches();
    }
}
```

The interviewer's prompt: "Validation occasionally returns wrong results under load. Pattern is compiled once. What's wrong?"

Thinking out loud: 1. "Pattern is thread-safe. Matcher is NOT." 2. "Shared matcher reset+match across threads — state corruption." 3. "Either create a fresh Matcher per call, or use ThreadLocal."

The actual bug: Sharing a Matcher across threads. Matcher is stateful and not thread-safe.

The fix (1-3 options if applicable): - **Option A:** `PATTERN.matcher(s).matches();` — fresh Matcher per call. Cheap. - **Option B:** `ThreadLocal<Matcher>` — one Matcher per thread. - **Option C:** For very hot validation, hand-written validator beats both.

What NOT to say: - "Pattern handles concurrency for the Matcher" — only the compile step is thread-safe. - "Synchronize matcher" — works but serializes the validation.

Common follow-up: "What's the cost of creating a Matcher from an already-compiled Pattern?"

Scenario 145 · The catch-rethrow that loses the cause

Asked at: TCS · Cognizant · **Difficulty:** Easy · **Topic:** Throw new without cause

The code:

```
try {
    repo.fetch(id);
} catch (SQLException e) {
    throw new RuntimeException("DB error");
}
```

The interviewer's prompt: "Stack trace shows only RuntimeException — no SQL details. Debug is painful. Fix?"

Thinking out loud: 1. " `new RuntimeException(message)` loses the cause. Use the 2-arg constructor." 2. " `new RuntimeException(message, e)` — chains the SQLException as cause." 3. "Stack trace then shows 'Caused by: SQLException ...'."

The actual bug: New exception created without chaining the cause. Original stack trace lost.

The fix (1-3 options if applicable): - **Option A:** `throw new RuntimeException("DB error", e);` — chains. - **Option B:** `throw new MyDomainException("DB error", e);` — typed domain exception with chain. - **Option C:** Don't catch if you can let it propagate — usually best.

What NOT to say: - "Logger logs the original" — not if logged elsewhere or only the wrapper. - "Use `printStackTrace` on the cause first" — pollutes logs.

Common follow-up: "How do you read 'Caused by' in a Java stack trace?"

Scenario 146 · The empty array passed to varargs

Asked at: Cognizant · TCS · **Difficulty:** Medium · **Topic:** Varargs with empty array

The code:


```
public void log(String tag, Object... params) {
    System.out.println(tag + ": " + Arrays.toString(params));
}

Object[] empty = {};
log("checkout", empty); // prints "checkout: []"
log("checkout"); // prints "checkout: []"

Object[] singleNull = { null };
log("checkout", singleNull); // prints "checkout: [null]"
log("checkout", (Object) null); // prints "checkout: [null]"
log("checkout", null); // params is null – NPE on toString
```

The interviewer's prompt: "Walk through what each call does and where the trap is."

Thinking out loud: 1. "Empty array → passed as the array → length 0. Fine." 2. "No args → varargs is an empty array. Fine." 3. "Array of one null → fine, params is array of length 1." 4. "`(Object) null` — single-element array wrapping null. Fine." 5. "Bare `null` — ambiguous; binds to the array as null. `params == null`."

The actual bug: Bare `null` as the only varargs argument is bound as the array, not a single null element.

The fix (1-3 options if applicable): - **Option A:** Defensive in callee: `if (params == null) params = new Object[0];` . - **Option B:** Cast at call site: `log("checkout", (Object) null);` . - **Option C:** Avoid varargs in APIs where null is a valid value.

What NOT to say: - "Varargs always wraps to non-null" — not for null literal. - "Compiler warns" — sometimes; not always loud enough.

Common follow-up: "What's the compile-time warning for this pattern?"

Scenario 147 · The lambda that throws checked exception

Asked at: Cred · Walmart Labs · **Difficulty:** Medium · **Topic:** Checked exceptions in functional interfaces

The code:

```
List<URL> urls = endpoints.stream()
    .map(e -> new URL(e)) // doesn't compile – MalformedURLException
    .toList();
```

The interviewer's prompt: "map doesn't accept a lambda that throws checked exception. We need to convert strings to URLs. Options?"

Thinking out loud: 1. " `Function<T, R>.apply` doesn't declare throws. Lambdas can't throw checked exceptions through these standard FIs." 2. "Three options: wrap in `RuntimeException`, use a custom FI that declares throws, or refactor to handle exceptions per element." 3. "Common pattern: utility that wraps `Callable`-style with checked throws."

The actual bug: Functional interfaces in `java.util.function` don't declare checked throws. Lambdas with checked exceptions don't compile.

The fix (1-3 options if applicable): - **Option A:** Wrap and rethrow as unchecked: `.map(e -> { try { return new URL(e); } catch (MalformedURLException ex) { throw new RuntimeException(ex); } })`. - **Option B:** Custom FI: `interface ThrowingFunction<T,R> { R apply(T t) throws Exception; }`. Adapter to standard Function wraps the exception. - **Option C:** Don't use stream; old-school for-loop with try-catch.

What NOT to say: - "Java's lambdas auto-catch checked" — they do not. - "Mark `MalformedURLException` as unchecked" — you can't; it's a JDK type.

Common follow-up: "How do libraries like Vavr handle checked exceptions in streams?"

Scenario 148 · The integer parse that returns 0 instead of throwing

Asked at: TCS · Wipro · **Difficulty:** Easy · **Topic:** `NumberFormatException` ignored

The code:

```
public int parseAge(String s) {
    try {
        return Integer.parseInt(s);
    } catch (NumberFormatException e) {
        return 0;
    }
}
```

The interviewer's prompt: "Customer reports age 0 in their profile. Original input was 'twenty-five'. What's the right call?"

Thinking out loud: 1. "Silent default to 0 masks the input error. Customer can't tell why their age is wrong." 2. "Better: throw a user-facing validation error, or return `Optional.empty`, or use `OptionalInt`." 3. "Silent fallbacks hide bugs in domains where the value matters."

The actual bug: Catching and returning a magic default. Caller can't distinguish 'invalid input' from 'genuinely 0'.

The fix (1-3 options if applicable): - **Option A:** Throw a domain validation exception: `throw new ValidationException("age is not a number: " + s);` . - **Option B:** Return `OptionalInt`: `return OptionalInt.empty();` . Caller decides what to do with absence. - **Option C:** Let `NumberFormatException` propagate — caller's controller layer handles to a 400 response.

What NOT to say: - "0 is a safe default" — only if 0 isn't a meaningful value, and even then, hides the input error. - "Log and return -1" — same problem.

Common follow-up: "What's the difference between `Optional` and `OptionalInt` and why both?"

Scenario 149 · The cyclic dependency between Spring beans

Asked at: Walmart Labs · JP Morgan · **Difficulty:** Medium · **Topic:** Constructor cycle in Spring

The code:

```
@Service
class A { A(B b) {} }
@Service
class B { B(A a) {} }
```

The interviewer's prompt: "Spring 2.6+ refuses to start with this. Older versions handled it via setter injection. What's the right design?"

Thinking out loud: 1. "Constructor cycle — A needs B, B needs A. Spring can't construct either first." 2. "Pre-2.6 with setter injection, Spring would inject a proxy or set later. Newer versions disallow by default." 3. "Real fix: break the cycle in the design. Extract shared logic into a third bean."

The actual bug: Cyclic dependency. Indicates a design smell.

The fix (1-3 options if applicable): - **Option A:** Extract shared functionality into a third bean C that A and B both depend on. - **Option B:** Use `@Lazy` on one side to inject a proxy — works but masks the design issue. - **Option C:** Use `ApplicationEventPublisher` to communicate via events — fully decouples.

What NOT to say: - "Just enable cycles in config" — kicks the can; design is still wrong. - "Use field injection" — same cycle problem; only changes how Spring complains.

Common follow-up: "What's `spring.main.allow-circular-references=true` and why is it dangerous?"

Scenario 150 · The Map iteration order that changed

Asked at: TCS · Cognizant · **Difficulty:** Easy · **Topic:** HashMap iteration order not guaranteed

The code:

```
Map<String, Integer> map = new HashMap<>();
map.put("A", 1); map.put("B", 2); map.put("C", 3);
for (var e : map.entrySet()) System.out.println(e); // order varies
```

The interviewer's prompt: "Sometimes prints A B C, sometimes C A B. We rely on order in a downstream test."

Thinking out loud: 1. "HashMap doesn't guarantee iteration order. The hash function and bucket layout determine it." 2. "If you need insertion order, use LinkedHashMap. Sorted? TreeMap." 3. "Tests that depend on HashMap order are flaky."

The actual bug: HashMap iteration order is not specified. Implementations and inputs can change it.

The fix (1-3 options if applicable): - **Option A:** Use `LinkedHashMap` — predictable insertion order. - **Option B:** Use `TreeMap` — sorted by key (or comparator). - **Option C:** In tests, sort the entries before comparing rather than relying on iteration order.

What NOT to say: - "HashMap is alphabetical" — not specified; varies by hash and seed. - "Pin Java version" — order may change between minor versions too.

Common follow-up: "What's the difference between HashMap, LinkedHashMap, and TreeMap in terms of order guarantees?"

Category 2 — Design This

Java Interview Prep — 2026 Edition · Learn & Excel

These are 5-minute whiteboard problems — not full system design. An interviewer asks "design a thread-safe LRU cache" or "design an idempotent payment API" and watches how you decompose the problem out loud. The goal isn't a perfect answer; it's showing you can pick the right data structures, name the tradeoffs, and push back on missing requirements. Every scenario below follows the same shape: how to open, what to sketch, and the follow-up they'll almost certainly throw at you.

Section A — Cache & Memory Design (Scenarios 1–20)

Scenario 1 · Thread-safe LRU cache

Asked at: Razorpay · Cred · Postman · Atlassian GCC · **Difficulty:** Medium · **Topic:** Concurrency + Collections

The interviewer's prompt: "Design a thread-safe LRU cache with $O(1)$ get and put. Capacity is fixed at construction. Assume heavy read traffic."

Thinking out loud (what to say first): 1. "Quick clarification — is this in-process only, or do I need to think about distribution? Also, do I need TTL or just LRU eviction?" 2. "I'll start with the classic `LinkedHashMap` + access-order, then upgrade for concurrency. If contention is high I'd switch to segmented locking or Caffeine's design." 3. "Core data structure is a hash map for $O(1)$ lookup plus a doubly-linked list for $O(1)$ recency updates."

A solid answer (the structure you'd actually whiteboard):

```
public class LruCache<K, V> {
    private final int capacity;
    private final LinkedHashMap<K, V> map;
    private final ReentrantReadWriteLock lock = new ReentrantReadWriteLock();

    public LruCache(int capacity) {
        this.capacity = capacity;
        this.map = new LinkedHashMap<>(capacity, 0.75f, true) {
            @Override protected boolean removeEldestEntry(Map.Entry<K,V> e) {
                return size() > LruCache.this.capacity;
            }
        };
    }

    public V get(K key) {
        lock.writeLock().lock(); // access-order reorders, so write lock
        try { return map.get(key); } finally { lock.writeLock().unlock(); }
    }

    public void put(K key, V val) {
        lock.writeLock().lock();
        try { map.put(key, val); } finally { lock.writeLock().unlock(); }
    }
}
```

- `LinkedHashMap` with `accessOrder=true` reorders on `get()`, so even reads mutate state — read lock won't work
- For high read throughput, partition into N segments by `hash(key) % N` so locks don't serialize the world

Tradeoffs / what you'd push back on: - "Pure LRU is rarely what you want — LFU or W-TinyLFU (Caffeine) handles scan-resistant workloads better" - Single global lock won't scale past a few cores. Either segment or use a lock-free structure (`ConcurrentHashMap` + CLHM-style ring buffer)

What NOT to say: - "Just use `ConcurrentHashMap`" — doesn't give you LRU eviction at all - "Use `Collections.synchronizedMap`" — same global-lock problem and no eviction policy

Common follow-up: "Now add a 5-minute TTL on every entry. How do you handle expiry without a background thread per entry?"

Scenario 2 · LRU cache without `LinkedHashMap`

Asked at: Cred · Flipkart · Walmart Labs · **Difficulty:** Medium · **Topic:** Data structures

The interviewer's prompt: "Build an LRU cache from scratch using only `HashMap` and your own doubly-linked list. No `LinkedHashMap` shortcut."

Thinking out loud: 1. "This is the classic LeetCode 146 — they want to see I understand WHY `LinkedHashMap` works internally." 2. "Node holds key, value, prev, next. `HashMap` maps key → Node. Two sentinel nodes (head/tail) avoid null checks." 3. "On get: move node to head. On put: insert at head, evict tail if over capacity."

A solid answer:

```

class Node<K,V> { K key; V val; Node<K,V> prev, next; }

public class LruCache<K,V> {
    private final Map<K, Node<K,V>> map = new HashMap<>();
    private final Node<K,V> head = new Node<>(), tail = new Node<>();
    private final int cap;

    public LruCache(int cap) {
        this.cap = cap;
        head.next = tail; tail.prev = head;
    }

    public V get(K k) {
        Node<K,V> n = map.get(k);
        if (n == null) return null;
        moveToHead(n);
        return n.val;
    }

    public void put(K k, V v) {
        Node<K,V> n = map.get(k);
        if (n != null) { n.val = v; moveToHead(n); return; }
        n = new Node<>(); n.key = k; n.val = v;
        addToHead(n); map.put(k, n);
        if (map.size() > cap) {
            Node<K,V> evict = tail.prev;
            remove(evict); map.remove(evict.key);
        }
    }
    // remove(n), addToHead(n), moveToHead(n) are 4-line pointer swaps
}

```

- Sentinel head/tail removes 80% of the null-check bugs
- Why store `key` in the Node? Eviction needs it to remove the entry from the map

Tradeoffs: - Not thread-safe — wrapping with synchronized blocks works but kills concurrency - For thread-safe, look at ConcurrentHashMap + a separate eviction queue (Caffeine model)

What NOT to say: - "Use a queue plus a HashMap" — Queue can't remove from middle in $O(1)$; you'd be $O(n)$ on get - "Iterate map.entrySet() to find oldest" — $O(n)$; kills the whole point

Common follow-up: "How would you make this thread-safe without one giant lock?"

Scenario 3 · LFU cache

Asked at: Cred · Netflix GCC · Postman · **Difficulty:** Hard · **Topic:** Data structures

The interviewer's prompt: "Design an LFU cache — evict the least-frequently used entry. $O(1)$ get and put."

Thinking out loud: 1. "Naive: track count per key, scan for min on evict — that's $O(n)$. Need $O(1)$ so I'll group keys by frequency." 2. "Three structures: key→Node, freq→doubly-linked-list-of-Nodes, a `minFreq` pointer." 3. "On access: bump node from freq list F to freq list $F+1$; if F was `minFreq` and is now empty, increment `minFreq`."

A solid answer:

```
public class LfuCache<K,V> {
    private final Map<K, Node<K,V>> keyMap = new HashMap<>();
    private final Map<Integer, LinkedHashSet<Node<K,V>>> freqMap = new HashMap<>();
    private final int cap;
    private int minFreq = 0;

    public V get(K k) {
        Node<K,V> n = keyMap.get(k);
        if (n == null) return null;
        bumpFreq(n);
        return n.val;
    }

    public void put(K k, V v) {
        if (cap == 0) return;
        Node<K,V> n = keyMap.get(k);
        if (n != null) { n.val = v; bumpFreq(n); return; }
        if (keyMap.size() >= cap) evict();
        n = new Node<>(k, v, 1);
        keyMap.put(k, n);
        freqMap.computeIfAbsent(1, x -> new LinkedHashSet<>()).add(n);
        minFreq = 1;
    }

    private void bumpFreq(Node<K,V> n) {
        freqMap.get(n.freq).remove(n);
        if (freqMap.get(n.freq).isEmpty() && n.freq == minFreq) minFreq++;
        n.freq++;
        freqMap.computeIfAbsent(n.freq, x -> new LinkedHashSet<>()).add(n);
    }
}
```

- `LinkedHashSet` gives insertion-order eviction within a freq bucket (ties broken by recency)
- `minFreq` is the only thing that lets eviction stay $O(1)$

Tradeoffs: - Cold-start problem: a new entry has `freq=1` and is the first to be evicted, even if it's about to be heavily used. W-TinyLFU (Caffeine) fixes this with an admission filter. - LFU memorizes old hits forever — once a key has `freq=10000`, it dominates eviction even if no longer relevant. Aging counters fix this.

What NOT to say: - "Just sort entries by frequency on each get" — $O(n \log n)$; fails the $O(1)$ requirement
 - "Use a PriorityQueue keyed by frequency" — heap operations are $O(\log n)$, still not $O(1)$

Common follow-up: "What's the cold-start problem with LFU, and how does TinyLFU solve it?"

Scenario 4 · TTL cache

Asked at: Razorpay · Swiggy · Zomato · **Difficulty:** Medium · **Topic:** Concurrency + scheduling

The interviewer's prompt: "Design a cache where every entry expires after a configurable TTL. No background sweeper thread per entry — that doesn't scale."

Thinking out loud: 1. "Two strategies: lazy expiry on read, or active sweeping. Production caches do both." 2. "Lazy is simple: store `expiresAt` with the value; on get, check and evict if past." 3. "Active sweep avoids memory bloat for entries that are written-but-never-read. Use a single timer thread scanning expiry buckets."

A solid answer:

```
record Entry<V>(V value, long expiresAt) {}

public class TtlCache<K,V> {
    private final ConcurrentHashMap<K, Entry<V>> map = new ConcurrentHashMap<>();
    private final long defaultTtlMs;

    public V get(K k) {
        Entry<V> e = map.get(k);
        if (e == null) return null;
        if (System.currentTimeMillis() > e.expiresAt()) {
            map.remove(k, e); // CAS-style – only remove if still the same entry
            return null;
        }
        return e.value();
    }

    public void put(K k, V v) {
        map.put(k, new Entry<>(v, System.currentTimeMillis() + defaultTtlMs));
    }

    // Sweeper: ScheduledExecutorService runs every 30s
    void sweep() {
        long now = System.currentTimeMillis();
        map.forEach((k, e) -> { if (now > e.expiresAt()) map.remove(k, e); });
    }
}
```

- `map.remove(k, e)` uses the two-arg form — only removes if the value hasn't been replaced by a concurrent put
- Sweep frequency is a tradeoff: too rare = memory bloat, too often = CPU waste

Tradeoffs: - `ConcurrentHashMap.forEach` during sweep doesn't block writes, but iterates a weakly-consistent snapshot — some entries may be missed and caught next sweep. Fine. - For very large caches, bucket entries by expiry time (e.g., per-minute bucket) so sweep only scans relevant buckets

What NOT to say: - "Schedule a Timer task per entry" — N timers for N entries kills the JVM - "Spawn a thread per entry to sleep TTL ms" — same problem, worse

Common follow-up: "Now combine LRU + TTL. Which eviction wins when both apply?"

Scenario 5 · Write-through vs write-back cache

Asked at: Cred · Walmart Labs · Atlassian GCC · **Difficulty:** Medium · **Topic:** Caching strategy

The interviewer's prompt: "Design a cache sitting in front of a database. Compare write-through and write-back. Which would you pick for a payments ledger?"

Thinking out loud: 1. "Write-through: every write goes to DB AND cache synchronously. Write-back: writes go to cache, DB is updated async." 2. "Payments is a consistency-critical domain — write-back risks losing acknowledged writes on crash. I'd go write-through." 3. "Sketch the cache interface, then show how each strategy plugs in."

A solid answer:

```

interface CacheStore<K,V> {
    V get(K k);
    void put(K k, V v);
}

class WriteThroughCache<K,V> implements CacheStore<K,V> {
    private final Map<K,V> cache = new ConcurrentHashMap<>();
    private final Database db;
    public void put(K k, V v) {
        db.write(k, v);          // DB first – if this throws, cache stays clean
        cache.put(k, v);
    }
    public V get(K k) {
        return cache.computeIfAbsent(k, db::read);
    }
}

class WriteBackCache<K,V> implements CacheStore<K,V> {
    private final Map<K,V> cache = new ConcurrentHashMap<>();
    private final BlockingQueue<K> dirty = new LinkedBlockingQueue<>();
    public void put(K k, V v) {
        cache.put(k, v);
        dirty.offer(k);          // background worker drains queue → DB
    }
}

```

- Write-through: stronger durability, higher write latency
- Write-back: lower write latency, batched DB writes, but a crash loses the queue

Tradeoffs: - For payments: pick write-through. Losing a committed payment to a cache crash is unacceptable. - For analytics counters / view counts: write-back is fine — losing a few page views is acceptable - Write-around (write to DB only, populate cache on read) is a third option for write-heavy / read-rare workloads

What NOT to say: - "Write-back is always faster so use it" — speed without durability is dangerous in finance - "Cache and DB will eventually be consistent" — handwaves the real failure modes

Common follow-up: "How do you invalidate the cache when another service writes directly to the DB?"

Scenario 6 · Distributed cache invalidation

Asked at: Razorpay · Swiggy · Meesho · **Difficulty:** Hard · **Topic:** Distributed caching

The interviewer's prompt: "You have 50 app servers each running a local in-memory cache. When data changes, all 50 caches need to know. Design the invalidation."

Thinking out loud: 1. "Two main options: pub/sub broadcast (Redis pub/sub, Kafka), or TTL-only and accept staleness." 2. "For payments-critical data: pub/sub. For displayable data like product price: short TTL is simpler." 3. "I'll sketch a CacheBus interface and a Redis-pubsub implementation."

A solid answer:

```
interface CacheBus {
    void publish(String key);           // tell all nodes to invalidate
    void subscribe(Consumer<String> handler);
}

public class InvalidatingCache<K,V> {
    private final ConcurrentHashMap<String,V> local = new ConcurrentHashMap<>();
    private final CacheBus bus;
    private final Function<String,V> loader;

    public InvalidatingCache(CacheBus bus, Function<String,V> loader) {
        this.bus = bus; this.loader = loader;
        bus.subscribe(local::remove);    // every invalidation = drop local copy
    }

    public V get(String k) { return local.computeIfAbsent(k, loader); }

    public void invalidate(String k) {
        local.remove(k);
        bus.publish(k);                 // broadcast to peers
    }
}
```

- Pub/sub is "at-most-once" by default — message loss = stale cache. Pair with a short TTL as backstop.
- Don't publish the new VALUE on the bus; just the key. Each node re-reads from source of truth (avoids stale-write reorderings)

Tradeoffs: - "Cache stampede" risk when many nodes invalidate simultaneously and all hit DB. Use a per-key lock or single-flight pattern - For cross-region replication, Redis pub/sub doesn't cross regions — use Kafka topics with region-aware consumers

What NOT to say: - "Just sync the caches over HTTP" — N^2 network calls - "Set TTL to 1 second and ignore invalidation" — high DB load + stale-data window

Common follow-up: "What happens during a cache stampede when 50 nodes invalidate at the same instant?"

Scenario 7 · Bloom filter for cache miss avoidance

Asked at: Cred · Postman · Flipkart · **Difficulty:** Medium · **Topic:** Probabilistic data structures

The interviewer's prompt: "Design a layer that prevents cache misses from hitting the database for keys we KNOW don't exist."

Thinking out loud: 1. "This is the classic Bloom filter use case — accept false positives, never false negatives." 2. "Sketch: bit array of size m , k hash functions. $Add(x)$ sets k bits. $MightContain(x)$ checks all k bits." 3. "Size m and number of hashes k come from the formula based on expected n and target false-positive rate p ."

A solid answer:

```
public class BloomFilter<T> {
    private final BitSet bits;
    private final int size;
    private final int hashCount;
    private final Function<T, int[]> hasher;    // k hash values

    public BloomFilter(int n, double p) {
        this.size = (int) Math.ceil(-n * Math.log(p) / (Math.log(2) * Math.log(2)));
        this.hashCount = (int) Math.ceil(size * Math.log(2) / n);
        this.bits = new BitSet(size);
        this.hasher = buildHasher(hashCount, size);
    }

    public void add(T item) {
        for (int h : hasher.apply(item)) bits.set(Math.abs(h % size));
    }

    public boolean mightContain(T item) {
        for (int h : hasher.apply(item))
            if (!bits.get(Math.abs(h % size))) return false;
        return true;    // could be false positive
    }
}

// Usage in front of cache:
public V get(K key) {
    if (!bloom.mightContain(key)) return null;    // definitely not in store
    V v = cache.get(key);
    return v != null ? v : loadFromDb(key);
}
```

- For $n=1M$ and $p=1\%$, you get ~ 9.6 bits per element — under 2MB total
- k hash functions are usually two independent hashes combined as $h1 + i \cdot h2$ (double hashing)

Tradeoffs: - Can't delete entries (would clear bits used by other keys). Use Counting Bloom or rebuild periodically. - Choose p carefully: too aggressive ($p=0.001$) makes the filter huge; too loose ($p=0.1$) means 10% of misses still hit DB

What NOT to say: - "Bloom filters give exact membership" — wrong, they give "definitely not" or "maybe" - "Just cache the negatives" — works but unbounded memory; Bloom gives fixed-memory approximation

Common follow-up: "How would you delete from a Bloom filter? When would you rebuild it?"

Scenario 8 · Object pool

Asked at: Razorpay · Postman · Apache projects · **Difficulty:** Medium · **Topic:** Resource management

The interviewer's prompt: "Design a generic object pool — clients borrow, use, return. Pool has min/max size and idle eviction."

Thinking out loud: 1. "Classic example is DB connection pool (HikariCP). But the design generalizes." 2. "Need a thread-safe queue of idle objects, a factory to create new ones, and tracking of in-use count." 3. "Borrow blocks if pool is at max and none idle; idle eviction runs on a timer."

A solid answer:

```

public class ObjectPool<T> {
    private final BlockingQueue<T> idle;
    private final Supplier<T> factory;
    private final Consumer<T> destroyer;
    private final Semaphore permits;    // caps total in-flight

    public ObjectPool(int max, Supplier<T> factory, Consumer<T> destroyer) {
        this.idle = new LinkedBlockingQueue<>(max);
        this.permits = new Semaphore(max);
        this.factory = factory;
        this.destroyer = destroyer;
    }

    public T borrow(Duration timeout) throws InterruptedException {
        if (!permits.tryAcquire(timeout.toMillis(), TimeUnit.MILLISECONDS))
            throw new TimeoutException();
        T t = idle.poll();
        return t != null ? t : factory.get();
    }

    public void release(T t) {
        if (!idle.offer(t)) destroyer.accept(t);
        permits.release();
    }
}

```

- Semaphore caps total in-flight (idle + borrowed); BlockingQueue holds only idle
- On release: if queue is full (we have more idle than needed), destroy the object

Tradeoffs: - Validation on borrow (e.g., `Connection.isValid()`) costs latency but catches stale objects - "Slow leak" if a borrower forgets to release — track borrow time and log warnings past a threshold - For DB connections specifically, use HikariCP — it handles edge cases (network blip mid-query, etc.)

What NOT to say: - "Use Stack for the idle list" — not thread-safe, and Stack is legacy - "Synchronize the whole pool on one lock" — kills concurrency under high load

Common follow-up: "A borrower never returns the object — how does the pool detect and recover?"

Scenario 9 · Connection pool sizing

Asked at: Razorpay · Cred · most backend roles · **Difficulty:** Medium · **Topic:** Resource sizing

The interviewer's prompt: "How do you decide pool size for a JDBC connection pool? Walk me through the math."

Thinking out loud: 1. "Common myth: more connections = more throughput. False — past a point, DB CPU thrashes." 2. "HikariCP guide recommends: `connections = ((core_count * 2) + effective_spindle_count)` . For SSDs assume spindle=1." 3. "For an 8-core DB with SSDs, ~17 connections per app instance is a starting point."

A solid answer: - Default: `pool_size = (cpu_cores * 2) + 1` per app instance - If you have 10 app pods talking to one DB, total connections to DB = 10 × pool_size — keep this under DB's max_connections - Set short connection timeout (`connectionTimeout=2000ms`) and short idle timeout (`idleTimeout=600000ms`)

```
HikariConfig cfg = new HikariConfig();
cfg.setJdbcUrl("jdbc:postgresql://...");
cfg.setMaximumPoolSize(17);
cfg.setMinimumIdle(5);
cfg.setConnectionTimeout(2000);
cfg.setIdleTimeout(600_000);
cfg.setMaxLifetime(1_800_000); // recycle every 30 min; avoids stale conns
HikariDataSource ds = new HikariDataSource(cfg);
```

- `maxLifetime` < load balancer / DB-side connection idle timeout, otherwise you get RST surprises

Tradeoffs: - Larger pool = more memory at app + DB, more risk of starving the DB CPU - Smaller pool = requests queue inside the app; tune `connectionTimeout` to fail fast rather than block forever - For multi-region: each region needs its own pool to its regional read replica

What NOT to say: - "Set max to 100 to be safe" — almost always wrong; DB CPU will saturate at ~2x cores - "More connections = more throughput" — false past saturation

Common follow-up: "What's `maxLifetime` and why isn't infinity?"

Scenario 10 · Weak / soft reference cache

Asked at: Postman · Atlassian GCC · Oracle GCC · **Difficulty:** Medium · **Topic:** JVM internals

The interviewer's prompt: "Design a cache that releases entries when the JVM needs memory — no fixed capacity."

Thinking out loud: 1. "This is `SoftReference` / `WeakReference` territory. Soft refs are cleared only under memory pressure; weak refs are cleared at next GC." 2. "For a cache where 'I'd LIKE to keep this if I can', `SoftReference` is the fit." 3. "Wrap values in `SoftReference`, use a `ConcurrentHashMap`, and have a small reaper that clears dead entries."

A solid answer:

```
public class SoftCache<K,V> {
    private final ConcurrentHashMap<K, SoftReference<V>> map = new ConcurrentHashMap<>();
    private final ReferenceQueue<V> deadQueue = new ReferenceQueue<>();
    private final ConcurrentHashMap<Reference<? extends V>, K> reverse = new ConcurrentHashMap<>();

    public V get(K k) {
        SoftReference<V> ref = map.get(k);
        return ref == null ? null : ref.get(); // may be null if GC'd
    }

    public void put(K k, V v) {
        SoftReference<V> ref = new SoftReference<>(v, deadQueue);
        map.put(k, ref);
        reverse.put(ref, k);
    }

    public void reap() {
        Reference<? extends V> r;
        while ((r = deadQueue.poll()) != null) {
            K k = reverse.remove(r);
            if (k != null) map.remove(k, r);
        }
    }
}
```

- ReferenceQueue receives the wrapper after GC clears its referent — that's how you find which keys to clean up
- Without the reap loop, the map fills with **SoftReference** wrappers whose contents are gone

Tradeoffs: - SoftReference behavior is JVM-implementation-specific; you give up precise control over eviction - For business-critical data, prefer explicit LRU with bounded size — soft caches make tail latency unpredictable

What NOT to say: - "Use WeakHashMap so entries get GC'd" — weak refs go on the FIRST GC; too aggressive for a cache - "Just rely on -Xmx tuning" — doesn't actually evict anything, just OOMs

Common follow-up: "What's the difference between SoftReference, WeakReference, and PhantomReference?"

Scenario 11 · Two-level cache (L1 local + L2 Redis)

Asked at: Razorpay · Swiggy · Meesho · **Difficulty:** Hard · **Topic:** Cache hierarchy

The interviewer's prompt: "Design a two-tier cache: in-process L1 for hot data, Redis as L2 for shared data. How do they coordinate?"

Thinking out loud: 1. "L1 is fast (~nanoseconds) but per-node. L2 is shared but ~1ms round-trip. Read path goes L1 → L2 → DB." 2. "Write path: write to DB, invalidate L2, broadcast invalidation to all L1s." 3. "Each L1 needs its own short TTL as a backstop in case the invalidation bus drops a message."

A solid answer:

```
public class TwoTierCache<K,V> {
    private final Cache<K,V> l1;           // Caffeine, max=10k, TTL=60s
    private final RedisClient l2;
    private final InvalidationBus bus;     // Redis pub/sub

    public V get(K k) {
        V v = l1.get(k);
        if (v != null) return v;
        v = l2.get(k);
        if (v != null) { l1.put(k, v); return v; }
        v = loadFromDb(k);
        l2.put(k, v); l1.put(k, v);
        return v;
    }

    public void put(K k, V v) {
        writeDb(k, v);
        l2.del(k);
        bus.publish(k);                    // every node drops L1[k]
    }
}
```

- L1 TTL is short (seconds) so even if invalidation is lost, drift is bounded
- L2 is the "source of cached truth" — invalidation drops L2, then L1, and next read repopulates

Tradeoffs: - L1 hit means stale-window risk; for STRICT consistency, skip L1 for that read - L2 down? Fail-open to DB but tighten rate limit; fail-closed risks cascading 500s - Memory per L1 × N nodes can dwarf L2 — watch heap

What NOT to say: - "Just use Redis, skip L1" — wastes network on every cache hit - "Have L1 write to L2 in background" — write ordering becomes a nightmare

Common follow-up: "What if Redis pub/sub drops a message — how do you recover?"

Scenario 12 · Cache stampede prevention

Asked at: Razorpay · Cred · Postman · **Difficulty:** Hard · **Topic:** Concurrency

The interviewer's prompt: "When a hot key expires, 1000 requests miss the cache and all hit the DB simultaneously. Design a fix."

Thinking out loud: 1. "This is cache stampede / thundering herd. Three fixes: single-flight, probabilistic early refresh, or lock + serve-stale." 2. "Single-flight is the cleanest: only one thread loads, others wait for that result." 3. "Sketch using a ConcurrentHashMap."

A solid answer:

```
public class SingleFlightCache<K,V> {
    private final Cache<K,V> cache;
    private final ConcurrentHashMap<K, CompletableFuture<V>> inFlight = new ConcurrentHashMap<>();
    private final Function<K,V> loader;

    public V get(K k) throws Exception {
        V v = cache.get(k);
        if (v != null) return v;

        CompletableFuture<V> mine = new CompletableFuture<>();
        CompletableFuture<V> winner = inFlight.putIfAbsent(k, mine);
        if (winner != null) return winner.get();    // someone else is loading

        try {
            V loaded = loader.apply(k);
            cache.put(k, loaded);
            mine.complete(loaded);
            return loaded;
        } catch (Exception e) {
            mine.completeExceptionally(e);
            throw e;
        } finally {
            inFlight.remove(k, mine);
        }
    }
}
```

- `putIfAbsent` is the atomic election — first thread wins, others share its future
- All losers block on `winner.get()` — no extra DB load

Tradeoffs: - If the loader hangs, all waiters hang. Add a per-load timeout. - For multi-node single-flight, use a distributed lock (Redis SETNX) — but that adds a network hop per miss

What NOT to say: - "Lock the cache during load" — serializes all keys, not just the hot one - "Set longer TTL" — pushes the problem out but doesn't fix it

Common follow-up: "How would you do this across 50 nodes, not just within one JVM?"

Scenario 13 · Counting Bloom filter

Asked at: Cred · Postman · Atlassian GCC · **Difficulty:** Hard · **Topic:** Probabilistic data structures

The interviewer's prompt: "Standard Bloom filter can't delete. Design one that can."

Thinking out loud: 1. "Replace each bit with a small counter (4 bits). Add increments, delete decrements." 2. "Counter overflow is rare with $k=7$ hashes — but mention it as a known limit." 3. "4x memory cost over a regular Bloom — that's the price of delete."

A solid answer:

```
public class CountingBloom<T> {
    private final byte[] counters; // 4-bit counters packed into bytes (or just use byte)
    private final int size;
    private final int hashCount;

    public void add(T item) {
        for (int h : hashes(item)) {
            int idx = Math.abs(h % size);
            if (counters[idx] < 15) counters[idx]++; // saturate at 15
        }
    }

    public void remove(T item) {
        for (int h : hashes(item)) {
            int idx = Math.abs(h % size);
            if (counters[idx] > 0) counters[idx]--;
        }
    }

    public boolean mightContain(T item) {
        for (int h : hashes(item))
            if (counters[Math.abs(h % size)] == 0) return false;
        return true;
    }
}
```

- Saturating counters: once at 15, don't increment further. Decrement-from-15 is then unsafe — accept this as a leak
- For exact semantics, use a Cuckoo Filter — supports delete AND uses less space at low FP rates

Tradeoffs: - 4 bits per cell uses 4x memory vs Bloom — for $n=10M$, that's 5MB vs 1.25MB - If your workload allows it, Cuckoo Filter is strictly better

What NOT to say: - "Use 1-bit counters" — that's just a regular Bloom filter - "Rebuild Bloom on every delete" — $O(n)$ per delete is unacceptable at scale

Common follow-up: "Cuckoo Filter vs Counting Bloom — when is each better?"

Scenario 14 · Disk-backed cache with mmap

Asked at: Cred · Druva · Helpshift · **Difficulty:** Hard · **Topic:** I/O + Memory

The interviewer's prompt: "Design a cache that exceeds heap size — billions of small entries on a single node."

Thinking out loud: 1. "Heap can't hold this. Options: off-heap (DirectByteBuffer), memory-mapped files (MappedByteBuffer), or embedded KV store (RocksDB, MapDB)." 2. "For pure cache (no durability), off-heap is cleanest. For durability + size, mmap or RocksDB." 3. "Sketch a simple mmap-backed cache with a fixed-size slab allocator."

A solid answer:

```

public class MmapCache {
    private final MappedByteBuffer buf;
    private final ConcurrentHashMap<String, int[]> index; // key → [offset, length]
    private final AtomicInteger nextOffset = new AtomicInteger();

    public MmapCache(Path file, long sizeBytes) throws IOException {
        FileChannel ch = FileChannel.open(file, READ, WRITE, CREATE);
        this.buf = ch.map(MapMode.READ_WRITE, 0, sizeBytes);
        this.index = new ConcurrentHashMap<>();
    }

    public void put(String k, byte[] v) {
        int off = nextOffset.getAndAdd(v.length);
        synchronized (buf) { // single-writer per region
            buf.position(off);
            buf.put(v);
        }
        index.put(k, new int[]{off, v.length});
    }

    public byte[] get(String k) {
        int[] loc = index.get(k);
        if (loc == null) return null;
        byte[] out = new byte[loc[1]];
        ByteBuffer dup = buf.duplicate();
        dup.position(loc[0]); dup.get(out);
        return out;
    }
}

```

- *mmap lets the OS page cache do the hard work; data lives outside the JVM heap*
- *For production, prefer Chronicle Map or RocksDB — they handle compaction, fragmentation, and crashes*

Tradeoffs: - *This sketch leaks space on overwrite — production code needs a slab/log-structured allocator - mmap has page-fault latency spikes — for predictable p99, plain RAM is better*

What NOT to say: - *"Just increase -Xmx" — past ~32GB you lose compressed oops; GC pauses balloon*
 - *"Use Redis on the same box" — adds IPC, doesn't help much*

Common follow-up: *"How does RocksDB's LSM tree compare to mmap for cache use?"*

Scenario 15 · Read-through cache abstraction

Asked at: Razorpay · Postman · Walmart Labs · **Difficulty:** Medium · **Topic:** API design

The interviewer's prompt: "Design a cache that takes a loader function and never returns null — always populates on miss."

Thinking out loud: 1. "This is Caffeine's LoadingCache model. The loader is part of the cache's construction." 2. "Key methods: `get(K)` blocks if not present, calls loader. `refresh(K)` reloads in background." 3. "Crucial detail: handle loader failures — cache exceptions briefly or rethrow each time?"

A solid answer:

```
public interface LoadingCache<K,V> {
    V get(K k) throws ExecutionException;
    CompletableFuture<V> refresh(K k);
    void invalidate(K k);
}

public class SimpleLoadingCache<K,V> implements LoadingCache<K,V> {
    private final Map<K, V> store = new ConcurrentHashMap<>();
    private final Function<K, V> loader;
    private final ConcurrentHashMap<K, CompletableFuture<V>> inFlight = new ConcurrentHashMap<>();

    public V get(K k) {
        V v = store.get(k);
        if (v != null) return v;
        return inFlight.computeIfAbsent(k, key ->
            CompletableFuture.supplyAsync(() -> {
                V loaded = loader.apply(key);
                store.put(key, loaded);
                inFlight.remove(key);
                return loaded;
            })).join();
    }
}
```

- `computeIfAbsent` gives atomic "compute it once" semantics
- For an exception, the future completes exceptionally — callers see the error, then the next call retries

Tradeoffs: - "Cache exceptions" — do you cache a failure for N seconds to prevent retry storms? Yes for transient errors; no for "key doesn't exist" - Sync get blocks; offer an async variant for non-blocking I/O code

What NOT to say: - "Return null on miss, let the caller decide" — that's just a regular cache, not read-through - "Use synchronized inside get()" — kills concurrency across keys

Common follow-up: "What about negative caching — when a key truly doesn't exist?"

Scenario 16 · Refresh-ahead cache

Asked at: Cred · Atlassian GCC · Adobe GCC · **Difficulty:** Medium · **Topic:** Caching strategy

The interviewer's prompt: "Don't wait for cache miss to reload — refresh hot entries before they expire."

Thinking out loud: 1. "This is refresh-ahead: when an entry is X% through its TTL, kick off async reload while serving the old value." 2. "Caffeine has this with `refreshAfterWrite`. I'll sketch the policy." 3. "Tradeoff: more DB load (preemptive refresh of unused entries) but zero latency spikes on miss."

A solid answer:

```
public class RefreshAheadCache<K,V> {
    record Entry<V>(V value, long writtenAt) {}
    private final Map<K, Entry<V>> cache = new ConcurrentHashMap<>();
    private final long refreshAfterMs;
    private final Function<K,V> loader;
    private final ExecutorService refreshPool = Executors.newSingleThreadExecutor();

    public V get(K k) {
        Entry<V> e = cache.get(k);
        if (e == null) return loadSync(k);
        if (System.currentTimeMillis() - e.writtenAt() > refreshAfterMs)
            refreshPool.submit(() -> cache.put(k, new Entry<>(loader.apply(k),
                System.currentTimeMillis())));
        return e.value(); // serve possibly-stale value immediately
    }
}
```

- Serve-stale-while-revalidate: caller never blocks on a refresh
- Use a bounded pool — unbounded async refresh on every read floods the DB

Tradeoffs: - For rarely-read keys, you refresh entries no one wants. Pair with a hit-counter to only refresh "frequently accessed" - "Stale value served" — fine for most read-heavy domains (product catalog, settings); not fine for ledgers

What NOT to say: - "Always reload synchronously on expiry" — that's just a TTL cache; reintroduces latency spikes - "Refresh every key every minute" — load on DB scales with cache size, not access pattern

Common follow-up: "What if the async refresh fails — do you serve the stale value forever?"

Scenario 17 · Negative caching

Asked at: Razorpay · Swiggy · Postman · **Difficulty:** Easy · **Topic:** Caching strategy

The interviewer's prompt: "Users keep searching for product IDs that don't exist. Each miss hits the DB. Fix it."

Thinking out loud: 1. "Cache the 'not found' result, not just successful lookups." 2. "Sentinel value or wrap in Optional. TTL must be short — entities can be created later." 3. "For high-cardinality 'missing' keys, pair with a Bloom filter."

A solid answer:

```
private static final Object MISSING = new Object();

public Product get(String id) {
    Object v = cache.get(id);
    if (v == MISSING) return null;           // negative cache hit
    if (v != null) return (Product) v;
    Product p = db.findById(id);
    cache.put(id, p != null ? p : MISSING, p != null ? Duration.ofMinutes(10) : Duration.ofSeconds(10));
    return p;
}
```

- Sentinel `MISSING` instead of `Optional` saves allocation per call
- Negative TTL much shorter than positive TTL — entity creation should be visible quickly

Tradeoffs: - "Cache the null itself" requires Map that allows null values. ConcurrentHashMap doesn't. - Without short TTL, you'll cache misses for new entities indefinitely — user reports "I created the product but search says it doesn't exist"

What NOT to say: - "Don't cache misses, they're rare" — they're often the majority of traffic in search - "Use long TTL on misses to save DB" — breaks new-entity visibility

Common follow-up: "What's the right TTL for a negative cache entry — and why?"

Scenario 18 · Cache key design

Asked at: Cred · Razorpay · Postman · **Difficulty:** Medium · **Topic:** API design

The interviewer's prompt: "Design a cache key strategy for `getOrdersForUser(userId, status, dateRange)` . Multiple parameters, some optional."

Thinking out loud: 1. "Cache keys must be deterministic, collision-free across param combos, and human-debuggable." 2. "Don't just `toString()` the args — null vs missing matters, order matters, types matter." 3. "I'd build a typed key class or a canonicalized string with namespace prefix."

A solid answer:

```
record OrdersKey(String userId, OrderStatus status, LocalDate from, LocalDate to) {
    String toCacheKey() {
        return "orders:v1:" + userId + ":" + (status == null ? "ALL" : status) +
            ":" + (from == null ? "*" : from) + ":" + (to == null ? "*" : to);
    }
}

public List<Order> getOrders(String userId, OrderStatus status, LocalDate from, LocalDate to) {
    String key = new OrdersKey(userId, status, from, to).toCacheKey();
    return cache.get(key, () -> repo.find(userId, status, from, to));
}
```

- `v1` prefix lets you bump schema and bypass old entries
- Use a record so `equals/hashCode` are correct if you key the cache on the object itself

Tradeoffs: - Cardinality explosion: every parameter combo is a separate key. For high-cardinality date ranges, this fills the cache with one-shot entries — consider rounding `from / to` to day boundaries - Hash the key if it's long, but keep the prefix readable for debugging

What NOT to say: - "Just concat args with a separator" — null vs "null" string collisions - "Use SHA-256 of args" — unreadable in logs; harder to debug

Common follow-up: "How do you invalidate ALL keys for a user when their data changes?"

Scenario 19 · Sharded in-memory cache

Asked at: Cred · Postman · Atlassian GCC · **Difficulty:** Medium · **Topic:** Concurrency

The interviewer's prompt: "Single ConcurrentHashMap bottlenecks at 32 cores. Design a sharded cache that scales to 64+."

Thinking out loud: 1. "ConcurrentHashMap is already segmented internally, but for VERY high contention, an outer sharding layer helps." 2. "Each shard is a separate cache instance — separate eviction, separate lock, separate stats." 3. "Pick shard by `hash(key) % N` — $N = \text{power of } 2$ lets you use bitmask for speed."

A solid answer:

```
public class ShardedCache<K,V> {
    private final Cache<K,V>[] shards;
    private final int mask;

    @SuppressWarnings("unchecked")
    public ShardedCache(int numShards, int capacityPerShard) {
        int n = Integer.highestOneBit(numShards - 1) << 1; // round to power of 2
        this.shards = new Cache[n];
        this.mask = n - 1;
        for (int i = 0; i < n; i++) shards[i] = new LruCache<>(capacityPerShard);
    }

    public V get(K k) { return shards[k.hashCode() & mask].get(k); }
    public void put(K k, V v) { shards[k.hashCode() & mask].put(k, v); }
}
```

- `& mask` instead of `% n` — single CPU instruction; only works if `n` is a power of 2
- Each shard has its own eviction → uneven shard load can cause imbalance

Tradeoffs: - More shards = less contention but more memory overhead per shard - Hot key still hammers one shard — sharding helps for random access, not for skewed hot-key workloads - For hot-key skew, consider striped LongAdder-style or consistent hashing with virtual nodes

What NOT to say: - "Use a single synchronized HashMap" — exact opposite direction - "Shard count = number of cores" — usually overkill; 16 shards covers most workloads

Common follow-up: "What if one key has 90% of the traffic — sharding doesn't help. What now?"

Scenario 20 · Cache warming on deploy

Asked at: Razorpay · Cred · Swiggy · **Difficulty:** Medium · **Topic:** Operational design

The interviewer's prompt: "After deploy, new pod has cold cache. First N requests are slow and DB CPU spikes. Design warming."

Thinking out loud: 1. "Three strategies: load top-K from DB on startup, shadow-traffic the new pod, or stream the old pod's cache." 2. "For most apps, top-K on startup is simplest. Background-load before joining the load balancer." 3. "Sketch a startup hook that loads from a 'hot keys' table and only marks ready when done."

A solid answer:

```
@PostConstruct
public void warmCache() {
    List<String> hotKeys = repo.findTop1000HotKeys(); // tracked via access logs
    ExecutorService loader = Executors.newFixedThreadPool(10);
    CompletableFuture<?>[] futures = hotKeys.stream()
        .map(k -> CompletableFuture.runAsync(() -> cache.put(k, db.load(k)), loader))
        .toArray(CompletableFuture[]::new);
    CompletableFuture.allOf(futures).join();
    readiness.markReady(); // K8s readiness probe flips green here
}
```

- Readiness probe stays red until warming completes → load balancer doesn't route to cold pod
- Hot-keys list comes from previous pods' access logs / Redis ZADD counter

Tradeoffs: - Slow startup: warming top-1000 might take 30s. K8s rollingUpdate strategy needs `progressDeadlineSeconds` tuned - "Cold pod shedding" alternative: don't warm; let the new pod take 1% of traffic initially, ramp up - Warming wrong keys (top-1000 from yesterday, not today) wastes time. Recompute the hot set hourly

What NOT to say: - "Let the cache warm naturally" — DB CPU spike kicks in immediately under real traffic - "Pre-load every entry at startup" — fine for small caches, infeasible past a few MB

Common follow-up: "How do you identify the 'hot 1000 keys' to warm in the first place?"

Section B — Rate Limiting & Throttling (Scenarios 21–35)

Scenario 21 · Token bucket rate limiter

Asked at: Razorpay · Cred · Postman · most fintech · **Difficulty:** Medium · **Topic:** Rate limiting

The interviewer's prompt: "Design a rate limiter that allows 100 requests per second per user, with burst up to 200."

Thinking out loud (what to say first): 1. "Three classic algorithms: token bucket (allows burst), leaky bucket (smooths to constant rate), sliding window. Burst requirement → token bucket." 2. "Bucket holds tokens; refill at 100/sec; capacity = 200. Each request takes one token." 3. "Per-user state: tokens + lastRefillTimestamp. Refill is computed lazily on each request — no background thread."

A solid answer (the structure you'd actually whiteboard):

```

public class TokenBucket {
    private final long capacity;
    private final double tokensPerSecond;
    private double tokens;
    private long lastRefill;

    public TokenBucket(long capacity, double tokensPerSecond) {
        this.capacity = capacity;
        this.tokensPerSecond = tokensPerSecond;
        this.tokens = capacity;
        this.lastRefill = System.nanoTime();
    }

    public synchronized boolean tryConsume() {
        refill();
        if (tokens < 1) return false;
        tokens--;
        return true;
    }

    private void refill() {
        long now = System.nanoTime();
        double elapsed = (now - lastRefill) / 1e9;
        tokens = Math.min(capacity, tokens + elapsed * tokensPerSecond);
        lastRefill = now;
    }
}

```

- Lazy refill: no scheduled thread, math is done on the request path
- Per user: `ConcurrentHashMap<UserId, TokenBucket>` keyed by user

Tradeoffs / what you'd push back on: - `synchronized` per bucket is fine until very high QPS per user. For million-QPS per key, use atomic CAS loops - Distributed case (multiple app pods): centralize in Redis with Lua script for atomicity

What NOT to say: - "Use Guava's RateLimiter" — Guava's is process-local; doesn't solve distributed - "Just count requests in last second" — fixed-window has boundary spike (200 at second-end + 200 at second-start = 400 in 100ms)

Common follow-up: "Now distribute it across 50 pods sharing the limit."

Scenario 22 · Distributed rate limiter with Redis

Asked at: Razorpay · Cred · PhonePe · **Difficulty:** Hard · **Topic:** Distributed systems

The interviewer's prompt: "50 app pods serve one user's traffic. Limit per user is 100 QPS across the fleet. Design it."

Thinking out loud: 1. "Local token buckets won't work — each pod allows its own 100 QPS, total = 5000." 2. "Centralize the state in Redis. Use Lua script to make refill+consume atomic in one round-trip." 3. "Each request: one Redis call, single-digit ms latency. Tradeoff vs local: extra hop."

A solid answer:

```
-- token_bucket.lua: KEYS[1] = bucket key, ARGV = {capacity, rate, now}
local data = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens = tonumber(data[1]) or tonumber(ARGV[1])
local ts = tonumber(data[2]) or tonumber(ARGV[3])
local elapsed = (tonumber(ARGV[3]) - ts) / 1000
tokens = math.min(tonumber(ARGV[1]), tokens + elapsed * tonumber(ARGV[2]))
if tokens < 1 then
    redis.call('HMSET', KEYS[1], 'tokens', tokens, 'ts', ARGV[3])
    return 0
end
tokens = tokens - 1
redis.call('HMSET', KEYS[1], 'tokens', tokens, 'ts', ARGV[3])
redis.call('EXPIRE', KEYS[1], 3600)
return 1
```

```
public boolean allow(String userId) {
    Long ok = (Long) redis.eval(LUA_SCRIPT,
        List.of("rl:" + userId),
        List.of("200", "100", String.valueOf(System.currentTimeMillis())));
    return ok == 1L;
}
```

- Lua runs atomically inside Redis — no race between read and write
- EXPIRE garbage-collects inactive users automatically

Tradeoffs: - Adds a Redis round-trip per request — 1-2ms. For 100k QPS that's serious load on Redis. - Failure mode: Redis down → fail open (allow) or fail closed (deny)? Most fintech picks fail-open with logging - For ultra-high QPS, batch with sliding-window approximation locally + periodic sync to Redis

What NOT to say: - "Use Redis INCR with TTL" — that's fixed-window, has boundary-spike problem - "Have one pod be the leader" — single point of failure

Common follow-up: "Redis itself becomes the bottleneck — how do you scale it?"

Scenario 23 · Sliding window log rate limiter

Asked at: Razorpay · Postman · Atlassian GCC · **Difficulty:** Medium · **Topic:** Rate limiting

The interviewer's prompt: "Allow exactly 100 requests in any 60-second sliding window. No fixed-window boundary spikes."

Thinking out loud: 1. "Sliding window log: store timestamps of each request, on every check evict old, count remaining." 2. "Per user: a Deque of timestamps. $O(N)$ per request in worst case but N is small (window size = 100)." 3. "Memory cost: 100 longs $\times$ N users. For $N=10M$, that's 8GB — too much. Mention sliding window counter as the production fix."

A solid answer:

```
public class SlidingWindowLogLimiter {
    private final int limit;
    private final long windowMs;
    private final Map<String, Deque<Long>> requests = new ConcurrentHashMap<>();

    public boolean allow(String userId) {
        Deque<Long> log = requests.computeIfAbsent(userId, k -> new ArrayDeque<>());
        synchronized (log) {
            long now = System.currentTimeMillis();
            long cutoff = now - windowMs;
            while (!log.isEmpty() && log.peekFirst() < cutoff) log.pollFirst();
            if (log.size() >= limit) return false;
            log.addLast(now);
            return true;
        }
    }
}
```

- Per-user **synchronized** block keeps it thread-safe with no global contention
- Precise — counts the exact number of hits in the last windowMs

Tradeoffs: - Memory: $O(\text{limit})$ per user. For 10M users with $\text{limit}=100$, that's 8GB just of longs - Sliding window counter approximates with much less memory: track count in current bucket + (% into bucket) $\times$ previous bucket count

What NOT to say: - "Just truncate the deque on every N th request" — defeats the precision the algorithm is for - "Use a TreeSet of timestamps" — $\log N$ insert/remove; deque is $O(1)$ amortized

Common follow-up: "How would the sliding window COUNTER approximation work, and what's the tradeoff?"

Scenario 24 · Leaky bucket

Asked at: Razorpay · PhonePe · Walmart Labs · **Difficulty:** Medium · **Topic:** Rate limiting

The interviewer's prompt: "Smooth out bursty traffic to a constant downstream rate. Design a leaky bucket."

Thinking out loud: 1. "Token bucket allows bursts. Leaky bucket doesn't — it processes at a constant rate, queueing excess." 2. "Implemented as a FIFO queue with a single-thread drainer that pops one item every $1/\text{rate}$ seconds." 3. "If queue is full, reject (drop) or block."

A solid answer:

```
public class LeakyBucket {
    private final BlockingQueue<Runnable> queue;
    private final ScheduledExecutorService leaker = Executors.newSingleThreadScheduledExecutor();

    public LeakyBucket(int capacity, double ratePerSecond) {
        this.queue = new ArrayBlockingQueue<>(capacity);
        long periodMs = (long) (1000.0 / ratePerSecond);
        leaker.scheduleAtFixedRate(() -> {
            Runnable task = queue.poll();
            if (task != null) task.run();
        }, 0, periodMs, TimeUnit.MILLISECONDS);
    }

    public boolean submit(Runnable task) {
        return queue.offer(task); // false if full → caller backs off
    }
}
```

- Constant downstream rate — perfect for a third-party API with strict per-second limits
- Queue depth controls how much burst you absorb

Tradeoffs: - Token bucket vs leaky bucket: token allows bursts but downstream sees them; leaky smooths but adds latency - Single-thread leaker doesn't scale past ~10k ops/sec — for higher, use a token-bucket pattern

What NOT to say: - "Same as token bucket" — opposite intent; token allows bursts, leaky forbids them - "Drop everything when full" — leaky usually queues; rejection is a configuration choice

Common follow-up: "When would you pick leaky bucket over token bucket?"

Scenario 25 · Tiered rate limits

Asked at: Razorpay · Cred · Stripe-style fintech · **Difficulty:** Medium · **Topic:** Product design

The interviewer's prompt: "Free users: 100 req/hour. Paid: 10,000 req/hour. Enterprise: unlimited. Design the system."

Thinking out loud: 1. "Per-user limit depends on plan. Don't hardcode — fetch plan, pick policy." 2. "Cache user → plan mapping (changes rarely). Apply rate limit at API gateway, not in each service." 3. "Counter/state can be Redis token bucket with plan-driven config."

A solid answer:

```
record RateLimitPolicy(long capacity, double tokensPerSecond) {}

public class TieredLimiter {
    private final Map<Plan, RateLimitPolicy> policies = Map.of(
        Plan.FREE, new RateLimitPolicy(100, 100.0/3600),
        Plan.PAID, new RateLimitPolicy(10_000, 10_000.0/3600),
        Plan.ENTERPRISE, new RateLimitPolicy(Long.MAX_VALUE, Double.MAX_VALUE)
    );
    private final UserPlanCache planCache;
    private final RedisTokenBucket bucket;

    public boolean allow(String userId) {
        Plan plan = planCache.getPlan(userId);
        if (plan == Plan.ENTERPRISE) return true;
        RateLimitPolicy p = policies.get(plan);
        return bucket.tryConsume(userId, p);
    }
}
```

- Plan cache short TTL (60s) so upgrades take effect quickly
- ENTERPRISE short-circuits — no Redis call needed

Tradeoffs: - Plan change during burst: user upgrades mid-throttle. Wait for cache TTL (acceptable) or invalidate on plan change - For multi-region with regional Redis, an Enterprise customer on Mumbai pod gets unlimited locally but other regions track separately

What NOT to say: - "Use 3 separate rate limiters" — same logic, repeated - "Make Enterprise share a global pool" — usually they want no limit at all, not a shared one

Common follow-up: "How do you handle a plan downgrade — should current burst capacity carry over?"

Scenario 26 · Concurrent request limit (in-flight cap)

Asked at: Razorpay · Postman · Atlassian GCC · **Difficulty:** Medium · **Topic:** Backpressure

The interviewer's prompt: "Limit each user to 10 IN-FLIGHT requests, not requests per second. Stops them from holding all your threads."

Thinking out loud: 1. "Different from QPS limits — this is concurrent connections / in-flight calls." 2. "Semaphore per user: acquire on entry, release on exit (try/finally)." 3. "Track per-user semaphore in a ConcurrentHashMap; clean up idle ones eventually."

A solid answer:

```
public class ConcurrencyLimiter {
    private final Map<String, Semaphore> permits = new ConcurrentHashMap<>();
    private final int maxConcurrent;

    public <T> T run(String userId, Supplier<T> work) throws InterruptedException {
        Semaphore sem = permits.computeIfAbsent(userId, k -> new Semaphore(maxConcurrent,));
        if (!sem.tryAcquire(100, TimeUnit.MILLISECONDS))
            throw new TooManyConcurrentRequestsException();
        try { return work.get(); } finally { sem.release(); }
    }
}
```

- `tryAcquire` with timeout → bounded wait, not unbounded blocking
- For unbounded user growth, periodically remove semaphores with all permits released

Tradeoffs: - Semaphore leaks memory per user. For 10M users, that's 10M Semaphore objects — switch to weak-keyed map or active eviction - This protects YOUR threads but doesn't limit total system load — combine with QPS limit

What NOT to say: - "Just count requests in a counter" — concurrency is acquire/release, not increment/decrement (you'd race on the threshold) - "Reject when thread pool is full" — too coarse; one user can starve the entire pool first

Common follow-up: "What's the difference between this and just having a per-user thread pool?"

Scenario 27 · Adaptive rate limiter (based on downstream health)

Asked at: Cred · Razorpay · Netflix GCC · **Difficulty:** Hard · **Topic:** Adaptive systems

The interviewer's prompt: "Rate limit should adjust based on downstream latency/error rate. If downstream is healthy, allow more; if it's struggling, throttle."

Thinking out loud: 1. "AIMD (additive-increase, multiplicative-decrease) — like TCP congestion control." 2. "On every N successes, bump limit by +1. On any failure or p99 spike, halve it." 3. "This is what Netflix's concurrency-limits library does."

A solid answer:

```
public class AdaptiveLimiter {
    private final AtomicInteger limit = new AtomicInteger(10);
    private final AtomicInteger successCount = new AtomicInteger();
    private static final int BUMP_AFTER = 100;

    public boolean allow() {
        return inFlight.get() < limit.get(); // semaphore-like check
    }

    public void onSuccess(long latencyMs) {
        if (latencyMs > LATENCY_THRESHOLD) onSlowResponse();
        else if (successCount.incrementAndGet() % BUMP_AFTER == 0)
            limit.updateAndGet(v -> Math.min(v + 1, MAX_LIMIT));
    }

    public void onError() {
        limit.updateAndGet(v -> Math.max(v / 2, MIN_LIMIT));
        successCount.set(0);
    }

    public void onSlowResponse() {
        limit.updateAndGet(v -> Math.max((int)(v * 0.9), MIN_LIMIT));
    }
}
```

- AIMD: slow growth, fast shrink — proven stable in TCP
- Floor (`MIN_LIMIT`) prevents collapse to zero during transient errors

Tradeoffs: - Reacts AFTER downstream is hurt; predictive (machine-learned) limits are an active research area - AIMD oscillates around true capacity — not perfectly steady, but converges - Per-host vs global limit — global is simpler; per-host catches asymmetric bad hosts

What NOT to say: - "Just set a high static limit" — fails when downstream degrades; cascades failure - "Catch errors and retry" — without backoff, makes things worse

Common follow-up: "What's the BBR alternative to AIMD?"

Scenario 28 · Cost-weighted rate limiter

Asked at: Razorpay · Cred · API gateway teams · **Difficulty:** Medium · **Topic:** Rate limiting

The interviewer's prompt: "Not all API calls are equal. A /search is 1 unit, /generate-pdf is 50 units. Same per-user budget."

Thinking out loud: 1. "Token bucket where each request consumes >1 tokens based on its weight." 2. "Weights defined per endpoint in config; refill rate and capacity are user-level." 3. "Big request can be partially blocked — usually we deny outright if not enough tokens."

A solid answer:

```
public class WeightedLimiter {
    private final Map<String, Integer> endpointCost = Map.of(
        "/search", 1,
        "/orders", 5,
        "/generate-pdf", 50
    );
    private final TokenBucket bucket;    // per user

    public boolean allow(String userId, String endpoint) {
        int cost = endpointCost.getOrDefault(endpoint, 1);
        return bucket.tryConsume(userId, cost);
    }
}

// TokenBucket.tryConsume(int n) — needs n tokens, deducts atomically
```

- Endpoint cost is a knob you can tune without touching code (config-driven)
- Bucket capacity should be $\geq$ max single-request cost (otherwise that request can never succeed)

Tradeoffs: - "Fairness": a user calling /generate-pdf 5 times consumes their whole budget. Could be confusing — communicate the cost in API docs. - For mixed-weight workloads, this is much more intuitive than per-endpoint limits

What NOT to say: - "Have a separate limit per endpoint" — burst on /search shouldn't be cheaper than burst on /orders if both consume DB - "Set endpoint weights at request time based on actual cost" — non-deterministic, hard for users to predict

Common follow-up: "How does the user discover how many tokens an endpoint costs?"

Scenario 29 · IP-based rate limiter with subnet fairness

Asked at: Cloudflare-style infra · CDN teams · Postman · **Difficulty:** Medium · **Topic:** Anti-abuse

The interviewer's prompt: "Limit by IP, but an attacker controls a /24 subnet and round-robins. Design fairness."

Thinking out loud: 1. "Pure per-IP allows /24 attacker to send 256× the limit. Add a per-subnet limit too." 2. "Hierarchical: enforce both /32 and /24 limits; deny if either is exceeded." 3. "For IPv6, the equivalent is /64 (typical home subnet)."

A solid answer:

```
public class HierarchicalIpLimiter {
    private final RateLimiter perIp;           // 100/min per IP
    private final RateLimiter perSubnet24;     // 1000/min per /24

    public boolean allow(String ipv4) {
        String slash24 = ipv4.substring(0, ipv4.lastIndexOf('.')) + ".0/24";
        return perIp.tryConsume(ipv4) && perSubnet24.tryConsume(slash24);
    }
}
```

- Both must pass — denial if either bucket is empty
- Subnet aggregation prevents trivial IP-rotation attacks

Tradeoffs: - NATs (corporate, mobile carriers): thousands of legitimate users behind one /24 → false positives. Combine with cookie / device-id limit - IPv4 vs IPv6: IPv6 attackers can rotate cheaply; aggregate at /48 or /64

What NOT to say: - "Just limit per IP" — trivially defeated - "Ban /24 if any IP misbehaves" — collateral damage on shared infrastructure

Common follow-up: "What if the attacker uses a botnet across 1000 ASes?"

Scenario 30 · Per-tenant quota system

Asked at: Razorpay · Cred · API platforms · **Difficulty:** Medium · **Topic:** Multi-tenancy

The interviewer's prompt: "SaaS product. Each tenant pays for a monthly quota of API calls. Track usage, enforce, send overage alerts."

Thinking out loud: 1. "Daily/monthly counters per tenant — Redis HINCRBY is the fast path." 2. "At each request: increment, check against quota, return 429 if exceeded." 3. "For accurate billing, replay from request logs to source of truth (warehouse)."

A solid answer:

```
public class QuotaService {
    private final RedisClient redis;
    private final TenantPlanService plans;

    public boolean recordAndCheck(String tenantId) {
        String key = "quota:" + tenantId + ":" + YearMonth.now();
        long used = redis.hincrBy(key, "count", 1);
        redis.expire(key, Duration.ofDays(40)); // self-cleans
        long quota = plans.getQuota(tenantId);
        if (used == (long)(quota * 0.8)) alerts.send(tenantId, "80% used");
        return used <= quota;
    }
}
```

- Counter in Redis hot path; periodic dump to warehouse for billing
- 80% / 90% / 100% threshold alerts — let customers act before they get cut off

Tradeoffs: - Eventual consistency: bursts might temporarily exceed by a few before Redis catches up. Accept ~1% overage tolerance - Hard cut at 100% vs soft (allow + bill overage): product decision; many SaaS allow burst and bill - For multi-region, use region-local counters + periodic merge; accept slight overage

What NOT to say: - "Increment on every request and check vs DB" — DB row contention at scale - "Track usage in app memory" — lost on pod restart

Common follow-up: "How would you handle quota reset at month boundary across regions?"

Scenario 31 · Backpressure on a producer

Asked at: Razorpay · Cred · Postman · **Difficulty:** Medium · **Topic:** Backpressure

The interviewer's prompt: "Producer is filling a queue faster than consumer drains. Design backpressure."

Thinking out loud: 1. "Three options: bounded queue + block, bounded queue + reject, or pass back signal to producer." 2. "BlockingQueue with put() (blocks) or offer() (rejects) is the simplest. For reactive flows, Project Reactor handles backpressure first-class." 3. "In a network protocol context, this is TCP window / HTTP/2 flow control."

A solid answer:

```

public class BackpressureQueue<T> {
    private final BlockingQueue<T> queue;
    private final BackpressureStrategy strategy;

    public boolean submit(T item) throws InterruptedException {
        return switch (strategy) {
            case BLOCK -> { queue.put(item); yield true; }
            case DROP_NEW -> queue.offer(item);
            case DROP_OLDEST -> {
                if (!queue.offer(item)) { queue.poll(); queue.offer(item); }
                yield true;
            }
        };
    }
}

```

- *BLOCK* = correctness-preserving but caller stalls (bad for HTTP handlers)
- *DROP_OLDEST* = good for telemetry where freshness > completeness

Tradeoffs: - *BLOCK* on web threads risks tying up the request thread pool — combine with timeout - *DROP_NEW* loses NEW data (recent), *DROP_OLDEST* loses OLD data. Pick based on what's more valuable - Reactive Streams gives bidirectional flow control: consumer requests *N*, producer sends *N*

What NOT to say: - "Unbounded queue" — OOM under sustained overload - "Scale up the consumer" — doesn't help if you can't process fast enough

Common follow-up: "In a reactive (Flux) pipeline, how does backpressure propagate?"

Scenario 32 · Fair queueing across users

Asked at: Cred · Postman · Atlassian GCC · **Difficulty:** Hard · **Topic:** Scheduling

The interviewer's prompt: "Single thread pool serves all users. One noisy user shouldn't starve others. Design fair scheduling."

Thinking out loud: 1. "Single FIFO queue = unfair (one user can fill it). Need per-user queues with round-robin pickup." 2. "Deficit round robin (DRR) or weighted fair queueing — each user has a 'turn'." 3. "Or simpler: cap in-flight per user (already a semaphore problem)."

A solid answer:

```

public class FairScheduler {
    private final Map<String, Deque<Runnable>> userQueues = new ConcurrentHashMap<>();
    private final List<String> rotation = new CopyOnWriteArrayList<>();
    private int cursor = 0;

    public void submit(String userId, Runnable task) {
        Deque<Runnable> q = userQueues.computeIfAbsent(userId, k -> {
            rotation.add(k);
            return new ConcurrentLinkedDeque<>();
        });
        q.offer(task);
    }

    public synchronized Runnable nextTask() {
        for (int i = 0; i < rotation.size(); i++) {
            cursor = (cursor + 1) % rotation.size();
            Runnable t = userQueues.get(rotation.get(cursor)).poll();
            if (t != null) return t;
        }
        return null;
    }
}

```

- Round-robin across users; each cycle takes one task from each user with work pending
- Noisy user has long queue but their RATE is bounded by $1/N$ of total throughput

Tradeoffs: - Memory: per-user queue. For 10M dormant users this wastes memory — reap empty queues
 - Strict round-robin ignores priority. Weighted round robin (premium user gets 3 slots per cycle) is a small extension

What NOT to say: - "Use a single FIFO" — exactly the problem we're solving - "Allocate one thread per user" — fine for 100 users, not 100k

Common follow-up: "What if some users have priority — premium gets 3x weight?"

Scenario 33 · Concurrent request deduplication

Asked at: Cred · Razorpay · Postman · **Difficulty:** Medium · **Topic:** Optimization

The interviewer's prompt: "Three concurrent requests ask for the same data. Don't make three downstream calls — coalesce."

Thinking out loud: 1. "Same as single-flight pattern from cache stampede. Map from key to in-flight `CompletableFuture`." 2. "First request creates the future, others attach. All return when the first completes." 3. "Tradeoff: increased latency for one slow request affecting all coalesced."

A solid answer:

```
public class RequestCoalescer<K,V> {
    private final ConcurrentHashMap<K, CompletableFuture<V>> inFlight = new ConcurrentHashMap<>();
    private final Function<K, V> loader;

    public CompletableFuture<V> request(K key) {
        return inFlight.computeIfAbsent(key, k ->
            CompletableFuture.supplyAsync(() -> {
                try { return loader.apply(k); }
                finally { inFlight.remove(k); } // free for next round
            })
        );
    }
}
```

- `computeIfAbsent` gives atomic "I'm first, everyone else attach to my future"
- Removal on completion → next request triggers a new load

Tradeoffs: - A slow loader holds up N callers. If loader hangs, all coalesced requests hang. Add timeout.
 - Failure of the leader propagates to all → could be desirable (consistent behavior) or undesirable (retry storm). Choose explicitly.

What NOT to say: - "Just cache the result" — different problem; coalescing is about concurrent IN-FLIGHT requests, not stored results - "Synchronize on the key" — works but allocates a lock per key permanently

Common follow-up: "What if you want each caller to retry independently on failure?"

Scenario 34 · Bulkhead pattern

Asked at: Razorpay · Cred · Atlassian GCC · **Difficulty:** Medium · **Topic:** Resilience

The interviewer's prompt: "One slow downstream is exhausting your entire thread pool. Other downstreams still healthy but you can't serve them. Design a fix."

Thinking out loud: 1. "Bulkhead: isolate resources so failure of one downstream doesn't drain everything." 2. "Separate thread pool per downstream OR semaphore per downstream caps in-flight." 3. "Hystrix did this with thread isolation; Resilience4j uses semaphore by default."

A solid answer:

```
public class BulkheadExecutor {
    private final Map<String, ExecutorService> pools = new ConcurrentHashMap<>();

    public <T> CompletableFuture<T> submit(String downstream, Supplier<T> work) {
        ExecutorService pool = pools.computeIfAbsent(downstream,
            k -> Executors.newFixedThreadPool(20)); // each downstream gets 20 threads
        return CompletableFuture.supplyAsync(work, pool);
    }
}
```

- Each downstream gets its own pool — a slow one ties up only its 20 threads
- Caller's thread is free immediately (returned the Future)

Tradeoffs: - Thread-pool isolation costs context switches and memory; semaphore isolation is lighter but doesn't give timeout semantics - Per-pool sizing needs care: too small = artificial throttling, too large = bulkhead is meaningless

What NOT to say: - "One shared thread pool — that's why we have async" — async doesn't help if all 200 threads block on the slow downstream - "Just retry on timeout" — adds load to the broken downstream

Common follow-up: "Semaphore-based vs thread-based bulkhead — pick one for HTTP-client calls."

Scenario 35 · Distributed semaphore

Asked at: Razorpay · Postman · Adobe GCC · **Difficulty:** Hard · **Topic:** Distributed concurrency

The interviewer's prompt: "Across all your pods, only 50 concurrent calls to vendor X are allowed. Design the lock."

Thinking out loud: 1. "Single-pod semaphore doesn't work — $N \text{ pods} \times 50 \text{ each} = N \times 50 \text{ total}$." 2. "Centralize counter in Redis or use a distributed lock service (ZooKeeper, etcd)." 3. "Redis approach: INCR a counter; if $> \text{limit}$, DECR and reject. EXPIRE to recover from leaks."

A solid answer:

```
public class DistributedSemaphore {
    private final RedisClient redis;
    private final String key;
    private final int limit;

    public boolean tryAcquire(Duration ttl) {
        long count = redis.incr(key);
        redis.expire(key, ttl); // leak protection
        if (count > limit) {
            redis.decr(key); // give it back
            return false;
        }
        return true;
    }

    public void release() {
        redis.decr(key);
    }
}
```

- TTL on the key prevents permanent leaks if a holder crashes without releasing
- Race condition: incr-then-check-then-decr leaves a brief over-count window. Acceptable for "approximately 50".

Tradeoffs: - Strict semantics: use a Lua script for atomic check-and-increment - ZooKeeper / etcd give stronger guarantees (linearizable) but more operational complexity - Single Redis node = single point of failure; use Redis Cluster or Sentinel

What NOT to say: - "Use Java's Semaphore" — process-local; useless across pods - "Have each pod take limit/N permits" — uneven traffic distribution wastes capacity

Common follow-up: "A pod crashes holding 10 permits. How do they get released?"

Section C — Resilience & Reliability (Scenarios 36–50)

Scenario 36 · Circuit breaker

Asked at: Razorpay · Cred · most backend roles · **Difficulty:** Medium · **Topic:** Resilience

The interviewer's prompt: "Design a circuit breaker. Three states: CLOSED, OPEN, HALF_OPEN. Walk me through the transitions."

Thinking out loud: 1. "CLOSED = normal. OPEN = downstream is dead, fail fast without calling. HALF_OPEN = probe to see if recovered." 2. "Transition triggers: error rate > X% in window → OPEN. Wait Y seconds → HALF_OPEN. K probe successes → CLOSED." 3. "State + counters guarded by either lock or atomic refs."

A solid answer:

```
public class CircuitBreaker {
    enum State { CLOSED, OPEN, HALF_OPEN }
    private final AtomicReference<State> state = new AtomicReference<>(State.CLOSED);
    private final AtomicInteger failureCount = new AtomicInteger();
    private final AtomicLong openedAt = new AtomicLong();
    private final int failureThreshold = 5;
    private final long openDurationMs = 30_000;

    public <T> T call(Supplier<T> work) {
        if (state.get() == State.OPEN) {
            if (System.currentTimeMillis() - openedAt.get() < openDurationMs)
                throw new CircuitOpenException();
            state.compareAndSet(State.OPEN, State.HALF_OPEN);
        }
        try {
            T r = work.get();
            onSuccess(); return r;
        } catch (Exception e) {
            onFailure(); throw e;
        }
    }

    private void onSuccess() {
        failureCount.set(0);
        state.set(State.CLOSED);
    }

    private void onFailure() {
        if (failureCount.incrementAndGet() >= failureThreshold) {
            openedAt.set(System.currentTimeMillis());
            state.set(State.OPEN);
        }
    }
}
```

- HALF_OPEN allows ONE probe call; success closes, failure re-opens
- Failure counter is reset on success — sliding-window failure rate is a better signal in production

Tradeoffs: - Count-based threshold (5 errors → open) vs rate-based (50% error rate over 1 minute) — rate is more robust for low-traffic services - A naive HALF_OPEN sends only one probe; production breakers (Resilience4j) allow N probes during HALF_OPEN

What NOT to say: - "Just retry on failure" — retries cascade load when downstream is dying - "Check downstream health every N seconds" — that's polling, not circuit breaking

Common follow-up: "How is this different from a retry-with-backoff?"

Scenario 37 · Retry with exponential backoff and jitter

Asked at: Razorpay · Cred · Postman · **Difficulty:** Medium · **Topic:** Resilience

The interviewer's prompt: "Implement retry with exponential backoff. Why is jitter important?"

Thinking out loud: 1. "Without jitter, 1000 clients all retry at exactly $T+1$, $T+2$, $T+4$ — thundering herd." 2. "Add randomness to the delay (full jitter is simplest, AWS recommended)." 3. "Cap max attempts AND max delay; some failures shouldn't be retried at all (4xx)."

A solid answer:

```
public class RetryPolicy {
    public <T> T execute(Supplier<T> work, int maxAttempts) throws Exception {
        long baseMs = 100;
        long maxMs = 30_000;
        for (int attempt = 1; attempt <= maxAttempts; attempt++) {
            try {
                return work.get();
            } catch (RetryableException e) {
                if (attempt == maxAttempts) throw e;
                long ceiling = Math.min(baseMs * (1L << attempt), maxMs);
                long delay = ThreadLocalRandom.current().nextLong(ceiling); // full jitter
                Thread.sleep(delay);
            }
        }
        throw new IllegalStateException();
    }
}
```

- Full jitter: $\text{delay} = \text{random}(0, \text{exp_backoff})$. Spreads retry storm uniformly.
- Distinguish retryable (5xx, timeout) vs non-retryable (4xx) — retrying a 400 is wasted work

Tradeoffs: - Retry-Storm vs idempotency: only retry idempotent operations. POST without an idempotency key — don't retry blindly. - Combine retry + circuit breaker: circuit breaker prevents retries during a sustained outage

What NOT to say: - "Fixed delay between retries" — thundering herd - "Retry on any exception" — masks programmer bugs

Common follow-up: "What's 'decorrelated jitter' and when is it better than full jitter?"

Scenario 38 · Idempotent payment API

Asked at: Razorpay · Cred · PhonePe · **Difficulty:** Hard · **Topic:** API design

The interviewer's prompt: "Design an idempotent /charge endpoint. Customer's network blips; client retries; charge twice → lawsuit."

Thinking out loud: 1. "Client must send an Idempotency-Key (UUID) per logical operation. Server stores response by key." 2. "On retry with same key: replay stored response, don't re-execute." 3. "Key TTL ~24h. Response and request fingerprint stored together to detect parameter-tampering retries."

A solid answer:

```
@PostMapping("/charge")
public ResponseEntity<ChargeResp> charge(
    @RequestHeader("Idempotency-Key") String idemKey,
    @RequestBody ChargeReq req) {
    String reqHash = sha256(req);
    IdempotencyRecord existing = store.get(idemKey);
    if (existing != null) {
        if (!existing.reqHash().equals(reqHash))
            return ResponseEntity.status(422).build(); // same key, different body
        return ResponseEntity.ok(existing.response());
    }
    // First time – try to claim the key atomically
    if (!store.putIfAbsent(idemKey, IdempotencyRecord.pending(reqHash)))
        return chargeRetryWait(idemKey); // concurrent first attempt

    ChargeResp resp = paymentEngine.process(req);
    store.put(idemKey, IdempotencyRecord.done(reqHash, resp));
    return ResponseEntity.ok(resp);
}
```

- Atomic claim (`putIfAbsent`) handles concurrent first-time attempts
- Request hash prevents misuse — same key + different body = client bug, reject

Tradeoffs: - Store: Redis with TTL=24h is fast; for stronger durability use Postgres with a UNIQUE constraint on idem_key - "What if process() succeeds but server crashes before storing response?" — concurrent retry sees `pending` , waits or polls - Key collision across customers — namespace the key: `idem:<merchantId>:<key>`

What NOT to say: - "Just retry-safe everything" — most operations aren't naturally idempotent - "Use database UNIQUE on order_id" — works for the simple case; doesn't help when client sends same key + different amount

Common follow-up: "What if the API call crashes mid-processing — how does the retry know?"

Scenario 39 · Bulkhead with timeouts

Asked at: Razorpay · Postman · Cred · **Difficulty:** Medium · **Topic:** Resilience

The interviewer's prompt: "Every outbound call to a vendor should have a hard timeout — none should hang forever. Design it."

Thinking out loud: 1. "HTTP clients (OkHttp, Apache) support connect + read timeouts. Set both." 2. "For non-HTTP work: `Future.get(timeout)` or `CompletableFuture.orTimeout()`." 3. "Timeout values come from p99 latency budget — should be tight, not generous."

A solid answer:

```
OkHttpClient client = new OkHttpClient.Builder()
    .connectTimeout(Duration.ofSeconds(2))
    .readTimeout(Duration.ofSeconds(5))
    .writeTimeout(Duration.ofSeconds(5))
    .callTimeout(Duration.ofSeconds(10))    // hard ceiling on the whole call
    .build();

// For arbitrary work:
CompletableFuture<Result> f = CompletableFuture.supplyAsync(this::call, executor);
return f.orTimeout(3, TimeUnit.SECONDS)
    .exceptionally(t -> fallback());
```

- `callTimeout` is the catch-all — even retries / redirects respect it
- For thread-pool work, prefer interruptible operations; `Thread.interrupt()` doesn't stop pure CPU loops

Tradeoffs: - Too tight: false positives on slow-but-recovering downstreams - Too loose: long-tail latency dominates p99 of your service - Track timeout count as a metric — sudden spike = downstream regression

What NOT to say: - "We trust the vendor SLA" — vendor SLAs are violated; default timeouts are usually 60s+ (too long) - "Use `Thread.sleep` with manual cancellation" — error-prone, doesn't compose

Common follow-up: "What's the difference between connect timeout, read timeout, and request timeout?"

Scenario 40 · Retry budget

Asked at: Cred · Razorpay · Atlassian GCC · **Difficulty:** Medium · **Topic:** Resilience

The interviewer's prompt: "Retries amplify load. Cap total retries as % of original traffic — design the budget."

Thinking out loud: 1. "Every retry adds load. If 100 RPS turns into 300 RPS during failures (3x retry), downstream dies harder." 2. "Token bucket of retry permits: replenish at X% of recent request rate. No permit = no retry." 3. "Google's gRPC retry policy uses this — `retry_budget_ratio` default 0.1 (10% extra traffic max)."

A solid answer:

```
public class RetryBudget {
    private final AtomicLong requestCount = new AtomicLong();
    private final AtomicLong retryCount = new AtomicLong();
    private final double maxRetryRatio = 0.10;    // 10% extra

    public boolean tryRetry() {
        long reqs = requestCount.get();
        long retries = retryCount.get();
        if (retries >= reqs * maxRetryRatio + 10) return false;    // +10 minimum allowance
        retryCount.incrementAndGet();
        return true;
    }

    public void recordRequest() { requestCount.incrementAndGet(); }
}
```

- Without budget, a downstream failure → retries → more failures → more retries → meltdown
- Add a small fixed minimum (e.g. 10 retries always allowed) to handle low-traffic services

Tradeoffs: - Reset window: per-second? Sliding 10-second window? Cumulative since service start? Sliding is most accurate, cumulative is simplest - Budget exhausted means some requests fail without retry — they get an outright error. Make sure the client knows it's a budget exhaustion, not a downstream failure

What NOT to say: - "Unlimited retries with backoff" — backoff alone doesn't bound load - "Skip retries entirely" — overcorrects; transient errors deserve retries

Common follow-up: "How do you propagate retry budget across service hops?"

Scenario 41 · Hedged requests

Asked at: Netflix GCC · Cred · Razorpay · **Difficulty:** Hard · **Topic:** Latency optimization

The interviewer's prompt: "p99 latency is bad because of slow replicas. Design hedged requests: fire a backup after N ms if no response."

Thinking out loud: 1. "Send to replica A. If no response in (p95 latency) ms, send to replica B in parallel. Take first response." 2. "Cuts p99 dramatically but doubles request load at the threshold." 3. "Need idempotent operations — hedging non-idempotent ops causes double-action bugs."

A solid answer:

```
public CompletableFuture<Response> hedgedRequest(Request req) {
    CompletableFuture<Response> primary = client.send(replicaA, req);
    CompletableFuture<Response> hedged = new CompletableFuture<>();
    Executors.newSingleThreadScheduledExecutor().schedule(() -> {
        if (!primary.isDone()) {
            client.send(replicaB, req).whenComplete((r, e) -> {
                if (e == null) hedged.complete(r);
                else hedged.completeExceptionally(e);
            });
        }
    }, 50, TimeUnit.MILLISECONDS); // hedge after 50ms
    return primary.applyToEither(hedged, Function.identity());
}
```

- Hedge delay should be ~p95 latency — most calls finish before hedging
- After the first response, cancel the loser (cooperative; HTTP can't truly cancel a request mid-flight)

Tradeoffs: - Hedge threshold low (10ms) → high load amplification; high (500ms) → minimal benefit. p95 is the sweet spot. - Server-side dedup: when both replicas land, downstream may execute twice. Idempotency or write-once semantics required.

What NOT to say: - "Send to all replicas in parallel always" — 3x load for marginal latency benefit - "Just retry on timeout" — too late; tail latency already happened

Common follow-up: "How would you implement this on the server to dedupe the doubled request?"

Scenario 42 · Graceful degradation

Asked at: Razorpay · Swiggy · Meesho · **Difficulty:** Medium · **Topic:** Resilience

The interviewer's prompt: "Search backend is down. Don't 500 the whole product page — degrade. Design the fallback chain."

Thinking out loud: 1. "Each non-critical dependency has a fallback: cached version, simpler response, or empty." 2. "Critical vs non-critical: payment is critical (no fallback, just fail). Recommendations are non-critical (return empty list)." 3. "Fallback should be cheap and bounded — don't fall back to an even-slower path."

A solid answer:

```
public ProductPage loadPage(String productId) {
    Product p = productService.load(productId); // critical – let exception propagate

    List<String> recommendations = withFallback(
        () -> recoSvc.related(productId),
        () -> List.of(), // fallback: no recos
        Duration.ofMillis(200)
    );

    List<Review> reviews = withFallback(
        () -> reviewSvc.recent(productId),
        () -> reviewCache.lastKnown(productId),
        Duration.ofMillis(500)
    );

    return new ProductPage(p, recommendations, reviews);
}

private <T> T withFallback(Supplier<T> primary, Supplier<T> fallback, Duration timeout) {
    try {
        return CompletableFuture.supplyAsync(primary).get(timeout.toMillis(), TimeUnit.MILLISECONDS);
    } catch (Exception e) {
        metrics.fallbackTriggered();
        return fallback.get();
    }
}
```

- Fallback fires on timeout OR exception — both are degradations
- Track fallback rate as a metric — high rate = upstream incident

Tradeoffs: - Cached fallback can be stale (last-known reviews from yesterday). Communicate to user via a "data may be outdated" banner - Cascading fallbacks (fallback fails → fallback's fallback) get complex — keep it 1-2 levels deep

What NOT to say: - "Just return 500" — kills user trust for a non-critical feature outage - "Return mock data" — risky if it pollutes downstream analytics

Common follow-up: "How do you decide which dependency is 'critical' vs 'fallback-able'?"

Scenario 43 · Deadline propagation

Asked at: Cred · Razorpay · Atlassian GCC · **Difficulty:** Hard · **Topic:** Distributed systems

The interviewer's prompt: "Request comes in with 1-second SLA. It hits 4 services. Design how the remaining budget propagates."

Thinking out loud: 1. "Pass deadline (absolute timestamp) as a header. Each hop subtracts its time + sets a tighter timeout on its own outgoing calls." 2. "gRPC has this built-in via `Deadline`. For REST, custom header like `X-Deadline-At: <millis since epoch>`." 3. "If a service receives an already-expired deadline, fail fast — don't even try."

A solid answer:

```
public class DeadlineContext {
    private static final ThreadLocal<Long> DEADLINE = new ThreadLocal<>();

    public static void set(long deadlineMs) { DEADLINE.set(deadlineMs); }
    public static long remainingMs() {
        Long d = DEADLINE.get();
        if (d == null) return Long.MAX_VALUE;
        return Math.max(0, d - System.currentTimeMillis());
    }
}

// Inbound filter
public void doFilter(HttpServletRequest req, ...) {
    String header = req.getHeader("X-Deadline-At");
    if (header != null) {
        long deadline = Long.parseLong(header);
        if (System.currentTimeMillis() > deadline) {
            response.setStatus(504); return; // already expired
        }
        DeadlineContext.set(deadline);
    }
}

// Outbound caller
long remaining = DeadlineContext.remainingMs();
HttpResponse r = client.send(req.timeout(Duration.ofMillis(remaining)));
```

- Absolute timestamp, not "remaining ms" — clock drift between services matters less for tight deadlines
- Inbound filter rejects expired deadlines before any work

Tradeoffs: - Clock skew between services: tolerate a few ms; if your services drift by seconds, fix NTP first - Some libraries don't support custom timeouts per call — you may need to wrap them

What NOT to say: - "Set a static 200ms timeout everywhere" — doesn't propagate; deep calls have less budget than shallow ones - "Just retry if it takes too long" — retries inside a deadline make it worse

Common follow-up: "How do you handle deadline propagation in a fan-out call (one service calls 5 others)?"

Scenario 44 · Outbox pattern

Asked at: Razorpay · Cred · Meesho · **Difficulty:** Hard · **Topic:** Reliable messaging

The interviewer's prompt: "Save an order to DB AND publish an OrderCreated event. Without 2PC, how do you ensure both happen?"

Thinking out loud: 1. "Classic dual-write problem. Without distributed transactions, the outbox pattern is standard." 2. "Write the event row to an `outbox` table in the SAME DB transaction as the order." 3. "Separate poller (or Debezium CDC) reads outbox → publishes to Kafka → marks row processed."

A solid answer:

```
@Transactional
public Order createOrder(OrderReq req) {
    Order o = orderRepo.save(req.toEntity());
    OutboxEvent evt = new OutboxEvent(UUID.randomUUID(),
        "OrderCreated", json(o), Instant.now(), false);
    outboxRepo.save(evt);
    return o;
}

// Separate scheduled job (or Debezium CDC stream)
@scheduled(fixedDelay = 1000)
public void publishOutbox() {
    List<OutboxEvent> pending = outboxRepo.findUnpublishedBatch(100);
    for (OutboxEvent e : pending) {
        kafka.send(e.topic(), e.payload());
        outboxRepo.markPublished(e.id());
    }
}
```

- Both writes in one DB tx → atomic from the application's perspective
- Poller is at-least-once: a crash after `kafka.send` but before `markPublished` → event re-publishes; consumers must be idempotent

Tradeoffs: - Polling adds latency (1s+); CDC (Debezium reading the WAL) cuts latency to ms - Outbox table grows — archive published rows to keep it small - "Just publish from app code after DB commit" — fails if app crashes between commit and publish

What NOT to say: - "Use two-phase commit" — most DBs + Kafka don't support XA properly; ops nightmare - "Send the event first, then save to DB" — even worse failure mode

Common follow-up: "What's the consumer-side idempotency story for at-least-once delivery?"

Scenario 45 · Saga pattern for distributed transactions

Asked at: Razorpay · Cred · Walmart Labs · **Difficulty:** Hard · **Topic:** Distributed transactions

The interviewer's prompt: "Order workflow: reserve inventory, charge payment, create shipment. Spans 3 services. Design without 2PC."

Thinking out loud: 1. "Saga: each step has a compensating action for rollback. No global transaction." 2. "Two flavors: choreography (events trigger next step) or orchestration (central coordinator)." 3. "For payment workflows, orchestration is easier to reason about and debug."

A solid answer:

```

public class OrderSagaOrchestrator {
    public void execute(OrderRequest req) {
        List<Compensation> compensations = new ArrayList<>();
        try {
            String resId = inventory.reserve(req.items());
            compensations.add(() -> inventory.cancelReservation(resId));

            String chargeId = payments.charge(req.amount(), req.card());
            compensations.add(() -> payments.refund(chargeId));

            String shipId = shipping.book(req.items(), req.address());
            compensations.add(() -> shipping.cancel(shipId));

            db.markComplete(req.id());
        } catch (Exception e) {
            compensations.forEach(Compensation::run); // rollback in reverse-ish
            throw new SagaFailedException(e);
        }
    }
}

```

- Each forward step has a "compensating" action that undoes it (refund undoes charge, etc.)
- Compensations must themselves be idempotent and reliable — they may retry

Tradeoffs: - "Eventually consistent" not "atomic" — there's a window where payment succeeded but order not finalized - Compensations can fail too — need dead-letter queue + manual ops escalation - Choreography (event-driven) is more decoupled but harder to track end-to-end

What NOT to say: - "Use distributed transactions across services" — XA across microservices is rarely practical - "Just retry the whole thing" — payment charge isn't free to retry

Common follow-up: "What happens if a compensation itself fails?"

Scenario 46 · Dead letter queue

Asked at: Razorpay · Cred · Postman · **Difficulty:** Medium · **Topic:** Messaging

The interviewer's prompt: "Message processing fails repeatedly. Don't loop forever. Design DLQ."

Thinking out loud: 1. "After N retries, move message to a separate DLQ topic for human inspection." 2. "Track attempt count in message headers or in a sidecar tracking store." 3. "DLQ has its own consumer (often manual / dashboard-driven for replay)."

A solid answer:

```

public class MessageProcessor {
    private static final int MAX_ATTEMPTS = 5;

    @KafkaListener(topics = "orders")
    public void onMessage(ConsumerRecord<String, OrderEvent> record) {
        int attempt = Integer.parseInt(
            new String(record.headers().lastHeader("attempt").value()) );
        try {
            process(record.value());
        } catch (Exception e) {
            if (attempt >= MAX_ATTEMPTS) {
                kafkaTemplate.send("orders.DLQ", record.value(),
                    Map.of("error", e.getMessage(), "originalTopic", "orders"));
            } else {
                kafkaTemplate.send("orders.retry", record.value(),
                    Map.of("attempt", String.valueOf(attempt + 1)));
            }
        }
    }
}

```

- Header carries attempt count between retries
- Retry topic can have a delay (delayed-message topic) so retries back off

Tradeoffs: - DLQ messages need an operator to inspect / replay — set up a dashboard - "Poison pill" messages (always fail) shouldn't block the partition — DLQ moves them aside - Storing failure context (stack trace, attempt count, last error) helps debugging

What NOT to say: - "Loop until success" — blocks the partition forever - "Drop after N retries" — loses data without operator awareness

Common follow-up: "How do you safely replay messages from DLQ back into the main flow?"

Scenario 47 · Health check endpoint

Asked at: All backend roles · **Difficulty:** Easy · **Topic:** Operations

The interviewer's prompt: "Design /health for a service. K8s uses it. What does it check?"

Thinking out loud: 1. "Two distinct concepts: liveness (am I alive?) vs readiness (can I take traffic?)." 2. "Liveness checks bare-minimum — process is running. Don't check downstreams, or a downstream blip kills your pod." 3. "Readiness checks dependencies — DB reachable, downstream APIs OK, caches warm."

A solid answer:

```

@RestController
public class HealthController {
    @GetMapping("/health/live")
    public ResponseEntity<String> live() {
        return ResponseEntity.ok("alive");    // process is up; that's it
    }

    @GetMapping("/health/ready")
    public ResponseEntity<HealthStatus> ready() {
        Map<String, String> checks = new LinkedHashMap<>();
        checks.put("db", db.ping() ? "OK" : "FAIL");
        checks.put("redis", redis.ping() ? "OK" : "FAIL");
        checks.put("kafka", kafka.isConnected() ? "OK" : "FAIL");
        boolean allOk = checks.values().stream().allMatch("OK"::equals);
        return ResponseEntity.status(allOk ? 200 : 503)
            .body(new HealthStatus(allOk, checks));
    }
}

```

- Liveness simple: K8s restarts the pod on liveness fail
- Readiness fail = K8s removes from load balancer but doesn't restart

Tradeoffs: - Don't check ALL downstreams on readiness — if any non-critical one is flaky, you bounce traffic. Check only what you NEED. - Liveness too aggressive = restart loops on transient issues; too lenient = stuck process never restarts - Add startup probe (K8s 1.16+) for slow-starting apps (cache warming)

What NOT to say: - "One /health endpoint covers both" — conflates restart vs route decisions - "Always return 200" — defeats the purpose

Common follow-up: "What's the difference between liveness, readiness, and startup probes?"

Scenario 48 · Graceful shutdown

Asked at: Cred · Razorpay · Postman · **Difficulty:** Medium · **Topic:** Operations

The interviewer's prompt: "K8s sends SIGTERM. You have 30 seconds. Design clean shutdown — no dropped requests, no message loss."

Thinking out loud: 1. "Steps: stop accepting new requests, finish in-flight, drain queues, close DB/Kafka cleanly." 2. "Spring Boot has `server.shutdown=graceful`. Underneath it's a JVM shutdown hook." 3. "Timing budget: K8s gives 30s default. Leave buffer for queue drain."

A solid answer:


```

@Configuration
public class GracefulShutdownConfig {
    @PreDestroy
    public void shutdown() throws InterruptedException {
        log.info("Shutdown initiated");
        readiness.markNotReady();           // K8s LB stops routing new requests
        Thread.sleep(5000);                 // wait for LB to notice
        Tomcat.getEngine().getConnector().pause(); // stop new requests
        long deadline = System.currentTimeMillis() + 20_000;
        while (inFlight.get() > 0 && System.currentTimeMillis() < deadline)
            Thread.sleep(100);
        kafkaConsumer.commitSync();         // commit current offsets
        kafkaConsumer.close();
        db.close();
        log.info("Shutdown complete");
    }
}

```

- Mark not-ready **FIRST** so traffic stops; pause connector **SECOND**
- Don't kill in-flight requests — let them finish, with a deadline

Tradeoffs: - terminationGracePeriodSeconds in K8s must exceed your shutdown sequence - Long-running requests (file upload) may exceed budget — they'll be killed; document this for clients - Hooks running on JVM crash (SIGKILL) — graceful shutdown does nothing; design for crash-safety too

What NOT to say: - "Catch the signal and System.exit(0)" — drops in-flight work - "Pre-shutdown lifecycle: stop the JVM immediately" — defeats the purpose

Common follow-up: "What if a request takes 60s to finish, but your shutdown budget is 30s?"

Scenario 49 · Heartbeat / liveness detection

Asked at: Cred · Postman · Adobe GCC · **Difficulty:** Medium · **Topic:** Distributed systems

The interviewer's prompt: "A worker pulls jobs from a queue. If it crashes mid-job, another worker should pick up. Design the liveness signal."

Thinking out loud: 1. "Worker takes a job and writes a 'lease' record with TTL (e.g., 30s)." 2. "While alive: periodically renew the lease (every 10s)." 3. "On crash: lease expires → job goes back to available pool."

A solid answer:

```

public class LeasedJobWorker {
    private final String workerId = UUID.randomUUID().toString();
    private final ScheduledExecutorService heartbeat = Executors.newSingleThreadScheduledExecutor()

    public void claim(Job job) {
        if (!jobStore.tryClaim(job.id(), workerId, Duration.ofSeconds(30))) {
            return; // someone else got it
        }
        ScheduledFuture<?> renewer = heartbeat.scheduleAtFixedRate(
            () -> jobStore.renewLease(job.id(), workerId, Duration.ofSeconds(30)),
            10, 10, TimeUnit.SECONDS);
        try {
            process(job);
            jobStore.markComplete(job.id());
        } finally {
            renewer.cancel(false);
        }
    }
}

// jobStore.tryClaim: SQL UPDATE WHERE worker_id IS NULL OR lease_expires_at < NOW()
// SET worker_id = ?, lease_expires_at = NOW() + ttl

```

- TTL must be > worst-case heartbeat-pause (GC pause, network blip)
- Atomic claim via conditional UPDATE — no race between checking and claiming

Tradeoffs: - Short TTL = fast recovery, more lease-renewal traffic. Long TTL = less traffic, slow recovery from crashes - "Worker is alive but stuck in GC for 45s" → lease expires, two workers think they own the job. Make job processing idempotent.

What NOT to say: - "Check if process PID exists" — works only on same host - "Use ZooKeeper ephemeral nodes" — fine answer; just more operational complexity

Common follow-up: "What if worker is alive but stuck — heartbeat keeps firing but no progress?"

Scenario 50 · Chaos engineering primitives

Asked at: Netflix GCC · Razorpay · Cred · **Difficulty:** Medium · **Topic:** Resilience testing

The interviewer's prompt: "Design a 'fault injector' that randomly fails 1% of internal RPC calls in staging — verify our resilience patterns work."

Thinking out loud: 1. "Wrap the HTTP client with a decorator that consults a fault-injection config." 2. "Fault types: throw exception, add latency, return wrong status code." 3. "Config dynamic (runtime-flipable), per-endpoint, with target % chance."

A solid answer:

```
public class FaultInjector {
    record FaultConfig(double probability, FaultType type, Duration extraLatency) {}
    enum FaultType { EXCEPTION, LATENCY, FAKE_5XX, NONE }

    private final Map<String, FaultConfig> rules = new ConcurrentHashMap<>();

    public void maybeInject(String endpoint) throws InterruptedException {
        FaultConfig cfg = rules.get(endpoint);
        if (cfg == null || ThreadLocalRandom.current().nextDouble() > cfg.probability()) return;
        switch (cfg.type()) {
            case EXCEPTION -> throw new RuntimeException("Chaos: injected failure");
            case LATENCY -> Thread.sleep(cfg.extraLatency().toMillis());
            case FAKE_5XX -> throw new HttpException(503);
        }
    }
}

// Used in client decorator:
public Response call(Request req) {
    faultInjector.maybeInject(req.endpoint());
    return delegate.call(req);
}
```

- ENV-gated: only enabled in staging / shadow traffic, never production
- Config in Redis or LaunchDarkly so SREs can flip on demand

Tradeoffs: - Don't inject in customer-facing prod by default — opt-in feature flag with strict scope - Latency injection is the best for finding hidden timeouts; exception injection finds missing catch blocks - Real chaos (Chaos Monkey) kills pods — this is request-level only

What NOT to say: - "Test resilience patterns in unit tests" — unit tests don't exercise system-wide latency / cascade - "Run only in non-production forever" — gameday in prod (gated) is how you actually validate

Common follow-up: "How do you measure whether the fault injection improved your resilience posture?"

Section D — Queues, Workers & Async Processing (Scenarios 51–65)

Scenario 51 · Job queue with retries

Asked at: Razorpay · Cred · Swiggy · **Difficulty:** Medium · **Topic:** Async processing

The interviewer's prompt: "Design a job queue. Jobs run on workers; failures retry up to 5 times with backoff; permanent failures go to DLQ."

Thinking out loud: 1. "Job table in DB or messages in Kafka — depends on durability vs throughput needs." 2. "Attempt count per job + next-attempt-at timestamp. Worker picks jobs where `next_attempt_at < now` ." 3. "On failure: bump attempt, compute next_attempt_at with exponential backoff. At max attempts → DLQ."

A solid answer:

```
record Job(UUID id, String type, String payload, int attempts, Instant nextAttemptAt, JobStatus status) {
    // ...
}

public class JobWorker {
    private static final int MAX_ATTEMPTS = 5;

    public void runOne() {
        Job job = jobStore.claimNext(workerId, Duration.ofMinutes(5)); // sets visibility timeout
        if (job == null) return;
        try {
            jobExecutor.execute(job);
            jobStore.markComplete(job.id());
        } catch (Exception e) {
            int next = job.attempts() + 1;
            if (next >= MAX_ATTEMPTS) {
                jobStore.moveToDlq(job.id(), e.getMessage());
            } else {
                long backoffMs = (long) Math.pow(2, next) * 1000;
                Instant retry = Instant.now().plusMillis(backoffMs);
                jobStore.scheduleRetry(job.id(), next, retry, e.getMessage());
            }
        }
    }
}

// claimNext SQL:
// UPDATE jobs SET worker_id=?, claim_expires=NOW()+?
// WHERE id = (SELECT id FROM jobs WHERE status='PENDING' AND next_attempt_at <= NOW()
//             ORDER BY next_attempt_at LIMIT 1 FOR UPDATE SKIP LOCKED)
// RETURNING *;
```

- **FOR UPDATE SKIP LOCKED** (Postgres/MySQL 8+) lets many workers claim concurrently without blocking
- Visibility timeout protects against worker crashes mid-job

Tradeoffs: - DB-based queue is simple but doesn't scale past a few k/sec — switch to Kafka or SQS for higher throughput - Polling interval: too short = DB load, too long = added latency. NOTIFY/LISTEN (Postgres) avoids polling - Permanent vs transient failures: 4xx from downstream means don't retry; 5xx means retry

What NOT to say: - "Use a single worker" — single point of failure - "Pick jobs in INSERT order" — broken when retries reschedule jobs

Common follow-up: "What's the difference between this and using SQS / RabbitMQ?"

Scenario 52 · Delayed task queue

Asked at: Cred · Razorpay · Postman · **Difficulty:** Medium · **Topic:** Scheduling

The interviewer's prompt: "Send a 'cart abandoned' email exactly 30 minutes after the user leaves. Design the delay queue."

Thinking out loud: 1. "In-process: ScheduledExecutorService.schedule. Doesn't survive restart." 2. "DB-backed: row with `fire_at` timestamp, worker polls for `fire_at <= now`." 3. "Production: RabbitMQ x-delayed-message plugin, Redis sorted set keyed by timestamp, or SQS delay queue."

A solid answer:

```
// Redis-backed delayed queue
public class DelayedQueue {
    private final RedisClient redis;
    private static final String KEY = "delayed:tasks";

    public void schedule(String taskPayload, Instant fireAt) {
        redis.zadd(KEY, fireAt.toEpochMilli(), taskPayload);
    }

    // Worker loop
    public void poll() {
        long now = System.currentTimeMillis();
        Set<String> due = redis.zrangeByScore(KEY, 0, now, 0, 100);
        for (String t : due) {
            if (redis.zrem(KEY, t) == 1) {    // atomic claim
                process(t);
            }
        }
    }
}
```

- Sorted set scored by epoch ms — `zrangeByScore(0, now)` returns due tasks
- `zrem` is the atomic claim: if it returns 1, this worker got it; 0 means someone else did

Tradeoffs: - Polling adds up to (poll_interval) of latency — for second-precision, poll every 500ms - Redis SortedSet supports millions of entries fine; for billions, partition by hour - For wall-clock-critical scheduling (cron-style), use a job scheduler (Quartz, Temporal)

What NOT to say: - "Spawn a thread per task with sleep(30 min)" — doesn't survive restart, doesn't scale - "Schedule with java.util.Timer" — single-threaded, single-JVM, fragile

Common follow-up: "What if you need millisecond-precise firing across millions of tasks?"

Scenario 53 · Priority job queue

Asked at: Razorpay · Cred · Postman · **Difficulty:** Medium · **Topic:** Scheduling

The interviewer's prompt: "Premium customers' jobs run before free-tier. Design priority queue with fairness."

Thinking out loud: 1. "Naive PriorityQueue: high-priority always wins → low priority can starve." 2. "Weighted fair queueing: pull from priority N with probability proportional to weight, but always make progress." 3. "Or simpler: separate queues per priority, worker pulls weighted round-robin (3 high : 1 low)."

A solid answer:

```

public class WeightedPriorityQueue<T> {
    private final Deque<T> highPriority = new ConcurrentLinkedDeque<>();
    private final Deque<T> normalPriority = new ConcurrentLinkedDeque<>();
    private final Deque<T> lowPriority = new ConcurrentLinkedDeque<>();
    private int cursor = 0;
    private static final int[] WEIGHTS = {0,0,0,1,1,2}; // 3 high : 2 normal : 1 low

    public synchronized T poll() {
        for (int tries = 0; tries < WEIGHTS.length; tries++) {
            int p = WEIGHTS[(cursor++) % WEIGHTS.length];
            Deque<T> q = switch (p) { case 0 -> highPriority; case 1 -> normalPriority; default -> lowPriority; }
            T t = q.poll();
            if (t != null) return t;
        }
        return null;
    }

    public void offer(T item, int priority) {
        (priority == 0 ? highPriority : priority == 1 ? normalPriority : lowPriority).offer(item);
    }
}

```

- Weights array is a simple way to encode "3 high, 2 normal, 1 low" ratio
- Falls through to next priority if current is empty — no idle workers

Tradeoffs: - Pure PriorityQueue starves low priorities — only use if low *MUST* never run before high - Weight tuning matters: too aggressive on high = starvation; too even = priority is meaningless - For strict SLA on premium, allocate dedicated worker pools

What NOT to say: - "Use Java's PriorityQueue" — not thread-safe, and pure priority starves - "Run high-priority first, low only when high is empty" — starvation under load

Common follow-up: "What if high-priority traffic spikes and floods the queue?"

Scenario 54 · Producer-consumer with bounded queue

Asked at: Most Java interviews · **Difficulty:** Easy · **Topic:** Concurrency

The interviewer's prompt: "Implement producer-consumer with a bounded queue. Multiple producers, multiple consumers."

Thinking out loud: 1. "BlockingQueue handles all the wait/notify nightmare — `put` blocks if full, `take` blocks if empty." 2. "ArrayBlockingQueue (bounded, FIFO) is the canonical pick." 3. "For producer-only-blocking-on-full, use `offer(timeout)`."

A solid answer:

```

public class WorkPipeline<T> {
    private final BlockingQueue<T> queue;
    private final ExecutorService producers;
    private final ExecutorService consumers;
    private final Consumer<T> handler;
    private volatile boolean running = true;

    public WorkPipeline(int capacity, int numProducers, int numConsumers, Consumer<T> handler) {
        this.queue = new ArrayBlockingQueue<>(capacity);
        this.producers = Executors.newFixedThreadPool(numProducers);
        this.consumers = Executors.newFixedThreadPool(numConsumers);
        this.handler = handler;
        for (int i = 0; i < numConsumers; i++) consumers.submit(this::consumeLoop);
    }

    public void produce(T item) throws InterruptedException {
        queue.put(item); // blocks if full
    }

    private void consumeLoop() {
        while (running) {
            try {
                T item = queue.poll(1, TimeUnit.SECONDS);
                if (item != null) handler.accept(item);
            } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        }
    }
}

```

- `put` blocks producer on full queue — natural backpressure
- `poll(timeout)` not `take()` so consumers can check `running` flag periodically

Tradeoffs: - `ArrayBlockingQueue` (fixed size, lock-based) vs `LinkedBlockingQueue` (optional bound, separate put/take locks) — `LinkedBQ` scales better under contention - Single-producer or single-consumer? `Disruptor` (LMAX) is faster for those patterns - For very high throughput, batch take: `drainTo(localList, 100)`

What NOT to say: - "Use synchronized list and wait/notify" — reinventing `BlockingQueue` badly - "Unbounded queue, problem solved" — OOM under sustained overload

Common follow-up: "When would you pick `LinkedBlockingQueue` over `ArrayBlockingQueue`?"

Scenario 55 · Pub/sub event bus (in-process)**Asked at:** Postman · Cred · Atlassian GCC · **Difficulty:** Medium · **Topic:** Eventing**The interviewer's prompt:** "Design a typed in-process event bus — components publish events, others subscribe. Multiple subscribers per event."**Thinking out loud:** 1. "Map<Class<? extends Event>, List>. publish() looks up listeners by event class."
2. "Sync (caller thread runs listeners) vs async (dispatch to executor)? Async is safer — slow listener doesn't block publisher." 3. "For type safety, generic Listener with bounded wildcards."**A solid answer:**

```

public interface Event {}
public interface Listener<T extends Event> {
    void onEvent(T event);
}

public class EventBus {
    private final Map<Class<? extends Event>, List<Listener<?>>> subs = new ConcurrentHashMap<>();
    private final ExecutorService dispatcher;

    public <T extends Event> void subscribe(Class<T> type, Listener<T> listener) {
        subs.computeIfAbsent(type, k -> new CopyOnWriteArrayList<>()).add(listener);
    }

    @SuppressWarnings({"rawtypes","unchecked"})
    public void publish(Event event) {
        List<Listener<?>> listeners = subs.get(event.getClass());
        if (listeners == null) return;
        for (Listener<?> l : listeners) {
            dispatcher.submit(() -> ((Listener)l).onEvent(event));
        }
    }
}

```

- CopyOnWriteArrayList for subscriber list: cheap reads, expensive writes (subscribe is rare)
- Async dispatch — caller doesn't wait for listeners

Tradeoffs: - Event class hierarchy: if Listener is registered, should it receive PaymentEvent (subclass)?
Pick a policy and document - Exception in one listener shouldn't kill others — catch and log - For cross-JVM events, this becomes Kafka / NATS / RabbitMQ**What NOT to say:** - "Use Spring's @EventListener" — fine answer but the interviewer wants to see you design the primitive - "Use Observer pattern from GoF" — vague; show concrete data structures**Common follow-up:** "What if a listener is slow — should it block other listeners of the same event?"

Scenario 56 · Notification dedup system

Asked at: Swiggy · Meesho · Cred · **Difficulty:** Medium · **Topic:** Messaging

The interviewer's prompt: "User opens app 5 times in a minute. They get 5 'order delivered' push notifications. Design dedup so they get 1."

Thinking out loud: 1. "Dedup key: hash of (user_id, notification_type, content_hash, time_bucket)." 2. "On send: try-set the dedup key in Redis with NX. If already set, skip." 3. "TTL on dedup key = dedup window (e.g., 10 minutes)."

A solid answer:

```
public class NotificationDeduper {
    private final RedisClient redis;
    private static final Duration DEDUP_WINDOW = Duration.ofMinutes(10);

    public boolean shouldSend(String userId, String type, String contentHash) {
        String key = "notif:dedup:" + userId + ":" + type + ":" + contentHash;
        // SET NX EX: only set if not exists, expire in N seconds
        Boolean wasSet = redis.set(key, "1", SetArgs.Builder.nx().ex(DEDUP_WINDOW.getSeconds()));
        return Boolean.TRUE.equals(wasSet);
    }
}

public class NotificationSender {
    public void send(Notification n) {
        if (!deduper.shouldSend(n.userId(), n.type(), n.contentHash())) {
            metrics.deduped(); return;
        }
        pushProvider.send(n);
    }
}
```

- **SET NX EX** is atomic — no race between check and set
- contentHash is part of key so different content goes through; identical content within window is dropped

Tradeoffs: - Window length: too short = duplicates leak through; too long = legitimate updates suppressed (e.g., real status change) - For user-aggregated digest ("3 new likes"), need a different pattern: rate-limited bundling, not dedup - Redis is single-region by default; cross-region needs replicated dedup or per-region windows

What NOT to say: - "Check DB before sending" — adds latency per send, race conditions - "Compare to last sent notification" — works for sequence but not concurrent triggers

Common follow-up: "Now bundle 'you got 3 new messages' instead of dropping the duplicates."

Scenario 57 · Async task with progress tracking

Asked at: Postman · Razorpay · Adobe GCC · **Difficulty:** Medium · **Topic:** Async APIs

The interviewer's prompt: "User uploads a 100MB CSV for import. UI needs to show 'processing... 47% done'. Design the async API."

Thinking out loud: 1. "Submit returns a task_id immediately. Worker processes async. Client polls /tasks/{id} for status." 2. "Status: PENDING, RUNNING (with progress %), COMPLETED, FAILED." 3. "For real-time updates, WebSocket / SSE instead of polling. Polling is simpler and usually fine."

A solid answer:

```
@PostMapping("/imports")
public ResponseEntity<TaskResp> create(@RequestBody MultipartFile file) {
    String taskId = UUID.randomUUID().toString();
    taskStore.put(taskId, Task.pending(taskId));
    asyncExecutor.submit(() -> runImport(taskId, file));
    return ResponseEntity.accepted()
        .header("Location", "/tasks/" + taskId)
        .body(new TaskResp(taskId, "PENDING"));
}

@GetMapping("/tasks/{id}")
public Task get(@PathVariable String id) {
    Task t = taskStore.get(id);
    if (t == null) throw new NotFoundException();
    return t;
}

private void runImport(String taskId, MultipartFile file) {
    int total = countLines(file);
    int processed = 0;
    for (Row row : reader(file)) {
        process(row);
        processed++;
        if (processed % 100 == 0) {
            taskStore.update(taskId, t -> t.withProgress((double)processed/total));
        }
    }
    taskStore.update(taskId, t -> t.withStatus(COMPLETED));
}
```

- 202 Accepted on creation + Location header pointing to status URL
- Update progress in batches (every 100 rows) — not every row, that hammers the store

Tradeoffs: - Polling interval: 1-2s for UX; <1s wastes server resources - For very long jobs (hours), SSE/ WebSocket avoids constant polling but adds connection state - Task store: in-memory ConcurrentHashMap works for single-node; needs Redis/DB for multi-pod

What NOT to say: - "Block the HTTP call until done" — request times out, browser disconnects, terrible UX - "Send progress in response body chunks" — works (HTTP streaming) but breaks proxies; SSE is cleaner

Common follow-up: "How would you make this resumable if the worker pod restarts mid-import?"

Scenario 58 · At-least-once vs exactly-once delivery

Asked at: Razorpay · Cred · Walmart Labs · **Difficulty:** Hard · **Topic:** Messaging semantics

The interviewer's prompt: "Kafka gives at-least-once by default. You're told to make consumers exactly-once. How?"

Thinking out loud: 1. "True exactly-once is hard; usually we mean 'at-least-once delivery + idempotent processing = effectively-once'." 2. "Consumer tracks processed message IDs. On replay, skip if already processed." 3. "Kafka 0.11+ has transactional producer (EOS within Kafka), but consumer-side dedup still needed for external side effects."

A solid answer:

```

@KafkaListener(topics = "payments")
public void onMessage(ConsumerRecord<String, Payment> rec) {
    String msgId = rec.value().idempotencyKey();
    if (!processedStore.tryClaim(msgId)) {
        log.info("Already processed: {}", msgId);
        return;
    }
    try {
        applyPayment(rec.value());
    } catch (Exception e) {
        processedStore.release(msgId); // allow retry
        throw e;
    }
}

// processedStore: Redis SETNX with TTL=7 days
public boolean tryClaim(String id) {
    return "OK".equals(redis.set("processed:" + id, "1", SetArgs.Builder.nx().ex(604800)));
}

```

- The DB/Redis claim must be atomic with the side effect for true EOS
- For DB side effects, use a transactional outbox: insert into processed_ids AND do work in same TX

Tradeoffs: - Storage: tracking IDs forever doesn't scale. TTL based on max-realistic-replay-window (usually days, not years) - Kafka transactions span produce-and-consume but NOT external systems (HTTP calls, third-party APIs) - For HTTP side effects, the API must itself be idempotent (Idempotency-Key header)

What NOT to say: - "Just enable Kafka exactly-once" — only protects Kafka-to-Kafka; external systems still need dedup - "Disable retries" — at-most-once = data loss

Common follow-up: "How big does the 'processed_ids' table get over time? How do you bound it?"

Scenario 59 · Worker pool with backpressure

Asked at: Razorpay · Cred · Postman · **Difficulty:** Medium · **Topic:** Concurrency

The interviewer's prompt: "Design a worker pool that grows under load but stops accepting work when too overloaded."

Thinking out loud: 1. "ThreadPoolExecutor with a bounded queue + CallerRunsPolicy = natural backpressure." 2. "CorePoolSize + maxPoolSize: pool grows when queue fills." 3. "When pool is at max AND queue is full: reject. CallerRuns makes the SUBMITTER run the work (slowing them down)."

A solid answer:

```
ThreadPoolExecutor pool = new ThreadPoolExecutor(
    10, // core threads
    50, // max threads
    60, TimeUnit.SECONDS, // idle timeout for non-core
    new ArrayBlockingQueue<>(200), // bounded queue
    new ThreadFactoryBuilder().setNameFormat("worker-%d").build(),
    new ThreadPoolExecutor.CallerRunsPolicy()
);
```

- Queue fills first (up to 200) before pool grows past core
- At max + full queue: *CallerRunsPolicy* runs the task on the submitter's thread — that thread is now slow, so submission rate falls

Tradeoffs: - Counterintuitive: *ThreadPoolExecutor* grows beyond core ONLY when queue is full, not when core is busy - *AbortPolicy* (default) throws *RejectedExecutionException* — exposes overload to caller - *DiscardPolicy* / *DiscardOldestPolicy* silently drop — usually wrong for backend work

What NOT to say: - "Use *Executors.newCachedThreadPool*" — unbounded growth; OOM risk - "Use *SynchronousQueue*" — every task creates a thread; works for short tasks only

Common follow-up: "What's wrong with `Executors.newCachedThreadPool()` in production?"

Scenario 60 · Scheduled task with overlap prevention

Asked at: Cred · Razorpay · Postman · **Difficulty:** Medium · **Topic:** Scheduling

The interviewer's prompt: "@Scheduled runs every 10 seconds. Task usually takes 5s, sometimes 30s. Don't want two copies running."

Thinking out loud: 1. "Spring's @Scheduled doesn't run the next iteration until the previous returns IF you use `fixedDelay` — that's the simplest fix." 2. "With `fixedRate`, runs at fixed intervals regardless of duration → overlap possible." 3. "For cross-pod overlap prevention, distributed lock (Redis SETNX)."

A solid answer:

```

@Scheduled(fixedDelay = 10_000)    // 10s AFTER previous run completes
public void singleNode() {
    runHeavyJob();
}

// Cross-pod: only one pod should run at a time
@Scheduled(fixedDelay = 10_000)
public void crossPod() {
    String lockKey = "scheduled:heavyJob";
    String wasSet = redis.set(lockKey, "1", SetArgs.Builder.nx().ex(60));
    if (!"OK".equals(wasSet)) {
        log.info("Another pod is running; skipping");
        return;
    }
    try {
        runHeavyJob();
    } finally {
        redis.del(lockKey);
    }
}

```

- `fixedDelay` is the right choice if you don't WANT overlap; `fixedRate` is for "I want to run every N seconds regardless"
- Redis lock TTL > worst-case run duration so a crash doesn't permanently block

Tradeoffs: - Drift over time: `fixedDelay` means runs slowly drift if duration > delay. Use cron for wall-clock-aligned scheduling - ShedLock library (Spring) wraps the Redis-lock pattern cleanly

What NOT to say: - "Set `fixedRate` to 0" — invalid; defeats scheduling - "Use a semaphore in memory" — doesn't work across pods

Common follow-up: "What if the lock-holder pod crashes mid-job?"

Scenario 61 · Periodic batch processor

Asked at: Razorpay · Swiggy · Adobe GCC · **Difficulty:** Medium · **Topic:** Batching

The interviewer's prompt: "Process incoming items in batches of 100, OR every 5 seconds, whichever comes first. Design it."

Thinking out loud: 1. "Classic batching: collect into a buffer, flush on size OR time." 2. "Single `ScheduledExecutorService` for time-based flush + lock-protected buffer." 3. "Concurrent producers add to buffer; one flusher thread reads + clears."

A solid answer:

```

public class BatchProcessor<T> {
    private final int batchSize;
    private final Duration maxWait;
    private final Consumer<List<T>> flusher;
    private final List<T> buffer = new ArrayList<>();
    private final ScheduledExecutorService scheduler = Executors.newSingleThreadScheduledExecutor()

    public BatchProcessor(int batchSize, Duration maxWait, Consumer<List<T>> flusher) {
        this.batchSize = batchSize;
        this.maxWait = maxWait;
        this.flusher = flusher;
        scheduler.scheduleAtFixedRate(this::flush, maxWait.toMillis(), maxWait.toMillis(), TimeUnit.SECONDS);
    }

    public void add(T item) {
        List<T> toFlush = null;
        synchronized (buffer) {
            buffer.add(item);
            if (buffer.size() >= batchSize) {
                toFlush = new ArrayList<>(buffer);
                buffer.clear();
            }
        }
        if (toFlush != null) flusher.accept(toFlush);
    }

    private void flush() {
        List<T> toFlush;
        synchronized (buffer) {
            if (buffer.isEmpty()) return;
            toFlush = new ArrayList<>(buffer);
            buffer.clear();
        }
        flusher.accept(toFlush);
    }
}

```

- Flush outside the synchronized block — don't hold lock during downstream call
- Time-based flush always runs; size-based fires opportunistically

Tradeoffs: - Strict batch size + low traffic = high latency. Time bound bounds latency. - Synchronized contention high under heavy throughput; switch to per-thread buffers + periodic merge (Disruptor style) - Flush failure handling: retry the batch, or drop, or DLQ — depends on data criticality

What NOT to say: - "Use a TimerTask" — Timer is fragile (single thread, swallows exceptions) - "Flush on every add" — defeats the purpose of batching

Common follow-up: "What if flusher fails — do you retry, drop, or block additions?"

Scenario 62 · Message ordering guarantee

Asked at: Razorpay · Cred · Walmart Labs · **Difficulty:** Hard · **Topic:** Messaging

The interviewer's prompt: "Each user's events must be processed in order. Across users, parallelism is fine. Design it."

Thinking out loud: 1. "Partition by user_id — all events for same user go to same partition / queue / worker." 2. "In Kafka, set partition key = userId. Single-threaded consumer per partition preserves order." 3. "Within a single JVM, hash userId to one of N worker threads."

A solid answer:

```
// Producer side (Kafka)
producer.send(new ProducerRecord<>("user-events", userId, event));
// Kafka guarantees partition assignment by hash(key), and order WITHIN partition

// In-JVM equivalent
public class OrderedDispatcher {
    private final ExecutorService[] workers;

    public OrderedDispatcher(int parallelism) {
        this.workers = new ExecutorService[parallelism];
        for (int i = 0; i < parallelism; i++)
            workers[i] = Executors.newSingleThreadExecutor(); // single thread = order preserved
    }

    public void submit(String userId, Runnable task) {
        int shard = Math.abs(userId.hashCode() % workers.length);
        workers[shard].submit(task);
    }
}
```

- Each worker is single-threaded — within-shard order guaranteed
- Different users may share a worker (hash collision) but each user's events stay in submit order

Tradeoffs: - Hot user with 90% of traffic creates a hot worker — other users behind them queue - Rebalance during consumer-group change can deliver out of order briefly — Kafka 2.4+ "static membership" reduces this - Strict order forbids parallelism within a user — if order doesn't matter for some events, route those differently

What NOT to say: - "Use a thread pool, threads pick from a queue" — no order guarantee across threads - "Sort by timestamp on receive" — sender clock skew breaks this; can't sort a stream

Common follow-up: "What if user_id 'X' has 90% of traffic — how do you avoid a hot partition?"

Scenario 63 · Long-running task with cancellation

Asked at: Postman · Razorpay · Adobe GCC · **Difficulty:** Medium · **Topic:** Concurrency

The interviewer's prompt: "Async task can take 10 minutes. User clicks cancel. Design clean cancellation."

Thinking out loud: 1. "Cancellation in Java is cooperative — `Future.cancel(true)` interrupts the thread, but the work must check interruption." 2. "Worker periodically checks `Thread.currentThread().isInterrupted()` ." 3. "For HTTP downstream calls, use clients that respect interruption (or you need explicit timeout)."

A solid answer:

```
public class CancellableTask {
    private final Map<String, Future<?>> running = new ConcurrentHashMap<>();
    private final ExecutorService pool = Executors.newCachedThreadPool();

    public String submit(Runnable work) {
        String taskId = UUID.randomUUID().toString();
        Future<?> f = pool.submit(work);
        running.put(taskId, f);
        return taskId;
    }

    public boolean cancel(String taskId) {
        Future<?> f = running.remove(taskId);
        return f != null && f.cancel(true); // true = interrupt if running
    }
}

// Inside the work:
public void run() {
    for (Item item : items) {
        if (Thread.currentThread().isInterrupted()) {
            cleanup();
            return;
        }
        process(item);
    }
}
```

- Worker checks `isInterrupted()` between units of work
- Long blocking calls (`Thread.sleep`, `BlockingQueue.take`) throw `InterruptedException` — catch + clean up

Tradeoffs: - Pure CPU-bound code can't be interrupted — must add explicit checks - JDBC calls in older drivers don't respect interrupt — set query timeout explicitly - Cleanup on cancel matters — partial writes, holding locks: ensure finally blocks run

What NOT to say: - "Use `Thread.stop()`" — deprecated since 1.2, leaves objects in inconsistent state - "Set a flag and check it" — fine as backup, but interrupt is the standard mechanism

Common follow-up: "What if the task is stuck in a third-party library that ignores interrupts?"

Scenario 64 · Fan-out aggregation

Asked at: Cred · Razorpay · Postman · **Difficulty:** Medium · **Topic:** Concurrency

The interviewer's prompt: "Need to call 5 microservices in parallel and combine their results. Design it with proper error handling."

Thinking out loud: 1. "CompletableFuture.allOf + thenApply, OR Mono.zip in reactive." 2. "Partial failure: do you fail the whole call, return what succeeded, or use defaults for failed?" 3. "Apply per-call timeout so one slow service doesn't bottleneck."

A solid answer:

```
public PageData loadPage(String userId) {
    CompletableFuture<Profile> profile = CompletableFuture.supplyAsync(() -> profileSvc.load(userId))
        .orTimeout(500, MILLISECONDS).exceptionally(e -> Profile.empty());
    CompletableFuture<List<Order>> orders = CompletableFuture.supplyAsync(() -> orderSvc.recent(userId))
        .orTimeout(500, MILLISECONDS).exceptionally(e -> List.of());
    CompletableFuture<Wallet> wallet = CompletableFuture.supplyAsync(() -> walletSvc.balance(userId))
        .orTimeout(300, MILLISECONDS).exceptionally(e -> Wallet.unknown());
    CompletableFuture<List<Reward>> rewards = CompletableFuture.supplyAsync(() -> rewardSvc.list(userId))
        .orTimeout(500, MILLISECONDS).exceptionally(e -> List.of());
    CompletableFuture<List<Notification>> notifs = CompletableFuture.supplyAsync(() -> notifSvc.unread(userId))
        .orTimeout(500, MILLISECONDS).exceptionally(e -> List.of());

    return CompletableFuture.allOf(profile, orders, wallet, rewards, notifs)
        .thenApply(v -> new PageData(profile.join(), orders.join(), wallet.join(),
            rewards.join(), notifs.join()))
        .join();
}
```

- Per-call timeout + exception fallback prevents one slow/failing service from dragging down the page
- `allOf().thenApply(...).join()` waits for all then combines

Tradeoffs: - Critical vs optional: profile failure might warrant full failure; reward failure shouldn't. Encode this differently per dependency. - Use a separate ForkJoinPool for these calls — sharing common pool with everything is dangerous

What NOT to say: - "Call sequentially" — $5 \times 200\text{ms} = 1 \text{ second}$; parallel = 200ms - "Use single `CompletableFuture` chain (thenApply.thenApply)" — that's sequential

Common follow-up: "What if 3 of 5 services return errors? Do you serve a degraded page or fail entirely?"

Scenario 65 · Async result polling vs webhook

Asked at: Razorpay · Cred · Postman · **Difficulty:** Medium · **Topic:** API design

The interviewer's prompt: "Long-running operation. Client needs the result. Design polling vs webhook — and when to pick which."

Thinking out loud: 1. "Polling: client periodically GETs /tasks/{id}. Simple, works through firewalls." 2. "Webhook: client provides a callback URL; we POST when done. Faster, no polling waste." 3. "Hybrid: webhook + polling as fallback (in case webhook delivery fails)."

A solid answer:

```
@PostMapping("/jobs")
public ResponseEntity<JobResp> submit(@RequestBody JobReq req) {
    String jobId = UUID.randomUUID().toString();
    Job job = new Job(jobId, req, PENDING, req.callbackUrl());
    jobStore.save(job);
    asyncRunner.submit(() -> execute(job));
    return ResponseEntity.accepted().body(new JobResp(jobId,
        "/jobs/" + jobId,                // for polling
        req.callbackUrl()                // confirms webhook will fire
    ));
}

private void execute(Job job) {
    JobResult result = doWork(job.req());
    jobStore.update(job.id(), result);
    if (job.callbackUrl() != null) {
        webhookSender.deliver(job.callbackUrl(), result); // with retries
    }
}
```

- Both supported: client can choose. Webhook for speed, polling as backstop.

- Webhook delivery needs retries + dead-letter — don't lose results due to client's transient errors

Tradeoffs: - Webhook: client needs a public URL, signature verification, retry tolerance. Complex. - Polling: trivial for client, wastes resources at high frequency. Long-poll is a middle ground. - Webhook security: HMAC signature, IP allowlist, mutual TLS — pick at least one

What NOT to say: - "Use WebSocket" — overkill for fire-and-forget result; great for ongoing streams - "Block the request until done" — fails for 10-min jobs

Common follow-up: "How do you secure the webhook so the client knows it's actually from us?"

Section E — Concurrency Primitives (Scenarios 66–85)

Scenario 66 · Producer-consumer using wait/notify

Asked at: TCS · Infosys · classic concurrency interview · **Difficulty:** Medium · **Topic:** Concurrency primitives

The interviewer's prompt: "Implement producer-consumer using wait/notify only — no BlockingQueue. Bounded buffer."

Thinking out loud: 1. "Single monitor protects the buffer. Producers wait on full; consumers wait on empty." 2. "Always wait in a loop (spurious wakeups). notifyAll() not notify() — single notify on shared monitor can miss the right thread." 3. "Show the classic structure to prove you understand the primitives."

A solid answer:

```
public class BoundedBuffer<T> {
    private final Queue<T> buffer = new ArrayDeque<>();
    private final int capacity;
    public BoundedBuffer(int capacity) { this.capacity = capacity; }

    public synchronized void put(T item) throws InterruptedException {
        while (buffer.size() == capacity) wait();    // loop, not if!
        buffer.offer(item);
        notifyAll();
    }

    public synchronized T take() throws InterruptedException {
        while (buffer.isEmpty()) wait();
        T item = buffer.poll();
        notifyAll();
        return item;
    }
}
```

- `while` not `if` — spurious wakeups and missed condition checks need a re-test
- `notifyAll` wakes both producers and consumers; with single monitor you can't distinguish

Tradeoffs: - `notifyAll` is wasteful — many threads wake to find condition unmet, go back to wait. Use `ReentrantLock + 2 Conditions` (`notFull`, `notEmpty`) for surgical signaling - In production, just use `ArrayBlockingQueue` — it does this correctly

What NOT to say: - "Use `if (buffer.size() == capacity) wait();`" — fails on spurious wakeup - "Use `notify()` to save resources" — risk waking the wrong type of waiter (producer instead of consumer)

Common follow-up: "Why `notifyAll` vs `notify` here? What goes wrong with `notify`?"

Scenario 67 · ReentrantLock with two conditions

Asked at: Razorpay · Cred · Atlassian GCC · **Difficulty:** Hard · **Topic:** Concurrency primitives

The interviewer's prompt: "Rewrite the bounded buffer with `ReentrantLock + Conditions`. Why is it better?"

Thinking out loud: 1. "Two conditions on one lock: `notFull` (producers wait), `notEmpty` (consumers wait)." 2. "Producer signal: `notEmpty`. Consumer signal: `notFull`. No more 'wake everybody and pray'." 3. "Same wait-in-loop discipline; same correctness; better throughput."

A solid answer:

```

public class BoundedBuffer<T> {
    private final Queue<T> buffer = new ArrayDeque<>();
    private final int capacity;
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notFull = lock.newCondition();
    private final Condition notEmpty = lock.newCondition();

    public void put(T item) throws InterruptedException {
        lock.lock();
        try {
            while (buffer.size() == capacity) notFull.await();
            buffer.offer(item);
            notEmpty.signal();                // wake one consumer
        } finally { lock.unlock(); }
    }

    public T take() throws InterruptedException {
        lock.lock();
        try {
            while (buffer.isEmpty()) notEmpty.await();
            T item = buffer.poll();
            notFull.signal();                // wake one producer
            return item;
        } finally { lock.unlock(); }
    }
}

```

- `signal()` (not `signalAll()`) is now safe — each condition only has the right type of waiters
- Lock + Condition is what `BlockingQueue` uses internally

Tradeoffs: - More verbose than `synchronized` + `wait/notify`, but more precise and slightly faster - Lock fairness can be enabled (`new ReentrantLock(true)`) — first-come-first-served at cost of throughput - For very simple cases, `synchronized` is fine and easier to read

What NOT to say: - "Lock vs `synchronized` makes no difference" — false; conditions give per-condition signaling - "Always use fair locks" — fairness costs ~10x throughput; rarely needed

Common follow-up: "When would you use a fair `ReentrantLock`? What's the throughput cost?"

Scenario 68 · `CountDownLatch` usage

Asked at: Cred · Razorpay · classic Java concurrency · **Difficulty:** Easy · **Topic:** Concurrency primitives

The interviewer's prompt: "3 service health checks must complete before app accepts traffic. Design with `CountDownLatch`."

Thinking out loud: 1. "Main thread awaits latch; each check counts down on completion." 2. "CountDownLatch is single-use — counter only goes down, never up. For reusable, use CyclicBarrier or Phaser." 3. "Add timeout so a stuck check doesn't block forever."

A solid answer:

```
public boolean waitForReadiness() throws InterruptedException {
    CountDownLatch latch = new CountDownLatch(3);
    ExecutorService pool = Executors.newFixedThreadPool(3);

    pool.submit(() -> { checkDb();    latch.countDown(); });
    pool.submit(() -> { checkRedis(); latch.countDown(); });
    pool.submit(() -> { checkKafka(); latch.countDown(); });

    boolean ready = latch.await(30, TimeUnit.SECONDS);
    pool.shutdown();
    return ready;
}
```

- `await(timeout)` returns true if reached zero, false if timeout
- `countDown` in finally if checks can throw — otherwise a failing check stalls await

Tradeoffs: - Single-use. For "run 3 phases of work in lockstep", use CyclicBarrier or Phaser - Counter only goes down; you can't "undo" a countDown

What NOT to say: - "Use it for repeated synchronization" — single-use only; reset isn't a thing - "Use Thread.join" — works only for managed threads; latch decouples synchronization from threads

Common follow-up: "What if I need to do this every 5 minutes? CountDownLatch can't be reset."

Scenario 69 · CyclicBarrier for parallel computation

Asked at: Cred · Adobe GCC · scientific computing · **Difficulty:** Medium · **Topic:** Concurrency primitives

The interviewer's prompt: "4 worker threads compute partial sums. All must finish phase 1 before phase 2 starts. Design with CyclicBarrier."

Thinking out loud: 1. "CyclicBarrier(4): each thread calls await() at phase boundary; barrier releases when all 4 arrived." 2. "After release, barrier auto-resets — can be reused for next phase." 3. "Optional barrier action runs once when all threads arrive — useful for aggregation."

A solid answer:


```

public class ParallelComputation {
    private final int[][] data;
    private final long[] partials = new long[4];

    public void run() {
        CyclicBarrier barrier = new CyclicBarrier(4, () -> {
            // Runs once when all 4 arrive – aggregate phase 1 results
            long sum = Arrays.stream(partials).sum();
            System.out.println("Phase 1 sum: " + sum);
        });

        for (int i = 0; i < 4; i++) {
            final int idx = i;
            new Thread(() -> {
                partials[idx] = computePartial(data[idx]);
                try { barrier.await(); } // wait for others
                catch (Exception e) { return; }
                runPhase2(idx);
                try { barrier.await(); }
                catch (Exception e) { }
            }).start();
        }
    }
}

```

- Barrier reused across phases — same instance handles both await calls
- Barrier action is a great place for cross-thread aggregation, since it's guaranteed to run after all arrive

Tradeoffs: - One thread throws `BrokenBarrierException` → all other threads at the barrier also throw — barrier becomes useless until reset - For dynamic worker counts (add/remove during run), use `Phaser` instead

What NOT to say: - "Use `CountDownLatch`" — single use; can't reset for phase 2 - "Use `synchronized` + `condition`" — reinventing `CyclicBarrier` badly

Common follow-up: "What's `Phaser`, and when is it better than `CyclicBarrier`?"

Scenario 70 · Semaphore-based connection pool

Asked at: Razorpay · Postman · Adobe GCC · **Difficulty:** Medium · **Topic:** Concurrency primitives

The interviewer's prompt: "Use `Semaphore` to enforce 'max 10 concurrent users of this resource'."

Thinking out loud: 1. "Semaphore(10): acquire() blocks if no permits, release() returns a permit." 2. "Per usage: acquire on entry, release in finally." 3. "tryAcquire(timeout) is the production-safe variant — fail fast instead of hanging."

A solid answer:

```
public class LimitedResource {
    private final Semaphore permits = new Semaphore(10);

    public <T> T use(Function<Resource, T> work) throws TimeoutException, InterruptedException {
        if (!permits.tryAcquire(5, TimeUnit.SECONDS)) {
            throw new TimeoutException("Resource pool exhausted");
        }
        try {
            return work.apply(getResource());
        } finally {
            permits.release();
        }
    }
}
```

- finally + release: prevents permit leaks on exception
- Fair Semaphore (`new Semaphore(10, true)`) prevents starvation under heavy contention

Tradeoffs: - Fairness costs throughput — only enable when starvation matters - Semaphore is per-JVM; for cross-pod limit, use Redis SETNX-based distributed semaphore - Acquire-release imbalance → permits leak or go negative — Java doesn't enforce balance

What NOT to say: - "Use synchronized on a counter" — needs wait/notify; Semaphore is exactly this primitive - "Use AtomicInteger" — doesn't block when limit reached

Common follow-up: "What happens if I release() more than I acquire()? Should I worry?"

Scenario 71 · Phaser for dynamic parties

Asked at: Cred · Adobe GCC · scientific systems · **Difficulty:** Hard · **Topic:** Concurrency primitives

The interviewer's prompt: "Multi-phase computation where workers can join or leave mid-run. CyclicBarrier doesn't support dynamic membership. Use Phaser."

Thinking out loud: 1. "Phaser allows register() and arriveAndDeregister() at runtime — unlike CyclicBarrier's fixed count." 2. "arriveAndAwaitAdvance() = arrive at phase boundary AND wait for all parties." 3. "Useful for fork-join style workflows where the worker pool is fluid."

A solid answer:

```
public class DynamicComputation {
    private final Phaser phaser = new Phaser(1);    // 1 = main thread

    public void run() {
        for (int i = 0; i < 5; i++) {
            phaser.register();                      // new worker joins
            int idx = i;
            new Thread(() -> {
                for (int phase = 0; phase < 3; phase++) {
                    doWork(idx, phase);
                    phaser.arriveAndAwaitAdvance(); // sync to next phase
                }
                phaser.arriveAndDeregister();        // leave the party
            }).start();
        }
        phaser.arriveAndDeregister();                // main thread leaves
    }
}
```

- `register()` adds a party; `arriveAndDeregister()` removes
- `arriveAndAwaitAdvance()` is the per-phase barrier

Tradeoffs: - Phaser is more flexible than CyclicBarrier but harder to use correctly — terminate accidentally and all bets are off - For most parallel-loop cases, CyclicBarrier or even ForkJoinPool is simpler - Phaser supports hierarchical phasers for very large parties (avoids contention on a single phaser)

What NOT to say: - "Use CyclicBarrier" — fixed party count; the whole point is dynamic - "Spin up new CountdownLatch each phase" — works but verbose and error-prone

Common follow-up: "How does hierarchical Phaser help with large party counts?"

Scenario 72 · AtomicReference CAS loop

Asked at: Razorpay · Cred · Postman · **Difficulty:** Medium · **Topic:** Lock-free programming

The interviewer's prompt: "Implement a thread-safe 'update if newer' using AtomicReference. No locks."

Thinking out loud: 1. "Compare-and-swap loop: read current, compute new, CAS — retry if someone else won the race." 2. "AtomicReference.updateAndGet does this for us. Or compareAndSet in an explicit loop." 3. "Show the loop pattern — interviewers want to see I know what CAS looks like."

A solid answer:

```
public class LatestVersion<T> {
    record Versioned<T>(T value, long version) {}
    private final AtomicReference<Versioned<T>> ref = new AtomicReference<>();

    public void updateIfNewer(T value, long version) {
        Versioned<T> proposed = new Versioned<>(value, version);
        Versioned<T> current;
        do {
            current = ref.get();
            if (current != null && current.version() >= version) return;
        } while (!ref.compareAndSet(current, proposed));
    }

    // Using updateAndGet – cleaner:
    public void updateIfNewerV2(T value, long version) {
        ref.updateAndGet(curr ->
            curr != null && curr.version() >= version ? curr
            : new Versioned<>(value, version));
    }
}
```

- CAS loop is the lock-free idiom: read, compute, CAS, retry on failure
- updateAndGet abstracts the loop for simple cases

Tradeoffs: - High contention → many CAS retries → CPU burn (livelock-like). LongAdder uses striping to reduce contention. - ABA problem: if value goes A→B→A between read and CAS, CAS succeeds but state actually changed. AtomicStampedReference solves it.

What NOT to say: - "Use AtomicReference.set" — overwrites unconditionally; loses the "if newer" check
- "Lock the reference" — defeats the lock-free advantage

Common follow-up: "What's the ABA problem and when do you need AtomicStampedReference?"

Scenario 73 · LongAdder for high-contention counter

Asked at: Razorpay · Cred · Postman · **Difficulty:** Medium · **Topic:** High-perf concurrency

The interviewer's prompt: "AtomicLong is too slow at 64 cores incrementing same counter. Use LongAdder."

Thinking out loud: 1. "AtomicLong = single memory location; CAS contention scales badly past ~16 cores." 2. "LongAdder = striped counter; each thread tends to update a different cell." 3. "sum() walks all cells — fast for write-heavy / read-occasional workloads."

A solid answer:

```
public class RequestCounter {
    private final LongAdder count = new LongAdder();

    public void onRequest() {
        count.increment();           // O(1), low contention
    }

    public long getTotal() {
        return count.sum();          // walks ~Ncells, no concurrent guarantee
    }

    public long getAndReset() {
        return count.sumThenReset();
    }
}
```

- LongAdder.sum() is NOT atomic — it walks cells while threads may still be incrementing. Acceptable for monitoring.
- For exact-read-at-instant counters, use AtomicLong with the throughput hit

Tradeoffs: - Memory: LongAdder uses $\sim N \times 16$ bytes per active thread (vs 16 for AtomicLong) - Read-heavy workloads: AtomicLong is faster (single read vs N-cell walk) - For monotone counts (metrics), LongAdder is the right pick

What NOT to say: - "AtomicLong scales linearly" — false; CAS contention dominates past ~16 threads - "Use synchronized" — orders of magnitude worse

Common follow-up: "When does AtomicLong outperform LongAdder?"

Scenario 74 · ThreadLocal for per-thread state

Asked at: Razorpay · Cred · Postman · **Difficulty:** Medium · **Topic:** Concurrency

The interviewer's prompt: "Need a per-request user context accessible from anywhere in the call stack — no parameter passing. Design with ThreadLocal."

Thinking out loud: 1. "ThreadLocal. Set at request start (servlet filter), clear at request end." 2. "With thread pools (Tomcat), threads are reused — must clear to avoid leaking to next request." 3. "For async (CompletableFuture), context doesn't propagate automatically — need explicit handoff or TransmittableThreadLocal."

A solid answer:

```
public class RequestContext {
    private static final ThreadLocal<UserContext> CURRENT = new ThreadLocal<>();

    public static void set(UserContext ctx) { CURRENT.set(ctx); }
    public static UserContext get() { return CURRENT.get(); }
    public static void clear() { CURRENT.remove(); }
}

// In a Servlet filter:
public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain) {
    UserContext ctx = extractFromJwt(req);
    RequestContext.set(ctx);
    try { chain.doFilter(req, res); }
    finally { RequestContext.clear(); } // CRITICAL — prevents leak across requests
}
```

- `remove()` not `set(null)` — `set(null)` keeps the entry; `remove` actually frees the map slot
- For async work, must manually propagate: `var ctx = RequestContext.get();`
`CompletableFuture.runAsync(() -> { RequestContext.set(ctx); ... });`

Tradeoffs: - Memory leak risk: forgetting `remove()` keeps the entry alive for the thread's lifetime (= forever in a pool) - Async pitfalls: `ThreadLocal` doesn't follow your `CompletableFuture` chain. Spring's `MDCCContextMap` and similar libraries help. - Java 21 `ScopedValues` (preview) is the modern replacement — explicit scope, no leak risk

What NOT to say: - "ThreadLocal is automatic with thread pools" — exactly opposite; pools make leaks worse - "Just pass user context as parameter" — fine answer for shallow stacks; `ThreadLocal` shines in deep / framework calls

Common follow-up: "What's the memory leak risk with `ThreadLocal` in a Tomcat thread pool?"

Scenario 75 · Reader-writer lock for read-heavy workload

Asked at: Cred · Razorpay · Adobe GCC · **Difficulty:** Medium · **Topic:** Concurrency

The interviewer's prompt: "Cache with 99% reads, 1% writes. Single ReentrantLock serializes everything. Improve with reader-writer lock."

Thinking out loud: 1. "ReentrantReadWriteLock: many readers in parallel, writers exclusive." 2. "Caveat: write starvation possible — high reader load can keep writers waiting." 3. "For ultra-high read load + occasional writes, StampedLock's optimistic read is faster."

A solid answer:

```
public class StatsCache {
    private final Map<String, Stats> data = new HashMap<>();
    private final ReentrantReadWriteLock lock = new ReentrantReadWriteLock();

    public Stats get(String key) {
        lock.readLock().lock();
        try { return data.get(key); }
        finally { lock.readLock().unlock(); }
    }

    public void put(String key, Stats s) {
        lock.writeLock().lock();
        try { data.put(key, s); }
        finally { lock.writeLock().unlock(); }
    }
}
```

- Many readers in parallel; writers wait for all current readers to release
- `new ReentrantReadWriteLock(true)` enables fairness → writers won't starve, but throughput drops

Tradeoffs: - For very high read throughput, ConcurrentHashMap is simpler and often faster (no lock at all for reads) - StampedLock is even faster but harder to use correctly (optimistic read invalidation) - Read lock to write lock upgrade isn't supported — must release read, acquire write, re-check

What NOT to say: - "Read lock and write lock are interchangeable" — write lock excludes everything; read lock allows other readers - "Use synchronized for both" — single-threaded behavior; defeats the purpose

Common follow-up: "How would StampedLock's optimistic read be even faster here?"

Scenario 76 · StampedLock with optimistic read

Asked at: Razorpay · Cred · Adobe GCC · **Difficulty:** Hard · **Topic:** Concurrency primitives

The interviewer's prompt: "Implement a `Point` class with `x,y` coordinates, very high read throughput. Use `StampedLock`'s optimistic read."

Thinking out loud: 1. "Optimistic read: no lock acquired; read values; verify stamp wasn't invalidated by a write." 2. "If validation fails, fall back to a real read lock." 3. "Useful when writes are rare — 99% of reads complete without any locking."

A solid answer:

```
public class Point {
    private double x, y;
    private final StampedLock lock = new StampedLock();

    public void move(double dx, double dy) {
        long stamp = lock.writeLock();
        try { x += dx; y += dy; }
        finally { lock.unlockWrite(stamp); }
    }

    public double distanceFromOrigin() {
        long stamp = lock.tryOptimisticRead();
        double cx = x, cy = y;
        if (!lock.validate(stamp)) {           // a write happened during read
            stamp = lock.readLock();
            try { cx = x; cy = y; }
            finally { lock.unlockRead(stamp); }
        }
        return Math.sqrt(cx * cx + cy * cy);
    }
}
```

- Capture values into LOCAL variables first; validate AFTER. Don't dereference shared state after validation.
- Validation failure rare in low-write workloads → almost all reads are lock-free

Tradeoffs: - Not reentrant — can't acquire same lock twice in nested calls - Easy to misuse — using `stamp.toString()` inside the read block can deadlock - For complex state (multiple fields with invariants), risk of reading inconsistent snapshot if write happens partway

What NOT to say: - "Same as `ReentrantReadWriteLock`" — optimistic read is the differentiator - "Skip validation if it usually succeeds" — defeats correctness

Common follow-up: "What's the danger of accessing object fields between optimistic-read and validate?"

Scenario 77 · Atomic file operations

Asked at: Razorpay · Postman · Atlassian GCC · **Difficulty:** Medium · **Topic:** Concurrency + I/O

The interviewer's prompt: "Write a config file atomically — readers should never see a half-written file."

Thinking out loud: 1. "Write-then-rename pattern. Writers write to .tmp, then atomically rename to final name." 2. "On POSIX, rename within same filesystem is atomic. fsync the tmp file before rename for crash safety." 3. "Files.move with ATOMIC_MOVE handles cross-platform."

A solid answer:

```
public void writeConfigAtomic(Path target, String content) throws IOException {
    Path tmp = target.resolveSibling(target.getFileName() + ".tmp");
    try (var out = Files.newOutputStream(tmp, CREATE, TRUNCATE_EXISTING, WRITE)) {
        out.write(content.getBytes(StandardCharsets.UTF_8));
        ((FileOutputStream) out).getFD().sync();          // fsync before rename
    }
    Files.move(tmp, target, StandardCopyOption.ATOMIC_MOVE, StandardCopyOption.REPLACE_EXISTING);
}
```

- fsync before rename — crash between write and rename leaves a partial .tmp (cleanable) but never a partial target
- ATOMIC_MOVE: rename is an atomic syscall on same filesystem; throws if cross-filesystem

Tradeoffs: - Cross-filesystem move isn't atomic — falls back to copy+delete. Detect & route around if needed. - Some file systems (NFS) have weaker atomicity guarantees - For multi-file atomicity (config + checksum), use a directory-rename pattern: write to dir-tmp, rename dir

What NOT to say: - "Just write directly to the file" — readers see partial writes during the operation - "Use file locks" — readers using mmap or unlocked reads bypass them

Common follow-up: "How would you do this atomically across MULTIPLE files?"

Scenario 78 · Lock-free stack

Asked at: Cred · Adobe GCC · trading systems · **Difficulty:** Hard · **Topic:** Lock-free programming

The interviewer's prompt: "Build a thread-safe stack without locks — using CAS on AtomicReference."

Thinking out loud: 1. "Treiber stack: head is AtomicReference. Push = CAS head from oldHead to newNode." 2. "Loop until CAS succeeds. ABA problem real if nodes are reused — use

AtomicStampedReference or freshly allocate." 3. "Pop is symmetric: CAS head from oldHead to oldHead.next."

A solid answer:

```
public class TreiberStack<T> {
    record Node<T>(T value, Node<T> next) {}
    private final AtomicReference<Node<T>> head = new AtomicReference<>();

    public void push(T value) {
        Node<T> newHead;
        Node<T> oldHead;
        do {
            oldHead = head.get();
            newHead = new Node<>(value, oldHead);
        } while (!head.compareAndSet(oldHead, newHead));
    }

    public T pop() {
        Node<T> oldHead, newHead;
        do {
            oldHead = head.get();
            if (oldHead == null) return null;
            newHead = oldHead.next();
        } while (!head.compareAndSet(oldHead, newHead));
        return oldHead.value();
    }
}
```

- Allocating fresh Node per push avoids ABA — same node can't reappear
- CAS retry under heavy contention → wasted work but no deadlock

Tradeoffs: - Allocations cost; high-frequency push/pop benefits from pooled nodes — which reintroduces ABA - Under extreme contention, CAS livelock-like behavior — backoff or fall back to lock-based - Java has `ConcurrentLinkedDeque` for production use

What NOT to say: - "Lock-free means faster always" — not true under high contention with cache invalidation - "Use a synchronized stack" — works but kills throughput

Common follow-up: "How does the ABA problem manifest here, and how do you fix it without fresh allocation?"

Scenario 79 · Concurrent linked queue (Michael-Scott)

Asked at: Cred · Adobe GCC · trading systems · **Difficulty:** Hard · **Topic:** Lock-free programming

The interviewer's prompt: "Implement a lock-free FIFO queue. Tail and head are separately CAS'd."

Thinking out loud: 1. "Michael-Scott queue: head and tail are AtomicReference. Enqueue CAS-es tail.next, then advances tail." 2. "Subtle: tail might lag if a concurrent enqueue happens — must help advance it on next op." 3. "For Java production, ConcurrentLinkedQueue already implements this — show you understand the algorithm."

A solid answer:

```
public class MSQueue<T> {
    static class Node<T> {
        T value;
        AtomicReference<Node<T>> next = new AtomicReference<>();
        Node(T value) { this.value = value; }
    }
    private final AtomicReference<Node<T>> head, tail;

    public MSQueue() {
        Node<T> sentinel = new Node<>(null);
        head = new AtomicReference<>(sentinel);
        tail = new AtomicReference<>(sentinel);
    }

    public void enqueue(T value) {
        Node<T> newNode = new Node<>(value);
        while (true) {
            Node<T> t = tail.get();
            Node<T> next = t.next.get();
            if (t != tail.get()) continue; // re-check
            if (next != null) { tail.compareAndSet(t, next); continue; } // help advance
            if (t.next.compareAndSet(null, newNode)) {
                tail.compareAndSet(t, newNode); // try to advance tail
                return;
            }
        }
    }
}
```

- "Helping" pattern: any thread can advance tail if it lags. Otherwise queue could stall.
- Sentinel head node avoids special-casing empty queue

Tradeoffs: - Algorithm is famously tricky to get right — use ConcurrentLinkedQueue in production - Memory reclamation in non-GC languages is painful (hazard pointers, epoch-based) — Java's GC handles this for us - Performance: comparable to lock-based for moderate contention; wins at high contention

What NOT to say: - "Use synchronized" — fine but defeats the exercise - "Just one CAS for enqueue" — misses the tail-lag problem

Common follow-up: "Why is the 'helping' pattern necessary? What goes wrong without it?"

Scenario 80 · Thread-safe singleton (double-checked locking)

Asked at: TCS · Infosys · most Java interviews · **Difficulty:** Medium · **Topic:** Concurrency patterns

The interviewer's prompt: "Implement a thread-safe singleton. Show why double-checked locking needs volatile."

Thinking out loud: 1. "Naive synchronized getInstance() works but locks every call." 2. "Double-checked: check, lock, check again. Volatile required for the field — otherwise JMM allows reordering." 3. "Even better in 2026 — use an enum singleton or holder pattern."

A solid answer:

```
// Modern: holder pattern (best for lazy init)
public class Config {
    private Config() {}
    private static class Holder { static final Config INSTANCE = new Config(); }
    public static Config getInstance() { return Holder.INSTANCE; }
}

// Or: enum (best for serialization & reflection safety)
public enum ConfigEnum {
    INSTANCE;
    public void doWork() { ... }
}

// Classic DCL – useful when constructor takes runtime params
public class ConfigDcl {
    private static volatile ConfigDcl instance;           // volatile MANDATORY
    public static ConfigDcl getInstance() {
        ConfigDcl local = instance;                       // local read for perf
        if (local == null) {
            synchronized (ConfigDcl.class) {
                local = instance;
                if (local == null) {
                    instance = local = new ConfigDcl();
                }
            }
        }
        return local;
    }
}
```

- *Holder pattern: JVM class-loading rules guarantee lazy + thread-safe initialization*
- *Enum: serialization handles single-instance preservation for free*

Tradeoffs: - DCL without volatile is BROKEN — reader can see a partially-constructed object (JMM reordering) - Enum singleton resists reflection attacks; classes with private constructor don't - Don't use singletons for testability — DI is usually better

What NOT to say: - "Double-check without volatile" — broken; one of the classic bugs - "Holder pattern is old-school" — it's the standard recommendation for lazy singletons

Common follow-up: "What goes wrong if you forget `volatile`?"

Scenario 81 · Concurrent map with `computeIfAbsent`

Asked at: Razorpay · Cred · Postman · **Difficulty:** Medium · **Topic:** Concurrent collections

The interviewer's prompt: "Memoize an expensive function. Must be thread-safe — concurrent callers with same key compute it ONCE."

Thinking out loud: 1. "`ConcurrentHashMap.computeIfAbsent` is atomic — function runs at most once per key." 2. "BUT: function is called WHILE holding the bucket lock. If it does anything that re-enters the same map, deadlock." 3. "For long computations, store `CompletableFuture` as value — releases lock immediately."

A solid answer:

```
public class Memoizer<K,V> {
    private final ConcurrentHashMap<K, CompletableFuture<V>> cache = new ConcurrentHashMap<>();
    private final Function<K,V> compute;

    public V get(K key) throws Exception {
        CompletableFuture<V> f = cache.computeIfAbsent(key, k ->
            CompletableFuture.supplyAsync(() -> compute.apply(k))
        );
        return f.get();
    }
}
```

- Storing a Future, not a value — concurrent callers all get the same Future and block on `.get()`
- `compute` is called by ONE thread; others share the result

Tradeoffs: - If compute throws, the Future is stored with the exception. Cache it (and dedupe failures) or remove on failure (and allow retry) - Memory: every key holds a Future forever — for unbounded keys, use Caffeine's LoadingCache - Don't reenter same map inside compute — deadlock risk under heavy use

What NOT to say: - "Use putIfAbsent" — caller still computes the value first; multiple callers each compute - "Synchronized block around get" — kills concurrency across keys

Common follow-up: "What deadlock can happen if compute re-enters the same map?"

Scenario 82 · Compute aggregate with parallel streams (when it goes wrong)

Asked at: Razorpay · Postman · Atlassian GCC · **Difficulty:** Medium · **Topic:** Streams

The interviewer's prompt: "Sum 10M numbers using parallel stream. Then identify pitfalls — when parallel streams hurt."

Thinking out loud: 1. "`list.parallelStream().mapToLong(...).sum()` — splits across `ForkJoinPool.commonPool`." 2. "Pitfalls: shared mutable state in lambdas; common pool starvation; small tasks where overhead > benefit." 3. "Always benchmark — parallel isn't free."

A solid answer:

```
// Good: stateless, side-effect-free
long total = numbers.parallelStream()
    .mapToLong(Long::longValue)
    .sum();

// Bad: shared mutable state – race condition!
List<Integer> result = new ArrayList<>();
input.parallelStream().forEach(result::add);           // ArrayList not thread-safe

// Fix: use collector, not side effects
List<Integer> safeResult = input.parallelStream()
    .collect(Collectors.toList());

// Custom pool – avoid clogging commonPool
ForkJoinPool customPool = new ForkJoinPool(8);
long sum = customPool.submit(() ->
    numbers.parallelStream().mapToLong(Long::longValue).sum()
).get();
```

- `parallelStream` uses `commonPool` — sharing with all your other parallel work
- `forEach` with side effects is a race condition; use collectors

Tradeoffs: - For small N (<10k), parallel overhead exceeds gain - For I/O-bound work, parallelStream blocks commonPool threads — use custom pool - Stateful collectors (groupingBy with non-concurrent map) suffer in parallel — use concurrent variants

What NOT to say: - "Always use parallel for speed" — small data + overhead = slower - "Streams handle thread safety for you" — only for the framework parts; your lambdas must be safe

Common follow-up: "Why is parallel stream bad with the default common pool for I/O-bound work?"

Scenario 83 · CompletableFuture composition pitfalls

Asked at: Cred · Razorpay · Postman · **Difficulty:** Medium · **Topic:** Async programming

The interviewer's prompt: "Chain 3 async API calls; handle errors. Show common mistakes."

Thinking out loud: 1. " `thenApply` runs on the previous stage's thread — may block your pool unexpectedly." 2. " `thenApplyAsync(fn, executor)` lets you pick the pool — explicit is safer." 3. " `exceptionally` vs `handle` — handle gives access to both result AND exception."

A solid answer:

```
public CompletableFuture<Order> createOrder(OrderReq req) {
    return CompletableFuture
        .supplyAsync(() -> userService.load(req.userId()), apiPool)
        .thenComposeAsync(user -> inventory.reserve(user, req.items()), apiPool)
        .thenComposeAsync(reservation -> payment.charge(req.payment(), reservation.total()), apiPool)
        .thenApply(this::buildOrder)
        .handle((order, ex) -> {
            if (ex != null) {
                log.error("Order creation failed", ex);
                throw new OrderException(ex);
            }
            return order;
        });
}
```

- `thenCompose` (`flatMap`) when next stage returns a `Future`
- `handle` vs `exceptionally`: `handle` sees both success and failure; useful for unified logging

Tradeoffs: - `thenApply` (no executor) runs on the previous stage's thread or the caller's — context unpredictable - Always use `*Async` variants with explicit `Executor` in production code - `join()` rethrows wrapped `CompletionException` — annoying for unwrapping. Prefer `get()` with try/catch.

What NOT to say: - "thenApply and thenApplyAsync are the same" — different execution context - "Use get() everywhere to wait for results" — blocks the calling thread, defeats async

Common follow-up: "When would you use thenApply vs thenApplyAsync vs thenCompose?"

Scenario 84 · Virtual threads (Project Loom)

Asked at: Cred · Razorpay · Postman · 2026-aware shops · **Difficulty:** Medium · **Topic:** Modern concurrency

The interviewer's prompt: "Java 21 has virtual threads. Design a high-concurrency request handler using them. When NOT to use them?"

Thinking out loud: 1. "Virtual threads = lightweight, JVM-scheduled. Block freely; not OS thread-backed." 2. "Sweet spot: I/O-bound code that used to need reactive frameworks for scale." 3. "NOT for CPU-bound work — same constraints as OS threads."

A solid answer:

```
// Per-request virtual thread – millions cheap
try (ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (Request req : requests) {
        executor.submit(() -> {
            // Synchronous code – easy to read
            User user = userService.load(req.userId()); // blocking I/O is fine
            List<Order> orders = orderService.fetch(user); // blocking I/O is fine
            return process(user, orders);
        });
    }
}
```

- `newVirtualThreadPerTaskExecutor`: each task gets a fresh virtual thread
- Synchronous code: no reactor / no callback hell; the JVM transparently parks blocked threads

Tradeoffs: - "Pinning": `synchronized` blocks pin the virtual thread to its carrier — concurrency hit. Use `ReentrantLock` instead under load. - Not for CPU-bound work — virtual threads don't add parallelism; same CPU count limit - `ThreadLocal` works but is per virtual thread — millions × `ThreadLocal` can leak memory

What NOT to say: - "Virtual threads make everything faster" — they help I/O concurrency, not CPU throughput - "Synchronized is fine with virtual threads" — pinning is a real production issue

Common follow-up: "What's 'pinning' and why does it hurt virtual thread throughput?"

Scenario 85 · Async cancellation with CompletableFuture

Asked at: Cred · Razorpay · Postman · **Difficulty:** Medium · **Topic:** Async programming

The interviewer's prompt: "Cancel an in-flight CompletableFuture chain on user request. Why is cancel() so weak?"

Thinking out loud: 1. "CompletableFuture.cancel(true) only marks the future as cancelled — does NOT interrupt the underlying work." 2. "For real cancellation, the work must check a flag or be a Future.cancel-aware operation." 3. "Better pattern: pass an AtomicBoolean (or similar) to the worker, set on cancel."

A solid answer:

```
public class CancellableAsync {
    public CompletableFuture<Result> run(Task task) {
        AtomicBoolean cancelled = new AtomicBoolean(false);
        CompletableFuture<Result> future = CompletableFuture.supplyAsync(() -> {
            for (Step step : task.steps()) {
                if (cancelled.get()) throw new CancellationException();
                step.execute();
            }
            return task.result();
        }, taskPool);

        future.whenComplete((r, e) -> {
            if (e instanceof CancellationException) cancelled.set(true);
        });
        return future;
    }
}
```

- Worker polls the flag at safe points; clean cancellation requires cooperation
- cancel() returning true just means "I marked it cancelled"; doesn't guarantee the work stopped

Tradeoffs: - For HTTP downstreams, use a client that cancels in-flight requests (OkHttp.call.cancel()) - Java 21 virtual threads + structured concurrency (preview) make cancellation propagation much cleaner - Cancellation cleanup (releasing locks, partial state) often more complex than the cancel itself

What NOT to say: - "CompletableFuture.cancel() stops the work" — wrong; it just marks the future - "Don't bother with cancellation" — important for user-facing systems

Common follow-up: "How does structured concurrency in Java 21 improve cancellation propagation?"

Section F — OOP Modeling Problems (Scenarios 86–105)

Scenario 86 · Parking lot class hierarchy

Asked at: Razorpay · Cred · Walmart Labs · classic LLD · **Difficulty:** Medium · **Topic:** OO design

The interviewer's prompt: "Design classes for a parking lot — multiple levels, vehicle types, spot types. Park and unpark."

Thinking out loud: 1. "Quick scope-check — do I need pricing? Payment? Reservation? Or just allocate/free a spot?" 2. "Core entities: ParkingLot, Level, Spot, Vehicle. Spot types match vehicle sizes." 3. "Use polymorphism on Vehicle; Strategy pattern for spot allocation."

A solid answer:

```

enum VehicleSize { MOTORBIKE, COMPACT, LARGE }

abstract class Vehicle {
    final String licensePlate;
    final VehicleSize size;
    Vehicle(String p, VehicleSize s) { this.licensePlate = p; this.size = s; }
}
class Motorbike extends Vehicle { Motorbike(String p) { super(p, VehicleSize.MOTORBIKE); } }
class Car extends Vehicle { Car(String p) { super(p, VehicleSize.COMPACT); } }

class Spot {
    final int id;
    final VehicleSize size;
    Vehicle occupant;
    boolean canFit(Vehicle v) { return occupant == null && size.ordinal() >= v.size.ordinal(); }
}

class Level {
    final int id;
    final List<Spot> spots;
    synchronized Optional<Spot> assign(Vehicle v) {
        return spots.stream().filter(s -> s.canFit(v)).findFirst()
            .map(s -> { s.occupant = v; return s; });
    }
}

class ParkingLot {
    final List<Level> levels;
    final Map<String, Spot> activeTickets = new ConcurrentHashMap<>();

    public Optional<Ticket> park(Vehicle v) {
        for (Level lvl : levels) {
            var spot = lvl.assign(v);
            if (spot.isPresent()) {
                Ticket t = new Ticket(v, spot.get(), Instant.now());
                activeTickets.put(v.licensePlate, spot.get());
                return Optional.of(t);
            }
        }
        return Optional.empty();
    }

    public void unpark(String licensePlate) {
        Spot s = activeTickets.remove(licensePlate);
        if (s != null) s.occupant = null;
    }
}

```

- `Spot.canFit` uses ordinal comparison — bigger spot can hold smaller vehicle
- `Level` holds a `synchronized` assign — for high concurrency, switch to per-spot CAS

Tradeoffs / what you'd push back on: - Linear scan for free spot is $O(n)$. For huge lots, maintain a free-spot queue per size - Pricing is its own subsystem — Strategy on `PricingPolicy` (hourly, daily, validated) - Spot reservation (booking ahead) is a different problem — add a `ReservationManager`

What NOT to say: - "Use a giant 2D array" — works but doesn't model real-world hierarchies - "Single class does everything" — interviewer is testing OOP decomposition

Common follow-up: "Now add EV charging spots — a Tesla should prefer those but fall back to compact."

Scenario 87 · Chess game class hierarchy

Asked at: Cred · Postman · Atlassian GCC · LLD interviews · **Difficulty:** Medium · **Topic:** OO design

The interviewer's prompt: "Design classes for a chess game. Pieces, board, valid moves. Whiteboard for 5 minutes."

Thinking out loud: 1. "Each piece type has different move rules → abstract `Piece` + subclass per type with `validMoves()` ." 2. "Board is 8x8 grid of optional `Pieces`. Game state holds whose turn, history, check status." 3. "Skip optimization — focus on the class graph; algorithms come second."

A solid answer:

```

enum Color { WHITE, BLACK }
record Position(int row, int col) {}

abstract class Piece {
    final Color color;
    Position position;
    Piece(Color c, Position p) { this.color = c; this.position = p; }
    abstract List<Position> validMoves(Board board);
}

class Pawn extends Piece {
    List<Position> validMoves(Board board) {
        // forward 1 (or 2 from start), diagonal capture, en-passant
        List<Position> moves = new ArrayList<>();
        int dir = color == Color.WHITE ? 1 : -1;
        Position fwd = new Position(position.row() + dir, position.col());
        if (board.isEmpty(fwd)) moves.add(fwd);
        // ... captures, en-passant, promotion
        return moves;
    }
}

class Board {
    final Piece[][] grid = new Piece[8][8];
    Optional<Piece> at(Position p) { return Optional.ofNullable(grid[p.row()][p.col()]); }
    boolean isEmpty(Position p) { return at(p).isEmpty(); }
}

class Game {
    Board board;
    Color turn = Color.WHITE;
    Deque<Move> history = new ArrayDeque<>();

    public boolean makeMove(Position from, Position to) {
        Optional<Piece> p = board.at(from);
        if (p.isEmpty() || p.get().color != turn) return false;
        if (!p.get().validMoves(board).contains(to)) return false;
        executeMove(from, to);
        if (isCheckmate(opponent(turn))) endGame();
        turn = opponent(turn);
        return true;
    }
}

```

- Each piece encapsulates its own move rules — open-closed principle
- Move history enables undo, draw-by-repetition, and notation export

Tradeoffs: - "Check" detection requires asking "can opponent capture my king?" — recursive validMoves check; can be slow - Castling, en-passant, promotion are special cases that don't fit pure validMoves cleanly - For online play, the validation must run server-side too — clients can be hacked

What NOT to say: - "Use one class with a switch on piece type" — exactly the anti-pattern OOP solves - "Skip the rules, just track positions" — interview wants to see rule modeling

Common follow-up: "How do you check if the current player is in check?"

Scenario 88 · Vending machine state machine

Asked at: TCS · Infosys · Razorpay · LLD interviews · **Difficulty:** Medium · **Topic:** State pattern

The interviewer's prompt: "Design a vending machine. Insert coin, select product, dispense, return change. Handle invalid sequences."

Thinking out loud: 1. "Classic State pattern — IdleState, HasMoneyState, DispensingState." 2. "Each state defines what actions are valid. Invalid action returns an error, no state change." 3. "Show the state graph + the data the machine holds."

A solid answer:

```

abstract class State {
    abstract void insertCoin(VendingMachine m, int amount);
    abstract void selectProduct(VendingMachine m, String code);
    abstract void cancel(VendingMachine m);
}

class IdleState extends State {
    void insertCoin(VendingMachine m, int amount) {
        m.addBalance(amount);
        m.setState(new HasMoneyState());
    }
    void selectProduct(VendingMachine m, String code) {
        throw new IllegalStateException("Insert money first");
    }
    void cancel(VendingMachine m) { /* nothing to cancel */ }
}

class HasMoneyState extends State {
    void insertCoin(VendingMachine m, int amount) { m.addBalance(amount); }
    void selectProduct(VendingMachine m, String code) {
        Product p = m.getProduct(code);
        if (p == null) throw new IllegalArgumentException("No such product");
        if (m.getBalance() < p.price()) throw new IllegalStateException("Insufficient funds");
        if (m.getStock(code) == 0) throw new IllegalStateException("Out of stock");
        m.setState(new DispensingState(p));
    }
    void cancel(VendingMachine m) {
        m.refundBalance();
        m.setState(new IdleState());
    }
}

class VendingMachine {
    private State state = new IdleState();
    private int balance = 0;
    private final Map<String, Product> products;
    private final Map<String, Integer> stock;
    // ... actions delegate to state
}

```

- Each state owns its valid-transitions logic — adding new states doesn't modify existing ones
- *IllegalState** exceptions communicate "wrong sequence" clearly

Tradeoffs: - For a small state machine (3-4 states), an enum with switch is simpler than full State pattern - Concurrent customers: if real (multiple input panels?), state needs lock - Persistence: state has to survive restart — serialize state name + balance

What NOT to say: - "Big if-else on currentState string" — that's procedural; defeats the modeling exercise - "Skip cancellation logic" — interviewer often probes edge cases

Common follow-up: "What if a coin gets stuck mid-dispense? How does the machine recover?"

Scenario 89 · Elevator system

Asked at: Cred · Walmart Labs · LLD interviews · **Difficulty:** Hard · **Topic:** OO + scheduling

The interviewer's prompt: "Design an elevator system for a 50-floor building with 4 elevators. Pickup and destination requests."

Thinking out loud: 1. "Two queues per elevator: pending floors (up direction) and pending floors (down direction). Process in order." 2. "Scheduler decides which elevator gets a pickup — typically 'nearest idle' or 'least loaded along path'." 3. "Avoid greedy = nearest because you might assign an elevator already going away."

A solid answer:


```

enum Direction { UP, DOWN, IDLE }

class Elevator {
    int currentFloor = 0;
    Direction direction = Direction.IDLE;
    NavigableSet<Integer> stopsUp = new TreeSet<>(); // ascending
    NavigableSet<Integer> stopsDown = new TreeSet<>(Comparator.reverseOrder());

    void addStop(int floor) {
        if (floor > currentFloor) stopsUp.add(floor);
        else stopsDown.add(floor);
    }

    void step() {
        if (direction == Direction.UP) {
            if (stopsUp.isEmpty()) { direction = stopsDown.isEmpty() ? Direction.IDLE : Direction.DOWN; }
            int next = stopsUp.first();
            currentFloor = next; stopsUp.remove(next);
            openDoors();
        }
        // mirror for DOWN
    }
}

class ElevatorScheduler {
    List<Elevator> elevators;

    Elevator assign(int pickupFloor, Direction reqDir) {
        // SCAN-style: prefer an elevator already heading toward pickup
        return elevators.stream()
            .min(Comparator.comparingInt(e -> cost(e, pickupFloor, reqDir)))
            .orElseThrow();
    }

    int cost(Elevator e, int pickup, Direction reqDir) {
        if (e.direction == Direction.IDLE) return Math.abs(e.currentFloor - pickup);
        if (e.direction == reqDir && isOnPath(e, pickup)) return Math.abs(e.currentFloor - pickup);
        return 100; // big penalty for "wrong direction"
    }
}

```

- *TreeSet keeps stops sorted — natural "next stop in current direction"*
- *Scheduler heuristic = SCAN (elevator algorithm) — fairness + efficiency*

Tradeoffs: - *Pure nearest-neighbor: greedy and starves rooftop calls - Pure SCAN: fairer, but can have higher wait time when direction changes - Real systems use destination-dispatch: passenger inputs destination at lobby, system groups requests*

What NOT to say: - *"All elevators serve all floors greedily" — leads to multiple elevators converging on same call - "Round-robin among elevators" — ignores position; absurd*

Common follow-up: "How would destination-dispatch (Schindler PORT) change the design?"

Scenario 90 · ATM (state + transactions)

Asked at: Most Indian fintech · LLD interview classic · **Difficulty:** Medium · **Topic:** OO design

The interviewer's prompt: "Design ATM classes. Card insert, PIN auth, balance, withdraw. Atomic withdrawal — never give cash without debiting."

Thinking out loud: 1. "States: NoCardState, EnteringPinState, AuthenticatedState, TransactionInProgressState." 2. "Withdrawal must be atomic — debit account THEN dispense, with rollback if dispense fails." 3. "Pin attempts limited (3), card blocked on exceed."

A solid answer:

```
abstract class AtmState {
    abstract void insertCard(Atm atm, Card c);
    abstract void enterPin(Atm atm, String pin);
    abstract void withdraw(Atm atm, BigDecimal amount);
    abstract void ejectCard(Atm atm);
}

class Atm {
    private AtmState state;
    private Card insertedCard;
    private int failedPinAttempts;
    private final BankService bank;
    private final CashDispenser dispenser;

    public void withdraw(BigDecimal amount) {
        // Atomic two-phase: reserve from account, dispense, confirm
        String reservationId = bank.reserveFunds(insertedCard.accountId(), amount);
        try {
            dispenser.dispense(amount);
            bank.confirmDebit(reservationId);
        } catch (Exception e) {
            bank.releaseReservation(reservationId);
            throw e;
        }
    }
}
```

- Two-phase: reserve → dispense → confirm. Failure releases the reservation, restoring balance.
- 3-strikes-pin lives in EnteringPinState; on 3rd fail, transition to BlockedState + report to bank

Tradeoffs: - True hardware ATM: dispense step can succeed mechanically but fail sensor — manual reconciliation needed - Network failure during confirmDebit: customer has cash, bank doesn't know — eventually consistent, sometimes ops-handled - Audit log every step — regulatory requirement in India

What NOT to say: - "Just debit then dispense" — if dispense fails, customer loses money - "Skip the state machine" — invalid sequences slip through

Common follow-up: "What if the customer takes the cash but the network call to confirm fails?"

Scenario 91 · Library management system

Asked at: TCS · Infosys · LLD interview classic · **Difficulty:** Easy-Medium · **Topic:** OO design

The interviewer's prompt: "Books, members, borrow, return, due-date tracking, fines. Whiteboard the classes."

Thinking out loud: 1. "Books: title + many copies. A Member borrows a Copy, not a Book." 2. "Loan = (Copy, Member, dueDate). Fine derived from days late × rate." 3. "Search by title/author/ISBN — repository layer with indexed queries."

A solid answer:

```

class Book {
    final String isbn, title, author;
    final List<BookCopy> copies = new ArrayList<>();
}
class BookCopy {
    final String id;
    final Book book;
    BookCopyStatus status = BookCopyStatus.AVAILABLE;
}
class Member {
    final String id;
    final String name;
    final List<Loan> activeLoans = new ArrayList<>();
}
class Loan {
    final BookCopy copy;
    final Member borrower;
    final LocalDate borrowedAt;
    final LocalDate dueDate;
    LocalDate returnedAt;

    public BigDecimal fineIfOverdue(LocalDate today, BigDecimal ratePerDay) {
        if (returnedAt != null || !today.isAfter(dueDate)) return BigDecimal.ZERO;
        long daysLate = ChronoUnit.DAYS.between(dueDate, today);
        return ratePerDay.multiply(BigDecimal.valueOf(daysLate));
    }
}

class LibraryService {
    public Loan borrow(Member m, String isbn) {
        if (m.activeLoans.size() >= 5) throw new IllegalStateException("Loan limit");
        BookCopy copy = inventory.findAvailable(isbn)
            .orElseThrow(() -> new IllegalStateException("No copies"));
        copy.status = BookCopyStatus.BORROWED;
        Loan loan = new Loan(copy, m, LocalDate.now(), LocalDate.now().plusDays(14));
        m.activeLoans.add(loan);
        return loan;
    }
}

```

- Loan is the bridge entity — Book and Member don't reference each other directly
- Fine calculation lives on Loan — co-located with the data it depends on

Tradeoffs: - For reservations / waitlists, add Reservation entity with notification on availability - Concurrent borrows of last copy: needs DB-level locking or optimistic concurrency - For large catalogs, search lives in a separate read-optimized index (Elasticsearch)

What NOT to say: - "Member borrows a Book" — wrong; they borrow a specific copy - "Calculate fines in a separate Service class" — fine calc is loan-intrinsic

Common follow-up: "How do you handle reservations for a book where all copies are out?"

Scenario 92 · Snake & Ladder game

Asked at: Cred · Postman · LLD interview classic · **Difficulty:** Easy-Medium · **Topic:** OO modeling

The interviewer's prompt: "Design Snake & Ladder for N players on a 100-cell board."

Thinking out loud: 1. "Board has snakes (head → tail) and ladders (bottom → top) as maps." 2. "Game loops players, rolls dice, applies snakes/ladders, checks winner." 3. "Player position is just an int — keep it simple."

A solid answer:

```

record Snake(int head, int tail) {}
record Ladder(int bottom, int top) {}

class Board {
    final int size;
    final Map<Integer, Integer> jumps;           // start → end (combined snakes + ladders)

    public Board(int size, List<Snake> snakes, List<Ladder> ladders) {
        this.size = size;
        this.jumps = new HashMap<>();
        snakes.forEach(s -> jumps.put(s.head(), s.tail()));
        ladders.forEach(l -> jumps.put(l.bottom(), l.top()));
    }

    public int resolve(int position) {
        return jumps.getOrDefault(position, position);
    }
}

class Game {
    final Board board;
    final Deque<Player> queue;
    final Dice dice = new Dice(6);

    public Player play() {
        while (true) {
            Player p = queue.poll();
            int roll = dice.roll();
            int next = p.position + roll;
            if (next > board.size) { queue.offer(p); continue; } // overshoot – skip
            p.position = board.resolve(next);
            if (p.position == board.size) return p;
            queue.offer(p);
        }
    }
}

```

- Single map merges snakes + ladders — they're symmetric mechanically
- Queue rotation handles N players cleanly

Tradeoffs: - "Roll 6 again" rule: track consecutive sixes; 3 in a row resets player to 0 (variant rule) - Game could be modeled with State pattern for complex variants — overkill for vanilla rules - For UI, expose observables (Listener) so a renderer can subscribe

What NOT to say: - "Separate Snake and Ladder maps" — same behavior; merge - "Use a 2D grid" — position is 1D for game logic

Common follow-up: "Make it pluggable — different board sizes, different rules (rolling 6 grants another turn)."

Scenario 93 · Cab booking system

Asked at: Swiggy · Cred · Walmart Labs · Uber-clone LLD · **Difficulty:** Hard · **Topic:** OO + matching

The interviewer's prompt: "Design classes for a cab booking system. Rider requests, drivers around, matching, ride lifecycle."

Thinking out loud: 1. "Entities: Rider, Driver, Ride, Location. Matching service finds nearest available driver." 2. "Ride lifecycle: REQUESTED → MATCHED → DRIVER_ARRIVING → IN_PROGRESS → COMPLETED. Plus CANCELLED at most stages." 3. "Geospatial search — use a grid (Uber H3) or QuadTree for nearby drivers."

A solid answer:

```

record Location(double lat, double lng) {}
enum RideStatus { REQUESTED, MATCHED, IN_PROGRESS, COMPLETED, CANCELLED }

class Rider { String id; String name; Location current; }
class Driver { String id; String name; Vehicle vehicle; Location current; DriverStatus status; }

class Ride {
    String id;
    Rider rider;
    Driver driver;
    Location pickup, drop;
    RideStatus status;
    Instant requestedAt, startedAt, endedAt;
    BigDecimal fare;
}

class MatchingService {
    private final GeoIndex driverIndex;

    public Optional<Driver> findNearest(Location pickup, double radiusKm) {
        return driverIndex.nearby(pickup, radiusKm).stream()
            .filter(d -> d.status == DriverStatus.AVAILABLE)
            .min(Comparator.comparingDouble(d -> distance(d.current, pickup)));
    }
}

class RideService {
    public Ride request(Rider r, Location drop) {
        Driver d = matching.findNearest(r.current, 5.0)
            .orElseThrow(() -> new IllegalStateException("No drivers nearby"));
        d.status = DriverStatus.OCCUPIED;
        Ride ride = new Ride(/*...*/);
        ride.status = RideStatus.MATCHED;
        return ride;
    }
}

```

- Driver/Rider hold current Location; matching service queries spatial index
- Status transitions are linear — finite state machine

Tradeoffs: - Driver location updates: at $1\text{Hz} \times 100\text{k drivers} = 100\text{k writes/sec}$; needs spatial index that supports fast updates (H3 grid is great) - Matching contention: two riders simultaneously request near same driver — needs atomic claim - For Surge pricing, add a PricingPolicy that knows current supply/demand by zone

What NOT to say: - "Linear scan all drivers" — $O(N)$ per request; dies past 10k drivers - "Driver and Rider extend same class" — they have different fields and lifecycles

Common follow-up: "How do you avoid two riders being matched to the same driver?"

Scenario 94 · Tic-tac-toe with pluggable AI

Asked at: Postman · Cred · interview practice · **Difficulty:** Easy-Medium · **Topic:** OO + Strategy

The interviewer's prompt: "Design tic-tac-toe. Two-player and human-vs-AI. AI difficulty is pluggable."

Thinking out loud: 1. "Board = 3x3 grid. Players take turns marking. Win check on each move." 2.

"Player is abstract — HumanPlayer (asks input), AIPlayer (Strategy)." 3. "AI difficulty = different Strategy implementations: Random, Greedy, Minimax."

A solid answer:

```

abstract class Player {
    final char mark;    // 'X' or 'O'
    Player(char m) { this.mark = m; }
    abstract int[] chooseMove(Board board);
}

class HumanPlayer extends Player {
    private final Scanner in;
    int[] chooseMove(Board board) {
        System.out.println(board); System.out.print("Move (row col): ");
        return new int[]{in.nextInt(), in.nextInt()};
    }
}

interface AiStrategy {
    int[] choose(Board board, char myMark);
}

class RandomAi implements AiStrategy {
    public int[] choose(Board b, char m) {
        List<int[]> empties = b.emptyCells();
        return empties.get(ThreadLocalRandom.current().nextInt(empties.size()));
    }
}

class MinimaxAi implements AiStrategy {
    public int[] choose(Board b, char m) {
        // standard minimax with alpha-beta pruning
        return minimax(b, m, true, Integer.MIN_VALUE, Integer.MAX_VALUE).move;
    }
}

class AiPlayer extends Player {
    private final AiStrategy strategy;
    AiPlayer(char m, AiStrategy s) { super(m); this.strategy = s; }
    int[] chooseMove(Board b) { return strategy.choose(b, mark); }
}

```

- Strategy pattern lets you swap AI difficulty without changing Player class
- Same chooseMove signature → game loop is identical for human or AI

Tradeoffs: - For larger boards (Gomoku 15x15), minimax explodes — switch to Monte Carlo Tree Search - Multiple humans on same machine: HumanPlayer just takes input; abstraction works - Networked play: Player implementation calls remote endpoint — same interface

What NOT to say: - "Two separate game classes for AI and human" — duplicate game logic - "Hardcode AI moves" — strategy lets you A/B test difficulties

Common follow-up: "How does the minimax change when the board is 5x5 instead of 3x3?"

Scenario 95 · Hotel booking domain

Asked at: Razorpay · Cred · travel-tech (Cleartrip) · **Difficulty:** Medium · **Topic:** OO + invariants

The interviewer's prompt: "Design hotel booking. Hotel → Rooms → Availability → Booking. No double-booking same room same dates."

Thinking out loud: 1. "Hotel has many Rooms (by type/category). Room has a list of Bookings (date ranges)." 2. "Booking has roomId, checkIn, checkOut, guest, status." 3. "Atomic check + book → DB unique constraint on (room_id, date) or pessimistic lock during booking."

A solid answer:

```
class Hotel { String id; String name; List<Room> rooms; }
class Room { String id; RoomType type; BigDecimal nightlyRate; }
class Booking {
    String id;
    String roomId;
    String guestId;
    LocalDate checkIn, checkOut;
    BookingStatus status;        // CONFIRMED, CANCELLED, COMPLETED
}

class BookingService {
    @Transactional
    public Booking book(String roomId, String guestId, LocalDate in, LocalDate out) {
        // Pessimistic lock – prevents two concurrent bookings
        Room room = roomRepo.findByIdForUpdate(roomId);
        boolean conflict = bookingRepo.existsOverlapping(roomId, in, out);
        if (conflict) throw new RoomUnavailableException();
        Booking b = new Booking(roomId, guestId, in, out, CONFIRMED);
        bookingRepo.save(b);
        return b;
    }
}

// Or with daterange GIST index (Postgres):
// CREATE UNIQUE INDEX no_overlap ON bookings USING gist
// (room_id, daterange(check_in, check_out, '[')) WITH &&) WHERE status = 'CONFIRMED';
```

- Postgres GIST exclusion constraint = DB enforces "no overlapping bookings" — cheat-proof
- Without DB support, pessimistic lock on Room row is the simplest path

Tradeoffs: - Search ("rooms available 15-Oct to 17-Oct") is a different query path — denormalize to date-bucket index for fast availability scans - Group bookings: 3 rooms for a wedding — must reserve all or

none (saga or DB transaction) - Cancellation policy: partial refund based on lead time — add CancellationPolicy strategy

What NOT to say: - "Check availability, then book in two queries" — race condition between the two - "List all rooms with `availableDates` field" — date arrays don't index well

Common follow-up: "Two users hit /book at the exact same time for the same room. How does only one win?"

Scenario 96 · Pizza ordering (decorator pattern)

Asked at: TCS · Infosys · LLD interview classic · **Difficulty:** Easy · **Topic:** Decorator pattern

The interviewer's prompt: "Design pizza with toppings. Base price + each topping adds cost. Show with Decorator pattern."

Thinking out loud: 1. "Pizza interface with `cost()` and `description()` . BasePizza is a concrete impl." 2. "Topping wraps a Pizza, adds to cost. Stack toppings by wrapping repeatedly." 3. "This is the classic decorator — toppings are runtime-composable, not subclass explosion."

A solid answer:

```

interface Pizza {
    String description();
    BigDecimal cost();
}

class MargheritaBase implements Pizza {
    public String description() { return "Margherita"; }
    public BigDecimal cost() { return new BigDecimal("199"); }
}

abstract class ToppingDecorator implements Pizza {
    protected final Pizza inner;
    ToppingDecorator(Pizza p) { this.inner = p; }
}

class ExtraCheese extends ToppingDecorator {
    ExtraCheese(Pizza p) { super(p); }
    public String description() { return inner.description() + ", Extra Cheese"; }
    public BigDecimal cost() { return inner.cost().add(new BigDecimal("50")); }
}

class Olives extends ToppingDecorator { /* +30 */ }

// Usage:
Pizza p = new Olives(new ExtraCheese(new MargheritaBase()));
System.out.println(p.description() + " = ₹" + p.cost());

```

- Compose at runtime; no subclass per combination
- Each decorator is a tiny class — easy to add new topping

Tradeoffs: - Pure Decorator: lots of small classes. For 30 toppings, that's 30 classes. - Alternative:

List<Topping> field on Pizza, sum costs in cost() — simpler but less "design-patterny" - For pricing rules (combo discounts), inject a PricingPolicy — Decorator alone can't handle "buy 2 toppings, get 1 free"

What NOT to say: - "Subclass for each combination" — combinatorial explosion - "Hardcode topping list as enum" — works but inflexible

Common follow-up: "How would you add a 'buy 2 toppings get 1 free' discount?"

Scenario 97 · BookMyShow seat booking

Asked at: BMS · PayTM · Razorpay · LLD · **Difficulty:** Hard · **Topic:** OO + concurrency

The interviewer's prompt: "Design seat booking for a movie show. Concurrent users selecting same seats. Hold-then-pay flow."

Thinking out loud: 1. "Show has Seats. Booking is a 2-phase: HOLD seats for 5 minutes, then CONFIRM on payment." 2. "Atomic claim: SELECT FOR UPDATE on seats, UPDATE status = HELD with hold_until = now + 5min." 3. "Sweeper releases HELD seats whose hold_until passed."

A solid answer:

```
enum SeatStatus { AVAILABLE, HELD, BOOKED }

class Seat {
    String id;
    String showId;
    String label;           // "A12"
    BigDecimal price;
    SeatStatus status;
    Instant holdUntil;
    String heldBy;
}

class BookingService {
    @Transactional
    public Hold hold(String userId, String showId, List<String> seatIds) {
        List<Seat> seats = seatRepo.findForUpdate(seatIds);    // pessimistic lock
        for (Seat s : seats) {
            if (s.status != SeatStatus.AVAILABLE && !s.heldBy.equals(userId))
                throw new SeatsUnavailableException(s.label);
        }
        Instant holdUntil = Instant.now().plus(5, ChronoUnit.MINUTES);
        seats.forEach(s -> { s.status = HELD; s.heldBy = userId; s.holdUntil = holdUntil; });
        return new Hold(holdId, userId, seatIds, holdUntil);
    }

    @Transactional
    public Booking confirm(String holdId, PaymentInfo payment) {
        Hold h = holdRepo.findById(holdId);
        if (h.holdUntil.isBefore(Instant.now())) throw new HoldExpiredException();
        Payment pay = paymentService.charge(payment, computeTotal(h));
        h.seats.forEach(s -> s.status = BOOKED);
        return bookingRepo.save(new Booking(h.userId, h.seatIds, pay.id));
    }
}

@scheduled(fixedRate = 30_000)
public void releaseExpiredHolds() {
    seatRepo.releaseHoldsBefore(Instant.now());    // sets HELD back to AVAILABLE
}
```

- SELECT FOR UPDATE on the seats locks them for the duration of the hold attempt — atomic

- Sweeper releases stale holds; without it, abandoned holds permanently block seats

Tradeoffs: - Multi-seat hold: all or nothing; partial hold is confusing UX - High contention on popular shows: many concurrent `SELECT FOR UPDATE` → lock waits. Cache "show available count" outside the lock for the UI - Cross-region: usually shows are region-bound; per-region DB simplifies

What NOT to say: - "Allow booking and skip the hold step" — users abandon mid-payment, seats blocked forever - "Use timestamps without sweeper" — stale holds accumulate

Common follow-up: "What if the user pays AFTER the hold expires?"

Scenario 98 · Splitwise / expense split

Asked at: Cred · Splitwise itself · LLD classic · **Difficulty:** Medium · **Topic:** OO modeling

The interviewer's prompt: "Design Splitwise. Users add expenses, split equally or unequally, track who owes whom."

Thinking out loud: 1. "User, Expense, Group. Expense has `paid_by` + splits (per-user share)." 2. "Balance is derived — for each pair, sum of expenses one owes the other." 3. "For settlement, compute the minimum number of transactions (NP-hard in general; heuristic is fine)."

A solid answer:

```

enum SplitType { EQUAL, EXACT, PERCENTAGE }

class Expense {
    String id;
    String paidBy;
    BigDecimal amount;
    SplitType type;
    Map<String, BigDecimal> splits;    // userId → share
    Instant createdAt;

    public Expense(String paidBy, BigDecimal amount, SplitType type, Map<String, BigDecimal> raw) {
        this.paidBy = paidBy;
        this.amount = amount;
        this.type = type;
        this.splits = switch (type) {
            case EQUAL -> equalize(raw.keySet(), amount);
            case EXACT -> validateExact(raw, amount);
            case PERCENTAGE -> percentToAmounts(raw, amount);
        };
    }
}

class BalanceSheet {
    // balances.get("A").get("B") = +50 means A owes B 50
    private final Map<String, Map<String, BigDecimal>> balances = new HashMap<>();

    public void apply(Expense e) {
        for (var entry : e.splits.entrySet()) {
            String debtor = entry.getKey();
            if (debtor.equals(e.paidBy)) continue;
            BigDecimal amt = entry.getValue();
            owes(debtor, e.paidBy, amt);
        }
    }

    private void owes(String from, String to, BigDecimal amt) {
        balances.computeIfAbsent(from, k -> new HashMap<>())
            .merge(to, amt, BigDecimal::add);
    }

    public BigDecimal netBetween(String a, String b) {
        BigDecimal aOwesB = balances.getOrDefault(a, Map.of()).getOrDefault(b, ZERO);
        BigDecimal bOwesA = balances.getOrDefault(b, Map.of()).getOrDefault(a, ZERO);
        return aOwesB.subtract(bOwesA);    // negative = b owes a
    }
}

```

- Net balance = directional; cancel out reciprocal debts
- For simplify-debts, see follow-up — that's the NP-hard part

Tradeoffs: - *BigDecimal for money (never double)* — *rounding matters* - *Floating-point splits (33.33 + 33.33 + 33.34)* — *pick one user to absorb the rounding* - *Currency: enforce single currency per expense; cross-currency needs FX-rate snapshot*

What NOT to say: - *"Store balance as double"* — *floating-point errors on every operation* - *"Pre-compute all balances"* — *recompute is cheap; storage is just expenses*

Common follow-up: *"Simplify debts: A owes B 50, B owes C 50. Reduce to A owes C 50."*

Scenario 99 · Logger framework

Asked at: *Razorpay · Cred · LLD* · **Difficulty:** *Medium* · **Topic:** *OO + Chain of Responsibility*

The interviewer's prompt: *"Design a logger. Levels (DEBUG, INFO, WARN, ERROR). Multiple outputs (console, file, network). Configurable filters per output."*

Thinking out loud: 1. *"LogLevel enum. LogMessage carries level + content + context. Appenders are the outputs."* 2. *"Each Appender has its own min level filter. A log call goes to ALL matching appenders."* 3. *"Chain-of-responsibility OR observer pattern — appenders subscribed to logger."*

A solid answer:

```

enum Level { DEBUG, INFO, WARN, ERROR;
    boolean atLeast(Level other) { return this.ordinal() >= other.ordinal(); }
}

record LogMessage(Level level, String message, Map<String, String> context, Instant ts) {}

interface Appender {
    Level minLevel();
    void append(LogMessage msg);
}

class ConsoleAppender implements Appender {
    private final Level min;
    public Level minLevel() { return min; }
    public void append(LogMessage msg) { System.out.println(format(msg)); }
}

class FileAppender implements Appender { /* writes to BufferedWriter */ }
class NetworkAppender implements Appender { /* POSTs to a remote endpoint */ }

class Logger {
    private final List<Appender> appenders = new CopyOnWriteArrayList<>();

    public void log(Level level, String msg) {
        LogMessage lm = new LogMessage(level, msg, MDC.copy(), Instant.now());
        for (Appender a : appenders) {
            if (lm.level().atLeast(a.minLevel())) a.append(lm);
        }
    }
}

```

- Each appender filters independently — console can be *DEBUG* while network is *ERROR*-only
- MDC (mapped diagnostic context) = *ThreadLocal* for request-scoped key-value pairs

Tradeoffs: - Synchronous append blocks the caller — for high-throughput, async appender with a queue
 - Lossy queues for telemetry are usually fine; lossy queues for audit logs are not - For structured logging (JSON), the format step is per-appender (text vs JSON)

What NOT to say: - "One global *PrintStream*" — can't filter, can't route - "Use *System.out.println* everywhere" — fine for prototypes; production needs structure

Common follow-up: "How would you make appenders async without losing messages?"

Scenario 100 · Stock trading order book

Asked at: Razorpay (capital) · Zerodha · trading-tech · **Difficulty:** Hard · **Topic:** OO + data structures

The interviewer's prompt: "Design an order book. Buy and sell orders. Match best bid with best ask. Whiteboard the matching engine."

Thinking out loud: 1. "Buy side: priority queue keyed by max price. Sell side: priority queue keyed by min price." 2. "On new order: if it crosses the spread, match with best opposite; remainder enters the book." 3. "Within same price level, FIFO (price-time priority)."

A solid answer:

```
enum Side { BUY, SELL }
class Order {
    String id;
    Side side;
    BigDecimal price;
    int quantity;
    Instant ts;
}

class OrderBook {
    // Buy: highest price first → reverse order
    private final TreeMap<BigDecimal, Queue<Order>> bids = new TreeMap<>(Comparator.reverseOrder());
    // Sell: lowest price first
    private final TreeMap<BigDecimal, Queue<Order>> asks = new TreeMap<>();

    public List<Trade> place(Order incoming) {
        TreeMap<BigDecimal, Queue<Order>> opposite = incoming.side == BUY ? asks : bids;
        List<Trade> trades = new ArrayList<>();
        while (incoming.quantity > 0 && !opposite.isEmpty()) {
            var bestPrice = opposite.firstKey();
            if (incoming.side == BUY && incoming.price.compareTo(bestPrice) < 0) break;
            if (incoming.side == SELL && incoming.price.compareTo(bestPrice) > 0) break;
            Queue<Order> q = opposite.get(bestPrice);
            Order resting = q.peek();
            int matchQty = Math.min(incoming.quantity, resting.quantity);
            trades.add(new Trade(incoming, resting, matchQty, bestPrice));
            incoming.quantity -= matchQty;
            resting.quantity -= matchQty;
            if (resting.quantity == 0) {
                q.poll();
                if (q.isEmpty()) opposite.remove(bestPrice);
            }
        }
        if (incoming.quantity > 0) addToBook(incoming);
        return trades;
    }
}
```

- *TreeMap gives $O(\log n)$ best-price lookup; queue per price level gives FIFO time priority*
- *Match loop continues until incoming filled or no more crossing prices*

Tradeoffs: - For very high throughput, replace *TreeMap* with skip list or array-of-levels indexed by price - Cancel-by-orderId: need orderId → level reverse map for $O(\log n)$ cancel - Real exchanges use lock-free or single-writer-thread designs for microsecond matching

What NOT to say: - "Iterate the whole book to find a match" — $O(n)$ per place; can't match 100k orders/sec - "Sort orders on every place" — $O(n \log n)$; use ordered map

Common follow-up: "How do you support order cancellation by id?"

Scenario 101 · Notification system (multi-channel)

Asked at: Razorpay · Swiggy · Meesho · **Difficulty:** Medium · **Topic:** OO + Strategy

The interviewer's prompt: "User can opt into email, SMS, push, WhatsApp. Single notification triggers all chosen channels. Design."

Thinking out loud: 1. "Strategy pattern: Channel interface, one impl per channel." 2. "NotificationService takes user prefs, picks channels, fans out." 3. "Async dispatch — slow channel shouldn't block the others."

A solid answer:

```
enum ChannelType { EMAIL, SMS, PUSH, WHATSAPP }

interface Channel {
    ChannelType type();
    void send(String userId, NotificationPayload payload);
}

class EmailChannel implements Channel { /* SMTP */ }
class SmsChannel implements Channel { /* Twilio */ }
class PushChannel implements Channel { /* FCM */ }
class WhatsAppChannel implements Channel { /* MSG91 / Gupshup */ }

class NotificationService {
    private final Map<ChannelType, Channel> channels;
    private final UserPrefsService prefs;
    private final ExecutorService dispatcher;

    public void notify(String userId, NotificationPayload payload) {
        Set<ChannelType> opted = prefs.getOptedChannels(userId);
        for (ChannelType type : opted) {
            Channel ch = channels.get(type);
            dispatcher.submit(() -> {
                try { ch.send(userId, payload); }
                catch (Exception e) { metrics.failure(type, e); }
            });
        }
    }
}
```

- Per-channel strategy → easy to add WhatsApp without touching email code
- Async submit → email outage doesn't slow down SMS

Tradeoffs: - Per-channel retries: SMS retry policy ≠ email retry policy. Move retry inside each Channel. - Templating: don't hardcode templates per channel; use a TemplateRenderer (Channel + Locale + EventType → text) - Compliance: DND list, opt-out per channel — enforce centrally before dispatch

What NOT to say: - "One if/else for each channel type" — adding new channel modifies central code - "Send sequentially" — slow channels delay others

Common follow-up: "How do you handle a partial failure — email succeeds, SMS fails?"

Scenario 102 · Stack with min() in O(1)

Asked at: Most product cos · **LeetCode** 155 · **Difficulty:** Medium · **Topic:** Data structures

The interviewer's prompt: "Implement a stack with push, pop, top, and `min` — all in $O(1)$."

Thinking out loud: 1. "Auxiliary stack tracking running min. Push: also push $\min(\text{current min}, \text{new value})$. Pop: pop both." 2. "Or single-stack with delta encoding — saves space." 3. "For the interview, two-stack is clearer."

A solid answer:

```
public class MinStack {
    private final Deque<Integer> stack = new ArrayDeque<>();
    private final Deque<Integer> mins = new ArrayDeque<>();

    public void push(int v) {
        stack.push(v);
        mins.push(mins.isEmpty() ? v : Math.min(mins.peek(), v));
    }

    public int pop() {
        mins.pop();
        return stack.pop();
    }

    public int top() { return stack.peek(); }
    public int min() { return mins.peek(); }
}
```

- Both stacks grow together — pop in lockstep
- `mins.peek()` is always the min of all elements currently in stack

Tradeoffs: - 2x memory overhead. Single-stack with delta encoding (store diff from previous min) cuts memory but harder to read. - For max-min in a sliding window, switch to monotonic deque

What NOT to say: - "Scan stack on `min()` call" — that's $O(n)$, defeats the requirement - "Use `PriorityQueue`" — not LIFO; doesn't match stack semantics

Common follow-up: "Same but for a queue — `getMin` in $O(1)$."

Scenario 103 · LRU + LFU hybrid (W-TinyLFU)

Asked at: Cred · Netflix GCC · cache-aware roles · **Difficulty:** Hard · **Topic:** Advanced caching

The interviewer's prompt: "Caffeine uses W-TinyLFU. Explain the design at a high level."

Thinking out loud: 1. "Pure LFU has cold-start; pure LRU misses on scan workloads. Combine: small window (LRU) + main (LFU)." 2. "Admission filter (TinyLFU sketch) compares incoming key's frequency to victim's — only admit if more frequent." 3. "Sketch the 3-part structure: window, probation, protected."

A solid answer: - Window cache (~1% of total): LRU. Catches the cold start; new entries always go here. - Main cache (~99%): SLRU (probation + protected). LFU-driven admission. - Frequency sketch (Count-Min Sketch with aging): tracks recent frequency cheaply.

```
public class TinyLfuCache<K,V> {
    private final LinkedHashMap<K,V> window;           // 1% LRU
    private final LinkedHashMap<K,V> probation;        // 20% LRU
    private final LinkedHashMap<K,V> protect;         // 80% LRU
    private final FrequencySketch<K> sketch;          // CMS with aging

    public V get(K k) {
        sketch.increment(k);
        V v = protect.get(k);
        if (v != null) return v;
        v = probation.remove(k);
        if (v != null) { promoteToProtected(k, v); return v; }
        v = window.get(k);
        return v;
    }

    public void put(K k, V v) {
        window.put(k, v);
        if (window.size() > windowCap) {
            Map.Entry<K,V> evicted = removeOldest(window);
            tryAdmit(evicted.getKey(), evicted.getValue());
        }
    }

    private void tryAdmit(K candidate, V v) {
        if (probation.size() < probationCap) { probation.put(candidate, v); return; }
        K victim = oldestKey(probation);
        if (sketch.frequency(candidate) > sketch.frequency(victim)) {
            probation.remove(victim);
            probation.put(candidate, v);
        }
    }
}
```

- Sketch uses ~4 bits per counter — fixed memory regardless of cardinality
- Aging halves all counters periodically to prevent old hits from dominating

Tradeoffs: - Complex — for most use cases, Caffeine's library is the answer, not a hand-roll - Hit rate beats LRU by ~5-10% on real workloads; worth it for large / expensive caches - For purely temporal workloads (recent always wins), simple LRU is comparable

What NOT to say: - "LFU is always better than LRU" — both lose to combined adaptive policies - "Just use a HashMap" — interviewer wants to hear cache policy understanding

Common follow-up: "What does 'aging' mean in the Count-Min Sketch context?"

Scenario 104 · Inventory deduction with reservations

Asked at: Swiggy · Meesho · Flipkart · Razorpay (D2C) · **Difficulty:** Hard · **Topic:** OO + concurrency

The interviewer's prompt: "E-commerce: 10 stock left, 50 users add to cart. Don't oversell. Design the inventory model."

Thinking out loud: 1. "Soft reserve on add-to-cart (with TTL), hard deduct on checkout." 2. "Atomic check-and-decrement at SQL level: `UPDATE inventory SET available = available - 1 WHERE sku = ? AND available >= 1`." 3. "For high contention (flash sales), pre-allocate stock chunks per region/pod."

A solid answer:


```

class Inventory {
    String sku;
    int available;           // current free count
    int reserved;           // soft-reserved (in carts)
}

class InventoryService {
    @Transactional
    public boolean reserve(String sku, int qty, String cartId, Duration ttl) {
        int affected = jdbc.update(
            "UPDATE inventory SET reserved = reserved + ? " +
            "WHERE sku = ? AND (available - reserved) >= ?",
            qty, sku, qty);
        if (affected == 0) return false;
        reservationRepo.save(new Reservation(cartId, sku, qty,
            Instant.now().plus(ttl)));
        return true;
    }

    @Transactional
    public boolean confirm(String cartId) {
        Reservation r = reservationRepo.findActive(cartId);
        if (r == null) return false;
        jdbc.update("UPDATE inventory SET available = available - ?, reserved = reserved + ? WHERE sku = ?",
            r.qty(), r.qty(), r.sku());
        reservationRepo.delete(r);
        return true;
    }
}

@scheduled(fixedRate = 60_000)
public void releaseExpired() {
    List<Reservation> expired = reservationRepo.findExpired();
    for (Reservation r : expired) {
        jdbc.update("UPDATE inventory SET reserved = reserved - ? WHERE sku = ?", r.qty(), r.sku());
        reservationRepo.delete(r);
    }
}
}

```

- Atomic UPDATE with WHERE clause = no overselling, no race
- Reservation TTL releases stock if user abandons cart

Tradeoffs: - Single-row contention for hot SKUs (flash sale) — split into N shards per SKU and decrement a random shard - "Phantom" oversell from poorly-set isolation levels — READ COMMITTED is enough with the WHERE clause - For multi-warehouse, inventory is per warehouse + SKU; allocation picks nearest with stock

What NOT to say: - "Check available, then decrement" — race condition - "Lock the whole inventory table" — kills throughput

Common follow-up: "Flash sale: 10,000 buy in 1 second. How do you not have all writes hit the same row?"

Scenario 105 · Coupon / promotion engine

Asked at: Razorpay · Swiggy · Meesho · Cred · **Difficulty:** Medium · **Topic:** OO + rules engine

The interviewer's prompt: "Design coupon engine. Flat ₹100 off, % off, BOGO, first-order-only, category-specific. Whiteboard the abstraction."

Thinking out loud: 1. "Coupon has Conditions (when applicable) + Action (what discount)." 2. "Apply: evaluate conditions on Cart; if all pass, compute discount via Action." 3. "Strategy pattern for conditions and actions — open for new types."

A solid answer:

```

interface Condition {
    boolean check(Cart cart, User user);
}
class MinCartValueCondition implements Condition {
    BigDecimal min;
    public boolean check(Cart c, User u) { return c.total().compareTo(min) >= 0; }
}
class FirstOrderCondition implements Condition {
    public boolean check(Cart c, User u) { return u.orderCount() == 0; }
}
class CategoryCondition implements Condition {
    Set<String> categories;
    public boolean check(Cart c, User u) {
        return c.items().stream().anyMatch(i -> categories.contains(i.category()));
    }
}

interface DiscountAction {
    BigDecimal compute(Cart cart);
}
class FlatDiscount implements DiscountAction {
    BigDecimal amount;
    public BigDecimal compute(Cart c) { return c.total().min(amount); }
}
class PercentDiscount implements DiscountAction {
    BigDecimal percent;
    BigDecimal cap;
    public BigDecimal compute(Cart c) { return c.total().multiply(percent).min(cap); }
}

class Coupon {
    String code;
    List<Condition> conditions;
    DiscountAction action;
    LocalDate validUntil;

    public Optional<BigDecimal> apply(Cart cart, User user) {
        if (LocalDate.now().isAfter(validUntil)) return Optional.empty();
        if (!conditions.stream().allMatch(c -> c.check(cart, user))) return Optional.empty();
        return Optional.of(action.compute(cart));
    }
}

```

- All conditions must pass (AND semantics) — for OR, wrap in CompositeCondition
- Discount cap on percent prevents "50% off ₹100k = ₹50k" surprises

Tradeoffs: - Coupon stacking: most engines allow only one coupon per order; mixing requires conflict rules - Audit: log every coupon application with reason — vital for ops + chargebacks - For complex rules, consider a rules engine (Drools) — usually overkill

What NOT to say: - "Hardcode each coupon type as a class" — combinatorial explosion (flat + first-order + category) - "Big if-else inside Coupon.apply" — hard to extend; OO design exists to avoid this

Common follow-up: "How do you prevent users from stacking 5 coupons for a free order?"

Section G — API & Service Design (Scenarios 106–125)

Scenario 106 · Feature flag system

Asked at: Razorpay · Cred · Postman · Atlassian GCC · **Difficulty:** Medium · **Topic:** Platform

The interviewer's prompt: "Design a feature flag system. Boolean flags + % rollout + per-user override + tied-to-environment."

Thinking out loud: 1. "Flag = (name, defaultValue, rolloutPercent, userOverrides). FlagService.isOn(name, user) returns true/false." 2. "Rollout: hash(userId + flagName) % 100 < rolloutPercent — stable per user." 3. "Config in Redis / hosted (LaunchDarkly); cached locally with short TTL."

A solid answer:

```
record Flag(String name, boolean defaultValue, int rolloutPercent,
            Set<String> userAllowlist, Set<String> userBlocklist) {}

class FlagService {
    private final FlagStore store;
    private final Cache<String, Flag> cache = Caffeine.newBuilder()
        .expireAfterWrite(30, TimeUnit.SECONDS).build();

    public boolean isOn(String name, String userId) {
        Flag f = cache.get(name, store::loadFlag);
        if (f == null) return false;
        if (f.userBlocklist().contains(userId)) return false;
        if (f.userAllowlist().contains(userId)) return true;
        if (f.rolloutPercent() == 0) return f.defaultValue();
        if (f.rolloutPercent() == 100) return true;
        int bucket = Math.abs((name + userId).hashCode()) % 100;
        return bucket < f.rolloutPercent();
    }
}
```

- Hash uses flag name + user id → user is consistent within a flag, but a "10% rollout" of flag A and flag B targets different 10%
s
- 30s cache TTL — operator changes propagate within 30s; almost no Redis traffic

Tradeoffs: - Local cache means inconsistency window: pod A may have new flag value while pod B has old for up to 30s - For instant kill-switch, support both polling AND push (SSE / WebSocket from flag service) - A/B experimentation needs sticky assignment + event logging — beyond simple flag

What NOT to say: - "Check the flag config on every request" — Redis call per check kills latency - "Use the `userId modulo 100`" — same user is always in the same 10% across all flags, defeating randomization

Common follow-up: "How do you do instant kill-switch instead of waiting for the cache TTL?"

Scenario 107 · Pagination API

Asked at: Most backend roles · **Difficulty:** Medium · **Topic:** API design

The interviewer's prompt: "Design pagination for `/api/orders`. Compare offset vs cursor. Which would you pick?"

Thinking out loud: 1. "Offset: `?page=5&size=20` — simple but slow at high pages and breaks on inserts." 2. "Cursor: `?cursor=abc` — opaque token = (`lastId + lastSortValue`). Stable, fast." 3. "For UI with page numbers, offset (with caveats). For infinite scroll, cursor."

A solid answer:

```
record CursorPage<T>(List<T> items, String nextCursor) {}

public CursorPage<Order> listOrders(String userId, String cursor, int limit) {
    OrderId after = cursor == null ? null : decodeCursor(cursor);
    List<Order> items = orderRepo.findById(userId, after, limit + 1);
    boolean hasMore = items.size() > limit;
    if (hasMore) items = items.subList(0, limit);
    String next = hasMore ? encodeCursor(items.get(items.size() - 1).id()) : null;
    return new CursorPage<>(items, next);
}

// SQL (Postgres):
// SELECT * FROM orders
// WHERE user_id = ? AND (created_at, id) < (?, ?)
// ORDER BY created_at DESC, id DESC LIMIT ?
```

- Cursor = (`createdAt, id`) pair — handles ties on `createdAt`

- Fetch limit+1 to detect "has next page" without a separate count query

Tradeoffs: - Cursor doesn't support "jump to page 50" — fine for feeds, bad for admin tables - Offset gives total count for free; cursor needs separate count call (or estimate) - Cursor must encode all sort fields, not just id — otherwise sort changes break it

What NOT to say: - "Always use offset/limit" — at high offsets, DB scans + skips everything before - "Cursor is always better" — UI page numbers require offset

Common follow-up: "What goes wrong with offset pagination at page 10,000?"

Scenario 108 · Bulk API endpoint

Asked at: Razorpay · Cred · Postman · **Difficulty:** Medium · **Topic:** API design

The interviewer's prompt: "Client wants to fetch 1000 user profiles. Don't make 1000 HTTP calls. Design /users/batch."

Thinking out loud: 1. "POST /users/batch with body {userIds: [...]}. Returns array of profiles." 2. "Per-element error handling: one bad id shouldn't fail the batch." 3. "Cap batch size (100? 500?) to bound memory and avoid timeouts."

A solid answer:

```
@PostMapping("/users/batch")
public BatchResp<UserProfile> batchGet(@RequestBody BatchReq req) {
    if (req.ids().size() > 500) throw new BadRequestException("max 500");
    Map<String, UserProfile> found = userRepo.findAllById(req.ids()).stream()
        .collect(Collectors.toMap(UserProfile::id, Function.identity()));
    List<BatchItem<UserProfile>> results = new ArrayList<>();
    for (String id : req.ids()) {
        UserProfile p = found.get(id);
        results.add(p != null
            ? BatchItem.ok(id, p)
            : BatchItem.error(id, "NOT_FOUND", "User not found"));
    }
    return new BatchResp<>(results);
}

record BatchItem<T>(String id, T data, String errorCode, String errorMessage) {
    static <T> BatchItem<T> ok(String id, T data) { return new BatchItem<>(id, data, null, null); }
    static <T> BatchItem<T> error(String id, String code, String msg) { return new BatchItem<>(id,
```

- *Per-item error keeps the batch atomic from the API perspective — caller doesn't have to retry the whole thing*
- *Order of results matches order of request ids (or include id in each result)*

Tradeoffs: - *POST for read endpoint — pragmatic; GET with 1000 IDs in query string overflows URL limits - Idempotency: GET is implicitly idempotent; POST batch GET is too (no side effects) - For very large batches (10k+), consider async (return job ID, poll for result)*

What NOT to say: - *"All-or-nothing" — one bad id wastes the whole batch - "Use GET with very long URL" — URL length limits (~2k chars) make this fragile*

Common follow-up: *"What if 50% of the IDs are bad — should the response status be 200 or 207 Multi-Status?"*

Scenario 109 · API versioning strategy

Asked at: *Razorpay · Cred · Postman* · **Difficulty:** *Medium* · **Topic:** *API design*

The interviewer's prompt: *"Design API versioning. v1 in production; v2 needs new fields. How do clients migrate?"*

Thinking out loud: 1. *"URL path (/v1/orders), header (Accept-Version), or query param (?v=1) — pick one and document."* 2. *"Most public APIs use URL path — most discoverable, easiest to debug."* 3. *"v1 stays running indefinitely (or until announced sunset). v2 is additive."*

A solid answer:

```

@RestController
@RequestMapping("/v1/orders")
class OrderControllerV1 {
    @GetMapping("/{id}")
    public OrderV1 get(@PathVariable String id) {
        return OrderMapper.toV1(orderService.find(id));
    }
}

@RestController
@RequestMapping("/v2/orders")
class OrderControllerV2 {
    @GetMapping("/{id}")
    public OrderV2 get(@PathVariable String id) {
        return OrderMapper.toV2(orderService.find(id)); // same domain, different DTO
    }
}

// DTO V1: fields {id, amount}
// DTO V2: fields {id, amount, taxAmount, discountApplied}

```

- Domain layer stays single; only DTO + endpoint differ per version
- Adapters bridge: changes to domain push to all active versions automatically

Tradeoffs: - N versions = $N \times$ maintenance. Sunset old versions on a schedule (6-12 months notice) - Breaking changes always bump version; additive changes (new optional fields) can ship within same version - Internal microservice-to-microservice APIs use semantic versioning (semver) and protobuf for evolution

What NOT to say: - "Always backward compatible — never version" — adding fields fine; renaming fields breaks clients - "Bump version on every release" — pointless churn for clients

Common follow-up: "Header-based versioning (Accept: application/vnd.razorpay.v2+json) vs URL path — pros and cons."

Scenario 110 · Webhook delivery system

Asked at: Razorpay · Cred · Stripe-style · **Difficulty:** Hard · **Topic:** Reliable delivery

The interviewer's prompt: "Customer registers callback URL. Events fire → deliver POST with signature, retry on failure."

Thinking out loud: 1. "Event → enqueue webhook job → worker delivers → mark delivered or schedule retry." 2. "Retry: exponential backoff (1s, 5s, 30s, 5m, 30m, 2h)." 3. "After N attempts (e.g. 8), DLQ + alert customer via email/dashboard."

A solid answer:

```
class WebhookSender {
    public void deliver(WebhookJob job) {
        String body = job.payload();
        String signature = hmacSha256(body, customerSecret(job.customerId()));
        try {
            HttpResponse<String> resp = http.send(HttpRequest.newBuilder()
                .uri(URI.create(job.url()))
                .timeout(Duration.ofSeconds(10))
                .header("X-Signature", signature)
                .header("X-Idempotency-Key", job.id())
                .POST(BodyPublishers.ofString(body))
                .build(), BodyHandlers.ofString());
            if (resp.statusCode() >= 200 && resp.statusCode() < 300) {
                store.markDelivered(job.id());
            } else {
                scheduleRetry(job, resp.statusCode());
            }
        } catch (Exception e) {
            scheduleRetry(job, -1);
        }
    }

    private void scheduleRetry(WebhookJob job, int statusCode) {
        if (job.attempt() >= 8) {
            store.moveToDlq(job.id(), statusCode);
            alerter.notifyCustomer(job.customerId(), "Webhook delivery failed", job.url());
            return;
        }
        long delaySec = (long) Math.pow(4, job.attempt());
        store.scheduleRetry(job.id(), Instant.now().plusSeconds(delaySec), job.attempt() + 1);
    }
}
```

- HMAC signature lets customers verify origin; idempotency key lets them dedup
- DLQ + alert closes the loop — customer knows when delivery failed permanently

Tradeoffs: - Retry pool size: too small = backed up DLQs; too big = hammering slow customer endpoints
 - Customer's endpoint goes down for an hour → all events for that hour pile up and retry burst on recovery — needs rate limiting per customer - For high-throughput, separate workers per customer to prevent one bad URL from blocking others

What NOT to say: - "Fire and forget" — silent failures break customer integrations - "Single retry then give up" — transient blips are common; need at least 6-8 attempts over ~hours

Common follow-up: "Customer's URL is down for 6 hours. How do you not bombard them on recovery?"

Scenario 111 · Signed URLs for downloads

Asked at: Razorpay · Postman · file-heavy services · **Difficulty:** Medium · **Topic:** Security

The interviewer's prompt: "User clicks download. File lives in S3. Don't expose S3 directly; don't proxy through your servers. Design."

Thinking out loud: 1. "Pre-signed URLs (AWS SDK): server generates time-limited URL with embedded signature, client downloads direct from S3." 2. "TTL short (5 min). User shouldn't share or reuse beyond their session." 3. "For DIY (no cloud SDK): URL + HMAC of (path, expires) — server validates on serving."

A solid answer:

```
// Using AWS SDK
public String presignedDownload(String key) {
    GetObjectPresignRequest req = GetObjectPresignRequest.builder()
        .signatureDuration(Duration.ofMinutes(5))
        .getObjectRequest(GetObjectRequest.builder().bucket(BUCKET).key(key).build())
        .build();
    return s3Presigner.presignGetObject(req).url().toString();
}

// DIY equivalent
public String signedUrl(String path, Duration ttl) {
    long expires = Instant.now().plus(ttl).getEpochSecond();
    String toSign = path + ":" + expires;
    String sig = base64Url(hmacSha256(toSign, secretKey));
    return baseUrl + path + "?expires=" + expires + "&sig=" + sig;
}

@GetMapping("/download/{path}")
public ResponseEntity<Resource> download(@PathVariable String path,
    @RequestParam long expires, @RequestParam String sig) {
    if (Instant.now().getEpochSecond() > expires) return ResponseEntity.status(403).build();
    String expected = base64Url(hmacSha256(path + ":" + expires, secretKey));
    if (!MessageDigest.isEqual(expected.getBytes(), sig.getBytes()))
        return ResponseEntity.status(403).build();
    return ResponseEntity.ok(loadResource(path));
}
```

- HMAC uses server-side secret — client can't forge URLs
- Constant-time comparison (`MessageDigest.isEqual`) — prevents timing attacks

Tradeoffs: - Short TTL: too short = bad UX (download starts then fails); too long = link-sharing leakage - Per-user binding (include userId in signature) for stronger access control - Revocation: hard with stateless signing — for revocable, maintain a server-side "valid signatures" set

What NOT to say: - "Proxy through the app" — bandwidth + memory burden on your servers - "Make S3 public" — security disaster

Common follow-up: "What if the user shares the signed URL — how do you detect / prevent?"

Scenario 112 · GraphQL N+1 problem

Asked at: Postman · Cred · GraphQL shops · **Difficulty:** Medium · **Topic:** API patterns

The interviewer's prompt: "Query `{users{id, posts{title}}}` . Naive resolver runs 1 query for users + N for posts. Design DataLoader."

Thinking out loud: 1. "DataLoader pattern: batch + cache requests within a single graph execution." 2. "Per-request DataLoader". Each resolver calls `.load(id)`; batched lookup runs once." 3. "Cache scope = request — not application — so we don't serve stale data."

A solid answer:

```
class PostsByUserDataLoader extends DataLoader<String, List<Post>> {
    PostsByUserDataLoader(PostRepository repo) {
        super(userIds -> CompletableFuture.supplyAsync(() -> {
            Map<String, List<Post>> grouped = repo.findByUserIds(userIds).stream()
                .collect(Collectors.groupingBy(Post::userId));
            return userIds.stream().map(id -> grouped.getOrDefault(id, List.of())).toList();
        }));
    }
}

class UserResolver {
    private final DataLoader<String, List<Post>> postsLoader;

    public CompletableFuture<List<Post>> posts(User user) {
        return postsLoader.load(user.id());
    }
}
```

- DataLoader collects calls within an event loop tick, then issues ONE batched query
- For 1000 users, you get 1 user query + 1 post-batch query, not 1 + 1000

Tradeoffs: - Request-scoped: new DataLoader per request — careful with framework integration - Batch latency: DataLoader waits for the tick to collect — adds a few ms; usually outweighed by query batching - Cache scope inside DataLoader prevents redundant calls for same id within the request

What NOT to say: - "Just join in SQL" — works if always needed; resolver chains don't know in advance - "Cache the post lookups globally" — stale data risk across requests

Common follow-up: "What's request-scoped DataLoader vs application-scoped — when each?"

Scenario 113 · Multi-tenant data isolation

Asked at: Razorpay · Cred · Postman · SaaS roles · **Difficulty:** Hard · **Topic:** Multi-tenancy

The interviewer's prompt: "SaaS with 10k tenants. One bug + tenant A sees tenant B's data = lawsuit. Design isolation."

Thinking out loud: 1. "Three patterns: shared schema + tenant_id column, schema-per-tenant, database-per-tenant. Cost vs isolation." 2. "Shared schema is cheapest; isolation enforced at app layer with tenant_id filter on EVERY query." 3. "Centralize tenant filter in repository base / interceptor — easy to forget at call sites."

A solid answer:

```
// Tenant context ThreadLocal (set per request)
class TenantContext {
    private static final ThreadLocal<String> CURRENT = new ThreadLocal<>();
    static void set(String tid) { CURRENT.set(tid); }
    static String get() { return Objects.requireNonNull(CURRENT.get(), "tenant not set"); }
    static void clear() { CURRENT.remove(); }
}

// Hibernate filter – applied automatically
@FilterDef(name = "tenantFilter", parameters = @ParamDef(name = "tid", type = String.class))
@Filter(name = "tenantFilter", condition = "tenant_id = :tid")
@Entity
class Order { /* ... */ }

@Aspect
class TenantAspect {
    @Around("@annotation(Transactional)")
    public Object enableFilter(ProceedingJoinPoint pjp) throws Throwable {
        Session s = entityManager.unwrap(Session.class);
        s.enableFilter("tenantFilter").setParameter("tid", TenantContext.get());
        return pjp.proceed();
    }
}

// Postgres alternative: Row Level Security
// CREATE POLICY tenant_isolation ON orders FOR ALL TO app_user
// USING (tenant_id = current_setting('app.tenant_id')::uuid);
```

- ThreadLocal + Hibernate Filter = automatic WHERE clause on every query
- Postgres RLS = DB-level enforcement; even raw SQL can't leak

Tradeoffs: - App-layer filter: fast but a single missed query = leak. Use RLS or automated tests for safety
 - Database-per-tenant: best isolation, worst cost — only for enterprise customers - Cross-tenant analytics: need a separate aggregation layer; can't query "all tenants" with the filter on

What NOT to say: - "Add tenant_id to every query manually" — humans forget; one missed query = breach - "Use enterprise login + trust the user" — user-controlled tenant id = trivial leak

Common follow-up: "What about analytics queries that span tenants — how do you safely do that?"

Scenario 114 · GraphQL persisted queries

Asked at: Postman · Cred · GraphQL shops · **Difficulty:** Medium · **Topic:** GraphQL ops

The interviewer's prompt: "GraphQL exposes infinite query complexity. Limit production to known queries only."

Thinking out loud: 1. "Persisted queries: clients send query ID (hash) instead of full query string." 2. "Server has whitelist of known queries; rejects unknown queries in prod." 3. "Dev mode allows arbitrary; build step extracts queries into manifest."

A solid answer:

```
class PersistedQueryRegistry {
    private final Map<String, String> idToQuery = loadFromManifest();

    public String resolve(String id) {
        String q = idToQuery.get(id);
        if (q == null) throw new UnknownQueryException(id);
        return q;
    }
}

@PostMapping("/graphql")
public Object execute(@RequestBody GraphQLReq req) {
    String query;
    if (req.queryId() != null) {
        query = registry.resolve(req.queryId());
    } else if (env.isProduction()) {
        throw new ForbiddenException("Ad-hoc queries blocked in prod");
    } else {
        query = req.query();
    }
    return engine.execute(query, req.variables());
}
```

- Build step extracts queries from client code → hash + manifest → ships with backend
- Dev allows arbitrary queries for development; prod is whitelist-only

Tradeoffs: - Tight coupling between client and server deploys — out-of-band query lookup (Apollo Persisted Queries) is the middle ground - Saves bandwidth (id instead of full query) AND CPU (no parsing for known queries) - Adds release complexity — client must update manifest before backend's whitelist updates

What NOT to say: - "Allow any query, just rate limit" — DoS via complex queries still works - "Validate query complexity at runtime" — useful complement, not replacement

Common follow-up: "How do you handle the deploy ordering — client wants new query but backend hasn't loaded the new manifest yet?"

Scenario 115 · CQRS read/write split**Asked at:** Razorpay · Cred · Walmart Labs · **Difficulty:** Hard · **Topic:** Architecture**The interviewer's prompt:** "Write model is normalized for transactional integrity; reads are slow. Design CQRS."**Thinking out loud:** 1. "Two separate models: write side (normalized, OLTP) + read side (denormalized, optimized per query)." 2. "Writes go to OLTP; events published to update read models." 3. "Read models eventually consistent — usually OK for views, dashboards."**A solid answer:**

```
// Write side
@Transactional
public void placeOrder(OrderRequest req) {
    Order order = orderRepo.save(req.toEntity());
    eventBus.publish(new OrderPlaced(order.id(), order.userId(), order.amount()));
}

// Read side — separate consumer
@KafkaListener(topics = "order-events")
public void on(OrderPlaced evt) {
    // Update a denormalized view: user_order_summary
    summaryRepo.upsert(
        evt.userId(),
        s -> s.incrementOrderCount().addToTotal(evt.amount())
    );
    // Or write to Elasticsearch for fast search
    searchIndex.index(new OrderSearchDoc(evt));
}

// Read API
public UserSummary getUserSummary(String userId) {
    return summaryRepo.findByUserId(userId); // single-row lookup, no joins
}
```

- Write side cares about correctness; read side cares about query speed
- Each read model targets specific query — denormalize for the use case

Tradeoffs: - Eventual consistency: read-after-write may show stale data for a few seconds - "Show user's latest order right after they placed it" — bypass read model and hit write store for that specific case - Maintaining multiple read models = ops complexity; only adopt when single model can't serve

What NOT to say: - "CQRS everywhere" — overkill for simple CRUD - "Read replicas solve this" — they do for some cases; CQRS is for fundamentally different query shapes

Common follow-up: "How do you handle 'read after write' when the read model is 2s behind?"

Scenario 116 · Event sourcing primer

Asked at: Cred · Razorpay · finance/banking · **Difficulty:** Hard · **Topic:** Architecture

The interviewer's prompt: "Bank account: store every transaction as an event, derive current balance. Design event sourcing."

Thinking out loud: 1. "Append-only event log. Current state = fold(events) — replay from start (or snapshot)." 2. "Snapshot every N events to avoid replaying all history." 3. "Audit trail comes for free — every state change is a stored event."

A solid answer:


```
sealed interface AccountEvent permits Deposited, Withdrawn, AccountOpened {}
record AccountOpened(String accountId, String owner, Instant at) implements AccountEvent {}
record Deposited(String accountId, BigDecimal amount, Instant at) implements AccountEvent {}
record Withdrawn(String accountId, BigDecimal amount, Instant at) implements AccountEvent {}

class Account {
    String id;
    BigDecimal balance = BigDecimal.ZERO;
    String owner;

    void apply(AccountEvent e) {
        switch (e) {
            case AccountOpened ao -> { this.id = ao.accountId(); this.owner = ao.owner(); }
            case Deposited d -> this.balance = this.balance.add(d.amount());
            case Withdrawn w -> this.balance = this.balance.subtract(w.amount());
        }
    }

    static Account rehydrate(List<AccountEvent> events) {
        Account a = new Account();
        events.forEach(a::apply);
        return a;
    }
}

class AccountService {
    public void deposit(String id, BigDecimal amount) {
        Account a = load(id); // rehydrate from event log
        if (a == null) throw new NotFoundException();
        Deposited e = new Deposited(id, amount, Instant.now());
        eventLog.append(id, e); // append to log
        // a.apply(e) - apply in memory if you keep an in-memory cache
    }
}
```

- Events are facts; state is a derived view
- Snapshot every 100 events: save current balance + last applied event id; rehydrate from snapshot + tail events

Tradeoffs: - Storage grows forever — archive old events; snapshots make rehydration fast - Schema evolution: events are immutable history. Adding fields → consumers must handle missing fields - "Replay from start" is slow for old accounts; snapshots are essential

What NOT to say: - "Skip the log, just update balance" — destroys audit trail; can't undo bugs - "Replay all events every time" — works for small N, infeasible at scale

Common follow-up: "How do you handle a bug in event interpretation discovered 6 months later?"

Scenario 117 · Distributed unique ID generator

Asked at: Razorpay · Cred · Postman · **Difficulty:** Medium · **Topic:** Distributed systems

The interviewer's prompt: "Need globally unique IDs across 50 services. No central DB. Compare UUID, Snowflake, ULID."

Thinking out loud: 1. "UUID v4: random, 128 bit — globally unique but unsortable, bad for DB index locality." 2. "Snowflake: 64-bit = timestamp + machine id + sequence. Time-ordered, compact." 3. "ULID: time-ordered like Snowflake, but UUID-compatible (128-bit)."

A solid answer:

```
public class SnowflakeIdGenerator {
    private final long epoch = 1577836800000L; // 2020-01-01
    private final int machineId;
    private long lastTimestamp = -1L;
    private int sequence = 0;

    public SnowflakeIdGenerator(int machineId) {
        this.machineId = machineId & 0x3FF; // 10 bits
    }

    public synchronized long nextId() {
        long ts = System.currentTimeMillis();
        if (ts < lastTimestamp) throw new IllegalStateException("Clock moved back");
        if (ts == lastTimestamp) {
            sequence = (sequence + 1) & 0xFFF; // 12 bits
            if (sequence == 0) ts = waitNextMillis();
        } else sequence = 0;
        lastTimestamp = ts;
        return ((ts - epoch) << 22) | (machineId << 12) | sequence;
    }

    private long waitNextMillis() {
        long ts;
        do { ts = System.currentTimeMillis(); } while (ts <= lastTimestamp);
        return ts;
    }
}
```

- Snowflake: 41-bit timestamp + 10-bit machine + 12-bit sequence = 64 bits, fits in BIGINT
- Time-ordered → great for index locality, but exposes creation time

Tradeoffs: - UUID v4: zero coordination, no clock dependency; bad for B-tree indexes due to randomness - Snowflake: requires unique machine id assignment (config or coordination); clock jumps cause issues - ULID: 26-char string; sortable; cross-system friendly

What NOT to say: - "Use database auto-increment" — single point of failure across services - "Random UUID is sortable" — not by time

Common follow-up: "What happens to Snowflake if a server's clock goes backward 5 seconds?"

Scenario 118 · Audit log design

Asked at: Razorpay · Cred · banking systems · **Difficulty:** Medium · **Topic:** Compliance

The interviewer's prompt: "Every sensitive action (refund, account close) must be auditable. Design the log."

Thinking out loud: 1. "Append-only table. Each row: who, what, when, why, before/after state, request id." 2. "Write must succeed with the action — same transaction, or outbox." 3. "Immutable: no UPDATES ever; archival to cold storage for old entries."

A solid answer:

```
CREATE TABLE audit_log (  
    id BIGSERIAL PRIMARY KEY,  
    actor_id VARCHAR(64) NOT NULL,  
    action VARCHAR(64) NOT NULL,          -- 'REFUND_ISSUED', 'ACCOUNT_CLOSED'  
    target_type VARCHAR(64) NOT NULL,     -- 'order', 'account'  
    target_id VARCHAR(64) NOT NULL,  
    before_state JSONB,  
    after_state JSONB,  
    request_id VARCHAR(64),              -- trace correlation  
    ip_address INET,  
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
CREATE INDEX ON audit_log(target_type, target_id);  
CREATE INDEX ON audit_log(actor_id);  
CREATE INDEX ON audit_log(created_at);  
-- No UPDATE permission for app role
```

```
@Transactional
public void issueRefund(String orderId, BigDecimal amount, String reason) {
    Order before = orderRepo.findById(orderId);
    refundService.process(orderId, amount);
    Order after = orderRepo.findById(orderId);
    auditLog.write(AuditEntry.builder()
        .actorId(SecurityContext.userId())
        .action("REFUND_ISSUED")
        .targetType("order").targetId(orderId)
        .beforeState(before).afterState(after)
        .requestId(MDC.get("requestId"))
        .build());
}
```

- Same transaction = audit can't be silently skipped; if audit fails, action rolls back
- Before/after JSON enables forensic reconstruction

Tradeoffs: - Synchronous write in TX: slows action; for high-volume events, fire async to Kafka + audit consumer (accept eventual consistency) - Storage explosion: archive after 1 year to cold storage (S3 Glacier); keep recent in DB for query - PII concerns: encrypt sensitive fields at rest; redact in views

What NOT to say: - "Log to a file" — files lost, rotated, hard to query - "Write audit async with no guarantee" — regulator wants no gaps

Common follow-up: "How do you prove the audit log itself hasn't been tampered with?"

Scenario 119 · Distributed lock with fencing token

Asked at: Razorpay · Cred · distributed systems roles · **Difficulty:** Hard · **Topic:** Distributed systems

The interviewer's prompt: "You use Redis SETNX as a distributed lock. Martin Kleppmann famously says it's broken. Why? Design a fix."

Thinking out loud: 1. "Problem: client A acquires lock, GC pauses for 30s, lock expires, client B acquires, A wakes up and acts thinking it still holds the lock." 2. "Fix: fencing token — every lock acquisition returns a monotonically increasing token. Downstream rejects writes with stale tokens." 3. "Redis can give you the token via INCR; Zookeeper gives sequential ephemeral nodes."

A solid answer:

```

record LockHandle(String lockId, long fencingToken) {}

class DistributedLock {
    private final RedisClient redis;

    public LockHandle acquire(String resource, Duration ttl) {
        long token = redis.incr("token:" + resource);
        String value = workerId + ":" + token;
        boolean got = "OK".equals(redis.set("lock:" + resource, value,
            SetArgs.Builder.nx().px(ttl.toMillis())));
        if (!got) throw new LockNotAvailableException();
        return new LockHandle(resource, token);
    }

    public void release(LockHandle h) {
        // Lua: only delete if value matches (we still hold it)
        String script = "if redis.call('get', KEYS[1]) == ARGV[1] then return redis.call('del', KEYS[1]) else return 0 end";
        redis.eval(script, List.of("lock:" + h.lockId()), List.of(workerId + ":" + h.fencingToken));
    }
}

// Downstream (e.g., file write API):
public void write(String resource, byte[] data, long fencingToken) {
    Long lastSeen = fencingStore.get(resource);
    if (lastSeen != null && fencingToken <= lastSeen)
        throw new StaleTokenException("token " + fencingToken + " < " + lastSeen);
    fencingStore.put(resource, fencingToken);
    actualWrite(resource, data);
}

```

- Fencing token incremented on every successful acquire — strictly monotonic
- Downstream rejects stale tokens — old client's write fails even if it thinks it still holds the lock

Tradeoffs: - Requires downstream cooperation — if your downstream doesn't check fencing tokens, this whole exercise doesn't help - For most apps, idempotent operations + short lock TTLs are enough; fencing is for correctness-critical writes - Zookeeper / etcd give linearizable locks with built-in fencing (zxid / mod_index)

What NOT to say: - "Just trust the lock" — exactly the failure Kleppmann warned about - "Use longer TTL" — pushes the window, doesn't eliminate it

Common follow-up: "Why does Redis Redlock not actually solve this?"

Scenario 120 · API rate limit headers

Asked at: Razorpay · Cred · API platforms · **Difficulty:** Easy · **Topic:** API design

The interviewer's prompt: "You return 429 when rate limited. What headers should you send so client can back off intelligently?"

Thinking out loud: 1. "Standard: X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset (epoch when bucket refills)." 2. "Retry-After: when we 429, tell client EXACT seconds to wait." 3.

"Documentation > headers — make sure the rules are also in API docs."

A solid answer:

```
@Component
class RateLimitFilter implements Filter {
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain) {
        HttpServletResponse hr = (HttpServletResponse) res;
        String userId = extractUser(req);
        TokenBucketState state = limiter.tryConsume(userId);

        hr.setHeader("X-RateLimit-Limit", String.valueOf(state.limit()));
        hr.setHeader("X-RateLimit-Remaining", String.valueOf(state.remaining()));
        hr.setHeader("X-RateLimit-Reset", String.valueOf(state.resetAtEpochSecond()));

        if (!state.allowed()) {
            long waitSec = state.resetAtEpochSecond() - Instant.now().getEpochSecond();
            hr.setHeader("Retry-After", String.valueOf(Math.max(1, waitSec)));
            hr.setStatus(429);
            hr.getWriter().write("{\"error\": \"rate_limited\"}");
            return;
        }
        chain.doFilter(req, res);
    }
}
```

- Send headers on EVERY response (not just 429) — clients can preemptively slow down
- Retry-After tells the client when to retry — prevents thundering herd

Tradeoffs: - For multi-tenant API, headers may need namespacing (X-RateLimit-User vs X-RateLimit-Org)
- Some clients ignore Retry-After — implement jitter on server side too

What NOT to say: - "Just return 429 with no headers" — client has no info to back off; will hammer -
"Use 503 instead" — wrong semantic; 503 = server down

Common follow-up: "What headers do you set when the request is NOT rate limited?"

Scenario 121 · Idempotency table garbage collection**Asked at:** Razorpay · Cred · PhonePe · **Difficulty:** Medium · **Topic:** Operations**The interviewer's prompt:** "Idempotency key table grows by 100M rows per day. Design retention."**Thinking out loud:** 1. "Each row has TTL (e.g., 24-72h depending on retry-window guarantee)." 2. "For SQL: scheduled job `DELETE WHERE created_at < now() - INTERVAL '3 days'`." 3. "For partition-friendly DBs: partition by day, drop old partitions = instant."**A solid answer:**

```
-- Partitioned daily
CREATE TABLE idempotency_keys (
    key TEXT PRIMARY KEY,
    response_json JSONB,
    created_at TIMESTAMPTZ NOT NULL
) PARTITION BY RANGE (created_at);

CREATE TABLE idempotency_keys_2026_05_25 PARTITION OF idempotency_keys
    FOR VALUES FROM ('2026-05-25') TO ('2026-05-26');

-- Daily job:
DROP TABLE idempotency_keys_2026_05_22; -- 3 days old; instant
```

```
@Scheduled(cron = "0 0 3 * * *") // 3 AM daily
public void retainKeys() {
    LocalDate cutoff = LocalDate.now().minusDays(3);
    partitionManager.dropPartitionsBefore(cutoff);
    partitionManager.createPartitionForTomorrow();
}
```

- DROP partition is instant; DELETE WHERE on 100M rows is hours of pain
- Pre-create tomorrow's partition; if you forget, inserts fail

Tradeoffs: - Redis TTL is simpler but storage cost vs durability matters — DB for high-value (payments) keys, Redis for low-stakes - Cross-region: partition strategy must be replicable - Customers requesting receipts older than retention window — limit clearly documented**What NOT to say:** - "DELETE in batches of 1000" — works at small scale, kills DB at 100M - "Keep forever" — storage costs balloon; index performance degrades**Common follow-up:** "What if a customer needs to dispute a charge older than your retention window?"

Scenario 122 · Search autocomplete (typeahead)**Asked at:** Swiggy · Meesho · Flipkart · **Difficulty:** Hard · **Topic:** Search**The interviewer's prompt:** "Show suggestions as user types. 'sw' → 'swiggy', 'sweater', 'sweet shop'. Sub-50ms p99."**Thinking out loud:** 1. "Trie or sorted index. For 1M+ keywords, Trie is memory-heavy." 2. "Production approach: precomputed prefix → top-K terms map, kept in Redis or in-memory." 3. "Ranking: combine popularity + personalization. Refresh hourly."**A solid answer:**

```

public class AutocompleteIndex {
    // Precomputed prefix → ordered list of (term, score)
    private final Map<String, List<TermScore>> prefixToTop10 = new ConcurrentHashMap<>();

    public List<String> suggest(String prefix) {
        List<TermScore> hits = prefixToTop10.get(prefix.toLowerCase());
        if (hits == null) return List.of();
        return hits.stream().map(TermScore::term).toList();
    }

    public void rebuildFromQueryLog(List<QueryLogEntry> logs) {
        Map<String, Map<String, Long>> agg = new HashMap<>(); // prefix → term → count
        for (QueryLogEntry e : logs) {
            String term = e.query().toLowerCase();
            for (int len = 1; len <= Math.min(10, term.length()); len++) {
                agg.computeIfAbsent(term.substring(0, len), k -> new HashMap<>())
                    .merge(term, 1L, Long::sum);
            }
        }
        Map<String, List<TermScore>> built = new HashMap<>();
        agg.forEach((prefix, terms) -> {
            built.put(prefix, terms.entrySet().stream()
                .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
                .limit(10)
                .map(e -> new TermScore(e.getKey(), e.getValue()))
                .toList());
        });
        this.prefixToTop10.putAll(built);
    }
}

```

- Lookup is O(1) HashMap — sub-millisecond
- Rebuild runs offline (hourly or daily); index swapped atomically

Tradeoffs: - Memory: for 1M terms with 10 prefixes each, ~10M entries. Manageable. - Stale: prefix index doesn't see "today's hot search" until rebuild — combine with realtime popular-recent overlay - Fuzzy match (typos): out of scope for prefix; need Levenshtein index or trigram (Elastic suggester)

What NOT to say: - "Query DB on every keystroke" — even with index, latency too high - "Just use SQL LIKE 'prefix%'" — works at small scale, dies at high QPS

Common follow-up: "How do you handle typos — 'swigy' should still suggest 'swiggy'?"

Scenario 123 · Rate-limited send queue

Asked at: Razorpay · Swiggy · Meesho · **Difficulty:** Medium · **Topic:** Throttling

The interviewer's prompt: "Send SMS via vendor. Vendor allows 100/sec. You have a queue of 1M to send. Don't exceed; don't waste time."

Thinking out loud: 1. "Producer dumps into queue (Kafka, Redis list). Worker pulls, throttled to 100/sec." 2. "Token bucket on the worker — gates send rate." 3. "Multi-worker: shared distributed rate limiter (Redis-based)."

A solid answer:

```
public class SmsSender {
    private final RedisTokenBucket bucket = new RedisTokenBucket("vendor-sms", 100, 100.0); // 100/sec
    private final Queue<SmsMessage> queue;

    public void run() {
        while (true) {
            SmsMessage msg = queue.poll();
            if (msg == null) { Thread.sleep(100); continue; }
            while (!bucket.tryConsume(1)) Thread.sleep(10); // wait for token
            try { vendor.send(msg); }
            catch (Exception e) { queue.requeueWithBackoff(msg, e); }
        }
    }
}
```

- Token bucket lets a burst absorb queue spikes up to bucket capacity
- Polling for token with sleep — fine for 100/sec; for higher rates, use blocking acquire

Tradeoffs: - Across N workers, total send rate = N × per-worker rate UNLESS rate limiter is centralized (Redis) - Vendor errors (5xx): re-queue with backoff. 4xx (bad number): DLQ. - For 1M messages at 100/sec, total time ~3 hours. Negotiate higher quota or split across vendors.

What NOT to say: - "Use `Thread.sleep(10ms)` between sends" — drift over time; not precise - "Skip rate limit, just retry on 429" — vendor banks the violations, may block

Common follow-up: "You have 3 SMS vendors with different rate limits. How do you split traffic optimally?"

Scenario 124 · Bulk import with progress

Asked at: Postman · Razorpay · Adobe GCC · **Difficulty:** Medium · **Topic:** Operations

The interviewer's prompt: "Customer uploads 100MB CSV. Import asynchronously, show progress, handle row-level errors."

Thinking out loud: 1. "Upload → store file → async worker → row-by-row processing → progress + error log." 2. "Errors: skip+collect (continue) vs abort (stop on first). Usually skip+collect for bulk." 3. "Resumable from row N if worker restarts."

A solid answer:

```

class BulkImport {
    String id;
    String fileKey;           // S3 key
    int totalRows;
    int processedRows;
    int errorRows;
    Instant startedAt;
    JobStatus status;
}

class BulkImportWorker {
    public void run(String importId) {
        BulkImport job = importRepo.get(importId);
        try (var lines = csvReader.read(s3.get(job.fileKey()))) {
            int row = 0;
            for (CsvRow csvRow : lines) {
                row++;
                if (row <= job.processedRows()) continue; // resume
                try {
                    domainService.upsert(csvRow.toDto());
                } catch (Exception e) {
                    importErrorRepo.save(new ImportError(importId, row, csvRow.raw(), e.getMessage()));
                    importRepo.incrementErrors(importId);
                }
                if (row % 100 == 0) importRepo.updateProgress(importId, row);
            }
        }
        importRepo.markComplete(importId);
    }
}

```

- **processedRows** checkpoint enables resume after crash
- Error rows recorded with original content for customer download

Tradeoffs: - Batch DB writes (every 100) instead of row-by-row — 10-100x faster - Memory: stream lines, don't load full file - For very large files, parallel processing by file offset chunks — requires CSV's row boundaries

What NOT to say: - "Load entire CSV into memory" — OOM on big files - "Abort on first error" — frustrating UX; user fixes one row, re-runs, finds next error

Common follow-up: "Worker pod dies at row 500,000 of a 1M-row file. How do you resume?"

Scenario 125 · Background email with templates

Asked at: Razorpay · Swiggy · Meesho · Adobe GCC · **Difficulty:** Easy-Medium · **Topic:** Async + templating

The interviewer's prompt: "Trigger transactional emails (order confirmation, password reset). Async send. Designs?"

Thinking out loud: 1. "Trigger publishes event → email worker consumes → renders template → sends via SMTP / SES." 2. "Templates: external (DB / S3) for non-engineer edits; render with engine like Handlebars." 3. "Per-email retry on send failure; DLQ after N attempts."

A solid answer:

```
record EmailRequest(String to, String templateId, Map<String, Object> variables, String correlationId)

@KafkaListener(topics = "email-requests")
public void onEmail(EmailRequest req) {
    Template tpl = templateRepo.get(req.templateId());
    String subject = render(tpl.subjectTemplate(), req.variables());
    String body = render(tpl.bodyTemplate(), req.variables());
    try {
        mailer.send(MailMessage.builder()
            .to(req.to())
            .subject(subject)
            .htmlBody(body)
            .header("X-Correlation-Id", req.correlationId())
            .build());
    } catch (Exception e) {
        // Kafka retry topic + DLQ
        throw e;
    }
}

// Triggering:
public void onOrderPlaced(Order o) {
    kafka.send("email-requests", new EmailRequest(
        o.userEmail(),
        "order-confirmation",
        Map.of("orderId", o.id(), "items", o.items(), "total", o.total()),
        MDC.get("requestId")));
}
```

- Decouples business logic from email delivery — order code doesn't block on SMTP
- Templates externalized so marketing can edit without code changes

Tradeoffs: - Template versioning: changes mid-flight may render emails with new template against old data — pin `template_version` in event - Per-locale templates: extend `templateId` with locale or have render auto-select - Spam compliance (unsubscribe link, sender reputation) — usually delegate to ESP (SES, SendGrid)

What NOT to say: - "Send synchronously from request handler" — SMTP latency adds 500ms+ to API - "Hardcode email content in code" — every wording change needs deploy

Common follow-up: "How do you handle a 'send failed' for a transactional email like password reset — user is waiting?"

Section H — Type System & Language Design (Scenarios 126–140)

Scenario 126 · Result type

Asked at: Cred · Postman · functional-leaning shops · **Difficulty:** Medium · **Topic:** Type design

The interviewer's prompt: "Java exceptions are intrusive. Design a Result like Rust — return success or failure as a value."

Thinking out loud: 1. "Sealed interface with Ok and Err records. Pattern match for handling." 2. "Methods: map, flatMap, orElse, fold — typical functional combinators." 3. "Useful for expected failures (validation, lookup) — keep exceptions for truly exceptional."

A solid answer:

```

public sealed interface Result<T, E> permits Result.Ok, Result.Err {
    record Ok<T, E>(T value) implements Result<T, E> {}
    record Err<T, E>(E error) implements Result<T, E> {}

    static <T, E> Result<T, E> ok(T value) { return new Ok<>(value); }
    static <T, E> Result<T, E> err(E error) { return new Err<>(error); }

    default <U> Result<U, E> map(Function<T, U> fn) {
        return switch (this) {
            case Ok<T, E> o -> new Ok<>(fn.apply(o.value()));
            case Err<T, E> e -> new Err<>(e.error());
        };
    }

    default <U> Result<U, E> flatMap(Function<T, Result<U, E>> fn) {
        return switch (this) {
            case Ok<T, E> o -> fn.apply(o.value());
            case Err<T, E> e -> new Err<>(e.error());
        };
    }

    default T orElse(T fallback) {
        return switch (this) {
            case Ok<T, E> o -> o.value();
            case Err<T, E> e -> fallback;
        };
    }

    default <R> R fold(Function<T, R> onOk, Function<E, R> onErr) {
        return switch (this) {
            case Ok<T, E> o -> onOk.apply(o.value());
            case Err<T, E> e -> onErr.apply(e.error());
        };
    }
}

```

- *Sealed permits* = compiler knows exhaustive switch is possible
- *map / flatMap / fold* = standard combinators; familiar from *Optional/Stream*

Tradeoffs / what you'd push back on: - Java doesn't have HKT — you can't generically combine Results across operations like in Haskell/Rust - Verbose compared to exceptions for deep call stacks (must thread Result through everything) - Use for expected failures (parsing, validation); exceptions for truly unexpected

What NOT to say: - "Just use Optional" — loses error type info - "Throw an exception, catch it later" — same problem we're solving

Common follow-up: "How would you combine 3 Results — only succeed if all 3 are Ok?"

Scenario 127 · Typed event bus

Asked at: Postman · Cred · Atlassian GCC · **Difficulty:** Medium · **Topic:** Type design

The interviewer's prompt: "Design a type-safe event bus. Subscribing to event X must give a typed callback, not Object."

Thinking out loud: 1. "Map<Class<? extends Event>, List> keyed by event class." 2. "Generic methods: subscribe(Class, Consumer) and publish(T)." 3. "At publish time, look up by event.getClass() — fan out to matching listeners."

A solid answer:

```
public class TypedEventBus {
    private final Map<Class<?>, List<Consumer<?>>> subs = new ConcurrentHashMap<>();

    public <T> Subscription subscribe(Class<T> type, Consumer<T> handler) {
        List<Consumer<?>> list = subs.computeIfAbsent(type, k -> new CopyOnWriteArrayList<>());
        list.add(handler);
        return () -> list.remove(handler);
    }

    @SuppressWarnings({"unchecked", "rawtypes"})
    public <T> void publish(T event) {
        List<Consumer<?>> handlers = subs.get(event.getClass());
        if (handlers == null) return;
        for (Consumer<?> h : handlers) {
            ((Consumer) h).accept(event);    // safe by construction
        }
    }
}

@FunctionalInterface
interface Subscription { void unsubscribe(); }

// Usage:
bus.subscribe(OrderPlaced.class, evt -> sendEmail(evt.userEmail()));
bus.publish(new OrderPlaced("u1", "u@x.com"));
```

- Subscription handle for clean unsubscribe — closes a memory-leak source
- Type checked at subscription site; raw cast at publish is safe because of how we populate the map

Tradeoffs: - Inheritance: listener for parent class doesn't auto-receive subclass events. Either traverse class hierarchy at publish, or document the constraint. - Async dispatch (executor) vs sync — sync is simpler; async needs care for ordering guarantees

What NOT to say: - "Use Object as the type and cast in the handler" — defeats type safety - "Use reflection to figure out the type" — slow + brittle

Common follow-up: "If I subscribe to Event (parent), should my listener get OrderPlaced (subclass) events?"

Scenario 128 · Builder pattern with required fields

Asked at: Razorpay · Cred · Postman · **Difficulty:** Medium · **Topic:** Type design

The interviewer's prompt: "Builder pattern lets caller forget required fields and find out at runtime. Design a builder where the compiler enforces required-first."

Thinking out loud: 1. "Stepwise builder: each setter returns a DIFFERENT builder type. Final type has the build() method." 2. "Caller MUST call required setters in order to reach the final type." 3. "More verbose than plain builder, but errors caught at compile time."

A solid answer:


```

public class Person {
    final String name; final int age; final String email;
    private Person(String n, int a, String e) { name = n; age = a; email = e; }

    public static NameStep builder() { return new Steps(); }

    public interface NameStep { AgeStep name(String name); }
    public interface AgeStep { OptionalStep age(int age); }
    public interface OptionalStep {
        OptionalStep email(String email);
        Person build();
    }

    private static class Steps implements NameStep, AgeStep, OptionalStep {
        private String name; private int age; private String email;
        public AgeStep name(String n) { this.name = n; return this; }
        public OptionalStep age(int a) { this.age = a; return this; }
        public OptionalStep email(String e) { this.email = e; return this; }
        public Person build() { return new Person(name, age, email); }
    }
}

// Usage – compiler forces correct sequence:
Person p = Person.builder()
    .name("Alice")    // returns AgeStep – only age() available
    .age(30)          // returns OptionalStep – email and build available
    .email("a@b.c")   // optional
    .build();

```

- Each step interface exposes only the methods valid at that stage
- Compiler catches `Person.builder().build()` and `Person.builder().name(...).build()` — missing required fields

Tradeoffs: - More verbose: N required fields = $N+1$ interfaces + 1 implementation - For many required + optional fields, the ergonomics get awkward; consider records with default values - Lombok @Builder doesn't do this; you'd hand-roll it

What NOT to say: - "Throw NullPointerException in build() if required missing" — that's the runtime check we're trying to avoid - "Use @NotNull annotations" — runtime, not compile time

Common follow-up: "What if I have 10 required fields — does the boilerplate explode?"

Scenario 129 · Phantom types for state

Asked at: Cred · Postman · advanced shops · **Difficulty:** Hard · **Topic:** Type design

The interviewer's prompt: "Connection has states (NEW, OPEN, CLOSED). Operations valid only in specific states. Use type system to enforce."

Thinking out loud: 1. "Phantom types: parametrize Connection with a state marker that's only at type level." 2. "Methods are defined only on the type for the valid state." 3. "Java's generics let you fake this — not as clean as Rust/Haskell, but works."

A solid answer:

```
public class Connection<S extends State> {
    interface State {}
    static class New implements State {}
    static class Open implements State {}
    static class Closed implements State {}

    private Connection(/*internal state*/) {}

    public static Connection<New> create() { return new Connection<>(); }

    public Connection<Open> open() {
        // (only callable on Connection<New>... wait, that's the problem)
        return new Connection<>();
    }

    public Connection<Closed> close() {
        // ...
        return new Connection<>();
    }

    public byte[] read() {
        // valid only on Open
        return new byte[0];
    }
}

// To restrict methods to specific states, use static methods:
public class Connections {
    public static Connection<Open> open(Connection<New> conn) { return conn.open(); }
    public static byte[] read(Connection<Open> conn) { return conn.read(); }
    public static Connection<Closed> close(Connection<Open> conn) { return conn.close(); }
}

// Usage:
var c1 = Connections.open(Connection.create());
byte[] data = Connections.read(c1);
var c2 = Connections.close(c1);
// Connections.read(c2); // compile error - c2 is Connection<Closed>
```

- Generic parameter acts as a state marker at type level
- Static helpers restrict valid operations by state type

Tradeoffs: - Java can't enforce "exactly one state per type" — work around with sealed marker interfaces
 - Verbose; rarely worth it outside protocol libraries - For most state machines, runtime enum + check is clearer to maintainers

What NOT to say: - "Use enum" — fine for runtime, but doesn't catch errors at compile time - "Java can't do this" — Java's generics are weak vs Haskell/Rust but you can fake a lot

Common follow-up: "Why doesn't Java's type system enforce this as cleanly as Rust's?"

Scenario 130 · Domain primitives (no primitive obsession)

Asked at: Razorpay · Cred · Postman · DDD-leaning shops · **Difficulty:** Medium · **Topic:** Type design

The interviewer's prompt: "You pass `String userId` and `String orderId` everywhere — easy to swap them. Design a fix."

Thinking out loud: 1. "Domain primitives: wrap each ID type in its own class. `UserId` and `OrderId` can't accidentally substitute." 2. "Records in Java 17+ make this nearly free." 3. "Validation happens at construction — invalid IDs never exist."

A solid answer:

```
public record UserId(String value) {
    public UserId {
        if (value == null || value.isBlank()) throw new IllegalArgumentException("UserId blank");
        if (!value.startsWith("usr_")) throw new IllegalArgumentException("Invalid UserId format");
    }
    public static UserId of(String value) { return new UserId(value); }
    @Override public String toString() { return value; }
}

public record OrderId(String value) {
    public OrderId {
        if (value == null || !value.startsWith("ord_")) throw new IllegalArgumentException();
    }
}

// Now this is a compile error:
public Order findOrder(OrderId oid) { ... }
findOrder(userId); // ERROR - can't pass UserId where OrderId expected
```

- Record canonical constructor enforces invariants at construction
- ID can never be malformed in your domain — validation happens once

Tradeoffs: - Verbose at boundaries (REST, DB): need explicit unwrap (`uid.value()`) for JSON/SQL - Many small types — IDE auto-import friction; codebase grows - Wins compound: replace primitive money with Money type, primitive email with Email, etc.

What NOT to say: - "Add `_uid` suffix to variable names" — no compile-time guarantee - "Use Lombok `@Value`" — fine pre-Java 17; records are now built-in

Common follow-up: "How do you handle serialization — JSON wants 'usr_123', not '{value: 'usr_123'}'?"

Scenario 131 · Builder for immutable config

Asked at: Razorpay · Cred · Postman · **Difficulty:** Medium · **Topic:** Type design

The interviewer's prompt: "Config has 20 fields, all optional. Design a builder so caller picks what they want; result is immutable."

Thinking out loud: 1. "Standard Builder pattern. Defaults set in the builder; final constructor takes builder." 2. "Records in Java 17 are immutable by default; combine with a Builder static class for fluent API." 3. "Validate in build()."

A solid answer:

```

public record HttpConfig(
    Duration connectTimeout,
    Duration readTimeout,
    int maxRetries,
    boolean followRedirects,
    int maxConnections,
    String userAgent
) {
    public static Builder builder() { return new Builder(); }

    public static class Builder {
        private Duration connectTimeout = Duration.ofSeconds(5);
        private Duration readTimeout = Duration.ofSeconds(30);
        private int maxRetries = 3;
        private boolean followRedirects = true;
        private int maxConnections = 50;
        private String userAgent = "MyApp/1.0";

        public Builder connectTimeout(Duration t) { this.connectTimeout = t; return this; }
        public Builder readTimeout(Duration t) { this.readTimeout = t; return this; }
        public Builder maxRetries(int n) { this.maxRetries = n; return this; }
        public Builder followRedirects(boolean f) { this.followRedirects = f; return this; }
        public Builder maxConnections(int n) { this.maxConnections = n; return this; }
        public Builder userAgent(String ua) { this.userAgent = ua; return this; }

        public HttpConfig build() {
            if (maxRetries < 0) throw new IllegalArgumentException("maxRetries < 0");
            return new HttpConfig(connectTimeout, readTimeout, maxRetries, followRedirects, maxConnections, userAgent);
        }
    }
}

HttpConfig cfg = HttpConfig.builder()
    .readTimeout(Duration.ofSeconds(10))
    .maxRetries(5)
    .build();

```

- Record auto-generates equals/hashCode/toString — immutable for free
- Defaults in builder make most fields truly optional

Tradeoffs: - Verbose for many fields — consider AutoValue or Lombok if 30+ fields - "withX" methods on record itself for single-field tweaks: `cfg.withReadTimeout(...)` — needs hand-rolling

What NOT to say: - "Use a Map" — loses type safety; bag of "anything goes" - "Public constructor with 20 args" — caller can't tell which arg is which

Common follow-up: "How do you support `withX` mutations for a single field on an immutable record?"

Scenario 132 · Custom annotation processor

Asked at: Postman · Cred · platform teams · **Difficulty:** Hard · **Topic:** Compile-time meta

The interviewer's prompt: "Design an `@AutoMetric` annotation that generates Micrometer-instrumented wrappers at compile time."

Thinking out loud: 1. "Annotation processor reads annotated classes during compilation, generates source code." 2. "javax.annotation.processing.AbstractProcessor — process() invoked per round." 3. "For each `@AutoMetric` class, emit a subclass that records latency/error metrics around each method."

A solid answer:

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.SOURCE)
public @interface AutoMetric {
    String prefix() default "";
}

@SupportedAnnotationTypes("com.example.AutoMetric")
@SupportedSourceVersion(SourceVersion.RELEASE_17)
public class AutoMetricProcessor extends AbstractProcessor {
    @Override public boolean process(Set<? extends TypeElement> annotations, RoundEnvironment env) {
        for (Element annotated : env.getElementsAnnotatedWith(AutoMetric.class)) {
            TypeElement cls = (TypeElement) annotated;
            JavaFileObject file = processingEnv.getFiler()
                .createSourceFile(cls.getQualifiedName() + "Metered");
            try (var w = file.openWriter()) {
                w.write(generateSource(cls));
            }
        }
        return true;
    }

    private String generateSource(TypeElement cls) {
        // Emit: public class FooMetered extends Foo { @Override Bar method() { ... timer.record(...) } }
        // ...
    }
}
```

- META-INF/services/javax.annotation.processing.Processor lists processor class
- Generated code lives in `build/generated/sources/...` — committed to compile output, not source tree

Tradeoffs: - Build complexity: processor itself must compile before user code; gradle multi-module setup
 - Generated code is brittle to refactors — IDE may not always reflect generated symbols cleanly - For most teams, AOP (Spring AOP, AspectJ) is easier than compile-time generation

What NOT to say: - "Use reflection at runtime" — slow; compile-time avoids overhead - "Hand-write the metric calls" — fine for 5 methods, painful for 500

Common follow-up: "What's the difference between compile-time annotation processing and runtime annotation reading?"

Scenario 133 · Type-safe SQL queries

Asked at: Cred · Postman · static-typing shops · **Difficulty:** Hard · **Topic:** Type design

The interviewer's prompt: " `JdbcTemplate.query("SELECT id, name FROM users", rowMapper)` — typo in column name crashes at runtime. Design a type-safe query API."

Thinking out loud: 1. "jOOQ or QueryDSL: code-generated meta-model. Compile-time-safe column references." 2. "Roll your own: define columns as typed constants; QueryBuilder uses them." 3. "Fundamentally, the SQL string must come from somewhere typed — either codegen or DSL."

A solid answer:

```
// Generated (or hand-written) meta-model
public class UserTable {
    public static final Column<String> ID = new Column<>("id", String.class);
    public static final Column<String> NAME = new Column<>("name", String.class);
    public static final Column<Integer> AGE = new Column<>("age", Integer.class);
    public static final String TABLE = "users";
}

public record Column<T>(String name, Class<T> type) {}

public class Query<T> {
    private final List<Column<?>> columns;
    private final String table;
    private String where;

    public static <T> SelectBuilder<T> select(Column<?>... cols) {
        return new SelectBuilder<>(Arrays.asList(cols));
    }

    public <V> Query<T> where(Column<V> c, V value) {
        this.where = c.name() + " = ?";
        // ...
        return this;
    }

    public List<Row> execute(JdbcTemplate jdbc) { /* ... */ }
}

// Usage – compiler checks column existence:
Query.select(UserTable.ID, UserTable.NAME)
    .from(UserTable.TABLE)
    .where(UserTable.AGE, 18)
    .execute(jdbc);
```

- Column typed by SQL type → builder can enforce compatible operations
- Typos in column names = compile error

Tradeoffs: - DIY DSL never as complete as jOOQ — for production, just use jOOQ - Code generation needs build step; introduces compile-then-codegen dependencies - For complex queries, falling back to raw SQL is sometimes simpler

What NOT to say: - "Use Hibernate criteria" — verbose, leaks abstractions - "Test the SQL in CI" — catches typos but not enough; type system can prevent them entirely

Common follow-up: "What does jOOQ do that hand-rolled DSLs miss?"

Scenario 134 · Immutable collection builder**Asked at:** Razorpay · Cred · Postman · **Difficulty:** Medium · **Topic:** Type design**The interviewer's prompt:** "Design `ImmutableList.of(1, 2, 3)` -style API. Once built, no mutation. Java has `List.of` — design from scratch."**Thinking out loud:** 1. "Wrap an array; expose only read methods. Throw `UnsupportedOperationException` on mutators." 2. "Builder for adds; finalize to immutable instance." 3. "For deeply nested immutability, use Guava's `ImmutableMap`, `ImmutableSet` patterns."**A solid answer:**

```

public class ImmutableList<T> implements Iterable<T> {
    private final Object[] data;

    private ImmutableList(Object[] d) { this.data = d; }

    @SafeVarargs
    public static <T> ImmutableList<T> of(T... elements) {
        return new ImmutableList<>(elements.clone()); // defensive copy
    }

    public static <T> Builder<T> builder() { return new Builder<>(); }

    @SuppressWarnings("unchecked")
    public T get(int i) { return (T) data[i]; }
    public int size() { return data.length; }

    @SuppressWarnings("unchecked")
    public Iterator<T> iterator() {
        return new Iterator<>() {
            int i = 0;
            public boolean hasNext() { return i < data.length; }
            public T next() { return (T) data[i++]; }
            public void remove() { throw new UnsupportedOperationException(); }
        };
    }

    public static class Builder<T> {
        private final List<T> tmp = new ArrayList<>();
        public Builder<T> add(T element) { tmp.add(element); return this; }
        public ImmutableList<T> build() { return new ImmutableList<>(tmp.toArray()); }
    }
}

```

- Defensive copy in `of()` — caller can't modify the array out from under us
- `Iterator.remove()` throws — no escape hatch for mutation

Tradeoffs: - Defensive copy costs allocation; for very small lists (size 0, 1, 2), specialized subclasses avoid the array - Java 9+ `List.of` is the production answer — covers this and integrates with the JDK - For immutable maps, structural sharing (Clojure-style HAMT) avoids full copy on modify

What NOT to say: - "Use `Collections.unmodifiableList`" — wraps a mutable list; original holder can still mutate - "Make the field private — done" — Java doesn't have const reference semantics

Common follow-up: "How does Java's `List.of` differ from `Collections.unmodifiableList`?"

Scenario 135 · Optional vs null

Asked at: Most product cos · **Difficulty:** Easy-Medium · **Topic:** Type design

The interviewer's prompt: "Design a method that may not return a value. Why Optional instead of nullable T?"

Thinking out loud: 1. "Optional makes 'maybe-absent' explicit in the type signature — caller HAS to handle it." 2. "Null is silent — caller forgets, NPE at runtime." 3. "Caveat: don't use Optional for fields or method parameters; it's for return types."

A solid answer:

```
// Good: return Optional from methods
public Optional<User> findByEmail(String email) {
    User u = repository.findByEmail(email);
    return Optional.ofNullable(u);
}

// Caller — compiler forces handling:
Optional<User> u = svc.findByEmail("a@b.com");
String name = u.map(User::name).orElse("Unknown");

// Anti-pattern: Optional as field or parameter
class User {
    private Optional<String> nickname;    // BAD — extra allocation, can be null itself
}

void doStuff(Optional<String> name) {    // BAD — caller can pass null instead
}

// Better:
class User {
    private String nickname;    // nullable, document with @Nullable
    public Optional<String> nickname() { return Optional.ofNullable(nickname); }
}
```

- *Optional return types force callers to handle absence*
- *Don't store Optional as a field; it's not Serializable, and Optional itself can be null*

Tradeoffs: - *Optional adds allocation on every call — for hot paths, plain nullable + @Nullable annotation may be better* - *Some teams aggressively use Optional everywhere; the JDK guidance is "return values only" - Optional.get() without check defeats the purpose — use map/orElse/ifPresent*

What NOT to say: - *"Optional makes code safer everywhere" — overuse hurts; field/param Optional is anti-pattern* - *"Use Optional.get() with isPresent() check" — verbose and re-creates null-check pattern*

Common follow-up: *"When should I NOT use Optional?"*

Scenario 136 · Functional interfaces / SAM types

Asked at: *Razorpay · Cred · most Java interviews* · **Difficulty:** *Medium* · **Topic:** *Type design*

The interviewer's prompt: *"Design a callback API for download progress. Show why functional interface beats inheritance."*

Thinking out loud: 1. *"Functional interface (Single Abstract Method) = lambdas work as implementations."* 2. *"Caller passes lambda — no anonymous subclass boilerplate."* 3. *"@FunctionalInterface annotation enforces SAM contract at compile time."*

A solid answer:

```

@FunctionalInterface
public interface ProgressCallback {
    void onProgress(long bytesDownloaded, long totalBytes);

    // Default method — extends without breaking SAM
    default int percent(long downloaded, long total) {
        return total == 0 ? 0 : (int)(downloaded * 100 / total);
    }
}

public void download(String url, ProgressCallback callback) {
    long total = getContentLength(url);
    try (var in = open(url); var out = file()) {
        byte[] buf = new byte[8192];
        long downloaded = 0;
        int n;
        while ((n = in.read(buf)) != -1) {
            out.write(buf, 0, n);
            downloaded += n;
            callback.onProgress(downloaded, total);
        }
    }
}

// Caller uses a lambda:
downloader.download("https://...", (down, total) ->
    System.out.println("Progress: " + (down * 100 / total) + "%"));

```

- `@FunctionalInterface` = compile error if you accidentally add a second abstract method
- default methods extend without breaking lambda compatibility

Tradeoffs: - SAM interfaces locked to one method — if you need pre/post hooks, add default methods or use multiple SAM args - For backward compat, adding methods to interfaces uses default — but those don't enforce semantics, just defaults - For 2+ params, use `Function2` / `BiFunction` / write custom

What NOT to say: - "Use abstract class for callbacks" — caller must subclass; can't pass lambda - "Use multiple interfaces, `ProgressListener` and `CompletionListener`" — fine if events are unrelated; awkward for cohesive callback

Common follow-up: "Why does `@FunctionalInterface` allow default methods but not multiple abstract methods?"

Scenario 137 · Type token (super type token)

Asked at: Cred · Postman · framework authors · **Difficulty:** Hard · **Topic:** Generics

The interviewer's prompt: "Java erases generics. How do you preserve `List<Order>` at runtime — for deserialization, etc.?"

Thinking out loud: 1. "Trick from Neal Gafter: subclass a generic class. Reflection on the subclass's super type gives you the parameter." 2. "Jackson's `TypeReference`, Guice's `TypeLiteral` all use this." 3. "Caller passes `new TypeReference<List<Order>>() {}` — anonymous subclass captures the type info."

A solid answer:

```
public abstract class TypeReference<T> {
    private final Type type;
    protected TypeReference() {
        Type superClass = getClass().getGenericSuperclass();
        this.type = ((ParameterizedType) superClass).getActualTypeArguments()[0];
    }
    public Type getType() { return type; }
}

public class JsonParser {
    public <T> T parse(String json, TypeReference<T> ref) {
        return (T) mapper.readValue(json, ref.getType());
    }
}

// Usage:
List<Order> orders = parser.parse(json, new TypeReference<List<Order>>() {});
```

- `{}` creates an anonymous subclass; that subclass's generic param is reflectable
- Without the `{}`, you'd just have a raw `TypeReference` with `type=Object`

Tradeoffs: - Verbose at call site; some accept it for safety, others prefer raw `Class` + cast - Allocates an anonymous class instance per call — usually fine - Doesn't work for generic types within generic types perfectly (e.g., `Map<>`) — needs nested `TypeReferences`

What NOT to say: - "Java erasure means you can't" — false; super type token works - "Pass `Class` as parameter" — loses parameter info (`Class` not `Class<>`)

Common follow-up: "How does Jackson use `TypeReference` internally?"

Scenario 138 · Sealed types for ADTs

Asked at: Cred · Postman · Java-17+ shops · **Difficulty:** Medium · **Topic:** Modern Java

The interviewer's prompt: "Design a payment outcome — success with txnId, or failure with reason, or pending with poll URL. Use sealed types."

Thinking out loud: 1. "Sealed interface with finite permits — closed set of subtypes." 2. "Switch expression with pattern matching is exhaustive (compiler enforces)." 3. "This is Java's algebraic data type."

A solid answer:

```
public sealed interface PaymentOutcome permits Success, Failure, Pending {}
public record Success(String txnId, BigDecimal amount) implements PaymentOutcome {}
public record Failure(String reason, String code) implements PaymentOutcome {}
public record Pending(String pollUrl, Duration suggestedWait) implements PaymentOutcome {}

public String render(PaymentOutcome outcome) {
    return switch (outcome) {
        case Success s -> "Paid " + s.amount() + " (txn " + s.txnId() + ")";
        case Failure f -> "Failed: " + f.reason() + " [" + f.code() + "]";
        case Pending p -> "Pending; check " + p.pollUrl() + " in " + p.suggestedWait();
    };
}
```

- Compiler enforces all cases handled in switch — add a new permits, every switch is a compile error
- Pattern matching with records deconstructs cleanly

Tradeoffs: - Adding a new variant breaks every switch — usually a feature (you WANT to be reminded) - For open hierarchies (plugin-style), use a regular interface - Older Java code can't use them; modernizing to Java 17 is a prerequisite

What NOT to say: - "Use null check on optional fields in a single class" — loses the discriminated-union semantics - "Throw exception for failure case" — exceptions for expected outcomes muddy control flow

Common follow-up: "What's the difference between sealed types and enum?"

Scenario 139 · Trait-like mixin via interface default methods

Asked at: Postman · Cred · Java 8+ teams · **Difficulty:** Medium · **Topic:** Modern Java

The interviewer's prompt: "Java doesn't have traits. Use interface default methods to add `loggable`, `cacheable` behavior to multiple classes."

Thinking out loud: 1. "Default methods on interfaces add implementation — implementor can override or use as-is." 2. "Multiple interfaces = multiple mixins. Diamond problem resolved by explicit override." 3. "State can't go in interfaces — for stateful mixins, use composition."

A solid answer:

```
public interface Loggable {
    default Logger logger() { return LoggerFactory.getLogger(getClass()); }
    default void logInfo(String msg) { logger().info(msg); }
    default void logError(String msg, Throwable t) { logger().error(msg, t); }
}

public interface Timed {
    default <T> T timed(String name, Supplier<T> fn) {
        long start = System.nanoTime();
        try { return fn.get(); }
        finally { System.out.println(name + " took " + (System.nanoTime()-start)/1_000_000 + "ms") }
    }
}

public class OrderService implements Loggable, Timed {
    public Order create(OrderReq req) {
        logInfo("Creating order");
        return timed("createOrder", () -> doCreate(req));
    }
    private Order doCreate(OrderReq req) { /* ... */ return new Order(); }
}
```

- Default methods give "mix in" capabilities without inheritance
- Multiple interfaces stack — OrderService gets both `logInfo` and `timed`

Tradeoffs: - State requires instance fields — defaults can't add them. For stateful mixins, use composition. - Diamond problem: two interfaces with same default method → implementor must override and pick or combine - Overuse leads to "implements 8 interfaces" — code becomes magic; favor composition for non-trivial state

What NOT to say: - "Use abstract class" — single-inheritance; can only have ONE mixin - "Use inheritance" — same problem

Common follow-up: "What goes wrong if two interfaces define the same default method?"

Scenario 140 · Visitor pattern with sealed types

Asked at: Cred · Postman · compiler-adjacent roles · **Difficulty:** Hard · **Topic:** Type design

The interviewer's prompt: "Process AST nodes with different logic per type. Visitor pattern, but modern Java with sealed types and pattern matching."

Thinking out loud: 1. "Traditional Visitor: each node has `accept(visitor)`. Visitor has `visit(NodeA)`, `visit(NodeB)` for each type." 2. "Modern Java with sealed types: skip the accept method, use pattern-match switch directly." 3. "Cleaner, no double-dispatch dance."

A solid answer:

```
public sealed interface Expr permits Literal, Add, Multiply, Variable {}
public record Literal(double value) implements Expr {}
public record Add(Expr left, Expr right) implements Expr {}
public record Multiply(Expr left, Expr right) implements Expr {}
public record Variable(String name) implements Expr {}

public class Evaluator {
    private final Map<String, Double> env;

    public double evaluate(Expr expr) {
        return switch (expr) {
            case Literal l -> l.value();
            case Add(Expr left, Expr right) -> evaluate(left) + evaluate(right);
            case Multiply(Expr left, Expr right) -> evaluate(left) * evaluate(right);
            case Variable v -> env.getOrDefault(v.name(), 0.0);
        };
    }
}

public class Printer {
    public String print(Expr expr) {
        return switch (expr) {
            case Literal l -> String.valueOf(l.value());
            case Add(Expr l, Expr r) -> "(" + print(l) + " + " + print(r) + ")";
            case Multiply(Expr l, Expr r) -> "(" + print(l) + " * " + print(r) + ")";
            case Variable v -> v.name();
        };
    }
}
```

- New operation? Just add a new switch — no need to modify Expr hierarchy
- Pattern destructuring `Add(Expr l, Expr r)` is sugar for `add.left()` / `add.right()`

Tradeoffs: - Adding a new node type forces every switch to update — usually desired (exhaustiveness) - Old-school Visitor wins when you have closed operations and open types - New-style with sealed wins when types are closed and operations are open — typical compiler AST

What NOT to say: - "instanceof checks" — works but loses exhaustiveness; add new type, no warning - "Inheritance with abstract method on node" — every operation forced into Node API

Common follow-up: "When would the classic Visitor pattern still be the right choice?"

Section I — Observability & Operations (Scenarios 141–150)

Scenario 141 · Metric counter under high contention

Asked at: Razorpay · Cred · Postman · trading-tech · **Difficulty:** Medium · **Topic:** Observability

The interviewer's prompt: "Count requests across 64 cores serving 1M QPS. Atomic counter contention is killing you. Design the fix."

Thinking out loud: 1. "Single AtomicLong becomes a cache-line ping-pong at scale." 2. "LongAdder (Java 8+) stripes the counter across cells — adds happen to different cells, sum walks all." 3. "For Prometheus / Micrometer, the underlying counter primitive is exactly LongAdder."

A solid answer:

```
public class RequestMetrics {
    private final LongAdder requests = new LongAdder();
    private final LongAdder errors = new LongAdder();
    private final LongAdder bytes = new LongAdder();

    public void recordRequest(int byteCount, boolean error) {
        requests.increment();
        bytes.add(byteCount);
        if (error) errors.increment();
    }

    // Reader (scrape, dashboard) — non-atomic but consistent enough for monitoring
    public Snapshot snapshot() {
        return new Snapshot(requests.sum(), errors.sum(), bytes.sum());
    }
}
```

- LongAdder uses Striped64 internally: cell array sized to ~num CPUs
- sum() walks all cells; not atomic with concurrent updates, but monitoring tolerates this

Tradeoffs / what you'd push back on: - For per-request, per-tenant cardinality (millions of distinct LongAdders), memory adds up — cap your label cardinality - Histogram percentiles are harder — use HDRHistogram or Micrometer Timer (which handles striping)

What NOT to say: - "Use AtomicLong" — scaling breaks past 16 cores - "Use synchronized" — orders of magnitude slower

Common follow-up: "Now I need p99 latency, not just count. What primitive?"

Scenario 142 · Distributed tracing context propagation

Asked at: Razorpay · Cred · Postman · all observability-aware shops · **Difficulty:** Medium · **Topic:** Observability

The interviewer's prompt: "Request flows through 5 services. Want a single trace ID across all. Design the propagation."

Thinking out loud: 1. "W3C Trace Context: `traceparent` HTTP header carries trace-id + parent-span-id + flags." 2. "Each service: extract on incoming request, set in MDC, inject on outgoing requests." 3. "For async (CompletableFuture, queues), MDC doesn't propagate automatically — wrap with context."

A solid answer:

```
public class TracingFilter implements Filter {
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain)
        throws IOException, ServletException {
        HttpServletRequest hr = (HttpServletRequest) req;
        String traceparent = hr.getHeader("traceparent");
        SpanContext ctx = traceparent != null
            ? W3CParser.parse(traceparent)
            : new SpanContext(newTraceId(), newSpanId(), 1);
        MDC.put("traceId", ctx.traceId());
        MDC.put("spanId", ctx.spanId());
        try {
            chain.doFilter(req, res);
        } finally { MDC.clear(); }
    }
}

// Outgoing HTTP client interceptor:
public Response intercept(Chain chain) throws IOException {
    String traceId = MDC.get("traceId");
    String spanId = newSpanId();
    String header = "00-" + traceId + "-" + spanId + "-01";
    Request req = chain.request().newBuilder().header("traceparent", header).build();
    return chain.proceed(req);
}

// Kafka headers – same pattern with producer interceptor adding traceparent header
```

- W3C trace context = vendor-neutral; OpenTelemetry uses this
- Every span has parent → tree of spans = the trace

Tradeoffs: - Sampling: tracing every request is expensive. Head-based sampling (random %) or tail-based (sample errors + slow only) - Async context: ThreadLocal MDC + CompletableFuture default executor = lost context. Use OpenTelemetry's contextWrap or pass context explicitly - Storage cost: dropping sampled-out spans early is key

What NOT to say: - "Generate a new trace id per service" — defeats the entire point - "Pass trace id in request body" — invasive; headers are designed for this

Common follow-up: "How does tracing context flow through Kafka? Through CompletableFuture?"

Scenario 143 · Log aggregation with correlation

Asked at: Razorpay · Cred · Postman · **Difficulty:** Medium · **Topic:** Observability

The interviewer's prompt: "50 pods, 1000 req/sec. When a customer complains, you need to find every log line for their request. Design."

Thinking out loud: 1. "Structured logging (JSON) + correlation id (trace id) in every log line." 2. "Ship logs to centralized store (Elasticsearch, Loki, CloudWatch). Search by correlation id." 3. "Filter at pod-level — don't ship DEBUG to remote; keeps cost down."

A solid answer:

```
// Slf4j with MDC
public class RequestController {
    public ResponseEntity<Order> create(@RequestHeader("X-Request-Id") String requestId,
                                       @RequestBody OrderReq req) {
        MDC.put("requestId", requestId);
        MDC.put("userId", req.userId());
        try {
            log.info("Creating order");
            Order order = orderService.create(req);
            log.info("Order created with id={}", order.id());
            return ResponseEntity.ok(order);
        } finally { MDC.clear(); }
    }
}

// logback.xml – JSON encoder:
/*
<appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
    <encoder class="net.logstash.logback.encoder.LogstashEncoder">
        <includeMdcKeyName>requestId</includeMdcKeyName>
        <includeMdcKeyName>userId</includeMdcKeyName>
    </encoder>
</appender>
*/

// All logs include {"requestId": "...", "userId": "...", "msg": "..."}
// Search Kibana: requestId:"abc-123" → all log lines for that request
```

- MDC + JSON encoder = structured logs with correlation
- Search across cluster by correlation id

Tradeoffs: - Log volume: $1000 \text{ req/sec} \times 5 \text{ log lines} \times 50 \text{ pods} = 250k \text{ log lines/sec}$. Use sampling or shipping pipeline (Fluentbit/Vector) buffering - PII redaction: don't log full credit card / Aadhaar. Have an interceptor or annotation - Cost: Elasticsearch storage is expensive; use S3 + Athena for cold tier

What NOT to say: - "Just grep on each pod" — doesn't scale to 50 pods - "Log to file and tar them later" — too slow for incidents

Common follow-up: "How do you handle log volume at 100k QPS — cost optimization?"

Scenario 144 · Metrics: counter vs gauge vs histogram

Asked at: Razorpay · Cred · Postman · **Difficulty:** Easy · **Topic:** Observability basics

The interviewer's prompt: "Counter, gauge, histogram, summary. Which one for: requests, current memory, latency p99, in-flight count?"

Thinking out loud: 1. "Counter: monotonic increasing. Requests, errors." 2. "Gauge: snapshot of current value. Memory, in-flight count, queue size." 3. "Histogram: distribution. Latency (then derive p50, p99). Pre-aggregated, queryable across nodes." 4. "Summary: client-side percentile calculation. Cheaper but can't aggregate across instances."

A solid answer:

```
// Micrometer examples
private final MeterRegistry registry;

// Counter – requests served
Counter requests = registry.counter("http.requests", "method", "GET");
requests.increment();

// Gauge – current in-flight (must give a reference for it to track)
AtomicInteger inFlight = new AtomicInteger();
registry.gauge("http.inflight", inFlight);

// Histogram – request latency (queryable percentiles)
Timer latency = Timer.builder("http.latency")
    .publishPercentileHistogram() // ships buckets, server-side aggregation
    .register(registry);
latency.record(Duration.ofMillis(120));

// Summary – client-side p99 (can't merge across pods)
DistributionSummary requestSize = DistributionSummary.builder("http.requestSize")
    .publishPercentiles(0.5, 0.95, 0.99)
    .register(registry);
requestSize.record(2048);
```

- Counter: `rate()` in Prometheus = derivative. Useful for "QPS"
- Histogram with `publishPercentileHistogram()` ships pre-bucketed counts — aggregable across pods
- Summary calculates percentiles in-process — can't combine; mostly avoid for multi-pod

Tradeoffs: - Histogram cardinality: bucket count × labels = many series; tune buckets to your latency range - Gauge requires a live reference; auto-decay if not kept up to date can mislead

What NOT to say: - "Counter for everything" — gauge for in-flight, histogram for latency are different shapes - "Use Summary always for percentiles" — can't aggregate across pods

Common follow-up: "Why can't you average histograms but you can aggregate their buckets?"

Scenario 145 · Error budget / SLO tracking

Asked at: Razorpay · Cred · SRE-aware shops · **Difficulty:** Medium · **Topic:** SRE

The interviewer's prompt: "Service has 99.9% availability SLO. Design a service to track current error budget consumption."

Thinking out loud: 1. "99.9% = 0.1% error budget = 43m 12s per month." 2. "Error budget = (1 - SLO) × total time. Consumed = (failed requests / total requests) × budget." 3. "Burn rate alarms: alert when burning faster than steady consumption rate."

A solid answer:

```
class SloTracker {
    private final double targetAvailability = 0.999;
    private final Duration windowDuration = Duration.ofDays(30);

    public Slo state() {
        long total = metrics.totalRequests(windowDuration);
        long failed = metrics.failedRequests(windowDuration);
        long allowedFailures = (long) (total * (1 - targetAvailability));
        long remaining = allowedFailures - failed;
        double consumedPercent = (double) failed / Math.max(1, allowedFailures) * 100;
        return new Slo(total, failed, allowedFailures, remaining, consumedPercent);
    }
}

// Burn-rate alerts (Prometheus query):
// rate(http_requests_total{status=~"5.."}[1h]) /
//   rate(http_requests_total[1h]) > 14.4 * (1 - 0.999)
// "burning 14.4x normal rate over 1h = will deplete monthly budget in ~2d"
```

- Multi-window, multi-burn-rate alerts (Google SRE) catch both fast burns (page now) and slow burns (page later)
- Visualize budget over time: dashboard with "X% consumed, Y days into the window"

Tradeoffs: - Choosing the SLI carefully: counts of failed HTTP responses? Successful but slow?

Synthetic probes? - Rolling window vs calendar window: rolling is more accurate; calendar is easier to reason about for ops - When budget exhausted, freeze risky deploys — encodes "reliability vs feature velocity" tradeoff

What NOT to say: - "Set SLO at 99.99% to be safe" — that's a 4m 23s/month budget; almost no room for legitimate ops - "Use uptime monitoring tool" — doesn't capture user-facing SLI

Common follow-up: "What's a 'multi-window burn rate alert' and why is it better than threshold alerts?"

Scenario 146 · Sampling strategy for high-volume tracing

Asked at: Razorpay · Cred · Atlassian GCC · **Difficulty:** Hard · **Topic:** Observability

The interviewer's prompt: "1M traces/sec. Storing all is \$100k/month. Design a sampling strategy that keeps the useful ones."

Thinking out loud: 1. "Head sampling: decide at root span before any work. Easy but blind to outcome." 2. "Tail sampling: collect spans, decide after trace completes (success/fail, latency). Smarter, more memory." 3. "Combine: head-sample 1%, tail-keep all errors + slow traces."

A solid answer:

```
class TailSamplingDecider {
    public SampleDecision decide(Trace trace) {
        // Always keep errors
        if (trace.hasError()) return SampleDecision.KEEP;
        // Always keep slow traces
        if (trace.duration().compareTo(Duration.ofSeconds(2)) > 0) return SampleDecision.KEEP;
        // Always keep traces with rare endpoints
        if (rareEndpoints.contains(trace.rootEndpoint())) return SampleDecision.KEEP;
        // Random sample of the rest
        return ThreadLocalRandom.current().nextDouble() < 0.01 ? SampleDecision.KEEP : SampleDecision.DROP;
    }
}

// Head sampling at SDK level – for cost ceiling:
class HeadSampler {
    private final double rate;    // 1% = 0.01
    public boolean sample(String traceId) {
        // Hash to make decision deterministic for the trace
        return Math.abs(traceId.hashCode() % 10000) < rate * 10000;
    }
}
```

- Tail sampling needs a collector layer (OpenTelemetry Collector) that buffers spans until trace completes
- Hash-based head sampling: same trace id always sampled same way — consistent across services

Tradeoffs: - Tail sampling: more useful data per byte stored, but buffer memory cost + complexity - Head sampling: predictable cost, simpler — but loses many error traces - Probabilistic sampling: blind to outcome; rare-error traces nearly always missed

What NOT to say: - "Sample 0.1% randomly" — most errors lost - "Keep everything for 24h then delete" — storage cost same as keep-everything

Common follow-up: "Why can't you do tail sampling at the SDK?"

Scenario 147 · Application config hot reload

Asked at: Razorpay · Cred · Postman · **Difficulty:** Medium · **Topic:** Operations

The interviewer's prompt: "Change a config value (rate limit, feature flag, retry count) without restarting pods. Design."

Thinking out loud: 1. "Config in centralized store (Consul, etcd, Spring Cloud Config, AWS AppConfig)." 2. "App watches for changes, refreshes in-memory copy on event." 3. "Per-config refresh callback — code that depends on the value re-initializes."

A solid answer:

```
class DynamicConfig {
    private final AtomicReference<ConfigSnapshot> current = new AtomicReference<>();
    private final List<Consumer<ConfigSnapshot>> listeners = new CopyOnWriteArrayList<>();

    public void start() {
        current.set(loadFromStore());
        configStore.watchForChanges(change -> {
            ConfigSnapshot newCfg = loadFromStore();
            current.set(newCfg);
            for (var l : listeners) l.accept(newCfg);
        });
    }

    public <T> T get(String key, T defaultValue) {
        return current.get().get(key, defaultValue);
    }

    public void onChange(Consumer<ConfigSnapshot> cb) {
        listeners.add(cb);
    }
}

// Usage:
config.onChange(cfg -> {
    int newLimit = cfg.get("rate.limit", 100);
    rateLimiter.updateLimit(newLimit);
});
```

- AtomicReference swap = lock-free refresh; readers see consistent snapshot
- Listeners notify subsystems that need re-initialization

Tradeoffs: - Some configs trigger expensive re-init (rebuild a thread pool); guard with "did this actually change?" check - Validate new config *BEFORE* swapping — bad config rolled out across fleet = outage. Canary first. - Spring Cloud has @RefreshScope; simpler for Spring users but adds complexity

What NOT to say: - "Restart pods on config change" — kills connections, bumps latency - "Read from file each call" — high IO; race conditions during reload

Common follow-up: "What if the new config is broken — how do you avoid rolling out the breakage to all pods?"

Scenario 148 · Per-tenant noisy neighbor detection

Asked at: Razorpay · Cred · multi-tenant SaaS · **Difficulty:** Medium · **Topic:** Operations

The interviewer's prompt: "Multi-tenant DB. One tenant's bad query is killing performance for everyone. How do you detect?"

Thinking out loud: 1. "Per-tenant metrics: query count, latency, CPU time, bytes scanned." 2. "Alert when one tenant exceeds, say, 50% of total CPU or 10x median query time." 3. "Mitigation: throttle that tenant temporarily; engage them to optimize."

A solid answer:

```

class PerTenantMetrics {
    private final ConcurrentHashMap<String, TenantStats> stats = new ConcurrentHashMap<>();

    public void recordQuery(String tenantId, long latencyMs, long cpuMs) {
        TenantStats s = stats.computeIfAbsent(tenantId, k -> new TenantStats());
        s.queries.increment();
        s.totalCpu.add(cpuMs);
        s.latencyHistogram.record(latencyMs);
    }

    // Periodic check (every minute):
    public List<String> detectNoisyNeighbors() {
        long totalCpu = stats.values().stream().mapToLong(s -> s.totalCpu.sum()).sum();
        return stats.entrySet().stream()
            .filter(e -> e.getValue().totalCpu.sum() > totalCpu * 0.5) // > 50% of total
            .map(Map.Entry::getKey)
            .toList();
    }
}

// Mitigation: temporary rate limit for noisy tenants
class TenantThrottler {
    public boolean allow(String tenantId) {
        if (noisyTenants.contains(tenantId)) return strictBucket.tryConsume(tenantId);
        return normalBucket.tryConsume(tenantId);
    }
}

```

- Per-tenant cardinality is a metrics concern — bound it (e.g., top-100 active tenants per minute, others bucketed)
- Detection + automated throttle = self-healing

Tradeoffs: - High-cardinality metrics: 10k tenants × 5 metrics = 50k series. Watch Prometheus memory. - False positives: a legitimate big tenant looks noisy. Combine with anomaly detection (is this a sudden spike?) - Throttling can hurt legitimate workloads — usually escalate to human-in-the-loop for tier-1 tenants

What NOT to say: - "Look at slow query log manually" — doesn't scale to thousands of tenants - "Round-robin all queries equally" — doesn't solve the underlying problem

Common follow-up: "How do you avoid metric cardinality explosion with 10k tenants?"

Scenario 149 · Profiling production safely

Asked at: Cred · Razorpay · platform-engineering · **Difficulty:** Hard · **Topic:** Operations

The interviewer's prompt: "CPU spiked on prod pod. Want to know which method is hot. Heap dump? Profiler? Design the on-demand profiling flow."

Thinking out loud: 1. "async-profiler — low-overhead sampling profiler. Attach to running JVM, get flame graph." 2. "JFR (Java Flight Recorder) for continuous low-overhead profiling —

`-XX:StartFlightRecording` ." 3. "On-demand: HTTP endpoint that starts a 30s recording, returns the file."

A solid answer:

```
@RestController
@RequestMapping("/internal/debug")
class DebugController {
    @PostMapping("/profile")
    public ResponseEntity<byte[]> profileFor(@RequestParam int seconds) throws Exception {
        if (!isAdminCaller()) return ResponseEntity.status(403).build();
        Path output = Files.createTempFile("profile", ".jfr");
        // Start JFR via JMX or programmatic API:
        Recording recording = new Recording();
        recording.enable("jdk.CPULoad");
        recording.enable("jdk.MethodSample");
        recording.setDestination(output);
        recording.start();
        Thread.sleep(seconds * 1000L);
        recording.stop();
        recording.close();
        return ResponseEntity.ok()
            .header("Content-Disposition", "attachment; filename=profile.jfr")
            .body(Files.readAllBytes(output));
    }
}
```

- JFR runs at <2% overhead — safe to leave on continuously at low sampling
- async-profiler captures kernel + user stack — finds GC pauses, page faults too

Tradeoffs: - Heap dumps are expensive (stop-the-world, multi-GB files). Sample-based profiling is usually better. - Restrict access — profile data leaks code structure and inputs - For container envs (K8s), trigger via debug sidecar or kubectl exec — don't expose this endpoint publicly

What NOT to say: - "Use jstack" — single thread dump; useless for finding hot methods - "Restart the pod and hope" — loses state; doesn't diagnose

Common follow-up: "What's the overhead difference between async-profiler and JFR?"

Scenario 150 · Synthetic monitoring / canary

Asked at: Razorpay · Cred · Postman · SRE roles · **Difficulty:** Medium · **Topic:** Operations

The interviewer's prompt: "Real users don't always exercise every code path. Design synthetic monitoring that fakes a transaction every minute."

Thinking out loud: 1. "Cron-style runner hits your API endpoints with known inputs, verifies outputs." 2. "Run from multiple regions (or third-party tools like Pingdom, Datadog Synthetics)." 3. "Alert on failures; track p99 latency over time."

A solid answer:

```
@Scheduled(fixedRate = 60_000)
public void syntheticCheck() {
    SyntheticResult result = runFlow();
    metrics.gauge("synthetic.latency", result.latencyMs(),
        Tags.of("flow", "checkout"));
    if (!result.success()) {
        alerts.page("Synthetic checkout failed: " + result.error());
    }
}

private SyntheticResult runFlow() {
    long start = System.currentTimeMillis();
    try {
        String token = api.login("synthetic-user@example.com", SYNTH_PASSWORD);
        String orderId = api.createOrder(token, new OrderReq(items, address));
        Order order = api.pollOrderUntil(orderId, OrderStatus.CONFIRMED, Duration.ofSeconds(30));
        api.cancelOrder(token, orderId); // clean up
        return SyntheticResult.success(System.currentTimeMillis() - start);
    } catch (Exception e) {
        return SyntheticResult.failure(e.getMessage(), System.currentTimeMillis() - start);
    }
}
```

- Run from outside your network (Datadog Synthetics, AWS CloudWatch Synthetics) — catches DNS / LB issues
- Designate test user accounts; ensure synthetic data doesn't pollute analytics

Tradeoffs: - Don't synthetic-trigger expensive operations (real payment, real shipping). Use shadow mode where possible - Synthetic-only failures vs real-user issues — sometimes diverge. Combine synthetic with RUM (real user monitoring) - Single point of failure: synthetic runner pod down → no signal → false confidence. Run from multiple sources.

What NOT to say: - "Real user monitoring is enough" — quiet periods or new code paths go untested - "Hit /health every minute" — checks process aliveness only, not end-to-end behavior

Common follow-up: "Synthetic monitor passes but real users complain — how do you reconcile?"

Category 3 — What Will This Output

Java Interview Prep — 2026 Edition · Learn & Excel · Scenario Library

100 short Java snippets where the interviewer drops the code on a shared screen and asks one question: what does this print? Each one hides a non-obvious behaviour — Integer cache boundaries, switch fall-through, finally swallowing returns, lambda capture rules, stream short-circuiting, generic erasure surprises, double precision, and the rest of the trapdoors that Indian product-co interviewers love. Walk through the reasoning out loud the way you would in the room; the gotcha explanation tells you exactly what to anchor on so you don't get caught next time.

Scenario 1 · Integer cache boundary at 127

Asked at: Razorpay · Swiggy · TCS Digital · Cred

Difficulty: Medium · **Topic:** Autoboxing — Integer cache

The code:

```
public class Cache127 {  
    public static void main(String[] args) {  
        Integer a = 127;  
        Integer b = 127;  
        Integer c = 128;  
        Integer d = 128;  
        System.out.println(a == b);  
        System.out.println(c == d);  
    }  
}
```

The interviewer's prompt: "What does this print? Walk me through both lines."

Thinking out loud: 1. "Both lines look identical — same literal value assigned twice, compared with `==`. But `Integer` is a reference type, so `==` checks identity, not value." 2. "The JVM caches Integer instances for values in the range -128 to 127. So `a` and `b` both point to the same cached object — true. But 128 is outside the cache, so `c` and `d` are two separate new Integer objects — false."

The actual output:

```
true  
false
```

Why (the non-obvious thing): `IntegerCache` (an inner class of `Integer`) pre-creates instances for -128..127 during JVM startup. Autoboxing of any `int` in that range returns the cached reference. Outside that range, autoboxing calls `new Integer(value)` — fresh object every time, so `==` returns false.

What NOT to say: - "Both print true because the values are equal" — `==` on wrapper types compares references, not values - "Both print false because they're separate objects" — misses the cache optimization

Common follow-up: "How would you fix the second line so it returns true without changing the values?" — use `.equals()` or compare unboxed `int` s.

Scenario 2 · String literal vs new String() pool behaviour

Asked at: Infosys · Wipro · Accenture · TCS

Difficulty: Easy-Medium · **Topic:** String pool

The code:

```
public class Pool {  
    public static void main(String[] args) {  
        String a = "hello";  
        String b = "hello";  
        String c = new String("hello");  
        System.out.println(a == b);  
        System.out.println(a == c);  
        System.out.println(a.equals(c));  
    }  
}
```

The interviewer's prompt: "Three booleans — give me each one with reasoning."

Thinking out loud: 1. "`a` and `b` are both string literals — the compiler interns them, so both point to the same object in the string pool. `a == b` is true." 2. "`c` uses `new String(...)` — that bypasses the pool and allocates a fresh object on the heap, even though its content is also `"hello"`. `a == c` compares references, so false." 3. "`a.equals(c)` compares content character by character — true."

The actual output:

```
true  
false  
true
```

Why (the non-obvious thing): String literals are interned at class-load time into the string pool. `new String(...)` explicitly creates a new heap object outside the pool, even when an identical literal exists.

What NOT to say: - "`new String(\"hello\")` returns the pooled string" — false, that's the whole point of using `new` - "All three return true because the string is the same" — confuses identity with equality

Common follow-up: "What does `c.intern() == a` print?" — true, because `intern()` returns the pool reference.

Scenario 3 · finally overrides return

Asked at: Cred · Razorpay · JP Morgan · Goldman Sachs

Difficulty: Medium · **Topic:** try-finally control flow

The code:

```
public class FinallyReturn {  
    static int compute() {  
        try {  
            return 1;  
        } finally {  
            return 2;  
        }  
    }  
    public static void main(String[] args) {  
        System.out.println(compute());  
    }  
}
```

The interviewer's prompt: "What does `compute()` return?"

Thinking out loud: 1. "The try block has `return 1` — normally that would push 1 onto the stack and exit. But before the method actually returns, the finally block runs." 2. "The finally block has its own

`return 2` . A return inside finally overrides any pending return from try. So the method returns 2, and the try's return value is silently discarded."

The actual output:

2

Why (the non-obvious thing): A `return` inside `finally` always wins over a `return` (or thrown exception) from `try` or `catch` . This is why putting `return` in finally is a serious code smell — it swallows exceptions and overrides intended values without warning.

What NOT to say: - "It returns 1, finally just cleans up" — finally can hijack the return - "It throws an exception" — no exception path here

Common follow-up: "What if try threw a RuntimeException and finally had `return 2` ?" — the exception is swallowed, method returns 2 silently.

Scenario 4 · Switch fall-through without break

Asked at: TCS · Infosys · Wipro · Cognizant

Difficulty: Easy · **Topic:** switch statement

The code:

```
public class FallThrough {
    public static void main(String[] args) {
        int x = 2;
        switch (x) {
            case 1: System.out.println("one");
            case 2: System.out.println("two");
            case 3: System.out.println("three");
            default: System.out.println("default");
        }
    }
}
```

The interviewer's prompt: "What prints?"

Thinking out loud: 1. "`x` is 2. The switch jumps to `case 2` . None of the cases have `break` , so execution falls through every subsequent case until the end of the switch." 2. "So we print 'two', then 'three', then 'default'."

The actual output:

```
two
three
default
```

Why (the non-obvious thing): The classic C-style switch falls through to subsequent cases unless `break` is explicitly written. `default` is not a special escape — it's just another label, and if execution reaches it, it runs.

What NOT to say: - "It prints only 'two'" — assumes implicit break - "It throws an error because there's no break" — switch without break is legal, just dangerous

Common follow-up: "How does the Java 14+ arrow-form switch (`case 2 -> ...`) change this?" — arrow form never falls through.

Scenario 5 · Integer overflow at MAX_VALUE

Asked at: Razorpay · PhonePe · Zerodha · Walmart Labs

Difficulty: Easy-Medium · **Topic:** Integer overflow

The code:

```
public class Overflow {
    public static void main(String[] args) {
        int max = Integer.MAX_VALUE;
        System.out.println(max + 1);
        System.out.println(max * 2);
    }
}
```

The interviewer's prompt: "Two prints — give me both values."

Thinking out loud: 1. " `Integer.MAX_VALUE` is 2,147,483,647. Adding 1 overflows — int wraps around to `Integer.MIN_VALUE`, which is -2,147,483,648." 2. " `max * 2` is overflow too. $2,147,483,647 \times 2$ in unbounded math is 4,294,967,294. In 32-bit signed int, that wraps to -2."

The actual output:

```
-2147483648
-2
```

Why (the non-obvious thing): Java's int arithmetic is silent two's-complement wrap-around on overflow — no exception, no warning. The only way to detect it is `Math.addExact(...)` / `Math.multiplyExact(...)` which throw `ArithmeticException`.

What NOT to say: - "It throws an exception on overflow" — Java doesn't, by default - "It saturates at MAX_VALUE" — that's some other languages, not Java

Common follow-up: "How would you compute `max + 1` safely?" — cast one operand to `long`, or use `Math.addExact`.

Scenario 6 · Static initializer exception

Asked at: Cred · Razorpay · Amazon India

Difficulty: Hard · **Topic:** Static initialization

The code:

```
public class StaticInit {
    static int x;
    static {
        x = 10 / 0;
    }
    public static void main(String[] args) {
        System.out.println(x);
    }
}
```

The interviewer's prompt: "Compile or runtime? If runtime, what's the exception?"

Thinking out loud: 1. "Compiles fine — the divide-by-zero is a runtime expression, not a compile-time constant." 2. "At runtime, before `main` runs, the static initializer block executes. `10 / 0` throws `ArithmeticException`." 3. "But that exception comes from inside a static initializer — the JVM wraps it in `ExceptionInInitializerError` and the class fails to initialize. `main` never runs."

The actual output:

```
Exception in thread "main" java.lang.ExceptionInInitializerError
Caused by: java.lang.ArithmeticException: / by zero
```

Why (the non-obvious thing): Any exception (checked or unchecked) thrown from a static initializer gets wrapped in `ExceptionInInitializerError`. The class enters a permanently failed state — any subsequent reference triggers `NoClassDefFoundError`, not a re-attempt.

What NOT to say: - "Prints 0 because x defaults to 0" — initialization fails before main runs - "Throws `ArithmeticException` directly" — it's wrapped

Common follow-up: "What happens if another class later does `new StaticInit()` ?" — `NoClassDefFoundError`, not a re-initialization.

Scenario 7 · Lambda capturing a loop variable

Asked at: Razorpay · Swiggy · Postman · Chargebee

Difficulty: Medium · **Topic:** Lambda capture

The code:

```
import java.util.*;
import java.util.function.*;

public class LambdaCapture {
    public static void main(String[] args) {
        List<Supplier<Integer>> suppliers = new ArrayList<>();
        for (int i = 0; i < 3; i++) {
            final int captured = i;
            suppliers.add(() -> captured);
        }
        suppliers.forEach(s -> System.out.println(s.get()));
    }
}
```

The interviewer's prompt: "What does this print? Is the loop variable handling correct?"

Thinking out loud: 1. "The lambda captures `captured`, a fresh effectively-final variable per iteration. Each lambda holds its own snapshot — 0, 1, 2." 2. "If the code had captured `i` directly, that wouldn't compile — `i` is mutated by `i++`, so it's not effectively final." 3. "Each supplier returns its captured value in insertion order."

The actual output:

```
0
1
2
```

Why (the non-obvious thing): Lambdas can only capture effectively final local variables. The fresh `final int captured = i` per iteration is the standard workaround. Each lambda gets its own captured snapshot — unlike JavaScript's classic `var` closure trap, this works correctly in Java.

What NOT to say: - "All three print 3 because of the closure" — that's JavaScript's old `var` behaviour, not Java - "Compile error" — the explicit `captured` makes it fine

Common follow-up: "What if I replace `captured` with `i` in the lambda?" — compile error: local variable referenced from lambda must be final or effectively final.

Scenario 8 · Stream short-circuit with findFirst

Asked at: Razorpay · Cred · Walmart Labs

Difficulty: Medium · **Topic:** Streams — short-circuit

The code:

```
import java.util.stream.*;

public class StreamFindFirst {
    public static void main(String[] args) {
        int result = Stream.of(1, 2, 3, 4, 5)
            .peek(n -> System.out.println("peek " + n))
            .filter(n -> n > 2)
            .findFirst()
            .get();
        System.out.println("result " + result);
    }
}
```

The interviewer's prompt: "Trace the output line by line."

Thinking out loud: 1. "Streams are lazy. `findFirst` is a short-circuit terminal operation — it stops as soon as it finds the first match." 2. "Element 1: peek prints 'peek 1', filter rejects (1 > 2 false)." 3. "Element 2: peek prints 'peek 2', filter rejects." 4. "Element 3: peek prints 'peek 3', filter accepts, `findFirst` captures 3 and short-circuits — elements 4 and 5 never flow through." 5. "Then `result 3` prints."

The actual output:

```
peek 1
peek 2
peek 3
result 3
```

Why (the non-obvious thing): Streams pull elements one at a time through the pipeline. Short-circuiting terminals (`findFirst` , `findAny` , `anyMatch` , `allMatch` , `noneMatch`) stop processing as soon as the answer is known. `peek` only fires for elements that actually flow through.

What NOT to say: - "peek prints all 5 elements first, then the filter runs" — wrong; streams aren't loops over an intermediate list - "result is 1 because `findFirst` returns the first element of the source" — `findFirst` is post-filter

Common follow-up: "What if I replace `findFirst` with `count` ?" — `peek` fires for all 5; `count` needs to traverse the whole filter chain.

Scenario 9 · ConcurrentModificationException

Asked at: TCS · Infosys · Cognizant · most Java rounds

Difficulty: Easy-Medium · **Topic:** Fail-fast iterator

The code:

```
import java.util.*;

public class CME {
    public static void main(String[] args) {
        List<Integer> list = new ArrayList<>(Arrays.asList(1, 2, 3, 4, 5));
        for (Integer n : list) {
            if (n == 3) list.remove(n);
        }
        System.out.println(list);
    }
}
```

The interviewer's prompt: "Will this print the list without the 3? Compile? Runtime?"

Thinking out loud: 1. "Enhanced-for uses the Iterator under the hood. ArrayList's iterator tracks `modCount` . If the list is structurally modified outside the iterator (i.e. directly via `list.remove(...)`), the next call to `iterator.next()` throws `ConcurrentModificationException`." 2. "After removing 3, the loop advances — `next()` checks `modCount` mismatch — boom, CME."

The actual output:

```
Exception in thread "main" java.util.ConcurrentModificationException
```

Why (the non-obvious thing): The fail-fast iterator detects modification via a `modCount` counter.

Removing through the iterator (`it.remove()`) updates both `modCount` and `expectedModCount`, keeping them in sync. Direct list modification only bumps `modCount`.

What NOT to say: - "Prints [1, 2, 4, 5]" — assumes the loop survives the removal - "ArrayList isn't thread-safe so it throws CME" — CME is about concurrent modification of the iterator state, not threading

Common follow-up: "What's the safe way to remove during iteration?" — `Iterator.remove()` , `removeIf(...)` , or stream's `filter`.

Scenario 10 · HashSet ignoring a broken hashCode

Asked at: Razorpay · Swiggy · Cred

Difficulty: Medium · **Topic:** hashCode>equals contract

The code:

```
import java.util.*;

class Bad {
    int id;
    Bad(int id) { this.id = id; }
    @Override public boolean equals(Object o) {
        return o instanceof Bad && ((Bad) o).id == this.id;
    }
    // hashCode not overridden
}

public class BrokenContract {
    public static void main(String[] args) {
        Set<Bad> s = new HashSet<>();
        s.add(new Bad(1));
        System.out.println(s.contains(new Bad(1)));
    }
}
```

The interviewer's prompt: "Should this print true or false?"

Thinking out loud: 1. "`equals` is overridden but `hashCode` is not. The default `Object.hashCode()` returns an identity-based hash — different for every instance." 2. "HashSet first hashes the lookup key, then walks the bucket calling equals. Two `Bad(1)` instances have different hash codes, so they go to different buckets. `contains` looks in the wrong bucket and finds nothing."

The actual output:

```
false
```

Why (the non-obvious thing): Overriding `equals` without `hashCode` silently breaks every hash-based collection. The objects look equal by your contract but live in different buckets, so lookups, removals, and set semantics all fail.

What NOT to say: - "true because equals returns true" — HashSet doesn't scan the whole table; it goes to a bucket - "Throws an exception" — Java doesn't enforce the contract; it just gives wrong answers

Common follow-up: "Fix it — what's the minimal change?" — add `@Override public int hashCode() { return Integer.hashCode(id); }`.

Scenario 11 · Double precision and 0.1 + 0.2

Asked at: Goldman Sachs · JP Morgan · Razorpay · fintech rounds

Difficulty: Easy · **Topic:** Floating-point precision

The code:

```
public class DoubleSum {
    public static void main(String[] args) {
        double sum = 0.1 + 0.2;
        System.out.println(sum);
        System.out.println(sum == 0.3);
    }
}
```

The interviewer's prompt: "Two prints. What and why?"

Thinking out loud: 1. "0.1 and 0.2 can't be represented exactly in binary floating-point (IEEE 754). Their sum has a tiny representation error." 2. "`0.1 + 0.2` prints as 0.30000000000000004 — the visible tail of the binary representation when rendered as a decimal." 3. "0.3 also has its own tiny error but it's a different one. The two values don't compare equal."

The actual output:

```
0.30000000000000004  
false
```

Why (the non-obvious thing): Doubles are base-2; many base-10 decimals are infinite repeating binary fractions, so they round to the nearest representable double. The errors compound through arithmetic. Never use `==` with floating-point — compare with a tolerance, or use `BigDecimal`.

What NOT to say: - "Prints 0.3 and true" — assumes infinite precision - "Java is buggy here" — IEEE 754 is the spec, working as designed

Common follow-up: "How would you represent money in Java?" — `BigDecimal`, never `double`.

Scenario 12 · Ternary type widening to double

Asked at: Infosys · Wipro · Accenture

Difficulty: Medium · **Topic:** Ternary expression typing

The code:

```
public class TernaryWiden {  
    public static void main(String[] args) {  
        int x = 5;  
        Object o = (x > 0) ? 1 : 1.0;  
        System.out.println(o.getClass().getName());  
    }  
}
```

The interviewer's prompt: "What's the runtime type of `o`?"

Thinking out loud: 1. "The ternary needs a single result type. The two branches are `int` (1) and `double` (1.0). Java widens both to the common numeric type — `double`." 2. "So the result type is `double`. To assign to `Object`, it autoboxes to `Double` — even when the chosen branch is the `int`." 3. "Class name is `java.lang.Double`."

The actual output:

```
java.lang.Double
```


Why (the non-obvious thing): The ternary's static type is determined at compile time from both arms, not the chosen arm. If one arm is `int` and the other is `double`, both arms are typed as `double`. The `int 1` becomes `1.0d`, then autoboxes to `Double`.

What NOT to say: - "It's Integer because the condition is true" — type isn't chosen at runtime - "Compile error — mixed types" — Java widens silently

Common follow-up: "What if both branches are different reference types?" — Java picks the common supertype (`Object` worst case).

Scenario 13 · Varargs vs array ambiguity

Asked at: Razorpay · BrowserStack · senior backend

Difficulty: Hard · **Topic:** Varargs resolution

The code:

```
public class Varargs {
    static void run(Object... args) { System.out.println("Object varargs"); }
    static void run(Integer... args) { System.out.println("Integer varargs"); }
    public static void main(String[] args) {
        run(1, 2, 3);
        run((Object) null);
    }
}
```

The interviewer's prompt: "Two calls — which overload wins each time?"

Thinking out loud: 1. "`run(1, 2, 3)`" — both overloads match after autoboxing (1, 2, 3 box to `Integer`). Java picks the most specific. `Integer...` is more specific than `Object...`, so `Integer varargs` wins." 2. "`run((Object) null)`" — explicit cast forces it to `Object`, so only `Object...` matches. `Object varargs` wins."

The actual output:

```
Integer varargs
Object varargs
```

Why (the non-obvious thing): Java's overload resolution picks the most specific applicable method. `Integer` is a subtype of `Object`, so `Integer...` beats `Object...` when both apply. An explicit cast on a null literal disambiguates which overload to bind.

What NOT to say: - "Both call Object varargs" — Integer is more specific - "First call is ambiguous and won't compile" — autoboxing + specificity disambiguates

Common follow-up: "What does plain `run(null)` do?" — ambiguous, compile error.

Scenario 14 · Generic erasure — overloading conflict

Asked at: Cred · Razorpay · senior rounds

Difficulty: Hard · **Topic:** Generic type erasure

The code:

```
import java.util.List;

public class Erasure {
    static void process(List<String> list) {}
    static void process(List<Integer> list) {}

    public static void main(String[] args) {
        System.out.println("compiles?");
    }
}
```

The interviewer's prompt: "Compiles or not? If not, why?"

Thinking out loud: 1. "Generics are erased after compilation — both methods become `process(List list)` at the bytecode level." 2. "That's a duplicate method signature. The compiler rejects the class before runtime ever sees it."

The actual output:

```
Compile error: name clash: process(java.util.List<java.lang.Integer>) and
process(java.util.List<java.lang.String>) have the same erasure
```

Why (the non-obvious thing): Java generics are compile-time only. After erasure, `List<String>` and `List<Integer>` are both just `List`. Two methods can't have identical erased signatures, even if their generic parameter types differ.

What NOT to say: - "Compiles fine — different generic types are different overloads" — erasure makes them identical - "Runtime ambiguity — picks one at random" — fails at compile time

Common follow-up: "How would you make these two overloads coexist?" — different method names, or different non-generic parameter types.

Scenario 15 · `Optional.of(null)` vs `ofNullable`

Asked at: Walmart Labs · Postman · senior backend

Difficulty: Easy-Medium · **Topic:** Optional

The code:

```
import java.util.Optional;

public class OptionalNull {
    public static void main(String[] args) {
        Optional<String> a = Optional.ofNullable(null);
        System.out.println(a.isPresent());
        Optional<String> b = Optional.of(null);
        System.out.println(b.isPresent());
    }
}
```

The interviewer's prompt: "What prints, in order? Any exceptions?"

Thinking out loud: 1. "`Optional.ofNullable(null)` returns `Optional.empty()`. `isPresent` is false. Prints `false`." 2. "`Optional.of(null)` throws `NullPointerException` immediately — `of` rejects null." 3. "So we print `false`, then NPE before line 2's `isPresent` ever runs."

The actual output:

```
false
Exception in thread "main" java.lang.NullPointerException
```

Why (the non-obvious thing): `Optional.of(value)` is a contract — value must be non-null, NPE if violated. `Optional.ofNullable(value)` is null-safe and returns `empty()`. Use `of` only when null would itself be a bug.

What NOT to say: - "Both print false" — `of(null)` is an NPE trap - "Both throw NPE" — `ofNullable` is explicitly null-safe

Common follow-up: "When would you choose `of` over `ofNullable`?" — when null would represent a contract violation you want to catch loudly.

Scenario 16 · Try-with-resources suppressed exception

Asked at: Cred · JP Morgan · senior backend

Difficulty: Hard · **Topic:** Exception chaining/suppression

The code:

```
public class Suppressed {
    static class Res implements AutoCloseable {
        public void close() { throw new RuntimeException("close-fail"); }
    }
    public static void main(String[] args) {
        try (Res r = new Res()) {
            throw new RuntimeException("body-fail");
        }
    }
}
```

The interviewer's prompt: "What exception bubbles out? What about the other one?"

Thinking out loud: 1. "The body throws first — `body-fail` . Then `close()` runs (try-with-resources guarantees it) and throws `close-fail` ." 2. "Java keeps `body-fail` as the primary exception and adds `close-fail` to the suppressed list. The thrown exception that escapes the main method is `body-fail` ." 3. "Stack trace prints `body-fail` and then 'Suppressed: close-fail'."

The actual output:

```
Exception in thread "main" java.lang.RuntimeException: body-fail
    at Suppressed.main(...)
    Suppressed: java.lang.RuntimeException: close-fail
        at Suppressed$Res.close(...)
```

Why (the non-obvious thing): Without try-with-resources, the second exception would silently overwrite the first. The TWR construct preserves the original via `Throwable.addSuppressed(...)` , retrievable via `getSuppressed()` .

What NOT to say: - "close-fail wins because it's later" — TWR specifically prevents that - "Compile error — can't throw from close()" — close can throw, that's why suppression exists

Common follow-up: "What if I use plain try-finally and both throw?" — finally's exception wins, body's is lost.

Scenario 17 · Integer cache with explicit valueOf

Asked at: TCS · Infosys · Razorpay

Difficulty: Medium · **Topic:** Autoboxing

The code:

```
public class ValueOf {  
    public static void main(String[] args) {  
        Integer a = Integer.valueOf(100);  
        Integer b = Integer.valueOf(100);  
        Integer c = new Integer(100);  
        System.out.println(a == b);  
        System.out.println(a == c);  
    }  
}
```

The interviewer's prompt: "Two booleans — what and why?"

Thinking out loud: 1. " `valueOf(100)` uses the Integer cache. Both `a` and `b` get the same cached object. `a == b` is true." 2. " `new Integer(100)` explicitly allocates — bypasses cache. `a == c` compares cached vs heap object — false." 3. "Note: `new Integer(int)` is deprecated since Java 9 and removed in Java 16+. The interviewer's snippet might compile-warn or fail depending on JDK version."

The actual output (on JDK 17, with the deprecated constructor):

```
true  
false
```

(On JDK 16+, `new Integer(int)` is removed and the snippet won't compile.)

Why (the non-obvious thing): `Integer.valueOf` is cache-aware; the deprecated `new Integer(int)` always allocates fresh. Modern code should never use `new Integer(...)` — even when allowed, it defeats the cache.

What NOT to say: - "Both are true because the value is the same" — `==` is reference equality - "Both are false — `valueOf` doesn't cache" — `valueOf` is the cache entry point

Common follow-up: "Why was `new Integer(int)` removed?" — encourages cached `valueOf`, reduces accidental allocations.

Scenario 18 · String concatenation in loop vs StringBuilder

Asked at: TCS · Infosys · most fresher rounds

Difficulty: Easy · **Topic:** String immutability

The code:

```
public class StringLoop {
    public static void main(String[] args) {
        String s = "";
        for (int i = 0; i < 3; i++) {
            s += i;
        }
        System.out.println(s);
        System.out.println(s.length());
    }
}
```

The interviewer's prompt: "What prints?"

Thinking out loud: 1. " `s += i` is sugar for `s = s + i` . Each iteration creates a new String." 2. "After three iterations the value is '012'. Length 3."

The actual output:

```
012
3
```

Why (the non-obvious thing): The output is unsurprising — but the implementation is. Modern javac compiles `s += i` to `StringBuilder.append` , but only within one statement. Across iterations of a loop, the compiler can't pool a single builder, so you get N intermediate String objects. Acceptable for 3 iterations, catastrophic for 100k.

What NOT to say: - "Throws because String is immutable" — `+=` is legal sugar - "Prints '0 1 2' with spaces" — `i` autoboxes to its bare digit string

Common follow-up: "How would you do this for a million iterations?" — `StringBuilder` outside the loop.

Scenario 19 · Char arithmetic surprise

Asked at: TCS · Cognizant · Wipro

Difficulty: Easy · **Topic:** Char/int promotion

The code:

```
public class CharMath {  
    public static void main(String[] args) {  
        char a = 'A';  
        System.out.println(a + 1);  
        System.out.println((char)(a + 1));  
    }  
}
```

The interviewer's prompt: "What does each line print?"

Thinking out loud: 1. " `a + 1` " — char is promoted to int for arithmetic. 'A' is ASCII 65. $65 + 1 = 66$. Prints 66." 2. " `(char)(a + 1)` " casts back to char. 66 is 'B'. Prints B."

The actual output:

```
66  
B
```

Why (the non-obvious thing): Java promotes char to int for arithmetic. Without an explicit cast back, `println` sees an int and prints the numeric value, not the character.

What NOT to say: - "Both print B" — first line prints the int promotion - "Compile error — can't add char and int" — char promotes implicitly

Common follow-up: "What's `'1' + '1'` in Java?" — 98 (49 + 49 as ints), not '2' or 2.

Scenario 20 · Boolean autoboxing in collections

Asked at: Razorpay · senior backend

Difficulty: Medium · **Topic:** Wrapper caching

The code:

```
public class BoolCache {  
    public static void main(String[] args) {  
        Boolean a = true;  
        Boolean b = Boolean.valueOf(true);  
        Boolean c = new Boolean(true);  
        System.out.println(a == b);  
        System.out.println(a == c);  
    }  
}
```

The interviewer's prompt: "Two booleans — what and why?"

Thinking out loud: 1. " `Boolean.valueOf(true)` always returns `Boolean.TRUE` — a single cached static instance. So `a` and `b` are the same object. `a == b` is true." 2. " `new Boolean(true)` allocates fresh. `a == c` is false. Note this constructor is deprecated since Java 9 and removed in Java 16+."

The actual output (on a JDK that still allows the deprecated constructor):

```
true  
false
```

Why (the non-obvious thing): Boolean has only two valid values — Java caches both as `Boolean.TRUE` and `Boolean.FALSE` static finals. Autoboxing and `valueOf` always hit the cache. Explicit construction defeats it.

What NOT to say: - "Both true — Boolean is a singleton" — only the cache is, `new` still works (where supported) - "Both false — Booleans are always different references" — cache is universal

Common follow-up: "Why was `new Boolean(boolean)` removed?" — forces use of cached constants, lower allocation.

Scenario 21 · Equals on different wrapper types

Asked at: Infosys · TCS · Wipro

Difficulty: Medium · **Topic:** equals across wrapper types

The code:


```
public class WrapperEquals {  
    public static void main(String[] args) {  
        Integer i = 100;  
        Long l = 100L;  
        System.out.println(i.equals(l));  
        System.out.println(i.equals(100));  
    }  
}
```

The interviewer's prompt: "Two booleans — which is true?"

Thinking out loud: 1. " `Integer.equals(Object)` checks `if (obj instanceof Integer)` . `l` is a Long, not an Integer — fails the instanceof. Returns false." 2. " `i.equals(100)` — 100 autoboxes to Integer, instanceof Integer passes, values match. Returns true."

The actual output:

```
false  
true
```

Why (the non-obvious thing): Wrapper `equals` is strictly type-checking.

`Integer(100).equals(Long(100))` is false even though both represent the same numeric value. This trips people up when mixing collection types or doing cross-type comparisons.

What NOT to say: - "Both true — same numeric value" — equals checks class, not value - "First throws `ClassCastException`" — equals takes Object, never throws on type mismatch

Common follow-up: "How would you compare numeric values across wrapper types?" — compare unboxed `long` values: `i.longValue() == l.longValue()` .

Scenario 22 · Switch on null String

Asked at: Razorpay · BrowserStack

Difficulty: Easy-Medium · **Topic:** switch null behaviour

The code:

```
public class SwitchNull {  
    public static void main(String[] args) {  
        String s = null;  
        switch (s) {  
            case "x": System.out.println("x"); break;  
            default: System.out.println("default");  
        }  
    }  
}
```

The interviewer's prompt: "What happens?"

Thinking out loud: 1. "Classic switch on a String unboxes via String's hashCode internally. Passing null calls null.hashCode() — NPE." 2. "In modern Java (21+) with the pattern-matching switch, you can write `case null`. But classic switch always NPEs on null."

The actual output:

```
Exception in thread "main" java.lang.NullPointerException
```

Why (the non-obvious thing): Old-style switch on String dispatches via the String's hashCode. Java 21 introduced pattern-matching switch with `case null` support; until then, you must null-check before switching.

What NOT to say: - "Prints 'default' because no case matches" — null can't even enter the switch - "Compile error" — compiles fine, fails at runtime

Common follow-up: "How does Java 21 pattern switch handle this?" — `case null -> ...` is allowed as an explicit label.

Scenario 23 · Recursive call with overflow

Asked at: TCS Digital · Razorpay · senior rounds

Difficulty: Easy · **Topic:** StackOverflowError

The code:

```
public class Recurse {
    static int count = 0;
    static void run() {
        count++;
        run();
    }
    public static void main(String[] args) {
        try {
            run();
        } catch (StackOverflowError e) {
            System.out.println("depth approx " + (count / 1000) + "k");
        }
    }
}
```

The interviewer's prompt: "Does this run forever? Or terminate? With what?"

Thinking out loud: 1. "`run()` calls itself with no base case. Each call pushes a stack frame. Eventually the thread's stack overflows." 2. "StackOverflowError is an Error, not an Exception. It's catchable but generally shouldn't be caught. Here the main method catches it and prints the approximate depth." 3. "Depth depends on JVM settings (-Xss); roughly 10-20k frames for default 512KB stack."

The actual output (typical):

```
depth approx 14k
```

Why (the non-obvious thing): StackOverflowError is recoverable (the stack unwinds), but the depth depends on `-Xss`, the frame size, and how much was already in use. You shouldn't catch Errors in production code — this is for demonstration.

What NOT to say: - "Runs forever" — stack is finite - "Throws OutOfMemoryError" — that's heap; this is stack

Common follow-up: "How would you make `run()` not overflow?" — make it tail-recursive in a TCO language, or rewrite as iteration (Java's JIT doesn't do tail-call optimization).

Scenario 24 · Static field initialization order

Asked at: Razorpay · Cred · senior backend

Difficulty: Hard · **Topic:** Class initialization order

The code:

```
public class InitOrder {
    static int a = b + 1;
    static int b = 10;

    public static void main(String[] args) {
        System.out.println("a=" + a + " b=" + b);
    }
}
```

The interviewer's prompt: "What are a and b at the end?"

Thinking out loud: 1. "Static fields are initialized top-to-bottom in declaration order. When `a` is being initialized, `b` hasn't run yet — it still has its default value, 0." 2. "So `a = 0 + 1 = 1`. Then `b = 10` runs." 3. "Final values: a=1, b=10."

The actual output:

```
a=1 b=10
```

Why (the non-obvious thing): Forward references to static fields use the default value (0 for int, null for object, false for boolean), not the eventual assigned value. This is the static analog of the instance "this leak" problem.

What NOT to say: - "Compile error — forward reference" — Java allows it as long as you don't read in the initializer chain (you can write `b + 1` because `b` is referenced by simple name from a sibling initializer, which is permitted) - "a=11" — assumes `b` is already 10

Common follow-up: "What if I declare them as `static final`?" — then forward reference is illegal at compile time.

Scenario 25 · Exception in lambda inside stream

Asked at: Razorpay · Cred · senior

Difficulty: Medium · **Topic:** Stream + checked exception

The code:

```
import java.util.*;
import java.util.stream.*;

public class StreamCheckedEx {
    public static void main(String[] args) {
        List<String> list = Arrays.asList("1", "2", "abc", "4");
        int sum = list.stream()
            .mapToInt(Integer::parseInt)
            .sum();
        System.out.println(sum);
    }
}
```

The interviewer's prompt: "What prints?"

Thinking out loud: 1. "The stream parses each element. `'abc'` throws `NumberFormatException` (unchecked) when `Integer.parseInt` runs." 2. "Streams don't catch exceptions — the exception propagates up and aborts the pipeline." 3. "Nothing is printed; the JVM dumps the stack trace."

The actual output:

```
Exception in thread "main" java.lang.NumberFormatException: For input string: "abc"
```

Why (the non-obvious thing): Streams don't have a built-in error-handling stage. A `RuntimeException` in any element aborts the pipeline mid-flight, no partial result returned. For checked exceptions, you can't even pass the method reference — compile error.

What NOT to say: - "Sum is 7 — skips the bad element" — streams don't skip on error - "Catches and ignores by default" — they don't

Common follow-up: "How would you skip parse failures?" — wrap in a try/catch lambda or use a helper that returns `Optional`.

Scenario 26 · == vs equals on Strings from substring

Asked at: Infosys · TCS

Difficulty: Medium · **Topic:** String pool

The code:

```
public class SubstringPool {  
    public static void main(String[] args) {  
        String a = "hello";  
        String b = "hello world".substring(0, 5);  
        System.out.println(a == b);  
        System.out.println(a.equals(b));  
        System.out.println(a == b.intern());  
    }  
}
```

The interviewer's prompt: "Three booleans — give me each."

Thinking out loud: 1. " `a` is the pooled literal 'hello'. `b` is built at runtime by substring — heap object, not pooled. `a == b` compares references — false." 2. " `a.equals(b)` compares content — true." 3. " `b.intern()` returns the pool entry for 'hello' if one exists, otherwise adds b to the pool and returns it. Since 'hello' is already in the pool from `a`, intern returns the same reference as `a`. True."

The actual output:

```
false  
true  
true
```

Why (the non-obvious thing): String pool is opt-in for runtime-created strings — only literals and `intern()` go in. Most String operations (substring, concat, replace) return new heap objects unless the result equals the receiver exactly.

What NOT to say: - "All true because the content matches" — `==` is reference equality - "Intern always returns a new object" — intern returns the canonical pool reference

Common follow-up: "When does substring return the original string reference?" — when start=0 and end=length, substring may return `this` (implementation-detail in modern JDKs).

Scenario 27 · finally with System.exit

Asked at: Razorpay · senior backend

Difficulty: Hard · **Topic:** finally edge cases

The code:

```
public class FinallyExit {  
    public static void main(String[] args) {  
        try {  
            System.out.println("try");  
            System.exit(0);  
        } finally {  
            System.out.println("finally");  
        }  
    }  
}
```

The interviewer's prompt: "Does finally always run? What prints?"

Thinking out loud: 1. " `try` prints. Then `System.exit(0)` terminates the JVM immediately." 2. " `finally` is supposed to always run — but `System.exit` skips it. The JVM is shutting down; no Java code, including finally blocks, runs after that." 3. "Only 'try' prints."

The actual output:

```
try
```

Why (the non-obvious thing): `System.exit` is the one common case where finally doesn't run. Other cases: JVM crash (segfault, OOM in some configs), `Runtime.halt`, killing the process. Shutdown hooks run on `System.exit`, but try/finally do not.

What NOT to say: - "Both print — finally always runs" — exit short-circuits - "Prints in reverse order" — sequential

Common follow-up: "What if the finally block had `System.exit(1)`?" — same; exit always wins, prevents the rest of the JVM from continuing.

Scenario 28 · Array covariance runtime check

Asked at: Cred · senior backend

Difficulty: Hard · **Topic:** Array covariance

The code:

```
public class ArrayCovariance {  
    public static void main(String[] args) {  
        Object[] arr = new String[3];  
        arr[0] = "hello";  
        arr[1] = 42;  
        System.out.println(arr[0]);  
    }  
}
```

The interviewer's prompt: "Compile or runtime error? If runtime, which line?"

Thinking out loud: 1. "Arrays are covariant: `String[]` is a subtype of `Object[]`. So the assignment compiles." 2. "`arr[0] = \"hello\"` — fine, String into String slot." 3. "`arr[1] = 42` — autoboxes to Integer. The JVM checks the array's actual component type (String) against Integer. Mismatch — throws `ArrayStoreException`."

The actual output:

```
Exception in thread "main" java.lang.ArrayStoreException: java.lang.Integer
```

Why (the non-obvious thing): Array covariance is unsound — the compiler accepts `Object[] arr = new String[3]`, but every write has a runtime type check. Generics solved this with invariance, but arrays kept the original Java design for backward compatibility.

What NOT to say: - "Compile error — type mismatch" — arrays are covariant at compile time - "Stores anyway because it's an `Object[]`" — runtime check enforces component type

Common follow-up: "Why don't generics have this problem?" — generics are invariant: `List<String>` is not a subtype of `List<Object>`.

Scenario 29 · Boxing in Collections.min on empty

Asked at: Razorpay · TCS

Difficulty: Medium · **Topic:** Collections API

The code:


```
import java.util.*;

public class CollMin {
    public static void main(String[] args) {
        List<Integer> list = new ArrayList<>();
        System.out.println(Collections.min(list));
    }
}
```

The interviewer's prompt: "Result?"

Thinking out loud: 1. " `Collections.min` on an empty collection throws `NoSuchElementException`."
2. "It needs at least one element to return."

The actual output:

```
Exception in thread "main" java.util.NoSuchElementException
```

Why (the non-obvious thing): `Collections.min/max` predate `Optional` and don't have a sentinel return value. Always pre-check emptiness, or use streams:

`list.stream().min(Comparator.naturalOrder())` returns `Optional`.

What NOT to say: - "Returns null" — throws instead - "Returns 0 — int default" — `Integer` is a reference type, no default

Common follow-up: "How would the stream equivalent behave?" — returns `Optional.empty()`, no exception.

Scenario 30 · Comparator returning `Integer.MIN_VALUE`

Asked at: Cred · senior backend

Difficulty: Hard · **Topic:** Comparator contract

The code:

```
import java.util.*;

public class BadComparator {
    public static void main(String[] args) {
        Comparator<Integer> bad = (a, b) -> a - b;
        System.out.println(bad.compare(Integer.MIN_VALUE, 1));
    }
}
```

The interviewer's prompt: "What does compare return? Spot any bug."

Thinking out loud: 1. "a - b where a is Integer.MIN_VALUE and b is 1 — that's MIN_VALUE - 1, which overflows to MAX_VALUE." 2. "The compare method is supposed to return a negative number (a < b). It returns MAX_VALUE — positive — meaning the comparator thinks a > b. Wrong sort order."

The actual output:

```
2147483647
```

Why (the non-obvious thing): The subtraction trick for comparators silently breaks on overflow. Use `Integer.compare(a, b)` instead — it returns -1, 0, or 1 without arithmetic.

What NOT to say: - "Returns a negative number" — overflow makes it positive - "Throws ArithmeticException" — int arithmetic wraps silently

Common follow-up: "What's the fix?" — `Integer.compare(a, b)` or `Comparator.naturalOrder()`.

Scenario 31 · HashMap put with null key

Asked at: TCS · Infosys · Razorpay

Difficulty: Easy-Medium · **Topic:** HashMap null semantics

The code:

```
import java.util.*;

public class MapNull {
    public static void main(String[] args) {
        Map<String, String> m = new HashMap<>();
        m.put(null, "first");
        m.put(null, "second");
        System.out.println(m.get(null));
        System.out.println(m.size());
    }
}
```

The interviewer's prompt: "Two prints — what and why?"

Thinking out loud: 1. "HashMap allows one null key (stored in bucket 0). Second put with same key — overwrite." 2. "Get(null) returns 'second'. Size is 1."

The actual output:

```
second
1
```

Why (the non-obvious thing): HashMap is the lone Map in the standard library that accepts a null key. TreeMap, ConcurrentHashMap, and Hashtable all throw NullPointerException on null keys.

What NOT to say: - "Throws NPE on null key" — HashMap specifically allows it - "Both nulls coexist — size 2" — keys are equal, second overwrites

Common follow-up: "What if I used ConcurrentHashMap?" — NPE on null keys or values.

Scenario 32 · TreeMap with custom comparator inconsistent with equals

Asked at: Cred · senior backend

Difficulty: Hard · **Topic:** TreeMap ordering vs equals

The code:

```
import java.util.*;

public class TreeMapEquals {
    public static void main(String[] args) {
        TreeMap<String, Integer> m = new TreeMap<>(
            Comparator.comparing(String::length));
        m.put("abc", 1);
        m.put("xyz", 2);
        System.out.println(m.size());
        System.out.println(m.get("xyz"));
    }
}
```

The interviewer's prompt: "Size? Lookup result?"

Thinking out loud: 1. "TreeMap uses the comparator for ordering AND for key equality (not `equals()`). 'abc'.length() == 'xyz'.length() — comparator returns 0 — keys treated as equal." 2. "Second put overwrites the first. Size is 1." 3. "Get('xyz') — comparator says equal to 'abc' which holds value 2. Returns 2."

The actual output:

```
1
2
```

Why (the non-obvious thing): TreeMap defines key equality via the comparator, not `equals()`. If your comparator returns 0 for keys that aren't actually equal, the map silently collapses them. This is why `Comparator` documentation insists on consistency with equals for sorted collections.

What NOT to say: - "Size 2 — different keys" — comparator says they're equal - "Throws because the keys aren't equal" — TreeMap trusts the comparator

Common follow-up: "How would you fix this?" — chain a tiebreaker:

`Comparator.comparing(String::length).thenComparing(Comparator.naturalOrder())`.

Scenario 33 · String == after compile-time concat

Asked at: Infosys · TCS

Difficulty: Medium · **Topic:** String pool — compile-time constants

The code:

```
public class CompileConcat {  
    public static void main(String[] args) {  
        String a = "hello";  
        String b = "hel" + "lo";  
        String c = "hel";  
        String d = c + "lo";  
        System.out.println(a == b);  
        System.out.println(a == d);  
    }  
}
```

The interviewer's prompt: "Two booleans."

Thinking out loud: 1. `"hel" + "lo"` is a compile-time constant — the compiler folds it into the literal `"hello"`, which is the same pool entry as `a`. True." 2. `c + "lo"` — `c` is a variable. Concat happens at runtime via `StringBuilder`; result is a fresh heap String, not pooled. False."

The actual output:

```
true  
false
```

Why (the non-obvious thing): The compiler can fold compile-time constants (literals, `final` constants) at javac time. Anything involving a non-final variable defers to runtime concatenation and lands on the heap.

What NOT to say: - "Both true — same content" — runtime concat lives outside the pool - "Both false — compiler doesn't optimize String concat" — it does, for compile-time constants

Common follow-up: "What if I declare `final String c = "hel"`?" — compiler folds it, becomes true.

Scenario 34 · Calling overridden method from constructor

Asked at: Razorpay · senior backend

Difficulty: Hard · **Topic:** Constructor + virtual dispatch

The code:

```
class Parent {
    Parent() { init(); }
    void init() { System.out.println("Parent.init"); }
}
class Child extends Parent {
    int value = 42;
    @Override void init() { System.out.println("Child.init value=" + value); }
}
public class CtorOverride {
    public static void main(String[] args) {
        new Child();
    }
}
```

The interviewer's prompt: "What prints when I instantiate Child?"

Thinking out loud: 1. "Constructing Child first runs Parent's constructor. Parent calls `init()` — virtual dispatch resolves to Child.init." 2. "BUT: Child's field initializers haven't run yet. `value` is still its default — 0." 3. "So Child.init prints 'value=0', then Parent's constructor completes, then Child's field init runs (value = 42), then Child's constructor body (none here)."

The actual output:

```
Child.init value=0
```

Why (the non-obvious thing): Calling overridable methods from constructors is dangerous. The subclass's override runs before the subclass's fields are initialized. Effective Java's "don't call overridable methods from constructors" rule exists exactly for this.

What NOT to say: - "value=42" — fields aren't yet initialized - "Parent.init prints — not yet a Child" — virtual dispatch sees the runtime type Child

Common follow-up: "How would you make this safe?" — mark `init()` as final or private.

Scenario 35 · String.split with trailing empties

Asked at: TCS · Infosys · Razorpay

Difficulty: Medium · **Topic:** String.split

The code:

```
public class SplitTrailing {  
    public static void main(String[] args) {  
        String s = "a,b,,c,,";  
        String[] parts = s.split(",");  
        System.out.println(parts.length);  
    }  
}
```

The interviewer's prompt: "How many parts?"

Thinking out loud: 1. " `split` with no limit argument uses the default limit 0 — which strips trailing empty strings but keeps internal ones." 2. "Splitting 'a,b,,c,,' gives the conceptual array ['a', 'b', '', 'c', '', '']. Strip trailing empties — ['a', 'b', '', 'c']. Length 4."

The actual output:

4

Why (the non-obvious thing): `split(regex)` with no limit is `split(regex, 0)` — it discards trailing empty strings. Use `split(regex, -1)` to keep all of them, including trailing.

What NOT to say: - "Length 6 — every comma creates a slot" — trailing empties are stripped - "Length 4 — but only because there are 4 non-empty values" — the internal empty between 'b' and 'c' is kept

Common follow-up: "How would I get length 6?" — `split(",", -1)`.

Scenario 36 · Multiple catch with class hierarchy

Asked at: Infosys · TCS

Difficulty: Easy-Medium · **Topic:** catch ordering

The code:

```
public class CatchOrder {
    public static void main(String[] args) {
        try {
            throw new RuntimeException("x");
        } catch (Exception e) {
            System.out.println("Exception");
        } catch (RuntimeException e) {
            System.out.println("RuntimeException");
        }
    }
}
```

The interviewer's prompt: "Output? Or does it not compile?"

Thinking out loud: 1. "RuntimeException extends Exception. A catch clause for the superclass before the subclass is unreachable — the compiler rejects this." 2. "Compile error: 'exception RuntimeException has already been caught'."

The actual output:

```
Compile error: exception java.lang.RuntimeException has already been caught
```

Why (the non-obvious thing): The compiler enforces ordering: more specific catch clauses must come first. Reversing them makes the subclass clause dead code, which is a compile error in Java.

What NOT to say: - "Prints 'Exception' — outer catch wins" — compile error - "Prints 'RuntimeException' — most specific match" — second catch is unreachable

Common follow-up: "How would I make this compile?" — put `RuntimeException` first, or merge with multi-catch `catch (RuntimeException | OtherException e)`.

Scenario 37 · Map.compute returning null

Asked at: Razorpay · senior backend

Difficulty: Medium · **Topic:** Map.compute semantics

The code:


```
import java.util.*;

public class ComputeNull {
    public static void main(String[] args) {
        Map<String, Integer> m = new HashMap<>();
        m.put("a", 1);
        m.compute("a", (k, v) -> null);
        System.out.println(m.containsKey("a"));
        System.out.println(m.size());
    }
}
```

The interviewer's prompt: "Two prints."

Thinking out loud: 1. " `compute` invokes the remapping function. If it returns null, the entry is removed." 2. "After compute, 'a' is gone. `containsKey` false. Size 0."

The actual output:

```
false
0
```

Why (the non-obvious thing): `Map.compute` (and `merge`) interpret a null result as 'remove the key'. Useful trick for atomic delete-if-condition, but a footgun if you didn't intend it.

What NOT to say: - "Stores null as value, `containsKey` true" — null result means delete - "Throws NPE" — null is legal return from the remapper

Common follow-up: "How would I explicitly store null as a value?" — use `put(key, null)` directly.

Scenario 38 · Switch on enum with missing case

Asked at: TCS Digital · Razorpay

Difficulty: Easy · **Topic:** switch on enum

The code:

```
enum Color { RED, GREEN, BLUE }
public class SwitchEnum {
    public static void main(String[] args) {
        Color c = Color.BLUE;
        switch (c) {
            case RED: System.out.println("R"); break;
            case GREEN: System.out.println("G"); break;
        }
        System.out.println("done");
    }
}
```

The interviewer's prompt: "What prints?"

Thinking out loud: 1. "c is BLUE. Switch has cases for RED and GREEN, no default. BLUE matches nothing — switch falls through to the end silently." 2. "Then 'done' prints."

The actual output:

```
done
```

Why (the non-obvious thing): Classic switch is non-exhaustive — unmatched values just skip the block. No compile error, no warning by default. Java 21's pattern-matching switch on sealed types enforces exhaustiveness; classic enum switch does not (without `-Xlint:switch`).

What NOT to say: - "Compile error — missing case BLUE" — classic switch isn't exhaustive - "Throws an exception at runtime" — silently no-ops

Common follow-up: "How can I get a compile-time exhaustiveness check?" — use arrow-form switch expression with no default.

Scenario 39 · Iterator next without hasNext

Asked at: Infosys · TCS

Difficulty: Easy · **Topic:** Iterator contract

The code:

```
import java.util.*;

public class IterEnd {
    public static void main(String[] args) {
        List<Integer> list = new ArrayList<>();
        Iterator<Integer> it = list.iterator();
        System.out.println(it.next());
    }
}
```

The interviewer's prompt: "What happens?"

Thinking out loud: 1. "Empty list. Iterator's next() on an exhausted (or empty) iterator throws NoSuchElementException."

The actual output:

```
Exception in thread "main" java.util.NoSuchElementException
```

Why (the non-obvious thing): The Iterator contract requires calling hasNext() first. next() with nothing left is undefined behaviour as far as the API is concerned, but all standard impls throw NSE.

What NOT to say: - "Returns null" — never null, always throws - "Returns 0" — Integer, no default

Common follow-up: "How does forEach avoid this?" — internally calls hasNext before each next.

Scenario 40 · Long autoboxing cache range

Asked at: Razorpay · senior backend

Difficulty: Medium · **Topic:** Long cache

The code:

```
public class LongCache {  
    public static void main(String[] args) {  
        Long a = 50L;  
        Long b = 50L;  
        Long c = 200L;  
        Long d = 200L;  
        System.out.println(a == b);  
        System.out.println(c == d);  
    }  
}
```

The interviewer's prompt: "Two booleans — like the Integer case?"

Thinking out loud: 1. "Long has the same -128 to 127 cache as Integer (and Short, Byte). 50 is in range — `a == b` is true." 2. "200 is out of range — `c == d` is false."

The actual output:

```
true  
false
```

Why (the non-obvious thing): The wrapper cache range -128..127 is identical for Integer, Long, Short, Byte. Character caches 0..127 (ASCII). Boolean caches both. Float and Double never cache (NaN equality complicates it).

What NOT to say: - "Long doesn't cache, both false" — it does - "Cache range is larger for Long" — same range

Common follow-up: "What about Double — does it cache?" — no, Double has no autobox cache.

Scenario 41 · Try-finally with return value modification

Asked at: Razorpay · senior

Difficulty: Hard · **Topic:** finally + return value

The code:

```
import java.util.*;
public class FinallyMutation {
    static List<Integer> run() {
        List<Integer> list = new ArrayList<>();
        try {
            list.add(1);
            return list;
        } finally {
            list.add(2);
        }
    }
    public static void main(String[] args) {
        System.out.println(run());
    }
}
```

The interviewer's prompt: "What does main print?"

Thinking out loud: 1. "try adds 1, then `return list` evaluates `list` (gets the reference) and stages it for return." 2. "finally runs before the method actually returns. It mutates the list — `list.add(2)`." 3. "The return value is the reference, which still points to the same (now mutated) list." 4. "Main prints [1, 2]."

The actual output:

```
[1, 2]
```

Why (the non-obvious thing): `return list` captures the reference, not a snapshot of the list contents. `finally` can mutate the referenced object after the return value is staged but before the method exits — those mutations are visible to the caller.

What NOT to say: - "Prints [1] — return runs first" — reference-typed returns are not snapshots - "Throws because list is modified mid-return" — no, mutation is fine

Common follow-up: "What if the return value were `int`?" — `finally` can't change the staged primitive value (it's already on the stack).

Scenario 42 · Implicit widening in compound assignment

Asked at: TCS · Infosys

Difficulty: Easy-Medium · **Topic:** Compound assignment

The code:

```
public class Compound {  
    public static void main(String[] args) {  
        int i = 10;  
        i += 1.5;  
        System.out.println(i);  
    }  
}
```

The interviewer's prompt: "Does this compile? If yes, what's i?"

Thinking out loud: 1. " `i += 1.5` is sugar for `i = (int)(i + 1.5)` — an implicit narrowing cast. So it compiles." 2. "10 + 1.5 = 11.5, cast to int truncates to 11."

The actual output:

11

Why (the non-obvious thing): Compound assignment operators include an implicit narrowing cast back to the LHS type. `i = i + 1.5` would NOT compile (no implicit narrowing from double to int), but `i += 1.5` does. Sneaky source of silent precision loss.

What NOT to say: - "Compile error — can't add double to int" — compound assignment auto-narrows - "Prints 11.5" — `i` is int, truncates

Common follow-up: "What if I wrote `i = i + 1.5` ?" — compile error, lossy conversion.

Scenario 43 · Float vs double literal mismatch

Asked at: TCS · Infosys

Difficulty: Easy · **Topic:** Numeric literals

The code:

```
public class FloatLit {  
    public static void main(String[] args) {  
        float f = 3.14;  
        System.out.println(f);  
    }  
}
```

The interviewer's prompt: "Compile? Or runtime?"

Thinking out loud: 1. "3.14 is a double literal. Assigning double to float is a narrowing conversion — requires explicit cast." 2. "Compile error: possible lossy conversion."

The actual output:

```
Compile error: incompatible types: possible lossy conversion from double to float
```

Why (the non-obvious thing): Without an `f` or `F` suffix, decimal literals are double. To make it a float literal, write `3.14f`.

What NOT to say: - "Compiles and prints 3.14" — needs explicit suffix - "Java is dynamically typed here" — no, strictly typed

Common follow-up: "What's the fix?" — `float f = 3.14f;` or `float f = (float) 3.14;`.

Scenario 44 · Equals on auto-boxed comparisons

Asked at: Razorpay · TCS

Difficulty: Medium · **Topic:** Autoboxing in comparison

The code:

```
public class IntBoxCompare {
    public static void main(String[] args) {
        Integer a = 1000;
        int b = 1000;
        System.out.println(a == b);
        System.out.println(a.equals(b));
    }
}
```

The interviewer's prompt: "Two booleans — beat the autoboxing trap."

Thinking out loud: 1. "`a == b`" — when one operand is a primitive and the other is a wrapper, the wrapper unboxes to primitive. So this is `1000 == 1000` as int — true." 2. "`a.equals(b)`" — b autoboxes to Integer to fit the parameter. equals compares values — true."

The actual output:

```
true
true
```

Why (the non-obvious thing): The Integer cache trap only fires when BOTH sides of `==` are wrappers. Mixed wrapper-and-primitive `==` always unboxes the wrapper, so the comparison is on primitives — straightforward value equality, no cache concerns.

What NOT to say: - "First false because 1000 is outside the Integer cache" — cache only matters for wrapper-vs-wrapper `==` - "Second false because `Integer.equals(int)` is illegal" — autoboxing for the argument

Common follow-up: "When would the cache trap actually fire?" — `Integer a = 1000; Integer b = 1000; a == b` — false.

Scenario 45 · Generics with raw type assignment

Asked at: Infosys · TCS · Razorpay

Difficulty: Hard · **Topic:** Raw types + erasure

The code:

```
import java.util.*;

public class RawType {
    public static void main(String[] args) {
        List<String> typed = new ArrayList<>();
        List raw = typed;
        raw.add(42);
        String s = typed.get(0);
        System.out.println(s);
    }
}
```

The interviewer's prompt: "Compile? Runtime? Which line fails?"

Thinking out loud: 1. "`List raw = typed`" — assigning generic to raw is allowed (with an unchecked warning). 2. "`raw.add(42)`" — raw type, no compile-time check. Integer goes into the underlying list. 3. "`typed.get(0)`" — compiler thinks it's a String, inserts cast. At runtime, the cast tries to convert Integer to String — `ClassCastException`."

The actual output:


```
Exception in thread "main" java.lang.ClassCastException:
class java.lang.Integer cannot be cast to class java.lang.String
```

Why (the non-obvious thing): Generics are compile-time only. The compiler inserts casts at every `get()` based on the declared parameter, but doesn't track what raw operations did to the underlying object. The "heap pollution" shows up only when you try to use the badly typed element.

What NOT to say: - "Compile error — can't add int to List" — raw type bypasses check - "Prints 42" — assigned to String fails the inserted cast

Common follow-up: "What's the warning the compiler gives?" — 'unchecked call to `add(E)`'.

Scenario 46 · String.format vs printf precision

Asked at: TCS · Infosys

Difficulty: Easy · **Topic:** printf format

The code:

```
public class Printf {
    public static void main(String[] args) {
        System.out.printf("%.2f%n", 3.14159);
        System.out.printf("%5d%n", 42);
    }
}
```

The interviewer's prompt: "What two lines print?"

Thinking out loud: 1. "`%.2f`" — float with 2 decimals. 3.14159 rounds to 3.14." 2. "`%5d`" — int with minimum width 5, right-aligned by default. 42 becomes ' 42' (three spaces + 42)."

The actual output:

```
3.14
 42
```

Why (the non-obvious thing): `%n` is platform-independent line separator (vs `\n` which is always LF). Format flags: `%5d` right-aligns to width 5; `%-5d` left-aligns; `%05d` zero-pads.

What NOT to say: - "Prints 3.14159" — `.2` truncates to 2 decimals (rounded) - "Prints '42 '" — right-aligned by default, not left

Common follow-up: "How to left-align?" — `%-5d` .

Scenario 47 · Comparator chained with reversed

Asked at: Razorpay · senior backend

Difficulty: Medium · **Topic:** Comparator chaining

The code:

```
import java.util.*;
import java.util.stream.*;

public class CompChain {
    public static void main(String[] args) {
        List<String> list = Arrays.asList("aa", "b", "ccc", "d");
        list.stream()
            .sorted(Comparator.comparingInt(String::length).reversed())
            .forEach(System.out::println);
    }
}
```

The interviewer's prompt: "What prints, in what order?"

Thinking out loud: 1. "Sort by length, reversed — longest first." 2. "'ccc' length 3, 'aa' length 2, then 'b' and 'd' both length 1." 3. "Within the same length, the sort is stable — original order preserved. 'b' before 'd'."

The actual output:

```
ccc
aa
b
d
```

Why (the non-obvious thing): Stream's `sorted` is stable — equal elements keep their original order. `.reversed()` flips the entire comparator's ordering, not just the primary key.

What NOT to say: - "Reverses each tier — d before b" — reversed flips the primary order, not within-tier - "Length 1 elements throw because comparator returns 0" — 0 is fine, just means equal

Common follow-up: "What if I want to sort by length desc, then alphabetically asc?" —

```
comparingInt(String::length).reversed().thenComparing(Comparator.naturalOrder())
```

Scenario 48 · Generic method type inference

Asked at: Razorpay · senior

Difficulty: Hard · **Topic:** Generic type inference

The code:

```
import java.util.*;

public class GenInfer {
    static <T> List<T> makeList(T a, T b) {
        return Arrays.asList(a, b);
    }
    public static void main(String[] args) {
        List<?> list = makeList(1, "two");
        System.out.println(list);
        System.out.println(list.get(0).getClass().getSimpleName());
    }
}
```

The interviewer's prompt: "Compile? Output?"

Thinking out loud: 1. "Java infers *T* as the most specific common supertype of *Integer* and *String* — which is `Object` (well, technically `Comparable<? extends ...> & Serializable`, but `Object` is the simple answer)." 2. "So `makeList(1, "two")` returns `List<Object>` containing *Integer* 1 and *String* 'two'." 3. "Prints `[1, two]`. First element is *Integer*."

The actual output:

```
[1, two]
Integer
```

Why (the non-obvious thing): Java infers the most specific common type of the arguments — when types diverge, it falls back to their nearest common ancestor. The compiler is happy, but you may end up with `List<Object>` when you expected `List<Integer>`.

What NOT to say: - "Compile error — can't mix types" — inference picks the common supertype - "Prints class type 'Object'" — runtime type is still *Integer*, regardless of erasure

Common follow-up: "How would I force *T* to be *Integer*?" — explicit type witness:

`MyClass.<Integer>makeList(1, "two")` would then fail because *String* doesn't fit.

Scenario 49 · *Stream.peek without terminal*

Asked at: Razorpay · senior

Difficulty: Medium · **Topic:** Stream laziness

The code:

```
import java.util.stream.*;

public class StreamLazy {
    public static void main(String[] args) {
        Stream.of(1, 2, 3)
            .peek(n -> System.out.println("peek " + n))
            .map(n -> n * 2);
        System.out.println("done");
    }
}
```

The interviewer's prompt: "What prints?"

Thinking out loud: 1. "No terminal operation — no collect, count, forEach, etc. Streams are lazy; intermediate operations never trigger without a terminal." 2. "peek never runs. Just 'done' prints."

The actual output:

```
done
```

Why (the non-obvious thing): Stream pipelines are pull-based and built around terminal operations. Without one, the JVM has no reason to run anything. `peek` is purely for debugging — if no terminal pulls, it's silent.

What NOT to say: - "Prints peek 1, peek 2, peek 3, done" — no terminal, no execution - "Compile error — incomplete stream" — pipelines without terminals are legal, just useless

Common follow-up: "What if I add `.count()` at the end?" — peek fires for each of the 3 elements before count returns.

Scenario 50 · Comparator on Strings with null

Asked at: Razorpay · senior backend

Difficulty: Medium · **Topic:** Comparator + null

The code:

```
import java.util.*;

public class SortNull {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>(Arrays.asList("b", null, "a"));
        Collections.sort(list);
        System.out.println(list);
    }
}
```

The interviewer's prompt: "What happens — does it sort? Throw?"

Thinking out loud: 1. "Natural order on String calls String.compareTo. When the sort algorithm hits a null, it calls null.compareTo(...) — NPE."

The actual output:

```
Exception in thread "main" java.lang.NullPointerException
```

Why (the non-obvious thing): Natural-order comparators dereference both operands; nulls are not allowed. Use `Comparator.nullsFirst(...)` or `Comparator.nullsLast(...)` to handle nulls safely.

What NOT to say: - "Sorts with null at the start" — natural ordering doesn't handle null - "Compile error" — nulls are allowed in collections, fails at sort time

Common follow-up: "How to put nulls last?" —

```
list.sort(Comparator.nullsLast(Comparator.naturalOrder())) .
```

Scenario 51 · String.chars() summing as int

Asked at: Razorpay · senior

Difficulty: Medium · **Topic:** Stream of primitives

The code:

```
public class CharsStream {  
    public static void main(String[] args) {  
        int sum = "abc".chars().sum();  
        System.out.println(sum);  
    }  
}
```

The interviewer's prompt: "What does sum equal?"

Thinking out loud: 1. " `String.chars()` returns `IntStream` of char codepoints. 'a'=97, 'b'=98, 'c'=99." 2. "Sum = 97 + 98 + 99 = 294."

The actual output:

```
294
```

Why (the non-obvious thing): `chars()` returns `IntStream` of int values (the codepoints), not chars. Sum operates on those ints, not on character objects. This trips people who expect 'abc' summing to '6' or some such.

What NOT to say: - "Sum is 3 — number of chars" — that's `count()`, not `sum` - "Throws because you can't sum chars" — `chars()` is an `IntStream`, `sum` is legal

Common follow-up: "How would you sum the digits in '123'?" — `"123".chars().map(c -> c - '0').sum()`.

Scenario 52 · Inheritance and field hiding

Asked at: Razorpay · senior backend

Difficulty: Hard · **Topic:** Field hiding vs method override

The code:

```
class A { int x = 1; int get() { return x; } }
class B extends A { int x = 2; int get() { return x; } }
public class FieldHide {
    public static void main(String[] args) {
        A a = new B();
        System.out.println(a.x);
        System.out.println(a.get());
    }
}
```

The interviewer's prompt: "Two prints — field access vs method call."

Thinking out loud: 1. " `a.x` — field access is resolved by the declared (compile-time) type. `a` is declared as A, so `a.x` reads A's `x` — 1." 2. " `a.get()` — method call is virtual, resolved by runtime type. B's `get` returns B's `x` — 2."

The actual output:

```
1
2
```

Why (the non-obvious thing): Fields don't polymorphism — they're shadowed (hidden), not overridden. The compile-time type chooses which field you see. Methods are virtual and use the runtime type. Mixing field declarations across subclasses is a serious code smell exactly because of this asymmetry.

What NOT to say: - "Both print 2 — runtime type is B" — fields don't polymorph - "Both print 1 — declared type is A" — methods do polymorph

Common follow-up: "What's the inheritance term for this?" — field hiding (or shadowing).

Scenario 53 · finally + uncaught exception

Asked at: TCS Digital · Razorpay

Difficulty: Medium · **Topic:** finally + uncaught throw

The code:

```
public class FinallyThrow {  
    public static void main(String[] args) {  
        try {  
            throw new RuntimeException("original");  
        } finally {  
            System.out.println("finally");  
            throw new RuntimeException("from finally");  
        }  
    }  
}
```

The interviewer's prompt: "What prints, and which exception escapes?"

Thinking out loud: 1. "try throws 'original'. finally runs: prints 'finally'." 2. "finally then throws 'from finally'. This new exception replaces 'original' — the original is silently lost." 3. "Main exits with 'from finally' in the stack trace."

The actual output:

```
finally  
Exception in thread "main" java.lang.RuntimeException: from finally
```

Why (the non-obvious thing): Without try-with-resources, a throw in finally completely replaces any pending exception from try. No suppressed list, no chain — the original is gone. This is one reason cleanup code in finally should rarely throw.

What NOT to say: - "Both exceptions show up" — only the finally one escapes - "Original wins because it came first" — finally replaces

Common follow-up: "How to preserve both?" — manual: catch the original, store it, then in finally re-throw with `addSuppressed`.

Scenario 54 · String + null concatenation

Asked at: TCS · Infosys

Difficulty: Easy · **Topic:** String concat with null

The code:


```
public class StringNull {  
    public static void main(String[] args) {  
        String s = null;  
        String result = "hello " + s;  
        System.out.println(result);  
        System.out.println(result.length());  
    }  
}
```

The interviewer's prompt: "NPE? Or what does it print?"

Thinking out loud: 1. "String concat with null calls `String.valueOf(null)` which returns the literal string 'null'." 2. "result = 'hello null'. Length 10."

The actual output:

```
hello null  
10
```

Why (the non-obvious thing): `+` on String never NPEs on a null operand — it stringifies via `String.valueOf` which returns `"null"`. Useful but dangerous: 'null' string in your logs often masks a real null reference bug.

What NOT to say: - "NPE on concat" — concat is null-safe - "Prints just 'hello '" — null becomes the literal text 'null'

Common follow-up: "What about `null + ""`?" — also 'null'.

Scenario 55 · Comparable on auto-generated record

Asked at: Razorpay · Cred · senior

Difficulty: Medium · **Topic:** Records + equals

The code:

```
import java.util.*;

public class RecordEquals {
    record Point(int x, int y) {}
    public static void main(String[] args) {
        Set<Point> s = new HashSet<>();
        s.add(new Point(1, 2));
        s.add(new Point(1, 2));
        System.out.println(s.size());
    }
}
```

The interviewer's prompt: "What's the set's size?"

Thinking out loud: 1. "Records auto-generate equals, hashCode, and toString based on all components." 2. "Two Point(1, 2) are equal and have the same hashCode. HashSet deduplicates. Size 1."

The actual output:

1

Why (the non-obvious thing): Records (standardized in Java 16) automatically derive equals/hashCode from the canonical components. This is one of the reasons records are the right call for data carriers — no boilerplate, no broken contracts.

What NOT to say: - "Size 2 — different objects" — record equals compares fields - "Compile error — records can't be in a Set" — they implement equals/hashCode properly

Common follow-up: "Can I customize a record's equals?" — yes, but you must keep it consistent with hashCode and the canonical constructor.

Scenario 56 · Switch expression with yield

Asked at: Razorpay · Cred · senior

Difficulty: Medium · **Topic:** Switch expression

The code:

```
public class SwitchExpr {  
    public static void main(String[] args) {  
        int x = 2;  
        String result = switch (x) {  
            case 1 -> "one";  
            case 2 -> { yield "two"; }  
            default -> "other";  
        };  
        System.out.println(result);  
    }  
}
```

The interviewer's prompt: "What does this print?"

Thinking out loud: 1. "x is 2. Matches case 2. Block form uses `yield` to produce the value." 2. "yield 'two'. Prints 'two'."

The actual output:

two

Why (the non-obvious thing): Switch expressions (Java 14+) use `->` for arrow form. Single-expression arms return directly; block arms must use `yield` to produce the value. `return` inside a switch arm exits the enclosing method, not just the switch.

What NOT to say: - "Compile error — yield not a keyword" — yield is a contextual keyword in switch expressions - "Prints 'one' — default case" — case 2 matches first

Common follow-up: "What if I use `return` instead of `yield`?" — exits the enclosing method, switch expression has no value.

Scenario 57 · String reversal via reverse + join

Asked at: TCS · Infosys

Difficulty: Easy · **Topic:** StringBuilder

The code:

```
public class Reverse {
    public static void main(String[] args) {
        String s = "Java";
        System.out.println(new StringBuilder(s).reverse());
    }
}
```

The interviewer's prompt: "Output?"

Thinking out loud: 1. "StringBuilder.reverse() reverses in place and returns `this`. println calls its toString." 2. "'Java' reversed is 'avaJ'."

The actual output:

```
avaJ
```

Why (the non-obvious thing): StringBuilder's reverse handles only the BMP (basic multilingual plane) correctly for surrogate pairs — Java handles them as a special case but bidi reversal of multi-codepoint glyphs (like flag emojis) still gives nonsense.

What NOT to say: - "Compile error — StringBuilder doesn't have reverse" — it does, in-place - "Returns a new String" — returns this StringBuilder, toString is called by println

Common follow-up: "What about emoji like 🧑🏿?" — these are grapheme clusters; reverse will break them into a meaningless sequence.

Scenario 58 · Boolean conversion in if

Asked at: TCS · Infosys

Difficulty: Easy · **Topic:** Boxed boolean in if

The code:

```
public class IfBool {
    public static void main(String[] args) {
        Boolean b = null;
        if (b) System.out.println("yes");
        else System.out.println("no");
    }
}
```

The interviewer's prompt: "What happens?"

Thinking out loud: 1. "`if` requires a boolean. Boolean wrapper is unboxed via `b.booleanValue()` — `null.booleanValue()` throws NPE."

The actual output:

```
Exception in thread "main" java.lang.NullPointerException
```

Why (the non-obvious thing): Unboxing any null wrapper in a primitive context throws NPE. This includes if conditions, arithmetic, comparisons — anywhere the JVM needs the primitive value. The fix: explicit null check before unboxing.

What NOT to say: - "Prints 'no' — null is falsy" — Java has no truthy/falsy, only true/false - "Compile error — Boolean in if" — Boolean is auto-unboxed

Common follow-up: "How would you write this safely?" — `if (Boolean.TRUE.equals(b))` — null-safe.

Scenario 59 · Recursive lambda via this

Asked at: Razorpay · senior

Difficulty: Hard · **Topic:** Lambda self-reference

The code:

```
import java.util.function.*;

public class RecLambda {
    public static void main(String[] args) {
        Function<Integer, Integer> fact = n -> n <= 1 ? 1 : n * fact.apply(n - 1);
        System.out.println(fact.apply(5));
    }
}
```

The interviewer's prompt: "Compile? Output?"

Thinking out loud: 1. "The lambda references `fact` inside its body. But `fact` is a local variable being initialized — it's not 'effectively final' yet when the lambda is created." 2. "Compile error: 'fact' might not have been initialized."

The actual output:

Compile error: variable fact might not have been initialized

Why (the non-obvious thing): A local variable can't refer to itself in its own initializer. The standard workaround is to declare a static field, then assign the lambda to it — the field exists before the initializer runs.

What NOT to say: - "Prints 120 — recursion works" — fails at compile - "Throws StackOverflowError" — doesn't get that far

Common follow-up: "How to write a self-recursive lambda?" — declare it as a static field or wrap it in a helper class.

Scenario 60 · Stream.generate with limit

Asked at: Razorpay · senior

Difficulty: Medium · **Topic:** Infinite stream

The code:

```
import java.util.stream.*;

public class GenLimit {
    public static void main(String[] args) {
        long count = Stream.generate(() -> "x")
            .limit(5)
            .filter(s -> s.equals("x"))
            .count();
        System.out.println(count);
    }
}
```

The interviewer's prompt: "What's the count?"

Thinking out loud: 1. " `generate(() -> "x")` produces infinite stream of 'x's. `limit(5)` truncates to first 5." 2. "Filter keeps all 5 (all match). Count returns 5."

The actual output:

5

Why (the non-obvious thing): `Stream.generate` is unbounded — without `limit` (or another short-circuiting op), terminal operations like `count` would never terminate. Always pair `generate/iterate` with `limit`.

What NOT to say: - "Throws OOM — infinite stream" — `limit` makes it finite - "Count is 1 — duplicates collapse" — `Stream` doesn't deduplicate (unless `distinct()`)

Common follow-up: "What if I omit `limit`?" — `count` never returns; infinite loop.

Scenario 61 · NaN equality

Asked at: Goldman Sachs · JP Morgan

Difficulty: Medium · **Topic:** NaN

The code:

```
public class Nan {
    public static void main(String[] args) {
        double x = Double.NaN;
        System.out.println(x == x);
        System.out.println(Double.isNaN(x));
        System.out.println(Double.valueOf(x).equals(Double.valueOf(x)));
    }
}
```

The interviewer's prompt: "Three booleans."

Thinking out loud: 1. "IEEE 754: NaN is not equal to anything, including itself. `x == x` is false." 2. "`Double.isNaN(x)` is the standard test — true." 3. "`Double.equals` is overridden specifically so `NaN.equals(NaN)` is true — to keep `Double` consistent in hash collections. True."

The actual output:

```
false
true
true
```

Why (the non-obvious thing): The `==` behavior for NaN follows IEEE 754 (never equal to anything), but `Double.equals` overrides this to keep `Double` objects behave consistently as map keys. The disconnect bites people who use double primitives vs `Double` wrappers interchangeably.

What NOT to say: - "All true — NaN equals NaN" — `primitive ==` says false - "All false — NaN is undefined" — `Double.equals` is specifically defined to handle this

Common follow-up: "What if I use `Double.NaN` as a `HashMap` key?" — works fine: `equals` + `hashCode` are consistent for `Double` wrappers.

Scenario 62 · Casting between primitive and wrapper

Asked at: TCS · Infosys

Difficulty: Easy-Medium · **Topic:** Wrapper unboxing

The code:

```
public class CastUnbox {
    public static void main(String[] args) {
        Integer i = null;
        int j = (int) i;
        System.out.println(j);
    }
}
```

The interviewer's prompt: "Compile? Runtime?"

Thinking out loud: 1. "`(int) i`" — explicit cast triggers unboxing on the null wrapper. NPE."

The actual output:

```
Exception in thread "main" java.lang.NullPointerException
```

Why (the non-obvious thing): Casting from wrapper to primitive triggers `intValue()` — null can't be unboxed. The cast looks 'safe' but actually invokes a method, which fails on null.

What NOT to say: - "Compile error — can't cast null to int" — compiles, fails at runtime - "Prints 0 — int default" — primitives have defaults only in fields, not after a failed unbox

Common follow-up: "How to handle this safely?" — `int j = (i == null) ? 0 : i;` or `Optional.ofNullable(i).orElse(0)`.

Scenario 63 · Substring memory leak (legacy JDK)

Asked at: senior backend

Difficulty: Medium · **Topic:** Substring implementation history

The code:

```
public class SubMem {  
    public static void main(String[] args) {  
        String huge = "x".repeat(1_000_000);  
        String tiny = huge.substring(0, 1);  
        huge = null;  
        System.out.println(tiny.length());  
    }  
}
```

The interviewer's prompt: "In Java 7 vs Java 8+: would the huge array be garbage-collected after `huge = null`?"

Thinking out loud: 1. "In Java 6 and earlier, substring shared the backing char array — `tiny` held a reference to the same 1M-char array. Setting `huge = null` wouldn't free it because `tiny` still pinned it." 2. "In Java 7u6 and later, substring copies into a new array. `tiny` is independent. `huge` can be collected." 3. "Either way, `tiny.length()` prints 1."

The actual output:

1

Why (the non-obvious thing): Substring's implementation changed in Java 7u6 to copy rather than share — eliminated a real-world memory leak pattern but also made substring $O(n)$ instead of $O(1)$.

What NOT to say: - "Always was a copy" — historically wasn't - "Substring is still a view" — modern JDK copies

Common follow-up: "What about `String.intern()` in modern JVMs?" — pool is in main heap (since Java 7), not PermGen.

Scenario 64 · Generic array creation

Asked at: Razorpay · senior

Difficulty: Hard · **Topic:** Generic array creation

The code:

```
import java.util.*;

public class GenArr {
    public static void main(String[] args) {
        List<String>[] arr = new List<String>[3];
        System.out.println(arr.length);
    }
}
```

The interviewer's prompt: "Does this compile?"

Thinking out loud: 1. "Java disallows creating arrays of parameterized types — `new List<String>[3]` is a compile error: 'generic array creation'."

The actual output:

```
Compile error: generic array creation
```

Why (the non-obvious thing): Arrays are reified (carry type at runtime); generics are erased. The two clash: an array of `List<String>` would have to do a runtime check that's impossible after erasure. Java forbids the creation outright. You can declare the variable, just not instantiate it directly.

What NOT to say: - "Compiles, length 3" — disallowed - "Throws `ClassCastException` at runtime" — doesn't compile

Common follow-up: "How would I work around this?" — use `(List<String>[]) new List[3]` with an unchecked cast warning, or use `List<List<String>>` instead.

Scenario 65 · Pre/post increment

Asked at: TCS · Infosys

Difficulty: Easy · **Topic:** Increment operators

The code:

```
public class IncOp {  
    public static void main(String[] args) {  
        int i = 5;  
        int a = i++;  
        int b = ++i;  
        System.out.println("a=" + a + " b=" + b + " i=" + i);  
    }  
}
```

The interviewer's prompt: "Final values?"

Thinking out loud: 1. " `i++` (post-increment): returns the old value (5) then increments. $a = 5$, $i = 6$." 2. " `++i` (pre-increment): increments first then returns the new value. $i = 7$, $b = 7$."

The actual output:

```
a=5 b=7 i=7
```

Why (the non-obvious thing): Post-increment reads then writes; pre-increment writes then reads. Both are atomic-looking but `i = i++` in particular is a famous gotcha — assigns the old value, then increments, so i ends as its original value.

What NOT to say: - " $a=6$, $b=7$ " — post-increment returns old value - "Both same — order doesn't matter" — it does

Common follow-up: "What does `int i = 5; i = i++;` end at?" — $i = 5$; the increment is overwritten by the assignment of the old value.

Scenario 66 · Map.Entry mutation through iterator

Asked at: Razorpay · senior

Difficulty: Medium · **Topic:** Map iteration

The code:

```
import java.util.*;

public class EntryMutate {
    public static void main(String[] args) {
        Map<String, Integer> m = new HashMap<>();
        m.put("a", 1);
        m.put("b", 2);
        for (Map.Entry<String, Integer> e : m.entrySet()) {
            e.setValue(e.getValue() * 10);
        }
        System.out.println(m);
    }
}
```

The interviewer's prompt: "Will this throw CME? What does the map look like?"

Thinking out loud: 1. " `Entry.setValue` is allowed during iteration — it doesn't structurally modify the map (no add/remove), so modCount stays stable. No CME." 2. "Each value × 10. Map becomes {a=10, b=20}."

The actual output (HashMap insertion order is not guaranteed, but Java 8+ tends to print in bucket order):

```
{a=10, b=20}
```

Why (the non-obvious thing): `setValue` on an Entry is the one mutation that fail-fast iterators tolerate, because it doesn't affect the structure (bucket layout). Adding or removing keys during iteration is what trips CME.

What NOT to say: - "Throws CME because we're mutating during iteration" — only structural mods trigger it - "Values stay the same — Entry is a copy" — Entry is a live view in HashMap

Common follow-up: "What about TreeMap?" — same rule: `setValue` is fine, structural changes throw.

Scenario 67 · Stream.iterate without limit

Asked at: Razorpay · senior

Difficulty: Medium · **Topic:** Infinite stream — iterate

The code:

```
import java.util.stream.*;

public class IterLimit {
    public static void main(String[] args) {
        int sum = Stream.iterate(1, n -> n + 1)
            .limit(5)
            .mapToInt(Integer::intValue)
            .sum();
        System.out.println(sum);
    }
}
```

The interviewer's prompt: "Sum?"

Thinking out loud: 1. "iterate(1, n -> n+1) produces 1, 2, 3, 4, 5, ..." 2. "limit(5) keeps the first 5: 1, 2, 3, 4, 5." 3. "Sum = 15."

The actual output:

15

Why (the non-obvious thing): `Stream.iterate` is lazy infinite; pair it with `limit`. Java 9 added a 3-arg `iterate(seed, hasNext, next)` for bounded iteration without needing limit.

What NOT to say: - "Loops forever" — limit bounds it - "Sum is 5 — count of elements" — sum sums values

Common follow-up: "3-arg iterate equivalent?" — `Stream.iterate(1, n -> n <= 5, n -> n + 1)`.

Scenario 68 · Equals comparing two Lists of different types

Asked at: Razorpay · senior

Difficulty: Medium · **Topic:** List.equals

The code:

```
import java.util.*;

public class ListEq {
    public static void main(String[] args) {
        List<Integer> a = Arrays.asList(1, 2, 3);
        LinkedList<Integer> b = new LinkedList<>(Arrays.asList(1, 2, 3));
        System.out.println(a.equals(b));
    }
}
```

The interviewer's prompt: "True or false?"

Thinking out loud: 1. "List.equals is defined by the List interface: equal if both are Lists, same size, and pairwise elements equal." 2. "ArrayList vs LinkedList — both implement List — equals across implementations is fine. Same elements in same order. True."

The actual output:

```
true
```

Why (the non-obvious thing): The Collection equals contract is defined per-interface, not per-class. List.equals compares any two Lists; Set.equals compares any two Sets. Mixing implementations is intentional.

What NOT to say: - "False — different classes" — equals is by content for Lists - "Throws ClassCastException" — equals takes Object, never throws on class

Common follow-up: "Does Set.equals work the same way?" — yes, ignores impl class.

Scenario 69 · Map.getOrDefault evaluating the default

Asked at: Razorpay · senior

Difficulty: Medium · **Topic:** Eager vs lazy default

The code:

```
import java.util.*;

public class GetOrDef {
    static String expensive() {
        System.out.println("computing");
        return "default";
    }
    public static void main(String[] args) {
        Map<String, String> m = new HashMap<>();
        m.put("k", "v");
        System.out.println(m.getOrDefault("k", expensive()));
    }
}
```

The interviewer's prompt: "Does 'computing' print? Output?"

Thinking out loud: 1. " `m.getOrDefault("k", expensive())` — Java evaluates all method arguments before calling `getOrDefault`. So `expensive()` runs first." 2. "'computing' prints, `expensive` returns 'default'. `getOrDefault` sees the key 'k' exists, returns 'v'. 'default' is discarded."

The actual output:

```
computing
v
```

Why (the non-obvious thing): `getOrDefault` is eager — the default is always evaluated even when the key is present. For lazy evaluation, use `computeIfAbsent` (which takes a Function) or guard with `containsKey`.

What NOT to say: - "Just prints v — default is lazy" — Java evaluates args eagerly - "Prints 'computing' and 'default'" — `getOrDefault` returns the existing value

Common follow-up: "What's the lazy alternative?" — `m.containsKey("k") ? m.get("k") : expensive()`, or `Optional.ofNullable(m.get("k")).orElseGet(this::expensive)`.

Scenario 70 · Lambda invocation count

Asked at: Razorpay · senior

Difficulty: Medium · **Topic:** Stream + side effects

The code:

```
import java.util.*;
import java.util.stream.*;

public class StreamSideEffect {
    static int calls = 0;
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(1, 2, 3, 4, 5);
        list.stream()
            .filter(n -> { calls++; return n % 2 == 0; })
            .findFirst();
        System.out.println(calls);
    }
}
```

The interviewer's prompt: "How many times does filter run?"

Thinking out loud: 1. "findFirst short-circuits. The stream pulls elements through the pipeline until it gets the first match." 2. "Element 1: filter runs, returns false. Element 2: filter runs, returns true. findFirst captures 2." 3. "Total filter calls: 2."

The actual output:

2

Why (the non-obvious thing): Streams pull lazily through the pipeline; intermediate operations only run for elements that reach a terminal that needs them. Short-circuiting terminals can drastically reduce work.

What NOT to say: - "5 — runs on every element" — short-circuits - "1 — only the first matching" — needs to check rejected elements first

Common follow-up: "What if findFirst is replaced with count?" — calls = 5.

Scenario 71 · Equal Strings hashCode collision

Asked at: TCS · Infosys

Difficulty: Easy · **Topic:** hashCode contract

The code:


```
public class HashCode {  
    public static void main(String[] args) {  
        String a = "Aa";  
        String b = "BB";  
        System.out.println(a.hashCode());  
        System.out.println(b.hashCode());  
        System.out.println(a.equals(b));  
    }  
}
```

The interviewer's prompt: "Three lines."

Thinking out loud: 1. "'Aa' and 'BB' are a famous String hashCode collision. Both produce 2112." 2. "They're not equal — different content."

The actual output:

```
2112  
2112  
false
```

Why (the non-obvious thing): hashCode contract allows collisions — equal hashCodes don't imply equal objects. The reverse holds: equal objects must have equal hashCodes.

What NOT to say: - "Different strings, different hashCodes" — collisions are allowed - "If hashCodes match, equals must be true" — backwards contract

Common follow-up: "Why does Java's String hash these the same?" — sum/poly formula coincidence: 'A'(65)31 + 'a'(97) = 'B'(66)31 + 'B'(66) = 2112.

Scenario 72 · Conditional operator type inference

Asked at: Infosys · TCS

Difficulty: Medium · **Topic:** Conditional expression

The code:

```
public class TernaryNull {  
    public static void main(String[] args) {  
        Integer x = null;  
        int y = true ? x : 0;  
        System.out.println(y);  
    }  
}
```

The interviewer's prompt: "What happens?"

Thinking out loud: 1. "Ternary has Integer (x) and int (0). Java widens int 0 to Integer? Actually — both arms are numeric, so it unboxes to int. x = null, unboxing throws NPE."

The actual output:

```
Exception in thread "main" java.lang.NullPointerException
```

Why (the non-obvious thing): When one ternary arm is a wrapper and the other a primitive, Java unboxes both to the primitive type. If the wrapper is null, you get NPE — and the trap is that the chosen arm (x) is the null one even though it 'should' be the literal 0.

What NOT to say: - "Prints null" — y is int, no null - "Compile error — mixed types" — Java unifies via unboxing

Common follow-up: "How to be safe?" — `int y = (x != null) ? x : 0;` or change the type to Integer.

Scenario 73 · Anonymous inner class capturing field

Asked at: Razorpay · senior

Difficulty: Hard · **Topic:** Inner class capture

The code:

```
public class InnerCapture {
    int x = 10;
    Runnable r = new Runnable() {
        public void run() { System.out.println(x); }
    };
    void mutate() { x = 20; }
    public static void main(String[] args) {
        InnerCapture ic = new InnerCapture();
        ic.mutate();
        ic.r.run();
    }
}
```

The interviewer's prompt: "What prints — 10 or 20?"

Thinking out loud: 1. "Inner class captures the enclosing `this` reference. Inside `run`, `x` is `OuterClass.this.x` — a live access." 2. "After mutate, `x = 20`. `r.run()` reads current value through this reference — prints 20."

The actual output:

20

Why (the non-obvious thing): Inner classes (anonymous, local, non-static nested) hold a hidden reference to the enclosing instance. Field accesses are live, not snapshots. This differs from lambda capture of local variables (which must be effectively final).

What NOT to say: - "Prints 10 — captured at construction" — instance fields aren't snapshotted -
"Compile error — `x` not final" — instance fields don't need to be final

Common follow-up: "What if `x` were a local variable in the enclosing method?" — must be effectively final.

Scenario 74 · Final field reassignment

Asked at: TCS · Infosys

Difficulty: Easy · **Topic:** final

The code:

```
import java.util.*;
public class FinalList {
    public static void main(String[] args) {
        final List<Integer> list = new ArrayList<>();
        list.add(1);
        list.add(2);
        System.out.println(list);
    }
}
```

The interviewer's prompt: "Compile? Output?"

Thinking out loud: 1. " `final` prevents reassignment of the reference, not mutation of the object. `list.add` is fine." 2. "List contains [1, 2]."

The actual output:

```
[1, 2]
```

Why (the non-obvious thing): `final` is shallow — it locks the reference, not the referenced object. To get a truly immutable view, use `List.of(...)` or `Collections.unmodifiableList(...)`.

What NOT to say: - "Compile error — final list" — final means the reference, not the contents - "Empty list — add silently fails" — add works

Common follow-up: "How to make the list immutable too?" — `List.of(1, 2)` or wrap in `Collections.unmodifiableList`.

Scenario 75 · Object.equals on null

Asked at: TCS · Infosys

Difficulty: Easy · **Topic:** equals + null

The code:

```
import java.util.Objects;

public class EqualsNull {
    public static void main(String[] args) {
        String a = null;
        String b = "x";
        System.out.println(Objects.equals(a, b));
        System.out.println(a.equals(b));
    }
}
```

The interviewer's prompt: "Two prints — which throws?"

Thinking out loud: 1. "`Objects.equals(a, b)`" — null-safe utility. Returns false (one is null, other isn't)." 2. "`a.equals(b)`" — calling equals on null — NPE."

The actual output:

```
false
Exception in thread "main" java.lang.NullPointerException
```

Why (the non-obvious thing): `Objects.equals` is the null-safe wrapper around `.equals`. Use it whenever either side could be null — saves an explicit null check.

What NOT to say: - "Both false — both null-safe" — only `Objects.equals` is - "Both throw NPE" — `Objects.equals` handles null

Common follow-up: "What does `Objects.equals(null, null)` return?" — true.

Scenario 76 · Switch expression exhaustiveness

Asked at: Razorpay · senior

Difficulty: Medium · **Topic:** Switch expression

The code:

```
enum Day { MON, TUE, WED }
public class SwitchExpr2 {
    public static void main(String[] args) {
        Day d = Day.MON;
        String s = switch (d) {
            case MON -> "Monday";
            case TUE -> "Tuesday";
        };
        System.out.println(s);
    }
}
```

The interviewer's prompt: "Compile?"

Thinking out loud: 1. "Switch expressions must be exhaustive. *d* is an enum with 3 values but we only cover 2 — no default, no WED." 2. "Compile error."

The actual output:

Compile error: the switch expression does not cover all possible input values

Why (the non-obvious thing): Switch expressions (arrow form, returning a value) require exhaustiveness. Switch statements (classic, no return) do not. The compiler enforces this for safety — no silent gaps.

What NOT to say: - "Prints Monday — first case matches" — fails before runtime - "Throws at runtime" — fails at compile

Common follow-up: "How to fix?" — add `case WED -> ...` or `default -> ...`.

Scenario 77 · Stream collect to map with duplicate keys

Asked at: Razorpay · senior

Difficulty: Hard · **Topic:** Collectors.toMap

The code:

```
import java.util.*;
import java.util.stream.*;

public class ToMapDup {
    public static void main(String[] args) {
        Map<Integer, String> m = Stream.of("aa", "bb", "cc")
            .collect(Collectors.toMap(String::length, s -> s));
        System.out.println(m);
    }
}
```

The interviewer's prompt: "Output?"

Thinking out loud: 1. "All three strings have length 2 — same key. `toMap` (the 2-arg form) doesn't allow duplicate keys; throws `IllegalStateException`."

The actual output:

```
Exception in thread "main" java.lang.IllegalStateException: Duplicate key 2
```

Why (the non-obvious thing): The 2-arg `Collectors.toMap` is strict — duplicate keys throw. Use the 3-arg form with a merge function: `toMap(k, v, (existing, replacement) -> ...)`. This trap caught countless people migrating from `Map.put` loops.

What NOT to say: - "Last value wins — Map semantics" — `toMap` fails on duplicates by default -
"Returns map with 1 entry" — throws instead

Common follow-up: "How to keep last value?" — `toMap(k, v, (a, b) -> b)`.

Scenario 78 · Boxing in arithmetic with null

Asked at: Razorpay · TCS

Difficulty: Easy-Medium · **Topic:** Unboxing NPE

The code:

```
public class BoxNPE {  
    public static void main(String[] args) {  
        Integer x = null;  
        int y = x + 1;  
        System.out.println(y);  
    }  
}
```

The interviewer's prompt: "Compile? Runtime?"

Thinking out loud: 1. "x + 1" — "+" on Integer + int unboxes x. null.intValue() — NPE."

The actual output:

```
Exception in thread "main" java.lang.NullPointerException
```

Why (the non-obvious thing): Any arithmetic involving a wrapper triggers unboxing. Null wrappers are unboxing landmines hiding in plain sight — especially in code that uses `Map.get` which can return null.

What NOT to say: - "Prints 1 — null treated as 0" — Java doesn't coerce null to 0 - "Compile error" — compiles, fails at runtime

Common follow-up: "Common place this bites?" — `Integer count = map.get(key); int n = count + 1;` — NPE if key absent.

Scenario 79 · Static method inheritance — hiding

Asked at: Razorpay · senior

Difficulty: Hard · **Topic:** Static method hiding

The code:

```
class Parent { static void hi() { System.out.println("Parent"); } }  
class Child extends Parent { static void hi() { System.out.println("Child"); } }  
public class StaticHide {  
    public static void main(String[] args) {  
        Parent p = new Child();  
        p.hi();  
    }  
}
```


The interviewer's prompt: "What prints?"

Thinking out loud: 1. "Static methods are not virtual — they don't override. The compile-time type of `p` is `Parent`, so `Parent.hi()` is called."

The actual output:

```
Parent
```

Why (the non-obvious thing): Static method calls are dispatched at compile time based on the declared (static) type, not the runtime type. Calling a static via an instance is itself a bad style — your IDE typically warns about it.

What NOT to say: - "Prints `Child` — runtime type is `Child`" — static dispatch is by declared type - "Compile error — static can't be inherited" — they are inherited (hidden), just not virtual

Common follow-up: "What if I call `Child.hi()` directly?" — prints 'Child'.

Scenario 80 · `String.format` with mismatched arg count

Asked at: Infosys · TCS

Difficulty: Easy · **Topic:** `printf`

The code:

```
public class FmtMismatch {
    public static void main(String[] args) {
        System.out.printf("%s %s%n", "hello");
    }
}
```

The interviewer's prompt: "Result?"

Thinking out loud: 1. "Two `%S` placeholders but only one argument. `printf` throws `MissingFormatArgumentException`."

The actual output:

```
hello Exception in thread "main" java.util.MissingFormatArgumentException: Format specifier '%s'
```

Why (the non-obvious thing): `printf` is variadic and does runtime checking. Mismatch fails at runtime, never compile time. It prints partial output (the first `%s` and a space) before throwing — annoying when debugging.

What NOT to say: - "Compiles and prints 'hello null'" — Java doesn't pad with null - "Compile error — too few args" — `varargs` accept any count

Common follow-up: "What about extra args?" — extras are silently ignored, no exception.

Scenario 81 · Object initialization with this leak

Asked at: Cred · senior backend

Difficulty: Hard · **Topic:** Object construction order

The code:

```
public class Init {
    int x = init();
    int y = 10;
    int init() { return y + 1; }
    public static void main(String[] args) {
        Init i = new Init();
        System.out.println("x=" + i.x + " y=" + i.y);
    }
}
```

The interviewer's prompt: "Final x and y?"

Thinking out loud: 1. "Field initializers run top to bottom. When `x = init()` runs, `init` reads `y`. `y` hasn't been assigned 10 yet — still default 0." 2. "`init` returns `0 + 1 = 1`. `x = 1`." 3. "Then `y = 10`."

The actual output:

```
x=1 y=10
```

Why (the non-obvious thing): Same forward-reference trap as static initializers, but for instance fields. Calling methods from initializers reads other fields at their default values if they haven't been assigned yet.

What NOT to say: - "`x=11 y=10`" — assumes `y` is already 10 - "NPE" — `int` defaults to 0, no nulls involved

Common follow-up: "How would I make this safer?" — initialize in a constructor where order is explicit.

Scenario 82 · *BigDecimal vs double precision*

Asked at: Goldman Sachs · JP Morgan · fintech rounds

Difficulty: Medium · **Topic:** BigDecimal

The code:

```
import java.math.BigDecimal;

public class BigDec {
    public static void main(String[] args) {
        BigDecimal a = new BigDecimal(0.1);
        BigDecimal b = new BigDecimal("0.1");
        System.out.println(a);
        System.out.println(b);
    }
}
```

The interviewer's prompt: "Two prints."

Thinking out loud: 1. " `new BigDecimal(0.1)` takes a double. 0.1 in double is not exact — it's the nearest representable value. BigDecimal converts that exact double value, which is the long ugly tail." 2. " `new BigDecimal("0.1")` parses the decimal string exactly. Result is exactly 0.1."

The actual output:

```
0.1000000000000000055511151231257827021181583404541015625
0.1
```

Why (the non-obvious thing): The double constructor is a footgun — it captures the inexact double, not the literal value you wrote. Always use the String constructor (or `BigDecimal.valueOf`) for predictable values.

What NOT to say: - "Both print 0.1" — only the String constructor does - "First prints 0.0999..." — BigDecimal preserves the exact double bits

Common follow-up: "Which constructor should I use in production?" — String, always.

Scenario 83 · Switch with String case insensitivity

Asked at: TCS · Infosys

Difficulty: Easy · **Topic:** switch on String

The code:

```
public class SwitchString {  
    public static void main(String[] args) {  
        String s = "Hello";  
        switch (s) {  
            case "hello": System.out.println("lower"); break;  
            case "Hello": System.out.println("Mixed"); break;  
            default: System.out.println("other");  
        }  
    }  
}
```

The interviewer's prompt: "What prints?"

Thinking out loud: 1. "switch on String uses equals (which is case-sensitive). 'Hello' matches 'Hello' exactly." 2. "Prints 'Mixed'."

The actual output:

Mixed

Why (the non-obvious thing): switch on String is case-sensitive. For case-insensitive matching, normalize first: `switch (s.toLowerCase())`. Switch on String compiles down to hashCode + equals on the case labels.

What NOT to say: - "Prints 'lower' — case-insensitive" — switch is case-sensitive - "Compile error — duplicate cases" — they're different strings

Common follow-up: "How to make it case-insensitive?" — normalize on entry: `switch (s.toLowerCase())`.

Scenario 84 · Final method override

Asked at: TCS · Infosys

Difficulty: Easy · **Topic:** final method

The code:

```
class Base { final void hi() { System.out.println("Base"); } }  
class Sub extends Base { void hi() { System.out.println("Sub"); } }  
public class FinalMethod {  
    public static void main(String[] args) {  
        new Sub().hi();  
    }  
}
```

The interviewer's prompt: "Compile?"

Thinking out loud: 1. "Base.hi is final. Sub can't override it. Compile error."

The actual output:

```
Compile error: hi() in Sub cannot override hi() in Base; overridden method is final
```

Why (the non-obvious thing): `final` on a method blocks override. Use it for security-critical methods, performance hotspots (allows JIT inlining), or template methods that subclasses must use unchanged.

What NOT to say: - "Compiles, prints 'Sub'" — final blocks override - "Compiles but prints 'Base'" — final is virtual" — fails before runtime

Common follow-up: "What's a good real-world use case?" — Object's `getClass()` is final for a reason.

Scenario 85 · *IntStream.range* exclusivity

Asked at: TCS · Infosys

Difficulty: Easy · **Topic:** *IntStream*

The code:

```
import java.util.stream.*;

public class IntRange {
    public static void main(String[] args) {
        int sum = IntStream.range(1, 5).sum();
        int sum2 = IntStream.rangeClosed(1, 5).sum();
        System.out.println(sum);
        System.out.println(sum2);
    }
}
```

The interviewer's prompt: "Two numbers."

Thinking out loud: 1. " `range(1, 5)` is half-open: 1, 2, 3, 4. Sum = 10." 2. " `rangeClosed(1, 5)` is closed: 1, 2, 3, 4, 5. Sum = 15."

The actual output:

```
10
15
```

Why (the non-obvious thing): `range` excludes the upper bound; `rangeClosed` includes it. Mirrors Python's `range` (exclusive) — easy to mix up if you forget which is which.

What NOT to say: - "Both 15 — same range" — different inclusivity - "Both 10 — exclusive upper bound" — only range is

Common follow-up: "Which is more common?" — `range` for index-style loops, `rangeClosed` for natural counting.

Scenario 86 · String == across method returns

Asked at: Infosys · TCS

Difficulty: Medium · **Topic:** String pool

The code:

```
public class StringReturn {  
    static String get() { return "x"; }  
    public static void main(String[] args) {  
        String a = "x";  
        String b = get();  
        System.out.println(a == b);  
    }  
}
```

The interviewer's prompt: "True or false?"

Thinking out loud: 1. " `get()` returns the literal 'x' — same pool entry as a. Both `a` and `b` point to the same pooled object." 2. "True."

The actual output:

```
true
```

Why (the non-obvious thing): String literals are pooled at class-load time, regardless of where they appear (method body, field, parameter). The same literal anywhere in your class points to the same pool entry.

What NOT to say: - "False — different method scopes" — pool is JVM-wide - "Compile error — String equality" — `==` on String is legal, just compares references

Common follow-up: "What if get returns `new String("x")` ?" — false.

Scenario 87 · Map remove during iteration

Asked at: Razorpay · senior

Difficulty: Medium · **Topic:** ConcurrentModificationException

The code:

```
import java.util.*;

public class MapRemove {
    public static void main(String[] args) {
        Map<String, Integer> m = new HashMap<>();
        m.put("a", 1); m.put("b", 2);
        for (String k : m.keySet()) {
            if (k.equals("a")) m.remove(k);
        }
        System.out.println(m);
    }
}
```

The interviewer's prompt: "CME? Or what map prints?"

Thinking out loud: 1. "Removing from the underlying map while iterating its keySet throws ConcurrentModificationException — same fail-fast iterator mechanism as List."

The actual output:

```
Exception in thread "main" java.util.ConcurrentModificationException
```

Why (the non-obvious thing): keySet, entrySet, values are all views — iterating them shares the underlying modCount with the map. Structural modification through any path triggers CME.

What NOT to say: - "Removes 'a' and continues" — fails as soon as iterator advances -
"ConcurrentHashMap would also throw" — it wouldn't; ConcurrentHashMap iterators are weakly consistent

Common follow-up: "Safe removal?" — `Iterator.remove()` , `removeIf` (on entrySet/keySet), or `Map.entrySet().removeIf(...)` .

Scenario 88 · Inner class accessing private field

Asked at: Razorpay · senior

Difficulty: Medium · **Topic:** Inner class access

The code:


```
public class InnerAccess {
    private int x = 5;
    class Inner {
        int read() { return x; }
    }
    public static void main(String[] args) {
        InnerAccess outer = new InnerAccess();
        InnerAccess.Inner inner = outer.new Inner();
        System.out.println(inner.read());
    }
}
```

The interviewer's prompt: "Compile? Output?"

Thinking out loud: 1. "Inner class can access the enclosing class's private fields — they're in the same compilation unit, javac generates synthetic accessor methods." 2. "inner.read returns 5."

The actual output:

5

Why (the non-obvious thing): Inner classes have privileged access to the enclosing class's private members via compiler-generated synthetic methods (visible as `access$000` style names in bytecode). Java 11+ nestmates removed the need for synthetics by adding 'nested types' to bytecode.

What NOT to say: - "Compile error — private is private" — same-class access includes inner classes - "Returns 0 — inner can't see x" — inner can see it

Common follow-up: "Why was nestmate access added in Java 11?" — eliminates synthetic bridge methods, smaller bytecode, better reflection.

Scenario 89 · Stream collect counting with grouping

Asked at: Razorpay · senior

Difficulty: Medium · **Topic:** Collectors.groupingBy

The code:

```
import java.util.*;
import java.util.stream.*;

public class GroupCount {
    public static void main(String[] args) {
        Map<Integer, Long> m = Stream.of("a", "bb", "cc", "ddd")
            .collect(Collectors.groupingBy(String::length, Collectors.counting()));
        System.out.println(m);
    }
}
```

The interviewer's prompt: "What does the map look like?"

Thinking out loud: 1. "groupingBy(length, counting()) groups by length, value is count." 2. "length 1: 'a' (count 1). length 2: 'bb', 'cc' (count 2). length 3: 'ddd' (count 1)." 3. "Result: {1=1, 2=2, 3=1}."

The actual output:

```
{1=1, 2=2, 3=1}
```

Why (the non-obvious thing): `Collectors.counting()` returns `Long`, not `int` — useful for very large streams but a typical source of "why is my map value type wrong" bugs.

What NOT to say: - "Returns Integer values" — counting returns Long - "Returns a List of strings per key" — no, the downstream is counting

Common follow-up: "How would I get a Map<String, Long> instead?" — `groupingBy(String::length)` without the second arg.

Scenario 90 · Method overload + null literal

Asked at: Razorpay · senior

Difficulty: Hard · **Topic:** Overload resolution

The code:

```
public class OverloadNull {
    static void hi(String s) { System.out.println("String"); }
    static void hi(Object o) { System.out.println("Object"); }
    public static void main(String[] args) {
        hi(null);
    }
}
```

The interviewer's prompt: "What prints?"

Thinking out loud: 1. "Both overloads accept null. Java picks the most specific. String is more specific than Object — String wins."

The actual output:

String

Why (the non-obvious thing): For `null`, Java picks the most specific applicable overload. String is a subtype of Object, so it wins. If you add a third overload `hi(Integer i)`, the call becomes ambiguous because Integer and String are siblings — neither is more specific.

What NOT to say: - "Object — null is treated as Object" — null binds to most specific - "Ambiguous — compile error" — only ambiguous when overloads aren't comparable

Common follow-up: "What if I add `hi(Integer i)`?" — compile error: ambiguous.

Scenario 91 · Math.min with mixed types

Asked at: TCS · Infosys

Difficulty: Easy · **Topic:** Math overloads

The code:

```
public class MathMin {
    public static void main(String[] args) {
        System.out.println(Math.min(0.0, -0.0));
        System.out.println(Math.min(-0.0, 0.0));
    }
}
```

The interviewer's prompt: "What prints?"

Thinking out loud: 1. "IEEE 754 distinguishes +0.0 and -0.0. Math.min defines -0.0 as less than +0.0." 2. "Both calls: -0.0 is the smaller one, returned regardless of order."

The actual output:

```
-0.0  
-0.0
```

Why (the non-obvious thing): `Math.min` follows IEEE 754 ordering: $-0.0 < +0.0$ (even though `0.0 == -0.0` is true). It's a rare distinction that matters in numerical libraries and almost nowhere else.

What NOT to say: - "Both 0.0 — they're equal" — `Math.min` special-cases the sign - "Throws because 0 has no sign" — IEEE 754 has signed zero

Common follow-up: "What's `0.0 == -0.0` ?" — true.

Scenario 92 · Var with diamond inference

Asked at: Razorpay · senior

Difficulty: Medium · **Topic:** var + diamond

The code:

```
import java.util.*;  
  
public class VarDiamond {  
    public static void main(String[] args) {  
        var list = new ArrayList<>();  
        list.add("hello");  
        list.add(42);  
        System.out.println(list.get(1).getClass().getName());  
    }  
}
```

The interviewer's prompt: "Compile? Output?"

Thinking out loud: 1. "`var list = new ArrayList<>()`" — diamond with no LHS hint. The compiler infers the type parameter as `Object`." 2. "`list` is `ArrayList`. Both `String` and `Integer` fit. `list.get(1)` is `Object` — runtime type `Integer`." 3. "Prints `java.lang.Integer`."

The actual output:

```
java.lang.Integer
```

Why (the non-obvious thing): With `var`, you lose the LHS type hint that normally guides diamond inference. The diamond falls back to `Object`. Use `var list = new ArrayList<String>()` to be explicit.

What NOT to say: - "Compile error — diamond needs LHS" — with `var`, falls back to `Object` - "ArrayList of String — first element wins" — `Object` holds both

Common follow-up: "What's the fix?" — specify the generic: `new ArrayList<String>()`.

Scenario 93 · String + char arithmetic

Asked at: TCS · Infosys

Difficulty: Easy · **Topic:** String + numeric

The code:

```
public class StringChar {
    public static void main(String[] args) {
        System.out.println("Value: " + 1 + 2);
        System.out.println("Value: " + (1 + 2));
    }
}
```

The interviewer's prompt: "Two prints."

Thinking out loud: 1. "`+`" is left-associative. 'Value: ' + 1 is a String, then + 2 concatenates the 2. Result: 'Value: 12'." 2. "With parens, $1 + 2 = 3$ first, then String concat. Result: 'Value: 3'."

The actual output:

```
Value: 12
Value: 3
```

Why (the non-obvious thing): Once any operand is a String, all subsequent `+` becomes concatenation. Left-to-right associativity means a leading String poisons the rest. Parenthesize to force numeric precedence.

What NOT to say: - "Both 'Value: 3'" — ignores left associativity - "Both 'Value: 12'" — parens force arithmetic

Common follow-up: "What's `1 + 2 + "Value"`?" — '3Value', numeric first.

Scenario 94 · Catch and rethrow with different exception type

Asked at: Razorpay · senior

Difficulty: Hard · **Topic:** Exception chaining

The code:

```
public class Chain {
    public static void main(String[] args) {
        try {
            try {
                throw new RuntimeException("inner");
            } catch (RuntimeException e) {
                throw new RuntimeException("outer", e);
            }
        } catch (RuntimeException e) {
            System.out.println(e.getMessage());
            System.out.println(e.getCause().getMessage());
        }
    }
}
```

The interviewer's prompt: "Two prints."

Thinking out loud: 1. "Inner throws 'inner', caught, wrapped in new RuntimeException with cause = inner." 2. "Outer catch prints e.getMessage() — 'outer'. Then getCause().getMessage() — 'inner'."

The actual output:

```
outer
inner
```

Why (the non-obvious thing): Exception chaining preserves the original via `cause`. Always use the two-arg constructor `new XxxException(message, cause)` when wrapping — losing the cause is a debug nightmare.

What NOT to say: - "Throws because cause isn't a Throwable" — `RuntimeException(String, Throwable)` is standard - "Prints 'inner' twice — same cause" — message of outer is 'outer'

Common follow-up: "What does `printStackTrace` show?" — outer's trace, then 'Caused by: inner'.

Scenario 95 · `Stream.toList` immutability

Asked at: Razorpay · senior

Difficulty: Medium · **Topic:** `Stream.toList` vs `collect`

The code:

```
import java.util.*;
import java.util.stream.*;

public class ToList {
    public static void main(String[] args) {
        List<Integer> list = Stream.of(1, 2, 3).toList();
        try {
            list.add(4);
            System.out.println("added");
        } catch (UnsupportedOperationException e) {
            System.out.println("immutable");
        }
    }
}
```

The interviewer's prompt: "What prints?"

Thinking out loud: 1. "`Stream.toList()` (Java 16+) returns an unmodifiable List. `add` throws `UnsupportedOperationException`."

The actual output:

```
immutable
```

Why (the non-obvious thing): `Stream.toList()` returns an immutable List — different from `Collectors.toList()` which returns a mutable `ArrayList`. The naming similarity catches people; the immutability is intentional, matching `List.of`.

What NOT to say: - "Prints 'added' — Lists are mutable" — `toList` returns unmodifiable - "Throws `ClassCastException`" — `UnsupportedOperationException` is the right one

Common follow-up: "How to get a mutable list?" —

`Stream.collect(Collectors.toCollection(ArrayList::new))` or `new ArrayList<>(stream.toList())`.

Scenario 96 · Long arithmetic with int literal

Asked at: TCS · Infosys

Difficulty: Easy-Medium · **Topic:** int overflow before assignment

The code:

```
public class LongLit {  
    public static void main(String[] args) {  
        long days = 365 * 24 * 60 * 60 * 1000;  
        System.out.println(days);  
    }  
}
```

The interviewer's prompt: "What's printed?"

Thinking out loud: 1. $365 \times 24 \times 60 \times 60 \times 1000 = 31,536,000,000$ — exceeds `Integer.MAX_VALUE` (2,147,483,647). 2. "Each multiplication is `int × int = int`. Overflow happens before the assignment to long." 3. "Result is the overflowed int value, then widened to long with that wrong value preserved." 4. "Actual int result: 1471228928 — overflow wrap."

The actual output:

```
1471228928
```

Why (the non-obvious thing): The RHS computes entirely in `int` — the long target doesn't promote intermediate results. Force long arithmetic by writing `365L * 24 * 60 * 60 * 1000` (one long operand promotes the whole expression).

What NOT to say: - "Prints 31536000000" — overflow truncates - "Throws `ArithmeticException`" — silent wrap

Common follow-up: "Fix?" — make the first literal `365L` or use `TimeUnit.DAYS.toMillis(365)`.

Scenario 97 · Try-with-resources auto-close order

Asked at: Razorpay · senior

Difficulty: Medium · **Topic:** TWR close order

The code:

```
public class TWROrder {
    static class R implements AutoCloseable {
        String name;
        R(String n) { name = n; System.out.println("open " + n); }
        public void close() { System.out.println("close " + name); }
    }
    public static void main(String[] args) {
        try (R a = new R("a"); R b = new R("b")) {
            System.out.println("body");
        }
    }
}
```

The interviewer's prompt: "Trace the prints."

Thinking out loud: 1. "Resources are declared left-to-right: open a, open b." 2. "Body runs: 'body'." 3. "Close runs in reverse declaration order: close b, then close a."

The actual output:

```
open a
open b
body
close b
close a
```

Why (the non-obvious thing): TWR closes resources in reverse declaration order — mirrors stack unwinding semantics. If b depends on a, you want a to outlive b's close, which is exactly what reverse order gives you.

What NOT to say: - "Closes in declaration order — a then b" — reverse order - "Closes in random order" — strictly reverse declaration

Common follow-up: "What if close() throws?" — exception is added to suppressed list of the body's exception (if any), or propagated as the primary.

Scenario 98 · Iterator over Stream

Asked at: Razorpay · senior

Difficulty: Medium · **Topic:** Stream consumption

The code:

```
import java.util.stream.*;

public class StreamReuse {
    public static void main(String[] args) {
        Stream<Integer> s = Stream.of(1, 2, 3);
        System.out.println(s.count());
        System.out.println(s.count());
    }
}
```

The interviewer's prompt: "Two counts? Or?"

Thinking out loud: 1. "First `count` consumes the stream. Streams are one-shot — you can't traverse twice." 2. "Second `count` throws `IllegalStateException`."

The actual output:

```
3
Exception in thread "main" java.lang.IllegalStateException: stream has already been operated upon
```

Why (the non-obvious thing): A Stream is consumed by any terminal operation. To use 'the same data' twice, either store the source (e.g. a List), or build a Supplier that creates a fresh stream each time.

What NOT to say: - "Both print 3 — Stream is reusable" — strictly one-shot - "Throws first time" — first works, second doesn't

Common follow-up: "How would I count twice?" — `Supplier<Stream<Integer>> sup = () -> Stream.of(1,2,3); sup.get().count(); sup.get().count();` .

Scenario 99 · Equality with autoboxed boolean in collection

Asked at: TCS · Infosys

Difficulty: Easy-Medium · **Topic:** Boolean cache

The code:

```
import java.util.*;
public class BoolList {
    public static void main(String[] args) {
        List<Boolean> list = Arrays.asList(true, true, false);
        System.out.println(list.get(0) == list.get(1));
        System.out.println(list.get(0) == Boolean.TRUE);
    }
}
```

The interviewer's prompt: "Two booleans."

Thinking out loud: 1. "Both `list.get(0)` and `list.get(1)` are autoboxed from `true` → both are `Boolean.TRUE` (cached). Same reference. True." 2. "`list.get(0)` is `Boolean.TRUE`. Compared to `Boolean.TRUE` — same reference. True."

The actual output:

```
true
true
```

Why (the non-obvious thing): Boolean autoboxing always returns one of two cached singletons (`TRUE`, `FALSE`). So `==` on autoboxed Booleans is always safe — unlike Integer's bounded cache.

What NOT to say: - "False — different boxing calls" — Boolean cache is universal - "Compile error — can't `==` on List elements" — Boolean is an object, `==` is reference

Common follow-up: "What's the difference from Integer?" — Integer cache is -128..127; Boolean cache is exhaustive (only 2 values).

Scenario 100 · Final variable + lambda + reassignment

Asked at: Razorpay · senior

Difficulty: Medium · **Topic:** Effectively final

The code:

```
import java.util.function.*;

public class EffFinal {
    public static void main(String[] args) {
        int x = 10;
        Supplier<Integer> s = () -> x;
        x = 20;
        System.out.println(s.get());
    }
}
```

The interviewer's prompt: "Compile or runtime? If both, what prints?"

Thinking out loud: 1. "The lambda captures x. After capture, x is reassigned — so x is NOT effectively final anymore." 2. "Compile error: 'local variables referenced from a lambda expression must be final or effectively final'."

The actual output:

Compile error: local variables referenced from a lambda expression must be final or effectively final

Why (the non-obvious thing): The 'effectively final' check looks at the variable across its whole scope — any reassignment, anywhere, disqualifies it. The compiler enforces this before runtime to prevent the closures-over-mutable-locals confusion that plagues JavaScript.

What NOT to say: - "Prints 10 — captured by value" — fails at compile - "Prints 20 — captured by reference" — fails at compile

Common follow-up: "What's the workaround if I need a mutable captured value?" — wrap in an array (`int[] holder = {10}`), use `AtomicInteger`, or use an instance field.

Category 4 — Memory & Performance Scenarios

Java Interview Prep — 2026 Edition · Learn & Excel

These are the scenarios senior engineers face on real production incidents — `OutOfMemoryError` at 2 AM, GC pause spikes during a sale, a service that runs fine for 4 hours and then dies. Indian product companies (Razorpay, Cred, Swiggy, Zerodha, Flipkart, PhonePe) screen heavily on this exact stack of problems in SDE-2 and SDE-3 rounds. Each scenario walks you through the diagnosis the way you'd actually do it — which tool, what to look at, how to land on root cause, and the specific lazy answer that gets you marked down.

Scenario 1 · Heap OOM after 4 hours

Asked at: Razorpay · Swiggy · Cred · **Difficulty:** Medium · **Topic:** Memory leak diagnosis

The interviewer's prompt: "Your Spring Boot service runs fine for 4 hours then memory grows to 95% and the JVM dies with `java.lang.OutOfMemoryError: Java heap space`. Walk me through how you'd debug this."

Thinking out loud (the diagnosis path): 1. "First I'd confirm it's actually a heap leak and not just an undersized heap. Check the GC log — if old gen keeps growing across full GCs and never shrinks, it's a leak." 2. "Next, capture a heap dump right before death. I'd start the JVM with `-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/var/dumps/`. Or trigger one manually with `jcmd <pid> GC.heap_dump /tmp/dump.hprof` once memory crosses 80%." 3. "Open the `.hprof` in Eclipse MAT, run Leak Suspects report. It'll point me at the dominator tree — usually one class holding millions of objects via a static collection, a `ThreadLocal`, or a listener list nobody unsubscribed from."

The likely root cause: A static `Map` or `List` accumulating entries with no eviction policy, a `ThreadLocal` in a thread-pool thread that never gets cleared, or a cache (Caffeine, Guava) configured without size or time bounds.

The fix (and the verification step): - Bound the offending collection — `Caffeine.newBuilder().maximumSize(10_000).expireAfterWrite(Duration.ofMinutes(10))` instead of plain `HashMap`. - For `ThreadLocal`s, always call `.remove()` in a `finally` block after the request completes. - Verify by re-running with `-Xlog:gc*:file=gc.log` for 8 hours and confirming old gen stabilizes after the first few full GCs.

What NOT to say: - "Just increase `-Xmx` to 8 GB" — that delays the crash, doesn't fix the leak. Senior interviewers explicitly listen for this answer to mark down.

Common follow-up: "You see one class holds 70% of the heap in MAT. How do you find which code path is adding to it?"

Scenario 2 · GC Overhead Limit Exceeded

Asked at: Flipkart · PhonePe · **Difficulty:** Medium · **Topic:** GC tuning

The interviewer's prompt: "You see `java.lang.OutOfMemoryError: GC overhead limit exceeded` in the logs. What does this actually mean, and how do you fix it?"

Thinking out loud: 1. "This error means the JVM spent more than 98% of recent CPU time doing GC and reclaimed less than 2% of the heap. It's a heap-pressure signal — the heap is too small or has a leak." 2. "I'd check GC logs first — `-Xlog:gc*` — and look at the time between full GCs and how much old gen they free. If full GCs run back-to-back and free almost nothing, it's a leak. If they free a lot but allocation rate is just very high, it's undersizing." 3. "Then heap dump on OOM, open in MAT, look at retained sizes. If a single dominator holds most of the heap, it's a leak. If memory is spread across many short-lived objects, raise heap or tune GC."

The likely root cause: Either a memory leak in user code, or the application's working set genuinely doesn't fit in the configured heap (common when migrating to containers with lower limits than the old VM).

The fix: - If it's a leak: fix the leak (see Scenario 1's patterns). - If it's undersizing: raise `-Xmx`, switch to G1GC or ZGC, and verify with `jstat -gcutil <pid> 1000` that old gen utilization stabilizes below 80% under load.

What NOT to say: - "Disable the GC overhead check with `-XX:-UseGCOverheadLimit`" — that just lets the JVM thrash longer before the inevitable crash. The check is doing you a favor.

Common follow-up: "What's the difference between this error and a regular `Java heap space` OOM?"

Scenario 3 · Metaspace OOM after redeloys

Asked at: TCS · Infosys · Cognizant · **Difficulty:** Medium · **Topic:** Classloader leak

The interviewer's prompt: "You hot-deploy your WAR file every few hours during testing. After the 6th redeploy, Tomcat dies with `java.lang.OutOfMemoryError: Metaspace`. What's happening?"

Thinking out loud: 1. "Metaspace stores class metadata. Each redeploy creates a new web-app classloader that loads all your application classes again. If the old classloader isn't garbage-collected, its classes stay in Metaspace forever." 2. "I'd take a heap dump and look for `WebappClassLoader` instances. If I see more than one, the old ones are leaking. In MAT, the path-to-GC-roots will show what's holding them — usually a `ThreadLocal` in a thread-pool thread, a JDBC driver registered in `DriverManager`, or a static field in a JDK class referencing an app class." 3. "Common culprits: log4j context maps, Apache Commons Logging, `java.beans.Introspector` cache, JDBC drivers not deregistered in `contextDestroyed()`."

The likely root cause: A reference outside the web-app classloader pointing back into it, preventing the old classloader (and all its classes) from being collected.

The fix: - Implement `ServletContextListener.contextDestroyed()` to deregister JDBC drivers, clear `ThreadLocal`s, shut down executors, and flush log appenders. - In Tomcat, enable the

`JreMemoryLeakPreventionListener` and `ThreadLocalLeakPreventionListener` in `server.xml` . - Verify: after deploy/undeploy cycle, `jcmd <pid> VM.classloader_stats` should not show old web-app classloaders.

What NOT to say: - "Just restart Tomcat between deploys" — defeats the point of hot deploy and signals you don't understand classloader semantics. - "Increase `-XX:MaxMetaspaceSize`" — only delays the failure.

Common follow-up: "Why does removing a `ThreadLocal` from a pooled thread matter for classloaders?"

Scenario 4 · Direct Buffer OOM

Asked at: Cred · Razorpay · **Difficulty:** Hard · **Topic:** Native memory

The interviewer's prompt: "Heap looks healthy at 40% but you get `java.lang.OutOfMemoryError: Direct buffer memory` . What's going on?"

Thinking out loud: 1. "Direct buffers (`ByteBuffer.allocateDirect()`) live off-heap. They count against `-XX:MaxDirectMemorySize` , which defaults to roughly `-Xmx` . Netty, NIO sockets, and some HTTP clients use them heavily." 2. "The reference to a direct buffer sits on heap, but the actual memory is native. A direct buffer is freed only when its `Cleaner` runs — which happens during GC. So if you have low heap pressure but high direct allocation, the cleaner runs rarely and direct memory accumulates." 3. "I'd enable Native Memory Tracking: `-XX:NativeMemoryTracking=summary` , then `jcmd <pid> VM.native_memory summary` . That'll show 'Internal' or specific buffer pool usage. Also `ManagementFactory.getPlatformMXBeans(BufferPoolMXBean.class)` exposes direct buffer counts."

The likely root cause: Netty channel pipeline leaking `ByteBuffer` s (forgotten `release()`), a NIO server not pooling buffers, or an HTTP client like Apache async whose connection pool grows unbounded.

The fix: - For Netty: enable leak detection with `-Dio.netty.leakDetection.level=paranoid` during testing. - Always `release()` `ByteBuffer` s in a `finally` block, or use `ReferenceCountUtil.safeRelease()` . - Cap `-XX:MaxDirectMemorySize` explicitly and monitor `java.nio:type=BufferPool,name=direct` via JMX.

What NOT to say: - "Heap is fine so it can't be a memory issue" — direct memory is a separate budget.

Common follow-up: "Why does forcing a `System.gc()` sometimes fix this temporarily?"

Scenario 5 · Slow response time investigation

Asked at: Swiggy · Zomato · **Difficulty:** Medium · **Topic:** Latency profiling

The interviewer's prompt: "P99 latency on your API jumped from 200ms to 2 seconds yesterday. No code deploy. How do you debug?"

Thinking out loud: 1. "First, narrow the layer. Is it the JVM (GC, lock contention), the database (slow query, lock wait), the network (downstream service), or the OS (high load average, swapping)?" 2. "I'd look at GC logs first — quick win. If pause times went from 50ms to 800ms, that's GC. Next, thread dump: `jstack <pid>` three times, 5 seconds apart. Look for threads stuck on the same DB call, lock, or external HTTP." 3. "If GC is fine and threads aren't blocked, attach async-profiler in wall-clock mode: `./profiler.sh -d 30 -e wall -f wall.html <pid>`. That'll show where time is being spent including waits — usually points at one downstream call or a regex backtracking blowup."

The likely root cause: One downstream call (DB, payment gateway, third-party) got slower; or a recently-added feature introduced a synchronous network call on the request path; or the dataset grew past an index's selectivity threshold.

The fix: - If downstream is slow, add a timeout + circuit breaker (Resilience4j). - If it's a DB query, **EXPLAIN ANALYZE** and add/fix the index. - Verify by re-running load and confirming P99 returns to baseline.

What NOT to say: - "Restart the service and see if it helps" — that's not a diagnosis, it's a hope. Interviewer wants methodology.

Common follow-up: "How is async-profiler different from `-Xprof` or VisualVM's sampler?"

Scenario 6 · GC pause spike during sale

Asked at: Flipkart · Myntra · **Difficulty:** Hard · **Topic:** GC tuning under load

The interviewer's prompt: "During a flash sale, P99 spikes from 100ms to 3 seconds every minute or so. Heap is 16 GB on G1GC. Where do you start?"

Thinking out loud: 1. "Sounds like full GC or a long mixed-collection pause. I'd enable `-Xlog:gc*,gc+phases=debug,safepoint:file=gc.log:time,uptime,level,tags` and look for 'Pause Full' or 'To-space exhausted' events." 2. "On G1, a sudden 3-second pause usually means an old-gen evacuation failure — humongous objects (>50% of region size) filling up, or allocation rate outpacing concurrent marking." 3. "I'd check the average region size (`-XX:G1HeapRegionSize`), look for humongous allocations in the log, and check if old gen is filling faster than the IHOP threshold (default 45%)."

The likely root cause: Allocation rate during sale outpaces the G1 concurrent cycle, causing a fall-back to a full GC. Or humongous objects (large arrays, big response payloads) fragmenting old gen.

The fix: - Raise `-XX:G1HeapRegionSize=32m` to reduce humongous allocations. - Lower `-XX:InitiatingHeapOccupancyPercent=30` so concurrent marking starts earlier. - For sub-100ms pauses on heaps this size, consider ZGC: `-XX:+UseZGC -XX:+ZGenerational` on Java 21 — sub-millisecond pauses regardless of heap size. - Verify with `jstat -gcutil 1000` during the next sale rehearsal.

What NOT to say: - "Run `System.gc()` periodically" — explicit GC on G1 forces a full GC; you're causing pauses, not preventing them.

Common follow-up: "When would you pick ZGC vs Shenandoah vs G1?"

Scenario 7 · ThreadLocal memory leak

Asked at: Razorpay · **Cred:** · **Difficulty:** Medium · **Topic:** Memory leak

The interviewer's prompt: "You added a `ThreadLocal<UserContext>` for request-scoped data. A week later, memory is leaking. Why?"

Thinking out loud: 1. "`ThreadLocal` entries live as long as the thread does. In Tomcat or any thread-pool server, threads live for the entire JVM lifetime. So every entry you `set()` stays until you `remove()` it." 2. "Worse: a `ThreadLocal`'s value can reference your app's classloader, which then can't be unloaded — turns a memory leak into a classloader leak on redeploy." 3. "In a heap dump, I'd look for `ThreadLocalMap$Entry` arrays inside `Thread` instances. MAT's 'Leak Suspects' usually surfaces this immediately."

The likely root cause: A filter or interceptor calls `userContext.set(ctx)` but never calls `userContext.remove()` in a `finally` block, so the previous user's context lingers on the worker thread until the next request overwrites it (and the last request of the day stays forever).

The fix: - Wrap usage: `java try { USER_CONTEXT.set(ctx); chain.doFilter(req, res); } finally { USER_CONTEXT.remove(); }` - For scoped data in Spring, prefer `RequestContextHolder` or a request-scoped bean. - Verify: under sustained load, heap should not grow beyond the expected steady state.

What NOT to say: - "Use `InheritableThreadLocal` instead" — that makes the leak worse by propagating to child threads.

Common follow-up: "Why doesn't the JVM auto-clean `ThreadLocal`s when their key is GC'd?"

Scenario 8 · Static collection memory leak

Asked at: Most product cos · **Difficulty:** Easy-Medium · **Topic:** Memory leak

The interviewer's prompt: "You have a static `Map<String, Order>` used as an in-memory cache. After a few days, heap is full. What's the issue?"

Thinking out loud: 1. "Static collections live for the entire JVM lifetime. Without an eviction policy, every entry you put in stays forever — that's not a cache, it's a leak with extra steps." 2. "In MAT, the dominator tree would show this `HashMap` retaining gigabytes."

The likely root cause: An unbounded static `HashMap` (or `ConcurrentHashMap`) used as a "cache" but never evicting.

The fix: - Replace with Caffeine:

```
Caffeine.newBuilder().maximumSize(50_000).expireAfterAccess(Duration.ofMinutes(30)).b
```

- If you genuinely need indefinite lookup, use a `WeakHashMap` (keys collected when no strong refs remain) — but understand its semantics. - Verify with `jcmd <pid> GC.heap_info` after a sustained run.

What NOT to say: - "Just call `System.gc()` periodically" — won't reclaim entries with live references.

Common follow-up: "What's the difference between `WeakHashMap`, `SoftReference`, and `PhantomReference`?"

Scenario 9 · Listener registration leak

Asked at: Swiggy · Zerodha · **Difficulty:** Medium · **Topic:** Memory leak

The interviewer's prompt: "Your event bus accumulates listeners over time. Memory grows. What's happening?"

Thinking out loud: 1. "Classic observer-pattern leak. A component registers itself as a listener, but never unregisters when destroyed. The event bus holds a strong reference to the listener, which keeps the whole component (and everything it references) alive." 2. "This is especially bad with anonymous inner classes — they capture the enclosing instance implicitly."

The likely root cause: Missing `unregister()` / `removeListener()` calls in destroy/close methods.

The fix: - Always pair `register()` with `unregister()` in `close()` / `destroy()`. - For Spring beans, use `@PreDestroy` to unregister. - Alternative: use `WeakReference`-based listener lists

(Guava `EventBus`'s weak variant) so listeners can be collected even if the producer forgets to unregister.

What NOT to say: - "Just trust the GC" — strong references win, GC never runs.

Common follow-up: "How does Guava's `EventBus` handle weak listeners differently from a plain `List<Listener>`?"

Scenario 10 · Thread dump shows BLOCKED threads

Asked at: Razorpay · Cred · PhonePe · **Difficulty:** Medium · **Topic:** Lock contention

The interviewer's prompt: "Your service is unresponsive. `jstack` shows 200 threads in BLOCKED state. How do you find the culprit?"

Thinking out loud: 1. "BLOCKED means a thread is waiting to acquire a monitor (synchronized lock) held by another thread. I'd look at the stack trace of each BLOCKED thread and find what lock they're waiting on — `- waiting to lock <0x000000076b...>` ." 2. "Then find which thread owns that lock — search the dump for `- locked <0x000000076b...>` . That thread's stack tells me what it's doing — usually a slow I/O call inside a `synchronized` block." 3. "If the owning thread is itself BLOCKED waiting on another lock, and that one waits on the first — classic deadlock. `jstack` will print 'Found one Java-level deadlock' at the bottom."

The likely root cause: A `synchronized` block contains a slow operation (DB call, HTTP, file I/O). 200 incoming requests queue behind it.

The fix: - Move slow I/O out of synchronized blocks. Lock only the critical section. - Switch to `ReentrantLock` with `tryLock(timeout)` to fail fast instead of blocking forever. - For read-heavy state, `StampedLock` or `ReadWriteLock` . - Verify: re-run load test; `jstack` should show no BLOCKED threads under normal load.

What NOT to say: - "Add `synchronized` to more methods to be safe" — that makes the contention worse.

Common follow-up: "What's the difference between BLOCKED, WAITING, and TIMED_WAITING in a thread dump?"

Scenario 11 · Deadlock detection

Asked at: Cred · Razorpay · **Difficulty:** Medium · **Topic:** Concurrency

The interviewer's prompt: "Two threads each hold one lock and wait for the other's. How do you confirm a deadlock in production?"

Thinking out loud: 1. "Take a thread dump: `jstack <pid> > dump.txt`. The JVM auto-detects monitor deadlocks and prints a 'Found one Java-level deadlock' section at the bottom listing the cycle of threads and locks." 2. "For deadlocks involving `ReentrantLock`, `jstack` may not auto-detect — I'd look manually for threads in WAITING on `AbstractQueuedSynchronizer$ConditionObject` with circular dependencies." 3. "`jcmd <pid> Thread.print -l` is equivalent and gives lock ownership too."

The likely root cause: Two code paths acquire locks in different orders. Lock A then B in one place, B then A in another.

The fix: - Establish a global lock-ordering invariant (e.g. by `System.identityHashCode` order, or by domain priority). - Or use `tryLock(timeout, TimeUnit)` everywhere so circular waits eventually break. - Verify: write a JUnit test that triggers both code paths concurrently with `CountDownLatch`; should complete without timeout.

What NOT to say: - "Add `synchronized(this)` everywhere" — increases the surface area for deadlock.

Common follow-up: "How would you prevent deadlocks in a banking transfer that locks both source and destination accounts?"

Scenario 12 · Heap dump analysis — largest objects

Asked at: Flipkart · PhonePe · **Difficulty:** Medium · **Topic:** Heap dump

The interviewer's prompt: "You have a 12 GB heap dump. Walk me through finding the biggest memory consumer."

Thinking out loud: 1. "Open in Eclipse MAT (or `jvisualvm` for smaller dumps). Run 'Histogram' to list classes by retained size. The top entries are usually `byte[]`, `char[]`, and `HashMap$Node[]` — that's expected; I look at the next layer to see what's holding them." 2. "Run 'Dominator Tree' — this shows objects sorted by retained heap. One object dominating 60% of the heap is the smoking gun." 3. "Right-click → Path to GC Roots → exclude weak/soft refs. That tells me the strong-reference chain keeping the object alive."

The likely root cause: Depends on the dump — could be a cache, an unbounded queue, a session map, etc.

The fix: - Bound or weakly-reference the dominator. - Add a metric (`HikariCP` -style) so the size is observable in production going forward.

What NOT to say: - "I'd just look at the histogram and increase heap" — histograms show shallow size; retained size is what matters for leaks.

Common follow-up: "What's the difference between shallow and retained heap in MAT?"

Scenario 13 · Duplicate strings dominating heap

Asked at: Flipkart · Swiggy · **Difficulty:** Medium · **Topic:** Memory optimization

The interviewer's prompt: "Heap dump shows 800 MB of `String` objects, and many are duplicates of the same values (`'INR'` , `'India'` , country codes). How do you fix this?"

Thinking out loud: 1. "Java has `String.intern()` to deduplicate, but it puts strings in the string table which has its own size limits. Better option on modern JVMs: enable `-XX:+UseStringDeduplication` with G1GC or ZGC — the GC will automatically merge duplicate `char[]` / `byte[]` backing arrays." 2. "For known-finite enumerated values (country codes, currency codes), use enums instead of strings — fixed memory, faster comparisons."

The likely root cause: Many objects each holding their own copy of the same logical string (often from JSON deserialization without interning).

The fix: - Enable `-XX:+UseStringDeduplication` (free on G1/ZGC, minor CPU cost). - For enumerated values, replace `String` fields with `enum`. - For Jackson, configure `JsonFactory.Feature.INTERN_FIELD_NAMES` for key strings. - Verify: heap dump after change should show one shared `byte[]` instead of N copies.

What NOT to say: - "Call `.intern()` on every string in the app" — intern table contention, slow, fills PermGen-like metadata structures.

Common follow-up: "How is `-XX:+UseStringDeduplication` different from `String.intern()` ?"

Scenario 14 · CPU spike to 100%

Asked at: Cred · Razorpay · Zerodha · **Difficulty:** Medium · **Topic:** CPU profiling

The interviewer's prompt: "One JVM is pegged at 100% CPU. The others on the same load balancer are fine. How do you find what's burning CPU?"

Thinking out loud: 1. "`top -H -p <pid>` shows per-thread CPU on Linux. Find the high-CPU thread's TID, convert to hex, and grep `jstack` output for `nid=0x<hex>` — that gives the Java stack

of the offending thread." 2. "Better: async-profiler in CPU mode. `./profiler.sh -d 60 -f cpu.html <pid>` — gives a flame graph showing exactly which methods are burning CPU." 3. "Common patterns: regex with catastrophic backtracking, infinite loop, JSON serialization of a cyclic object graph, GC thrashing."

The likely root cause: Depends — but on production Spring services, the top 3 are: (1) GC thrashing (look at `jstat -gcutil`), (2) regex backtracking on user input, (3) `String.replaceAll` in a tight loop.

The fix: - Fix the hot method (cache the compiled `Pattern`, sanitize input, etc.). - Add a circuit on regex-on-user-input — either pre-validate or use `Matcher.region()` with a timeout via watchdog. - Verify: rerun the flame graph; previous hot frame should be gone.

What NOT to say: - "Restart and add more replicas" — works for capacity, not for a single-thread CPU bug.

Common follow-up: "What's the difference between async-profiler and JFR for CPU profiling?"

Scenario 15 · JIT deoptimization symptoms

Asked at: Cred · Zerodha · **Difficulty:** Hard · **Topic:** JIT

The interviewer's prompt: "After running fine for an hour, your method's latency suddenly degrades 5x with no code change. Logs show JIT deoptimization. What happened?"

Thinking out loud: 1. "The JIT speculates on observed types. If the code only ever saw `ArrayList`, it inlines `ArrayList.get()`. When suddenly a `LinkedList` arrives, the assumption is invalid and the JIT deoptimizes back to the interpreter, then re-profiles and recompiles." 2. "I'd enable `-XX:+PrintCompilation -XX:+UnlockDiagnosticVMOptions -XX:+PrintInlining` (verbose, dev only) or use JFR with the 'Compiler' event family enabled to see deoptimization events."

The likely root cause: A polymorphic call site that was monomorphic for the first hour suddenly becomes bimorphic or megamorphic — a new code path started feeding a different concrete type.

The fix: - Reduce polymorphism on hot paths — use sealed hierarchies or split into separate methods. - If a class hierarchy is genuinely polymorphic, accept the cost; JIT will stabilize at bimorphic inline cache. - Verify via JFR that deopt events stop after warm-up.

What NOT to say: - "JIT issues never affect production" — wrong, deopt storms are a real cause of latency cliffs.

Common follow-up: "What's the difference between C1 and C2 compilers, and when does tiered compilation switch between them?"

Scenario 16 · Connection pool exhaustion

Asked at: Razorpay · Swiggy · Cred · **Difficulty:** Medium · **Topic:** Connection pool

The interviewer's prompt: "Threads are stuck on `HikariDataSource.getConnection()` and you see `Connection is not available, request timed out after 30000ms`. Diagnose."

Thinking out loud: 1. "HikariCP has a fixed pool size (default 10). If all connections are in use, new requests wait up to `connectionTimeout`. Two reasons: queries take too long, or connections aren't being returned to the pool." 2. "Thread dump first. If many threads are in `socketRead` on the DB driver, the DB itself is slow — check slow query log." 3. "If threads are sitting on application work after `getConnection()`, the connection is being held during slow non-DB work — refactor to release first." 4. "Hikari logs leak detection if you set `leakDetectionThreshold=2000` — surfaces forgotten `.close()`."

The likely root cause: Either a slow query saturating the pool, a leaked connection (forgot `try-with-resources`), or pool size too small for $\text{load} \times \text{query time}$.

The fix: - Use `try (Connection c = ds.getConnection()) { ... }` — auto-close. - Tune pool size using the formula: `connections = ((core_count * 2) + effective_spindle_count)` as a rough Hikari guideline, then load-test. - Enable `leakDetectionThreshold=2000` in dev. - Verify: under load, `hikaricp_connections_active` metric should not stay at max.

What NOT to say: - "Increase pool size to 1000" — overwhelms the DB, makes things worse.

Common follow-up: "Why does increasing the pool size sometimes hurt throughput?"

Scenario 17 · N+1 query through JPA

Asked at: Most Spring/Hibernate shops · **Difficulty:** Easy-Medium · **Topic:** ORM performance

The interviewer's prompt: "Your `/orders` endpoint takes 4 seconds. DB shows 200 SQL queries per request. What's happening?"

Thinking out loud: 1. "Classic N+1. You loaded `List<Order>` (one query), then for each order you accessed `order.getCustomer()` lazily, triggering one query per order — $1 + N$ queries." 2. "Enable `spring.jpa.show-sql=true` and `spring.jpa.properties.hibernate.format_sql=true` to see the queries. Or use Hibernate's statistics: `org.hibernate.stat.Statistics`." 3. "Fix: fetch eagerly when you know you need it — `@EntityGraph(attributePaths = {"customer"})` on the repository method, or `JOIN FETCH` in JPQL."

The likely root cause: Lazy associations being walked in a loop after the initial fetch.

The fix: - `@EntityGraph` or `JOIN FETCH` for known access patterns. - DTO projection (`SELECT new com.foo.OrderDTO(o.id, c.name) FROM Order o JOIN o.customer c`) — fewer columns, no N+1. - Verify by counting queries: 1, not N.

What NOT to say: - "Set `FetchType.EAGER` on everything" — over-fetches everywhere, causes other slowness.

Common follow-up: "What's the difference between `JOIN FETCH` and `@EntityGraph` ?"

Scenario 18 · Pinned virtual thread

Asked at: Cred · Razorpay (Java 21 shops) · **Difficulty:** Hard · **Topic:** Virtual threads

The interviewer's prompt: "You migrated to virtual threads on Java 21. Throughput didn't improve. Some threads are 'pinned'. What does that mean?"

Thinking out loud: 1. "A virtual thread normally unmounts from its carrier (platform thread) when it blocks on I/O. But if it's inside a `synchronized` block or a native (JNI) call when it blocks, it can't unmount — it 'pins' the carrier." 2. "Pinning collapses your scalability — if 200 virtual threads all pin, you're back to 200 platform threads of capacity." 3. "Detect with `-Djdk.tracePinnedThreads=full` — JVM prints the pin location to stderr."

The likely root cause: A library uses `synchronized` heavily (e.g. older JDBC drivers, log4j with sync appenders, some `ConcurrentHashMap` paths in older JDKs).

The fix: - Replace `synchronized` blocks on hot paths with `ReentrantLock` — virtual-thread aware. - For libraries you can't change, isolate them behind a bounded `Executors.newFixedThreadPool(N)` so pinning doesn't starve the entire VT scheduler. - Java 23+ removes pinning for `synchronized` (JEP 491 et al.) — verify by upgrading.

What NOT to say: - "Just use more virtual threads" — pinning is per-carrier, more VTs don't help.

Common follow-up: "What's the difference between pinning and capturing in the virtual thread model?"

Scenario 19 · Java 8 HashMap concurrent resize bug

Asked at: Senior rounds at Cred, Razorpay · **Difficulty:** Hard · **Topic:** Concurrency bug

The interviewer's prompt: "A legacy service on Java 8 occasionally hangs with one thread at 100% CPU inside `HashMap.get()`. What's the cause?"

Thinking out loud: 1. "`HashMap` is not thread-safe. If two threads call `put()` concurrently and one triggers a resize, the internal linked list (pre-Java 8 it was a list, Java 8+ may treeify) can form a cycle. Subsequent `get()` enters an infinite loop walking the cycle." 2. "Symptom: one thread permanently at 100% CPU with `HashMap.getNode` or `HashMap.transfer` on top of the stack in `jstack`."

The likely root cause: `HashMap` (not `ConcurrentHashMap`) shared across threads without external synchronization.

The fix: - Replace with `ConcurrentHashMap` everywhere shared maps are used. - If you genuinely need single-threaded access, document it and add `assert Thread.currentThread() == ownerThread`. - Verify with stress test: 50 threads calling put concurrently for 10 seconds — should not hang.

What NOT to say: - "Wrap with `Collections.synchronizedMap()`" — fixes the bug but kills throughput; `ConcurrentHashMap` is strictly better.

Common follow-up: "Is this still possible in Java 17? What changed?"

Scenario 20 · False sharing in counter array

Asked at: Cred · Zerodha · Quant shops · **Difficulty:** Hard · **Topic:** CPU cache

The interviewer's prompt: "You have an array of `AtomicLong` counters, one per thread. Throughput is bad even though there's no logical contention. Why?"

Thinking out loud: 1. "False sharing. Cache lines are typically 64 bytes. An `AtomicLong` is 16 bytes (header + value). Eight `AtomicLong`s packed into an array share cache lines. When thread A updates `counter[0]` and thread B updates `counter[1]`, both invalidate each other's cache line — even though they touch different variables." 2. "Detection: profile with `perf stat -e cache-misses` or async-profiler's hardware events. Telltale sign — high L1 cache misses with low logical contention."

The likely root cause: Hot counters in adjacent memory sharing CPU cache lines.

The fix: - Pad with `@Contended` (requires `-XX:-RestrictContended` to enable for user code in some JDKs). - Or use `LongAdder` — internally striped and padded for exactly this case. - Verify: throughput scales linearly with thread count after the change.

What NOT to say: - "Just use volatile" — doesn't help; this is a hardware-level issue.

Common follow-up: "Why does `LongAdder` outperform `AtomicLong` under high contention?"

Scenario 21 · Escape analysis failure

Asked at: Cred · Senior backend rounds · **Difficulty:** Hard · **Topic:** JIT optimization

The interviewer's prompt: "You expected the JIT to scalar-replace your short-lived `Point` object, but heap allocation is high. Why might escape analysis fail?"

Thinking out loud: 1. "Escape analysis tries to prove an object never escapes the method. If proven, the JIT scalar-replaces — no heap allocation, fields live in registers." 2. "It fails if the object is stored in a static field, returned from the method, passed to a method the JIT can't inline, or used inside a synchronized block (in older JDKs)." 3. "Verify with `-XX:+PrintEscapeAnalysis -XX:+UnlockDiagnosticVMOptions` (verbose) or by checking allocation counts with JFR."

The likely root cause: Method call chain prevents inlining, so the JIT must assume the object can escape.

The fix: - Keep hot methods small to encourage inlining (default `MaxInlineSize=35` bytes). - Avoid returning short-lived objects from hot methods — refactor to fill caller-provided arrays. - Use records — easier for the JIT to scalar-replace than mutable classes (in modern JVMs).

What NOT to say: - "Object allocation is free in modern JVMs" — true only when escape analysis succeeds.

Common follow-up: "What's the difference between scalar replacement and stack allocation?"

Scenario 22 · async-profiler usage

Asked at: Razorpay · Cred · Flipkart · **Difficulty:** Medium · **Topic:** Profiling tools

The interviewer's prompt: "Walk me through how you'd use async-profiler on a running production JVM to find a CPU hotspot."

Thinking out loud: 1. "async-profiler is a sampling profiler that uses `perf_events` or `AsyncGetCallTrace` — no safepoint bias, unlike older sampling profilers." 2. "`./profiler.sh -d 60 -e cpu -f /tmp/cpu.html <pid>` — 60 seconds, CPU event, flame-graph output." 3. "For wall-clock (including blocked time): `-e wall`. For allocation: `-e alloc`. For lock contention: `-e lock`." 4. "Read flame graphs bottom-up: the wide frames at the top are where time is spent. Click to filter, search for class names."

The likely root cause: N/A — this is a tool question. The point is to demonstrate fluency.

The fix: - Always profile in production-like conditions; dev profiling lies. - Use `-e alloc` to find allocation hotspots, often the cause of GC pressure.

What NOT to say: - "I use VisualVM in production" — safepoint sampling bias, often misleading; async-profiler is the modern answer.

Common follow-up: "What's safepoint bias and why does it make some profilers lie?"

Scenario 23 · JFR continuous recording

Asked at: Razorpay · Swiggy · **Difficulty:** Medium · **Topic:** JFR

The interviewer's prompt: "How would you set up Java Flight Recorder to always-on capture in production?"

Thinking out loud: 1. "JFR is built into the JDK and has minimal overhead (~1-2%) on the default profile. Always-on flight recording is the standard pattern in production now." 2.

" `-XX:StartFlightRecording=duration=24h,filename=/var/jfr/recording.jfr,settings=profile,dumpsonexit=true` — captures last 24 hours, dumps on shutdown, uses 'profile' settings (more detailed than 'default')." 3. "For investigating live issues: `jcmd <pid> JFR.start name=adhoc duration=120s filename=/tmp/adhoc.jfr` — 2-minute recording for ad-hoc analysis." 4. "Open `.jfr` files in JDK Mission Control (JMC) — shows GC, allocations, lock contention, hot methods, exceptions, etc."

The likely root cause: N/A — tool question.

The fix: - Always-on JFR + a sidecar that uploads dumps on OOM signal is the modern production pattern.

What NOT to say: - "JFR is a commercial feature" — true pre-Java 11, false now; it's free and open-source since OpenJDK 11.

Common follow-up: "What's the difference between the 'default' and 'profile' JFR settings, in terms of overhead?"

Scenario 24 · jstat for live GC monitoring

Asked at: TCS · Cognizant · Infosys · **Difficulty:** Easy-Medium · **Topic:** GC monitoring

The interviewer's prompt: "You don't have JMX exposed. How do you check GC behavior on a live JVM via SSH?"

Thinking out loud: 1. "`jstat -gcutil <pid> 1000`" — refreshes every second, shows percentages of S0, S1, Eden, Old, Metaspace; plus YGC count, YGCT total time, FGC count, FGCT total time." 2. "If Eden fills and Old grows steadily across full GCs and never shrinks, that's a leak." 3. "`jstat -gccause <pid> 1000`" adds the cause of the last GC — useful for diagnosing allocation-failure vs explicit-GC vs ergonomics-driven."

The likely root cause: N/A — tool question.

The fix: - For long-term, instrument with Micrometer + Prometheus and graph these counters.

What NOT to say: - "I'd need to enable a profiler" — `jstat` ships with the JDK and works on any live JVM with no extra setup.

Common follow-up: "What does the YGCT vs FGCT ratio tell you?"

Scenario 25 · jmap heap dump on prod

Asked at: Cred · PhonePe · **Difficulty:** Easy-Medium · **Topic:** Heap dump

The interviewer's prompt: "You need a heap dump from a 16 GB production JVM. How do you do it without killing the service?"

Thinking out loud: 1. "`jcmd <pid> GC.heap_dump /var/dumps/heap.hprof`" — preferred over `jmap` on modern JDKs, equivalent under the hood." 2. "Heap dump triggers a full GC and a STW pause proportional to heap size — 16 GB usually 5-15 seconds. So I'd drain the JVM from the load balancer first." 3. "For OOM auto-capture, run with `-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/var/dumps/` so you get the dump even when the JVM dies."

The likely root cause: N/A — tool question.

The fix: - Use `live` filter to only capture reachable objects: `jcmd <pid> GC.heap_dump -all=false /var/dumps/heap.hprof` — smaller file.

What NOT to say: - "Use `jmap -F` (force mode)" — attaches to a hung JVM but produces unreliable dumps; use only when JVM is unresponsive.

Common follow-up: "Why is the dump so much smaller when you use `live` filter?"

Scenario 26 · Stop-the-world pauses on Java 8 CMS

Asked at: Older codebases at MNCs · **Difficulty:** Medium · **Topic:** Legacy GC

The interviewer's prompt: "A legacy Java 8 service uses CMS GC. P99 latency is fine 99% of the time but spikes to 8 seconds occasionally. What's happening?"

Thinking out loud: 1. "CMS does most work concurrently but falls back to a single-threaded stop-the-world full GC ('concurrent mode failure') if old gen fills before concurrent collection finishes." 2. "Telltale GC log line: `concurrent mode failure` followed by an `Full GC (Allocation Failure)` that takes seconds." 3. "Causes: allocation rate too high, or fragmented old gen (CMS doesn't compact)."

The likely root cause: Old gen fragmentation, or initiating-occupancy threshold too high so CMS starts too late.

The fix: - Migrate to G1GC: `-XX:+UseG1GC` — G1 compacts incrementally, no concurrent mode failure. - Or, if stuck on CMS short-term: lower `-XX:CMSInitiatingOccupancyFraction=60` and force it with `-XX:+UseCMSInitiatingOccupancyOnly`. - Long term: upgrade to Java 17/21 and use G1 or ZGC.

What NOT to say: - "CMS is the best GC for low latency" — deprecated in Java 9, removed in Java 14; this answer dates you.

Common follow-up: "What does ZGC do differently to avoid full GCs entirely?"

Scenario 27 · Off-heap cache memory growth

Asked at: Cred · Razorpay · **Difficulty:** Hard · **Topic:** Native memory

The interviewer's prompt: "You use Chronicle Map / EhCache off-heap. Heap is fine but the container OOMs. What's happening?"

Thinking out loud: 1. "Container memory = heap + Metaspace + code cache + direct buffers + JNI/native (off-heap caches, libraries) + thread stacks. If off-heap cache grows beyond what you sized the container for, kernel kills the process (OOMKilled in K8s)." 2. "Enable Native Memory Tracking: `-XX:NativeMemoryTracking=detail`, then `jcmd <pid> VM.native_memory detail` — breaks down where native bytes are." 3. "Container memory limit must account for all of these, not just heap."

The likely root cause: Off-heap cache size + heap + JVM overhead exceeds container limit.

The fix: - Budget: $\text{container limit} \geq \text{heap} + \text{Metaspace cap} + \text{direct memory} + \text{cache size} + 25\% \text{ headroom}$. - Cap off-heap cache explicitly. - Verify with NMT and cgroup memory metrics.

What NOT to say: - "Heap looks fine so the JVM isn't the problem" — heap is one slice of process memory.

Common follow-up: "How do you size container memory limits when running on Kubernetes?"

Scenario 28 · JNI native leak

Asked at: Cred · Senior backend · **Difficulty:** Hard · **Topic:** Native memory

The interviewer's prompt: "You use a native library via JNI (e.g. JNA, OpenCV). Container RSS grows but heap is flat. How do you find the leak?"

Thinking out loud: 1. "The leak is in native code, not Java. Standard heap dumps won't show it." 2. "Tools: `jemalloc` with profiling enabled (`MALLOC_CONF=prof:true`), or `valgrind --tool=memcheck` (slow, dev only). On Linux, `pmap -x <pid>` shows the process memory map — large anon regions are suspicious." 3. "Native Memory Tracking shows JVM-owned native allocations but won't see leaks inside user-loaded native libraries."

The likely root cause: A native call allocates memory but the corresponding free/release call is missed in error paths.

The fix: - Wrap every JNI allocation in a Java `AutoCloseable` so it's released in a try-with-resources. - Audit error paths for missed releases. - Use `jemalloc` heap profiling to capture the actual leak stacks.

What NOT to say: - "Just bounce the JVM weekly" — workaround, not a fix.

Common follow-up: "Why is Java's garbage collector unaware of native allocations?"

Scenario 29 · Code cache full

Asked at: Senior rounds · **Difficulty:** Hard · **Topic:** JIT

The interviewer's prompt: "You see `CodeCache is full. Compiler has been disabled.` in JVM logs. Performance degrades. What's happening?"

Thinking out loud: 1. "The JIT compiler stores compiled native code in the code cache. If it fills up, the JIT shuts off and the JVM falls back to interpretation — order-of-magnitude slowdown." 2. "Default size varies by JVM and tiered-compilation mode — often 240 MB. Long-running JVMs with lots of code paths can exhaust it." 3. "Verify with `jcmd <pid> Compiler.codecache` — shows used vs available."

The likely root cause: Code cache size too small for the application's compiled code.

The fix: - Raise `-XX:ReservedCodeCacheSize=512m`. - Enable code cache sweeping: `-XX:+UseCodeCacheFlushing` (usually on by default). - Verify with `Compiler.codecache` after warmup.

What NOT to say: - "Disable the JIT compiler" — interpretation is 10-50x slower; never the right answer in production.

Common follow-up: "What's tiered compilation and how does it affect code cache usage?"

Scenario 30 · `CompletableFuture` pool starvation

Asked at: Razorpay · Cred · **Difficulty:** Medium · **Topic:** Async

The interviewer's prompt: "You chain `CompletableFuture.supplyAsync()` calls. Under load, throughput collapses. What's the issue?"

Thinking out loud: 1. "By default, `supplyAsync()` uses `ForkJoinPool.commonPool()` — sized to `Runtime.availableProcessors() - 1`. If your tasks block (DB call, HTTP), all common-pool threads can be blocked simultaneously." 2. "When the pool is saturated, new async submissions queue up indefinitely. Other code in the JVM that also uses common pool (parallel streams, etc.) is starved." 3. "`jstack` shows all `ForkJoinPool-1-worker-N` threads in WAITING on an external call."

The likely root cause: Blocking I/O on the common pool, plus shared-pool contention.

The fix: - Always provide an explicit `Executor` for blocking work: `supplyAsync(task, executor)`. - Size the executor based on blocking time × throughput. - For Java 21+, use virtual threads: `Executors.newVirtualThreadPerTaskExecutor()` — virtually unlimited, blocking is cheap.

What NOT to say: - "Increase common pool size with `-Djava.util.concurrent.ForkJoinPool.common.parallelism`" — affects everything else in the JVM; explicit pool is cleaner.

Common follow-up: "Why is the common pool sized to N-1 by default?"

Scenario 31 · Heap dump on cgroup limit

Asked at: PhonePe · Cred · **Difficulty:** Medium · **Topic:** Containers

The interviewer's prompt: "Your container is OOMKilled by Kubernetes but heap looks healthy. Why?"

Thinking out loud: 1. "Kubernetes OOMKill is based on RSS exceeding the container's memory limit. RSS includes heap + Metaspace + code cache + native (direct buffers, JNI, off-heap caches) + thread stacks + JVM internals." 2. "On older JDKs (pre-8u131), the JVM didn't honor cgroup limits — `Runtime.maxMemory()` returned the host's total memory, leading to oversizing." 3. "`kubectl describe pod` shows `OOMKilled` in the last termination reason. `dmesg` on the node shows the kernel oom-killer's pick."

The likely root cause: Heap + non-heap memory exceeds container limit.

The fix: - Use `-XX:MaxRAMPercentage=75` instead of fixed `-Xmx` — JVM auto-sizes heap to 75% of container RAM, leaving room for non-heap. - Cap Metaspace: `-XX:MaxMetaspaceSize=256m` . - Cap direct memory: `-XX:MaxDirectMemorySize=256m` . - Verify with NMT and container memory metrics.

What NOT to say: - "Just give it more memory" — without diagnosis, you'll do the same thing again next quarter.

Common follow-up: "What's the difference between `Xmx` and `MaxRAMPercentage` in container environments?"

Scenario 32 · `ConcurrentHashMap computeIfAbsent` deadlock

Asked at: Senior rounds · **Difficulty:** Hard · **Topic:** Concurrency

The interviewer's prompt: "You have a `ConcurrentHashMap` and call `computeIfAbsent()` . Inside the lambda, you call `computeIfAbsent()` on the same map for a different key. The JVM hangs. Why?"

Thinking out loud: 1. " `ConcurrentHashMap.computeIfAbsent()` locks the bin (bucket) for the key. If the lambda computes a value for a different key whose bin happens to be the same one, you self-deadlock — same thread holding two locks on the same bin, but trying to re-acquire from inside the existing critical section." 2. "In Java 9+, this was made to throw an `IllegalStateException` in detectable cases, but on older versions, you got a silent infinite loop."

The likely root cause: Recursive `computeIfAbsent` on the same `ConcurrentHashMap` .

The fix: - Never nest `compute*` calls on the same map. - If you need dependent computation, do the inner computation first, then store the result. - Verify with a stress test.

What NOT to say: - "Lambda recursion isn't allowed in Java" — it is; this is a specific `ConcurrentHashMap` semantic issue.

Common follow-up: "How does `ConcurrentHashMap.compute()` differ from `synchronized` get-then-put?"

Scenario 33 · `ScheduledExecutorService` swallowing exceptions

Asked at: Razorpay · Swiggy · **Difficulty:** Medium · **Topic:** Concurrency

The interviewer's prompt: "You scheduled a periodic task with `scheduleAtFixedRate()`. It stops running silently after a few hours. Why?"

Thinking out loud: 1. "If the task throws an unchecked exception and you didn't catch it, `scheduleAtFixedRate()` cancels future runs silently. The exception is wrapped in the `Future` but never logged." 2. "Always wrap the task body in a try/catch and log."

The likely root cause: Uncaught exception in the scheduled task.

The fix: - `java scheduler.scheduleAtFixedRate(() -> { try { doWork(); } catch (Throwable t) { log.error("scheduled task failed", t); } }, 0, 1, TimeUnit.MINUTES);` - Or set a `Thread.UncaughtExceptionHandler` on the executor's `ThreadFactory`. - Verify by deliberately throwing in dev and checking logs.

What NOT to say: - "Exceptions in lambdas always propagate" — true in synchronous code, false in scheduled/async paths.

Common follow-up: "What's the difference between `Future.get()` exception behavior and `CompletableFuture.exceptionally()`?"

Scenario 34 · Slow log4j async appender

Asked at: Most product cos · **Difficulty:** Medium · **Topic:** Logging perf

The interviewer's prompt: "Under load, you see request threads blocked on `Logger.info()`. Why?"

Thinking out loud: 1. "By default, log4j2's synchronous appender blocks the calling thread on disk I/O. Under high request volume, disk write becomes the bottleneck." 2. "Switch to `AsyncAppender` or, better, `AsyncLogger` (LMAX Disruptor-backed) — log calls return immediately, a separate thread writes to disk."

The likely root cause: Synchronous logging on a slow disk under high request rate.

The fix: - Use log4j2 `AsyncLogger` globally: `log4j2.component.properties` with `Log4jContextSelector=org.apache.logging.log4j.core.async.AsyncLoggerContextSelector`. - Cap log volume; sample DEBUG; structure logs efficiently. - Verify with a flame graph — `Logger.info` frames should disappear from request path.

What NOT to say: - "Just turn off logging in prod" — loses observability when you need it most.

Common follow-up: "What's the difference between `AsyncAppender` and `AsyncLogger`?"

Scenario 35 · Hibernate session memory leak

Asked at: Spring/Hibernate shops · **Difficulty:** Medium · **Topic:** ORM

The interviewer's prompt: "A long-running batch job using Hibernate runs out of memory. Heap dump shows millions of entities in the persistence context. Why?"

Thinking out loud: 1. "Hibernate's first-level cache (the `Session`) accumulates every entity it loads. In a long-running session, it grows unbounded." 2. "For batch jobs, call `session.flush()` then `session.clear()` every N entities — typically every 50-100."

The likely root cause: Persistence context not cleared in a batch loop.

The fix: - `java for (int i = 0; i < total; i++) { Order o = ...; session.persist(o); if (i % 100 == 0) { session.flush(); session.clear(); } }`
- Or use `StatelessSession` for pure read/write batches — no first-level cache, no dirty checking. - Verify heap doesn't grow over the batch.

What NOT to say: - "Increase heap to 32 GB" — kicks the can; batch should run in constant memory.

Common follow-up: "What's the difference between `flush()` and `commit()` in Hibernate?"

Scenario 36 · Reflection-heavy framework startup

Asked at: Spring shops · **Difficulty:** Medium · **Topic:** Startup time

The interviewer's prompt: "Spring Boot startup takes 90 seconds in production. How do you cut it?"

Thinking out loud: 1. "Profile startup with `-XX:StartFlightRecording=duration=60s,filename=startup.jfr` from `main()`. Open in JMC — usually shows reflection (`ClassLoader.loadClass`, `Method.invoke`) dominating." 2. "Spring's autoconfiguration scans tons of classes. Disable unused autoconfigs with `@SpringBootApplication(exclude = ...)` or `spring.autoconfigure.exclude`." 3. "For drastic improvements: GraalVM native image — 50ms startup, no reflection at runtime. Spring Boot 3 supports this natively."

The likely root cause: Heavy reflection scanning, slow classloading, unused autoconfigs running.

The fix: - Exclude unused autoconfigs. - Use `spring.jmx.enabled=false`, lazy initialization (`spring.main.lazy-initialization=true`). - For containerized restarts under SLA, consider CRaC (Coordinated Restore at Checkpoint) on Java 21+. - Verify with `-Dspring-boot.run.jvmArguments="-Xlog:class+load:file=class-load.log"` — should show fewer classes loaded.

What NOT to say: - "Just give it a longer startup probe in K8s" — masks symptom, doesn't help cold starts.

Common follow-up: "What does GraalVM native image gain at startup and what does it lose at runtime?"

Scenario 37 · OOM during JSON serialization

Asked at: Most product cos · **Difficulty:** Medium · **Topic:** Allocation

The interviewer's prompt: " `/orders/export` endpoint OOMs trying to serialize a 2 GB result set. How do you fix?"

Thinking out loud: 1. "Loading 2 GB into memory just to serialize it isn't going to work. The fix is streaming." 2. "Use Jackson's `JsonGenerator` to stream directly to the `HttpServletResponse.getOutputStream()`. For JPA, use `Stream<Entity>` from a query and serialize one entity at a time." 3. "Pair with `session.clear()` after each batch to avoid accumulating in the persistence context."

The likely root cause: Eager materialization of the entire result set before serialization.

The fix: - Stream:

```
java try (Stream<Order> stream = orderRepo.streamAll())
{ stream.forEach(o -> { jsonGen.writeObject(o); session.detach(o); }); } - Set
fetchSize appropriately on the query (Postgres needs a setFetchSize > 0 AND a transaction).
- Verify with sustained heap stable during export.
```

What NOT to say: - "Just return all rows as a List" — that's the bug.

Common follow-up: "Why does Postgres need a transaction for streaming to work?"

Scenario 38 · Image library native leak

Asked at: Swiggy · Zomato · **Difficulty:** Hard · **Topic:** Native

The interviewer's prompt: "Your image-processing service uses ImageIO. Heap is fine but RSS climbs over hours. Why?"

Thinking out loud: 1. " `ImageIO` uses native buffers via `sun.awt.image`. The `BufferedImage.flush()` or `ImageReader.dispose()` calls release native resources — if you skip them, native memory leaks." 2. "Diagnose with NMT and `pmap -x <pid>`. Look for growing anon regions."

The likely root cause: Missing `dispose()` / `flush()` calls on `ImageReader` / `BufferedImage`.

The fix: - Wrap in try/finally with `dispose()` and `flush()`. - Consider migrating to a pure-Java library (Marvin, Scrimage) or a separate process pool for image work. - Verify RSS stays flat under sustained load.

What NOT to say: - "GC will eventually clean it up" — only Java references; native bytes need explicit release.

Common follow-up: "How does `Cleaner` differ from `finalize()` in modern Java?"

Scenario 39 · Thread leak from unbounded executor

Asked at: Cred · Razorpay · **Difficulty:** Medium · **Topic:** Threads

The interviewer's prompt: "Your service has 12,000 threads after a few hours of running. What did you misconfigure?"

Thinking out loud: 1. "`Executors.newCachedThreadPool()` has no upper bound — it creates a new thread whenever no idle thread is available. Under burst load, this spawns thousands of threads." 2. "Each thread costs ~1 MB of stack memory. 12k threads = 12 GB just in stacks." 3. "`jstack | grep -c java.lang.Thread.State` counts threads. Compare to expected pool size."

The likely root cause: Unbounded executor used for high-frequency tasks.

The fix: - Use `Executors.newFixedThreadPool(N)` or a `ThreadPoolExecutor` with `maximumPoolSize` and a bounded queue. - For high-fan-out blocking I/O on Java 21+, use virtual threads — millions of them are cheap. - Verify thread count remains bounded under load.

What NOT to say: - "Cached pool is fine, threads are cheap" — they're not, especially with platform threads.

Common follow-up: "How is a virtual thread's memory footprint different from a platform thread's?"

Scenario 40 · Big regex catastrophic backtracking

Asked at: Razorpay · Cred · **Difficulty:** Medium · **Topic:** CPU

The interviewer's prompt: "A specific user request makes one CPU pin at 100% for 30 seconds, then the request times out. Other requests are fine. What's happening?"

Thinking out loud: 1. "Sounds like regex catastrophic backtracking — a pathological input forces a NFA-based regex engine to try exponential combinations." 2. "Example: `(a+)+$` on input `aaaaaaaaaaaaaaaaaaaaaab` — engine tries every grouping before giving up." 3. "Thread dump shows the thread stuck in `java.util.regex.Pattern$GroupTail.match`."

The likely root cause: User-controlled input fed into a regex with nested quantifiers, no input length cap.

The fix: - Rewrite the regex to avoid nested quantifiers (use possessive quantifiers `a++` or atomic groups `(?>a+)`). - Validate input length before regex. - Run regex on a separate thread with a timeout; cancel if exceeded. - For untrusted input, consider RE2/J — Google's RE2 port with no backtracking.

What NOT to say: - "Regex is always fast" — false; depends on pattern and input.

Common follow-up: "What's the difference between possessive and greedy quantifiers?"

Scenario 41 · Memory leak via Spring `@Cacheable`

Asked at: Spring shops · **Difficulty:** Medium · **Topic:** Cache

The interviewer's prompt: "You added `@Cacheable` everywhere. A week later, heap is full. Why?"

Thinking out loud: 1. " `@Cacheable` defaults to a `ConcurrentHashMap`-backed `ConcurrentMapCacheManager` with no eviction. Every distinct argument combination adds an entry forever." 2. "Heap dump shows the cache map dominating retained heap."

The likely root cause: Default in-memory cache with unbounded growth.

The fix: - Configure Caffeine: `@Bean CacheManager cacheManager() { return new CaffeineCacheManager(...); }` with sizes and TTLs. - Or use Redis as the cache. - Verify with cache metrics: hits, misses, size, evictions.

What NOT to say: - "Spring's cache abstraction handles eviction automatically" — only if you configure it.

Common follow-up: "What's the difference between `@Cacheable` and `@CachePut`?"

Scenario 42 · Stack overflow in deep recursion

Asked at: Most rounds · **Difficulty:** Easy-Medium · **Topic:** Stack

The interviewer's prompt: "Your recursive tree walker throws `StackOverflowError` on deep trees. How do you fix without rewriting to iterative?"

Thinking out loud: 1. "Default thread stack is 512 KB (Linux x86_64). Each Java frame eats ~100-200 bytes, so you get ~2500-5000 frames before overflow." 2. "Bump stack: `-Xss2m` per thread. Note this costs RAM × thread count." 3. "For very deep recursion: tail-recursion is not optimized by HotSpot. Switch to an explicit `Deque<Node>` and iterate."

The likely root cause: Tree depth exceeds stack capacity.

The fix: - Iterative refactor with an explicit stack. - Or `-Xss2m` if you can't refactor and recursion depth is bounded. - For Java 21+ recursive parsing, see if a `StructuredTaskScope` and virtual threads help (virtual threads have a much smaller initial stack).

What NOT to say: - "Java does tail-call optimization like Scala" — it doesn't, and probably won't.

Common follow-up: "Why doesn't HotSpot do tail-call optimization?"

Scenario 43 · JFR allocation profile

Asked at: Senior rounds at product cos · **Difficulty:** Medium · **Topic:** GC pressure

The interviewer's prompt: "You suspect allocation pressure but aren't sure which code path. How do you find the worst allocators?"

Thinking out loud: 1. "JFR has 'TLAB allocation' and 'allocation outside TLAB' events. Run `jcmd <pid> JFR.start duration=60s settings=profile filename=alloc.jfr`, open in JMC, look at the 'Allocations' tab." 2. "Sort by allocation rate descending. Top entries are usually `byte[]`, `char[]`, `String`, internal collection arrays." 3. "For deeper analysis, async-profiler with `-e alloc` produces an allocation flame graph showing the call stacks responsible."

The likely root cause: Hot path generating short-lived garbage — usually string concatenation in a loop, autoboxing, or oversized buffers.

The fix: - Cache `StringBuilder`, switch to `LongAdder` /primitives, size collections initially. - Verify by re-running JFR — top allocator should drop.

What NOT to say: - "Allocation is free thanks to TLAB" — bump-pointer allocation is fast, but the garbage still has to be collected; high allocation rate = high GC rate.

Common follow-up: "What's TLAB and why does it speed up allocation?"

Scenario 44 · Slow HikariCP connection acquisition

Asked at: Razorpay · Swiggy · **Difficulty:** Medium · **Topic:** DB pool

The interviewer's prompt: "P99 `getConnection()` time from HikariCP is 800ms. What could cause that?"

Thinking out loud: 1. "Either the pool is too small and waiters queue up, or the underlying database is slow to hand out new physical connections during pool growth." 2. "Hikari has a `connectionAcquisitionTimeout` and a metric `hikaricp_connections_pending` — if pending is high, demand exceeds pool size." 3. "Also check `hikaricp_connections_creation_time` — high values point at slow DB handshake, often SSL or DNS issues."

The likely root cause: Pool size too small for concurrent load, or slow connection setup (TLS handshake, DNS resolution per connect).

The fix: - Right-size: `maximumPoolSize = (cores_on_db * 2) + storage_spindles` is the Hikari heuristic; tune via load test. - Set `idleTimeout` long enough to avoid frequent reconnects. - Pre-warm: `minimumIdle = maximumPoolSize` keeps the pool full. - Verify with `getConnection p99 < 5ms` under load.

What NOT to say: - "Just disable connection pooling" — every request paying TCP+TLS+auth is suicidal.

Common follow-up: "What's the difference between `minimumIdle` and `maximumPoolSize`?"

Scenario 45 · ZGC for low-latency service

Asked at: Cred · Zerodha · Razorpay · **Difficulty:** Medium · **Topic:** Modern GC

The interviewer's prompt: "Your trading service needs sub-10ms P99.9. What GC do you pick on Java 21?"

Thinking out loud: 1. "ZGC: concurrent, sub-millisecond pauses regardless of heap size. Enable: `-XX:+UseZGC`." 2. "Java 21 added generational ZGC: `-XX:+UseZGC -XX:+ZGenerational` — much better throughput than non-generational ZGC for typical workloads." 3. "Shenandoah is similar — concurrent compaction, sub-ms pauses; pick based on JDK distribution (Shenandoah is OpenJDK + RedHat, ZGC is Oracle-led but in OpenJDK)."

The likely root cause: G1 pause spikes on large heaps; want guaranteed low pause.

The fix: - ZGC + generational mode + appropriate heap sizing. - Don't over-set heap — ZGC reads from page tables, very large heaps incur metadata overhead. - Verify with histogram of GC pause times after switch.

What NOT to say: - "ZGC is experimental" — production-ready since Java 15; generational ZGC GA in 21.

Common follow-up: "Why does ZGC pause time not depend on heap size?"

Scenario 46 · Excessive GC roots from JNI globals

Asked at: Senior rounds · **Difficulty:** Hard · **Topic:** GC roots

The interviewer's prompt: "Heap dump's GC root list contains 50,000 `JNI global` references. What's that?"

Thinking out loud: 1. "JNI code can hold Java objects via `NewGlobalRef()`. These are GC roots — they pin objects forever until `DeleteGlobalRef()` is called." 2. "Leaking globals is a common JNI bug — the C code allocates them but doesn't release in error paths."

The likely root cause: Native library leaking JNI global references.

The fix: - Audit C/C++ code: pair every `NewGlobalRef` with `DeleteGlobalRef`. - Use `NewWeakGlobalRef` if you don't actually need to pin. - Verify root count drops in heap dump.

What NOT to say: - "JNI is mostly safe" — JNI is notoriously easy to misuse.

Common follow-up: "What's the difference between local, global, and weak global JNI references?"

Scenario 47 · GC-induced thread starvation

Asked at: Cred · Quant shops · **Difficulty:** Hard · **Topic:** GC

The interviewer's prompt: "Under heavy GC, your real-time threads miss deadlines even with low pause times. Why?"

Thinking out loud: 1. "Even concurrent GCs do some STW work — root scanning, barriers, safepoints. Frequent safepoints add jitter." 2. "GC's concurrent workers compete with application threads for CPU. High allocation rate → high GC CPU → less CPU for app threads."

The likely root cause: Allocation rate forces GC to consume substantial CPU; safepoint frequency adds tail latency.

The fix: - Reduce allocation: object pools (carefully — pool reuse vs GC is a trade-off), primitive arrays instead of object arrays. - For hard-real-time, look at Azul Zing or compile-time-allocated structures. - Pin GC threads to specific cores if cores are dedicated.

What NOT to say: - "Java is unsuitable for real-time" — too broad; depends on what "real time" means.

Common follow-up: "What is a safepoint and why does the JVM need them?"

Scenario 48 · `ParallelStream` blocking common pool

Asked at: Spring shops · **Difficulty:** Medium · **Topic:** Streams

The interviewer's prompt: "You called `list.parallelStream().forEach(x -> httpClient.get(x))`. Throughput collapses. Why?"

Thinking out loud: 1. "`parallelStream()` runs on the common `ForkJoinPool`. Each blocking HTTP call holds a pool thread. With default pool size (`cores - 1`), only a handful of HTTP calls run concurrently." 2. "Worse, other code using parallel streams or `CompletableFuture.supplyAsync` is starved."

The likely root cause: Blocking I/O on common pool.

The fix: - For blocking I/O, never use `parallelStream`. Use an explicit executor. - `java ExecutorService ex = Executors.newFixedThreadPool(50); list.forEach(x -> ex.submit(() -> httpClient.get(x)));` - For Java 21+, `Executors.newVirtualThreadPerTaskExecutor()` is ideal here.

What NOT to say: - "Parallel streams are always faster" — only for CPU-bound work on the right data sizes.

Common follow-up: "When is `parallelStream` actually the right choice?"

Scenario 49 · Memory leak from forgotten `Closeable`

Asked at: Most rounds · **Difficulty:** Easy · **Topic:** Resources

The interviewer's prompt: "Your service runs out of file descriptors after a few hours. What's the bug?"

Thinking out loud: 1. "Resource leak. Each `new FileInputStream` allocates an OS file descriptor. Without `close()`, the FD stays until the object is GC'd, which is non-deterministic." 2. "Default limit is often 1024 (`ulimit -n`). Hit it, you get `Too many open files`." 3. "`lsof -p <pid> | wc -l` shows live FD count."

The likely root cause: Missing `try-with-resources` somewhere.

The fix: - Always: `try (FileInputStream in = new FileInputStream(...)) { ... }` — auto-closes via `AutoCloseable`. - Audit with static analysis (SonarLint, ErrorProne) — they catch unclosed resources. - Raise ulimit only as belt-and-suspenders; don't rely on it.

What NOT to say: - "Java's GC closes file handles eventually" — non-deterministic; not a fix.

Common follow-up: "What's the difference between `try-with-resources` and a finally block?"

Scenario 50 · GC log not being written

Asked at: TCS · Infosys · **Difficulty:** Easy · **Topic:** GC config

The interviewer's prompt: "You set `-XX:+PrintGCDetails` but no GC log appears. Why?"

Thinking out loud: 1. "`-XX:+PrintGCDetails` was the Java 8 flag. In Java 9+, GC logging was unified — use `-Xlog:gc*:file=gc.log:time,uptime,level,tags`." 2. "On modern JVMs, the old flag is still parsed but emits a deprecation warning and logs to stdout, not a file."

The likely root cause: Using legacy flags on modern JVMs.

The fix: - Java 11+: `-Xlog:gc*,gc+heap=debug,gc+phases=debug:file=/var/logs/gc-%t.log:time,uptime,level,tags:filecount=10,filesize=100M`. - Rotate with `filecount` and `filesize` parameters.

What NOT to say: - "GC logging slows the JVM" — overhead is negligible (<1%), always-on in production is standard.

Common follow-up: "What's the difference between the `default` and `info` log levels in unified logging?"

Scenario 51 · Hibernate second-level cache leak

Asked at: Spring/Hibernate shops · **Difficulty:** Medium · **Topic:** ORM cache

The interviewer's prompt: "You enabled Hibernate's second-level cache with EhCache. Memory leaks. Why?"

Thinking out loud: 1. "L2 cache is shared across sessions. If you don't configure region sizes, entries accumulate." 2. "In `ehcache.xml`: every cached region needs `maxEntriesLocalHeap` or `maxBytesLocalHeap` plus `timeToLiveSeconds`."

The likely root cause: Unbounded L2 cache regions.

The fix: - Configure caps per region. - Use Hibernate stats (`enable_statistics=true`) and verify hit rates. - Verify heap stays stable under sustained load.

What NOT to say: - "L2 cache is always a win" — depends on access patterns; sometimes hurts.

Common follow-up: "What's the difference between query cache and entity cache?"

Scenario 52 · Big string allocation in hot path

Asked at: Cred · Razorpay · **Difficulty:** Medium · **Topic:** Allocation

The interviewer's prompt: "Allocation profile shows 60% of allocation is `byte[]` s in a string concatenation. How do you fix?"

Thinking out loud: 1. " `String + String` in Java 9+ uses `StringConcatFactory` (invokedynamic) and is usually efficient. But in a loop, each iteration still allocates." 2. "Use `StringBuilder` with explicit initial capacity: `new StringBuilder(estimatedSize)` — avoids reallocation." 3. "For high-throughput logging, switch to structured logging with `Logger.atInfo().setMessage(...).log()` — log4j2's fluent API avoids varargs allocation."

The likely root cause: Repeated string concatenation in loops.

The fix: - Pre-size `StringBuilder` . - For logging, use parameterized messages: `log.info("user {} bought {}", userId, sku)` — avoids string concat when level disabled.

What NOT to say: - "String concat is free now" — true for single concat outside loops; false in tight loops.

Common follow-up: "How does `StringConcatFactory` work under the hood?"

Scenario 53 · Slow Spring bean wiring at startup

Asked at: Spring shops · **Difficulty:** Medium · **Topic:** Startup

The interviewer's prompt: "A specific `@Bean` takes 20 seconds to initialize at startup. How do you find which one?"

Thinking out loud: 1. "Spring Boot exposes `/actuator/startup` (since 2.4) — JSON of bean creation timings." 2. "Or `-Ddebug` + `grep startup log` for slowest creation events." 3. "Common culprits: lazy DB schema validation, blocking calls in `@PostConstruct`."

The likely root cause: Synchronous network call or expensive computation in a constructor or `@PostConstruct`.

The fix: - Defer expensive work with `@EventListener(ApplicationReadyEvent.class)` — runs after context is up. - Or `@Async` initialization with a callback. - Verify with re-run of `/actuator/startup`.

What NOT to say: - "Just disable that bean" — losing functionality isn't a fix.

Common follow-up: "What's the order of bean initialization vs `@PostConstruct` vs `ApplicationReadyEvent`?"

Scenario 54 · Memory fragmentation on G1

Asked at: Senior rounds at product cos · **Difficulty:** Hard · **Topic:** GC internals

The interviewer's prompt: "You see 'humongous allocation' warnings in G1 GC log. What does that mean?"

Thinking out loud: 1. "G1 divides heap into regions (default ~2 MB, configurable). An allocation > 50% of a region size is 'humongous' — bypasses normal allocation, goes directly to a contiguous span of old-gen regions." 2. "Humongous allocations fragment the heap and trigger more frequent concurrent cycles."

The likely root cause: Large arrays or large strings being allocated frequently.

The fix: - Raise `-XX:G1HeapRegionSize=32m` (max is 32 MB). Reduces allocations classified as humongous. - Break large arrays into chunks if possible. - Verify with `gc+humongous=trace` log tag — should show fewer events.

What NOT to say: - "Just allocate smaller objects" — sometimes you genuinely need a big buffer; configure G1 to handle it.

Common follow-up: "How do humongous allocations interact with concurrent marking?"

Scenario 55 · Time-based memory leak in cache

Asked at: Cred · PhonePe · **Difficulty:** Medium · **Topic:** Cache

The interviewer's prompt: "Your Caffeine cache uses `expireAfterAccess(30 minutes)`. Memory grows steadily. Why?"

Thinking out loud: 1. "`expireAfterAccess` doesn't eagerly evict — entries stay in memory until the next read/write touches the segment, or a cleanup task runs (Caffeine schedules these via the executor)." 2. "If the cache is rarely read but constantly written, evictions lag. The cache effectively grows to whatever the write rate × 30 min is."

The likely root cause: Cache size policy `maximumSize()` not set; relying solely on time-based expiry.

The fix: - Always combine size + time:

`maximumSize(10_000).expireAfterAccess(Duration.ofMinutes(30))` . - Configure a `scheduler` for proactive cleanup: `.scheduler(Scheduler.systemScheduler())` . - Verify with `cache.estimatedSize()` over time.

What NOT to say: - "Caffeine evicts immediately on TTL" — it doesn't; eviction is amortized.

Common follow-up: "What's the difference between `expireAfterWrite` and `expireAfterAccess`?"

Scenario 56 · Allocation in lambda capture

Asked at: Senior rounds · **Difficulty:** Medium · **Topic:** Lambdas

The interviewer's prompt: "A lambda inside a hot loop seems to allocate per iteration. Why?"

Thinking out loud: 1. "If a lambda captures local variables (a capturing lambda), the JVM allocates a new instance per evaluation. Non-capturing lambdas are cached as singletons." 2. "Verify with allocation profile — look for synthetic `Lambda$$` classes."

The likely root cause: Loop creates a capturing lambda each iteration.

The fix: - Make the lambda non-capturing — pass arguments explicitly to the called method. - Or hoist the lambda outside the loop. - For one-shot use, accept the allocation cost; it's only an issue in tight loops.

What NOT to say: - "Lambdas are free" — true for non-capturing; not for capturing in hot paths.

Common follow-up: "What's the difference between method references and lambdas in allocation terms?"

Scenario 57 · Autoboxing in hot loop

Asked at: Most rounds · **Difficulty:** Easy-Medium · **Topic:** Performance

The interviewer's prompt: " `Map<Integer, Integer>` access in a hot loop is allocating. How do you avoid?"

Thinking out loud: 1. "Every `map.get(intKey)` autoboxes the int to `Integer`. Integers up to 127 use the cache; outside that range, each call allocates." 2. "For high-throughput primitive maps, use Eclipse Collections or fastutil — `IntIntHashMap` avoids boxing entirely."

The likely root cause: Autoboxing on every map access.

The fix: - Replace `HashMap<Integer, Integer>` with `IntIntHashMap` (Eclipse Collections) or `Int2IntOpenHashMap` (fastutil). - Verify allocation profile.

What NOT to say: - "JIT optimizes boxing away" — sometimes via escape analysis, often not in megamorphic call sites.

Common follow-up: "What's `Integer.valueOf()` caching and when does it not help?"

Scenario 58 · CompletableFuture chain leak

Asked at: Razorpay · Cred · **Difficulty:** Medium · **Topic:** Async

The interviewer's prompt: "A chain of `thenApply().thenCompose().thenApply()` leaks memory if never completed. Why?"

Thinking out loud: 1. " `CompletableFuture` holds references to its dependents until completion. A chain that's started but never completed (e.g. caller stops waiting, or the source future is GC'd while async chain holds it) leaks." 2. "Heap dump shows many `CompletableFuture$UniApply` instances retaining lambdas."

The likely root cause: Orphaned future chains.

The fix: - Always complete or cancel futures. - Use `.orTimeout(Duration)` (Java 9+) to fail futures after a deadline. - Verify with allocation profile showing fewer `CompletableFuture$*` instances.

What NOT to say: - "Futures are GC'd automatically" — only when no references remain; orphaned chains can be reachable.

Common follow-up: "What's the difference between `thenApply` and `thenApplyAsync`?"

Scenario 59 · Allocation inside synchronized block

Asked at: Senior rounds · **Difficulty:** Hard · **Topic:** Concurrency

The interviewer's prompt: "You see throughput drop because of contention on a synchronized block. Profile shows allocation inside it. What's the link?"

Thinking out loud: 1. "A long-running synchronized block holds the monitor longer, increasing contention. Allocation inside it (object creation, string concat) prolongs the lock-held duration." 2. "Also: synchronization can interfere with escape analysis and inhibit scalar replacement of objects allocated inside the block."

The likely root cause: Heavyweight operations inside a critical section.

The fix: - Move allocation outside the synchronized block. Lock only the minimal critical update. - Consider lock-free alternatives (`AtomicReference.updateAndGet` , `ConcurrentHashMap.compute`).

What NOT to say: - "Synchronization cost is fixed" — wrong; it scales with contention and critical-section length.

Common follow-up: "What's the difference between biased locking and lightweight locking?"

Scenario 60 · Profile shows safepoint sync time high

Asked at: Cred · Zerodha · **Difficulty:** Hard · **Topic:** Safepoints

The interviewer's prompt: "GC log shows

`Total time for which application threads were stopped` is high but actual GC time is low. What's the gap?"

Thinking out loud: 1. "GC pause = safepoint sync time + GC work time. If sync time is high, threads are slow to reach a safepoint after the JVM requests one." 2. "Causes: long-running JIT loops without back-edge safepoints (e.g. counted int loops in old HotSpot), JNI calls that don't poll, or thread doing system calls." 3. "Enable `-Xlog:safepoint*` for diagnostics."

The likely root cause: A thread inside a JIT-compiled loop without safepoint polls.

The fix: - Modern JDKs add safepoint polls more aggressively; upgrade. - Break long loops into chunks. - Profile to identify the offending method.

What NOT to say: - "Safepoints are an internal detail" — they directly impact tail latency in production.

Common follow-up: "What's a 'time to safepoint' (TTSP) and why does it matter?"

Scenario 61 · GraalVM JIT vs HotSpot

Asked at: Cred · Cutting-edge product cos · **Difficulty:** Hard · **Topic:** JIT alternatives

The interviewer's prompt: "You evaluated GraalVM as JIT instead of HotSpot C2 and got 15% throughput improvement. Why?"

Thinking out loud: 1. "GraalVM's JIT (the Graal compiler, when used as JIT under HotSpot via `-XX:+UseJVMCICompiler`) does better escape analysis, partial escape analysis, and more aggressive inlining than C2." 2. "For workloads heavy in Streams, lambdas, and short-lived objects, Graal can give 10-30% gains."

The likely root cause: Better optimization for modern Java idioms.

The fix: - Pilot on a non-critical service. Measure with realistic load. - Note: Graal as JIT requires Oracle/GraalVM JDK or `-XX:+EnableJVMCI -XX:+UseJVMCICompiler`.

What NOT to say: - "Graal is only for native image" — no, it's both an AOT compiler (native image) and a JIT.

Common follow-up: "What's partial escape analysis and how is it different from regular EA?"

Scenario 62 · Polymorphic call site megamorphic

Asked at: Cred · Senior backend · **Difficulty:** Hard · **Topic:** JIT

The interviewer's prompt: "A method that calls an interface's method got slower as your codebase grew. Why might that be?"

Thinking out loud: 1. "The JIT optimizes interface calls via inline caches. Monomorphic (one impl) is fastest — fully inlined. Bimorphic (two impls) is still fast — switch on klass. Megamorphic (>2 impls) falls back to vtable lookup, no inlining." 2. "As more implementations of an interface accumulate, call sites flip from mono → mega."

The likely root cause: Hot interface call site became megamorphic.

The fix: - For very-hot paths, replace interface with concrete type or sealed interface (Java 17+). - Consider class hierarchy redesign: fewer impls of interfaces on the hot path.

What NOT to say: - "Interface dispatch is always free" — it has predictable cost only when monomorphic.

Common follow-up: "How does the JIT decide when to deoptimize?"

Scenario 63 · Compilation thresholds and warm-up

Asked at: Cred · Zerodha · **Difficulty:** Medium · **Topic:** JIT warmup

The interviewer's prompt: "Your service's first 100 requests are 10x slower than steady state. How do you fix?"

Thinking out loud: 1. "JIT compilation is invocation-count-driven. C1 kicks in around 1500 invocations, C2 around 10000 (tiered)." 2. "For latency-sensitive cold start: pre-warm critical paths by replaying a representative request stream before joining the load balancer." 3. "Or use AOT: GraalVM native image, or HotSpot's AOT (JEP 295, jaotc — though jaotc was removed in JDK 17)."

The likely root cause: Cold JIT.

The fix: - Warm-up requests before LB join. - AppCDS (Application Class Data Sharing) to speed class loading. - Project Leyden / CRaC for snapshotting (Java 21+ work in progress).

What NOT to say: - "JIT warm-up is unavoidable" — for cold start it's a real problem; multiple options exist.

Common follow-up: "What's AppCDS and how does it help startup?"

Scenario 64 · Memory grows on Tomcat redeploy

Asked at: Java EE shops · **Difficulty:** Medium · **Topic:** Classloader leak

The interviewer's prompt: "Tomcat redeploys are leaking memory. Heap dump shows two `WebappClassLoader` instances. What's the typical fix?"

Thinking out loud: 1. "The old classloader can't be GC'd because something outside the app holds a reference to a class loaded by it." 2. "Common culprits: JDBC drivers registered in `java.sql.DriverManager`, log4j context map, ThreadLocals on Tomcat's request threads, MBeans not unregistered." 3. "Tomcat's manager app has a 'Find leaks' button — uses some heuristics."

The likely root cause: Reference outside the webapp's classloader to a webapp class.

The fix: - Enable `JreMemoryLeakPreventionListener` in `server.xml`. - Implement `contextDestroyed()` to deregister JDBC drivers and shut down executors. - Verify with a heap dump after several redeploys.

What NOT to say: - "Tomcat is the problem, switch to Jetty" — same issue applies; it's a JVM/app concern, not server-specific.

Common follow-up: "Why does JDBC driver registration in `DriverManager` leak the classloader?"

Scenario 65 · Concurrent allocation contention

Asked at: Senior rounds · **Difficulty:** Hard · **Topic:** TLAB

The interviewer's prompt: "Why does allocation throughput degrade with more threads even though TLAB should isolate them?"

Thinking out loud: 1. "TLABs (Thread-Local Allocation Buffers) make per-thread allocation cheap — bump pointer in a per-thread chunk." 2. "But when a TLAB is exhausted, the thread requests a new one from the shared Eden — that requires a CAS or lock." 3. "With many threads allocating fast, TLAB refill contention becomes a bottleneck."

The likely root cause: Small TLABs and high allocation rate.

The fix: - Tune `-XX:TLABSize=1m` (and `-XX:-ResizeTLAB` to disable auto-shrink). - Reduce allocation rate where possible. - Use object reuse for very high allocation paths.

What NOT to say: - "Allocation is always lock-free" — TLAB refill is not.

Common follow-up: "How does TLAB sizing interact with `-XX:MinTLABSize`?"

Scenario 66 · Soft reference cache too aggressive

Asked at: Senior backend · **Difficulty:** Medium · **Topic:** References

The interviewer's prompt: "You used `SoftReference` to make a cache. Under memory pressure, the entire cache evicts in one GC. Why?"

Thinking out loud: 1. "SoftReferences are cleared by the GC at its discretion, typically just before OOM. Under pressure, all softs may be cleared at once — cache cliff." 2. "Tune with `-XX:SoftRefLRUPolicyMSPerMB=1000` (default ~1000) — controls how long softs survive based on free heap."

The likely root cause: GC clearing all softs simultaneously.

The fix: - Don't use SoftReferences for caches you care about. Use Caffeine with explicit size/TTL. - If you must, tune `SoftRefLRUPolicyMSPerMB` to keep entries longer.

What NOT to say: - "Soft references are a great cache primitive" — modern caches (Caffeine, Guava) handle eviction better.

Common follow-up: "What's the difference between `SoftReference`, `WeakReference`, and `PhantomReference`?"

Scenario 67 · Memory pressure from log MDC

Asked at: Spring shops · **Difficulty:** Medium · **Topic:** Logging

The interviewer's prompt: "You added MDC (Mapped Diagnostic Context) for distributed tracing. Memory grows in pooled threads. Why?"

Thinking out loud: 1. "MDC is backed by a `ThreadLocal<Map<String, String>>`. Set values stay on the thread until explicitly cleared." 2. "In a pooled-thread server, the previous request's MDC values linger until the next request overwrites them — and some keys may never be overwritten."

The likely root cause: MDC not cleared at end of request.

The fix: - Call `MDC.clear()` in a filter's finally block. - For Spring, the `MDCInsertingServletFilter` (from Logback) does this. - Verify by inspecting a thread dump — MDC `ThreadLocal` should be empty when no request is active.

What NOT to say: - "MDC manages itself" — it doesn't; it's just a `ThreadLocal`.

Common follow-up: "How does MDC interact with `CompletableFuture` chains?"

Scenario 68 · OkHttp/Apache HTTP client connection leak

Asked at: Razorpay · Swiggy · **Difficulty:** Medium · **Topic:** HTTP client

The interviewer's prompt: "You see `Connection pool exhausted` from Apache `HttpClient`. Where's the leak?"

Thinking out loud: 1. "Apache `HttpClient` requires explicit consumption + release of `HttpEntity`. Missing `EntityUtils.consume(entity)` or `response.close()` leaks the underlying connection." 2. "OkHttp requires `response.close()` or `body.close()`."

The likely root cause: Forgetting to close response bodies.

The fix: - Always use try-with-resources for responses. - For Apache:

`EntityUtils.consumeQuietly(entity)` in finally. - Verify with pool stats:

`PoolStats.getLeased() == 0` after request.

What NOT to say: - "HTTP clients pool connections so leaks heal themselves" — they don't; pool exhaustion is permanent until restart.

Common follow-up: "What's the difference between OkHttp's `Response.body().close()` and `Response.close()`?"

Scenario 69 · Kafka consumer rebalance memory

Asked at: Swiggy · Razorpay · **Difficulty:** Medium · **Topic:** Kafka

The interviewer's prompt: "During a Kafka rebalance, your consumer JVM allocates a lot. Why?"

Thinking out loud: 1. "Rebalance triggers callbacks (`onPartitionsRevoked` , `onPartitionsAssigned`). Allocations come from metadata fetches, offset commits, and per-partition state reset." 2. "Also, in-flight records being processed get dropped/redelivered, causing temp object churn."

The likely root cause: Normal but high-rate allocation during rebalance.

The fix: - Tune `max.poll.records` so the per-poll allocation is bounded. - Use static membership (`group.instance.id`) to avoid full rebalances on transient disconnects. - Verify with allocation profile during rebalance.

What NOT to say: - "Rebalances should never happen" — they happen; design for them.

Common follow-up: "What's static group membership and when does it help?"

Scenario 70 · Hibernate dirty checking cost

Asked at: Spring/Hibernate shops · **Difficulty:** Medium · **Topic:** ORM

The interviewer's prompt: "Your `@Transactional` method does only reads but profile shows high CPU in `DefaultFlushEntityEventListener`. Why?"

Thinking out loud: 1. "Hibernate dirty-checks every managed entity at flush time, comparing current state to the snapshot taken at load. With thousands of loaded entities, this is $O(\text{entities} \times \text{fields})$." 2. "For

read-only ops, use `@Transactional(readOnly = true)` — Hibernate skips dirty checking." 3. "Or `StatelessSession` — no persistence context, no dirty checking."

The likely root cause: Dirty checking on a large persistence context.

The fix: - `@Transactional(readOnly = true)` for read methods. - DTO projections to avoid loading entities you don't need. - Verify with profiler — dirty-check frames should disappear.

What NOT to say: - "Dirty checking is free" — it's $O(\text{entity} \times \text{fields})$ per flush.

Common follow-up: "What's the difference between `flush` and `commit` in JPA?"

Scenario 71 · Spring proxy CGLIB heap usage

Asked at: Spring shops · **Difficulty:** Medium · **Topic:** AOP

The interviewer's prompt: "You see thousands of CGLIB-generated classes in Metaspace. Where do they come from?"

Thinking out loud: 1. "Spring uses CGLIB for proxying classes (interfaces use JDK dynamic proxies). Each proxied bean generates a synthetic class." 2. "With prototype-scoped beans plus heavy AOP, you can generate thousands of these."

The likely root cause: Excessive prototype-scoped proxies.

The fix: - Use singleton-scoped beans where possible. - For interfaces, JDK proxies (no synthetic class) — but they apply to interface methods only. - Cap Metaspace and monitor.

What NOT to say: - "Spring overhead is fixed" — depends on configuration.

Common follow-up: "What's the difference between JDK proxies and CGLIB proxies in Spring?"

Scenario 72 · Spike in old gen on TTL expiry

Asked at: Cred · PhonePe · **Difficulty:** Hard · **Topic:** Cache + GC

The interviewer's prompt: "Every hour, old gen jumps when a cache TTL expires. Latency spikes too. Why?"

Thinking out loud: 1. "All entries expire at the same time → all references released → next GC has lots of garbage in old gen → potentially triggers a mixed/concurrent cycle and pauses." 2. "This is a 'cache stampede' pattern."

The likely root cause: Synchronized TTL expiry.

The fix: - Jitter expiry:

`expireAfterWrite(baseTTL.plus(Duration.ofSeconds(random.nextInt(60))))` . - Refresh ahead (Caffeine `refreshAfterWrite`) — async refresh before expiry, so cache never goes empty.

What NOT to say: - "TTL is supposed to expire" — yes, but stampeding causes correlated GC and downstream load.

Common follow-up: "What's the difference between `refreshAfterWrite` and `expireAfterWrite`?"

Scenario 73 · Stream groupBy memory

Asked at: Most rounds · **Difficulty:** Easy-Medium · **Topic:** Streams

The interviewer's prompt: "You do `stream.collect(Collectors.groupingBy(...))` on a huge dataset and OOM. Alternative?"

Thinking out loud: 1. "`groupingBy` materializes the entire result map in memory. For large data, switch to a streaming approach — process one group at a time." 2. "If data is already sorted by key, use a custom collector or iterator-based grouping that emits each group as it's complete." 3. "For DB-sourced data, push the `GROUP BY` to the database."

The likely root cause: In-memory full materialization.

The fix: - Push aggregation to DB. - Or process via a sorted stream + window-based grouping.

What NOT to say: - "Just give it more heap" — won't help if dataset grows.

Common follow-up: "What's the difference between `Collectors.groupingBy` and `Collectors.partitioningBy`?"

Scenario 74 · Spike in thread count after deploy

Asked at: Razorpay · Swiggy · **Difficulty:** Medium · **Topic:** Threads

The interviewer's prompt: "After a deploy, thread count jumps from 200 to 800 and stays there. What might have changed?"

Thinking out loud: 1. "Someone added a new library or feature that spawns its own threads. Common culprits: HTTP clients (Apache, Netty), Kafka consumers, scheduled executors, async logging." 2.

"Compare `jstack` thread names before/after — new thread name prefixes give away the source."

The likely root cause: Library default thread pool size, not adjusted for the use case.

The fix: - Configure libraries' thread pool sizes explicitly. - Audit `Executors.newCachedThreadPool` usage. - Cap total threads.

What NOT to say: - "Threads are cheap" — at 800+ platform threads, memory and scheduling overhead is real.

Common follow-up: "How would virtual threads change this picture in Java 21?"

Scenario 75 · Slow `String.split` in hot path

Asked at: Cred · Razorpay · **Difficulty:** Easy-Medium · **Topic:** Strings

The interviewer's prompt: "You profile and see `String.split` is the top CPU hotspot. Why is it slow?"

Thinking out loud: 1. "`String.split` compiles the regex on every call (with fast paths for single-char patterns, but still). For hot paths, precompile and reuse a `Pattern`." 2. "For simple single-character splits, `Pattern.compile("\\|\\|\\|\\|").split(s)` reused is much faster than `s.split("\\|\\|\\|\\|\\|")` repeated."

The likely root cause: Regex recompilation per call.

The fix: - Hoist `Pattern.compile()` to a static final field. - Or use Guava's `Splitter` for cleaner API. - For very simple delimiters, write a manual `indexOf`-based split.

What NOT to say: - "String.split is always fast" — it's fine for one-shot use, slow in tight loops.

Common follow-up: "What's the fast-path optimization in `String.split` for single-char patterns?"

Scenario 76 · Out of memory during file upload

Asked at: Razorpay · Cred · **Difficulty:** Easy-Medium · **Topic:** I/O

The interviewer's prompt: "File uploads sometimes OOM. The endpoint receives 200 MB files. What's the issue?"

Thinking out loud: 1. "Default `MultipartFile` (Spring) buffers the entire upload to memory if below `spring.servlet.multipart.file-size-threshold`. Otherwise to disk." 2. "Set threshold low (e.g. 1 MB) so big files go to disk."

The likely root cause: Multipart configuration buffering entire file in heap.

The fix: - `spring.servlet.multipart.file-size-threshold=1MB`
`spring.servlet.multipart.max-file-size=500MB`
`spring.servlet.multipart.location=/tmp/uploads` - Or stream uploads directly to S3/storage without intermediate buffer.

What NOT to say: - "Bigger heap will fix it" — until next time someone uploads a 2 GB file.

Common follow-up: "How do you stream a file directly from request body to S3 without loading into heap?"

Scenario 77 · Compaction phase in Cassandra-style

Asked at: Senior backend at data shops · **Difficulty:** Hard · **Topic:** Heap pressure

The interviewer's prompt: "Your service does background compaction work. During compaction, GC pauses spike. Why?"

Thinking out loud: 1. "Compaction reads many records, creates many temp objects, and writes new ones. High allocation rate → high GC pressure." 2. "Plus, compaction may hold large structures live in old gen, increasing concurrent-cycle duration."

The likely root cause: Compaction allocation pressure.

The fix: - Throttle compaction rate. - Reuse byte buffers — `ByteBuffer.clear()` and refill instead of allocate. - Run compaction on a dedicated thread with a separate executor; consider isolating with GC settings via `-XX:+UseG1GC -XX:G1HeapRegionSize=...`.

What NOT to say: - "Background work shouldn't affect GC" — it does; GC is global.

Common follow-up: "How would generational ZGC handle this differently?"

Scenario 78 · Excessive GC roots from static finals

Asked at: Senior rounds · **Difficulty:** Medium · **Topic:** GC roots

The interviewer's prompt: "Heap dump shows a class with 50 static final fields, each holding a large object. Memory never reduces. Why?"

Thinking out loud: 1. "Static fields of loaded classes are GC roots. They live as long as the classloader does." 2. "In a long-running server, classes loaded by the system classloader live forever."

The likely root cause: Static fields holding large data structures.

The fix: - Avoid static caches for large data. Use a managed bean with explicit lifecycle. - For configuration, use static finals only for small primitives or interned strings.

What NOT to say: - "Statics are free" — they're permanent for the lifetime of their classloader.

Common follow-up: "What's the lifetime difference between a class loaded by the bootstrap classloader and the application classloader?"

Scenario 79 · Memory leak through anonymous inner class

Asked at: Senior rounds · **Difficulty:** Medium · **Topic:** Memory leak

The interviewer's prompt: "An anonymous `Runnable` keeps its enclosing class alive. Why?"

Thinking out loud: 1. "Non-static inner classes (and anonymous classes) hold an implicit reference to the enclosing instance via the synthetic `this$0` field." 2. "If you submit this `Runnable` to a long-lived executor, the enclosing instance is pinned for the duration."

The likely root cause: Implicit outer-class reference.

The fix: - Use `static` inner classes when no outer state is needed. - Or use lambdas (which capture only what they use, not the enclosing `this`). - Verify with heap dump — anonymous-class instances should not pin outer.

What NOT to say: - "Inner classes are syntactic sugar" — they have real reference semantics.

Common follow-up: "How is a lambda's capture different from an anonymous inner class's?"

Scenario 80 · WebSocket session leak

Asked at: Swiggy · Zomato · **Difficulty:** Medium · **Topic:** Memory leak

The interviewer's prompt: "You store `WebSocketSession` in a `Map<UserId, Session>`. Memory grows. What's wrong?"

Thinking out loud: 1. "Sessions need explicit removal when disconnected. Spring's WebSocket lifecycle calls `afterConnectionClosed` — if you don't remove the entry, you leak."

The likely root cause: Missing `afterConnectionClosed` cleanup.

The fix: - Implement `afterConnectionClosed` to remove from the map. - Verify with map size matching active session count.

What NOT to say: - "Sessions GC when they disconnect" — they don't; the map strong-refs them.

Common follow-up: "How would you implement heartbeat-based cleanup as a safety net?"

Scenario 81 · Exception construction cost

Asked at: Cred · Senior backend · **Difficulty:** Medium · **Topic:** Performance

The interviewer's prompt: "Your error path is slow. Profile shows `Throwable.fillInStackTrace` as a hotspot. Why?"

Thinking out loud: 1. "Constructing an exception walks the stack. Stack-walking is expensive (allocates `StackTraceElement[]`, native call)." 2. "If your code throws exceptions on a hot path (e.g. as control flow), this dominates."

The likely root cause: Exceptions used as control flow.

The fix: - Don't use exceptions for control flow. Return `Optional`, sentinel, or a result type. - If you must throw frequently, consider overriding `fillInStackTrace` to be a no-op (for specific exception classes).

What NOT to say: - "Exceptions are cheap" — they're cheap to catch, expensive to throw.

Common follow-up: "Why is `fillInStackTrace` so slow?"

Scenario 82 · ZipFile leak

Asked at: Most rounds · **Difficulty:** Easy · **Topic:** I/O

The interviewer's prompt: "Your code reads many `ZipFile`s. RSS grows. Heap is fine. Why?"

Thinking out loud: 1. " `ZipFile` allocates native memory (memory-mapped data) and a file descriptor. Without `close()`, both leak." 2. "Until JDK 9, even closing wasn't enough — `ZipFile` used native allocations with weak cleanup."

The likely root cause: Missing `close()` .

The fix: - Always try-with-resources. - Verify with NMT and `lsof` .

What NOT to say: - "GC will eventually close it" — wrong; native resources need explicit release.

Common follow-up: "What's the difference between `ZipFile` and `ZipInputStream`?"

Scenario 83 · Thread dump shows `IN_NATIVE`

Asked at: Senior backend · **Difficulty:** Medium · **Topic:** Thread dumps

The interviewer's prompt: "A thread is stuck in `IN_NATIVE` state for minutes. What's it doing?"

Thinking out loud: 1. "`IN_NATIVE` means the thread is executing native code (JNI, OS syscall). Common cases: a blocking socket read, a `nanosleep` , a JNI library call." 2. "Look at the topmost native frame for hints — `recv` , `read` , library-specific function."

The likely root cause: Blocking syscall.

The fix: - For network calls, set socket timeouts. - For JNI work, ensure native side has timeouts.

What NOT to say: - "Native threads can't be interrupted" — true for `Thread.interrupt()` , but you can close the underlying socket.

Common follow-up: "How does `Thread.interrupt()` interact with NIO `InterruptibleChannel` ?"

Scenario 84 · GC log shows promotion failures

Asked at: Senior backend · **Difficulty:** Hard · **Topic:** G1 internals

The interviewer's prompt: "GC log says 'to-space exhausted' or 'promotion failed'. What happened?"

Thinking out loud: 1. "G1 needs space in old gen to promote survivors. If old gen is full enough that there's no contiguous region available, promotion fails — fall back to a full GC." 2. "Triggers: allocation rate too high vs concurrent cycle speed, or many humongous objects fragmenting old gen."

The likely root cause: Old gen too full when young GC starts.

The fix: - Lower `-XX:InitiatingHeapOccupancyPercent` so concurrent cycle starts earlier. - Raise heap. - Investigate humongous allocations.

What NOT to say: - "Promotion failures are inevitable" — they're a tuning failure.

Common follow-up: "What's the difference between concurrent marking and mixed collection in G1?"

Scenario 85 · Spring `RestTemplate` connection pool

Asked at: Spring shops · **Difficulty:** Medium · **Topic:** HTTP

The interviewer's prompt: " `RestTemplate` calls a slow downstream. Service threads accumulate. Why?"

Thinking out loud: 1. "Default `RestTemplate` uses `SimpleClientHttpRequestFactory` — no connection pool, no proper timeouts. Under slow downstream, threads pile up waiting." 2. "Switch to `HttpComponentsClientHttpRequestFactory` with proper timeouts and a connection pool."

The likely root cause: No timeout, no pool.

The fix: - Configure read/connect timeouts. - Use a pooled connection manager. - Better: migrate to `WebClient` (reactive, non-blocking).

What NOT to say: - "RestTemplate is fine for everything" — defaults are unsafe.

Common follow-up: "What's the difference between `RestTemplate` and `WebClient`?"

Scenario 86 · GC time exceeds 50% of CPU

Asked at: Razorpay · Cred · **Difficulty:** Medium · **Topic:** GC pressure

The interviewer's prompt: "jstat shows GCT taking 50% of CPU time. What does that mean?"

Thinking out loud: 1. "GC consuming half the CPU = application getting half the CPU. Heap too small, or allocation rate too high." 2. "jstat -gccapacity to see heap sizes; jstat -gccause for the cause of recent collections."

The likely root cause: Allocation rate vs heap size mismatch.

The fix: - Raise heap, or reduce allocation (object reuse, primitives). - Switch GC algorithm if appropriate (ZGC, Shenandoah).

What NOT to say: - "GC is supposed to use CPU" — at 50%, it's a problem.

Common follow-up: "What's a good target for GC overhead percentage?"

Scenario 87 · Class loading slowness

Asked at: Senior rounds · **Difficulty:** Medium · **Topic:** Startup

The interviewer's prompt: "Profile shows `ClassLoader.loadClass` dominating startup. How do you speed it up?"

Thinking out loud: 1. "AppCDS: dump class metadata to a shared archive, JVM mmap's it at startup. 10-30% startup gain." 2. "Generate the archive:

```
java -XX:ArchiveClassesAtExit=app.jsa -jar app.jar . Use: java -XX:SharedArchiveFile=app.jsa -jar app.jar ."
```

The likely root cause: Class loading from many JAR files.

The fix: - AppCDS. - Or GraalVM native image (no class loading at runtime).

What NOT to say: - "Startup time doesn't matter" — for serverless, K8s autoscale, it absolutely does.

Common follow-up: "What's the difference between CDS and AppCDS?"

Scenario 88 · Spring `@Async` thread pool

Asked at: Spring shops · **Difficulty:** Easy-Medium · **Topic:** Async

The interviewer's prompt: "You use `@Async` everywhere. Throughput collapses. Why?"

Thinking out loud: 1. "Default `@Async` uses `SimpleAsyncTaskExecutor` — creates a new thread per task, no pool. Under load, thread count explodes." 2. "Configure a proper `ThreadPoolTaskExecutor` bean."

The likely root cause: Default unbounded thread creation.

The fix: - Define a `@Bean("taskExecutor") ThreadPoolTaskExecutor` with bounded core/max and a queue. - Or `@Async("named")` with named executor.

What NOT to say: - "Spring handles it automatically" — defaults are bad.

Common follow-up: "What's the difference between `ThreadPoolTaskExecutor` and `ThreadPoolExecutor`?"

Scenario 89 · JsonNode tree retention

Asked at: Spring + Jackson shops · **Difficulty:** Medium · **Topic:** JSON

The interviewer's prompt: "You parse JSON into a `JsonNode` tree, extract a single field, and return it. Memory leaks. Why?"

Thinking out loud: 1. "If you return a `JsonNode` (subtree), it can hold a reference back to the parent tree via parent links. The whole document stays alive." 2. "Extract the value to a String or POJO before returning."

The likely root cause: Subtree retention via parent links.

The fix: - Convert the needed field to a primitive value before returning. - Use `ObjectMapper.readValue(..., TargetClass.class)` instead of tree model.

What NOT to say: - "Jackson trees are GC'd as soon as you're done" — they're not, if you hold subtrees.

Common follow-up: "What's the difference between tree model and data binding in Jackson?"

Scenario 90 · GC pauses in trading system

Asked at: Zerodha · Trading firms · **Difficulty:** Hard · **Topic:** Low latency

The interviewer's prompt: "Trading system has sub-1ms latency targets. G1 gives 50ms pauses. What's the fix?"

Thinking out loud: 1. "For sub-ms pauses, GC must be concurrent. Options: ZGC (sub-1ms pauses), Shenandoah (sub-10ms typical), Azul Zing (commercial, pauseless)." 2. "Also: reduce allocation drastically. Reuse buffers. Use off-heap (Chronicle Bytes). Avoid autoboxing."

The likely root cause: GC algorithm not suited for latency target.

The fix: - ZGC + generational mode. - Allocation-free design on hot paths. - Bench with histogram of latency percentiles to verify.

What NOT to say: - "Java can't do low latency" — it can; you have to design for it.

Common follow-up: "What's an allocation-free programming style and what does it cost?"

Scenario 91 · OOM in Spring scheduled task

Asked at: Spring shops · **Difficulty:** Medium · **Topic:** Schedulers

The interviewer's prompt: "A `@Scheduled` task accumulates state in a member field and eventually OOMs. Why?"

Thinking out loud: 1. "The scheduled bean is singleton-scoped — same instance lives forever. Member fields are not reset between runs." 2. "If the task accumulates results in a `List<Result>` field, it grows forever."

The likely root cause: Singleton state accumulating.

The fix: - Use local variables in the scheduled method, not member fields. - Or explicitly clear member fields at the end of each run.

What NOT to say: - "Spring resets beans between calls" — no, singletons persist.

Common follow-up: "How is `@Scheduled` different from a `ScheduledExecutorService`?"

Scenario 92 · Heap dump comparison

Asked at: Senior rounds · **Difficulty:** Medium · **Topic:** Diagnosis

The interviewer's prompt: "You have two heap dumps — one healthy, one leaking. How do you diff them?"

Thinking out loud: 1. "MAT supports comparing two snapshots. Histogram diff shows which classes grew." 2. "Or use Eclipse MAT's `OQL` (Object Query Language) — `SELECT * FROM com.foo.Bar` and count by classloader."

The likely root cause: N/A — methodology question.

The fix: - Compare histograms, sort by delta, investigate the top growers.

What NOT to say: - "Take only one dump" — diff is much more informative.

Common follow-up: "What's MAT's OQL and when would you use it?"

Scenario 93 · BufferedReader memory

Asked at: Most rounds · **Difficulty:** Easy · **Topic:** I/O

The interviewer's prompt: "Your CSV reader OOMs on a 5 GB file. How do you fix?"

Thinking out loud: 1. "You're probably reading into a `List<String>` or `String[]`. Don't — stream line by line." 2. "`Files.lines(path)` returns a `Stream<String>` — process one line at a time."

The likely root cause: Full materialization.

The fix: - `try (Stream<String> lines = Files.lines(path))`
`{ lines.forEach(this::process); }` . - Process line by line, don't accumulate.

What NOT to say: - "Just read the whole file" — won't scale.

Common follow-up: "What's the difference between `BufferedReader.readLine` in a loop and `Files.lines`?"

Scenario 94 · Memory leak from unmodifiableMap wrapper

Asked at: Senior rounds · **Difficulty:** Medium · **Topic:** Collections

The interviewer's prompt: "You wrap a `HashMap` with `Collections.unmodifiableMap()` and pass it around. The wrapper holds the original. Why does it matter?"

Thinking out loud: 1. "`unmodifiableMap` is a view, not a copy. The wrapper holds a reference to the backing map. If you mutate the backing map, the view changes." 2. "For true immutability, use `Map.copyOf(map)` (Java 10+) — copies eagerly."

The likely root cause: Confusing view with copy.

The fix: - Use `Map.copyOf()` when you need a snapshot. - Use the unmodifiable wrapper only when you want to share a live read-only view.

What NOT to say: - "Unmodifiable means immutable" — different concepts.

Common follow-up: "What's the difference between `Collections.unmodifiableMap` and `Map.copyOf`?"

Scenario 95 · Recovering from prod OOM

Asked at: Razorpay · Cred · **Difficulty:** Medium · **Topic:** Ops

The interviewer's prompt: "Your service OOM'd in production at 2 AM. What do you do first, second, third?"

Thinking out loud: 1. "First, recover service — restart the JVM, restore traffic. Heap dump is captured automatically via `-XX:+HeapDumpOnOutOfMemoryError` ." 2. "Second, copy the dump off the host, replace the host (or restart), confirm metrics back to baseline." 3. "Third, analyze the dump in MAT. Find the leak suspect. Open a bug, write a regression test, fix, deploy."

The likely root cause: Varies — but the response process matters more than guessing root cause.

The fix: - Always run with `-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/var/dumps/` . - Maintain a runbook for OOM response. - Add a metric/alert for old-gen utilization approaching limit (catch it before OOM).

What NOT to say: - "Restart and move on" — you'll see it again.

Common follow-up: "What's a 'kill switch' for a runaway service?"

Scenario 96 · Spring's `@Transactional` self-invocation

Asked at: Spring shops · **Difficulty:** Medium · **Topic:** Spring proxy

The interviewer's prompt: "Your `@Transactional` method calls another `@Transactional` method in the same class. The inner transaction doesn't behave as expected. Why?"

Thinking out loud: 1. "Spring AOP proxies intercept external calls to the bean. Self-invocation (`this.method()`) bypasses the proxy — annotations don't apply." 2. "This isn't strictly a memory issue, but related questions about Spring internals come up."

The likely root cause: Self-invocation bypasses proxy.

The fix: - Inject `self` reference (or `ApplicationContext.getBean(this.getClass())`) and call through it. - Or split into two beans. - Or use AspectJ weaving instead of Spring's proxy AOP.

What NOT to say: - "Annotations always work" — false in self-invocation.

Common follow-up: "How does AspectJ weaving differ from Spring's proxy-based AOP?"

Scenario 97 · Memory leak from `ScheduledFuture`

Asked at: Senior backend · **Difficulty:** Medium · **Topic:** Schedulers

The interviewer's prompt: "You schedule one-off tasks with `ScheduledExecutorService.schedule(...)` . Memory grows. Why?"

Thinking out loud: 1. " `ScheduledThreadPoolExecutor` keeps completed tasks' `ScheduledFuture` in its internal queue until polled. With many short scheduled tasks, the queue grows." 2. "Workaround: call `setRemoveOnCancelPolicy(true)` and `purge()` periodically."

The likely root cause: Internal queue retention.

The fix: - Configure `setRemoveOnCancelPolicy(true)` . - Call `purge()` periodically. - Or batch scheduling.

What NOT to say: - "Scheduled tasks GC after they run" — `Future` references stay.

Common follow-up: "How does `ScheduledThreadPoolExecutor` differ from `Timer` ?"

Scenario 98 · Off-heap memory monitoring

Asked at: Cred · Razorpay · **Difficulty:** Medium · **Topic:** Monitoring

The interviewer's prompt: "How do you monitor off-heap memory usage in production?"

Thinking out loud: 1. "Native Memory Tracking: `-XX:NativeMemoryTracking=summary` , then JMX exposure via JFR or `BufferPoolMXBean` ." 2. "For Micrometer: `JvmMemoryMetrics` includes buffer pools." 3. "Pair with cgroup memory metrics to compare process RSS to JVM-internal accounting."

The likely root cause: N/A — monitoring question.

The fix: - Always-on NMT (low overhead) + Prometheus + alerting on direct memory pool % full.

What NOT to say: - "Off-heap is invisible" — it's not, with NMT.

Common follow-up: "What's the overhead of `-XX:NativeMemoryTracking=detail` vs `summary` ?"

Scenario 99 · jcmd for live diagnostics

Asked at: Most product cos · **Difficulty:** Medium · **Topic:** Tools

The interviewer's prompt: "List the `jcmd` commands you'd use on a live JVM and what each tells you."

Thinking out loud: 1. " `jcmd <pid> VM.uptime` — how long it's been up." 2. " `jcmd <pid> GC.heap_info` — current heap usage." 3. " `jcmd <pid> GC.heap_dump /tmp/dump.hprof` —

heap dump." 4. " `jcmd <pid> Thread.print` — thread dump." 5. " `jcmd <pid> VM.native_memory summary` — NMT (requires `-XX:NativeMemoryTracking`)." 6. " `jcmd <pid> JFR.start name=adhoc duration=120s filename=/tmp/r.jfr` — JFR recording." 7. " `jcmd <pid> Compiler.codecache` — JIT code cache status." 8. " `jcmd <pid> VM.classloader_stats` — classloader info."

The likely root cause: N/A — tool question.

The fix: - These are your everyday tools; memorize the top 5.

What NOT to say: - "I'd use JConsole" — needs JMX exposed; `jcmd` works on any local JVM.

Common follow-up: "What's the difference between `jcmd` and `jstack` / `jmap` ?"

Scenario 100 · Predicting OOM before it happens

Asked at: Razorpay · PhonePe · Cred · **Difficulty:** Medium · **Topic:** Observability

The interviewer's prompt: "How do you build alerting so you catch a memory leak before the OOM in production?"

Thinking out loud: 1. "Track old gen utilization post-GC. If post-GC utilization grows monotonically over hours, it's a leak." 2. "Micrometer's `JvmGcMetrics` exposes `jvm_gc_max_data_size_bytes` and `jvm_gc_live_data_size_bytes` (after collection). Alert when `live/max > 0.8` and trending up." 3. "Pair with a heap dump trigger: `-XX:+HeapDumpBeforeFullGC` (custom logic) or a sidecar that calls `jcmd GC.heap_dump` when old gen crosses 85%."

The likely root cause: N/A — observability question.

The fix: - Alert on post-GC old-gen growth, not on instantaneous heap usage. - Auto-trigger heap dump before the JVM dies. - Verify by simulating a leak in staging.

What NOT to say: - "Wait for the OOM and react" — too late, traffic already lost.

Common follow-up: "How would you build the same alerting for native memory growth?"

Category 5 — Concurrency Bugs

These are the production landmines. Code that passes every unit test, hums quietly in staging, then blows up at 2 AM on Diwali eve when traffic peaks. Each scenario below is a real bug — the kind that

hides behind 99.99% of green builds. Read the code, predict the failure mode, then check your diagnosis.

Scenario 1 · Counter that loses increments under load

Asked at: Razorpay · Cred · Swiggy · **Difficulty:** Medium · **Topic:** Race condition

The code:

```
public class TxnCounter {
    private volatile long count = 0;

    public void recordTxn() {
        count++;
    }

    public long current() {
        return count;
    }
}
```

The interviewer's prompt: "This counter tracks transactions processed per minute. The volatile keeps it thread-safe, right? Yet under load we see 9,983 increments when we expected 10,000. What's wrong?"

Thinking out loud (the diagnosis): 1. "volatile gives visibility — every read sees the latest write — but it doesn't make compound operations atomic." 2. "count++ is three operations: read, add, write. Two threads can read the same value, increment locally, and write back the same new value. One increment vanishes."

The actual bug: count++ is a read-modify-write. volatile guarantees that each individual read/write is visible across threads, but the sequence is not atomic. Lost updates result.

The fix: - **Option A:** AtomicLong count = new AtomicLong(); then count.incrementAndGet(); — CAS-based, lock-free, fast for moderate contention. - **Option B:** LongAdder count = new LongAdder(); then count.increment(); — better under very high contention; reads via sum() aggregate striped cells.

What NOT to say: - "Add synchronized to recordTxn()" — fixes correctness but throttles throughput; AtomicLong is faster. - "Mark count final" — final on a primitive value field is meaningless; the field is reassigned every call.

Common follow-up: "When does LongAdder beat AtomicLong, and when is the reverse true?"

Scenario 2 · `ConcurrentHashMap.get-then-put` loses entries

Asked at: Razorpay · Flipkart · **Difficulty:** Medium · **Topic:** Race condition

The code:

```
private final ConcurrentHashMap<String, List<Order>> orders = new ConcurrentHashMap<>();

public void track(String userId, Order order) {
    List<Order> list = orders.get(userId);
    if (list == null) {
        list = new ArrayList<>();
        orders.put(userId, list);
    }
    list.add(order);
}
```

The interviewer's prompt: "We use `ConcurrentHashMap` so this is thread-safe. Yet we sometimes lose orders for the same user. Why?"

Thinking out loud (the diagnosis): 1. "`ConcurrentHashMap`'s individual operations are atomic. The composite get-then-put is not." 2. "Two threads can both see `list == null`, both create new `ArrayList`s, both `put()` — one overwrites the other, losing the order added to the loser's list." 3. "Even after that, `ArrayList` itself isn't thread-safe — concurrent `add()` calls corrupt internal state."

The actual bug: Two races in one method. (1) check-then-act on the map. (2) Unsynchronized mutation of `ArrayList` from multiple threads.

The fix: - Option A: `orders.computeIfAbsent(userId, k -> new CopyOnWriteArrayList<>()).add(order);` — atomic compute + thread-safe list. - **Option B:** Use `Collections.synchronizedList(new ArrayList<>())` inside `computeIfAbsent` if reads are rare; `CopyOnWriteArrayList` is best when reads outnumber writes.

What NOT to say: - "Wrap the whole method in `synchronized`" — kills the concurrency the CHM was bought for. - "Use `putIfAbsent` instead of `put`" — closes the map race but still allows two threads to construct lists unnecessarily, and the list-mutation race remains.

Common follow-up: "Why is `computeIfAbsent` atomic even though it accepts a user-supplied lambda?"

Scenario 3 · Singleton broken under JIT

Asked at: Cred · Zerodha · **Difficulty:** Hard · **Topic:** Double-checked locking

The code:

```
public class ConfigCache {
    private static ConfigCache instance;

    public static ConfigCache get() {
        if (instance == null) {
            synchronized (ConfigCache.class) {
                if (instance == null) {
                    instance = new ConfigCache();
                }
            }
        }
        return instance;
    }
}
```

The interviewer's prompt: "This is textbook double-checked locking. We see `NullPointerException` intermittently in flight when callers touch the returned instance. Why?"

Thinking out loud (the diagnosis): 1. "DCL pattern needs `volatile` on the instance field — that's the missing piece here." 2. "`new ConfigCache()` is three steps: allocate, run constructor, assign reference. Without `volatile`, the JIT or CPU can reorder allocate-assign-construct." 3. "A second thread can read a non-null reference, but the object's fields are still default values — NPE on access of a field that should be initialized."

The actual bug: Missing `volatile`. The publication of the reference is not safely ordered against the constructor's writes. JMM allows the reordering.

The fix: - **Option A:** `private static volatile ConfigCache instance;` — minimum correct DCL. - **Option B:** Initialization-on-demand holder idiom:
`private static class Holder { static final ConfigCache INSTANCE = new ConfigCache(); } and return Holder.INSTANCE;`. JVM class-init guarantees thread safety, no `volatile` needed, no synchronized cost on the hot path.

What NOT to say: - "Eagerly initialize as a static field" — works but defeats the lazy goal if the constructor is expensive. - "DCL is broken in Java, never use it" — pre-Java 5 yes; with `volatile`, since JSR-133 it is correct.

Common follow-up: "Why is the holder idiom faster than `volatile` DCL on the steady-state read path?"

Scenario 4 · Visibility bug — shutdown flag never seen

Asked at: Swiggy · Cred · **Difficulty:** Medium · **Topic:** Visibility

The code:

```
public class PollLoop {
    private boolean stopRequested = false;

    public void run() {
        while (!stopRequested) {
            poll();
        }
    }

    public void shutdown() {
        stopRequested = true;
    }
}
```

The interviewer's prompt: "Calling shutdown() does nothing — the loop keeps polling forever. The flag is definitely set; we logged it. Why doesn't the loop see it?"

Thinking out loud (the diagnosis): 1. "stopRequested" is a plain field. No memory barrier between writer thread and reader thread." 2. "JIT is free to hoist the read out of the loop after seeing it never changes within the thread — the loop becomes while (true)." 3. "Even without hoisting, the reader can cache the field in a register indefinitely with no happens-before relationship from the writer."

The actual bug: No happens-before edge from shutdown()'s write to the loop's read. JMM lets the JIT speculate that the field is loop-invariant.

The fix: - **Option A:** private volatile boolean stopRequested; — establishes write-to-read visibility, defeats the hoist. - **Option B:** AtomicBoolean stopRequested = new AtomicBoolean(false); — same visibility, plus CAS if you ever need set-once semantics.

What NOT to say: - "Add a sleep in the loop, that'll flush memory" — superstition; sleep doesn't establish happens-before. - "Synchronize the whole loop body" — works but locks every iteration; volatile is the right tool.

Common follow-up: "What's the JMM rule that makes volatile sufficient here?"

Scenario 5 · Deadlock from inconsistent lock order

Asked at: Zerodha · Razorpay · **Difficulty:** Medium · **Topic:** Deadlock

The code:

```
public class Wallet {
    private final Object lock = new Object();
    private long balance;

    public void transfer(Wallet from, Wallet to, long amount) {
        synchronized (from.lock) {
            synchronized (to.lock) {
                from.balance -= amount;
                to.balance += amount;
            }
        }
    }
}
```

The interviewer's prompt: "Settlement service hangs under load. jstack shows two transfer threads BLOCKED forever. Diagnose."

Thinking out loud (the diagnosis): 1. "Two locks held in nested fashion — classic deadlock setup." 2. "Thread 1: transfer(A, B) locks A.lock, waits for B.lock. Thread 2: transfer(B, A) locks B.lock, waits for A.lock. Circular wait — Coffman's fourth condition satisfied."

The actual bug: No global lock ordering. Pair-wise transfers can acquire locks in opposite orders simultaneously.

The fix: - **Option A:** Order locks by `System.identityHashCode` or a stable per-wallet id. Always acquire the smaller hash first. Breaks circular wait. - **Option B:** Use a single shared lock for all transfers — simple but kills concurrency. Use only if transfer throughput is low. - **Option C:** `ReentrantLock.tryLock(timeout)` and back off — releases held locks on timeout, retry with jitter.

What NOT to say: - "Add a third lock to coordinate the other two" — adds more deadlock surface, not less. - "Lock the higher-balance account first" — balance changes during execution; the order isn't stable.

Common follow-up: "How would you implement the identity-hash lock-ordering trick with the rare hash-collision edge case handled?"

Scenario 6 · ThreadLocal leaks across Tomcat requests

Asked at: Cred · Razorpay · enterprise rounds · **Difficulty:** Medium · **Topic:** ThreadLocal leak

The code:


```

public class RequestContext {
    private static final ThreadLocal<String> tenantId = new ThreadLocal<>();

    public static void setTenant(String id) {
        tenantId.set(id);
    }

    public static String currentTenant() {
        return tenantId.get();
    }
}

@RestController
class OrdersController {
    @GetMapping("/orders")
    public List<Order> list(@RequestHeader("X-Tenant") String t) {
        RequestContext.setTenant(t);
        return repo.findByTenant(RequestContext.currentTenant());
    }
}

```

The interviewer's prompt: "User from tenant A occasionally sees tenant B's orders. We can't reproduce locally — only in production behind the LB. What's the bug?"

Thinking out loud (the diagnosis): 1. "Tomcat reuses worker threads across requests. A ThreadLocal set on a thread persists until explicitly cleared." 2. "If a request omits the header, `setTenant` is never called — the thread still has the previous tenant's value. Caller reads stale data." 3. "Even with the header, no `remove()` after the request means slow leaks of header-keyed entries."

The actual bug: ThreadLocal lifecycle mismatch with request lifecycle. No `remove()` in a finally block.

The fix: - **Option A:** Wrap the controller in a filter / interceptor: `try { setTenant(t); chain.doFilter(...); } finally { tenantId.remove(); }`. Standard Spring `OncePerRequestFilter` pattern. - **Option B:** Java 21 `ScopedValue` — immutable, scoped to a `where().run()` block, automatically cleared. Cleaner if the codebase is on JDK 21+.

What NOT to say: - "Make the ThreadLocal `static final` — that fixes it" — already is; the bug is in lifecycle, not declaration. - "Use a new ExecutorService per request" — solves the leak by being absurd.

Common follow-up: "Does this leak still happen with `InheritableThreadLocal` and virtual threads?"

Scenario 7 · Broken happens-before with array publication

Asked at: Cred · Zerodha · **Difficulty:** Hard · **Topic:** Visibility

The code:

```
public class ConfigLoader {
    private int[] thresholds;

    public void load() {
        int[] data = readFromDb();
        thresholds = data;
    }

    public int thresholdFor(int idx) {
        return thresholds[idx];
    }
}
```

The interviewer's prompt: "After load() returns on one thread, another thread sometimes reads zeros instead of the values. The DB is correct. What's happening?"

Thinking out loud (the diagnosis): 1. "Two visibility hazards: the reference assignment and the array contents." 2. "Plain reference write — no guarantee the reader sees the new array pointer at all." 3. "Even if the pointer is seen, no happens-before edge guarantees the reader sees the populated array elements vs the default-zero state immediately after `new int[n]`."

The actual bug: Unsafe publication of the array and its contents. Without a happens-before edge, JMM permits the reader to see the post-allocation, pre-fill state.

The fix: - **Option A:** `private volatile int[] thresholds;` — volatile write of the reference establishes a happens-before edge that also covers the array's prior writes. - **Option B:** `AtomicReference<int[]>` with `set` and `get` — same effect, clearer intent. - **Option C:** For truly immutable config, build with all fields then `final` — final-field freeze gives safe publication without volatile.

What NOT to say: - "Synchronize the read in `thresholdFor`" — works but bottlenecks reads; volatile is enough. - "Copy the array in the reader" — doesn't fix visibility; copy reads stale data too.

Common follow-up: "Why does volatile on the reference field also make the array's prior writes visible?"

Scenario 8 · ExecutorService rejecting tasks silently

Asked at: Swiggy · Razorpay · **Difficulty:** Medium · **Topic:** ExecutorService rejection

The code:

```

ExecutorService pool = new ThreadPoolExecutor(
    4, 4, 0L, TimeUnit.MILLISECONDS,
    new ArrayBlockingQueue<>(100));

public void enqueue(WebhookEvent e) {
    pool.submit(() -> handle(e));
}

```

The interviewer's prompt: "Webhook delivery randomly drops events under bursts. The `handle()` method is never called for some events. No exception in logs. What's the default behavior we're hitting?"

Thinking out loud (the diagnosis): 1. "Bounded queue of 100, 4 threads. Burst > 100 plus 4-in-flight overflows." 2. "Default `RejectedExecutionHandler` is `AbortPolicy` — throws `RejectedExecutionException`." 3. "`pool.submit(...)` returns a `Future`; the exception is wrapped in that `Future`. We never call `get()` on it, so the exception is silently swallowed."

The actual bug: `AbortPolicy` fires under overflow; the rejection lives inside the returned `Future`, which we ignore. Lost events with no log line.

The fix: - **Option A:** `CallerRunsPolicy` — submitter executes the task itself. Acts as natural backpressure on the producer; never drops. - **Option B:** Custom handler that writes to a dead-letter queue / Kafka topic with a clear log line. - **Option C:** `pool.execute(...)` instead of `submit(...)` — rejection now propagates synchronously to the caller, which can decide.

What NOT to say: - "Increase the queue to 100,000" — moves the failure point; doesn't fix the silent-drop. - "Catch all exceptions in `handle()`" — irrelevant; `handle` never ran.

Common follow-up: "With `CallerRunsPolicy`, what risk does the calling thread now take on, and how do you prevent it from blocking your HTTP thread pool?"

Scenario 9 · `CompletableFuture` eats the exception

Asked at: Cred · Razorpay · Swiggy · **Difficulty:** Medium · **Topic:** `CompletableFuture`

The code:

```

public void chargeAndNotify(Order o) {
    CompletableFuture
        .supplyAsync(() -> charge(o))
        .thenApply(receipt -> sendEmail(receipt))
        .thenAccept(r -> log.info("Done: {}", r));
}

```

The interviewer's prompt: "Charges sometimes fail silently. We catch nothing; emails never go out; no exception in logs. What's the issue?"

Thinking out loud (the diagnosis): 1. "The chain never observes the result. No `get()`, no `join()`, no `exceptionally`, no `whenComplete`." 2. "If `charge()` throws, the `CompletableFuture` completes exceptionally. The exception sits in the future, and the dependent stages skip. No one ever surfaces it."

The actual bug: Fire-and-forget `CompletableFuture` with no terminal exception handler. Failures vanish.

The fix: - **Option A:** Chain `.exceptionally(ex -> { log.error("charge failed", ex); return null; })` — terminal handler logs and absorbs. - **Option B:** `.whenComplete((res, ex) -> { if (ex != null) log.error(...); })` — fires on both success and failure. - **Option C:** Structured concurrency in JDK 21 — `try (var scope = new StructuredTaskScope.ShutdownOnFailure())` — exceptions propagate to the parent via `throwIfFailed()`.

What NOT to say: - "Wrap the lambda in try-catch" — catches only the body of that single stage; doesn't help if `sendEmail` throws. - "Use `join()` at the end" — turns async into sync, defeats the point; rethrows checked-as-unchecked.

Common follow-up: "How is exception propagation different between `thenApply` and `thenCompose` in failure paths?"

Scenario 10 · `wait()` without while-loop misses condition

Asked at: Zerodha · Razorpay · **Difficulty:** Medium · **Topic:** Missed wakeup

The code:

```
class Buffer {
    private Object item;

    public synchronized Object take() throws InterruptedException {
        if (item == null) {
            wait();
        }
        Object out = item;
        item = null;
        return out;
    }

    public synchronized void put(Object x) {
        item = x;
        notifyAll();
    }
}
```

The interviewer's prompt: "Consumer sometimes returns null even though we never call put(null). What's the bug?"

Thinking out loud (the diagnosis): 1. " `if (item == null) wait();` — if instead of while." 2. "Spurious wakeups happen — JLS explicitly allows wait() to return without notify()." 3. "Even without spurious wakeups: thread A waits, B puts and notifies, C wakes first and takes the item. A then unblocks with `item == null` but skips the check — returns null."

The actual bug: Predicate not rechecked after wait. `if` instead of `while`. Spurious wakeup or barge-in by another consumer leads to stale read.

The fix: - **Option A:** `while (item == null) wait();` — the textbook fix. Always wait inside a while-loop. - **Option B:** Use a `BlockingQueue` — pre-built, correct, gives `take()` / `put()` semantics out of the box.

What NOT to say: - "Use `notify()` instead of `notifyAll()`" — orthogonal; doesn't fix the spurious-wakeup hole. - "Spurious wakeups are theoretical, they never happen on Hotspot" — they do; production reproduces them.

Common follow-up: "Why does JLS explicitly permit spurious wakeups instead of forbidding them?"

Scenario 11 · Lazy init publishes a half-built object

Asked at: Cred · Flipkart · **Difficulty:** Hard · **Topic:** Broken lazy init

The code:

```
public class PriceTable {
    private Map<String, BigDecimal> prices;

    public BigDecimal of(String sku) {
        if (prices == null) {
            prices = new HashMap<>();
            for (var row : loadFromDb()) {
                prices.put(row.sku(), row.price());
            }
        }
        return prices.get(sku);
    }
}
```

The interviewer's prompt: "Two threads call of() concurrently. One returns null for SKUs that exist. The other returns the correct value. What broke?"

Thinking out loud (the diagnosis): 1. "Plain non-volatile reference. Reader can see the assignment before the map is fully populated." 2. "Even if it saw the full map: HashMap isn't thread-safe. Concurrent puts during the initial load + reads from a second thread can corrupt internal arrays — historically infinite loops in Java 7." 3. "And: `if (prices == null)` plus the body — racey initialization. Two threads may both run the loader."

The actual bug: Three concurrency hazards stacked — unsafe publication, non-thread-safe map, racey init.

The fix: - **Option A:** Holder idiom — `private static class Holder { static final Map<String, BigDecimal> PRICES = load(); }` . JVM class-init guarantees thread-safe one-shot init. - **Option B:** `AtomicReference<Map<String, BigDecimal>>` with `compareAndSet(null, builtMap)` + an `unmodifiableMap` wrapper for safe publication. - **Option C:** Build into an immutable `Map.copyOf(...)` (since Java 10) — the field gets a fully populated immutable map, then safe-published via volatile.

What NOT to say: - "Use ConcurrentHashMap" — fixes the corruption but not the publication or the double-load. - "Synchronize the whole method" — works but penalises every call after the first.

Common follow-up: "Why does the holder idiom avoid the synchronization cost entirely?"

Scenario 12 · Atomic on the wrong granularity

Asked at: Razorpay · **Difficulty:** Medium · **Topic:** Race condition

The code:

```
private final AtomicInteger seq = new AtomicInteger();
private final Map<Integer, String> events = new ConcurrentHashMap<>();

public void publish(String e) {
    int id = seq.get();
    events.put(id, e);
    seq.incrementAndGet();
}
```

The interviewer's prompt: "We use `AtomicInteger` and `ConcurrentHashMap`. Yet events occasionally overwrite each other. What's wrong?"

Thinking out loud (the diagnosis): 1. "`seq.get()` then `seq.incrementAndGet()` is a get-then-increment. Two threads can read the same `id`, both write at that key." 2. "The map and the counter are individually atomic; the composite is not."

The actual bug: Atomicity on individual operations is not atomicity over the sequence. Classic check-then-act misuse of `AtomicInteger`.

The fix: - **Option A:** `int id = seq.getAndIncrement(); events.put(id, e);` — single atomic operation hands out unique ids. - **Option B:** If id-collision detection matters: `events.putIfAbsent(seq.getAndIncrement(), e)` and assert null return.

What NOT to say: - "Use `incrementAndGet()` first and decrement at the end" — pretzel logic; `getAndIncrement` is the right primitive. - "Add `synchronized` around `publish()`" — fixes it but throws away the atomic class.

Common follow-up: "What's the difference between `getAndIncrement` and `incrementAndGet`, and when does it matter?"

Scenario 13 · Virtual thread pinning under synchronized

Asked at: Cred · Razorpay · senior backend · **Difficulty:** Hard · **Topic:** Virtual thread pinning

The code:

```
public class RateLimiter {
    public synchronized boolean tryAcquire() {
        if (tokens > 0) {
            tokens--;
            return true;
        }
        return false;
    }

    public synchronized void refill() throws InterruptedException {
        Thread.sleep(1_000); // wait next window
        tokens = maxTokens;
    }
}
```

The interviewer's prompt: "We migrated to virtual threads with `Executors.newVirtualThreadPerTaskExecutor()`. Throughput tanked instead of improving. Why?"

Thinking out loud (the diagnosis): 1. "Virtual threads unmount from carrier OS threads on blocking IO — unless they're inside a `synchronized` block." 2. "Inside `synchronized`, the virtual thread is pinned to its carrier. The carrier OS thread cannot be reused. With many vthreads piling up in `refill()`, the small carrier pool starves." 3. "`Thread.sleep` inside `synchronized` pins the carrier for the full second. Throughput collapses."

The actual bug: Pinning under `synchronized` blocks the carrier thread, defeating the unmounting advantage of virtual threads.

The fix: - **Option A:** Replace `synchronized` with `ReentrantLock` — vthreads release the carrier when waiting on `lock.lock()` and blocking ops inside. - **Option B:** Don't block inside the critical section — release the lock before `sleep`, re-acquire after. - **Option C:** JDK 21+: enable `-Djdk.tracePinnedThreads=full` to find every such site during testing.

What NOT to say: - "Increase the number of carriers" — band-aid; doesn't fix the root cause. - "Virtual threads are slower than platform threads for synchronized code" — they're equivalent when not pinned; the issue is pinning, not virtual threads.

Common follow-up: "In JDK 24/25 there's work to remove pinning under `synchronized` — what changes?"

Scenario 14 · `CompletableFuture` on common pool blocks the world

Asked at: Cred · Swiggy · **Difficulty:** Medium · **Topic:** `ForkJoinPool` starvation

The code:

```
public CompletableFuture<UserDetails> load(long userId) {  
    return CompletableFuture  
        .supplyAsync(() -> jdbcLookup(userId))  
        .thenApply(this::enrich);  
}
```

The interviewer's prompt: "Parallel streams in other parts of the app slow to a crawl when load() is called heavily. Diagnose."

Thinking out loud (the diagnosis): 1. "supplyAsync without an executor runs on ForkJoinPool.commonPool() — shared by parallel streams, every other Async call, and any library that uses it." 2. "jdbcLookup blocks on a JDBC socket. JDBC isn't a ManagedBlocker. The carrier in the common pool is stuck." 3. "Common pool defaults to availableProcessors - 1. A handful of blocked JDBC calls starves everyone else."

The actual bug: IO-bound blocking work on the common pool. The pool was sized for CPU-bound tasks.

The fix: - **Option A:** Pass a dedicated ExecutorService — supplyAsync(() -> jdbcLookup(id), jdbcExecutor). Pool sized for IO concurrency. - **Option B:** JDK 21+ — Executors.newVirtualThreadPerTaskExecutor() as the executor. Each vthread unmounts on socket read, so blocking is free.

What NOT to say: - "Use parallelStream() instead" — also uses the common pool; same problem. - "Increase common pool size globally" — affects every consumer; surgical executor is cleaner.

Common follow-up: "Why does ManagedBlocker not fix this for arbitrary JDBC drivers?"

Scenario 15 · ABA in lock-free stack

Asked at: Cred · senior systems · **Difficulty:** Hard · **Topic:** ABA problem

The code:

```

public class LockFreeStack<T> {
    private final AtomicReference<Node<T>> top = new AtomicReference<>();

    public void push(T v) {
        Node<T> n = new Node<>(v);
        do {
            n.next = top.get();
        } while (!top.compareAndSet(n.next, n));
    }

    public T pop() {
        Node<T> cur;
        do {
            cur = top.get();
            if (cur == null) return null;
        } while (!top.compareAndSet(cur, cur.next));
        return cur.value;
    }
}

```

The interviewer's prompt: "This stack passes single-threaded tests. Under contention with pooled nodes, sometimes pop returns a value with the wrong `next` chain. Why?"

Thinking out loud (the diagnosis): 1. "CAS compares object identity, not version. With a node pool that recycles freed nodes, a node A can be popped, recycled, and re-pushed." 2. "Thread X reads `top = A`, intends `top.next = B`. Thread Y pops A, pops B, pushes A back. Now `top = A` but `A.next` is something different. X's CAS still succeeds — `A == A` — but the stack is now corrupt."

The actual bug: Classic ABA. CAS sees the same reference, doesn't notice the intervening pops/push.

The fix: - **Option A:** `AtomicStampedReference<Node<T>>` — pairs the reference with a stamp; CAS includes the stamp. - **Option B:** Don't recycle nodes — let GC free them. Then ABA reduces to garbage retention, which is fine for correctness. - **Option C:** Use `ConcurrentLinkedDeque` from JDK and stop reinventing.

What NOT to say: - "Just don't use a node pool" — works but loses the pool's allocation savings if those mattered. - "Add a synchronized around CAS" — defeats lock-free design entirely.

Common follow-up: "How does `AtomicStampedReference` internally avoid being subject to ABA?"

Scenario 16 · False sharing tanks `AtomicLong` throughput

Asked at: Cred · low-latency rounds · **Difficulty:** Hard · **Topic:** False sharing

The code:

```
public class Stats {
    public final AtomicLong reads = new AtomicLong();
    public final AtomicLong writes = new AtomicLong();
    public final AtomicLong errors = new AtomicLong();
}
```

The interviewer's prompt: "Profiling shows AtomicLong updates are 3-5x slower in this class than in isolated micro-benchmarks. We're not even close to memory bandwidth limits. What's the issue?"

Thinking out loud (the diagnosis): 1. "Three AtomicLongs adjacent in memory — likely sharing a CPU cache line (64 bytes on x86)." 2. "Thread A updates `reads`. The cache line invalidates on Thread B's core. Thread B updates `writes`. Cache-line bounces between cores." 3. "Each core's CAS retries because the cache line keeps getting invalidated by the others. False sharing."

The actual bug: Three counters fit in one cache line. Independent updates trigger false sharing — cache coherence traffic kills throughput.

The fix: - **Option A:** `@jdk.internal.vm.annotation.Contended` (JDK 8+ with `-XX:-RestrictContended`) — JVM pads the field to its own cache line. - **Option B:** Use `LongAdder` — internally striped; designed for high-contention counters. Solves false sharing and contention at once. - **Option C:** Manual padding — `long p1, p2, p3, p4, p5, p6, p7;` around the field; brittle but works on old JVMs.

What NOT to say: - "Increase clock speed of the CPU" — not a thing your code can do. - "Use `volatile` instead of `AtomicLong`" — slower, doesn't address false sharing.

Common follow-up: "Why does `@Contended` need a JVM flag to enable in user code?"

Scenario 17 · Structured concurrency drops sibling cancellation

Asked at: Cred · senior · **Difficulty:** Hard · **Topic:** Structured concurrency

The code:

```
public Result fetch(long orderId) throws Exception {
    try (var scope = new StructuredTaskScope<Object>()) {
        var f1 = scope.fork(() -> callPayments(orderId));
        var f2 = scope.fork(() -> callShipping(orderId));
        scope.join();
        return new Result(f1.get(), f2.get());
    }
}
```

The interviewer's prompt: "Payments call fails fast, but shipping keeps running for 30s and the caller waits anyway. We want shipping cancelled when payments fails. What's wrong?"

Thinking out loud (the diagnosis): 1. "Plain `StructuredTaskScope` doesn't cancel siblings on failure — that's `ShutdownOnFailure`'s job." 2. "Plain scope just gathers all subtasks until everyone finishes. Failure of one doesn't propagate."

The actual bug: Wrong scope. `StructuredTaskScope` is the base class without cancellation policy. Need a policy that fits "fail fast".

The fix: - **Option A:** `var scope = new StructuredTaskScope.ShutdownOnFailure()` then `scope.join(); scope.throwIfFailed();` — first failure cancels siblings, raises the exception. - **Option B:** `ShutdownOnSuccess` if you want "first one wins" semantics (e.g. mirrored downstream services).

What NOT to say: - "Wrap each subtask in a timeout" — masks the issue; doesn't propagate the actual failure reason. - "Check `f1.state()` manually and cancel `f2`" — reinvents what `ShutdownOnFailure` does for you.

Common follow-up: "How does `ShutdownOnFailure` actually cancel siblings — what method gets called on them?"

Scenario 18 · Read-modify-write across two collections

Asked at: Flipkart · Swiggy · **Difficulty:** Medium · **Topic:** Race condition

The code:

```
private final ConcurrentHashMap<Long, Cart> carts = new ConcurrentHashMap<>();
private final AtomicLong total = new AtomicLong();

public void addItem(long cartId, Item item) {
    Cart c = carts.get(cartId);
    c.add(item);
    total.addAndGet(item.price());
}
```

The interviewer's prompt: "Cart contents and the aggregate total disagree after high concurrency. What's the bug?"

Thinking out loud (the diagnosis): 1. "Two state updates that need to happen atomically: `Cart.add` and total addition. They aren't." 2. "Thread A adds item ₹100 to cart 1, fails between `Cart.add` and

`total.addAndGet` (e.g. crash). Cart has the item; total doesn't." 3. "Even without failures: read of `total` interleaved between the two updates returns a value that doesn't match the cart sum."

The actual bug: Two related state updates that must be atomic, treated as if independent. Lack of compound atomicity.

The fix: - **Option A:** Lock per cart: `synchronized (c) { c.add(item); total.addAndGet(item.price()); }`. Reads of total are still racy w.r.t. uncompleted updates; design accordingly. - **Option B:** Derive `total` from cart contents lazily — single source of truth. Eliminates the consistency problem at the cost of a scan. - **Option C:** Event sourcing — append an event, project state. Atomic single write.

What NOT to say: - "Use AtomicLong for both" — Cart isn't a long. - "Synchronize across the whole map" — kills concurrency.

Common follow-up: "What property of multi-step state updates is at the core of transactional systems?"

Scenario 19 · Condition signal without holding the lock

Asked at: Razorpay · senior · **Difficulty:** Medium · **Topic:** Condition variable

The code:

```
private final ReentrantLock lock = new ReentrantLock();
private final Condition ready = lock.newCondition();
private volatile boolean done = false;

public void awaitReady() throws InterruptedException {
    lock.lock();
    try {
        while (!done) ready.await();
    } finally {
        lock.unlock();
    }
}

public void markReady() {
    done = true;
    ready.signal(); // no lock held
}
```

The interviewer's prompt: "awaitReady() sometimes hangs forever even though markReady was clearly called. Diagnose."

Thinking out loud (the diagnosis): 1. "`ready.signal()` from `markReady` is called without holding the lock — `IllegalMonitorStateException` at runtime, or in some paths just no-op via wrappers." 2. "Even if signal happens to work: a missed wakeup window — the waiter sees `done == false`, calls `await`; then `markReady` sets `done` and signals. But because signaling without the lock has no happens-before guarantee w.r.t. the `await` call's predicate check, you can race past the signal."

The actual bug: `Condition.signal()` requires holding the associated `Lock`. Calling it without throws `IllegalMonitorStateException`. The intermediate volatile makes it look almost correct but the signal contract is violated.

The fix: - **Option A:** Acquire and release the lock in `markReady` :

```
java lock.lock(); try { done = true; ready.signal(); } finally
{ lock.unlock(); }
```

- **Option B:** Use a `CountDownLatch(1)` if it's a one-shot "ready" gate. Simpler, no lock juggling.

What NOT to say: - "Use `signalAll()` instead of `signal()`" — doesn't matter; both require the lock. - "Drop the lock since `done` is volatile" — wrong contract; `Condition.signal` requires the lock.

Common follow-up: "What's the analogous rule for `Object.notify()`?"

Scenario 20 · synchronized on a Long autobox

Asked at: Razorpay · code-review trap · **Difficulty:** Medium · **Topic:** Lock object choice

The code:

```
public class FxRates {
    private Long lock = 0L;

    public BigDecimal rate(String pair) {
        synchronized (lock) {
            return fetch(pair);
        }
    }

    public void update(String pair, BigDecimal r) {
        synchronized (lock) {
            store(pair, r);
        }
    }
}
```

The interviewer's prompt: "This compiles, runs, and looks thread-safe. In production we see interleaved updates that should have been serialized. What's wrong?"

Thinking out loud (the diagnosis): 1. " `lock` is a `Long` — boxed. Each instance of `FxRates` has its own boxed zero (well — `Long` values `-128..127` are cached, so they're the same object across instances)." 2. "Worse: any code anywhere in the JVM that synchronizes on `Long.valueOf(0)` is contending on the SAME monitor." 3. "And if `lock` is ever reassigned, the previous monitor is abandoned — threads block on different objects."

The actual bug: Locking on a cached boxed primitive. Cross-class contention plus potential reassignment hazard.

The fix: - **Option A:** `private final Object lock = new Object();` — private, dedicated, immutable reference. - **Option B:** `private final ReentrantLock lock = new ReentrantLock();` if you want timed / interruptible.

What NOT to say: - "Make `lock` final but keep it `Long`" — final fixes reassignment but the cache problem remains. - "Synchronize on `this`" — better than `Long` but leaks the lock to external callers.

Common follow-up: "Why does locking on a `String` literal have the same class of bug?"

Scenario 21 · `CountDownLatch` reused as if it were `CyclicBarrier`

Asked at: senior backend · **Difficulty:** Medium · **Topic:** Synchronizer misuse

The code:

```
public class BatchRunner {
    private CountDownLatch latch = new CountDownLatch(5);

    public void runBatch(List<Runnable> tasks) throws InterruptedException {
        for (var t : tasks) executor.submit(() -> { t.run(); latch.countDown(); });
        latch.await();
        // process next batch using the same latch
    }
}
```

The interviewer's prompt: "First batch processes correctly. Second batch's `await` returns immediately with no tasks having run. Why?"

Thinking out loud (the diagnosis): 1. " `CountDownLatch` is single-use. Once the count reaches zero, it stays zero — every subsequent `await` returns immediately." 2. "For repeated phases, you need a fresh latch per batch or a `CyclicBarrier` / `Phaser`."

The actual bug: Wrong synchronizer. `CountDownLatch` is one-shot; cannot be reset.

The fix: - **Option A:** Construct a fresh `CountDownLatch(n)` per batch. - **Option B:** `CyclicBarrier` — reusable, parties wait until all arrive; auto-resets for the next round. - **Option C:** `Phaser` if batch size is dynamic — supports register/deregister.

What NOT to say: - "Call `latch = new CountDownLatch(5)` inside the method and forget to make it final" — works mechanically but is fragile and racey with concurrent batches. - "Use synchronized + count manually" — reinventing.

Common follow-up: "When would you reach for `Phaser` over `CyclicBarrier`?"

Scenario 22 · synchronized method overridden, lock lost

Asked at: enterprise rounds · **Difficulty:** Medium · **Topic:** Inheritance + synchronization

The code:

```
class Counter {
    private int n;
    public synchronized void inc() { n++; }
    public synchronized int get() { return n; }
}

class LoggingCounter extends Counter {
    @Override
    public void inc() {
        super.inc();
        log.info("inc");
    }
}
```

The interviewer's prompt: "Counter is thread-safe. LoggingCounter extends it and adds logging. Yet LoggingCounter occasionally returns wrong reads. Diagnose."

Thinking out loud (the diagnosis): 1. "`synchronized` is not inherited as part of the method signature. `LoggingCounter.inc` overrides without `synchronized`." 2. "Inside override: `super.inc()` acquires the monitor briefly, releases, then `log.info` runs without the lock — `n` updated, no longer protected by surrounding mutual exclusion." 3. "A concurrent `get()` between `super.inc()` and the log call sees an updated value that was supposed to be committed together with the log line."

The actual bug: Overriding `synchronized` method without re-declaring `synchronized`. The override silently drops the lock contract on its full body.

The fix: - Option A: `public synchronized void inc() { super.inc(); log.info("inc"); }` — re-declare synchronized. - **Option B:** Composition over inheritance: `LoggingCounter` holds a `Counter` and forwards through its own lock.

What NOT to say: - "synchronized is inherited" — flat-out wrong; it's a method-implementation attribute. - "Mark the method `final`" — doesn't add the lock; would prevent further override.

Common follow-up: "Why does Java not propagate `synchronized` through inheritance — what would break if it did?"

Scenario 23 · Iterating a `ConcurrentHashMap` inside `compute`

Asked at: Cred · **Difficulty:** Medium-Hard · **Topic:** `ConcurrentHashMap` misuse

The code:

```
ConcurrentHashMap<String, Stats> map = new ConcurrentHashMap<>();

public void merge(String k, long v) {
    map.compute(k, (key, cur) -> {
        if (cur == null) cur = new Stats();
        cur.add(v);
        // recompute other keys based on this one
        for (var other : map.keySet()) {
            map.computeIfPresent(other, (ok, ov) -> { ov.recompute(); return ov; });
        }
        return cur;
    });
}
```

The interviewer's prompt: "This runs fine for a while then hangs or throws bizarre `IllegalStateException`. What's the issue?"

Thinking out loud (the diagnosis): 1. "`compute`"'s lambda runs under a per-bin lock inside `ConcurrentHashMap`. 2. "Inside, we iterate keys and call `computeIfPresent` on potentially the SAME bin — recursive bin-lock acquisition, often leading to deadlock or undefined behavior." 3. "Javadoc explicitly says: 'do not attempt to update any other mappings of this map' within `compute`'s function."

The actual bug: Modifying the map from within `compute` violates the documented contract. CHM uses bin-level locks that aren't reentrant across bins safely.

The fix: - Option A: Snapshot keys first, then iterate outside the `compute` lambda: `var keys = new ArrayList<>(map.keySet()); ...`. - **Option B:** Two-phase update — gather changes in a local map, then apply via `forEach` or per-key `compute`.

What NOT to say: - "Wrap in synchronized" — masks the issue; the contract violation remains. - "Use HashMap with explicit lock" — gives up CHM's concurrency benefits entirely.

Common follow-up: "What are the documented restrictions on the function passed to compute/merge/computeIfPresent?"

Scenario 24 · BlockingQueue.offer without checking return

Asked at: Swiggy · **Difficulty:** Easy-Medium · **Topic:** Queue API misuse

The code:

```
BlockingQueue<Event> q = new ArrayBlockingQueue<>(1000);

public void emit(Event e) {
    q.offer(e);
}
```

The interviewer's prompt: "Events get silently dropped under load. We use a bounded queue intentionally. What's the API mistake?"

Thinking out loud (the diagnosis): 1. "`offer(e)` returns `false` immediately when the queue is full — non-blocking semantics. We ignore the boolean." 2. "Producer fires events into a full queue, all return false, all silently dropped."

The actual bug: `offer` non-blocking variant used where backpressure was needed; return value discarded.

The fix: - **Option A:** `q.put(e)` — blocking. Producer blocks when queue is full; natural backpressure.

- **Option B:** `if (!q.offer(e, timeout, TimeUnit.MILLISECONDS)) { metrics.dropped(); }` — bounded wait, observable drop. - **Option C:** `if (!q.offer(e)) { spillToDeadLetter(e); }` — fall back to a dead letter.

What NOT to say: - "Increase queue size" — moves the bug, doesn't fix it. - "Use `add(e)` instead" — `add` throws on full, slightly better than silent drop but uncaught exceptions are still hidden.

Common follow-up: "What's the contract difference between `add`, `offer`, and `put` on BlockingQueue?"

Scenario 25 · Stale read after volatile write — wrong assumption

Asked at: Cred · **Difficulty:** Hard · **Topic:** Happens-before

The code:

```
public class Cache {
    private Map<String, String> data = new HashMap<>();
    private volatile boolean ready = false;

    public void load() {
        data.put("a", "1");
        data.put("b", "2");
        ready = true;
    }

    public String read(String k) {
        if (!ready) return null;
        return data.get(k);
    }
}
```

The interviewer's prompt: "We use volatile to publish ready safely. Yet read() occasionally returns null for present keys. Why?"

Thinking out loud (the diagnosis): 1. "Volatile write of `ready = true` establishes happens-before — prior writes ARE visible to a thread that sees `ready == true`. That part is fine." 2. "But the data map itself is HashMap — internal arrays etc. Once both threads start reading and only one thread writes, that's technically safe-after-publication, BUT..." 3. "...nothing prevents another writer from also calling `load()` later or `data.put` later. And `HashMap`'s `get()` interleaved with any concurrent structural modification — including the constructor's internal init — is unsafe." 4. "Specifically: if `load()` runs more than once, or if any other thread mutates the map post-publication, the reader's `HashMap.get` can return null for a key that's present (mid-resize)."

The actual bug: Safe initial publication, but no protection from subsequent mutation. HashMap remains non-thread-safe after the volatile write.

The fix: - **Option A:** Use `ConcurrentHashMap` if mutations continue. Volatile only safely publishes one snapshot, not ongoing mutability. - **Option B:** If load is one-shot: replace data with an `unmodifiableMap(new HashMap(...))` populated inside load, then assign once. Readers see immutable contents. - **Option C:** Holder idiom for one-shot loaders — JVM guarantees safe class-init publication.

What NOT to say: - "Volatile is enough" — only for the publication moment, not for ongoing mutation. - "Make the map field final" — final on a mutable HashMap doesn't prevent mutation; just prevents reassignment.

Common follow-up: "What's the difference between safe publication and thread-safe mutation?"

Scenario 26 · Static SimpleDateFormat shared across threads

Asked at: TCS · Infosys · Razorpay · **Difficulty:** Easy-Medium · **Topic:** Thread-unsafe library class

The code:

```
public class Audit {  
    private static final SimpleDateFormat FMT = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");  
  
    public static String stamp(Date d) {  
        return FMT.format(d);  
    }  
}
```

The interviewer's prompt: "Audit logs sometimes have malformed timestamps — months that don't exist, jumbled fields. Where's the corruption coming from?"

Thinking out loud (the diagnosis): 1. "SimpleDateFormat is not thread-safe. Holds mutable internal calendar state." 2. "Concurrent format calls share that calendar — fields get overwritten mid-format. Corrupted output."

The actual bug: SimpleDateFormat is documented as not thread-safe. Used as a static singleton — classic Java gotcha.

The fix: - **Option A:** DateTimeFormatter (Java 8+) — immutable, thread-safe by design. Use DateTimeFormatter.ofPattern(...) . - **Option B:** ThreadLocal<SimpleDateFormat> with withInitial(...) — works but adds ThreadLocal cleanup concerns in thread pools. - **Option C:** Construct per-call — wasteful but correct.

What NOT to say: - "Synchronize the format method" — works but bottleneck. - "SimpleDateFormat is thread-safe in Java 8+" — false, never was.

Common follow-up: "What other JDK classes have similar 'thread-unsafe singleton' traps?"

Scenario 27 · Scheduled task swallows exception and stops running

Asked at: Cred · Swiggy · **Difficulty:** Medium · **Topic:** ScheduledExecutorService

The code:

```
scheduler.scheduleAtFixedRate(() -> {
    refreshRates();
}, 0, 30, TimeUnit.SECONDS);
```

The interviewer's prompt: "FX rates stop refreshing after some hours. The scheduler is alive — other tasks run. What stops this one?"

Thinking out loud (the diagnosis): 1. " `scheduleAtFixedRate` cancels future executions if the task throws an exception. The `ScheduledFuture` completes exceptionally." 2. " `refreshRates` throws once (transient HTTP error). The task is silently removed from the scheduler. No more executions, no log."

The actual bug: Uncaught exception kills the periodic schedule. Documented behavior of `ScheduledExecutorService`.

The fix: - **Option A:** Wrap in try-catch: `java scheduler.scheduleAtFixedRate(() -> { try { refreshRates(); } catch (Throwable t) { log.error("rates", t); } }, ...);` - **Option B:** Override `ScheduledThreadPoolExecutor.afterExecute` to log thrown exceptions from completed tasks — catches the issue centrally.

What NOT to say: - "Use `scheduleWithFixedDelay` instead" — same exception-cancels behavior. - "Just restart the scheduler if a task fails" — would need to detect; the whole point is detection is missing.

Common follow-up: "How is this behavior different in Spring's `@Scheduled` annotation?"

Scenario 28 · Future.get with no timeout hangs the request thread

Asked at: Razorpay · **Difficulty:** Easy-Medium · **Topic:** Future timeout

The code:

```
@GetMapping("/quote")
public Quote quote(long id) throws Exception {
    Future<Quote> f = executor.submit(() -> upstream.quote(id));
    return f.get();
}
```

The interviewer's prompt: "When upstream is slow, our HTTP threads pile up — Tomcat eventually rejects. Diagnose."

Thinking out loud (the diagnosis): 1. " `f.get()` blocks forever until the task completes." 2. "Tomcat's request thread is held captive; under a slow upstream, threads accumulate; pool fills."

The actual bug: Unbounded blocking on a downstream's Future. No timeout, no fallback.

The fix: - **Option A:** `f.get(2, TimeUnit.SECONDS);` — bounded wait, throw `TimeoutException` on overflow, return a graceful error. - **Option B:** Use `CompletableFuture` with `.orTimeout(2, SECONDS)` and `.exceptionally(...)` — async fallback. - **Option C:** Servlet 3 async (`DeferredResult`) — release the Tomcat thread while waiting.

What NOT to say: - "Increase Tomcat max threads" — moves the failure point. - "Wrap in try-catch" — won't catch a hang; you need a deadline, not error handling.

Common follow-up: "Why does cancelling the Future not actually stop the upstream HTTP call most of the time?"

Scenario 29 · CopyOnWriteArrayList in a write-heavy path

Asked at: Cred · **Difficulty:** Easy-Medium · **Topic:** Wrong collection

The code:

```
private final List<Tick> ticks = new CopyOnWriteArrayList<>();

public void onTick(Tick t) {
    ticks.add(t);
}

public List<Tick> recent() {
    return ticks.subList(Math.max(0, ticks.size() - 100), ticks.size());
}
```

The interviewer's prompt: "Tick handler latency goes from microseconds to 20 milliseconds as the list grows. CPU spikes. Why?"

Thinking out loud (the diagnosis): 1. "`CopyOnWriteArrayList.add` copies the entire underlying array on every write." 2. "At 100k ticks, every add allocates and copies 100k references. $O(n)$ per write, $O(n^2)$ cumulative." 3. "Designed for read-heavy, write-rare workloads. We have the opposite."

The actual bug: CoW costs $O(n)$ per write. Misapplied to a write-heavy stream.

The fix: - **Option A:** `ConcurrentLinkedQueue` if you only need FIFO and `recent()` can iterate. - **Option B:** Bounded ring buffer — `ArrayBlockingQueue` or a custom array-backed circular buffer. Fixed memory; $O(1)$ writes. - **Option C:** `LongAdder` -style sharded per-thread buffers; aggregate on read.

What NOT to say: - "CoWArrayList is always slow" — fast for read-heavy; correct in that niche. - "Synchronize ArrayList writes" — works but no upper bound on memory.

Common follow-up: "What's the actual append complexity of `ArrayList.add(E)` and why isn't it $O(1)$ worst case?"

Scenario 30 · `notify()` wakes the wrong waiter

Asked at: Razorpay · **Difficulty:** Hard · **Topic:** Missed wakeup

The code:

```
class BoundedBuffer<T> {
    private final Queue<T> q = new ArrayDeque<>();
    private final int cap;

    BoundedBuffer(int cap) { this.cap = cap; }

    public synchronized void put(T x) throws InterruptedException {
        while (q.size() == cap) wait();
        q.add(x);
        notify();
    }

    public synchronized T take() throws InterruptedException {
        while (q.isEmpty()) wait();
        T v = q.poll();
        notify();
        return v;
    }
}
```

The interviewer's prompt: "Under several producers and consumers, the buffer occasionally locks up — producers blocked on full, consumers blocked on empty, and nothing moves. Diagnose."

Thinking out loud (the diagnosis): 1. "Single monitor, single wait set. `notify()` wakes ONE arbitrary waiter." 2. "Producer P fills last slot, calls `notify()`. The notify can wake another producer (also waiting on full), not a consumer. That producer rechecks, still full, waits again. Deadlock window." 3. "In short: with a shared wait set for two different conditions, `notify()` can wake the wrong species."

The actual bug: Single condition for two semantic states. `notify()` may wake a thread that can't proceed, missing the one that could.

The fix: - **Option A:** `notifyAll()` everywhere — wakes everyone, the right one progresses. Wasteful but correct. - **Option B:** Use `ReentrantLock` with two `Condition`s (`notFull`, `notEmpty`). `put` signals `notEmpty`; `take` signals `notFull`. Each wait-set is homogeneous. - **Option C:** Use `ArrayBlockingQueue` — already implements exactly this pattern correctly.

What NOT to say: - "Notify works fine; you're imagining it" — happens in production; documented hazard. - "Use `Object.notify()` with a sleep" — useless.

Common follow-up: "What's the asymptotic difference between `notifyAll`-everywhere and two-condition design under contention?"

Scenario 31 · Sync block leaks the lock on lambda capture

Asked at: Cred · senior backend · **Difficulty:** Hard · **Topic:** Lock scope

The code:

```
class Hub {
    private final Object lock = new Object();
    private final List<Listener> listeners = new ArrayList<>();

    public void emit(Event e) {
        synchronized (lock) {
            for (Listener l : listeners) {
                CompletableFuture.runAsync(() -> l.handle(e));
            }
        }
    }
}
```

The interviewer's prompt: "This deadlocks when a listener tries to register another listener back into the same Hub. Why?"

Thinking out loud (the diagnosis): 1. "`synchronized(lock)` is held while we fire-and-forget async tasks. The tasks run on the FJ pool — outside the lock." 2. "That's actually fine for the emit thread releasing the lock — once emit returns, the lock is gone." 3. "But: if a listener (running in the pool) immediately calls `register()`, which itself acquires `lock` ... and the original emit thread is in the same JVM still inside `synchronized` ... no, that releases when emit returns." 4. "Actual bug emerges if a listener calls back synchronously somewhere — but here, async, the deeper issue: holding a lock while iterating, and the iteration calls user-provided code (`Listener.handle`) before exiting `synchronized`." 5. "If `Listener.handle` is implemented to call `Hub.register` synchronously (not async), and the listener implementation grabs internal locks of its own, you've enabled a deadlock by holding `lock` across unknown callbacks."

The actual bug: Holding a lock while invoking arbitrary callback/external code. Even if the calls are async, the iteration submits tasks that might recursively touch the locked region.

The fix: - **Option A:** Snapshot the listeners under the lock, then iterate outside:

```
java List<Listener> snap; synchronized (lock) { snap =
List.copyOf(listeners); } for (var l : snap) executor.submit(() ->
l.handle(e));
```

- **Option B:** Use `CopyOnWriteArrayList<Listener>` — its iterator is lock-free; emit doesn't need synchronization.

What NOT to say: - "Listeners should not call back" — you can't enforce this on external code. - "Add a re-entrant lock" — same hazard.

Common follow-up: "What's the general rule about holding locks across external callbacks?"

Scenario 32 · AtomicReference compound update

Asked at: Cred · low-latency · **Difficulty:** Medium · **Topic:** Atomic misuse

The code:

```
private final AtomicReference<Stats> stats = new AtomicReference<>(new Stats());

public void record(long latency) {
    Stats s = stats.get();
    s.count++;
    s.totalLatency += latency;
    stats.set(s);
}
```

The interviewer's prompt: "We use AtomicReference but counts are still lost. What?"

Thinking out loud (the diagnosis): 1. "`stats.get()`" returns the same mutable Stats every call. We mutate it concurrently. Atomic on the reference doesn't make the pointed-to object atomic." 2.

"`stats.set(s)`" writes back the same object we read — no CAS, no protection against concurrent modifications."

The actual bug: Atomic reference to a mutable object isn't atomic over field updates. Same object mutated without synchronization.

The fix: - **Option A:** Make Stats immutable; use `updateAndGet` : `java stats.updateAndGet(s -> new Stats(s.count + 1, s.totalLatency + latency));` - **Option B:** Use two `LongAdder` s — one for count, one for total. Striped, no shared mutation.

What NOT to say: - "Synchronize the set call" — Stats fields are still racy. - "Volatile Stats" — same hazard; volatility on a reference doesn't help field-level updates.

Common follow-up: "Why does immutability remove the entire class of bugs we just discussed?"

Scenario 33 · LazyHolder accessed during class init recursion

Asked at: Cred · **Difficulty:** Hard · **Topic:** Class init deadlock

The code:

```
public class Config {
    private static final Map<String, String> M = init();

    private static Map<String, String> init() {
        return App.bootstrap().configMap();
    }
}

public class App {
    private static final Config CFG = new Config();
    public static App bootstrap() { return new App(); }
}
```

The interviewer's prompt: "Application hangs at startup, only sometimes. jstack shows both classes mid-init, blocked on each other. What's the bug?"

Thinking out loud (the diagnosis): 1. "JVM holds a per-class initialization lock during `<clinit>`. Static field initializers run under that lock." 2. "`Config.<clinit>` triggers `App.bootstrap`, which touches `App.CFG`, which triggers `App.<clinit>`." 3. "If two different threads hit Config and App concurrently first, each holds one class's init lock and waits for the other. Cycle. Deadlock."

The actual bug: Circular static initialization across two classes. JVM class-init locks form a cycle.

The fix: - **Option A:** Break the cycle — initialize Config lazily (holder idiom), not at class-load time. -

Option B: Move `bootstrap` out of `init()` — pass configuration explicitly, eliminate the static back-reference. - **Option C:** Use the JVM

`-XX:+PrintClassInitialization` to detect such cycles in tests.

What NOT to say: - "Add `synchronized` to static init" — already implicitly synchronized; not the issue. - "Reorder field declarations" — doesn't fix; the cycle is structural.

Common follow-up: "What does JLS say about a thread observing a partially-initialized class?"

Scenario 34 · CompletionStage exception path skipped

Asked at: Cred · **Swiggy** · **Difficulty:** Medium-Hard · **Topic:** CompletableFuture

The code:

```
return CompletableFuture
    .supplyAsync(this::fetch)
    .thenApply(this::transform)
    .exceptionally(ex -> "fallback")
    .thenApply(s -> s.toUpperCase());
```

The interviewer's prompt: "If `transform` throws, we get the fallback. If `toUpperCase` throws after the fallback, the result is an exceptionally completed future. Where's the asymmetry?"

Thinking out loud (the diagnosis): 1. "`exceptionally` is between `transform` and `toUpperCase`. Exceptions from `fetch/transform` are caught." 2. "`toUpperCase` is AFTER `exceptionally`. If it throws (say a null), no further handler catches it."

The actual bug: `exceptionally` handles only upstream errors. Anything thrown downstream of it isn't handled.

The fix: - **Option A:** Move `exceptionally` to the end — handles all upstream errors. - **Option B:** Use `handle((res, ex) -> ...)` at the end — handles both result and exception, can return a new value either way. - **Option C:** Add multiple `exceptionally` stages if you have different fallback semantics for different stages.

What NOT to say: - "Wrap each stage in try-catch" — uglier than `handle`. - "`exceptionally` catches all exceptions" — only those upstream of itself in the chain.

Common follow-up: "What's the difference between `handle` and `whenComplete` when both can observe an exception?"

Scenario 35 · Synchronized block on `String.intern()` lock

Asked at: Cred · **Difficulty:** Medium · **Topic:** Lock object choice

The code:

```
public void debit(String accountId, long amt) {
    synchronized (accountId.intern()) {
        repo.debit(accountId, amt);
    }
}
```

The interviewer's prompt: "This was meant to serialize per-account debits across the JVM. It works mostly, but we see unrelated paths in other classes blocking on the same lock. Why?"

Thinking out loud (the diagnosis): 1. "`String.intern()` returns the canonical instance from the string pool — global JVM singleton." 2. "Any code in any class that does `synchronized(otherAccountId.intern())` with the same value contends on the same monitor." 3. "Worse: ANY code that calls `synchronized(someString.intern())` for unrelated strings could collide if values happen to match." 4. "And: `intern()` can leak into the permgen/metaspaces under high cardinality."

The actual bug: Cross-class lock contention via the JVM-wide intern pool. Plus potential memory pressure.

The fix: - **Option A:** A `ConcurrentMap<String, Object>` of per-account locks: `locks.computeIfAbsent(accountId, k -> new Object())`. Caller controls lifecycle. Add eviction if needed. - **Option B:** `Striped<Lock>` from Guava — fixed pool of locks, hashed by key. Bounded memory. - **Option C:** Push to the DB — `SELECT FOR UPDATE` or optimistic lock.

What NOT to say: - "Just use `intern()` — Java pools strings anyway" — that's the whole problem. - "Synchronize the whole method" — kills per-account concurrency.

Common follow-up: "What's the bounded analog — `Striped` — and how does it differ from per-key locks?"

Scenario 36 · ReadWriteLock writer starvation

Asked at: Cred · senior · **Difficulty:** Medium-Hard · **Topic:** ReadWriteLock

The code:

```
private final ReentrantReadWriteLock rw = new ReentrantReadWriteLock();

public T read() {
    rw.readLock().lock();
    try { return cur; } finally { rw.readLock().unlock(); }
}

public void write(T v) {
    rw.writeLock().lock();
    try { cur = v; } finally { rw.writeLock().unlock(); }
}
```

The interviewer's prompt: "Read traffic dominates 1000:1. Occasional writes sometimes wait 10+ seconds. Why?"

Thinking out loud (the diagnosis): 1. "Default `ReentrantReadWriteLock` is non-fair. Readers can barge in front of waiting writers if any other reader holds the lock." 2. "With a constant trickle of readers, a writer can wait forever — never gets an empty window."

The actual bug: Writer starvation under continuous reader load with non-fair `RWLock`.

The fix: - **Option A:** `new ReentrantReadWriteLock(true)` — fair mode. Writer takes precedence on its queue position. - **Option B:** `StampedLock` with optimistic reads — readers don't block writers at all in the optimistic path; validate at the end. - **Option C:** If reads dominate so heavily, the immutable-snapshot pattern (volatile reference to immutable object) outperforms `RWLock` entirely.

What NOT to say: - "Switch to `synchronized`" — single-mutual exclusion, much worse for reads. - "`RWLock` is broken" — non-fair is by design; just need fair mode.

Common follow-up: "Why does fair `RWLock` cost more in throughput than non-fair?"

Scenario 37 · `synchronized` inside a virtual thread holds the carrier

Asked at: Cred · 2025+ · **Difficulty:** Hard · **Topic:** Virtual thread pinning

The code:

```

ExecutorService vt = Executors.newVirtualThreadPerTaskExecutor();

void process(List<Long> ids) {
    var futures = ids.stream().map(id -> vt.submit(() -> {
        synchronized (sharedMonitor) {
            jdbcCall(id);
        }
    })).toList();
    futures.forEach(f -> { try { f.get(); } catch (Exception e) {} });
}

```

The interviewer's prompt: "We expected near-unlimited concurrency from virtual threads. JDBC throughput is no better than 10 concurrent calls. Why?"

Thinking out loud (the diagnosis): 1. "Each task wraps the JDBC call in `synchronized(sharedMonitor)` ." 2. "Inside synchronized, a vthread that blocks (JDBC socket read) PINS its carrier. Carrier OS thread can't be reused." 3. "With N carriers (= cores), only N vthreads can be pinned-and-blocked at once. Everything else queues up. Concurrency collapses to N."

The actual bug: Pinning under `synchronized` + blocking IO. Cancels the virtual-thread advantage. Plus the synchronized itself serializes all calls — even worse.

The fix: - **Option A:** Drop the `synchronized` — if the JDBC driver is thread-safe, no monitor needed. - **Option B:** If serialization is required: `ReentrantLock` instead of `synchronized` — vthreads release their carrier when blocked on a lock or when blocking inside it. - **Option C:** Connection pool with N connections — natural concurrency cap without pinning.

What NOT to say: - "Increase carrier threads" — band-aid; doesn't fix pinning class. - "Use platform threads" — defeats the migration.

Common follow-up: "Why is `ReentrantLock` pinning-free while `synchronized` is not (pre-JDK 24)?"

Scenario 38 · removeAll on a synchronized list iterator

Asked at: enterprise · **Difficulty:** Medium · **Topic:** ConcurrentModificationException

The code:

```
List<Order> orders = Collections.synchronizedList(new ArrayList<>());

public void purge(Predicate<Order> p) {
    for (Order o : orders) {
        if (p.test(o)) orders.remove(o);
    }
}
```

The interviewer's prompt: "Throws ConcurrentModificationException intermittently — even when called from one thread. What's happening?"

Thinking out loud (the diagnosis): 1. "Collections.synchronizedList synchronizes individual method calls. Iteration with for-each does NOT lock around the whole loop." 2. "Even single-threaded: removing during iteration modifies the list mid-iter — modCount changes — next iterator.next() throws CME." 3. "Multi-threaded: another thread modifying via remove/add during iteration — same CME via the fail-fast iterator."

The actual bug: Fail-fast iterator protests when modCount diverges from expectedModCount. Plus synchronizedList doesn't help compound iter+modify.

The fix: - **Option A:** Use the iterator's `remove()` : `java synchronized (orders) { Iterator<Order> it = orders.iterator(); while (it.hasNext()) if (p.test(it.next())) it.remove(); }` - **Option B:** `orders.removeIf(p)` — atomic on synchronizedList; documented as safe. - **Option C:** `CopyOnWriteArrayList` — iterator is a snapshot, modifications don't throw CME; trade-off is write cost.

What NOT to say: - "Use synchronizedList; it handles iteration" — it doesn't. - "Catch CME and retry" — papering over the bug.

Common follow-up: "How does fail-fast differ from fail-safe iterator semantics?"

Scenario 39 · CompletableFuture chained on common pool starves itself

Asked at: Cred · **Difficulty:** Hard · **Topic:** Pool starvation

The code:

```
public CompletableFuture<List<Order>> all() {
    List<CompletableFuture<Order>> per = ids.stream()
        .map(id -> CompletableFuture.supplyAsync(() -> fetch(id)))
        .toList();
    return CompletableFuture.allOf(per.toArray(new CompletableFuture[0]))
        .thenApply(v -> per.stream().map(CompletableFuture::join).toList());
}
```

The interviewer's prompt: "Works for small lists. With 1000 ids and `fetch` calling another async future internally, we hit deadlock. Why?"

Thinking out loud (the diagnosis): 1. "All tasks run on the common pool — `availableProcessors - 1` threads." 2. "Each task calls `fetch(id)`, which internally submits another future to the same pool and blocks on `.join()`." 3. "All pool threads are occupied by outer tasks waiting on inner tasks that can't be scheduled — classic pool-thread starvation deadlock."

The actual bug: Re-entrant blocking on a fixed thread pool. Caller threads can't make progress because workers are pending.

The fix: - **Option A:** Separate executors — outer level uses one, inner uses another. No back-pressure within one pool. - **Option B:** Non-blocking composition — `thenCompose` instead of `.join()` inside the lambda. No blocking, pool freed during wait. - **Option C:** Virtual thread executor — unbounded concurrency, no starvation since vthreads unmount on join.

What NOT to say: - "Increase common pool size to 100" — moves the threshold; same class of bug for 100+ tasks. - "Add timeouts" — degrades to timeout, not throughput.

Common follow-up: "How does `thenCompose` avoid blocking while `.join` inside a lambda does not?"

Scenario 40 · `synchronized(this)` leaked to external code

Asked at: enterprise · **code-review** · **Difficulty:** Medium · **Topic:** Lock encapsulation

The code:


```
public class Counter {
    private int count;
    public synchronized void inc() { count++; }
    public synchronized int get() { return count; }
}

// Elsewhere
Counter c = new Counter();
synchronized (c) {
    Thread.sleep(60_000);
}
```

The interviewer's prompt: "All calls to Counter freeze for a minute. Why?"

Thinking out loud (the diagnosis): 1. "Counter's methods synchronize on `this`. The lock is the Counter instance." 2. "External code can `synchronized(c)` on the same object — holds the monitor." 3. "Inside the sleep, any call to `c.inc()` blocks on the very same monitor."

The actual bug: Locking on `this` exposes the lock to external code that can hold it indefinitely.

The fix: - **Option A:** Private lock object inside Counter: `java private final Object lock = new Object(); public void inc() { synchronized (lock) { count++; } }` - **Option B:** Use `AtomicInteger` — no monitor at all.

What NOT to say: - "External code shouldn't synchronize on someone else's object" — true but unenforceable. - "Mark the class final" — orthogonal; doesn't hide the monitor.

Common follow-up: "What's the same hazard with locking on a Class object via `synchronized(MyClass.class)`?"

Scenario 41 · Reading a `long` without atomicity guarantee on 32-bit

Asked at: legacy enterprise · **Difficulty:** Medium · **Topic:** Word-tearing

The code:

```
private long lastEventNanos;

public void record() {
    lastEventNanos = System.nanoTime();
}

public long readLast() {
    return lastEventNanos;
}
```

The interviewer's prompt: "On a 32-bit JVM we sometimes read garbage values from `lastEventNanos`. Why might that be?"

Thinking out loud (the diagnosis): 1. "On 32-bit JVMs, JLS does not require non-volatile long/double writes to be atomic — word-tearing is allowed." 2. "Reader can observe the high 32 bits of the old value and the low 32 bits of the new value — values that never existed."

The actual bug: Word-tearing on non-volatile 64-bit primitives. Spec-permitted on 32-bit; rare but real.

The fix: - **Option A:** `volatile long lastEventNanos;` — JLS guarantees atomicity for volatile longs. - **Option B:** `AtomicLong` — explicit, plus CAS support. - **Option C:** Move to a 64-bit JVM — most modern JVMs treat all reads/writes atomically anyway, but the spec lets 32-bit ones not.

What NOT to say: - "longs are always atomic on JVM" — only volatile ones per JLS; modern 64-bit JVMs do this in practice but spec is the contract. - "Use synchronized" — works but heavier than volatile.

Common follow-up: "What's the JLS rule about volatile longs versus plain longs?"

Scenario 42 · Static initializer thread is interrupted

Asked at: senior · **Difficulty:** Hard · **Topic:** Initialization

The code:

```
public class Plugins {
    static final List<Plugin> ALL = load();

    private static List<Plugin> load() {
        try { return remoteRegistry.fetch(); }
        catch (InterruptedException ie) { Thread.currentThread().interrupt(); return List.of(); }
    }
}
```

The interviewer's prompt: "Plugin loading sometimes returns an empty list. Plugin team swears they're available. Why?"

Thinking out loud (the diagnosis): 1. "If the class-loading thread is interrupted while inside `load()`, we catch and return empty." 2. "Worse: we re-interrupt the current thread. Any subsequent blocking call in that thread now throws immediately or behaves weirdly." 3. "Static init runs once. If the first attempt fails to interrupt, ALL is forever empty — no retry."

The actual bug: Swallowing interrupt with empty fallback during static init. The class is now permanently broken with empty plugins.

The fix: - **Option A:** Don't initialize via blocking call in static init — make `load` lazy and idempotent on retry. - **Option B:** If you must: throw `ExceptionInInitializerError` on failure — the class doesn't initialize at all; the JVM throws `NoClassDefFoundError` on next access, allowing the app to recover/reattempt at the higher layer. - **Option C:** Use the holder idiom + lazy retry: each call to `Plugins.all()` retries until success.

What NOT to say: - "Catch and return empty" — exactly the present bug. - "Ignore interrupt" — interrupt is a directive; ignoring it breaks shutdown semantics.

Common follow-up: "What's the semantic contract of `Thread.currentThread().interrupt()` after catching `InterruptedException`?"

Scenario 43 · Memoization map grows unbounded

Asked at: Razorpay · **Difficulty:** Easy-Medium · **Topic:** Concurrent cache misuse

The code:

```
private final ConcurrentMap<String, BigDecimal> cache = new ConcurrentHashMap<>();

public BigDecimal compute(String key) {
    return cache.computeIfAbsent(key, k -> expensive(k));
}
```

The interviewer's prompt: "After 12 hours, heap grows steadily until OOM. The compute function returns correct results. What's the leak?"

Thinking out loud (the diagnosis): 1. "ConcurrentHashMap never evicts. Every distinct key lives forever." 2. "If keys are unbounded (user IDs, session IDs, query strings), the cache grows without bound."

The actual bug: Unbounded cache. CHM isn't an eviction-managed cache.

The fix: - **Option A:** Guava `Cache` or Caffeine — explicit size/time eviction. - **Option B:** `LinkedHashMap` with `removeEldestEntry` in a synchronized wrapper — simple LRU. - **Option C:** `WeakHashMap` if keys can be evicted when callers drop them — but note `WeakHashMap` is not thread-safe; wrap.

What NOT to say: - "`ConcurrentHashMap` evicts old entries" — it doesn't. - "`Use System.gc()`" — GC can't collect entries the map references.

Common follow-up: "Why is Caffeine's segmented design faster than a `synchronized LinkedHashMap` LRU?"

Scenario 44 · Phaser deregister forgotten

Asked at: senior · **Difficulty:** Medium · **Topic:** Phaser

The code:

```
Phaser phaser = new Phaser(1);

public void runPhase(Runnable r) {
    phaser.register();
    new Thread(() -> { r.run(); phaser.arrive(); }).start();
}

public void waitAll() {
    phaser.arriveAndAwaitAdvance();
}
```

The interviewer's prompt: "`waitAll()` hangs forever after several phases. Each task definitely completes — we see logs. Why?"

Thinking out loud (the diagnosis): 1. "`arrive()` only signals arrival; the party stays registered. Each new `runPhase` adds another party that never deregisters." 2. "After N phases, phaser expects N+1 parties to arrive (including the original 1). Tasks from prior phases haven't deregistered. New phases require all parties — old parties never re-arrive."

The actual bug: `arrive` increments arrival count but doesn't reduce registered parties. Tasks should use `arriveAndDeregister` if they're one-shot.

The fix: - **Option A:** `phaser.arriveAndDeregister()` from one-shot task threads. - **Option B:** Re-register at the start of each phase if tasks span phases. - **Option C:** For one-shot collective tasks, `CountDownLatch` is simpler.

What NOT to say: - "Phaser deregisters on thread exit" — it doesn't. - "Use `awaitAdvance` instead" — orthogonal.

Common follow-up: "When is `Phaser` actually the right tool over `CountDownLatch` or `CyclicBarrier`?"

Scenario 45 · Worker shutdown drops in-flight tasks

Asked at: Cred · **Difficulty:** Medium · **Topic:** `ExecutorService` shutdown

The code:

```
public void stop() {  
    pool.shutdownNow();  
}
```

The interviewer's prompt: "On `stop()`, in-flight payments sometimes succeed at the gateway but never get recorded locally. Diagnose."

Thinking out loud (the diagnosis): 1. "`shutdownNow()` interrupts running tasks and returns unstarted ones." 2. "If a task is mid-payment (gateway call sent, response pending), interrupt may cause `IOException` in the response handler — payment succeeded remotely but the local handler aborts before persisting."

The actual bug: Forceful shutdown of tasks with side effects spanning interrupt boundaries. Half-done work.

The fix: - **Option A:** `shutdown()` + `awaitTermination(timeout)` — graceful drain. - **Option B:** Tasks should be idempotent / use compensating logic so interrupt mid-flight is recoverable. - **Option C:** Use a separate "drain" state — stop accepting new tasks (`shutdown`), wait, only then `shutdownNow` .

What NOT to say: - "shutdownNow is fine; tasks should handle interrupt" — they do, but the issue is the side effect that already left the JVM. - "Just retry after restart" — gateway may charge again.

Common follow-up: "What's the right interrupt-response pattern for IO that has already produced a side effect?"

Scenario 46 · ThreadLocal with InheritableThreadLocal + thread pool**Asked at:** Cred · **Difficulty:** Hard · **Topic:** ThreadLocal leak**The code:**

```
static final InheritableThreadLocal<String> tenant = new InheritableThreadLocal<>();

class TenantPropagatingExecutor implements Executor {
    private final ExecutorService inner = Executors.newFixedThreadPool(8);
    public void execute(Runnable r) {
        String captured = tenant.get();
        inner.submit(() -> { tenant.set(captured); r.run(); });
    }
}
```

The interviewer's prompt: "We use `InheritableThreadLocal` so background tasks see the right tenant. They mostly do. Then tenants leak. Why?"

Thinking out loud (the diagnosis): 1. "`InheritableThreadLocal` is copied at child thread CREATION, not at task submission. With a pool, children are created once, with whatever tenant was set on the creator at pool startup — usually empty or wrong." 2. "The wrapper here sets `tenant.set(captured)` but never `remove()`. Worker now retains last seen tenant across tasks."

The actual bug: Two bugs — `InheritableThreadLocal` doesn't help with reused pool threads, and the explicit propagation leaks across tasks.

The fix: - **Option A:** Explicit propagation with cleanup: `java inner.submit(() -> { tenant.set(captured); try { r.run(); } finally { tenant.remove(); } });` - **Option B:** JDK 21 `ScopedValue` — `ScopedValue.where(tenant, captured).run(r)`. Auto-scoped, no remove needed.

What NOT to say: - "Switch to `ThreadLocal`" — won't propagate at all into pool workers. - "Spring's `RequestContext` handles this" — only if you wire its propagator correctly.

Common follow-up: "What problem does Java 21 `ScopedValue` solve that `ThreadLocal` doesn't?"

Scenario 47 · LongAdder.sum read during writes**Asked at:** Cred · **Difficulty:** Medium · **Topic:** LongAdder**The code:**

```
LongAdder requests = new LongAdder();

public void onRequest() { requests.increment(); }

public boolean overBudget() {
    return requests.sum() > 1_000_000;
}
```

The interviewer's prompt: "We check `overBudget` to throttle. Under load, we sometimes throttle far before reaching a million; other times far after. Diagnose."

Thinking out loud (the diagnosis): 1. "`LongAdder.sum` is a snapshot over striped cells — NOT atomic. Concurrent increments during the sum can be counted or missed." 2. "For a budget check, the value is only approximate. Not precise like `AtomicLong.get()`."

The actual bug: `LongAdder` gives high write throughput by trading away exact-at-read semantics.

The fix: - **Option A:** Accept the approximation; budget control rarely needs millisecond accuracy. -

Option B: `AtomicLong` if exact reads are required. - **Option C:** `LongAdder.sumThenReset()` for periodic measure-and-reset semantics, slightly stronger.

What NOT to say: - "LongAdder is broken" — works as documented. - "Synchronize the sum" — defeats the point of `LongAdder`.

Common follow-up: "In what range of contention does `LongAdder.sum()` deviate most from a hypothetical exact value?"

Scenario 48 · Sleep inside synchronized triggers everyone

Asked at: TCS · Razorpay · interview classic · **Difficulty:** Easy-Medium · **Topic:** Lock hold time

The code:

```
public synchronized void process(Order o) {
    Thread.sleep(5_000); // wait for downstream rate limit
    callDownstream(o);
}
```

The interviewer's prompt: "This serializes orders to comply with a 1-call-per-5-sec downstream limit. Throughput is 1 order per 5 seconds even though we have 8 cores. Why?"

Thinking out loud (the diagnosis): 1. " `synchronized` on the method means one thread at a time. Five-second sleep inside the lock means each call holds the lock 5+ seconds." 2. "Throughput is bounded by the longest critical section under contention. 8 cores don't help."

The actual bug: Holding a lock across a long blocking operation. Defeats parallelism.

The fix: - **Option A:** `Semaphore` for rate limiting — release/acquire don't serialize the actual work. - **Option B:** Guava `RateLimiter` — token-bucket rate limit, non-blocking. - **Option C:** Use a real downstream rate-limit handshake (per-second window with permits).

What NOT to say: - "Make sleep shorter" — doesn't help; lock still held. - "Sleep outside synchronized" — closer; but rate limiting is the actual requirement.

Common follow-up: "What's the difference between a semaphore and a lock for rate limiting?"

Scenario 49 · `Vector` operations don't compose

Asked at: legacy enterprise · **Difficulty:** Easy · **Topic:** Compound atomicity

The code:

```
Vector<String> v = new Vector<>();

public void addUnique(String x) {
    if (!v.contains(x)) {
        v.add(x);
    }
}
```

The interviewer's prompt: "Vector is thread-safe. Yet duplicates show up. Why?"

Thinking out loud (the diagnosis): 1. "Vector's methods are individually synchronized. Compound `contains-then-add` is not atomic." 2. "Two threads pass `contains` simultaneously, both add."

The actual bug: Method-level synchronization doesn't compose. Same class as the `ConcurrentHashMap` check-then-act bug.

The fix: - **Option A:** Don't use Vector — historical baggage. Use a Set: `new CopyOnWriteArraySet<>()` or `ConcurrentHashMap.newKeySet()`. - **Option B:** Synchronize externally over the compound: `java synchronized (v) { if (!v.contains(x)) v.add(x); }`

What NOT to say: - "Vector is deprecated" — not officially, but effectively yes; use modern collections. - "Use synchronizedList" — same compound-atomicity issue.

Common follow-up: "Why is `ConcurrentHashMap.newKeySet()` a Set view, not a real Set class?"

Scenario 50 · `synchronized` + `Thread.interrupt` deadlock recovery

Asked at: Cred · senior · **Difficulty:** Hard · **Topic:** Interrupt handling

The code:

```
public synchronized void run() {
    while (!shouldStop) {
        process();
    }
}
```

The interviewer's prompt: "Worker hangs forever after `shutdown()`. We set `shouldStop` and interrupt. Synchronized block won't respond. Why?"

Thinking out loud (the diagnosis): 1. "`synchronized` is NOT interruptible. A thread blocked on entering `synchronized` ignores interrupt." 2. "Even if the holder eventually exits, an interrupt to the blocked thread is just stored in its interrupt flag — doesn't break the wait."

The actual bug: Synchronized blocks don't honor `Thread.interrupt()`. For long-held locks, blocked waiters can't be cancelled.

The fix: - **Option A:** `ReentrantLock` with `lockInterruptibly()` — throws `InterruptedException` if interrupted while waiting. - **Option B:** Volatile `shouldStop` checked inside the loop frequently; ensure the loop body doesn't itself hold synchronized for long.

What NOT to say: - "Force-kill the thread with `Thread.stop`" — deprecated and unsafe; corrupts state. - "Interrupt should work" — JLS says it doesn't for synchronized acquisition.

Common follow-up: "What's the design trade-off in making synchronized non-interruptible?"

Scenario 51 · Listener leak via missed unregister

Asked at: Razorpay · enterprise · **Difficulty:** Medium · **Topic:** Listener lifecycle

The code:

```
public class EventBus {
    private final List<Listener> all = new CopyOnWriteArrayList<>();
    public void register(Listener l) { all.add(l); }
    public void fire(Event e) { all.forEach(l -> l.handle(e)); }
}

public class TransientView {
    public TransientView(EventBus bus) {
        bus.register(e -> updateUI(e));
    }
}
```

The interviewer's prompt: "TransientView is supposed to be ephemeral — appears, dismisses, GC'd. Heap shows thousands of them retained. Why?"

Thinking out loud (the diagnosis): 1. "`bus.register(...)` captures a lambda that closes over `this` of TransientView." 2. "EventBus.all holds a strong reference to the lambda — and through it to the TransientView." 3. "No unregister call. The bus retains them all forever."

The actual bug: Listener lifecycle leak. Common in long-lived event buses + short-lived subscribers.

The fix: - **Option A:** Return a Subscription handle from register, with `close()` for unregister. Use try-with-resources or explicit lifecycle. - **Option B:** Weak references inside the bus — `WeakHashMap` -style listener tracking. Trade-off: lambdas with no other strong ref get collected silently.

What NOT to say: - "GC should collect the views — they're transient" — it can't; the bus holds strong refs. - "Use System.gc()" — won't free strongly reachable objects.

Common follow-up: "What's the trade-off between strong-reference listener registries and weak-reference ones?"

Scenario 52 · ConcurrentLinkedQueue.size() is O(n)

Asked at: senior systems · **Difficulty:** Medium · **Topic:** Hidden cost

The code:

```
private final Queue<Tick> q = new ConcurrentLinkedQueue<>();

public void emit(Tick t) {
    q.offer(t);
    if (q.size() > THRESHOLD) drop();
}
```

The interviewer's prompt: "emit() latency grows linearly with queue size, then collapses to constant when we drop. Why the linear growth?"

Thinking out loud (the diagnosis): 1. " `ConcurrentLinkedQueue.size()` traverses the list to count nodes — $O(n)$ by spec." 2. "Every emit pays $O(\text{current size})$. As the queue grows, emit gets slower until drop catches it."

The actual bug: Using `size()` on a non-counting concurrent queue as if it were $O(1)$.

The fix: - **Option A:** Keep a separate `AtomicInteger` counter; increment on offer, decrement on poll. - **Option B:** Use `LinkedBlockingQueue` — its `size()` is $O(1)$ (counted internally). - **Option C:** Use a bounded `ArrayBlockingQueue` — natural cap, no size check needed.

What NOT to say: - "Queue.size is always $O(1)$ " — depends on the implementation. - "Just don't use size()" — possible, but the check is the requirement.

Common follow-up: "Why is CLQ's size $O(n)$ — what would $O(1)$ cost?"

Scenario 53 · synchronized method returning mutable field

Asked at: enterprise · **Difficulty:** Easy-Medium · **Topic:** Encapsulation

The code:

```
private final Map<String, String> conf = new HashMap<>();

public synchronized Map<String, String> getConf() {
    return conf;
}

public synchronized void put(String k, String v) {
    conf.put(k, v);
}
```

The interviewer's prompt: "Methods are synchronized. Yet readers see partial state and we sometimes get `ConcurrentModificationException`. Why?"

Thinking out loud (the diagnosis): 1. " `getConf()` returns the actual internal map. Caller now iterates/ mutates outside the lock." 2. " `put` synchronizes; reader iterates without — fail-fast iterator triggers." 3. "Or worse: caller mutates through the returned reference."

The actual bug: Synchronized API surface but mutable internal state leaked to callers. Callers can break invariants.

The fix: - **Option A:** Return an unmodifiable snapshot: `java return Map.copyOf(conf);` - **Option B:** Use `ConcurrentHashMap` internally; safe to expose for read (but still document that mutation isn't allowed).

What NOT to say: - "Document that callers shouldn't iterate" — unenforceable. - "Mark the field final" — orthogonal.

Common follow-up: "When is `unmodifiableMap` a snapshot vs a view?"

Scenario 54 · Compute lambda mutates shared state

Asked at: Cred · **Difficulty:** Hard · **Topic:** CHM compute

The code:

```
ConcurrentHashMap<String, Counter> map = new ConcurrentHashMap<>();
private long globalTotal;

public void inc(String k) {
    map.compute(k, (key, cur) -> {
        if (cur == null) cur = new Counter();
        cur.value++;
        globalTotal++;
        return cur;
    });
}
```

The interviewer's prompt: "globalTotal sometimes doesn't match the sum of map values. Why?"

Thinking out loud (the diagnosis): 1. "`compute` lambda runs under a per-bin lock — atomic per key. But `globalTotal++` touches a field outside that scope." 2. "Two threads computing different keys hold different bin locks. Both touch `globalTotal` without synchronization." 3. "Worse: `compute`'s lambda can be re-invoked internally during resize."

The actual bug: Lambda has side effects outside the key's scope. Documentation explicitly warns against this.

The fix: - **Option A:** Move `globalTotal++` outside compute, using `AtomicLong`. Don't mutate non-local state inside compute. - **Option B:** Derive globalTotal lazily: `map.values().stream().mapToLong(c -> c.value).sum()`.

What NOT to say: - "Synchronize globalTotal++" — works but the lambda's purity matters semantically too. - "Use volatile globalTotal" — visibility ok, still not atomic.

Common follow-up: "Why does CHM's documentation insist the function be 'short and simple'?"

Scenario 55 · Object.wait without owning the monitor

Asked at: TCS · interview classic · **Difficulty:** Easy · **Topic:** wait/notify

The code:

```
public class Holder {
    private boolean ready;

    public void waitForReady() throws InterruptedException {
        while (!ready) wait();
    }
}
```

The interviewer's prompt: "Throws IllegalMonitorStateException on the first call. Why?"

Thinking out loud (the diagnosis): 1. "wait()" must be called from a synchronized context — caller must hold the monitor of the object on which wait is invoked." 2. "No synchronized block here. Wait fails immediately."

The actual bug: Missing synchronized. wait/notify always require holding the monitor.

The fix: - **Option A:** `java public synchronized void waitForReady() throws InterruptedException { while (!ready) wait(); }` - **Option B:** Use `CountDownLatch(1)` — `latch.await()` doesn't need a monitor.

What NOT to say: - "Wait should be allowed anywhere" — by design it requires the lock. - "Add `volatile` to ready" — wait still throws.

Common follow-up: "Why does Object.wait require the monitor?"

Scenario 56 · Re-entrant synchronized re-acquires but masks bug

Asked at: senior · **Difficulty:** Medium · **Topic:** Reentrance hazard

The code:

```
public synchronized void a() {
    b();
}

public synchronized void b() {
    state++;
    if (state > 1) throw new IllegalStateException();
}
```

The interviewer's prompt: "From outside, calling `a()` can lead to `IllegalStateException` even when no other thread calls. Why does the `IllegalStateException` happen single-threaded?"

Thinking out loud (the diagnosis): 1. "Both methods are synchronized — same monitor (`this`). Java's synchronized is reentrant — same thread enters both." 2. "`a()` calls `b()` — both run; `state++` happens once. Not the bug yet." 3. "But: any external caller calling `b()` directly while `a()` is in flight on the same thread... no, single thread can only be in one method at a time." 4. "Bug: if `b()` is also called externally, state ends up >1 . But that's not single-threaded."

Actually re-read this — the bug is more subtle: if `a()` is called and increments via `b()`, then later `b()` is called directly from outside, state increases past 1 in normal usage. The setup is the latent class invariant — reentrance lets methods compose surprisingly.

The actual bug: Reentrance allows `b` to be called within `a` and externally. Designer assumed `state` would never reach 2 because of synchronization, but synchronization doesn't gate the number of invocations — only concurrency. The invariant was wrong about what synchronized guarantees.

The fix: - **Option A:** Decouple invariants from synchronization. Maintain state correctness regardless of who calls what. - **Option B:** If reentrance is the problem (i.e. design wanted a non-reentrant lock semantically), use `Lock.tryLock` with manual reentrance tracking, or restructure so internal calls don't re-enter the public surface.

What NOT to say: - "synchronized prevents recursion" — it doesn't; it's reentrant. - "Make state volatile" — doesn't help.

Common follow-up: "What would happen if `synchronized` weren't reentrant in Java?"

Scenario 57 · `ScheduledFuture` not held, gets cancelled

Asked at: Cred · **Difficulty:** Hard · **Topic:** `ScheduledExecutorService`

The code:

```
public void scheduleHeartbeat() {
    new ScheduledThreadPoolExecutor(1).schedule(this::ping, 30, TimeUnit.SECONDS);
}
```

The interviewer's prompt: "Heartbeat fires once, sometimes twice, then stops. Diagnose."

Thinking out loud (the diagnosis): 1. "We construct a new STPE per call, return nothing, hold no reference. The executor becomes unreachable." 2. "STPE's `keepAliveTime` defaults to 0; `allowCoreThreadTimeOut` may apply. Worker thread is non-daemon by default, holding the executor alive temporarily." 3. "At GC, the executor may be finalized — but more critically, if `allowCoreThreadTimeOut(true)` is set or if the worker exits, the scheduled task may stop firing."

The actual bug: Creating an executor per call leaks them and risks losing scheduled tasks when refs are dropped. Plus, daemon settings on STPE matter for JVM shutdown behavior.

The fix: - **Option A:** Single shared executor as a field. Set thread factory to daemon-or-not depending on lifetime. - **Option B:** `Executors.newSingleThreadScheduledExecutor(daemonFactory)` once at app init.

What NOT to say: - "STPE persists scheduled tasks across instances" — it doesn't. - "Daemon threads are always fine" — daemon dies on JVM exit; non-daemon prevents exit.

Common follow-up: "What's the impact of `allowCoreThreadTimeOut(true)` on a STPE?"

Scenario 58 · synchronized + I/O = throughput collapse

Asked at: enterprise · **Difficulty:** Easy-Medium · **Topic:** Lock hold time

The code:

```
public synchronized void save(Order o) {
    repo.save(o); // JDBC call, 50-200ms
}
```

The interviewer's prompt: "All order saves serialize. 200ms latency × 1000 orders = 200 seconds. Why so slow?"

Thinking out loud (the diagnosis): 1. "Method-level synchronization serializes all callers." 2. "JDBC save runs inside the lock. 200ms × N orders = N × 200ms total."

The actual bug: Lock held across slow IO, with no real reason — repo is presumably thread-safe.

The fix: - **Option A:** Remove `synchronized` entirely if repo is thread-safe. - **Option B:** Per-order or per-account synchronized — finer-grained.

What NOT to say: - "JDBC needs synchronized" — modern JDBC + connection pool is thread-safe per connection. - "Add more threads" — they all queue on the monitor.

Common follow-up: "What's the rule of thumb for what code goes inside a lock?"

Scenario 59 · Volatile array vs volatile element

Asked at: senior · **Difficulty:** Hard · **Topic:** Visibility nuance

The code:

```
private volatile int[] data = new int[1024];

public void update(int i, int v) {
    data[i] = v;
}

public int read(int i) {
    return data[i];
}
```

The interviewer's prompt: "We marked data volatile expecting per-element visibility. Reads sometimes return stale values. Why?"

Thinking out loud (the diagnosis): 1. "Volatile applies to the reference, not to elements. Writing `data[i] = v` is a plain write to the i-th slot of the array." 2. "Other threads have no happens-before edge to that write — may read stale."

The actual bug: Volatile on the array reference doesn't make element accesses volatile.

The fix: - **Option A:** `AtomicIntegerArray` — per-element atomic ops with visibility. - **Option B:** `VarHandle` for arrays with `setVolatile` / `getVolatile` per element. - **Option C:** Synchronize per region — coarser.

What NOT to say: - "volatile on array gives element-level visibility" — wrong, common confusion. - "Just use `synchronized` on every read/write" — slow.

Common follow-up: "What's the relationship between volatile fields and VarHandle's modes?"

Scenario 60 · ForkJoinPool deadlock from `join` ordering**Asked at:** senior · **Difficulty:** Hard · **Topic:** Fork/Join**The code:**

```
class Sum extends RecursiveTask<Long> {
    private final int[] a; private final int lo, hi;
    Sum(int[] a, int lo, int hi) { this.a=a; this.lo=lo; this.hi=hi; }

    protected Long compute() {
        if (hi - lo < 100) {
            long s = 0; for (int i=lo;i<hi;i++) s += a[i]; return s;
        }
        int mid = (lo+hi)/2;
        Sum left = new Sum(a, lo, mid);
        Sum right = new Sum(a, mid, hi);
        left.fork();
        right.fork();
        return left.join() + right.join();
    }
}
```

The interviewer's prompt: "This works but is slower than expected and occasionally stalls. What's the small optimization that fixes both?"

Thinking out loud (the diagnosis): 1. "Forking both then joining both leaves no work for the current thread. Worker is idle during joins; relies entirely on stealing." 2. "Better pattern: fork one, compute the other on this thread, then join the forked one. Current worker doesn't block during the local compute."

The actual bug: Suboptimal fork/join pattern. Not strictly a correctness bug but a real performance / occasional deadlock under pool pressure.

The fix: - **Option A:** `left.fork(); long r = right.compute(); long l = left.join(); return l + r;` — current worker busy, only one stealable task. - **Option B:** Use `invokeAll(left, right)` — F/J idiom that handles the pattern.

What NOT to say: - "F/J should figure this out" — it doesn't; the user's algorithm determines the pattern.

Common follow-up: "What's the canonical 'fork one, compute the other' pattern called in the F/J docs?"

Scenario 61 · Scheduled rate triggers tasks faster than they run**Asked at:** Cred · **Difficulty:** Medium · **Topic:** ScheduledExecutorService semantics

The code:

```
scheduler.scheduleAtFixedRate(this::heavyJob, 0, 1, TimeUnit.SECONDS);
```

The interviewer's prompt: "heavyJob takes 5 seconds. At fixed-rate 1s, we expected back-to-back execution. Yet only one runs at a time and they back up. Why?"

Thinking out loud (the diagnosis): 1. " `scheduleAtFixedRate` does NOT spawn overlapping tasks. If a run overtakes the next scheduled tick, subsequent ticks queue." 2. "Single executor thread; tasks back up. Memory or queue saturation possible."

The actual bug: `scheduleAtFixedRate` is sequential per-task. Overruns accumulate.

The fix: - **Option A:** Use `scheduleWithFixedDelay` if you want fixed gap between completions, not fixed period from start times. - **Option B:** Wrap the periodic kickoff to itself submit work to a worker pool: scheduled task is light; pool handles concurrency. - **Option C:** Monitor task duration vs period; alert if drift accumulates.

What NOT to say: - "FixedRate means parallel execution" — no, it means fixed start interval, but never overlapping for one schedule. - "Increase scheduler size" — doesn't change the per-task serial behavior of a single schedule.

Common follow-up: "What's the exact contract difference between `scheduleAtFixedRate` and `scheduleWithFixedDelay`?"

Scenario 62 · Synchronized field-mutation without final reference

Asked at: Cred · **Difficulty:** Hard · **Topic:** Race on lock object

The code:

```
private Object lock = new Object();

public void reset() {
    lock = new Object();
}

public void op() {
    synchronized (lock) {
        criticalSection();
    }
}
```

The interviewer's prompt: "Calling `reset()` while ops are in flight breaks mutual exclusion — two threads enter `criticalSection`. How?"

Thinking out loud (the diagnosis): 1. "Lock reference is reassigned. Different threads can synchronize on different objects." 2. "Thread A reads `lock` (old object), enters synchronized. Thread B (after reset) reads `lock` (new object), also enters synchronized. Two different monitors — both 'inside' the critical section simultaneously."

The actual bug: Lock object reassigned. Synchronized on a moving target gives no exclusion across threads that see different values.

The fix: - **Option A:** `private final Object lock = new Object();` — final reference, never reassigned. - **Option B:** If you need to reset state, reset the state behind the lock — don't replace the lock.

What NOT to say: - "Volatile the field" — visibility OK; still two different monitors. - "Synchronize the reset method" — doesn't fix the in-flight observers.

Common follow-up: "Why is `final` the only safe modifier for a lock-holding field?"

Scenario 63 · Lazy init via DCL with non-volatile string

Asked at: enterprise · **Difficulty:** Medium · **Topic:** Lazy init

The code:

```
private String cached;

public String get() {
    if (cached == null) {
        synchronized (this) {
            if (cached == null) {
                cached = compute();
            }
        }
    }
    return cached;
}
```

The interviewer's prompt: "DCL pattern. `cached` is a `String` — immutable. Why is volatile still needed?"

Thinking out loud (the diagnosis): 1. "String is immutable, so seeing a partially-constructed String can't happen via field reads." 2. "But the FIELD assignment itself isn't safely published without volatile." 3.

"Thread A may set `cached = "foo";` while Thread B's read of `cached` sees null indefinitely — no happens-before."

The actual bug: Even immutable values need safe publication of the reference itself. The field write isn't visible without volatile.

The fix: - **Option A:** `private volatile String cached;` — minimal fix. - **Option B:** Holder idiom or `Suppliers.memoize` style.

What NOT to say: - "String is immutable, DCL is safe without volatile" — wrong; reference visibility still required.

Common follow-up: "What types of values, if any, can be safely DCL'd without volatile?"

Scenario 64 · Async exception bubbles into user thread

Asked at: Cred · **Difficulty:** Medium · **Topic:** CompletableFuture composition

The code:

```
public Quote getQuote(long id) {
    return CompletableFuture
        .supplyAsync(() -> upstream.fetch(id))
        .thenApply(this::transform)
        .join();
}
```

The interviewer's prompt: "When upstream throws SQLException, the caller sees `CompletionException` wrapping a `SQLException`. Code catching `SQLException` directly never catches. Why?"

Thinking out loud (the diagnosis): 1. "`join()` rethrows the underlying exception wrapped in `CompletionException` — an unchecked wrapper." 2. "`get()` wraps in `ExecutionException` — also a wrapper. Callers expecting the raw exception type don't catch it."

The actual bug: `CompletableFuture`'s terminal getters wrap exceptions. Original cause is in `getCause()`.

The fix: - **Option A:** Catch `CompletionException` and rethrow `e.getCause()` — preserve type at the API boundary. - **Option B:** Use `.exceptionally(...)` to normalize errors to a domain type before terminal access.

What NOT to say: - "Catch Throwable broadly" — anti-pattern, masks bugs. - "join() throws the original" — it does not.

Common follow-up: "Why does join wrap in CompletionException while get wraps in ExecutionException?"

Scenario 65 · ForEach in parallel stream mutates shared list

Asked at: Cred · **Difficulty:** Easy-Medium · **Topic:** Parallel stream

The code:

```
List<String> out = new ArrayList<>();
list.parallelStream().forEach(x -> out.add(transform(x)));
return out;
```

The interviewer's prompt: "Output list has fewer entries than input, with occasional `ArrayIndexOutOfBoundsException`. Why?"

Thinking out loud (the diagnosis): 1. "ArrayList is not thread-safe. Parallel stream's `forEach` runs on multiple FJ workers, all mutating the same ArrayList." 2. "Concurrent `add` corrupts internal array — lost writes, size out of sync, AIOOBE."

The actual bug: Parallel-stream side effect on non-thread-safe collection.

The fix: - **Option A:** Use `collect` properly: `java return list.parallelStream().map(this::transform).collect(Collectors.toList());` - **Option B:** `Collections.synchronizedList(new ArrayList<>())` — works but slow. - **Option C:** `forEach` only on truly side-effect-free operations.

What NOT to say: - "parallelStream is broken" — it's the misuse. - "Synchronize the lambda body" — works but defeats parallelism.

Common follow-up: "Why is `collect(toList())` safe under parallel streams when `forEach(list::add)` isn't?"

Scenario 66 · synchronized on Optional.get path

Asked at: enterprise · **Difficulty:** Medium · **Topic:** Race condition

The code:

```
private Optional<Config> cfg = Optional.empty();

public Config require() {
    if (cfg.isEmpty()) {
        synchronized (this) {
            if (cfg.isEmpty()) cfg = Optional.of(loadFresh());
        }
    }
    return cfg.get();
}
```

The interviewer's prompt: "Sometimes throws NoSuchElementException on `cfg.get()` even after the lazy init block. Diagnose."

Thinking out loud (the diagnosis): 1. "`cfg` is non-volatile. After the synchronized block, the calling thread releases the lock, but the field read at `return cfg.get()` is plain." 2. "A different thread that didn't enter the synchronized may see the stale `Optional.empty()` — its earlier `cfg.isEmpty()` read." 3. "And the path `return cfg.get()` outside synchronized re-reads the field. Without volatile, no happens-before."

The actual bug: Plain field, DCL-style logic without volatile. Same class as the singleton DCL bug.

The fix: - **Option A:** `private volatile Optional<Config> cfg = Optional.empty();` . - **Option B:** Holder idiom; eliminate the dance. - **Option C:** Move return inside the synchronized's outer if — but that re-introduces lock per call. Use volatile.

What NOT to say: - "Optional is immutable so visibility is fine" — wrong; field visibility is independent.

Common follow-up: "Does Optional being final help here?"

Scenario 67 · ABA on AtomicReference for cached object

Asked at: Cred · **Difficulty:** Hard · **Topic:** ABA

The code:

```
private final AtomicReference<Snapshot> snap = new AtomicReference<>();

public void rotate() {
    Snapshot old = snap.get();
    Snapshot fresh = build();
    if (snap.compareAndSet(old, fresh)) {
        recycle(old); // back to pool
    }
}
```

The interviewer's prompt: "Snapshots get corrupted under concurrent rotate() calls. Why doesn't CAS protect us?"

Thinking out loud (the diagnosis): 1. "Recycle returns the Snapshot to a pool. The same instance can be re-acquired by build() later." 2. "Thread A reads snap = X, builds Y. Thread B reads snap = X, builds different Y'. Thread B CAS succeeds (X → Y'). Thread B recycles X back to pool. build() in Thread A's NEXT rotation later gets X back. Thread A is still pending CAS on (X, Y) — fails since current is Y'. OK so far." 3. "But: if Thread A's build retrieves a recycled instance that happens to be X again, and CAS sees X-as-current... we're back to classic ABA."

The actual bug: Recycling + CAS = ABA. The reference equality used by CAS doesn't detect intervening swaps.

The fix: - **Option A:** Don't recycle Snapshot objects; let GC handle them. ABA can't happen if values are never re-used. - **Option B:** AtomicStampedReference<Snapshot> with version counter. - **Option C:** Use immutable Snapshot + sequence number embedded in the swap.

What NOT to say: - "Synchronize rotate" — kills lock-free design. - "CAS prevents all races" — only for unique reference values.

Common follow-up: "How does AtomicMarkableReference compare to AtomicStampedReference?"

Scenario 68 · Caller-runs policy hangs the calling thread

Asked at: Cred · senior · **Difficulty:** Hard · **Topic:** Rejection policy

The code:

```
ThreadPoolExecutor pool = new ThreadPoolExecutor(
    4, 4, 0, TimeUnit.MILLISECONDS,
    new ArrayBlockingQueue<>(8),
    new ThreadPoolExecutor.CallerRunsPolicy());

@GetMapping("/fast")
public String fast() {
    pool.submit(this::slowTask);
    return "queued";
}
```

The interviewer's prompt: "We picked `CallerRunsPolicy` for backpressure. Under load, `/fast` endpoint sometimes takes 30 seconds. Why?"

Thinking out loud (the diagnosis): 1. "`CallerRunsPolicy` runs the rejected task on the `SUBMITTING` thread." 2. "Submitting thread is a Tomcat HTTP worker. When the pool is saturated, the HTTP thread executes the slow task itself — blocking the request for 30s."

The actual bug: `CallerRunsPolicy` doesn't free HTTP threads; it makes them run the work themselves. Backpressure for the producer becomes latency for end users.

The fix: - **Option A:** Don't use `CallerRunsPolicy` on HTTP threads — instead reject with `429 Too Many Requests` and let the load balancer back off. - **Option B:** Use a separate `Bulkhead` /semaphore in front of the pool to limit concurrency without piggybacking on HTTP threads.

What NOT to say: - "`CallerRunsPolicy` is the safe default" — only if the caller can afford to run the task. - "Increase the queue size" — defers, doesn't fix.

Common follow-up: "What's the difference between bulkhead and backpressure?"

Scenario 69 · Final field modification via reflection

Asked at: senior · **Difficulty:** Hard · **Topic:** Immutability + safe publication

The code:


```
public final class Box {
    private final int val;
    public Box(int v) { this.val = v; }
    public int val() { return val; }
}

// Some library does:
Field f = Box.class.getDeclaredField("val");
f.setAccessible(true);
f.setInt(box, 42);
```

The interviewer's prompt: "After this reflective mutation, other threads sometimes see the old value, sometimes new — even after thread synchronization. Why?"

Thinking out loud (the diagnosis): 1. "Final fields have JMM 'freeze' semantics — guaranteed visible to readers without further synchronization, but only the value at end of constructor." 2. "Reflective mutation of final fields is allowed but undefined w.r.t. memory ordering. Readers may see either the original or the mutated value depending on JIT optimizations."

The actual bug: Mutating final fields breaks JMM guarantees. JLS explicitly says behavior is undefined for non-Object-Field-style mutators.

The fix: - **Option A:** Don't mutate final fields. Use a non-final field with explicit synchronization or volatile.
- **Option B:** If the library needs to inject — make Box's val non-final, controlled via a guarded setter.

What NOT to say: - "Reflection should always work" — JLS specifically warns about final-field mutation visibility.

Common follow-up: "What's the JLS exception that allows reflective final-field setting?"

Scenario 70 · Multiple monitors leak via Object.notify()

Asked at: senior · **Difficulty:** Hard · **Topic:** Notify

The code:

```

class Order {
    private final Object lock = new Object();
    private boolean delivered;
    public void waitForDelivery() throws InterruptedException {
        synchronized (lock) {
            while (!delivered) lock.wait();
        }
    }
    public void deliver() {
        synchronized (lock) {
            delivered = true;
            lock.notify();
        }
    }
}

// Caller waits on multiple orders
order1.waitForDelivery();
order2.waitForDelivery();

```

The interviewer's prompt: "Caller wants to wait for both orders. They wait in sequence. If order2 is delivered first, the waitForDelivery on order1 still blocks even after order2 returns. Surely both are independent? Yes — but with N waits, latency = sum, not max. Worse, sometimes order1 never delivers — yet from our logs it did. What's wrong?"

Thinking out loud (the diagnosis): 1. "Sequential waits is N-sum latency — orthogonal to correctness here." 2. "Real bug: `notify()` instead of `notifyAll()`. If somehow multiple threads waited on order1.lock (e.g. operator + auto-system), one is left waiting forever after a single delivery." 3. "Or: deliver() called BEFORE the waiter enters wait — wait happens after notify; signal lost. Spurious-wakeup-style hazard."

The actual bug: `notify()` races with the wait-set; lost wakeup possible if delivery happens between the predicate check and the wait. Plus single notify on a multi-waiter monitor is a hazard.

The fix: - **Option A:** `notifyAll()` — wakes everyone, the right one progresses. - **Option B:** Use `CompletableFuture<Void>` per order — caller composes via `allOf` for parallel waits. - **Option C:** `CountDownLatch(1)` per order — natural one-shot wait.

What NOT to say: - "notify and notifyAll are equivalent" — they aren't. - "Sequential waits is fine" — depends on whether parallel composition is needed.

Common follow-up: "Why does Java's wait() require a while-loop even with notifyAll?"

Scenario 71 · Concurrent modification via stream + side effect

Asked at: Cred · **Difficulty:** Medium · **Topic:** Stream side effects

The code:

```
private final List<Job> jobs = Collections.synchronizedList(new ArrayList<>());

public void purge() {
    jobs.stream()
        .filter(Job::isExpired)
        .forEach(jobs::remove);
}
```

The interviewer's prompt: "ConcurrentModificationException, even though we use synchronizedList. Why?"

Thinking out loud (the diagnosis): 1. " `synchronizedList` synchronizes individual ops, not iteration. Stream sources from `list.stream()` use the underlying iterator without external locking." 2. "forEach during iteration calls `jobs.remove(job)` — modifies the list mid-iter — modCount changes — CME."

The actual bug: Side-effecting modification during stream iteration. Documented contract violation.

The fix: - **Option A:** `jobs.removeIf(Job::isExpired)` — atomic on synchronizedList. - **Option B:** Collect to remove, then iterate: `java var expired = jobs.stream().filter(Job::isExpired).toList(); jobs.removeAll(expired);`

What NOT to say: - "synchronizedList handles streams" — it doesn't. - "Wrap in try-catch" — anti-pattern.

Common follow-up: "Why does `removeIf` avoid CME even on a synchronizedList?"

Scenario 72 · CompletableFuture cancellation doesn't propagate

Asked at: Cred · **Difficulty:** Hard · **Topic:** Cancellation

The code:

```
public CompletableFuture<Result> fetch() {
    return CompletableFuture.supplyAsync(() -> {
        return upstream.call(); // 30s blocking HTTP
    });
}

// Elsewhere
var f = fetch();
f.cancel(true);
```

The interviewer's prompt: "We cancel the future, expecting the upstream HTTP call to abort. It runs to completion anyway. Why?"

Thinking out loud (the diagnosis): 1. "`CompletableFuture.cancel(true)` marks the future cancelled and rejects subsequent stages." 2. "It does NOT interrupt the thread running the supplier — the worker keeps executing the lambda." 3. "The `mayInterruptIfRunning` parameter is unused for `CompletableFuture`; the lambda finishes naturally."

The actual bug: `CompletableFuture`'s `cancel` doesn't interrupt the upstream computation. The supplier completes; the result is discarded.

The fix: - **Option A:** Track the running Thread in the supplier and interrupt it explicitly on cancel: `java AtomicReference<Thread> worker = new AtomicReference<>(); var f = CompletableFuture.supplyAsync(() -> { worker.set(Thread.currentThread()); try { return upstream.call(); } finally { worker.set(null); } }); f.whenComplete((r, ex) -> { var t = worker.get(); if (ex instanceof CancellationException && t != null) t.interrupt(); });` - **Option B:** Pass a cancellation token to `upstream.call(token)` if the API supports it. - **Option C:** Use `StructuredTaskScope` in JDK 21+ — `scope.shutdown()` interrupts all forked tasks.

What NOT to say: - "`cancel(true)` interrupts the worker" — it doesn't in `CompletableFuture`. - "Use `Future.cancel`" — same limitation for arbitrary suppliers.

Common follow-up: "How does `StructuredTaskScope`'s cancellation differ?"

Scenario 73 · Re-entry into synchronized with `this` escape

Asked at: senior · **Difficulty:** Hard · **Topic:** This-escape

The code:

```
public class Loader {
    public Loader() {
        registry.add(this);
        loadInBackground();
    }

    private synchronized void loadInBackground() {
        new Thread(() -> {
            synchronized (this) {
                state = loadFromDb();
            }
        }).start();
    }
}
```

The interviewer's prompt: "Other threads pulling from `registry` sometimes see a `Loader` before its state is set. Race?"

Thinking out loud (the diagnosis): 1. "Constructor calls `registry.add(this)` — the partially-constructed `Loader` is now visible to other threads." 2. "`loadInBackground` starts a separate thread to populate state. Other threads can access the `Loader` from `registry` before that thread has run." 3. "Plus: `this` escapes during construction — JLS warns this prevents final-field freeze guarantees."

The actual bug: `this` escape during construction. Other threads see an object whose constructor hasn't finished its observable work.

The fix: - **Option A:** Don't expose `this` in the constructor. Use a static factory: `java public static Loader create() { var l = new Loader(); l.loadInBackground(); registry.add(l); return l; }` - **Option B:** Use a "ready" gate (`CountDownLatch`) that other consumers await before using a `Loader`.

What NOT to say: - "Add `volatile` to state" — doesn't fix `this` escape. - "Final fields fix this" — only if `this` doesn't escape before constructor end.

Common follow-up: "What does JLS say about this-escape and final-field freeze?"

Scenario 74 · `StructuredTaskScope` leaked across method boundary

Asked at: Cred · 2025+ · **Difficulty:** Hard · **Topic:** Structured concurrency

The code:

```
public Future<String> kickOff() {  
    var scope = new StructuredTaskScope.ShutdownOnFailure();  
    return scope.fork(this::work);  
}
```

The interviewer's prompt: "Compiles cleanly. At runtime, tasks behave oddly — sometimes complete, sometimes leak threads. What's the structural violation?"

Thinking out loud (the diagnosis): 1. "Structured concurrency: the scope must be `join` ed before being `close` d (which is itself before the method returns)." 2. "This code returns a Subtask before joining, before closing the scope. Threads created inside the scope outlive the method." 3. "Worse: the scope is GC-managed but its threads aren't auto-cancelled."

The actual bug: Scope leaked out of its try-with-resources lifetime. Structured concurrency's contract is violated.

The fix: - **Option A:** Don't expose scope state across method boundaries. Do all join/close inside: `java public String kickOff() throws Exception { try (var scope = new StructuredTaskScope.ShutdownOnFailure()) { var t = scope.fork(this::work); scope.join(); scope.throwIfFailed(); return t.get(); } }` - **Option B:** If you need async return, use a `CompletableFuture` instead — scope is the wrong abstraction.

What NOT to say: - "Just call `scope.close()` in a finalizer" — finalizers are unreliable and slow. - "Structured concurrency forbids this" — it does at the API level, and the static checker (if enabled) would flag it.

Common follow-up: "What's the principle behind 'structured' in structured concurrency?"

Scenario 75 · Deadlock in logger MDC propagation

Asked at: Cred · enterprise · **Difficulty:** Medium · **Topic:** MDC + threads

The code:

```
MDC.put("requestId", id);  
CompletableFuture.runAsync(() -> {  
    log.info("processing");  
    process();  
});
```

The interviewer's prompt: "MDC requestId doesn't appear in async log lines. Why?"

Thinking out loud (the diagnosis): 1. "MDC is backed by a ThreadLocal." 2. " `runAsync` runs on a different thread (FJ pool). That thread's MDC is empty." 3. "Logs from the async path have no requestId."

The actual bug: ThreadLocal context not propagated across thread boundaries. Async tasks lose request context.

The fix: - **Option A:** Capture MDC at submit, restore in the task: `java var ctx =`

`MDC.getCopyOfContextMap(); CompletableFuture.runAsync(() ->`

`{ MDC.setContextMap(ctx); try { process(); } finally { MDC.clear(); } });` -

Option B: A `TaskDecorator` /wrapper executor that auto-propagates MDC for every submit. - **Option**

C: JDK 21 `ScopedValue` if migrating off ThreadLocal.

What NOT to say: - "MDC propagates automatically" — only in specific frameworks with their own propagators.

Common follow-up: "What's the difference between `MDC.setContextMap` and `MDC.put` for restoration?"

Scenario 76 · Lazy holder accidentally not lazy

Asked at: senior · **Difficulty:** Medium · **Topic:** Lazy init

The code:

```
public class Services {
    public static final ExpensiveThing THING = new ExpensiveThing();
}

public class Main {
    public static void main(String[] args) {
        if (someCondition) {
            Services.THING.use();
        }
    }
}
```

The interviewer's prompt: "Startup takes 5 seconds even when `someCondition` is false. `THING` is built but never used. Why?"

Thinking out loud (the diagnosis): 1. " `Services.THING` is a static field initialized at class load." 2. "Class loading is triggered by any reference to `Services`, including the import or the `if` body... but only if `Services` itself is touched at runtime." 3. "If `Services.THING` appears in code path before the `if`,

or class loading happens for other reasons (like reflection scan, Spring component scan, etc.), *THING* is constructed."

The actual bug: Static fields aren't lazy w.r.t. usage — they initialize when the class loads.

The fix: - **Option A:** Holder idiom: `java public class Services { private static class Holder { static final ExpensiveThing THING = new ExpensiveThing(); } public static ExpensiveThing thing() { return Holder.THING; } }` - **Option B:** `Supplier<ExpensiveThing>` cached on first call.

What NOT to say: - "Static fields are lazy" — they're class-init-time, not first-use unless accessed via a Holder.

Common follow-up: "What exactly triggers class initialization per JLS?"

Scenario 77 · `CompletableFuture`'s `applyAsync` runs on wrong executor

Asked at: Cred · **Difficulty:** Hard · **Topic:** `CompletableFuture` executor

The code:

```
public CompletableFuture<Result> chain(long id) {
    return CompletableFuture.supplyAsync(() -> stepA(id), customA)
        .thenApply(this::stepB)
        .thenApplyAsync(this::stepC, customC);
}
```

The interviewer's prompt: "stepA runs on customA. stepC on customC. stepB sometimes runs on customA, sometimes on the common pool. We want it on customA. Why the inconsistency?"

Thinking out loud (the diagnosis): 1. "`thenApply` (without Async) runs on either the thread that completed the predecessor or the thread that called `thenApply` — whichever happens later." 2. "If stepA completes BEFORE `thenApply` is registered, stepB runs on the caller's thread. If stepA hasn't completed yet, stepB runs on customA (which completed it)."

The actual bug: `thenApply` without Async has undefined executor — depends on the timing of completion vs registration.

The fix: - **Option A:** Use `thenApplyAsync(this::stepB, customA)` — explicit executor for every stage. - **Option B:** Use a single executor throughout the chain — pass it on every Async call.

What NOT to say: - "thenApply uses the predecessor's executor" — sometimes; not guaranteed.

Common follow-up: "What's the executor for `thenApply` (without Async) per the spec?"

Scenario 78 · Two-phase commit without barrier

Asked at: Cred · senior · **Difficulty:** Hard · **Topic:** Coordination

The code:

```
public class TwoPhase {
    private volatile boolean phase1Done;
    private volatile boolean phase2Done;

    public void worker1() { phase1Done = true; while (!phase2Done) {} commit1(); }
    public void worker2() { while (!phase1Done) {} phase2Done = true; commit2(); }
}
```

The interviewer's prompt: "Workers complete out of expected order. We want phase1 before phase2 commits. Why doesn't volatile guarantee this?"

Thinking out loud (the diagnosis): 1. "Volatile gives per-field visibility, not ordering across multiple variables in a global sense." 2. "Spinning on booleans is dependent on the JIT not eliminating the spin — which it won't for volatile." 3. "But the order of `commit1` and `commit2` is what matters. The actual race: w1 sets phase1Done; w2 sees, sets phase2Done, calls commit2. Meanwhile w1 sees phase2Done, calls commit1. Order: commit2 before commit1 — usually wrong direction."

The actual bug: Two-phase coordination needs a proper barrier. Hand-rolled flags are correct on each side but tricky to compose.

The fix: - **Option A:** `CyclicBarrier(2)` — both parties await, then proceed simultaneously. - **Option B:** `Phaser` for variable parties. - **Option C:** A clear lock + Condition; or just use a higher-level pattern like an actor.

What NOT to say: - "Volatile gives full ordering" — it gives per-variable ordering, not multi-variable.

Common follow-up: "What is `CyclicBarrier`'s effective happens-before edge?"

Scenario 79 · Misusing AtomicBoolean for `if not yet`

Asked at: Razorpay · **Difficulty:** Medium · **Topic:** CAS misuse

The code:

```
private final AtomicBoolean initialized = new AtomicBoolean(false);

public void init() {
    if (!initialized.get()) {
        initialized.set(true);
        doInit();
    }
}
```

The interviewer's prompt: "AtomicBoolean. Yet `doInit` runs twice occasionally. Why?"

Thinking out loud (the diagnosis): 1. "`get` then `set` is not atomic. Two threads both see false, both set true, both call `doInit`."

The actual bug: Get-then-set, not compare-and-set.

The fix: - **Option A:** `if (initialized.compareAndSet(false, true)) doInit();` — atomic transition. - **Option B:** `getAndSet(true)` and check the returned value.

What NOT to say: - "AtomicBoolean.set is atomic" — it is; the bug is in the compound check-then-set.

Common follow-up: "What's the difference between `compareAndSet` and `getAndSet` for this use case?"

Scenario 80 · Synchronized chain doesn't compose

Asked at: senior · **Difficulty:** Medium · **Topic:** Lock composition

The code:

```
class Account {
    public synchronized void debit(long x) { bal -= x; }
    public synchronized void credit(long x) { bal += x; }
}

void transfer(Account a, Account b, long x) {
    a.debit(x);
    b.credit(x);
}
```

The interviewer's prompt: "Account.debit/credit are atomic. Between debit and credit, another thread can see an intermediate state where money has vanished. Diagnose."

Thinking out loud (the diagnosis): 1. "Each method is atomic individually. The composition isn't." 2. "A query thread between debit and credit sees A reduced, B unchanged — money temporarily missing."

The actual bug: Atomicity doesn't compose without external coordination. Class invariant ("conservation of money") spans two methods.

The fix: - **Option A:** Higher-level lock or transaction that spans both calls. Ordered lock acquisition to prevent deadlock. - **Option B:** A single mutator that does both: `Bank.transfer(a, b, x)` locks both accounts in canonical order. - **Option C:** STM or event-sourced approach — observe via consistent snapshots.

What NOT to say: - "Just synchronize transfer" — needed, but lock ordering still required to avoid deadlock.

Common follow-up: "Why does each Account being thread-safe not make `transfer` thread-safe?"

Scenario 81 · `wait` without re-checking after interrupt

Asked at: enterprise · **Difficulty:** Medium · **Topic:** Interrupt + wait

The code:

```
public synchronized void awaitReady() throws InterruptedException {
    if (!ready) wait();
    consume();
}
```

The interviewer's prompt: "After an `InterruptedException`, the next call to `awaitReady` consumes a stale `ready` value. Why?"

Thinking out loud (the diagnosis): 1. "Predicate checked once. After `wait()` returns (via notify, spurious wakeup, or interrupt), we go to consume without rechecking." 2. "`wait` throws `InterruptedException` on interrupt — exits the method. But the next caller doesn't re-check 'ready' if it skipped the wait." 3. "Actually wait the question — `InterruptedException` causes the method to throw, which is fine. The bug is the `if` instead of `while` for the spurious-wakeup / barge-in case."

The actual bug: Predicate not in a while-loop. Classic wait/notify mistake.

The fix: - **Option A:** `while (!ready) wait();` — standard pattern.

What NOT to say: - "Catch `InterruptedException` and continue" — separate issue; the bug is the missing loop.

Common follow-up: "Why is the `while` mandatory rather than just recommended?"

Scenario 82 · ForkJoin tasks blocking on JDBC

Asked at: Cred · **Difficulty:** Hard · **Topic:** FJ pool starvation

The code:

```
ids.parallelStream().forEach(id -> {  
    repo.find(id); // JDBC  
});
```

The interviewer's prompt: "Parallel JDBC fetches are no faster than sequential — sometimes slower. Why?"

Thinking out loud (the diagnosis): 1. "`parallelStream` uses ForkJoin common pool — sized ~ `availableProcessors`." 2. "Each JDBC call blocks waiting on a socket. Common pool threads are blocked, can't pick up other tasks." 3. "Result: a few JDBC calls in flight, others queued. Plus you're contending on the common pool with everyone else."

The actual bug: IO-bound work on FJ common pool — wrong tool. Common pool is CPU-bound sized.

The fix: - **Option A:** Use a dedicated executor sized for IO concurrency (~ connection pool size). -

Option B: JDK 21 virtual threads — `IntStream.range(0, ids.size()).parallel()` is still common pool; use `Executors.newVirtualThreadPerTaskExecutor()` + futures. - **Option C:** Async JDBC alternative (R2DBC) — non-blocking IO.

What NOT to say: - "Increase parallelism via `setProperty`" — affects every parallel stream globally.

Common follow-up: "Why are virtual threads particularly suited to JDBC-style blocking work?"

Scenario 83 · Concurrent map putIfAbsent + value mutation race

Asked at: Cred · **Difficulty:** Medium · **Topic:** Compound atomicity

The code:

```
ConcurrentMap<String, Set<String>> tagsByUser = new ConcurrentHashMap<>();

public void tag(String user, String t) {
    Set<String> tags = tagsByUser.putIfAbsent(user, new HashSet<>());
    if (tags == null) tags = tagsByUser.get(user);
    tags.add(t);
}
```

The interviewer's prompt: "Tags occasionally lost. Why?"

Thinking out loud (the diagnosis): 1. " `putIfAbsent` returns null if it inserted, or the existing value. We then fetch via `get` if null." 2. "That dance is fine for getting the set. But: the set is `new HashSet<>()` — not thread-safe. Concurrent `tags.add` corrupts." 3. "Plus: `putIfAbsent(user, new HashSet<>())` allocates the set on every call, even when not needed."

The actual bug: Inner collection isn't thread-safe. Concurrent add can corrupt or lose entries.

The fix: - **Option A:** `computeIfAbsent` + thread-safe set: `java tagsByUser.computeIfAbsent(user, k -> ConcurrentHashMap.newKeySet()).add(t);` - **Option B:** `synchronized` on the per-user set if it's mostly read-only after init.

What NOT to say: - "ConcurrentHashMap handles inner mutation" — it doesn't; only outer atomicity.

Common follow-up: "Why does `computeIfAbsent` avoid the per-call allocation?"

Scenario 84 · CountdownLatch await with timeout, ignored return

Asked at: Cred · **Difficulty:** Medium · **Topic:** API misuse

The code:

```
CountDownLatch latch = new CountDownLatch(N);
// fire tasks
latch.await(5, TimeUnit.SECONDS);
collectResults();
```

The interviewer's prompt: "Sometimes `collectResults` sees partial results — half the tasks haven't completed. Diagnose."

Thinking out loud (the diagnosis): 1. " `await(timeout, unit)` returns `false` if the latch didn't reach zero. We ignore the return." 2. "Under slow tasks, we time out, proceed to collect anyway. Partial results."

The actual bug: Discarding boolean from timed-await. Same class as `BlockingQueue.offer` return-discard.

The fix: - **Option A:** Check the return; throw or fall back: `java if (!latch.await(5, TimeUnit.SECONDS)) throw new TimeoutException();` - **Option B:** Increase timeout or use unbounded `await()` — only if hangs are acceptable.

What NOT to say: - "await blocks until done" — not the timed variant.

Common follow-up: "How does CountdownLatch's await contract differ from CyclicBarrier's?"

Scenario 85 · `CompletableFuture.allOf` doesn't fail fast

Asked at: Cred · **Difficulty:** Medium · **Topic:** `CompletableFuture`

The code:

```
var futures = List.of(callA(), callB(), callC());
CompletableFuture
    .allOf(futures.toArray(new CompletableFuture[0]))
    .thenAccept(v -> use(futures));
```

The interviewer's prompt: "If callA fails fast in 10ms, we still wait for callB and callC for 30 seconds. We want fail-fast. Why doesn't `allOf`?"

Thinking out loud (the diagnosis): 1. "`allOf` waits for ALL futures to complete (success or failure). It doesn't short-circuit on first failure." 2. "For fail-fast: implement manually or use structured concurrency."

The actual bug: `allOf` is "wait for all", not "fail on first". Different semantics.

The fix: - **Option A:** `anyOf` with logic to check completion type — coarse. - **Option B:** Custom: register `whenComplete` on each future; on first failure, cancel siblings. - **Option C:** JDK 21 `StructuredTaskScope.ShutdownOnFailure` — built for this.

What NOT to say: - "`allOf` cancels on failure" — no, it gathers all completions.

Common follow-up: "How does `ShutdownOnFailure` actually achieve fail-fast in Java's API?"

Scenario 86 · Shared `SecureRandom` contention

Asked at: Razorpay · **Difficulty:** Medium · **Topic:** Library bottleneck

The code:

```
private static final SecureRandom RNG = new SecureRandom();

public String token() {
    byte[] b = new byte[16];
    RNG.nextBytes(b);
    return Base64.getEncoder().encodeToString(b);
}
```

The interviewer's prompt: "Token generation latency spikes under load — 100x slower under concurrency than single-threaded. Why?"

Thinking out loud (the diagnosis): 1. " `SecureRandom` is thread-safe — internally synchronized." 2. "All callers serialize on the same instance's internal lock." 3. "At high concurrency: contention bottleneck."

The actual bug: Thread-safe but not concurrent. Single shared instance becomes a serialization point.

The fix: - **Option A:** `ThreadLocal<SecureRandom>` — one per thread. - **Option B:** `SecureRandom.getInstanceStrong()` cached per thread. - **Option C:** Pool of instances.

What NOT to say: - "SecureRandom is fast" — it is per call; bottleneck is contention.

Common follow-up: "Why doesn't the JDK give us a contention-free SecureRandom out of the box?"

Scenario 87 · Implicit lock release on exception

Asked at: enterprise · **Difficulty:** Easy-Medium · **Topic:** Lock release

The code:

```
ReentrantLock lock = new ReentrantLock();

public void op() {
    lock.lock();
    work();
    lock.unlock();
}
```

The interviewer's prompt: "If `work()` throws, all subsequent `op()` calls hang forever. Why?"

Thinking out loud (the diagnosis): 1. "No try-finally. Exception inside work skips unlock." 2. "Lock held by the dead thread forever. Other threads block indefinitely."

The actual bug: `ReentrantLock` requires explicit unlock in finally. `Synchronized` auto-releases; `ReentrantLock` doesn't.

The fix: - **Option A:** `java lock.lock(); try { work(); } finally { lock.unlock(); } -`
Option B: Stick with `synchronized` if no advanced features needed — auto-release on exception.

What NOT to say: - "ReentrantLock auto-releases" — it doesn't.

Common follow-up: "How might `Lock.lock` be wrapped in a try-with-resources style helper?"

Scenario 88 · synchronized lock from constructor `this` arg

Asked at: senior · **Difficulty:** Hard · **Topic:** Construction race

The code:

```
public class Subscriber {
    public Subscriber(EventBus bus) {
        bus.register(this::handle);
    }
    private void handle(Event e) { ... }
}
```

The interviewer's prompt: "After registering, the Subscriber receives an event and `handle` is invoked. Sometimes handle sees `null` for a field initialized later in the constructor. Why?"

Thinking out loud (the diagnosis): 1. "Constructor escapes `this` (via method reference) before init completes." 2. "EventBus may fire from another thread immediately. Subscriber's later field initialization isn't visible yet."

The actual bug: `this` escape during construction. The handle is callable before the rest of the constructor runs.

The fix: - **Option A:** Use a static factory: `java public static Subscriber create(EventBus bus) { Subscriber s = new Subscriber(); bus.register(s::handle); return s; } -`
Option B: Make fields final + initialize before register — but register is the last line.

What NOT to say: - "Synchronize the constructor" — doesn't fix escape semantics.

Common follow-up: "What's special about final-field freeze when this-escape doesn't happen?"

Scenario 89 · Cancellation token not checked in tight loop

Asked at: Cred · **Difficulty:** Medium · **Topic:** Cancellation

The code:

```
public void crunch(List<Long> ids) {
    for (long id : ids) {
        compute(id);
    }
}

// Caller
Future<?> f = executor.submit(() -> crunch(longList));
f.cancel(true);
```

The interviewer's prompt: "cancel(true) returns; the cruncher keeps running for minutes. Diagnose."

Thinking out loud (the diagnosis): 1. "cancel(true) interrupts the worker thread." 2. "The crunch loop doesn't check Thread.interrupted() and compute doesn't either — interrupt is silently set but never observed."

The actual bug: Tight CPU loop with no interrupt check. Cancellation is a hint, not a halt.

The fix: - **Option A:** Check periodically: `java for (long id : ids) { if (Thread.interrupted()) throw new InterruptedException(); compute(id); }` -

Option B: Use a cancellation flag (volatile boolean) the caller can toggle.

What NOT to say: - "cancel kills the thread" — doesn't; cooperative cancellation only.

Common follow-up: "Why does Java avoid Thread.stop() style termination?"

Scenario 90 · Race in lazy init via Map.compute

Asked at: Cred · **Difficulty:** Hard · **Topic:** CHM compute timing

The code:

```
ConcurrentHashMap<String, Heavy> cache = new ConcurrentHashMap<>();

public Heavy get(String k) {
    if (!cache.containsKey(k)) {
        cache.compute(k, (key, cur) -> cur != null ? cur : new Heavy(key));
    }
    return cache.get(k);
}
```

The interviewer's prompt: "Sometimes Heavy is created multiple times for the same key, even with the contains check + compute. Why?"

Thinking out loud (the diagnosis): 1. "containsKey then compute is two operations. Between them, another thread can also see absent and call compute." 2. "Actually, compute does check cur != null — so simultaneous computes are serialized per key. Only one Heavy per key... unless the lambda itself has side effects we count." 3. "Wait — compute's function is per-bin synchronized. So only one Heavy(key) is constructed per missing entry. Where's the bug?" 4. "Bug: cache.compute(k, ...) returns the value, but we discard it and re-fetch via cache.get(k). Between compute and get, another thread could evict / overwrite. Plus the containsKey check is racy and wasteful."

The actual bug: Two-step lookup is racy; should use computeIfAbsent directly which is atomic and returns the value.

The fix: - **Option A:** `java return cache.computeIfAbsent(k, key -> new Heavy(key));` - **Option B:** For very expensive constructions, consider `Caffeine.get(key, loader)` which has stronger guarantees.

What NOT to say: - "containsKey is fast" — it is; but combining it with compute defeats the atomicity.

Common follow-up: "What's the difference between compute and computeIfAbsent for missing keys?"

Scenario 91 · Daemon thread dies with JVM, abandons work

Asked at: Cred · **Difficulty:** Medium · **Topic:** Thread lifecycle

The code:

```
Thread t = new Thread(() -> {
    while (true) flush(buffer);
});
t.setDaemon(true);
t.start();
```

The interviewer's prompt: "After JVM shutdown, in-memory buffer entries are lost. We thought daemon was a good choice. Why was it wrong?"

Thinking out loud (the diagnosis): 1. "Daemon threads are killed mid-execution when the JVM exits — no chance to flush." 2. "Buffer in memory at shutdown is discarded."

The actual bug: Daemon for work that must complete before shutdown. Daemons are abruptly killed.

The fix: - **Option A:** Non-daemon thread + JVM `Runtime.addShutdownHook` to signal shutdown gracefully. - **Option B:** `ExecutorService` with `shutdown` + `awaitTermination` in a shutdown hook.

What NOT to say: - "Daemon flushes before exit" — it doesn't.

Common follow-up: "What's the right pattern for graceful flush-on-shutdown?"

Scenario 92 · Synchronized + virtual thread + sleeping = pinned carrier

Asked at: Cred · 2025+ · **Difficulty:** Hard · **Topic:** Virtual thread pinning

The code:

```
class TokenBucket {
    public synchronized boolean acquire() throws InterruptedException {
        while (tokens == 0) {
            Thread.sleep(100); // wait for refill
        }
        tokens--;
        return true;
    }
}
```

The interviewer's prompt: "Under virtual threads, throughput collapses when tokens are exhausted. Yet `acquire()` looks fine. Why?"

Thinking out loud (the diagnosis): 1. "Sleep inside `synchronized` — virtual thread holds its carrier OS thread for the entire sleep." 2. "Many virtual threads piling up in `acquire` all pinning their carriers — small carrier pool exhausted." 3. "Bonus: spinning sleep is a flawed waiting strategy."

The actual bug: Sleep inside synchronized pins the carrier. Plus polling-style wait is wasteful.

The fix: - **Option A:** Use `ReentrantLock` + `Condition`: `java lock.lock(); try { while (tokens == 0) cond.await(100, TimeUnit.MILLISECONDS); tokens--; } finally { lock.unlock(); }` - **Option B:** `Semaphore.acquire()` — internally uses AQS, no pinning.

What NOT to say: - "Sleep is fine in synchronized" — pinning issue is documented.

Common follow-up: "What's an AQS lock and why doesn't it pin?"

Scenario 93 · `CompletableFuture` exception type changes via `thenApply`

Asked at: Cred · **Difficulty:** Medium · **Topic:** Exception in chain

The code:

```
return CompletableFuture
    .supplyAsync(() -> { throw new IOException("net"); })
    .thenApply(this::transform)
    .exceptionally(ex -> {
        if (ex instanceof IOException io) return "fallback-io";
        return "fallback-other";
    });
```

The interviewer's prompt: "IOException is thrown but the exceptionally returns 'fallback-other'. Why?"

Thinking out loud (the diagnosis): 1. "`supplyAsync`" runs a lambda that throws checked `IOException` — but lambdas can't throw checked. The compiler would refuse. So actually this must wrap in `RuntimeException`, or the `IOException` becomes a `CompletionException`." 2. "In `exceptionally`, the `Throwable` passed is a `CompletionException` wrapping the original `IOException`." 3. "`ex instanceof IOException` is false — `ex` is the wrapper."

The actual bug: `CompletableFuture` wraps exceptions. The handler's `ex` is the wrapper, not the original.

The fix: - **Option A:** Unwrap explicitly: `java Throwable cause = ex instanceof CompletionException ? ex.getCause() : ex; if (cause instanceof IOException) ...` - **Option B:** Use `handle` which gets the raw exception in older variants — though it also wraps.

What NOT to say: - "ex is the original exception" — usually a wrapper.

Common follow-up: "Why does `CompletableFuture` wrap in `CompletionException` rather than passing the original?"

Scenario 94 · Multiple `signalAll` waste cycles

Asked at: senior · **Difficulty:** Medium · **Topic:** Condition signal

The code:

```
lock.lock();
try {
    for (int i = 0; i < 1000; i++) buffer.add(i);
    cond.signalAll();
} finally { lock.unlock(); }
```

The interviewer's prompt: "After this, all consumers wake up but only one finds an item; the rest go back to sleep. Inefficient. Why?"

Thinking out loud (the diagnosis): 1. "`signalAll` wakes EVERY waiter. They each contend on the lock; the lucky one consumes; others re-wait." 2. "Wakeup-storm — high CPU, low throughput."

The actual bug: `signalAll` is correct but wasteful when only N consumers can proceed. Use `signal N` times or a different design.

The fix: - **Option A:** `signal()` per item if you know how many items. - **Option B:** Use `BlockingQueue` — it signals exactly the right number.

What NOT to say: - "`signalAll` is always safer than `signal`" — only correctness-safer; throughput-cost.

Common follow-up: "When is `signalAll` the only correct choice?"

Scenario 95 · Read of `final` field before constructor completes

Asked at: senior · **Difficulty:** Hard · **Topic:** Construction visibility

The code:

```
public class Box {
    private final int x;
    public Box(int v) {
        sharedRef = this; // publish this
        this.x = v;
    }
    public int x() { return x; }
}
```

The interviewer's prompt: "Another thread reads `sharedRef.x()` and sees 0 instead of the constructor's value. Why?"

Thinking out loud (the diagnosis): 1. "`this` escapes before `x` is assigned. Other thread reads `x` which is still 0 (default)." 2. "Final-field freeze guarantees visibility AFTER the constructor returns — not during."

The actual bug: `this` escape during construction nullifies the final-field guarantees.

The fix: - **Option A:** Assign before publishing: `java public Box(int v) { this.x = v; sharedRef = this; }` - **Option B:** Factory pattern — no this-escape.

What NOT to say: - "Final fields always visible" — only post-constructor.

Common follow-up: "What's the JLS rule about final-field freeze timing?"

Scenario 96 · ThreadFactory not setting daemon, holds JVM exit

Asked at: enterprise · **Difficulty:** Easy-Medium · **Topic:** Thread lifecycle

The code:

```
ExecutorService bg = Executors.newSingleThreadExecutor();
bg.submit(() -> watchFiles());
```

The interviewer's prompt: "JVM doesn't exit even after `main()` returns. Why?"

Thinking out loud (the diagnosis): 1. "Default `ExecutorService` threads are non-daemon." 2. "`watchFiles` is a long-running task. Non-daemon thread alive => JVM doesn't exit."

The actual bug: Default thread factory creates non-daemon threads. JVM exit requires all non-daemon threads to terminate.

The fix: - **Option A:** Custom daemon factory: `java Executors.newSingleThreadExecutor(r -> { Thread t = new Thread(r); t.setDaemon(true); return t; });` - **Option B:** Call `bg.shutdown()` in a shutdown hook to halt politely.

What NOT to say: - "Set `System.exit(0)` at end" — abrupt; doesn't flush.

Common follow-up: "What's the JVM exit rule with daemon vs non-daemon threads?"

Scenario 97 · Spinning on volatile field for ordering

Asked at: Cred · low-latency · **Difficulty:** Hard · **Topic:** Volatile + atomicity

The code:

```
private volatile long seq;
private long[] data = new long[1024];

public void write(int idx, long v) {
    data[idx] = v;
    seq++;
}

public long read(int idx) {
    long s1 = seq;
    long v = data[idx];
    long s2 = seq;
    if (s1 == s2 && (s1 & 1) == 0) return v;
    // retry
}
```

The interviewer's prompt: "Pattern looks like a seqlock. But `seq++` isn't atomic — what's wrong?"

Thinking out loud (the diagnosis): 1. "`seq++` on volatile is read-modify-write — not atomic. Two writers can collide; counts get lost." 2. "And the protocol needs odd→even transitions for writers. With non-atomic increment, two writers can leave `seq` at an arbitrary value." 3. "Plus: writes to `data[idx]` aren't volatile — even with `seq` updates, the array writes may be reordered around the `seq` read."

The actual bug: Hand-rolled seqlock with wrong atomicity primitive. Should use `VarHandle.releaseFence` / `acquireFence` and CAS on the sequence.

The fix: - **Option A:** Use a proper concurrent data structure (CHM, `AtomicLongArray`). - **Option B:** If you really need a seqlock, use `VarHandle` for release/acquire fences and a single writer model. - **Option C:** `StampedLock.tryOptimisticRead` — JDK gives you this pattern out of the box.

What NOT to say: - "volatile is enough for seqlock" — atomicity and ordering both matter.

Common follow-up: "How does StampedLock's optimistic read implement a seqlock under the hood?"

Scenario 98 · synchronized on autoboxed Integer

Asked at: TCS · **code-review** · **Difficulty:** Easy-Medium · **Topic:** Lock object

The code:

```
public void inc(Integer key) {  
    synchronized (key) {  
        counters.merge(key, 1, Integer::sum);  
    }  
}
```

The interviewer's prompt: "Concurrent calls with the same Integer key seem to work. Concurrent calls with different keys sometimes block on each other. Why?"

Thinking out loud (the diagnosis): 1. "Integer.valueOf caches values -128..127 — same object across instances." 2. " `synchronized(5)` and `synchronized(5)` from anywhere in the JVM lock the same monitor — even from unrelated code." 3. "Big keys (> 127) won't share; you get inconsistent behavior at the boundary."

The actual bug: Locking on cached boxed primitives. Same hazard as boxed Long. Cross-class contention.

The fix: - **Option A:** Use `ConcurrentHashMap.merge` — already atomic; no synchronized needed. - **Option B:** Striped lock by hash if external locking is required.

What NOT to say: - "Synchronized on Integer is fine" — caching makes it broken.

Common follow-up: "What other JDK classes have hidden interning that breaks lock identity?"

Scenario 99 · Premature stream close in async pipeline

Asked at: Cred · **Difficulty:** Medium · **Topic:** Resource lifecycle

The code:


```
public CompletableFuture<List<String>> read(Path p) {
    return CompletableFuture.supplyAsync(() -> {
        try (Stream<String> s = Files.lines(p)) {
            return s.collect(Collectors.toList());
        } catch (IOException e) { throw new RuntimeException(e); }
    });
}
```

The interviewer's prompt: "This actually works. Now consider an alternate version that returns the Stream itself — what's the bug?"

Thinking out loud (the diagnosis): 1. "In the alternate (`return s;` outside try-with-resources), the stream is closed before the caller consumes. Reads on a closed stream fail." 2. "In the present code: stream consumed inside try-with-resources, list materialized, fine."

The actual bug (in the alternate): Returning a Stream from inside try-with-resources closes the resource before the caller can consume it. Common with async.

The fix: - **Option A:** Materialize inside the try (current code). - **Option B:** Pass the open stream to the caller with an explicit close contract — fragile. - **Option C:** Use `Files.readAllLines` — bounded memory but no resource lifecycle question.

What NOT to say: - "try-with-resources doesn't close until the future completes" — closes at the end of the try block, regardless of futures.

Common follow-up: "How does this differ in JDK 8 `Files.lines` vs `Files.readAllLines`?"

Scenario 100 · Async logging swallows shutdown messages

Asked at: Cred · enterprise · **Difficulty:** Medium · **Topic:** Async logger flush

The code:

```
@PreDestroy
public void shutdown() {
    log.info("shutting down");
    cleanup();
}
```

The interviewer's prompt: "On graceful shutdown, the 'shutting down' line never appears in the log file. Why?"

Thinking out loud (the diagnosis): 1. "Async appenders (Log4j2 AsyncAppender, Logback AsyncAppender) buffer log events to a queue, flushed by a dedicated thread." 2. "If JVM exits before the appender's thread drains the queue, in-flight log messages are lost."

The actual bug: Async log handler not given time to flush before shutdown. Daemon background flusher dies with JVM.

The fix: - **Option A:** Call `LoggerContext.stop()` explicitly in the shutdown hook to drain. - **Option B:** Use synchronous logging for shutdown-critical messages. - **Option C:** Configure shutdown hook in the logging framework (Logback: `<shutdownHook/>`, Log4j2: `shutdownHook` config).

What NOT to say: - "Logs are always flushed" — async appenders explicitly aren't.

Common follow-up: "What's the trade-off between async logging throughput and shutdown reliability?"

Category 6 — Refactor This

These are the snippets interviewers slide across the table with the line "clean this up." They look like working code — and they are — but each one carries a smell that a senior engineer should spot in under thirty seconds. Read the code, name the smell, then refactor it the way you would on a real PR.

Scenario 1 · Nested if-else hell on payment status

Asked at: Razorpay · PhonePe · **Difficulty:** Easy · **Topic:** Switch expression

The code:

```
public String describe(String status) {
    if (status.equals("INITIATED")) {
        return "Payment created, awaiting gateway";
    } else if (status.equals("AUTHORIZED")) {
        return "Funds blocked on customer card";
    } else if (status.equals("CAPTURED")) {
        return "Money landed in our account";
    } else if (status.equals("FAILED")) {
        return "Gateway declined";
    } else if (status.equals("REFUNDED")) {
        return "Money returned to customer";
    } else {
        return "Unknown";
    }
}
```

The interviewer's prompt: "Modernize this for Java 21. Single expression if you can."

Thinking out loud: 1. "Six branches keyed on a String — that's a switch expression. Modern Java returns directly from the arrow form." 2. " `default` keeps exhaustiveness. No fall-through, no break statements, no accidental empty return."

The refactored version:

```
public String describe(String status) {
    return switch (status) {
        case "INITIATED" -> "Payment created, awaiting gateway";
        case "AUTHORIZED" -> "Funds blocked on customer card";
        case "CAPTURED" -> "Money landed in our account";
        case "FAILED" -> "Gateway declined";
        case "REFUNDED" -> "Money returned to customer";
        default -> "Unknown";
    };
}
```

Why this is better: - One expression, one return point — easier to scan - Arrow form has no fall-through, eliminates a whole class of bugs - Compiler enforces exhaustiveness when used with sealed types or enums

What NOT to say: - "Replace with a `Map<String,String>` " — works, but allocates a map per call site and loses the compile-time check on cases - "Use ternary chaining" — readable for two branches, unreadable for six

Common follow-up: "How would this change if `status` were an enum instead of a String?"

Scenario 2 · Long parameter list on order creation

Asked at: Flipkart · Meesho · **Difficulty:** Easy · **Topic:** Builder / record

The code:

```
public Order createOrder(String customerId, String productId, int quantity,
                        BigDecimal unitPrice, String currency, String shippingAddr,
                        String billingAddr, String couponCode, boolean giftWrap,
                        String giftMessage, LocalDate deliveryDate) {
    Order o = new Order();
    o.setCustomerId(customerId);
    o.setProductId(productId);
    o.setQuantity(quantity);
    // ... eight more setters
    return o;
}
```

The interviewer's prompt: "Eleven parameters. Half optional. Clean it up."

Thinking out loud: 1. "Long parameter list is the smell — callers can swap two String args and the compiler stays quiet." 2. "Two clean options: a record for the request payload, or a builder if a lot of fields are optional. Gift wrap, coupon, message are optional — builder fits."

The refactored version:

```
public record OrderRequest(String customerId, String productId, int quantity,
                        BigDecimal unitPrice, String currency, String shippingAddr,
                        String billingAddr, Optional<String> couponCode,
                        boolean giftWrap, Optional<String> giftMessage,
                        LocalDate deliveryDate) {}

public Order createOrder(OrderRequest req) {
    return new Order(req); // Order has a constructor that takes the record
}
```

Why this is better: - Caller sees field names at the call site, can't swap two Strings by mistake - Record is immutable, equals/hashCode/toString auto-generated - Optional fields explicit in the type

What NOT to say: - "Just pass a `Map<String, Object>`" — loses every type guarantee - "Reduce to fewer parameters by hardcoding defaults" — silently changes behaviour for callers who relied on those args

Common follow-up: "When would you pick a builder over a record for this?"

Scenario 3 · Primitive obsession on money

Asked at: Razorpay · Cred · **Difficulty:** Medium · **Topic:** Value object

The code:

```
public class Invoice {
    private double amount;
    private String currency;

    public double convertTo(String target, double rate) {
        return amount * rate; // currency ignored
    }

    public boolean isExpensive() {
        return amount > 10000; // 10000 what? rupees? dollars?
    }
}
```

The interviewer's prompt: "Spot the bugs and fix them properly."

Thinking out loud: 1. "Two smells. `double` for money — floating-point rounding will bite on multi-currency totals. And amount + currency are two fields that should be one concept." 2. "Extract a Money value object. BigDecimal under the hood, currency baked in, arithmetic that fails fast on mismatch."

The refactored version:

```
public record Money(BigDecimal amount, Currency currency) {
    public Money add(Money other) {
        if (!currency.equals(other.currency))
            throw new IllegalArgumentException("Currency mismatch");
        return new Money(amount.add(other.amount), currency);
    }
    public Money convertTo(Currency target, BigDecimal rate) {
        return new Money(amount.multiply(rate), target);
    }
    public boolean exceeds(Money threshold) { return add(threshold.negate()).amount.signum() > 0; }
    private Money negate() { return new Money(amount.negate(), currency); }
}
```

Why this is better: - BigDecimal kills floating-point rounding - Currency mismatch fails at the call site, not in the ledger - `isExpensive()` becomes typed: `m.exceeds(new Money(10_000, INR))`

What NOT to say: - "Just use `float` instead of `double`" — same problem, smaller range - "Round to 2 decimals at the end" — doesn't help; intermediate rounding errors are the real issue

Common follow-up: "How would you handle currency conversion with rate fluctuations between read and write?"

Scenario 4 · Feature envy on customer discount

Asked at: Flipkart · Myntra · **Difficulty:** Medium · **Topic:** Move method

The code:

```
public class OrderService {
    public BigDecimal computeDiscount(Order order, Customer customer) {
        BigDecimal pct = customer.getLoyaltyTier().equals("GOLD")
            ? new BigDecimal("0.15")
            : customer.getLoyaltyTier().equals("SILVER")
            ? new BigDecimal("0.10")
            : customer.getLifetimeSpend().compareTo(new BigDecimal("50000")) > 0
            ? new BigDecimal("0.05")
            : BigDecimal.ZERO;
        return order.getTotal().multiply(pct);
    }
}
```

The interviewer's prompt: "OrderService is asking Customer too many questions. Refactor."

Thinking out loud: 1. "Feature envy — the method spends all its time inside `customer`. Discount percentage is Customer's concern, not OrderService's." 2. "Move `discountRate()` onto Customer. OrderService just multiplies."

The refactored version:

```

public class Customer {
    public BigDecimal discountRate() {
        return switch (loyaltyTier) {
            case "GOLD"    -> new BigDecimal("0.15");
            case "SILVER"  -> new BigDecimal("0.10");
            default        -> lifetimeSpend.compareTo(new BigDecimal("50000")) > 0
                           ? new BigDecimal("0.05") : BigDecimal.ZERO;
        };
    }
}

public class OrderService {
    public BigDecimal computeDiscount(Order order, Customer customer) {
        return order.getTotal().multiply(customer.discountRate());
    }
}

```

Why this is better: - Customer owns its own loyalty logic — change tiers in one place - OrderService shrinks to the one thing it should do - Unit-testable without an Order in scope

What NOT to say: - "Put the rate table in a `Map<String, BigDecimal>` constant" — fine, but doesn't fix the location problem - "Pass Customer's tier string to a static utility" — still leaves OrderService driving Customer's rules

Common follow-up: "What if loyalty tier becomes a configurable rules engine?"

Scenario 5 · God class managing everything

Asked at: TCS · Infosys · **Difficulty:** Medium · **Topic:** Split responsibilities

The code:

```

public class UserManager {
    public void register(...) { /* validate + DB save + send email + log audit */ }
    public void login(...) { /* check creds + create session + push to Kafka */ }
    public void resetPassword(...) { /* token gen + email + DB update */ }
    public void uploadAvatar(...) { /* S3 upload + DB update */ }
    public void exportGdpr(...) { /* gather + zip + sign */ }
    public void deleteAccount(...) { /* soft delete + cascade + notify */ }
    // ... 23 more methods
}

```

The interviewer's prompt: "This class is 1,400 lines. Split it."

Thinking out loud: 1. "Single Responsibility violation. Authentication, profile, GDPR, and account lifecycle are four different reasons to change." 2. "Split along the actual axes of change — UserRegistration, UserAuthentication, UserProfile, UserPrivacy. Each one tested in isolation."

The refactored version:

```
public class UserRegistration {
    void register(RegistrationRequest req) { /* validate + save */ }
}
public class UserAuthentication {
    Session login(Credentials c) { /* verify + session */ }
    void resetPassword(String email) { /* token + email */ }
}
public class UserProfile {
    void uploadAvatar(UserId id, byte[] image) { /* S3 + DB */ }
}
public class UserPrivacy {
    byte[] exportGdpr(UserId id) { /* gather + zip */ }
    void deleteAccount(UserId id) { /* soft delete */ }
}
```

Why this is better: - Each class has one reason to change - Easier to mock in tests — UserAuthentication doesn't need an S3 client - Teams can own slices without stepping on each other

What NOT to say: - "Split by method count, not responsibility" — gives you arbitrary chunks with no boundary - "Keep it as one class but add region comments" — region comments don't fix the dependency graph

Common follow-up: "How do you handle a method that genuinely needs three of these collaborators?"

Scenario 6 · Non-thread-safe cache exposed to many threads

Asked at: Swiggy · Zomato · **Difficulty:** Medium · **Topic:** Thread safety

The code:


```
public class PriceCache {
    private final Map<String, BigDecimal> cache = new HashMap<>();

    public BigDecimal get(String sku) {
        if (!cache.containsKey(sku)) {
            cache.put(sku, fetchFromDb(sku));
        }
        return cache.get(sku);
    }
}
```

The interviewer's prompt: "Multiple request threads hit this. Make it thread-safe without killing throughput."

Thinking out loud: 1. "HashMap under concurrent writes can corrupt internal state — Java 7 era had infinite loops on resize. Even now, `containsKey-then-put` is a TOCTOU race." 2.

"ConcurrentHashMap's `computeIfAbsent` is the atomic primitive I want — runs the loader at most once per key."

The refactored version:

```
public class PriceCache {
    private final ConcurrentMap<String, BigDecimal> cache = new ConcurrentHashMap<>();

    public BigDecimal get(String sku) {
        return cache.computeIfAbsent(sku, this::fetchFromDb);
    }
}
```

Why this is better: - Atomic check-then-load, no lost work - Per-bucket locking inside CHM — readers and writers on different keys don't block each other - Two lines shorter and bug-free

What NOT to say: - "Wrap the whole method in `synchronized`" — single global lock destroys read throughput - "Use `Collections.synchronizedMap`" — the compound op still isn't atomic across calls

Common follow-up: "What if `fetchFromDb` can return null or throw?"

Scenario 7 · Imperative loop that should be a stream

Asked at: Infosys · Wipro · **Difficulty:** Easy · **Topic:** Stream API

The code:

```
public List<String> activeUserEmails(List<User> users) {
    List<String> result = new ArrayList<>();
    for (int i = 0; i < users.size(); i++) {
        User u = users.get(i);
        if (u.isActive() && u.getEmail() != null) {
            result.add(u.getEmail().toLowerCase());
        }
    }
    return result;
}
```

The interviewer's prompt: "Rewrite this idiomatically."

Thinking out loud: 1. "Filter-map-collect — the canonical stream pipeline. Index-based loop adds nothing here." 2. "Use `Objects::nonNull` on the email check, `String::toLowerCase` on the map step, `toList()` for an unmodifiable result on Java 16+."

The refactored version:

```
public List<String> activeUserEmails(List<User> users) {
    return users.stream()
        .filter(User::isActive)
        .map(User::getEmail)
        .filter(Objects::nonNull)
        .map(String::toLowerCase)
        .toList();
}
```

Why this is better: - Each step says what it does — filter, map, collect - `toList()` returns an unmodifiable list; mutation accidents fail fast - Parallelizable with one keyword if the list ever gets large

What NOT to say: - "Wrap the whole loop in `stream().forEach()`" — that's an imperative loop in stream clothing - "Use `.collect(Collectors.toList())` on Java 21" — outdated; `.toList()` is the modern call

Common follow-up: "When would the stream version be slower than the loop?"

Scenario 8 · Stale Java 7 file-reading pattern

Asked at: TCS · Cognizant · **Difficulty:** Easy · **Topic:** Modernization

The code:

```
public String loadConfig(String path) throws IOException {
    BufferedReader reader = null;
    StringBuilder sb = new StringBuilder();
    try {
        reader = new BufferedReader(new FileReader(path));
        String line;
        while ((line = reader.readLine()) != null) {
            sb.append(line).append("\n");
        }
    } finally {
        if (reader != null) reader.close();
    }
    return sb.toString();
}
```

The interviewer's prompt: "This is Java 7 code. Bring it to Java 21."

Thinking out loud: 1. "Manual close in finally — that's pre-try-with-resources. Worse, `FileReader` uses platform default charset, which differs between Linux and Windows." 2. "`Files.readString(Path, UTF_8)` is one line. Charset explicit, resource closed automatically."

The refactored version:

```
public String loadConfig(String path) throws IOException {
    return Files.readString(Path.of(path), StandardCharsets.UTF_8);
}
```

Why this is better: - One line, no resource leak possible - Explicit UTF-8 — same output on dev laptop, CI, and prod box - `Path.of` replaces the older `Paths.get`

What NOT to say: - "Use try-with-resources around the `BufferedReader`" — better than the original, but `Files.readString` is the modern call for small files - "Read into a byte array and convert" — adds work for no gain

Common follow-up: "What if the file is 4 GB?"

Scenario 9 · Checked exception spaghetti

Asked at: Infosys · TCS · **Difficulty:** Medium · **Topic:** Exception handling

The code:

```

public String fetchAndParse(String url) {
    try {
        try {
            URI uri = new URI(url);
            try {
                String body = httpClient.get(uri);
                try {
                    return parser.parse(body);
                } catch (ParseException p) {
                    return "{}";
                }
            } catch (IOException io) {
                return "{}";
            }
        } catch (URISyntaxException u) {
            return "{}";
        }
    } catch (Exception e) {
        return "{}";
    }
}

```

The interviewer's prompt: "Four nested catches, all returning the same fallback. Clean it up."

Thinking out loud: 1. "Every catch returns the same `{}`. That's begging for a multi-catch or one outer catch." 2. "Java 7 added multi-catch. Java 14 added enhanced try. The point is one catch, one fallback, no pyramid."

The refactored version:

```

public String fetchAndParse(String url) {
    try {
        return parser.parse(httpClient.get(new URI(url)));
    } catch (URISyntaxException | IOException | ParseException e) {
        log.warn("fetchAndParse failed for {}: {}", url, e.getMessage());
        return "{}";
    }
}

```

Why this is better: - One try, one catch, one fallback — the intent reads top to bottom - Logging the exception preserves the diagnostic that nested catches were swallowing - Multi-catch keeps the original type info for each branch

What NOT to say: - "Catch `Exception`" — hides NPEs and runtime bugs that should propagate - "Add `throws Exception` to the signature" — pushes the problem to every caller

Common follow-up: "When is it acceptable to catch raw `Exception`?"

Scenario 10 · Optional misused as a null check

Asked at: Cred · Razorpay · **Difficulty:** Easy · **Topic:** Optional

The code:

```
public String customerName(Long id) {
    Optional<Customer> opt = repo.findById(id);
    if (opt.isPresent()) {
        Customer c = opt.get();
        if (c.getName() != null) {
            return c.getName();
        } else {
            return "Anonymous";
        }
    } else {
        return "Anonymous";
    }
}
```

The interviewer's prompt: "You added Optional but kept writing null-check code. Fix it."

Thinking out loud: 1. " `isPresent` / `get` is the anti-pattern Optional was designed to replace. Same shape as a null check." 2. "Chain `map` and `orElse` — one expression."

The refactored version:

```
public String customerName(Long id) {
    return repo.findById(id)
        .map(Customer::getName)
        .filter(Objects::nonNull)
        .orElse("Anonymous");
}
```

Why this is better: - Reads as a sentence: find, get the name, drop if null, else default - Impossible to forget the `else` branch - No `.get()` call — no IDE warning about unchecked Optional access

What NOT to say: - "Use `orElseGet(() -> \"Anonymous\")` " — fine, but for a constant `orElse` is cheaper (no supplier creation) - "Return Optional from this method" — pushes the unwrap to the caller; only do that if callers genuinely have a missing path

Common follow-up: "When should you return Optional from a method?"

Scenario 11 · Synchronized everywhere on mutable state

Asked at: PhonePe · **Cred:** · **Difficulty:** Medium · **Topic:** Immutability

The code:

```
public class Position {
    private double x, y;
    public synchronized void setX(double x) { this.x = x; }
    public synchronized void setY(double y) { this.y = y; }
    public synchronized double getX() { return x; }
    public synchronized double getY() { return y; }
    public synchronized double distanceFrom(Position other) {
        return Math.hypot(x - other.x, y - other.y);
    }
}
```

The interviewer's prompt: "Every method synchronized. There's a better way."

Thinking out loud: 1. "Synchronized getters hint that someone tried to fix a thread-safety problem the hard way. Better fix: make it immutable. No mutation, no locks needed." 2. "Convert to a record. Distance moves to a pure method. Updates become 'new Position(...)'. "

The refactored version:

```
public record Position(double x, double y) {
    public double distanceFrom(Position other) {
        return Math.hypot(x - other.x, y - other.y);
    }
    public Position withX(double newX) { return new Position(newX, y); }
    public Position withY(double newY) { return new Position(x, newY); }
}
```

Why this is better: - No locks, no visibility issues — immutable objects are inherently thread-safe - `equals` / `hashCode` / `toString` free - `withX` makes updates explicit and safe to share across threads

What NOT to say: - "Use a single lock object instead of `synchronized this` " — same throughput problem, just better encapsulated - "Mark the fields `volatile` " — fixes visibility, not the `distanceFrom` consistency problem

Common follow-up: "What if positions need to mutate millions of times per second?"

Scenario 12 · Lombok-heavy data class as a record candidate

Asked at: Flipkart · Myntra · **Difficulty:** Easy · **Topic:** Records

The code:

```
@Data
@AllArgsConstructor
@NoArgsConstructor
@Builder
public class ShippingAddress {
    private String line1;
    private String line2;
    private String city;
    private String state;
    private String pincode;
    private String country;
}
```

The interviewer's prompt: "Java 21 project. Do we still need Lombok here?"

Thinking out loud: 1. "Six fields, all immutable in spirit, no behaviour. Classic record territory." 2.

"NoArgsConstructor was needed for JPA/Jackson in the past — modern Jackson handles records, and JPA in Spring 3.x supports them via @Embeddable workarounds. If they're a free pass, drop Lombok here."

The refactored version:

```
public record ShippingAddress(
    String line1,
    String line2,
    String city,
    String state,
    String pincode,
    String country) {}
```

Why this is better: - Zero annotations, zero generated code, zero IDE plugin dependency - Immutable by construction — caller cannot mutate after passing - `equals` / `hashCode` / `toString` are spec-defined, not Lombok-defined

What NOT to say: - "Records can't have validation, keep Lombok" — records can validate inside the compact constructor - "Records can't be used with JPA" — true for `@Entity`, but fine for `@Embeddable` or DTOs

Common follow-up: "When would you keep Lombok over a record?"

Scenario 13 · For-loop with index instead of enhanced for

Asked at: TCS · Wipro · **Difficulty:** Easy · **Topic:** Enhanced for

The code:

```
public BigDecimal totalAmount(List<Invoice> invoices) {
    BigDecimal total = BigDecimal.ZERO;
    for (int i = 0; i < invoices.size(); i++) {
        Invoice inv = invoices.get(i);
        total = total.add(inv.getAmount());
    }
    return total;
}
```

The interviewer's prompt: "Two ways to clean this up — both acceptable."

Thinking out loud: 1. "Index `i` is never used. Enhanced for loop reads cleaner." 2. "Or go full stream — `mapToBigDecimal` doesn't exist, but `map().reduce(ZERO, BigDecimal::add)` does the job."

The refactored version:

```
// Option A — enhanced for, when you want a debuggable loop
public BigDecimal totalAmount(List<Invoice> invoices) {
    BigDecimal total = BigDecimal.ZERO;
    for (Invoice inv : invoices) {
        total = total.add(inv.getAmount());
    }
    return total;
}

// Option B — stream, when the rest of the codebase is stream-style
public BigDecimal totalAmount(List<Invoice> invoices) {
    return invoices.stream()
        .map(Invoice::getAmount)
        .reduce(BigDecimal.ZERO, BigDecimal::add);
}
```

Why this is better: - Enhanced for drops the index nobody used - Stream version composes naturally when filtering enters the picture later - Both avoid the off-by-one trap of `i <= invoices.size()`

What NOT to say: - "Use `Iterator` explicitly" — the enhanced-for desugars to exactly that - "Use `IntStream.range`" — only when you genuinely need the index

Common follow-up: "Why isn't there a `mapToBigDecimal` primitive specialization?"

Scenario 14 · Setter-heavy bean for an immutable concept

Asked at: Razorpay · Swiggy · **Difficulty:** Easy · **Topic:** Constructor + final

The code:

```
public class Coupon {
    private String code;
    private BigDecimal discount;
    private LocalDate expiry;

    public Coupon() {}
    public void setCode(String code) { this.code = code; }
    public void setDiscount(BigDecimal d) { this.discount = d; }
    public void setExpiry(LocalDate e) { this.expiry = e; }
    public String getCode() { return code; }
    public BigDecimal getDiscount() { return discount; }
    public LocalDate getExpiry() { return expiry; }
}
```

The interviewer's prompt: "A coupon's code and discount don't change after creation. Why are these setters here?"

Thinking out loud: 1. "Setter-driven beans were for legacy frameworks. Modern Spring, Jackson, JPA all handle constructor injection." 2. "Replace with a record or a final-field class. Whatever framework integration you need, pick the lighter shape."

The refactored version:

```
public record Coupon(String code, BigDecimal discount, LocalDate expiry) {
    public Coupon {
        Objects.requireNonNull(code);
        Objects.requireNonNull(discount);
        if (discount.signum() < 0) throw new IllegalArgumentException("Discount must be >= 0");
    }
}
```

Why this is better: - No setters means no half-constructed Coupon in a downstream cache - Compact constructor validates once, at creation - Three fields, three lines

What NOT to say: - "Add @Setter from Lombok and keep the class" — adds the wrong tool - "Mark setters package-private" — half measure; the field is still mutable inside the package

Common follow-up: "How does Jackson deserialize this record without an empty constructor?"

Scenario 15 · Stream forEach used to mutate external list

Asked at: Infosys · TCS · **Difficulty:** Medium · **Topic:** Stream idiom

The code:

```
public List<String> upperNames(List<Customer> customers) {
    List<String> out = new ArrayList<>();
    customers.stream()
        .filter(c -> c.getName() != null)
        .forEach(c -> out.add(c.getName().toUpperCase()));
    return out;
}
```

The interviewer's prompt: "Stream API but you're still mutating an external list. Why is that a smell?"

Thinking out loud: 1. "forEach on a stream that mutates external state breaks the moment someone switches to parallelStream — ArrayList.add isn't thread-safe." 2. "Use map and collect. That's what the stream API was built for."

The refactored version:

```
public List<String> upperNames(List<Customer> customers) {
    return customers.stream()
        .map(Customer::getName)
        .filter(Objects::nonNull)
        .map(String::toUpperCase)
        .toList();
}
```

Why this is better: - Pure pipeline — works correctly under parallelStream if you ever need it - No mutable accumulator visible from outside the stream - One step shorter

What NOT to say: - "Use Collectors.toList() with .collect()" — .toList() is the modern call and returns an unmodifiable list - "Use forEachOrdered to make it safe" — fixes ordering, not the thread-safety of add

Common follow-up: "When is forEach on a stream actually appropriate?"

Scenario 16 · Nested ternary that no one can read

Asked at: Cred · PhonePe · **Difficulty:** Easy · **Topic:** Readability

The code:

```
public String priorityLabel(int p) {  
    return p > 8 ? "P0" : p > 5 ? "P1" : p > 2 ? "P2" : p > 0 ? "P3" : "P4";  
}
```

The interviewer's prompt: "This passes tests but the next dev hates you. Refactor."

Thinking out loud: 1. "Right-associative ternary chain is parseable but unreadable. A reader has to mentally re-parenthesize." 2. "Switch with pattern matching on the int range is the modern fix. Java 21 supports guarded patterns."

The refactored version:

```
public String priorityLabel(int p) {  
    return switch (p) {  
        case int n when n > 8 -> "P0";  
        case int n when n > 5 -> "P1";  
        case int n when n > 2 -> "P2";  
        case int n when n > 0 -> "P3";  
        default -> "P4";  
    };  
}
```

Why this is better: - Each branch reads top-to-bottom in the order conditions are checked - Guards are explicit; no nesting precedence to mentally resolve - Compiler enforces exhaustiveness when used with sealed inputs

What NOT to say: - "Use `Math.signum` and clamp" — too clever for a label lookup - "Replace with if-else ladder" — works but loses the expression form

Common follow-up: "What's the syntax difference between a guarded pattern and an if inside a case?"

Scenario 17 · Date math with java.util.Date

Asked at: TCS · Infosys · Wipro · **Difficulty:** Easy · **Topic:** java.time

The code:

```
public Date sevenDaysFromNow() {
    Calendar c = Calendar.getInstance();
    c.setTime(new Date());
    c.add(Calendar.DAY_OF_MONTH, 7);
    return c.getTime();
}
```

The interviewer's prompt: "Project's still using `java.util.Date` and `Calendar`. Modernize one call."

Thinking out loud: 1. "`Date` is mutable, not timezone-aware, and overloaded with deprecated methods. `Calendar` is verbose." 2. "`java.time` since Java 8 — `LocalDate.now().plusDays(7)` if I want a date, `Instant.now().plus(7, DAYS)` if I want a timestamp."

The refactored version:

```
public LocalDate sevenDaysFromNow() {
    return LocalDate.now().plusDays(7);
}
```

Why this is better: - Immutable — no risk of a downstream caller mutating shared state - Timezone-aware variants (`ZonedDateTime` , `OffsetDateTime`) are explicit - One line, no `Calendar.getInstance` dance

What NOT to say: - "Use `new Date(System.currentTimeMillis() + 7L * 86400 * 1000)`" — manual arithmetic, ignores DST - "Keep `Date` because the legacy API needs it" — convert at the boundary, model in `java.time` internally

Common follow-up: "How would you convert at a legacy boundary that takes `java.util.Date`?"

Scenario 18 · Manual null checks instead of `Objects.requireNonNull`

Asked at: Razorpay · Swiggy · **Difficulty:** Easy · **Topic:** Defensive checks

The code:

```
public void transfer(Account from, Account to, BigDecimal amount) {  
    if (from == null) throw new IllegalArgumentException("from is null");  
    if (to == null) throw new IllegalArgumentException("to is null");  
    if (amount == null) throw new IllegalArgumentException("amount is null");  
    if (amount.signum() < 0) throw new IllegalArgumentException("amount must be >= 0");  
    // ...  
}
```

The interviewer's prompt: "Tighten the null checks."

Thinking out loud: 1. " `Objects.requireNonNull(x, name)` is the standard one-liner — throws NPE with the field name, which is what a stack trace consumer wants anyway." 2. "Business-rule check on signum stays as-is — it's not a null check, it's a domain rule."

The refactored version:

```
public void transfer(Account from, Account to, BigDecimal amount) {  
    Objects.requireNonNull(from, "from");  
    Objects.requireNonNull(to, "to");  
    Objects.requireNonNull(amount, "amount");  
    if (amount.signum() < 0) throw new IllegalArgumentException("amount must be >= 0");  
    // ...  
}
```

Why this is better: - NPE with the field name is grep-friendly in logs - Less ceremony per check - Standard idiom — every Java dev recognises it

What NOT to say: - "Use Lombok `@NonNull`" — works, but adds an annotation processor and a non-standard NPE message - "Catch the NPE elsewhere" — fail fast at entry, not at line 47

Common follow-up: "Why throw NPE instead of `IllegalArgumentException` for null arguments?"

Scenario 19 · String concatenation in a hot loop

Asked at: TCS · Cognizant · **Difficulty:** Easy · **Topic:** `StringBuilder`

The code:

```
public String csv(List<String> tokens) {
    String result = "";
    for (String t : tokens) {
        result += t + ",";
    }
    if (!result.isEmpty()) result = result.substring(0, result.length() - 1);
    return result;
}
```

The interviewer's prompt: "Fix the obvious issue and the not-so-obvious one."

Thinking out loud: 1. " `+=` in a loop allocates a new `String` each iteration — $O(n^2)$ work." 2. "And the trailing-comma cleanup is a smell. `String.join("\\",\\", tokens)` does both jobs in one call."

The refactored version:

```
public String csv(List<String> tokens) {
    return String.join(",", tokens);
}
```

Why this is better: - $O(n)$ instead of $O(n^2)$ - No special case for the last element - Reads as the intent: join with comma

What NOT to say: - "Switch to `StringBuilder`" — correct but misses that `String.join` exists - "Use `Collectors.joining`" — fine in a stream pipeline, overkill for a `List` input

Common follow-up: "What does the JIT do with `+=` on `Strings` — does it auto-translate to `StringBuilder`?"

Scenario 20 · Manual iteration over `Map.entrySet`

Asked at: Infosys · TCS · **Difficulty:** Easy · **Topic:** `forEach` on `Map`

The code:

```
public void logAll(Map<String, Integer> stockBySku) {
    for (Map.Entry<String, Integer> e : stockBySku.entrySet()) {
        log.info("SKU {} has stock {}", e.getKey(), e.getValue());
    }
}
```

The interviewer's prompt: "Cleanest form for Java 21?"

Thinking out loud: 1. "Map has `forEach((k, v) -> ...)` since Java 8. No Entry handling." 2. "Two-parameter lambda makes the intent obvious."

The refactored version:

```
public void logAll(Map<String, Integer> stockBySku) {
    stockBySku.forEach((sku, stock) -> log.info("SKU {} has stock {}", sku, stock));
}
```

Why this is better: - Named lambda parameters self-document - No `getKey` / `getValue` ceremony - One line

What NOT to say: - "Convert to a stream first" — `entrySet().stream().forEach` is a longer way to do the same thing - "Use parallel `forEach`" — only if the map is huge and the action is independent

Common follow-up: "Is `Map.forEach` thread-safe on a `ConcurrentHashMap`?"

Scenario 21 · Anonymous inner class for a one-line Runnable

Asked at: TCS · Wipro · **Difficulty:** Easy · **Topic:** Lambda

The code:

```
executor.submit(new Runnable() {
    @Override
    public void run() {
        notifier.sendInvoice(customerId);
    }
});
```

The interviewer's prompt: "Java 21. This is from 2012."

Thinking out loud: 1. "Single-method functional interface — `Runnable`. Lambda replaces the entire anonymous class." 2. "Method reference if the body is just a call: `() -> notifier.sendInvoice(customerId)` is the lambda, `notifier::sendInvoice` if `customerId` were captured differently."

The refactored version:

```
executor.submit(() -> notifier.sendInvoice(customerId));
```

Why this is better: - 5 lines collapse to 1 - No anonymous-class allocation overhead (lambdas use invokedynamic) - Reads like the intent: submit a task that sends the invoice

What NOT to say: - "Inline the body into a method reference" — only works when the call shape matches the SAM exactly - "Use `Executors.newSingleThreadExecutor()` instead" — unrelated concern

Common follow-up: "What's the bytecode difference between an anonymous class and a lambda?"

Scenario 22 · Defensive copy of immutable List

Asked at: Cred · Razorpay · **Difficulty:** Medium · **Topic:** List.copyOf

The code:

```
public class Cart {
    private final List<Item> items;

    public Cart(List<Item> items) {
        this.items = new ArrayList<>(items);
    }

    public List<Item> getItems() {
        return Collections.unmodifiableList(new ArrayList<>(items));
    }
}
```

The interviewer's prompt: "Two copies on every read. Tighten this."

Thinking out loud: 1. "Defensive copy at construction makes sense — caller could mutate. But on the read path, returning an unmodifiable view of an internal copy is overkill." 2. " `List.copyOf` since Java 10 returns an immutable list — share it freely."

The refactored version:


```
public class Cart {
    private final List<Item> items;

    public Cart(List<Item> items) {
        this.items = List.copyOf(items); // immutable copy, no further defense needed
    }

    public List<Item> getItems() {
        return items; // already immutable
    }
}
```

Why this is better: - One copy on construction, zero on read - `List.copyOf` returns a more memory-efficient immutable list than `ArrayList` + wrapper - Callers cannot mutate, regardless of how they hold the reference

What NOT to say: - "Return a Stream from getItems" — forces callers to re-collect; bad API - "Skip the constructor copy" — caller can then mutate after constructing the Cart

Common follow-up: "What if Item itself is mutable?"

Scenario 23 · Visitor implemented with instanceof ladder

Asked at: Cred · PhonePe · **Difficulty:** Medium · **Topic:** Pattern matching

The code:

```
public BigDecimal price(Shape shape) {
    if (shape instanceof Circle) {
        Circle c = (Circle) shape;
        return BigDecimal.valueOf(c.getRadius() * c.getRadius() * Math.PI);
    } else if (shape instanceof Rectangle) {
        Rectangle r = (Rectangle) shape;
        return BigDecimal.valueOf(r.getWidth() * r.getHeight());
    } else if (shape instanceof Triangle) {
        Triangle t = (Triangle) shape;
        return BigDecimal.valueOf(0.5 * t.getBase() * t.getHeight());
    }
    throw new IllegalArgumentException("Unknown shape");
}
```

The interviewer's prompt: "Java 21. Make the compiler enforce coverage."

Thinking out loud: 1. "Two smells. The `cast-after-instanceof` is replaced by pattern matching (Java 16+). And the `throw` for unknown types means the compiler can't help me — sealed types fix that." 2. "Switch over a sealed interface in Java 21 — compiler ensures every permitted subtype is handled."

The refactored version:

```
public sealed interface Shape permits Circle, Rectangle, Triangle {}
public record Circle(double radius) implements Shape {}
public record Rectangle(double width, double height) implements Shape {}
public record Triangle(double base, double height) implements Shape {}

public BigDecimal price(Shape shape) {
    return BigDecimal.valueOf(switch (shape) {
        case Circle c    -> c.radius() * c.radius() * Math.PI;
        case Rectangle r -> r.width() * r.height();
        case Triangle t  -> 0.5 * t.base() * t.height();
    });
}
```

Why this is better: - Sealed + switch — adding a new shape becomes a compile error until handled - Pattern variable replaces cast - No `default` branch needed; exhaustiveness is proved at compile time

What NOT to say: - "Add a `default -> throw`" — defeats the exhaustiveness check - "Move price into each Shape subclass" — viable, but the switch is just as clean when the operation lives outside the data

Common follow-up: "When would you put `price` on each subtype instead of in a switch?"

Scenario 24 · Magic numbers in a discount calculator

Asked at: Flipkart · Myntra · **Difficulty:** Easy · **Topic:** Named constants

The code:

```
public BigDecimal computeDiscount(BigDecimal cartTotal, int loyaltyYears) {
    if (cartTotal.compareTo(new BigDecimal("5000")) > 0) {
        return cartTotal.multiply(new BigDecimal("0.10"));
    }
    if (loyaltyYears >= 3) {
        return cartTotal.multiply(new BigDecimal("0.05"));
    }
    return BigDecimal.ZERO;
}
```

The interviewer's prompt: "Three magic numbers. Pull them out."

Thinking out loud: 1. "5000, 0.10, 3, 0.05 — none of these are self-explaining. A reader has to guess what they mean." 2. "Named constants at the top. If they ever come from config, the rename touches one place."

The refactored version:

```
public class DiscountRules {
    private static final BigDecimal LARGE_CART_THRESHOLD = new BigDecimal("5000");
    private static final BigDecimal LARGE_CART_DISCOUNT = new BigDecimal("0.10");
    private static final int LOYALTY_YEARS_MIN = 3;
    private static final BigDecimal LOYALTY_DISCOUNT = new BigDecimal("0.05");

    public BigDecimal computeDiscount(BigDecimal cartTotal, int loyaltyYears) {
        if (cartTotal.compareTo(LARGE_CART_THRESHOLD) > 0) return cartTotal.multiply(LARGE_CART_DISCOUNT);
        if (loyaltyYears >= LOYALTY_YEARS_MIN) return cartTotal.multiply(LOYALTY_DISCOUNT);
        return BigDecimal.ZERO;
    }
}
```

Why this is better: - Names explain the rule without a comment - Changing the threshold is a one-line edit - Easier to expose via configuration later

What NOT to say: - "Move the numbers to a properties file" — premature; do it when the business asks - "Use an enum for the discount tiers" — only if tiers grow beyond two

Common follow-up: "At what point do you move these to runtime config?"

Scenario 25 · ExecutorService never shut down

Asked at: Swiggy · Razorpay · **Difficulty:** Medium · **Topic:** Resource lifecycle

The code:

```
public class ReportGenerator {
    private final ExecutorService pool = Executors.newFixedThreadPool(8);

    public void run(List<Report> reports) {
        for (Report r : reports) {
            pool.submit(() -> generate(r));
        }
    }
}
```

The interviewer's prompt: "Pool created, never closed. Plus the run method returns before tasks finish. Fix both."

Thinking out loud: 1. "No shutdown means JVM exit hangs forever — non-daemon threads keep it alive." 2. "Java 21 has structured concurrency for the wait-for-all-tasks problem. For owned pools, [try-with-resources](#) on `ExecutorService` (Java 19+) is the modern shape."

The refactored version:

```
public class ReportGenerator implements AutoCloseable {
    private final ExecutorService pool = Executors.newFixedThreadPool(8);

    public void run(List<Report> reports) throws InterruptedException, ExecutionException {
        List<Future<?>> futures = new ArrayList<>();
        for (Report r : reports) futures.add(pool.submit(() -> generate(r)));
        for (Future<?> f : futures) f.get(); // wait for completion
    }

    @Override public void close() {
        pool.shutdown();
        try { pool.awaitTermination(30, TimeUnit.SECONDS); }
        catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    }
}
```

Why this is better: - Owner can use `try-with-resources` to guarantee shutdown - `run` waits for completion — caller can rely on the work being done - 30s grace period before forced shutdown

What NOT to say: - "Make the pool static" — moves the leak from per-instance to forever - "Mark threads as daemon" — JVM exits without finishing reports; data loss

Common follow-up: "How would structured concurrency change this?"

Scenario 26 · Boolean parameter that toggles behaviour

Asked at: Razorpay · Cred · **Difficulty:** Medium · **Topic:** Naming / API design

The code:

```
public Invoice generate(Order order, boolean isDraft) {
    if (isDraft) {
        return new Invoice(order, "DRAFT", null);
    } else {
        return new Invoice(order, "FINAL", LocalDate.now());
    }
}

// Call site
generate(order, true); // what does true mean here?
```

The interviewer's prompt: "Boolean flag tells you nothing at the call site. Refactor."

Thinking out loud: 1. "`generate(order, true)`" is unreadable. Reader has to jump to the method to learn what `true` means." 2. "Split into two methods, or take an enum. Two methods are clearer for binary choices."

The refactored version:

```
public Invoice generateDraft(Order order) {
    return new Invoice(order, "DRAFT", null);
}

public Invoice generateFinal(Order order) {
    return new Invoice(order, "FINAL", LocalDate.now());
}

// Call site
generateDraft(order); // intent obvious
```

Why this is better: - Call site reads as English - Each method's body is shorter — no branch on the flag - If draft and final diverge further later, each method evolves independently

What NOT to say: - "Rename the parameter to `draft`" — improves the signature but the call site still reads `true` - "Pass an enum `DRAFT / FINAL`" — fine for three or more values; for two, methods are cleaner

Common follow-up: "What if there are three states?"

Scenario 27 · Try-finally instead of try-with-resources

Asked at: TCS · Infosys · **Difficulty:** Easy · **Topic:** Resource management

The code:

```
public void writeReport(String path, String content) throws IOException {
    FileWriter writer = null;
    try {
        writer = new FileWriter(path);
        writer.write(content);
    } finally {
        if (writer != null) {
            try {
                writer.close();
            } catch (IOException ignored) {}
        }
    }
}
```

The interviewer's prompt: "Modernize."

Thinking out loud: 1. "Three problems. Manual close. Swallowed exception on close. And FileWriter uses platform charset." 2. "try-with-resources for the close. `Files.writeString` skips the manual writer entirely."

The refactored version:

```
public void writeReport(String path, String content) throws IOException {
    Files.writeString(Path.of(path), content, StandardCharsets.UTF_8);
}
```

Why this is better: - One line replaces ten - Explicit UTF-8 — no surprises on Windows - Any `IOException` propagates with its full stack trace

What NOT to say: - "Wrap `FileWriter` in try-with-resources" — better than the original but `Files.writeString` is the modern call - "Swallow `IOException` so callers don't have to handle it" — hides write failures from the caller

Common follow-up: "What if you need to append, not overwrite?"

Scenario 28 · *Optional.get without isPresent*

Asked at: Flipkart · Myntra · **Difficulty:** Easy · **Topic:** Optional

The code:

```
public String customerEmail(Long userId) {  
    Optional<User> user = repo.findById(userId);  
    return user.get().getEmail();  
}
```

The interviewer's prompt: "NoSuchElementException waiting to happen. Multiple fixes — which one?"

Thinking out loud: 1. " `get()` without `isPresent` defeats Optional's purpose. If user is missing, this throws." 2. "What should happen on missing user? If 'caller's fault', throw a meaningful exception. If 'return empty', propagate the Optional."

The refactored version:

```
// If a missing user is a programmer error (e.g. id was already validated)  
public String customerEmail(Long userId) {  
    return repo.findById(userId)  
        .map(User::getEmail)  
        .orElseThrow(() -> new UserNotFoundException(userId));  
}  
  
// If a missing user is a normal outcome  
public Optional<String> customerEmail(Long userId) {  
    return repo.findById(userId).map(User::getEmail);  
}
```

Why this is better: - Meaningful exception beats NoSuchElementException at debug time - Or, return Optional so the caller can decide - Either way, no naked `.get()`

What NOT to say: - "Wrap in try-catch for NoSuchElementException" — using exceptions as flow control - "Return null on missing" — re-introduces the null check Optional was supposed to eliminate

Common follow-up: "When is `.orElseThrow()` without an argument acceptable?"

Scenario 29 · *Eager Optional.of(null) trap*

Asked at: Cred · Razorpay · **Difficulty:** Easy · **Topic:** Optional

The code:

```
public Optional<String> findNickname(Long userId) {  
    User user = repo.findOne(userId);  
    return Optional.of(user.getNickname()); // NPE on null nickname  
}
```

The interviewer's prompt: "This throws an NPE you didn't expect. Why, and what's the fix?"

Thinking out loud: 1. "Optional.of throws if the argument is null. Optional.ofNullable is the safe variant." 2. "Use Optional.ofNullable whenever the value could be null."

The refactored version:

```
public Optional<String> findNickname(Long userId) {  
    return Optional.ofNullable(repo.findOne(userId))  
        .map(User::getNickname);  
}
```

Why this is better: - ofNullable handles the null case cleanly - Chain handles missing user too - Method does what its name promises

What NOT to say: - "Use Optional.of after a null check" — extra code where ofNullable already does it - "Return empty string instead of empty Optional" — loses the distinction between 'no nickname set' and 'nickname is empty string'

Common follow-up: "What's the difference between Optional.empty() and Optional.ofNullable(null) ?"

Scenario 30 · Inheritance used for code reuse

Asked at: Flipkart · PhonePe · **Difficulty:** Hard · **Topic:** Composition

The code:

```
public class Stack<T> extends ArrayList<T> {  
    public void push(T item) { add(item); }  
    public T pop() { return remove(size() - 1); }  
    public T peek() { return get(size() - 1); }  
}
```


The interviewer's prompt: "This 'works' but breaks the LSP. Refactor."

Thinking out loud: 1. "Stack-extends-ArrayList means callers can do `stack.add(0, item)` — bypasses LIFO." 2. "Composition: hold an ArrayList, expose only the stack API."

The refactored version:

```
public class Stack<T> {
    private final ArrayList<T> items = new ArrayList<>();
    public void push(T item) { items.add(item); }
    public T pop() {
        if (items.isEmpty()) throw new NoSuchElementException();
        return items.remove(items.size() - 1);
    }
    public T peek() {
        if (items.isEmpty()) throw new NoSuchElementException();
        return items.get(items.size() - 1);
    }
    public int size() { return items.size(); }
    public boolean isEmpty() { return items.isEmpty(); }
}
```

Why this is better: - Callers can only push/pop/peek — no backdoor insertions - ArrayList stays an implementation detail; could swap to ArrayDeque later - Empty-stack contract explicit

What NOT to say: - "Use `java.util.Stack`" — that class itself extends Vector, same problem - "Override `add` to throw" — surprising behaviour for a List subclass

Common follow-up: "Why is `java.util.Stack` considered a mistake in the JDK?"

Scenario 31 · Repeated null check chain

Asked at: Razorpay · Swiggy · **Difficulty:** Medium · **Topic:** Optional chaining

The code:

```
public String city(User user) {  
    if (user != null) {  
        Address addr = user.getAddress();  
        if (addr != null) {  
            City c = addr.getCity();  
            if (c != null) {  
                return c.getName();  
            }  
        }  
    }  
    return "Unknown";  
}
```

The interviewer's prompt: "Three nested null checks. Refactor."

Thinking out loud: 1. "Classic train wreck. Each field could be null and the call drills three levels." 2.

" `Optional.ofNullable` + `map` chain expresses the same idea linearly."

The refactored version:

```
public String city(User user) {  
    return Optional.ofNullable(user)  
        .map(User::getAddress)  
        .map(Address::getCity)  
        .map(City::getName)  
        .orElse("Unknown");  
}
```

Why this is better: - Linear top-to-bottom flow - One `orElse` covers any null in the chain - Each `map` skips itself if the upstream is empty

What NOT to say: - "Use `?.` operator" — Java doesn't have it - "Catch the NPE" — flow control via exceptions

Common follow-up: "What's the cost of the intermediate `Optional` allocations?"

Scenario 32 · Switch with fall-through

Asked at: TCS · Infosys · **Difficulty:** Medium · **Topic:** Switch expression

The code:

```

public int daysInMonth(int month, int year) {
    int days;
    switch (month) {
        case 1: case 3: case 5: case 7: case 8: case 10: case 12:
            days = 31;
            break;
        case 4: case 6: case 9: case 11:
            days = 30;
            break;
        case 2:
            days = (year % 4 == 0 && (year % 100 != 0 || year % 400 == 0)) ? 29 : 28;
            break;
        default:
            throw new IllegalArgumentException();
    }
    return days;
}

```

The interviewer's prompt: "Fall-through plus mutable temp. Modernize."

Thinking out loud: 1. "Arrow form with comma-separated case labels covers the fall-through cleanly." 2. "Switch expression returns directly — no temp, no break."

The refactored version:

```

public int daysInMonth(int month, int year) {
    return switch (month) {
        case 1, 3, 5, 7, 8, 10, 12 -> 31;
        case 4, 6, 9, 11 -> 30;
        case 2 -> (year % 4 == 0 && (year % 100 != 0 || year % 400 == 0)) ? 29 : 28;
        default -> throw new IllegalArgumentException();
    };
}

```

Why this is better: - Comma syntax replaces stacked `case` s - No `break` s, no `days` variable - Expression form means the compiler complains if a case is missing

What NOT to say: - "Use `YearMonth.of(year, month).lengthOfMonth()`" — best answer for production, but the question is about modernizing switch - "Use a `Map<Integer, Integer>`" — allocates a map every call

Common follow-up: "What does Java do if you forget `default` in a switch expression?"

Scenario 33 · Stream collector for a count

Asked at: Infosys · TCS · **Difficulty:** Easy · **Topic:** Stream idiom

The code:

```
long activeCount = users.stream()
    .filter(User::isActive)
    .collect(Collectors.toList())
    .size();
```

The interviewer's prompt: "Why is this wasteful?"

Thinking out loud: 1. "Materializes the whole filtered list just to count it. Allocates an ArrayList of N elements for nothing." 2. "`.count()` is a terminal op for exactly this."

The refactored version:

```
long activeCount = users.stream()
    .filter(User::isActive)
    .count();
```

Why this is better: - No intermediate collection - O(1) memory instead of O(n) - Reads as the intent

What NOT to say: - "Use `Collectors.counting()`" — fine in a `groupingBy` downstream; overkill for a top-level count - "Use `users.size()` after filtering" — you'd still materialize the filter

Common follow-up: "What's the difference between `count()` and `size()` for a Collection?"

Scenario 34 · Builder pattern hand-written

Asked at: Razorpay · Flipkart · **Difficulty:** Medium · **Topic:** Records / Lombok

The code:

```

public class Customer {
    private final String name;
    private final String email;
    private final int age;
    private Customer(Builder b) {
        this.name = b.name; this.email = b.email; this.age = b.age;
    }
    public static class Builder {
        private String name, email;
        private int age;
        public Builder name(String n) { this.name = n; return this; }
        public Builder email(String e) { this.email = e; return this; }
        public Builder age(int a) { this.age = a; return this; }
        public Customer build() { return new Customer(this); }
    }
}

```

The interviewer's prompt: "Hand-rolled builder. Cleaner option?"

Thinking out loud: 1. "All fields required, no optional fields, no validation order. Builder is overkill — record is the right shape." 2. "If callers want named-argument-style construction, builder makes sense — but for three required fields, no."

The refactored version:

```

public record Customer(String name, String email, int age) {
    public Customer {
        Objects.requireNonNull(name);
        Objects.requireNonNull(email);
        if (age < 0) throw new IllegalArgumentException("age must be >= 0");
    }
}

// Caller
new Customer("Ravi", "ravi@example.com", 28);

```

Why this is better: - 17 lines collapse to 7 - equals/hashCode/toString free - Validation runs once, at construction

What NOT to say: - "Use `@Builder` from Lombok" — preserves the same complexity, just hides it - "Keep the builder because it's 'extensible'" — YAGNI

Common follow-up: "When is a real builder actually worth it?"

Scenario 35 · Mutable static state

Asked at: Cred · PhonePe · **Difficulty:** Medium · **Topic:** Singleton hygiene

The code:

```
public class FeatureFlags {
    public static Map<String, Boolean> flags = new HashMap<>();

    public static boolean isOn(String name) {
        return flags.getOrDefault(name, false);
    }
}
```

The interviewer's prompt: "Multiple things wrong. Fix all of them."

Thinking out loud: 1. "Public static mutable field — any caller can replace the entire map. Plus HashMap under concurrent reads/writes is unsafe." 2. "Hide the map, expose load and read operations, store in a thread-safe container — or better, swap the whole map atomically."

The refactored version:

```
public final class FeatureFlags {
    private FeatureFlags() {}
    private static final AtomicReference<Map<String, Boolean>> FLAGS =
        new AtomicReference<>(Map.of());

    public static boolean isOn(String name) {
        return FLAGS.get().getOrDefault(name, false);
    }

    public static void reload(Map<String, Boolean> next) {
        FLAGS.set(Map.copyOf(next)); // immutable snapshot
    }
}
```

Why this is better: - Reads are lock-free against an immutable snapshot - Reloads swap the whole map atomically — no torn reads - Public mutation gone; callers go through **reload**

What NOT to say: - "Switch to **ConcurrentHashMap**" — fixes reads/writes on the map but doesn't prevent callers replacing the field - "Wrap reads in **synchronized**" — kills read throughput

Common follow-up: "How would you load the flags on startup?"

Scenario 36 · Throwing Exception from API

Asked at: Infosys · TCS · **Difficulty:** Medium · **Topic:** Exception types

The code:

```
public Invoice fetch(String id) throws Exception {  
    String raw = http.get("/invoices/" + id);  
    return mapper.readValue(raw, Invoice.class);  
}
```

The interviewer's prompt: " `throws Exception` is the lazy choice. Tighten."

Thinking out loud: 1. "Caller must `catch (Exception)` or re-throw — loses every distinction between IO failure and JSON parse failure." 2. "Declare what actually flies. Or wrap in a domain exception."

The refactored version:

```
// Option A — declare the real checked types  
public Invoice fetch(String id) throws IOException {  
    String raw = http.get("/invoices/" + id);  
    return mapper.readValue(raw, Invoice.class);  
}  
  
// Option B — wrap in a domain exception  
public Invoice fetch(String id) {  
    try {  
        return mapper.readValue(http.get("/invoices/" + id), Invoice.class);  
    } catch (IOException e) {  
        throw new InvoiceFetchException(id, e);  
    }  
}
```

Why this is better: - Caller sees specific failure types and can react - Domain exception preserves context (`id`) in the stack trace - No more catch-all at every call site

What NOT to say: - "Change to `throws RuntimeException`" — `RuntimeException` doesn't need declaring; meaningless to declare it - "Catch and ignore" — silent data loss

Common follow-up: "Checked vs unchecked — what's your default?"

Scenario 37 · Manual JDBC try-finally

Asked at: TCS · Infosys · **Difficulty:** Medium · **Topic:** try-with-resources

The code:

```
public List<String> emails() throws SQLException {
    Connection c = null; Statement s = null; ResultSet r = null;
    try {
        c = ds.getConnection();
        s = c.createStatement();
        r = s.executeQuery("SELECT email FROM users");
        List<String> out = new ArrayList<>();
        while (r.next()) out.add(r.getString(1));
        return out;
    } finally {
        if (r != null) try { r.close(); } catch (SQLException ignored) {}
        if (s != null) try { s.close(); } catch (SQLException ignored) {}
        if (c != null) try { c.close(); } catch (SQLException ignored) {}
    }
}
```

The interviewer's prompt: "Java 21. Clean this."

Thinking out loud: 1. "Three resources, three nested closes, three swallowed exceptions. try-with-resources handles all three in one declaration." 2. "Close order is reverse-declaration — exactly what JDBC needs."

The refactored version:

```
public List<String> emails() throws SQLException {
    try (Connection c = ds.getConnection();
        Statement s = c.createStatement();
        ResultSet r = s.executeQuery("SELECT email FROM users")) {
        List<String> out = new ArrayList<>();
        while (r.next()) out.add(r.getString(1));
        return out;
    }
}
```

Why this is better: - All three closed correctly, in reverse order - Suppressed exceptions chained, not swallowed - 5 lines shorter, zero null checks

What NOT to say: - "Use JdbcTemplate" — best practice in Spring, but the question is about modernizing raw JDBC - "Catch SQLException and convert to runtime" — orthogonal concern

Common follow-up: "What's a 'suppressed exception' and where does it surface?"

Scenario 38 · Map iteration to build another Map

Asked at: Flipkart · Cred · **Difficulty:** Medium · **Topic:** Stream toMap

The code:

```
public Map<String, BigDecimal> taxBySku(Map<String, BigDecimal> priceBySku) {
    Map<String, BigDecimal> out = new HashMap<>();
    for (Map.Entry<String, BigDecimal> e : priceBySku.entrySet()) {
        out.put(e.getKey(), e.getValue().multiply(new BigDecimal("0.18")));
    }
    return out;
}
```

The interviewer's prompt: "Stream version."

Thinking out loud: 1. "Map.entrySet().stream()" with `Collectors.toMap` — same key, transformed value." 2. "GST rate is a magic number; name it."

The refactored version:

```
private static final BigDecimal GST_RATE = new BigDecimal("0.18");

public Map<String, BigDecimal> taxBySku(Map<String, BigDecimal> priceBySku) {
    return priceBySku.entrySet().stream()
        .collect(Collectors.toMap(
            Map.Entry::getKey,
            e -> e.getValue().multiply(GST_RATE)));
}
```

Why this is better: - Stream version composes with later filters or sort - Named constant explains the 0.18 - No mutable intermediate map

What NOT to say: - "Use `forEach` to mutate `out`" — same shape as before, no improvement - "Mutate the input map" — surprising side effect; callers expect input untouched

Common follow-up: "What happens if there are duplicate keys in `toMap`?"

Scenario 39 · Reflection used to avoid a switch

Asked at: PhonePe · **Cred:** · **Difficulty:** Hard · **Topic:** Polymorphism

The code:

```
public Object handle(String typeName, Object payload) throws Exception {
    Class<?> handlerClass = Class.forName("com.acme.handlers." + typeName + "Handler");
    Object handler = handlerClass.getDeclaredConstructor().newInstance();
    return handlerClass.getMethod("process", Object.class).invoke(handler, payload);
}
```

The interviewer's prompt: "Reflection-driven dispatch. What's wrong and what's the fix?"

Thinking out loud: 1. "Class.forName accepts any string — code injection if `typeName` is user-controlled. No compile-time check that the handler exists or that it takes the right type." 2. "Register handlers in a Map at startup. Type-safe, no reflection, no string-based attack surface."

The refactored version:

```
public interface Handler<T> { Object process(T payload); }

private final Map<String, Handler<Object>> handlers = Map.of(
    "Order",    payload -> orderHandler.process((Order) payload),
    "Payment",  payload -> paymentHandler.process((Payment) payload)
);

public Object handle(String typeName, Object payload) {
    Handler<Object> h = handlers.get(typeName);
    if (h == null) throw new IllegalArgumentException("Unknown type: " + typeName);
    return h.process(payload);
}
```

Why this is better: - No reflection — code is JIT-friendly and grep-friendly - Unknown type fails with a clear message instead of `ClassNotFoundException` - Adding a handler is a one-line registration

What NOT to say: - "Add a whitelist of allowed type names" — bandaid for the security hole; the design problem remains - "Cache the reflected method" — caches the wrong abstraction

Common follow-up: "How would you wire up handlers in Spring?"

Scenario 40 · Equality on a mutable cache key

Asked at: Razorpay · **Cred:** · **Difficulty:** Medium · **Topic:** Immutability

The code:

```
public class CacheKey {
    public String region;
    public String resource;
    @Override public boolean equals(Object o) { /* compares fields */ return ...; }
    @Override public int hashCode() { return Objects.hash(region, resource); }
}

Map<CacheKey, String> cache = new HashMap<>();
CacheKey k = new CacheKey("ap-south-1", "user-42");
cache.put(k, "value");
k.region = "us-east-1"; // accident
cache.get(new CacheKey("ap-south-1", "user-42")); // null
```

The interviewer's prompt: "The cache lost the entry. Why and how do you prevent it?"

Thinking out loud: 1. "Mutable fields used in hashCode — when you mutate after insertion, the bucket is wrong forever. The entry is orphaned." 2. "Cache keys must be immutable. Record fits perfectly."

The refactored version:

```
public record CacheKey(String region, String resource) {}

Map<CacheKey, String> cache = new HashMap<>();
cache.put(new CacheKey("ap-south-1", "user-42"), "value");
// Caller can't mutate — has to construct a new one
```

Why this is better: - Fields are final; mutation is impossible - Record gives equals/hashCode that respect the contract automatically - Bucket location stays consistent for the lifetime of the key

What NOT to say: - "Tell callers not to mutate after `put`" — convention, not enforcement; breaks at 2 AM - "Recompute hashCode on every call" — defeats the cache that HashMap relies on

Common follow-up: "What's the relationship between equals/hashCode and immutability?"

Scenario 41 · String split into a list

Asked at: TCS · Wipro · **Difficulty:** Easy · **Topic:** Stream creation

The code:

```
public List<String> tags(String csv) {
    String[] parts = csv.split(",");
    List<String> result = new ArrayList<>();
    for (String p : parts) {
        result.add(p.trim());
    }
    return result;
}
```

The interviewer's prompt: "Stream version, please."

Thinking out loud: 1. "split into a Stream via `Stream.of` or `Arrays.stream`. Trim each, collect to list." 2. "Drop empty entries while we're at it — `\",,b\"` shouldn't produce a blank tag."

The refactored version:

```
public List<String> tags(String csv) {
    return Arrays.stream(csv.split(","))
        .map(String::trim)
        .filter(s -> !s.isEmpty())
        .toList();
}
```

Why this is better: - Pipeline expresses intent: split, trim, drop empties, collect - One pass, no mutable accumulator - Empty-token edge case handled explicitly

What NOT to say: - "Use `csv.split(\"\\\\s*,\\\\s*\")` " — handles whitespace but not empty tokens, and the regex is harder to read - "Use `StringTokenizer` " — legacy API, doesn't stream

Common follow-up: "What does `split` do with trailing empty strings by default?"

Scenario 42 · Conditional logging without guard

Asked at: Razorpay · PhonePe · **Difficulty:** Easy · **Topic:** Logging

The code:

```
log.debug("Processing order " + order.getId() + " with " + items.size()
    + " items, total " + computeExpensiveTotal(order));
```

The interviewer's prompt: "This always pays the cost, even when debug is off. Two fixes."

Thinking out loud: 1. "String concatenation happens before the log call — JVM builds the string even if log level filters it out. `computeExpensiveTotal` also always runs." 2. "SLF4J parameterized format defers concatenation. For the expensive computation, lazy `Supplier` (SLF4J 2.x) or an `isDebugEnabled` guard."

The refactored version:

```
// Option A — parameterized, computation still eager
log.debug("Processing order {} with {} items, total {}",
    order.getId(), items.size(), computeExpensiveTotal(order));

// Option B — lazy with isDebugEnabled when computation is expensive
if (log.isDebugEnabled()) {
    log.debug("Processing order {} with {} items, total {}",
        order.getId(), items.size(), computeExpensiveTotal(order));
}
```

Why this is better: - Parameterized form skips concatenation when the level is off - `isDebugEnabled` guard skips `computeExpensiveTotal` entirely - Standard SLF4J idiom

What NOT to say: - "Set log level to INFO and forget it" — debug logs are valuable, just need to cost zero when off - "Use `String.format`" — same eager concatenation problem

Common follow-up: "What does SLF4J 2.x give you that 1.x doesn't?"

Scenario 43 · Class with only static methods

Asked at: Cred · Razorpay · **Difficulty:** Easy · **Topic:** Final + private constructor

The code:

```
public class StringUtils {
    public static boolean isBlank(String s) { return s == null || s.isBlank(); }
    public static String reverse(String s) { /* ... */ }
}
```

The interviewer's prompt: "Anything missing?"

Thinking out loud: 1. "Utility class can be instantiated — `new StringUtils()` is legal and meaningless. Adds confusion in IDE autocomplete." 2. "Mark class `final`, give it a private constructor."

The refactored version:

```
public final class StringUtils {
    private StringUtils() {
        throw new UnsupportedOperationException("Utility class");
    }
    public static boolean isBlank(String s) { return s == null || s.isBlank(); }
    public static String reverse(String s) { /* ... */ }
}
```

Why this is better: - `final` blocks subclassing; nobody can extend a utility for no reason - Private constructor blocks instantiation - Throw in the constructor catches reflection-based instantiation attempts

What NOT to say: - "Make every method synchronized" — utility methods are typically pure; sync is unnecessary - "Convert to an interface with static methods" — interfaces still can't have private constructors

Common follow-up: "When should a utility class become a real object with instance state?"

Scenario 44 · Inheritance chain three levels deep

Asked at: Infosys · TCS · **Difficulty:** Hard · **Topic:** Composition over inheritance

The code:

```
public abstract class Animal { void breathe() { ... } }
public abstract class Mammal extends Animal { void giveBirth() { ... } }
public abstract class Carnivore extends Mammal { void hunt() { ... } }
public class Tiger extends Carnivore { void roar() { ... } }
public class Cat extends Carnivore { void purr() { ... } } // doesn't really "hunt" the same way
```

The interviewer's prompt: "This taxonomy keeps growing. The hierarchy is brittle. Refactor."

Thinking out loud: 1. "Deep inheritance forces every leaf to inherit traits it might not have — Cat extends Carnivore but doesn't hunt big game." 2. "Compose traits. Each capability is a small interface; classes implement what they actually do."

The refactored version:

```

public interface Breathing { void breathe(); }
public interface Birthing { void giveBirth(); }
public interface Hunting { void hunt(); }
public interface Vocal { void vocalize(); }

public class Tiger implements Breathing, Birthing, Hunting, Vocal {
    public void breathe() { ... }
    public void giveBirth() { ... }
    public void hunt() { ... }
    public void vocalize() { /* roar */ }
}

public class Cat implements Breathing, Birthing, Vocal {
    public void breathe() { ... }
    public void giveBirth() { ... }
    public void vocalize() { /* purr */ }
}

```

Why this is better: - Each class advertises exactly what it can do - Adding a new capability doesn't reshape the hierarchy - Default methods on interfaces share implementation where it makes sense

What NOT to say: - "Make Cat extend Mammal directly, skip Carnivore" — patches one symptom, the design's still rigid - "Use a Trait library" — Java's interface defaults are already trait-like

Common follow-up: "When is deep inheritance the right answer?"

Scenario 45 · Hashing strings manually

Asked at: Razorpay · Cred · **Difficulty:** Medium · **Topic:** MessageDigest

The code:

```

public String md5(String input) {
    int hash = 0;
    for (int i = 0; i < input.length(); i++) {
        hash = 31 * hash + input.charAt(i);
    }
    return Integer.toHexString(hash);
}

```

The interviewer's prompt: "This is called md5 but it's not MD5. Fix the name and the implementation."

Thinking out loud: 1. "That's `String.hashCode()` — 32-bit polynomial. Not MD5. The method name lies." 2. "Use `MessageDigest`. And honestly, MD5 is broken — SHA-256 is the modern default."

The refactored version:

```
public String sha256(String input) {
    try {
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        byte[] digest = md.digest(input.getBytes(StandardCharsets.UTF_8));
        return HexFormat.of().formatHex(digest);
    } catch (NoSuchAlgorithmException e) {
        throw new IllegalStateException("SHA-256 missing from JVM", e);
    }
}
```

Why this is better: - Actual cryptographic hash, not a 32-bit polynomial - `HexFormat.of()` (Java 17+) replaces hand-rolled hex conversion - UTF-8 explicit — same result on every platform

What NOT to say: - "Use MD5 because the method is called md5" — the name was wrong; the right fix is the algorithm - "Catch and ignore `NoSuchAlgorithmException`" — SHA-256 is mandatory; if it's missing, the JVM is broken

Common follow-up: "Why is MD5 considered broken?"

Scenario 46 · Comparator chain reimplemented

Asked at: Flipkart · Myntra · **Difficulty:** Easy · **Topic:** `Comparator.comparing`

The code:

```
Collections.sort(users, new Comparator<User>() {
    @Override
    public int compare(User a, User b) {
        int c = a.getCity().compareTo(b.getCity());
        if (c != 0) return c;
        return a.getName().compareTo(b.getName());
    }
});
```

The interviewer's prompt: "Modernize."

Thinking out loud: 1. "Anonymous Comparator with hand-rolled chain. `Comparator.comparing` and `thenComparing` chain naturally." 2. " `List.sort` over `Collections.sort` for instance-method form."

The refactored version:


```
users.sort(Comparator
    .comparing(User::getCity)
    .thenComparing(User::getName));
```

Why this is better: - 7 lines collapse to 3 - Chain reads as the sort intent: by city, then by name - Reverse, nulls-first, nulls-last all available via static helpers

What NOT to say: - "Use a Lambda inside the existing Comparator" — incomplete modernization - "Sort twice, once by each field" — wrong; second sort loses first ordering when keys collide

Common follow-up: "How does `Comparator.nullsFirst` plug into this chain?"

Scenario 47 · `Stream.collect` for joining strings

Asked at: Infosys · TCS · **Difficulty:** Easy · **Topic:** Collectors.joining

The code:

```
StringBuilder sb = new StringBuilder();
for (User u : users) {
    if (sb.length() > 0) sb.append(", ");
    sb.append(u.getName());
}
String csv = sb.toString();
```

The interviewer's prompt: "Stream + Collectors."

Thinking out loud: 1. "Map to name, join with comma. `Collectors.joining` handles the separator-only-between-elements problem." 2. "Prefix/suffix variant also exists, useful for `[a, b, c]` style output."

The refactored version:

```
String csv = users.stream()
    .map(User::getName)
    .collect(Collectors.joining(", "));
```

Why this is better: - No separator-bookkeeping - Stream pipeline composes with `filter` if needed later - Same allocation profile as the `StringBuilder` version

What NOT to say: - "Use `String.join("\", \", ...)` " — works if you already have a List; for the stream case `Collectors.joining` is cleaner - "Concatenate with `+=` " — quadratic and we already covered that

Common follow-up: "What's the prefix/suffix overload useful for?"

Scenario 48 · Volatile flag for thread coordination

Asked at: Cred · Razorpay · **Difficulty:** Hard · **Topic:** Modern concurrency

The code:

```
public class Worker implements Runnable {
    private volatile boolean running = true;

    public void run() {
        while (running) {
            doWork();
        }
    }

    public void stop() { running = false; }
}
```

The interviewer's prompt: "Works but blocks on `doWork` . Modern shape?"

Thinking out loud: 1. "volatile boolean is the classic pre-Java 5 pattern. Modern idiom uses interruption — `Thread.currentThread().isInterrupted()` ." 2. "Interruption integrates with blocking operations (sleep, wait, IO) so the loop can stop mid-work."

The refactored version:

```

public class Worker implements Runnable {
    public void run() {
        while (!Thread.currentThread().isInterrupted()) {
            try {
                doWork(); // doWork should propagate InterruptedException, not swallow it
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
        }
    }
}
// Caller stops via thread.interrupt() or ExecutorService.shutdownNow()

```

Why this is better: - Interruption stops blocking calls inside `doWork` - `ExecutorService` integrates with interruption out of the box - No custom volatile field to remember

What NOT to say: - "Add `Thread.sleep(1)` in the loop" — wastes cycles, doesn't fix the blocking concern - "Use `Thread.stop()`" — deprecated since the late 1990s, unsafe

Common follow-up: "Why does `Thread.interrupted()` clear the flag but `isInterrupted()` doesn't?"

Scenario 49 · Catching and rethrowing without cause

Asked at: TCS · Infosys · **Difficulty:** Easy · **Topic:** Exception chaining

The code:

```

public Invoice load(String id) {
    try {
        return repo.findById(id);
    } catch (SQLException e) {
        throw new RuntimeException("Failed to load invoice");
    }
}

```

The interviewer's prompt: "You'll see this and curse the dev. Why?"

Thinking out loud: 1. "RuntimeException without the cause — the original SQLException, with its stack trace and DB error message, is lost forever." 2. "Always pass the cause."

The refactored version:

```
public Invoice load(String id) {
    try {
        return repo.findById(id);
    } catch (SQLException e) {
        throw new InvoiceLoadException("Failed to load invoice " + id, e);
    }
}
```

Why this is better: - Stack trace includes the SQL error - Domain exception type (*InvoiceLoadException*) lets callers handle it specifically - Id in the message makes the log line greppable

What NOT to say: - "Log the exception then throw a new one" — duplicate stack traces, double the noise - "Just rethrow as `RuntimeException(e)`" — preserves cause but loses domain meaning

Common follow-up: "What's the difference between `Throwable.getCause` and `Throwable.getSuppressed`?"

Scenario 50 · Singleton with public static field

Asked at: Razorpay · Cred · **Difficulty:** Medium · **Topic:** Modern singleton

The code:

```
public class ConfigCache {
    public static ConfigCache INSTANCE = new ConfigCache();
    private Map<String, String> config = new HashMap<>();
    private ConfigCache() { /* load */ }
}
```

The interviewer's prompt: "Three problems. Find them."

Thinking out loud: 1. "Public field — anyone can reassign INSTANCE. Mutable map. And HashMap isn't thread-safe." 2. "Modern shape: enum singleton, or a record with a final *ConcurrentMap*, or a Spring bean."

The refactored version:

```
public enum ConfigCache {
    INSTANCE;
    private final ConcurrentMap<String, String> config = new ConcurrentHashMap<>();
    ConfigCache() { load(); }
    public String get(String key) { return config.get(key); }
    public void set(String key, String value) { config.put(key, value); }
    private void load() { /* ... */ }
}
```

Why this is better: - Enum guarantees one instance, serialization-safe, reflection-resistant - INSTANCE reference is intrinsically final - ConcurrentHashMap handles concurrent reads and writes

What NOT to say: - "Use double-checked locking" — works since Java 5 with volatile, but enum is simpler and harder to break - "Inject via Spring" — fine in a Spring app; the pattern still needs to be safe by itself

Common follow-up: "Why is enum the JVM's safest singleton form?"

Scenario 51 · String concatenation in SQL

Asked at: Infosys · TCS · **Difficulty:** Medium · **Topic:** PreparedStatement

The code:

```
public List<User> findByCity(String city) throws SQLException {
    String sql = "SELECT * FROM users WHERE city = '" + city + "'";
    try (Statement s = conn.createStatement();
        ResultSet r = s.executeQuery(sql)) {
        // ...
    }
}
```

The interviewer's prompt: "Two things wrong. Fix both."

Thinking out loud: 1. "SQL injection — any caller can pass `city = \"' OR 1=1 --\"` and dump the table." 2. "PreparedStatement with `?` placeholder fixes both injection and value escaping (apostrophes in legitimate inputs)."

The refactored version:

```
public List<User> findByCity(String city) throws SQLException {
    String sql = "SELECT * FROM users WHERE city = ?";
    try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, city);
        try (ResultSet r = ps.executeQuery()) {
            // ...
        }
    }
}
```

Why this is better: - SQL injection impossible — values bound separately from SQL text - Driver handles escaping for special characters - Statement plan is cached when the prepared statement is reused

What NOT to say: - "Escape apostrophes manually" — DIY escaping always loses to a clever payload - "Use a stored procedure" — defense in depth, but PreparedStatement is the actual fix

Common follow-up: "How does the driver handle PreparedStatement caching?"

Scenario 52 · Custom thread instead of ExecutorService

Asked at: Cred · PhonePe · **Difficulty:** Medium · **Topic:** Concurrency primitives

The code:

```
public void emailAll(List<User> users) {
    for (User u : users) {
        new Thread(() -> emailService.send(u)).start();
    }
}
```

The interviewer's prompt: "For 1,000 users this spawns 1,000 threads. Fix."

Thinking out loud: 1. "Unbounded thread creation — at scale, each thread eats ~1MB of stack. 1,000 users = ~1GB just for stacks." 2. "ExecutorService caps concurrency. Or Java 21 virtual threads if the work is IO-bound."

The refactored version:

```
public void emailAll(List<User> users) {
    // Java 21 virtual threads – millions of cheap threads for IO-bound work
    try (var exec = Executors.newVirtualThreadPerTaskExecutor()) {
        users.forEach(u -> exec.submit(() -> emailService.send(u)));
    } // try-with-resources awaits termination
}

// Or, pre-21 with bounded platform thread pool
ExecutorService pool = Executors.newFixedThreadPool(16);
users.forEach(u -> pool.submit(() -> emailService.send(u)));
```

Why this is better: - Virtual threads handle IO blocking efficiently — thousands without OS-thread cost - Bounded pool caps memory and connection-pool pressure - try-with-resources guarantees the executor is closed

What NOT to say: - "Add `Thread.sleep` between spawns to slow it down" — sleep doesn't reduce peak count - "Use `parallelStream`" — uses common ForkJoinPool, sized to CPUs, blocking calls choke other parallel work

Common follow-up: "When are virtual threads the wrong choice?"

Scenario 53 · Stream collect to a specific list type

Asked at: Flipkart · Cred · **Difficulty:** Easy · **Topic:** Stream collectors

The code:

```
LinkedList<String> names = users.stream()
    .map(User::getName)
    .collect(Collectors.toCollection(LinkedList::new));
```

The interviewer's prompt: "Why specifically a LinkedList here?"

Thinking out loud: 1. "LinkedList chosen 'just because'. Almost certainly the caller doesn't actually need O(1) head/tail inserts." 2. "Default to `toList()` (immutable) or `ArrayList` if mutation is needed."

The refactored version:

```
// If you don't mutate
List<String> names = users.stream().map(User::getName).toList();

// If you do
List<String> names = users.stream().map(User::getName).collect(Collectors.toCollection(ArrayList::new));
```

Why this is better: - ArrayList wins on cache locality and memory per element - `toList()` gives an unmodifiable list — defensive by default - LinkedList only earns its keep on deque workloads, and ArrayDeque is usually better even there

What NOT to say: - "LinkedList is faster for inserts" — only if you have the node reference; otherwise it's still $O(n)$ - "Switch to `Collectors.toUnmodifiableList()`" — fine, but `.toList()` is shorter

Common follow-up: "When does LinkedList beat ArrayList in practice?"

Scenario 54 · Static factory hidden behind a public constructor

Asked at: Razorpay · Cred · **Difficulty:** Medium · **Topic:** API design

The code:

```
public class Money {
    public Money(BigDecimal amount, Currency currency) { ... }
    public Money(double amount, String currencyCode) { ... }
    public Money(String amount, String currencyCode) { ... }
    public Money(long minorUnits, Currency currency) { ... }
}
```

The interviewer's prompt: "Four overloaded constructors. Caller can't tell which is which. Refactor."

Thinking out loud: 1. "Constructors with the same arity but different meaning are confusing — `new Money(100, ...)` could be minor units or major units." 2. "Named static factories make intent explicit."

The refactored version:


```
public final class Money {
    private final BigDecimal amount;
    private final Currency currency;

    private Money(BigDecimal amount, Currency currency) { ... }

    public static Money of(BigDecimal amount, Currency currency) { return new Money(amount, currency); }
    public static Money ofMinor(long minorUnits, Currency currency) {
        return new Money(BigDecimal.valueOf(minorUnits, currency.getDefaultFractionDigits()), currency);
    }
    public static Money parse(String amount, String currencyCode) {
        return new Money(new BigDecimal(amount), Currency.getInstance(currencyCode));
    }
}
```

Why this is better: - Caller writes `Money.ofMinor(10000, INR)` vs `Money.of(...)` — intent in the method name - Static factories can cache or return subtypes - Private constructor blocks the ambiguous direct-construction path

What NOT to say: - "Add Javadoc to each constructor" — readers don't always read Javadoc - "Pick one constructor and delete the others" — loses functionality

Common follow-up: "What's the JDK's example of this pattern?"

Scenario 55 · Composite key as concatenated string

Asked at: Flipkart · Myntra · **Difficulty:** Medium · **Topic:** Value object

The code:

```
Map<String, BigDecimal> stock = new HashMap<>();
stock.put(warehouseId + ":" + skuId, quantity);
// Later
BigDecimal q = stock.get(warehouseId + ":" + skuId);
```

The interviewer's prompt: "Composite key as a String. What's the smell and what's the fix?"

Thinking out loud: 1. "What if `warehouseId` contains a colon? Collision. Also, the format is duplicated across every call site — change the separator and you break everything." 2. "A record key captures the composite cleanly."

The refactored version:

```
public record StockKey(String warehouseId, String skuId) {}

Map<StockKey, BigDecimal> stock = new HashMap<>();
stock.put(new StockKey(warehouseId, skuId), quantity);
BigDecimal q = stock.get(new StockKey(warehouseId, skuId));
```

Why this is better: - Equality is field-by-field; no separator collision possible - Type-safe — caller can't pass two warehouse ids by accident - equals/hashCode generated correctly

What NOT to say: - "Use a different separator like `\\u0001`" — works until your input contains it - "Use a `Map<String, Map<String, BigDecimal>>`" — nested maps are awkward to iterate

Common follow-up: "When would you store separate maps instead of a composite-key one?"

Scenario 56 · Manual array copy

Asked at: TCS · Infosys · **Difficulty:** Easy · **Topic:** Arrays utility

The code:

```
public int[] firstN(int[] source, int n) {
    int[] out = new int[n];
    for (int i = 0; i < n; i++) {
        out[i] = source[i];
    }
    return out;
}
```

The interviewer's prompt: "Modern call."

Thinking out loud: 1. "Hand-rolled copy. `Arrays.copyOf` does this in one call, uses `System.arraycopy` internally which is JIT-optimized." 2. "Edge case: if `n > source.length`, we'd OOB. `Arrays.copyOf` pads with zeros instead."

The refactored version:

```
public int[] firstN(int[] source, int n) {
    return Arrays.copyOf(source, n);
}
```

Why this is better: - One call, uses native memcpy under the hood - Handles `n > source.length` by padding with 0 - Zero off-by-one risk

What NOT to say: - "Use `System.arraycopy` directly" — works, but `Arrays.copyOf` wraps it with the allocation - "Stream the array and collect" — boxes every int, allocates an Integer per element

Common follow-up: "What's the difference between `Arrays.copyOf` and `Arrays.copyOfRange`?"

Scenario 57 · Wildcard imports

Asked at: TCS · Infosys · **Difficulty:** Easy · **Topic:** Imports hygiene

The code:

```
import java.util.*;
import java.io.*;
import java.nio.file.*;
import com.acme.*;
```

The interviewer's prompt: "Wildcards. Why is this bad and what's the policy?"

Thinking out loud: 1. "Wildcard imports hide what you actually depend on. Two classes with the same name from different packages cause subtle compile errors when one of them moves." 2. "Explicit imports — IDE handles them, and they read as documentation."

The refactored version:

```
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import com.acme.Order;
import com.acme.Customer;
```

Why this is better: - Reader sees the dependency surface at a glance - Adding a new class to `java.util` doesn't risk a shadowing collision - Most IDEs (IntelliJ, Eclipse) auto-collapse wildcards on save when configured

What NOT to say: - "Wildcards are fine, just configure the IDE" — fine for personal projects, fails on shared codebases - "Star imports speed up compilation" — measurable difference is zero

Common follow-up: "When is `import static` acceptable?"

Scenario 58 · Boolean array as flags

Asked at: Cred · Razorpay · **Difficulty:** Medium · **Topic:** EnumSet

The code:

```
public class Order {
    private boolean[] flags = new boolean[10];
    // flags[0] = paid, flags[1] = shipped, flags[2] = refunded, ...

    public void markPaid()    { flags[0] = true; }
    public boolean isPaid()   { return flags[0]; }
    public void markShipped() { flags[1] = true; }
}
```

The interviewer's prompt: "Magic indices. Refactor."

Thinking out loud: 1. "Indices into a boolean array — typo and you flip the wrong flag. No safety." 2. "EnumSet of an enum — type-safe, dense bitset under the hood."

The refactored version:

```
public class Order {
    public enum Flag { PAID, SHIPPED, REFUNDED, CANCELLED, RETURNED, DISPUTED, FLAGGED }
    private final EnumSet<Flag> flags = EnumSet.noneOf(Flag.class);

    public void mark(Flag f)    { flags.add(f); }
    public void unmark(Flag f)  { flags.remove(f); }
    public boolean has(Flag f)  { return flags.contains(f); }
}
```

Why this is better: - Caller writes `order.mark(PAID)` — typo doesn't compile - EnumSet is a bitset internally — same memory as a long - Iteration order matches enum declaration order

What NOT to say: - "Use named integer constants" — fixes the typo problem but still allows arbitrary ints - "Use a HashSet" — string typos are also possible

Common follow-up: "How does EnumSet stay so memory-efficient?"

Scenario 59 · Comparator with Integer compareTo

Asked at: Flipkart · Myntra · **Difficulty:** Easy · **Topic:** Modern comparator

The code:

```
items.sort((a, b) -> a.getPriority() - b.getPriority());
```

The interviewer's prompt: "Subtle bug. Find and fix."

Thinking out loud: 1. "Subtraction-based compare overflows on extremes — `Integer.MIN_VALUE - 1` wraps to `Integer.MAX_VALUE`, breaking the sort contract." 2. "Use `Integer.compare(a, b)` or `Comparator.comparingInt`."

The refactored version:

```
items.sort(Comparator.comparingInt(Item::getPriority));
```

Why this is better: - No integer overflow risk - Method reference reads as the sort key - Chains cleanly with `.thenComparing`

What NOT to say: - "Wrap in `Math.abs`" — doesn't fix the contract violation, masks it - "Convert to long first" — works but `Integer.compare` is the standard idiom

Common follow-up: "What does `Comparable`'s contract say about sign consistency?"

Scenario 60 · Reading System.getProperty without default

Asked at: Razorpay · PhonePe · **Difficulty:** Easy · **Topic:** Defensive defaults

The code:

```
int timeout = Integer.parseInt(System.getProperty("http.timeout"));
```

The interviewer's prompt: "Production-unsafe. Fix."

Thinking out loud: 1. "If the property isn't set, `getProperty` returns null, `parseInt(null)` throws `NumberFormatException`, app dies at startup." 2. "Provide a default. And use the typed property reader."

The refactored version:

```
int timeout = Integer.getInteger("http.timeout", 30_000); // default 30s
```

Why this is better: - `Integer.getInteger` reads the system property and parses it in one call - Default applied when the property is missing or unparseable - Underscore in the literal is a readability win (Java 7+)

What NOT to say: - "Wrap in try-catch" — works but you still need a default value - "Throw if missing" — for a timeout, default is friendlier than crash

Common follow-up: "What does `Integer.getInteger` do that `Integer.parseInt(System.getProperty(...))` doesn't?"

Scenario 61 · Vector and Hashtable in modern code

Asked at: TCS · Cognizant · **Difficulty:** Easy · **Topic:** Legacy collections

The code:

```
Vector<String> names = new Vector<>();  
Hashtable<String, Integer> stock = new Hashtable<>();
```

The interviewer's prompt: "2026 codebase. Replace."

Thinking out loud: 1. "Vector and Hashtable are pre-JCF legacy — every method synchronized, kills throughput, and the locking is coarse-grained." 2. "ArrayList / HashMap for single-threaded. For concurrent, ConcurrentHashMap and CopyOnWriteArrayList depending on read/write mix."

The refactored version:

```
// Single-threaded
List<String> names = new ArrayList<>();
Map<String, Integer> stock = new HashMap<>();

// Concurrent
List<String> names = new CopyOnWriteArrayList<>(); // read-heavy
ConcurrentMap<String, Integer> stock = new ConcurrentHashMap<>();
```

Why this is better: - Modern collections have better algorithms (treeified buckets, etc.) - Concurrent options offer fine-grained locking instead of every-method-synchronized - Cleaner API — `entrySet` / `keySet` views, default methods

What NOT to say: - "Vector is fine because it's thread-safe" — coarse locking is rarely the right kind of thread-safe - "Wrap ArrayList in `Collections.synchronizedList`" — bandaid for non-existent problem in 99% of code

Common follow-up: "When was Vector deprecated?"

Scenario 62 · String.format for log lines

Asked at: Infosys · TCS · **Difficulty:** Easy · **Topic:** Logging

The code:

```
log.info(String.format("User %s logged in from %s at %d", user, ip, ts));
```

The interviewer's prompt: "SLF4J idiom."

Thinking out loud: 1. "String.format runs every call, even if the log level filters it out." 2. "SLF4J's `{}` parameterized format defers concatenation."

The refactored version:

```
log.info("User {} logged in from {} at {}", user, ip, ts);
```

Why this is better: - SLF4J skips formatting when level is off - Slightly faster even when level is on — no Formatter overhead - Standard SLF4J idiom every Java dev recognises

What NOT to say: - "Use `+` concatenation" — same eager-evaluation problem - "Wrap in `isEnabled` check" — overkill for a parameterized log; reserve for expensive computations in the argument list

Common follow-up: "What if you need locale-specific formatting?"

Scenario 63 · Method that returns null instead of empty collection

Asked at: Razorpay · Cred · **Difficulty:** Easy · **Topic:** Defensive return values

The code:

```
public List<Order> ordersFor(String userId) {
    List<Order> result = repo.find(userId);
    if (result.isEmpty()) {
        return null;
    }
    return result;
}
```

The interviewer's prompt: "Caller has to null-check, then size-check. Why?"

Thinking out loud: 1. "No-data case should return an empty collection, not null. Forcing callers to null-check is hostile." 2. "`List.of()` is an immutable empty list — cheap, shared, allocation-free."

The refactored version:

```
public List<Order> ordersFor(String userId) {
    return repo.find(userId); // assume repo returns empty list, not null
}

// If repo could return null
public List<Order> ordersFor(String userId) {
    List<Order> result = repo.find(userId);
    return result == null ? List.of() : result;
}
```

Why this is better: - Caller can iterate without a null check - `List.of()` returns a singleton empty list — no allocation cost - Removes a class of NPE bugs

What NOT to say: - "Return `Optional<List<Order>>`" — `Optional-of-collection` is the documented anti-pattern; an empty list expresses the same thing - "Throw if empty" — empty is a normal outcome, not an exception

Common follow-up: "Why is `Optional` considered an anti-pattern?"

Scenario 64 · `Iterator.remove` vs `removeIf`

Asked at: Flipkart · Cred · **Difficulty:** Easy · **Topic:** Modern collection ops

The code:

```
Iterator<Order> it = orders.iterator();
while (it.hasNext()) {
    Order o = it.next();
    if (o.isCancelled()) {
        it.remove();
    }
}
```

The interviewer's prompt: "One-liner with Java 8+."

Thinking out loud: 1. "This is the classic iterator-remove dance. `Collection.removeIf` since Java 8 does the same in one call." 2. "Implementations like `ArrayList` specialize `removeIf` to avoid $O(n^2)$ — important on large lists."

The refactored version:

```
orders.removeIf(Order::isCancelled);
```

Why this is better: - One line replaces six - `ArrayList`'s specialized `removeIf` is $O(n)$, not $O(n^2)$ - Predicate is the intent — no iterator bookkeeping

What NOT to say: - "Use `for` loop with index, decrement on remove" — error-prone and quadratic - "Stream-filter to a new list and reassign" — works but allocates a whole new list

Common follow-up: "Why does the indexed-for-loop approach get $O(n^2)$?"

Scenario 65 · BigDecimal constructor with double

Asked at: Razorpay · Cred · **Difficulty:** Medium · **Topic:** BigDecimal

The code:

```
BigDecimal price = new BigDecimal(0.1);  
System.out.println(price); // 0.1000000000000000055511151231257827021181583404541015625
```

The interviewer's prompt: "Why is the output ugly and how do you avoid it?"

Thinking out loud: 1. " `new BigDecimal(double)` captures the actual binary representation of the double, which isn't exactly 0.1." 2. "Use the String constructor or `BigDecimal.valueOf`."

The refactored version:

```
BigDecimal price = new BigDecimal("0.1");           // exact  
// or  
BigDecimal price = BigDecimal.valueOf(0.1);         // routes through Double.toString – exact for
```

Why this is better: - String constructor stores exactly 0.1 - `valueOf` is the documented way to get a "nice" BigDecimal from a double - No surprises on multiplication or comparison

What NOT to say: - "Round it after construction" — too late; the value is already wrong - "Use `MathContext`" — controls precision in operations, not initial value

Common follow-up: "Why does `BigDecimal.valueOf(0.1)` give a different result than `new BigDecimal(0.1)`?"

Scenario 66 · Manual logger init with class literal

Asked at: Infosys · TCS · **Difficulty:** Easy · **Topic:** SLF4J idiom

The code:

```
public class OrderService {  
    private static final Logger LOG = LoggerFactory.getLogger("com.acme.OrderService");  
    // ...  
}
```

The interviewer's prompt: "What's safer?"

Thinking out loud: 1. "Hard-coded string — rename the class and the logger name silently desyncs." 2. "Pass the class literal — refactors follow renames."

The refactored version:

```
public class OrderService {  
    private static final Logger LOG = LoggerFactory.getLogger(OrderService.class);  
    // ...  
}
```

Why this is better: - IDE rename updates everything - Logger name matches the class even when classes move packages - Standard SLF4J idiom

What NOT to say: - "Use `MethodHandles.lookup().lookupClass()`" — works in static contexts but more verbose; `Class.literal` is fine 99% of the time - "Use `getClass()`" — fails in static contexts

Common follow-up: "What's the trick for getting the right Logger in an abstract base class?"

Scenario 67 · Mutable global counter via static field

Asked at: Razorpay · Cred · **Difficulty:** Medium · **Topic:** Thread safety

The code:

```
public class IdGen {  
    public static long counter = 0;  
    public static long next() { return ++counter; }  
}
```

The interviewer's prompt: "Multi-threaded code calls this. Multiple problems. Fix all."

Thinking out loud: 1. "public static mutable field — any code can reset it. `++counter` is non-atomic. No visibility guarantees." 2. "AtomicLong is the right primitive. Wrap behind a method, hide the field."

The refactored version:

```
public final class IdGen {  
    private IdGen() {}  
    private static final AtomicLong COUNTER = new AtomicLong();  
    public static long next() { return COUNTER.incrementAndGet(); }  
}
```

Why this is better: - Atomic increment under any thread count - Field hidden — no external mutation - Utility-class shape: final + private constructor

What NOT to say: - "Synchronize the `next()` method" — works, but `AtomicLong` is lock-free and faster - "Use volatile long" — fixes visibility, not atomicity of `++`

Common follow-up: "What if you need monotonic IDs across multiple JVMs?"

Scenario 68 · String comparison with ==

Asked at: TCS · Wipro · **Difficulty:** Easy · **Topic:** equals

The code:

```
if (status == "ACTIVE") {  
    // ...  
}
```

The interviewer's prompt: "This sometimes works, sometimes doesn't. Why and what's the fix?"

Thinking out loud: 1. "`==`" compares references. Works for string-pool-interned literals, fails for strings built at runtime. 2. "`.equals`" is the comparison. Null-safe version:

```
"ACTIVE".equals(status) ."
```

The refactored version:

```
if ("ACTIVE".equals(status)) {  
    // ...  
}  
  
// Or Objects.equals if either could be null  
if (Objects.equals(status, "ACTIVE")) {  
    // ...  
}
```

Why this is better: - Compares value, not reference - Literal-on-the-left avoids NPE when `status` is null - Reads as "is status equal to ACTIVE"

What NOT to say: - "Use `==` because string pool" — works for literals only; brittle - "Use `intern()` then `==`" — adds work for no benefit

Common follow-up: "When does `==` legitimately make sense for strings?"

Scenario 69 · Catch-and-log without rethrow

Asked at: Razorpay · Cred · **Difficulty:** Medium · **Topic:** Exception handling

The code:

```
public void charge(Order order) {
    try {
        gateway.charge(order);
    } catch (PaymentException e) {
        log.error("Payment failed", e);
    }
}
```

The interviewer's prompt: "Caller thinks the charge succeeded. Why and what's the right shape?"

Thinking out loud: 1. "Catch-log-swallow. Caller has no way to know the charge failed — proceeds as if everything worked." 2. "Either rethrow, or return a Result/boolean, or propagate as a domain exception. Never silently swallow."

The refactored version:

```
// Option A – propagate as domain exception
public void charge(Order order) {
    try {
        gateway.charge(order);
    } catch (PaymentException e) {
        log.error("Payment failed for order {}", order.getId(), e);
        throw new ChargeFailedException(order.getId(), e);
    }
}

// Option B – return a result type
public ChargeResult charge(Order order) {
    try {
        gateway.charge(order);
        return ChargeResult.success();
    } catch (PaymentException e) {
        log.error("Payment failed for order {}", order.getId(), e);
        return ChargeResult.failed(e.getMessage());
    }
}
```

Why this is better: - Caller sees the failure and reacts - Log message includes the order id for debugging
- Exception cause preserved in the chain

What NOT to say: - "Catch and ignore" — silent failure is the worst failure mode - "Log and return null"
— caller has to remember to null-check the void-equivalent

Common follow-up: "When is swallowing an exception legitimate?"

Scenario 70 · Polling with Thread.sleep

Asked at: Cred · PhonePe · **Difficulty:** Medium · **Topic:** CompletableFuture

The code:

```
public Order waitForOrder(String id) throws InterruptedException {
    while (true) {
        Order o = repo.findById(id);
        if (o != null && o.isReady()) return o;
        Thread.sleep(500);
    }
}
```

The interviewer's prompt: "Spin-poll a DB every 500ms. Better shape?"

Thinking out loud: 1. "Polling wastes DB calls and adds latency. If the upstream supports async notification, use it." 2. "If polling is the only option, at least cap iterations and back off."

The refactored version:

```
// Option A – event-driven (e.g. Redis pub-sub, Kafka, DB LISTEN/NOTIFY)
public CompletableFuture<Order> waitForOrder(String id) {
    return eventBus.subscribe(id) // returns future that completes when order ready
        .orTimeout(30, TimeUnit.SECONDS);
}

// Option B – bounded poll with exponential backoff
public Order waitForOrder(String id) throws InterruptedException {
    long delay = 100;
    long deadline = System.currentTimeMillis() + 30_000;
    while (System.currentTimeMillis() < deadline) {
        Order o = repo.findById(id);
        if (o != null && o.isReady()) return o;
        Thread.sleep(delay);
        delay = Math.min(delay * 2, 5_000);
    }
    throw new TimeoutException("Order " + id + " not ready in 30s");
}
```

Why this is better: - Event-driven version uses zero DB queries while waiting - Bounded poll won't run forever if the order never appears - Exponential backoff reduces load on the DB

What NOT to say: - "Sleep longer to reduce DB load" — increases latency - "Spin-loop without sleep" — burns a CPU core

Common follow-up: "How do you implement event-driven waiting on top of a DB without pub-sub?"

Scenario 71 · Equals using getClass instead of instanceof

Asked at: Infosys · TCS · **Difficulty:** Medium · **Topic:** equals contract

The code:

```
@Override
public boolean equals(Object o) {
    if (o == null || getClass() != o.getClass()) return false;
    Point p = (Point) o;
    return p.x == x && p.y == y;
}
```

The interviewer's prompt: "Pros and cons of getClass vs instanceof here?"

Thinking out loud: 1. " `getClass()` rejects subclass equality — `Point.equals(ColouredPoint)` returns false. Sometimes that's right (subclass might add fields), sometimes it breaks LSP." 2. "Modern shape: make the class `final` (or use a record) and use `instanceof` pattern."

The refactored version:

```
public final class Point {
    private final int x, y;
    public Point(int x, int y) { this.x = x; this.y = y; }

    @Override
    public boolean equals(Object o) {
        return o instanceof Point p && p.x == x && p.y == y;
    }
    @Override
    public int hashCode() { return Objects.hash(x, y); }
}

// Or just
public record Point(int x, int y) {}
```

Why this is better: - `final` class eliminates the subclass-equality dilemma - Pattern variable replaces cast - Record version handles equals/hashCode automatically

What NOT to say: - "Keep `getClass` because it's symmetric" — fragile if anyone ever extends - "Use `Objects.equals` on each field" — works but verbose

Common follow-up: "Why is equality across subclass boundaries so hard?"

Scenario 72 · Returning List from a generic method**Asked at: Flipkart · Cred · Difficulty: Medium · Topic: Generics****The code:**

```
public List<Object> readAll(String collection) {  
    // ...  
}  
  
// Caller  
List<Order> orders = (List<Order>) (List<?>) repo.readAll("orders"); // unchecked cast
```

The interviewer's prompt: "Caller has to do unchecked casts. Fix the signature."

Thinking out loud: 1. "Returning `List<Object>` forces every caller to cast. Pushing type-knowledge to the caller defeats generics." 2. "Parameterize the method by the element type."

The refactored version:

```
public <T> List<T> readAll(String collection, Class<T> type) {  
    // use `type` to drive deserialization  
}  
  
// Caller  
List<Order> orders = repo.readAll("orders", Order.class);
```

Why this is better: - Caller never casts; compiler enforces the type - Class token gives the implementation enough to deserialize correctly - No unchecked-cast warnings

What NOT to say: - "Use raw types (`List`) instead of generic" — same problem, worse warnings - "Suppress unchecked warnings" — hides the design issue

Common follow-up: "When do you need a Class token vs a TypeReference?"

Scenario 73 · Manual checked-list pattern**Asked at: Razorpay · Cred · Difficulty: Medium · Topic: Validation**

The code:

```
public void process(List<Order> orders) {
    if (orders == null) return;
    if (orders.isEmpty()) return;
    for (Order o : orders) {
        if (o == null) continue;
        if (o.getId() == null) continue;
        if (o.getAmount() == null) continue;
        // process
    }
}
```

The interviewer's prompt: "Drowning in defensive checks. Refactor."

Thinking out loud: 1. "Each null check is a band-aid for an upstream contract being unclear. Combine them with a stream filter; or fail at the boundary." 2. "Better: validate at the API boundary, treat internal state as already-valid."

The refactored version:

```
public void process(List<Order> orders) {
    Objects.requireNonNull(orders, "orders");
    orders.stream()
        .filter(Objects::nonNull)
        .filter(o -> o.getId() != null && o.getAmount() != null)
        .forEach(this::processOne);
}

// Better still – enforce non-null fields on Order construction
public record Order(String id, BigDecimal amount, ...) {
    public Order {
        Objects.requireNonNull(id);
        Objects.requireNonNull(amount);
    }
}

public void process(List<Order> orders) {
    Objects.requireNonNull(orders);
    orders.forEach(this::processOne); // contract guarantees fields non-null
}
```

Why this is better: - Filter chain replaces nested `continue` s - Boundary validation lets internal code trust its inputs - Order constructor enforcement prevents the bad data from existing

What NOT to say: - "Wrap each access in `Optional.ofNullable`" — pushes the same checks into Optional form - "Use `@NonNull` annotations only" — annotations help but don't enforce at runtime without a processor

Common follow-up: "Where exactly is 'the boundary' in a layered application?"

Scenario 74 · Switch with mutable accumulator

Asked at: Cred · Razorpay · **Difficulty:** Medium · **Topic:** Switch expression

The code:

```
public BigDecimal fee(String tier) {
    BigDecimal fee;
    switch (tier) {
        case "GOLD":    fee = new BigDecimal("100"); break;
        case "SILVER":  fee = new BigDecimal("50");  break;
        case "BRONZE":  fee = new BigDecimal("25");  break;
        default:        fee = BigDecimal.ZERO;
    }
    fee = fee.multiply(new BigDecimal("1.18")); // GST
    return fee;
}
```

The interviewer's prompt: "Modernize the switch and tighten the variable."

Thinking out loud: 1. "Switch expression returns directly. GST multiplication can happen on the returned value." 2. "`var` or `final BigDecimal` for the local — once-assigned reads better."

The refactored version:

```
private static final BigDecimal GST = new BigDecimal("1.18");

public BigDecimal fee(String tier) {
    BigDecimal base = switch (tier) {
        case "GOLD"    -> new BigDecimal("100");
        case "SILVER"  -> new BigDecimal("50");
        case "BRONZE"  -> new BigDecimal("25");
        default        -> BigDecimal.ZERO;
    };
    return base.multiply(GST);
}
```

Why this is better: - No mutable `fee` variable - Switch expression — exhaustive on enums and sealed types - GST constant named

What NOT to say: - "Inline the multiplication into each case" — duplicates the constant - "Convert to a Map lookup" — extra allocation; switch is faster and exhaustiveness-checked

Common follow-up: "What's the difference between a switch statement and a switch expression's exhaustiveness?"

Scenario 75 · `String.valueOf` to convert null safely

Asked at: Infosys · TCS · Difficulty: Easy · Topic: Null safety

The code:

```
String name = user != null ? user.getName() : null;
String safe = name == null ? "" : name;
```

The interviewer's prompt: "Two checks for one chain. Cleaner?"

Thinking out loud: 1. "Two null checks for the same flow." 2.

" `Optional.ofNullable` chain, or for the simple null-to-empty conversion, `Objects.toString(name, "\\")` ."

The refactored version:

```
String safe = Optional.ofNullable(user)
    .map(User::getName)
    .orElse("");

// Or, if user is guaranteed non-null
String safe = Objects.toString(user.getName(), "");
```

Why this is better: - One expression replaces two checks -

`Objects.toString(obj, default)` is the standard null-to-default - No intermediate variable

What NOT to say: - "Use `String.valueOf(null)` " — returns the string "null", not empty - "Use `(user != null && user.getName() != null) ? user.getName() : \"\"` " — same checks, harder to read

Common follow-up: "What's the difference between `Objects.toString` and `String.valueOf` ?"

Scenario 76 · Class with a setter and a separate validate method

Asked at: Razorpay · Cred · **Difficulty:** Medium · **Topic:** Always-valid invariant

The code:

```
public class Email {
    private String value;
    public void setValue(String v) { this.value = v; }
    public String getValue() { return value; }
    public boolean isValid() {
        return value != null && value.contains("@") && value.contains(".");
    }
}

// Call site
Email e = new Email();
e.setValue("not an email");
// ... 200 lines later
if (e.isValid()) { ... }
```

The interviewer's prompt: "Object can exist in an invalid state. Refactor to make invalid Email impossible."

Thinking out loud: 1. "Always-valid principle — validate at construction, hold the value as final." 2. "Throwing in the constructor is the loud failure mode."

The refactored version:

```
public final class Email {
    private static final Pattern RX = Pattern.compile("[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+");
    private final String value;

    public Email(String value) {
        Objects.requireNonNull(value, "email");
        if (!RX.matcher(value).matches())
            throw new IllegalArgumentException("Invalid email: " + value);
        this.value = value;
    }
    public String value() { return value; }
}
```

Why this is better: - Email object can't exist in an invalid state - One validation point, not scattered `isValid` checks - Final field + final class — true value object

What NOT to say: - "Add `@Valid` annotation" — works for Bean Validation but only at certain call sites - "Set then `assert isValid`" — assertions are off by default in prod

Common follow-up: "What's the regex you'd actually use — RFC 5322 or a stricter approximation?"

Scenario 77 · Properties file loaded inside hot method

Asked at: TCS · Infosys · **Difficulty:** Medium · **Topic:** Configuration loading

The code:

```
public BigDecimal taxRate(String state) throws IOException {
    Properties p = new Properties();
    p.load(new FileInputStream("tax-rates.properties"));
    return new BigDecimal(p.getProperty(state, "0"));
}
```

The interviewer's prompt: *"This is called millions of times per day. What's the problem?"*

Thinking out loud: 1. *"Loads the file from disk every call. Disk IO inside a hot method."* 2. *"Load once at startup into an immutable Map. Reload on config-change signal if needed."*

The refactored version:

```
public class TaxRates {
    private static final Map<String, BigDecimal> RATES = loadRates();

    private static Map<String, BigDecimal> loadRates() {
        try (var in = TaxRates.class.getResourceAsStream("/tax-rates.properties")) {
            Properties p = new Properties();
            p.load(in);
            Map<String, BigDecimal> m = new HashMap<>();
            for (var name : p.stringPropertyNames()) {
                m.put(name, new BigDecimal(p.getProperty(name)));
            }
            return Map.copyOf(m);
        } catch (IOException e) {
            throw new IllegalStateException("tax-rates.properties missing", e);
        }
    }

    public BigDecimal rate(String state) {
        return RATES.getOrDefault(state, BigDecimal.ZERO);
    }
}
```

Why this is better: - One disk read at startup - Immutable map shared safely across threads - Missing file is a startup failure, not a per-request one

What NOT to say: - "Cache the Properties object" — same as caching, just half-done - "Use a ConcurrentHashMap" — unnecessary; the map is built once and frozen

Common follow-up: *"How do you refresh this without restart?"*

Scenario 78 · Old SimpleDateFormat as shared field**Asked at: Razorpay · Cred · Difficulty: Hard · Topic: java.time****The code:**

```
public class DateUtils {  
    private static final SimpleDateFormat FMT = new SimpleDateFormat("yyyy-MM-dd");  
  
    public static String format(Date d) {  
        return FMT.format(d);  
    }  
}
```

The interviewer's prompt: "Multi-threaded code calls this. There's a subtle bug. Fix and modernize."

Thinking out loud: 1. "SimpleDateFormat is not thread-safe. Shared static instance corrupts under concurrent calls — produces garbage strings or throws `ArrayIndexOutOfBoundsException`." 2. "DateTimeFormatter from `java.time` is thread-safe and immutable."

The refactored version:

```
public final class DateUtils {  
    private DateUtils() {}  
    private static final DateTimeFormatter FMT = DateTimeFormatter.ofPattern("yyyy-MM-dd");  
  
    public static String format(LocalDate d) {  
        return FMT.format(d);  
    }  
}
```

Why this is better: - `DateTimeFormatter` is immutable and thread-safe — share freely - `LocalDate` replaces error-prone `Date` - No timezone surprises

What NOT to say: - "Make FMT thread-local" — works but wastes memory per thread - "Synchronize the format method" — kills throughput

Common follow-up: "Why is SimpleDateFormat not thread-safe?"**Scenario 79 · Stream peek for side effects****Asked at: Cred · PhonePe · Difficulty: Medium · Topic: Stream purity****The code:**

```
List<Order> processed = orders.stream()
    .peek(o -> auditLog.write("Processing " + o.getId()))
    .map(this::process)
    .peek(o -> auditLog.write("Done " + o.getId()))
    .toList();
```

The interviewer's prompt: "peek for production side effects. Why is this fragile?"

Thinking out loud: 1. " **peek** is intended for debugging. The Stream API doesn't guarantee **peek** runs for every element if a downstream op short-circuits." 2. "Replace with an explicit loop or **forEach** if we need guaranteed side effects."

The refactored version:

```
List<Order> processed = orders.stream()
    .map(o -> {
        auditLog.write("Processing " + o.getId());
        Order result = process(o);
        auditLog.write("Done " + o.getId());
        return result;
    })
    .toList();

// Or just an enhanced-for loop if the pipeline isn't actually streaming
List<Order> processed = new ArrayList<>();
for (Order o : orders) {
    auditLog.write("Processing " + o.getId());
    processed.add(process(o));
    auditLog.write("Done " + o.getId());
}
```

Why this is better: - Side effects guaranteed to run for every element - No dependence on the unspecified peek-execution behaviour - Reads as the actual flow

What NOT to say: - "Just `peek` is fine, peek always runs" — Java docs explicitly call this out as undefined - "Use `forEach` for both peek lines and the result" — but you can't collect from `forEach`

Common follow-up: "When does Stream `peek` legitimately skip elements?"

Scenario 80 · ToString concatenation in loop

Asked at: Infosys · TCS · Difficulty: Easy · Topic: String building

The code:

```
@Override
public String toString() {
    String result = "Order{items=[";
    for (Item i : items) {
        result = result + i.toString() + ", ";
    }
    result = result + "]]";
    return result;
}
```

The interviewer's prompt: "Quadratic toString. Tighten."

Thinking out loud: 1. "String concat in a loop is $O(n^2)$ per the earlier scenario. For toString, use a `StringJoiner` or `Stream + Collectors.joining`." 2. "`StringJoiner` has a prefix/suffix mode — perfect for `[items, ...]`."

The refactored version:

```

@Override
public String toString() {
    StringJoiner sj = new StringJoiner(", ", "Order{items=[", "]}");
    for (Item i : items) sj.add(i.toString());
    return sj.toString();
}

// Or stream-style
@Override
public String toString() {
    return items.stream()
        .map(Object::toString)
        .collect(Collectors.joining(", ", "Order{items=[", "]}"));
}

```

Why this is better: - $O(n)$ instead of $O(n^2)$ - Prefix/suffix handled in one call - No trailing-comma cleanup

What NOT to say: - "Use `+=` because JIT will optimize" — JIT cannot turn loop-`+=` into one `StringBuilder` in all cases - "Hand-roll `StringBuilder`" — works but `StringJoiner` is purpose-built

Common follow-up: "What's the prefix/suffix `StringJoiner` constructor for?"

Scenario 81 · Static List mutated by multiple threads

Asked at: Cred · PhonePe · **Difficulty:** Hard · **Topic:** Concurrent collection

The code:

```

public class EventBuffer {
    private static final List<Event> events = new ArrayList<>();

    public static void record(Event e) { events.add(e); }
    public static List<Event> snapshot() { return new ArrayList<>(events); }
}

```

The interviewer's prompt: "Concurrent calls hitting this. `ArrayList` isn't thread-safe. Fix."

Thinking out loud: 1. "ArrayList.add under concurrent calls can lose entries or throw ArrayIndexOutOfBoundsException." 2. "Read pattern: snapshot is taken occasionally; writes are frequent. ConcurrentLinkedQueue fits — lock-free writes, weakly consistent snapshot."

The refactored version:

```
public class EventBuffer {  
    private static final Queue<Event> events = new ConcurrentLinkedQueue<>();  
  
    public static void record(Event e) { events.offer(e); }  
    public static List<Event> snapshot() { return new ArrayList<>(events); }  
}
```

Why this is better: - Lock-free writes under any thread count - Snapshot is a weakly consistent view — fast, no global lock - No CHM-style bucket overhead

What NOT to say: - "Wrap in `Collections.synchronizedList`" — coarse lock on every add - "Use CopyOnWriteArrayList" — write cost is $O(n)$ per add — disaster for high-frequency events

Common follow-up: "What does 'weakly consistent' mean for the snapshot?"

Scenario 82 · Boxed integer comparison with ==

Asked at: Flipkart · Cred · Difficulty: Medium · Topic: Autoboxing

The code:

```
Integer a = 200;  
Integer b = 200;  
if (a == b) { /* sometimes true, sometimes false */ }
```

The interviewer's prompt: "This works for small values, fails for big ones. Why and fix it."

Thinking out loud: 1. "Integer cache covers -128 to 127. Outside that range, autoboxing creates new objects; `==` compares references." 2. "`.equals` or unbox to primitive int."

The refactored version:

```
Integer a = 200;
Integer b = 200;
if (a.equals(b)) { /* always correct */ }
// Or
if (a.intValue() == b.intValue()) { /* primitive compare */ }
// Or
if (Objects.equals(a, b)) { /* null-safe */ }
```

Why this is better: - `.equals` compares values - `Objects.equals` adds null-safety - Primitive compare avoids autoboxing on every check

What NOT to say: - "Always use int, never Integer" — fine if you can, but collections force Integer - "Set

`-Djava.lang.Integer.IntegerCache.high=10000` " — works but coupling production correctness to a JVM flag is fragile

Common follow-up: "What's the size of the Integer cache and can it be tuned?"

Scenario 83 · Multiple constructors with overlapping logic

Asked at: Razorpay · Cred · Difficulty: Medium · Topic: Constructor chaining

The code:

```
public class Booking {
    public Booking(String userId) {
        this.userId = userId;
        this.createdAt = Instant.now();
        this.status = "NEW";
        this.tags = new ArrayList<>();
    }
    public Booking(String userId, List<String> tags) {
        this.userId = userId;
        this.createdAt = Instant.now();
        this.status = "NEW";
        this.tags = tags;
    }
    public Booking(String userId, List<String> tags, String status) {
        this.userId = userId;
        this.createdAt = Instant.now();
        this.status = status;
        this.tags = tags;
    }
}
```

The interviewer's prompt: "Three constructors, three places to update if you add a field. Refactor."

Thinking out loud: 1. "Constructor duplication. Chain with `this(...)` so all initialization runs through one path." 2. "Or use a builder if optional params keep multiplying."

The refactored version:

```
public class Booking {
    public Booking(String userId) {
        this(userId, List.of(), "NEW");
    }
    public Booking(String userId, List<String> tags) {
        this(userId, tags, "NEW");
    }
    public Booking(String userId, List<String> tags, String status) {
        this.userId = userId;
        this.tags = List.copyOf(tags);
        this.status = status;
        this.createdAt = Instant.now();
    }
}
```

Why this is better: - One canonical constructor sets all fields - Adding a field touches one place - Defensive copy on tags (`List.copyOf`)

What NOT to say: - *"Keep them as-is, just add a TODO" — kicks the can* - *"Use reflection in a helper init" — over-engineered*

Common follow-up: *"How does this look as a record with compact constructor?"*

Scenario 84 · Recursive method without base case for empty

Asked at: Infosys · TCS · **Difficulty:** Medium · **Topic:** Tail recursion / iterative

The code:

```
public int sum(List<Integer> nums) {  
    if (nums.size() == 1) return nums.get(0);  
    return nums.get(0) + sum(nums.subList(1, nums.size()));  
}
```

The interviewer's prompt: *"Two bugs. Find both and refactor."*

Thinking out loud: 1. *"Empty list throws IndexOutOfBoundsException. And recursion on a list of 10K elements blows the stack — JVM doesn't do TCO."* 2. *"Iterative or stream — both safer."*

The refactored version:

```
public int sum(List<Integer> nums) {  
    return nums.stream().mapToInt(Integer::intValue).sum();  
}  
  
// Or iterative  
public int sum(List<Integer> nums) {  
    int total = 0;  
    for (int n : nums) total += n;  
    return total;  
}
```

Why this is better: - Empty list returns 0 - No stack-overflow risk regardless of input size - `mapToInt` unboxes once

What NOT to say: - "Add a base case for empty and keep recursion" — fixes the bug but stack-overflow stays - "Increase stack size with -Xss" — hides the design problem

Common follow-up: "Why doesn't the JVM optimize tail recursion?"

Scenario 85 · Stream toMap collision

Asked at: Cred · Razorpay · **Difficulty:** Medium · **Topic:** toMap merge function

The code:

```
Map<String, BigDecimal> totalByCity = orders.stream()
    .collect(Collectors.toMap(Order::getCity, Order::getAmount));
```

The interviewer's prompt: "Throws IllegalStateException at runtime. Why and how do you intend to handle it?"

Thinking out loud: 1. "Two orders for the same city — `toMap` without a merge function throws on duplicate keys." 2. "If the intent is 'sum amounts by city', supply a merge function. If it's 'last wins', specify that explicitly."

The refactored version:

```
// Sum amounts per city
Map<String, BigDecimal> totalByCity = orders.stream()
    .collect(Collectors.toMap(
        Order::getCity,
        Order::getAmount,
        BigDecimal::add));

// Or use groupingBy if the operation is more general
Map<String, BigDecimal> totalByCity = orders.stream()
    .collect(Collectors.groupingBy(
        Order::getCity,
        Collectors.reducing(BigDecimal.ZERO, Order::getAmount, BigDecimal::add)));
```

Why this is better: - Merge function makes intent explicit - No runtime exception for legitimate input - `groupingBy` reads more naturally for aggregation

What NOT to say: - "Wrap toMap in try-catch" — flow control via exceptions - "Use a TreeMap to allow duplicates" — TreeMap doesn't allow duplicate keys either

Common follow-up: "When would 'last wins' be a legitimate merge function?"

Scenario 86 · CompletableFuture chained with get()

Asked at: PhonePe · Razorpay · **Difficulty:** Hard · **Topic:** Async composition

The code:

```
public Order enrichOrder(String id) throws Exception {
    Order o = orderService.fetchAsync(id).get();
    Customer c = customerService.fetchAsync(o.getCustomerId()).get();
    o.setCustomer(c);
    return o;
}
```

The interviewer's prompt: "You went async then blocked. Refactor."

Thinking out loud: 1. ".get()" blocks the calling thread, defeating the async API. Each call blocks until the future completes." 2. "Chain with **thenCompose** — second call starts when the first completes, all without blocking."

The refactored version:

```
public CompletableFuture<Order> enrichOrder(String id) {
    return orderService.fetchAsync(id)
        .thenCompose(order ->
            customerService.fetchAsync(order.getCustomerId())
                .thenApply(customer -> {
                    order.setCustomer(customer);
                    return order;
                })
        );
}
```

Why this is better: - Non-blocking — calling thread is free - Pipeline composes naturally with timeouts and error handlers - Caller can decide whether to block or chain further

What NOT to say: - "Use `.join()` instead of `.get()`" — still blocks; only difference is unchecked exception - "Spin a separate thread for the wait" — adds a thread to wait on a future; pointless

Common follow-up: "What's the difference between `thenApply`, `thenCompose`, and `thenAccept`?"

Scenario 87 · Manual cache with no expiry

Asked at: Cred · Razorpay · **Difficulty:** Hard · **Topic:** Caffeine

The code:

```
public class ProductCache {
    private final Map<String, Product> cache = new ConcurrentHashMap<>();

    public Product get(String sku) {
        return cache.computeIfAbsent(sku, k -> db.fetch(k));
    }
}
```

The interviewer's prompt: "Cache grows forever. Production OOMs at week 3. Fix."

Thinking out loud: 1. "No size cap, no eviction, no TTL — unbounded growth." 2. "Caffeine handles size/time/refresh policies in one config."

The refactored version:

```
public class ProductCache {
    private final LoadingCache<String, Product> cache = Caffeine.newBuilder()
        .maximumSize(10_000)
        .expireAfterWrite(Duration.ofMinutes(15))
        .refreshAfterWrite(Duration.ofMinutes(5))
        .build(db::fetch);

    public Product get(String sku) { return cache.get(sku); }
}
```

Why this is better: - Bounded memory via `maximumSize` - TTL via `expireAfterWrite` - Background refresh keeps hot keys warm without blocking the request thread

What NOT to say: - "Add a scheduled job to clear the map every hour" — coarse, drops all entries including hot ones - "Use a soft-reference Map" — GC pressure unpredictable, doesn't bound based on time

Common follow-up: "What's the difference between `expireAfterWrite` and `expireAfterAccess` ?"

Scenario 88 · Equals with floating point

Asked at: Razorpay · Cred · **Difficulty:** Medium · **Topic:** Float comparison

The code:

```
public class Sensor {
    private double reading;
    @Override
    public boolean equals(Object o) {
        if (!(o instanceof Sensor)) return false;
        return ((Sensor) o).reading == reading;
    }
    @Override public int hashCode() { return Double.hashCode(reading); }
}
```

The interviewer's prompt: "`0.1 + 0.2 != 0.3` and your equality reflects it. Refactor."

Thinking out loud: 1. "Direct `==` on double — two sensors with semantically equal but bit-different readings (e.g. from different summation orders) don't equal." 2. "Use `Double.compare` for IEEE-754-aware comparison (handles -0.0 and NaN), or `BigDecimal` if exactness matters."

The refactored version:

```

public class Sensor {
    private static final double EPSILON = 1e-9;
    private final double reading;

    public Sensor(double reading) { this.reading = reading; }

    @Override
    public boolean equals(Object o) {
        return o instanceof Sensor s && Math.abs(s.reading - reading) < EPSILON;
    }
    @Override public int hashCode() {
        // Note: equals-with-epsilon and hashCode don't combine cleanly – bucket on rounded value
        return Double.hashCode(Math.round(reading / EPSILON) * EPSILON);
    }
}

// Better: if exact equality matters, use BigDecimal as the field

```

Why this is better: - Epsilon-based comparison handles float drift - BigDecimal variant gives exact equality when needed - Pattern variable replaces cast

What NOT to say: - "Use `Double.compare`" — that's bit-exact compare, doesn't help with float drift - "Cast to int" — loses precision

Common follow-up: "Why is epsilon-equality and hashCode hard to combine correctly?"

Scenario 89 · Repeated DTO mapping by hand

Asked at: Flipkart · Cred · **Difficulty:** Medium · **Topic:** Record / MapStruct

The code:

```

public OrderDto toDto(Order o) {
    OrderDto d = new OrderDto();
    d.setId(o.getId());
    d.setCustomerId(o.getCustomerId());
    d.setAmount(o.getAmount());
    d.setCurrency(o.getCurrency());
    d.setStatus(o.getStatus().name());
    return d;
}

```

The interviewer's prompt: *"Forty more DTOs follow this exact pattern. Better approach?"*

Thinking out loud: 1. *"Hand-rolled mapping is error-prone — forget a field and the bug is silent."* 2. *"Two options: MapStruct generates the mapping at compile time, or records + a factory method live in the DTO."*

The refactored version:

```
// Option A – MapStruct
@Mapper
public interface OrderMapper {
    OrderMapper INSTANCE = Mappers.getMapper(OrderMapper.class);
    @Mapping(source = "status", target = "status", qualifiedByName = "statusToString")
    OrderDto toDto(Order order);
    @Named("statusToString") default String statusToString(Status s) { return s.name(); }
}

// Option B – record DTO with a static factory
public record OrderDto(String id, String customerId, BigDecimal amount,
                      String currency, String status) {
    public static OrderDto from(Order o) {
        return new OrderDto(o.getId(), o.getCustomerId(), o.getAmount(),
                           o.getCurrency(), o.getStatus().name());
    }
}
```

Why this is better: - MapStruct generates the mapper at compile time — zero reflection at runtime - Record version is concise and fail-fast (compile error on missing field) - No more bean setters spread across mapper code

What NOT to say: - "Use BeanUtils.copyProperties" — reflective copy, slow, silently skips type mismatches - "Write each mapper by hand 'because it's clearer'" — clear until field 38

Common follow-up: *"MapStruct vs ModelMapper — what's the tradeoff?"*

Scenario 90 · Configuration via global static setters

Asked at: Razorpay · Cred · **Difficulty:** Hard · **Topic:** Dependency injection

The code:

```
public class HttpClient {
    public static int timeoutMs = 5000;
    public static int maxRetries = 3;
    public static String baseUrl;

    public Response get(String path) {
        // uses static fields
    }
}

// Caller during test
HttpClient.timeoutMs = 100;
HttpClient.baseUrl = "http://localhost:8080";
```

The interviewer's prompt: "Tests are flaky because of test-order issues. Refactor."

Thinking out loud: 1. "Static mutable config — tests interfere with each other. Parallel tests see torn writes." 2. "Instance configuration, immutable per client. Builder pattern lets test wire its own client."

The refactored version:

```
public class HttpClient {
    private final Duration timeout;
    private final int maxRetries;
    private final String baseUrl;

    private HttpClient(Builder b) {
        this.timeout = b.timeout;
        this.maxRetries = b.maxRetries;
        this.baseUrl = b.baseUrl;
    }

    public Response get(String path) { /* uses fields */ }

    public static class Builder {
        private Duration timeout = Duration.ofSeconds(5);
        private int maxRetries = 3;
        private String baseUrl;

        public Builder timeout(Duration t) { this.timeout = t; return this; }
        public Builder maxRetries(int r) { this.maxRetries = r; return this; }
        public Builder baseUrl(String u) { this.baseUrl = u; return this; }
        public HttpClient build() { return new HttpClient(this); }
    }
}
```

Why this is better: - Each test wires its own client; no global state leak - Parallel tests safe - Immutable client — config can't change mid-request

What NOT to say: - "Reset statics in `@BeforeEach`" — works but easy to forget; doesn't help parallel tests - "Inject via Spring `@Value`" — works in Spring but doesn't fix the underlying state problem

Common follow-up: "How would you structure this in a Spring application?"

Scenario 91 · Anonymous class to capture enclosing field

Asked at: Cred · PhonePe · **Difficulty:** Medium · **Topic:** Lambda capture

The code:

```
public Runnable makeTask(final Order order) {
    return new Runnable() {
        @Override
        public void run() {
            processor.process(order);
        }
    };
}
```

The interviewer's prompt: "2026 syntax."

Thinking out loud: 1. "Single abstract method = functional interface = lambda. `final` is no longer required since Java 8 (effectively-final suffices)." 2. "Method reference if the call shape matches."

The refactored version:

```
public Runnable makeTask(Order order) {
    return () -> processor.process(order);
}
```

Why this is better: - Lambda replaces 4-line anonymous class - No **final** ceremony — **order** is effectively final - Same bytecode efficiency via *invokedynamic*

What NOT to say: - "Use **Runnable::run**" — that's not what's happening here - "Use **processor::process** directly" — only works if the lambda has no captured variable

Common follow-up: "What's effectively-final?"

Scenario 92 · Eager fetch all then filter in memory

Asked at: Razorpay · Flipkart · **Difficulty:** Medium · **Topic:** Push filtering to source

The code:

```
public List<Order> recentLargeOrders(String customerId) {
    List<Order> all = repo.findAll();
    return all.stream()
        .filter(o -> o.getCustomerId().equals(customerId))
        .filter(o -> o.getAmount().compareTo(new BigDecimal("10000")) > 0)
        .filter(o -> o.getCreatedAt().isAfter(Instant.now().minus(7, DAYS)))
        .toList();
}
```

The interviewer's prompt: "Fetch all + filter in memory. Refactor."

Thinking out loud: 1. "Pulling every order from the DB to filter three predicates that the DB could apply with an index." 2. "Push the predicates to the query."

The refactored version:

```
public List<Order> recentLargeOrders(String customerId) {
    return repo.findByCustomerIdAndAmountGreaterThanAndCreatedAtAfter(
        customerId,
        new BigDecimal("10000"),
        Instant.now().minus(7, DAYS));
}

// Or with a Specification / criteria builder for dynamic predicates
```


Why this is better: - DB returns only the rows you need — orders of magnitude less data over the wire - Indexed columns turn $O(n)$ scan into $O(\log n)$ - Memory pressure on the app server stays flat

What NOT to say: - "Cache the result of `findAll`" — keeps the giant in-memory dataset around forever - "Paginate the `findAll`" — still scans all rows; you want the index, not pagination

Common follow-up: "What if the predicate is dynamic — sometimes amount, sometimes not?"

Scenario 93 · Long-arg method when half are flags

Asked at: Razorpay · Cred · **Difficulty:** Medium · **Topic:** Options object

The code:

```
public Result render(String html, int width, int height, boolean grayscale,
                    boolean cropToContent, boolean includeBackground,
                    int dpi, boolean useFonts, boolean lazyImages) {
    // ...
}
```

The interviewer's prompt: "Five booleans. Call sites are unreadable. Refactor."

Thinking out loud: 1. "Boolean parameter list — caller writes

`render(html, 800, 600, false, true, false, 96, true, false)` — incomprehensible." 2. "Options record with named fields."

The refactored version:

```
public record RenderOptions(  
    int width,  
    int height,  
    boolean grayscale,  
    boolean cropToContent,  
    boolean includeBackground,  
    int dpi,  
    boolean useFonts,  
    boolean lazyImages) {  
    public static RenderOptions defaults() {  
        return new RenderOptions(800, 600, false, true, true, 96, true, true);  
    }  
}  
  
public Result render(String html, RenderOptions opts) { /* ... */ }
```

Why this is better: - Named fields at the call site - Defaults factory for the common case - Adding a new option doesn't break callers (give it a default in `defaults()`)

What NOT to say: - "Encode flags as an int bitmask" — terse but hostile to maintenance - "Add overloads for each combination" — combinatorial explosion

Common follow-up: "How would you handle adding a new option with a sensible default?"

Scenario 94 · Enum with hardcoded list

Asked at: Infosys · **Cred:** · **Difficulty:** Easy · **Topic:** Enum.values

The code:

```
public List<Status> activeStatuses() {  
    List<Status> result = new ArrayList<>();  
    result.add(Status.NEW);  
    result.add(Status.PROCESSING);  
    result.add(Status.SHIPPED);  
    return result;  
}
```

The interviewer's prompt: "You add a new active status and forget this list. Refactor to be self-maintaining."

Thinking out loud: 1. "Hardcoded list — forget to update it and the bug is silent." 2. "Put `isActive` on the enum and filter."

The refactored version:

```
public enum Status {
    NEW(true), PROCESSING(true), SHIPPED(true), DELIVERED(false), CANCELLED(false);
    private final boolean active;
    Status(boolean active) { this.active = active; }
    public boolean isActive() { return active; }
}

public List<Status> activeStatuses() {
    return Arrays.stream(Status.values()).filter(Status::isActive).toList();
}
```

Why this is better: - New status declares its active-ness inline -

`activeStatuses` derives from the enum's own knowledge - Single source of truth

What NOT to say: - "Document the convention in a comment" — comments rot -
"Use a `Set<Status>` constant" — same drift problem in a different shape

Common follow-up: "How would you cache the result of `activeStatuses()` to avoid recomputing?"

Scenario 95 · Final on local variable for no reason

Asked at: Cred · Razorpay · **Difficulty:** Easy · **Topic:** var keyword

The code:

```
public void process(List<Order> orders) {
    final Map<String, BigDecimal> totals = new HashMap<>();
    final BigDecimal taxRate = new BigDecimal("0.18");
    for (final Order order : orders) {
        final BigDecimal afterTax = order.getAmount().multiply(taxRate.add(BigDecimal.ONE));
        totals.put(order.getId(), afterTax);
    }
}
```

The interviewer's prompt: "Modern Java style — what changes?"

Thinking out loud: 1. " **final** on every local is noise that obscures the intent.

var infers the type and removes the dead annotation." 2. "Constants stay typed explicitly — they're API in spirit."

The refactored version:

```
private static final BigDecimal GST_MULTIPLIER = new BigDecimal("1.18");

public void process(List<Order> orders) {
    var totals = new HashMap<String, BigDecimal>();
    for (var order : orders) {
        var afterTax = order.getAmount().multiply(GST_MULTIPLIER);
        totals.put(order.getId(), afterTax);
    }
}
```

Why this is better: - **var** infers type, removes verbose duplication - **final** only where it adds value (e.g. lambda capture) - Constant pulled out and named

What NOT to say: - "Use **final** everywhere for safety" — noise that hides intent without runtime benefit - "Replace all **var** with explicit types" — Java 11+ prefers **var** for local readability

Common follow-up: "When should you avoid **var**?"

Scenario 96 · Try-catch around InterruptedException swallowed

Asked at: Cred · PhonePe · **Difficulty:** Medium · **Topic:** Interruption handling

The code:

```
public void awaitJob() {
    try {
        Thread.sleep(1000);
    } catch (InterruptedException e) {
        // ignored
    }
}
```

The interviewer's prompt: "Swallowed InterruptedException. What goes wrong and what's the fix?"

Thinking out loud: 1. "Caller signals 'stop' via interrupt. We catch it, clear the flag, and pretend nothing happened. Now the loop or pool that interrupted us has no way to know we got the message." 2. "Either restore the interrupt flag, or rethrow."

The refactored version:

```
public void awaitJob() throws InterruptedException {
    Thread.sleep(1000); // propagate
}

// If catching is necessary (e.g. wrapper hides checked exception)
public void awaitJob() {
    try {
        Thread.sleep(1000);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt(); // restore the flag
        throw new RuntimeException("Interrupted while waiting for job", e);
    }
}
```

Why this is better: - Restored interrupt flag propagates to callers checking `isInterrupted()` - ExecutorService treats restored flag as a stop signal - Cause preserved for debugging

What NOT to say: - "Catch and log only" — log is informational; flag still lost - "Catch then `Thread.interrupted()`" — that clears the flag, doesn't restore

Common follow-up: "What's the difference between `Thread.interrupted()` and `Thread.currentThread().isInterrupted()`?"

Scenario 97 · Verbose JSON parsing with old Jackson**Asked at: Razorpay · Cred · Difficulty: Medium · Topic: ObjectMapper****The code:**

```
public Map<String, Object> parse(String json) throws IOException {
    ObjectMapper mapper = new ObjectMapper();
    JsonFactory factory = mapper.getFactory();
    JsonParser parser = factory.createParser(json);
    return parser.readValueAs(new TypeReference<Map<String, Object>>() {});
}
```

The interviewer's prompt: "You're creating a new `ObjectMapper` every call. Plus the parser dance is unnecessary."

Thinking out loud: 1. "`ObjectMapper` construction is expensive — caches reflection metadata. Reuse a single instance." 2. "`readValue` directly on the mapper skips the parser scaffolding."

The refactored version:

```
public class JsonParser {
    private static final ObjectMapper MAPPER = new ObjectMapper();
    private static final TypeReference<Map<String, Object>> MAP_TYPE = new TypeReference<Map<String, Object>>() {};

    public Map<String, Object> parse(String json) throws IOException {
        return MAPPER.readValue(json, MAP_TYPE);
    }
}
```

Why this is better: - Single `ObjectMapper` amortizes setup cost - `TypeReference` singleton — created once - `ObjectMapper` is thread-safe after configuration

What NOT to say: - "Make `ObjectMapper` a singleton field on every class" — duplicates the instance - "Use `Gson` instead" — different library, doesn't address the design

Common follow-up: "When is `ObjectMapper` not thread-safe?"

Scenario 98 · Throw new RuntimeException as generic catch-all**Asked at: Infosys · Cred · Difficulty: Easy · Topic: Exception types****The code:**

```
public Invoice load(String id) {  
    try {  
        return repo.findById(id);  
    } catch (Exception e) {  
        throw new RuntimeException(e);  
    }  
}
```

The interviewer's prompt: "Type-erased exception. Refactor."**Thinking out loud: 1. " `catch (Exception)` swallows every checked and runtime exception. Then re-throw as generic RuntimeException loses all type info." 2.****"Catch what you can actually handle, let the rest propagate. If you must wrap, use a domain exception."****The refactored version:**

```
public Invoice load(String id) {  
    try {  
        return repo.findById(id);  
    } catch (SQLException e) {  
        throw new InvoiceLoadException("Failed to load invoice " + id, e);  
    }  
}
```

Why this is better: - Caller knows it's a load failure - Other exceptions (NPE, IllegalStateException) propagate untouched - Stack trace and cause preserved**What NOT to say: - "Catch Throwable to be extra safe" — never catch Throwable in app code; OOM and ThreadDeath should die - "Replace with `Optional.empty()` on failure" — silently hides errors****Common follow-up: "When is `catch (Exception)` ever right?"**

Scenario 99 · Lazy initialization with null check**Asked at: Cred · PhonePe · Difficulty: Hard · Topic: Lazy initialization****The code:**

```
public class HeavyService {
    private Connection conn;

    public Connection getConn() {
        if (conn == null) {
            conn = createConnection();
        }
        return conn;
    }
}
```

The interviewer's prompt: "Multi-threaded. What goes wrong and what's the modern fix?"

Thinking out loud: 1. "Two threads can both see null, both call `createConnection`. You get two connections, the leak hides until you run pool diagnostics." 2. "Modern Java: `Suppliers.memoize` (Guava) is the simplest. Or hand-rolled DCL with volatile."

The refactored version:


```

// Option A – initialize eagerly if the cost is acceptable
public class HeavyService {
    private final Connection conn = createConnection();
    public Connection getConn() { return conn; }
}

// Option B – DCL with volatile if lazy is genuinely needed
public class HeavyService {
    private volatile Connection conn;

    public Connection getConn() {
        Connection c = conn;
        if (c == null) {
            synchronized (this) {
                c = conn;
                if (c == null) {
                    c = createConnection();
                    conn = c;
                }
            }
        }
        return c;
    }
}

// Option C – holder idiom for static lazy init
public class HeavyService {
    private static class Holder { static final Connection CONN = createConnection(); }
    public Connection getConn() { return Holder.CONN; }
}

```

Why this is better: - Each option closes the race - Holder idiom — JVM guarantees class init runs once, lazily - Eager init is simplest when startup cost is acceptable

What NOT to say: - "Synchronize the getter" — works but pays the lock cost forever - "Mark `conn` volatile and ignore the if" — fixes visibility but not the double-create race

Common follow-up: "What does the JLS say about class initialization safety?"

Scenario 100 · Loop building a List from a Map

Asked at: Flipkart · **Cred:** · **Difficulty:** Easy · **Topic:** Stream pipeline

The code:

```
public List<String> formatStock(Map<String, Integer> stock) {  
    List<String> lines = new ArrayList<>();  
    for (Map.Entry<String, Integer> e : stock.entrySet()) {  
        if (e.getValue() > 0) {  
            lines.add(e.getKey() + ": " + e.getValue());  
        }  
    }  
    Collections.sort(lines);  
    return lines;  
}
```

The interviewer's prompt: "Three things in one method. Tighten it."

Thinking out loud: 1. "Filter, map, sort, collect — that's a stream pipeline." 2.

"Stream is cleaner than a loop with intermediate mutations and a separate sort."

The refactored version:

```
public List<String> formatStock(Map<String, Integer> stock) {  
    return stock.entrySet().stream()  
        .filter(e -> e.getValue() > 0)  
        .map(e -> e.getKey() + ": " + e.getValue())  
        .sorted()  
        .toList();  
}
```

Why this is better: - One pipeline expresses filter, format, sort - No mutable list in scope to corrupt accidentally - `toList()` returns unmodifiable result

What NOT to say: - "Use a TreeMap to get sorted iteration" — still need to filter and format - "Sort using a comparator over Entries first" — same shape but more verbose

Common follow-up: "What if the sort key is the integer value, not the formatted string?"

Category 7 — Real-World Tradeoffs

Java Interview Prep — 2026 Edition · Learn & Excel

These 100 scenarios cover the architectural micro-decisions that separate senior candidates from juniors. Every one of these comes up in real Indian-MNC and product-company interviews — and the right answer almost never starts with "always use X". The pattern that wins rounds is: name both options, name the dimension that matters, pick one, and admit what you're giving up.

Scenario 1 · HashMap vs ConcurrentHashMap for a read-heavy lookup cache

Asked at: Razorpay · Swiggy · Flipkart · **Difficulty:** Medium · **Topic:** Concurrency + collections

The interviewer's prompt: "You have an in-memory lookup cache populated once at startup and read by 200 request threads per second. Would you use HashMap or ConcurrentHashMap?"

Thinking out loud (the right way to think about it): 1. "First, who writes to it after startup? If nobody, a plain HashMap published safely via final field reference is enough — JLS final-field semantics guarantee visibility." 2. "If anything mutates it later — even a refresh every 5 minutes — concurrent reads can hit a resize and see a corrupted bucket array. That's the classic CPU-pegging infinite loop people have reported on HashMap."

The defensible answer: - Pure read-only after init: `Map.copyOf(...)` and store in a `final` field. Cheaper and simpler than ConcurrentHashMap. - If anything ever writes — even rarely — use ConcurrentHashMap. The cost of getting it wrong is much higher than the cost of using the concurrent version.

The tradeoff (what you give up): - ConcurrentHashMap has slightly higher memory footprint and per-bucket overhead due to its segment/CAS machinery. - Iteration is weakly consistent — your snapshot can include updates that happened mid-iteration.

When the OTHER option wins (situations where you'd flip): - Truly immutable after construction → plain `Map.copyOf` is faster, simpler, and gives a real immutability guarantee. - Tiny maps (under 20 keys) that fit in CPU cache → the constant factor doesn't matter, pick whichever is clearest.

What NOT to say: - "Always use `ConcurrentHashMap` to be safe" — shows you don't understand the cost of paranoia in hot paths. - "HashMap is fine because reads don't need locks" — wrong, because a concurrent writer during resize corrupts the structure for everyone.

Common follow-up: "What about `Collections.synchronizedMap` — when does that ever win?"

Scenario 2 · `synchronized` vs `ReentrantLock` for a payment processor

Asked at: PhonePe · Razorpay · Paytm · **Difficulty:** Medium-Hard · **Topic:** Concurrency

The interviewer's prompt: "You're guarding a critical section in a payment processor. Would you use `synchronized` or `ReentrantLock`?"

Thinking out loud: 1. "What features do I actually need? Mostly: do I need a timeout, fairness, or to be interruptible while waiting for the lock?" 2. "If it's a plain in-out critical section that's microseconds long, `synchronized` is simpler and the JIT optimizes it heavily — biased locking is gone in Java 15+ but lock coarsening and elimination still apply."

The defensible answer: - Default to `synchronized` for short critical sections — less ceremony, no chance of forgetting to unlock in a finally. - Pick `ReentrantLock` when I need `tryLock(timeout)`, lock interruptibility, fairness, or multiple condition variables.

The tradeoff: - `synchronized` can't time out. If the lock is held forever, the waiting thread blocks forever — no way to recover gracefully. - `ReentrantLock` requires `try { ... } finally { lock.unlock(); }` — one missing finally and you leak the lock permanently.

When the OTHER option wins: - A payment retry that should give up after 200ms instead of stacking up → `ReentrantLock.tryLock(200, MILLISECONDS)`. - Need to support cancellation from another thread → `ReentrantLock` supports `lockInterruptibly()`.

What NOT to say: - "ReentrantLock is faster than synchronized" — not true on modern JVMs for uncontended locks; they're roughly equivalent. - "synchronized is legacy, always use Lock" — opinionated and wrong; `synchronized` is still the right default for simple cases.

Common follow-up: "What about `StampedLock` — when does that beat `ReentrantLock`?"

Scenario 3 · AtomicInteger vs synchronized for a hit counter

Asked at: Swiggy · **Cred:** · **Difficulty:** Easy-Medium · **Topic:** Concurrency

The interviewer's prompt: "For a counter incremented by every request thread, would you use `AtomicInteger.incrementAndGet()` or `synchronized` around a `int` field?"

Thinking out loud: 1. "Is the operation just increment, or is it part of a multi-step compound action like check-then-update?" 2. "For a single-variable read-modify-write, CAS via `AtomicInteger` is purpose-built and lock-free."

The defensible answer: - `AtomicInteger.incrementAndGet()` . Lock-free, hardware-supported CAS, no thread parking, no monitor contention. - For very high contention (hundreds of threads pounding one counter), upgrade to `LongAdder` — it stripes internal state per thread to reduce CAS retries.

The tradeoff: - Atomic only protects ONE variable. The moment you need to update two related fields together, you're back to a lock. - `LongAdder.sum()` is not a single-instant snapshot — it's a sum across stripes computed lazily.

When the OTHER option wins: - Updating two fields atomically (e.g. `count + lastUpdated timestamp`) → need `synchronized` or `ReentrantLock` . - Need to read-and-decide based on the new value while excluding other updates → `synchronized` block.

What NOT to say: - "AtomicInteger is always faster than synchronized" — true for uncontended, but at extreme contention CAS retries can actually starve. - "Just make the field volatile" — volatile gives visibility, not atomicity. `volatile int counter; counter++;` is still a race.

Common follow-up: "What's `LongAdder` and when does it beat `AtomicLong` ?"

Scenario 4 · Optional vs nullable return vs custom Result type

Asked at: Atlassian India · Walmart Global Tech · **Difficulty:** Medium · **Topic:** API design

The interviewer's prompt: "A repository's `findById` can fail to find the row. Do you return `Optional<User>`, `User` (nullable), or a custom `Result<User, Error>` ?"

Thinking out loud: 1. "What's the caller going to do with absence? Is it a normal outcome or an error?" 2. "For an interactive lookup where 'not found' is a regular case (e.g. checking if a username is taken), absence is a value — `Optional` fits."

The defensible answer: - Return `Optional<User>` for genuinely optional return values where absence is a regular case — interactive lookups, cache misses, conditional finds. - Throw a specific exception (e.g. `EntityNotFoundException`) when the absence indicates a programming error or contract violation. - Use a custom `Result` type only when you need to carry multiple distinct error reasons that the caller will branch on.

The tradeoff: - `Optional` is for return values — not parameters, not fields, not collection elements. Misusing it leaks the type into your data model. - Custom `Result` types in Java feel awkward without a real sum type / sealed hierarchy and add boilerplate.

When the OTHER option wins: - A loader that may legitimately return null inside a stream pipeline → nullable + `Objects.requireNonNullElseGet`. - A multi-step workflow with 5+ distinct failure modes → sealed `Result` hierarchy or a Try-style API.

What NOT to say: - "Always wrap in `Optional`, never return null" — popular myth; doesn't apply to fields or collection elements. - "`Optional` is just a nullable in a box" — misses the point; the API forces the caller to confront absence.

Common follow-up: "Why is `Optional` a bad fit for fields in entity classes?"

Scenario 5 · Checked vs unchecked exception for a payment failure

Asked at: Razorpay · PhonePe · Difficulty: Medium · Topic: Exception design

The interviewer's prompt: "You're writing `chargeCard(...)`. The call can fail because the card was declined. Do you make `CardDeclinedException` checked or unchecked?"

Thinking out loud: 1. "Is this something every caller **MUST** handle? A declined card is a routine outcome that the caller needs to react to — show a different screen, retry with another card." 2. "But checked exceptions force `throws` declarations through every method between the call site and the handler — that becomes painful with lambdas and Streams."

The defensible answer: - Unchecked, with a specific class (`CardDeclinedException extends RuntimeException`). The framework layer (controller advice, error mapper) catches it and translates. - The exception class is part of the contract — document it in Javadoc, but don't force `throws` propagation everywhere.

The tradeoff: - Unchecked won't be caught by the compiler if forgotten — code reviews and integration tests have to enforce handling. - A checked exception is louder and harder to ignore — fits domains where 'forgetting to handle' is unacceptable.

When the **OTHER** option wins: - A small surface area with one or two callers and severe consequences for missing the case (banking core ledger writes) → checked. - A tightly scoped library API where the recovery action is obvious and local → checked can document intent better.

What **NOT** to say: - "Checked exceptions are dead, never use them" — Effective Java still recommends checked for recoverable conditions when the caller can act. - "Wrap everything in `RuntimeException`" — loses the specific type, makes handling impossible.

Common follow-up: "Why does Spring wrap JDBC's checked `SQLException` into an unchecked `DataAccessException`?"

Scenario 6 · @Async vs Reactor vs virtual threads for downstream HTTP fan-out

Asked at: Flipkart · Swiggy · Zomato · **Difficulty:** Hard · **Topic:** Concurrency models

The interviewer's prompt: "You need to call 5 downstream APIs in parallel and aggregate. `@Async`, Project Reactor, or Java 21 virtual threads?"

Thinking out loud: 1. "What's the rest of the codebase look like? Is it already reactive? Is it a Spring MVC blocking stack on Tomcat?" 2. "How many concurrent fan-outs per second am I doing? Five outbound calls per request times 500 RPS is 2500 inflight downstream calls — that's the scale that pushes you past a regular thread pool."

The defensible answer: - On Java 21 with a blocking codebase: virtual threads. `StructuredTaskScope.ShutdownOnFailure` for the fan-out, no thread pool, no callback hell. - Already reactive (WebFlux + Reactor): keep it reactive — `Mono.zip(...)` or `Flux.merge(...)`. Don't mix paradigms. - Stuck on Java 17 with Spring MVC: `@Async` with a tuned `ThreadPoolTaskExecutor` and `CompletableFuture.allOf(...)`. Not pretty, but it works.

The tradeoff: - Virtual threads pin to the carrier when they hit `synchronized` blocks or native code — JDBC driver internals can defeat the model in subtle ways. - Reactor's learning curve is steep — debugging stack traces and threading model is genuinely harder. - `@Async` proxy semantics: calling an `@Async` method from inside the same bean bypasses the proxy and runs synchronously.

When the OTHER option wins: - Strict backpressure (slow consumer, fast producer) → Reactor's flow control wins over any thread-per-task model. - CPU-bound work, not I/O → virtual threads don't help; use a `ForkJoinPool` or fixed pool sized to cores.

What NOT to say: - "Virtual threads make everything faster" — they don't help CPU-bound work, and they hurt in pinning scenarios. - "Reactive is always the modern choice" — the modern choice on Java 21 for I/O fan-out is virtual threads, not Reactor.

Common follow-up: "How does `StructuredTaskScope` improve over `CompletableFuture.allOf`?"

Scenario 7 · DB cursor paging vs offset paging for an export

Asked at: Walmart Global Tech · Target · Lowe's India · **Difficulty:** Medium · **Topic:** Database pagination

The interviewer's prompt: "You're exporting 5 million orders. Would you page through using `LIMIT/OFFSET` or a keyset/cursor based on `id > lastId`?"

Thinking out loud: 1. "`OFFSET 4000000 LIMIT 1000` makes the DB scan and discard 4 million rows for each page. That's quadratic over the export." 2. "Cursor (keyset) pagination uses `WHERE id > :lastId ORDER BY id LIMIT 1000` — the index seek is $O(\log n)$ each page, no scanning of skipped rows."

The defensible answer: - Cursor/keyset pagination. Stable performance regardless of how deep we are, no risk of duplicates if the table is mutating during the export. - For exports specifically: stream via JDBC `setFetchSize` + `ResultSet` cursor — don't materialize pages in memory at all if you don't have to.

The tradeoff: - Cursor pagination requires a stable, unique, indexed ordering column (usually PK or a composite of PK + timestamp). - Can't jump to arbitrary page numbers like "show me page 47" — only forward/backward by cursor.

When the OTHER option wins: - A user-facing admin UI with page numbers and 100s of rows total → `OFFSET` is fine and simpler. - The sort column isn't unique → `OFFSET` avoids the tie-breaking complexity.

What NOT to say: - "OFFSET is fine, the DB handles it" — at depth, it absolutely doesn't; production incidents are common. - "Just cache the count" — count isn't the issue, the row-skipping scan is.

Common follow-up: "How do you handle ties in keyset pagination when sorting by a non-unique column?"

Scenario 8 · Cache hit vs DB lookup — when does caching hurt?

Asked at: Razorpay · Swiggy · Flipkart · **Difficulty:** Medium · **Topic:** Caching

The interviewer's prompt: *"This lookup is hit 50 times per second. Do you cache it in Caffeine or just hit the DB?"*

Thinking out loud: 1. *"What's the hit pattern look like? Same 10 keys hammered repeatedly, or wide and uniform across millions of keys?"* 2. *"What's the consistency requirement? If the DB row changes, how stale can the cache be?"*

The defensible answer: - Cache when: hot key set is small (Pareto), DB lookup is expensive (joins, deep traversals), and stale-by-seconds is acceptable. - Skip cache when: hit ratio would be under 50%, the row mutates frequently and consumers need fresh data, or the DB query is already index-served in under a millisecond.

The tradeoff: - Caches add another invalidation problem — Phil Karlton's two hard things. - Memory pressure: a poorly sized cache evicts useful entries; Caffeine's W-TinyLFU helps but doesn't eliminate the issue.

When the OTHER option wins: - Sub-millisecond DB query on an already-indexed lookup → don't cache; the cost-benefit isn't there. - Strict read-after-write consistency → cache invalidation race conditions are real, sometimes the DB IS the cache.

What NOT to say: - "Cache everything to reduce DB load" — adds complexity, can mask real performance problems. - "Just set TTL to 1 minute, it's fine" — TTL is a hack when you should be doing event-driven invalidation.

Common follow-up: *"What's the cache stampede problem and how do you solve it?"*

Scenario 9 · In-memory state vs Redis for session data

Asked at: Flipkart · Zomato · Myntra · **Difficulty:** Medium · **Topic:** Distributed state

The interviewer's prompt: *"You need to store user session data. Keep it in-process in a ConcurrentHashMap or push it to Redis?"*

Thinking out loud: 1. *"How many instances of this service run? If one, in-memory is genuinely simpler. If three behind a load balancer, sticky sessions become a*

deployment headache." 2. "What happens on restart? In-memory means every active session is lost on deploy."

The defensible answer: - More than one instance, or rolling deploys without sticky sessions: Redis. Externalize the state so any instance can serve any request. - Single-instance, low-stakes, sessions can rebuild on restart: in-memory is fine and faster.

The tradeoff: - Redis adds a network hop per session access — roughly 0.5–1ms on the same VPC, more if cross-region. - Operational cost: another thing that can fail, needs monitoring, needs eviction policy, needs persistence config.

When the OTHER option wins: - A websocket gateway where each connection is naturally pinned to one instance → in-memory wins. - An admin tool with 10 users → don't pay the Redis tax for nothing.

What NOT to say: - "Always externalize state, it's a 12-factor rule" — context matters; a CLI tool doesn't need Redis. - "Just use sticky sessions" — works until a node dies mid-deploy and users get logged out.

Common follow-up: "What about Hazelcast or Infinispan as embedded-distributed alternatives?"

Scenario 10 · ORM (JPA/Hibernate) vs JdbcTemplate for a reporting endpoint

Asked at: TCS Digital · Infosys · Accenture · Difficulty: Medium · Topic: Persistence

The interviewer's prompt: "You're building a reporting endpoint that joins 5 tables and aggregates 100k rows. JPA/Hibernate or JdbcTemplate?"

Thinking out loud: 1. "Reporting queries usually return projections, not full entity graphs. JPA's entity-lifecycle machinery, dirty checking, and proxy unwrapping is pure overhead here." 2. "For complex SQL with window functions, CTEs, or vendor-specific tricks, hand-written SQL is always going to be cleaner than JPQL or Criteria."

The defensible answer: - JdbcTemplate (or jOOQ, or a `@Query(nativeQuery = true)` JPA repo). Map directly to a DTO via `RowMapper` . No entity overhead, no

lazy-loading surprises. - Keep JPA for the transactional CRUD entity layer — it earns its keep there.

The tradeoff: - You write SQL by hand — more typing, less compile-time safety unless you use jOOQ. - Two persistence paradigms in one codebase — developers must context-switch.

When the OTHER option wins: - Simple lookups returning fully-managed entities used elsewhere → JPA repository is one line. - Heavy domain logic where entities participate in business rules → keep JPA so cascades and lifecycle callbacks work.

What NOT to say: - "JPA is slow, always use raw JDBC" — wrong; JPA is fine for entity CRUD, it's just a poor fit for reports. - "Hibernate handles everything, don't worry" — until the N+1 query problem surfaces in production.

Common follow-up: "What's the N+1 query problem and how would you detect it in production?"

Scenario 11 · Field injection vs constructor injection

Asked at: Walmart · Atlassian · Razorpay · Difficulty: Easy-Medium · Topic: Spring DI

The interviewer's prompt: "In a Spring service class, do you use `@Autowired` on the field or pass dependencies via constructor?"

Thinking out loud: 1. "What does testability look like? With field injection I need Spring or reflection to inject a mock. With constructor injection I just `new MyService(mockA, mockB)` in a plain JUnit test." 2. "Field injection hides what the class needs — you can't tell from the constructor signature."

The defensible answer: - Constructor injection. Dependencies are explicit, fields can be `final` (true immutability), and unit tests don't need any Spring machinery. - With Lombok `@RequiredArgsConstructor` or Java 17 records, the boilerplate is essentially zero.

The tradeoff: - Constructors with 10 dependencies look ugly — but that's a useful smell; the class probably violates SRP. - Slightly more typing in plain Java without Lombok.

When the OTHER option wins: - Genuinely circular dependencies that can't be redesigned → setter injection or `ObjectProvider` deferred resolution. - Quick prototypes where you'll throw the code away in a week → not really, but some teams do it.

What NOT to say: - "Field injection is fine, it's the default" — Spring's own docs deprecate it. - "Constructor injection is too much boilerplate" — `@RequiredArgsConstructor` removes that argument entirely.

Common follow-up: "How would you handle a circular dependency between two services?"

Scenario 12 · WebClient vs RestClient vs RestTemplate

Asked at: Cred · Razorpay · Swiggy · **Difficulty:** Medium · **Topic:** HTTP clients

The interviewer's prompt: "You need an HTTP client to call a downstream service. WebClient, RestClient, or RestTemplate?"

Thinking out loud: 1. "RestTemplate is in maintenance mode — no new features, only bug fixes. Spring's own docs say start new development with WebClient or RestClient." 2. "WebClient is reactive (returns Mono/Flux). RestClient (Spring 6.1+) is a synchronous fluent API built on the same HTTP backbone."

The defensible answer: - New blocking codebase on Spring 6.1+: RestClient. Same fluent API as WebClient, synchronous semantics, no Reactor in your stack traces. - Reactive codebase (WebFlux): WebClient — it's native to the model. - Legacy code: keep RestTemplate for now, migrate when you touch the endpoint.

The tradeoff: - WebClient pulls in Reactor even if you `.block()` everywhere — your stack traces include reactive frames. - RestClient is newer (Spring 6.1) — older projects on Spring 5 don't have it.

When the OTHER option wins: - A massive legacy codebase on Spring 5 → not worth migrating to RestClient for one new endpoint; stay with RestTemplate. - Need true backpressure and streaming response → WebClient even in a blocking service.

What NOT to say: - "RestTemplate is deprecated, never use it" — it's in maintenance mode, not deprecated; existing code can stay. - "Just use WebClient.block() everywhere" — defeats the reactive model and adds overhead for zero benefit.

Common follow-up: "Why is WebClient.block() considered an anti-pattern?"

Scenario 13 · Stream vs traditional for-loop for a transformation

Asked at: TCS · Infosys · Wipro · **Difficulty:** Easy · **Topic:** Java 8+ idioms

The interviewer's prompt: "You're mapping a list of 50 items to a list of DTOs. Stream API or a traditional for-loop?"

Thinking out loud: 1. "For 50 items, performance is irrelevant — both finish in microseconds. The question is readability." 2. "A simple `map` → `collect` pipeline reads as 'transform each element' — exactly the intent."

The defensible answer: - Stream for declarative transforms (`map` , `filter` , `collect`). It expresses the intent at a higher level than the mechanics of iteration. - Traditional loop when there's complex state across iterations, multiple early exits, or checked-exception handling that lambdas make awkward.

The tradeoff: - Stream pipelines hide allocation costs (intermediate operations create objects); for hot paths this can show up. - Debugger stack frames inside a stream are noisier than a plain for-loop. - Checked exceptions inside lambdas force ugly wrappers.

When the OTHER option wins: - Inner loops at high frequency where every allocation matters → traditional `for` with primitive arrays. - Heavy checked exception handling → stick with `for` to avoid `try/catch` inside lambdas.

What NOT to say: - "Streams are always faster" — for small collections, the setup cost can make them slower than a for-loop. - "For-loops are old-school, always use streams" — context matters; clarity wins.

Common follow-up: "When is `parallelStream` actually worth it?"

Scenario 14 · Lambda vs method reference

Asked at: Most Java interviews · **Difficulty:** Easy · **Topic:** Java 8+ idioms

The interviewer's prompt: "Inside a `stream.map(...)`, you can write `s -> s.toUpperCase()` or `String::toUpperCase`. Which do you prefer and why?"

Thinking out loud: 1. "Method references are more concise and read like English — 'map each string to its uppercase form'." 2. "But they only work when the lambda body is exactly a single method call with arguments in order."

The defensible answer: - Method reference when it's a clean 1:1 mapping — `String::toUpperCase`, `User::getId`, `System.out::println`. - Lambda when you need to do anything more than dispatch — even one extra step like `s -> s.toUpperCase().trim()`.

The tradeoff: - Method references can be ambiguous to readers unfamiliar with the target's signature — `Foo::bar` could be instance, static, or bound. - IDE refactors sometimes rewrite one into the other without explanation, hurting git blame.

When the OTHER option wins: - A lambda that needs to capture local context → can't use method reference. - Reader audience that's junior — explicit lambda with parameter names is sometimes clearer.

What NOT to say: - "Method references are always shorter, so always use them" — readability beats brevity for junior team members. - "Lambdas are slower than method references" — JIT optimizes both to the same thing.

Common follow-up: "What's the difference between bound, unbound, and constructor method references?"

Scenario 15 · ArrayList vs ArrayDeque for a work queue

Asked at: Flipkart · Walmart · **Difficulty:** Medium · **Topic:** Collections

The interviewer's prompt: "You need a FIFO queue for in-process work items. ArrayList, ArrayDeque, or LinkedList?"

Thinking out loud: 1. "ArrayList for queue semantics is terrible — `remove(0)` is $O(n)$ because it shifts everything left." 2. "LinkedList works but allocates a node per element; cache locality is poor."

The defensible answer: - ArrayDeque. Backed by a circular array — $O(1)$ add and remove at both ends, excellent cache locality, no per-element node overhead. - For a thread-safe variant, `ArrayBlockingQueue` (bounded) or `ConcurrentLinkedQueue` (unbounded, lock-free).

The tradeoff: - ArrayDeque is not thread-safe — multiple producers/consumers need a concurrent variant. - Doesn't allow null elements (LinkedList does).

When the OTHER option wins: - Need a thread-safe blocking queue → `ArrayBlockingQueue` or `LinkedBlockingQueue`. - Need a deque AND a list interface together → LinkedList implements both.

What NOT to say: - "Use LinkedList for queues, that's what it's for" — outdated advice; ArrayDeque dominates in modern Java. - "ArrayList is fine, just use `remove(0)`" — $O(n)$ per dequeue, kills throughput at scale.

Common follow-up: "What's the difference between offer and add on a Queue interface?"

Scenario 16 · Bean Validation vs manual validation

Asked at: TCS Digital · Razorpay · Swiggy · **Difficulty:** Easy-Medium · **Topic:** Validation

The interviewer's prompt: "For request DTOs, use Bean Validation annotations (`@NotBlank`, `@Size`) or write manual `if (x == null) throw ...` checks?"

Thinking out loud: 1. "Annotations are declarative — the constraints live with the field, visible at a glance." 2. "For business rules that span multiple fields or need DB lookups (uniqueness check), annotations alone don't cut it."

The defensible answer: - Bean Validation (`jakarta.validation`) for field-level rules: not-null, size bounds, regex format, enum-of-values. - Manual validation in the service layer for cross-field rules and rules needing DB or external state. - `@Valid` on the controller method triggers automatic validation — Spring wires it into `MethodArgumentNotValidException` .

The tradeoff: - Annotation-based validators are reflective at runtime — not as fast as direct code, but rarely the bottleneck. - Custom constraint annotations add ceremony (annotation interface + validator class) — only pays off for reused rules.

When the OTHER option wins: - One-off validation used in exactly one place → manual `if` check is shorter than writing a constraint annotation. - Need to fail fast on the FIRST validation error (annotations collect all) → manual.

What NOT to say: - "Don't validate at the controller, do it in the service" — both layers should validate; controller for shape, service for business rules. - "Just write manual checks everywhere" — duplicates effort and rules drift apart.

Common follow-up: "How do you do cross-field validation with annotations?"

Scenario 17 · Lombok vs Java 17 records for DTOs

Asked at: Walmart Global Tech · Cred · Difficulty: Easy-Medium · Topic: Java 17 features

The interviewer's prompt: "For an immutable DTO, would you use Lombok `@Value` / `@Data` or a Java 17 record?"

Thinking out loud: 1. "Records are language-level — no annotation processor, no compile-time magic, fewer build issues." 2. "But records are STRICTLY immutable. If I need a setter or a builder, records don't fit."

The defensible answer: - Records for immutable data carriers (DTOs, value objects, response payloads). Built into the language, auto-generated `equals/`

`hashCode/toString` , deconstruction patterns coming. - Lombok for entity classes (need mutable state for JPA), builders (**`@Builder`**), or where you need fields that can be reassigned.

The tradeoff: - Records lack a builder by default — for DTOs with 15 fields, construction is verbose. - Lombok can break IDE refactoring and Java agent compatibility (sometimes interferes with Mockito 5+).

When the OTHER option wins: - Mutable JPA entities → can't use records; use Lombok or hand-written code. - Need a fluent builder → Lombok **`@Builder`** is simpler than rolling your own constructor.

What NOT to say: - "Lombok is dead, records replace it" — records cover only one of Lombok's many features. - "Records can't be entities" — partially true; Hibernate supports them as **`@Embeddable`** , but as **`@Entity`** they're awkward due to lifecycle requirements.

Common follow-up: "What's a compact constructor in a record and when would you use one?"

Scenario 18 · enum vs sealed interface for a fixed set of types

Asked at: Atlassian · **Cred:** · **Difficulty:** Medium · **Topic:** Java 17 features

The interviewer's prompt: "You have three payment methods — Card, UPI, NetBanking. Each carries different data. Enum or sealed interface with records?"

Thinking out loud: 1. "Enums are great when each variant carries the SAME shape — name + display label." 2. "When each variant carries DIFFERENT fields (card number vs UPI handle vs bank code), an enum forces you to put all fields on every variant and leave them null."

The defensible answer: - Sealed interface + record implementations. Each variant carries its own fields, pattern matching on switch becomes exhaustive — the compiler tells you when you add a variant and miss a case. - Enum when all variants share the same shape (status codes, log levels, day of week).

The tradeoff: - Sealed hierarchies are slightly more verbose to declare than an enum. - Pattern matching switch with sealed types requires Java 21 for full features.

When the OTHER option wins: - All variants are interchangeable values with no per-variant data → enum is one line per variant. - Need `valueOf(String)` and `values()` reflection → enum gives those for free.

What NOT to say: - "Sealed interfaces replace enums" — they cover different use cases; both are alive. - "Enums can have fields, just put everything there" — leads to nullable bloat and runtime branching that the type system can't help with.

Common follow-up: "How does pattern matching with sealed types give you exhaustiveness checking?"

Scenario 19 · YAML vs properties for Spring configuration

Asked at: TCS Digital · Walmart · **Difficulty:** Easy · **Topic:** Spring config

The interviewer's prompt: " `application.yml` or `application.properties` ?"

Thinking out loud: 1. "YAML is hierarchical — nested keys read better than dot-separated properties when you have deep structure." 2. "Properties are simpler — every line is `key=value` , no indentation issues, no parser quirks."

The defensible answer: - YAML for non-trivial config with nested structures, lists, and profile-specific overrides. The hierarchy maps directly to `@ConfigurationProperties` classes. - Properties for tiny apps or when the team is wary of YAML's indentation gotchas.

The tradeoff: - YAML's whitespace-sensitive — one wrong space and the parser silently misreads structure. - Properties files don't natively support lists — you end up with `app.list[0]=` , `app.list[1]=` which is ugly.

When the OTHER option wins: - Sensitive config injected via Kubernetes ConfigMap / Vault → properties format integrates more smoothly with most config-sync tools. - A small CLI app with 5 settings → properties is shorter.

What NOT to say: - "YAML is always better" — it isn't; sometimes simpler tooling matters more. - "Just use environment variables for everything" — works for secrets, painful for structured config.

Common follow-up: "How do Spring profiles interact with YAML files?"

Scenario 20 · Interface vs abstract class for shared behavior

Asked at: Most Java interviews · **Difficulty:** Medium · **Topic:** OOP design

The interviewer's prompt: "You have shared behavior across three classes. Interface with default methods or abstract class?"

Thinking out loud: 1. "Does the shared behavior need state (fields)? Interfaces can't carry state — only constants." 2. "How many interfaces should a class implement? Multiple is fine; multiple abstract class inheritance isn't."

The defensible answer: - Interface with default methods when the shared behavior is stateless and you want the implementing class to remain free to extend a different class. - Abstract class when you need shared mutable state, constructors, or template-method patterns that hold protected fields.

The tradeoff: - Default methods in interfaces can collide if a class implements two interfaces with the same default — the compiler forces explicit resolution. - Abstract class locks the implementer into single-inheritance.

When the OTHER option wins: - Need three orthogonal capability sets → multiple interfaces (`Comparable` , `Serializable` , `AutoCloseable`). - Heavy shared state + a few template methods → abstract class.

What NOT to say: - "Always use interfaces, abstract classes are dead" — interfaces with default methods don't carry state; abstract classes still earn their keep. - "Abstract classes are for inheritance, interfaces for behavior" — too neat; default methods blur the line.

Common follow-up: "What happens when two interfaces both provide a default method with the same signature?"

Scenario 21 · Primitive vs boxed in a hot loop

Asked at: Atlassian · Walmart · **Difficulty:** Medium · **Topic:** Performance

The interviewer's prompt: "In an inner loop processing millions of elements, would you use `int[]` or `List<Integer>`?"

Thinking out loud: 1. "Each `Integer` is a 16-byte object on the heap with a reference. `int` is a 4-byte primitive on the stack. The difference is 4x memory and an indirection on every access." 2. "Autoboxing in the hot path allocates short-lived objects — GC pressure follows."

The defensible answer: - `int[]` (or `IntStream` for streams). Stays in primitive form, cache-friendly, no boxing. - For sets and maps of primitives, libraries like Eclipse Collections or HPPC provide `IntHashSet`, `IntIntMap`.

The tradeoff: - Primitive arrays don't fit into the Collections framework cleanly — can't pass to `List<Integer>` APIs. - Generic algorithms can't be written over primitive arrays without separate specialization.

When the OTHER option wins: - Cold paths or small collections → `List<Integer>` is more readable. - Need to leverage Collections framework APIs (filtering, copying with utility methods) → boxed.

What NOT to say: - "Autoboxing is free, don't worry about it" — measurably slow in tight loops, GC visible in profilers. - "Always use primitives" — readability matters more in cold paths.

Common follow-up: "What does Project Valhalla aim to fix?"

Scenario 22 · Map vs EnumMap

Asked at: Walmart · Cred · **Difficulty:** Medium · **Topic:** Collections

The interviewer's prompt: "Your map keys are an enum. Plain `HashMap<MyEnum, V>` or `EnumMap<MyEnum, V>`?"

Thinking out loud: 1. "EnumMap is backed by a simple array indexed by `ordinal()`. No hashing, no buckets, no collisions." 2. "HashMap on enum keys works — hash + bucket lookup — but it's overkill for a fixed, dense set of keys."

The defensible answer: - EnumMap. Faster (array indexing), more memory-efficient (no Entry objects, no hash table), iterates in enum declaration order. - Same reasoning applies to `EnumSet` when keys-only is what you want.

The tradeoff: - EnumMap requires the enum class at construction time — slightly more code than `new HashMap<>()`. - Not thread-safe — needs `Collections.synchronizedMap` if shared.

When the OTHER option wins: - Map type erased to `Map<Object, V>` for generic plumbing → can't construct EnumMap there. - Rarely accessed config map where allocation cost doesn't matter → HashMap is fine.

What NOT to say: - "HashMap is general-purpose, use it everywhere" — leaves easy wins on the table. - "EnumMap is the same as HashMap, just typed" — different data structure entirely.

Common follow-up: "How is EnumSet internally represented for enums with 64+ values?"

Scenario 23 · synchronized list vs CopyOnWriteArrayList for listeners

Asked at: Flipkart · **Cred:** · **Difficulty:** Medium · **Topic:** Concurrent collections

The interviewer's prompt: "You're maintaining a listener list — 10 listeners, registered rarely, fired 1000 times per second. Synchronized list or CopyOnWriteArrayList?"

Thinking out loud: 1. "Reads dominate by orders of magnitude. Writes are once-in-a-blue-moon (when a listener registers)." 2. "CopyOnWriteArrayList means reads are lock-free and the iterator is snapshot-stable. Writes copy the whole array — expensive but rare."

The defensible answer: - CopyOnWriteArrayList. Reads are completely unsynchronized — perfect for event-fan-out and observer patterns. - The write cost (full array copy) is negligible because writes are infrequent.

The tradeoff: - Every write allocates a new array — bad if you're writing in a loop. - Iterator does NOT see writes that happen after iteration started — usually desirable for listener fan-out, sometimes surprising.

When the OTHER option wins: - Many writers, few readers → synchronized list or ConcurrentLinkedQueue . - Listeners must be removed and re-added often → COW's write cost compounds.

What NOT to say: - "CopyOnWriteArrayList is too expensive, never use it" — wrong for read-heavy fan-out; it's exactly what it's for. - "Just use Collections.synchronizedList" — reads block on writes, fan-out throughput suffers.

Common follow-up: "What's the iterator behavior of CopyOnWriteArrayList?"

Scenario 24 · Java serialization vs JSON vs Protobuf for inter-service messages

Asked at: Razorpay · Swiggy · Flipkart · Difficulty: Hard · Topic: Serialization

The interviewer's prompt: "For messages between two of your internal services, would you use Java serialization, JSON, or Protobuf?"

Thinking out loud: 1. "Java serialization is the classic security nightmare — Oracle has been trying to deprecate it for years. Untrusted input deserialization is a known RCE vector." 2. "JSON is human-readable, polyglot-friendly, but verbose on the wire and slower to parse than binary formats." 3. "Protobuf is schema-first, compact, fast, but requires shared .proto files and code generation."

The defensible answer: - High-throughput / latency-sensitive inter-service: Protobuf (with gRPC). Compact, schema-evolution rules are clear (field numbers). - Lower volume or public-facing: JSON. Universal tooling, easy to debug with curl , every language speaks it. - Java serialization: avoid in any new code. Maybe acceptable for a private trusted JVM-only RMI link, but even then the modern alternatives are better.

The tradeoff: - **Protobuf adds build tooling (protoc, generated classes), version skew complexity.** - **JSON forgives schema mistakes silently — typo a field name and you get null instead of an error.** - **Java serialization is JVM-only and brittle across version changes.**

When the OTHER option wins: - **A pure Java RPC channel with trusted endpoints behind a strict network boundary → Java serialization technically works (still avoid it).** - **Need maximum readability for debugging or audit → JSON.**

What NOT to say: - **"Java serialization is fine for internal services" — security and brittleness both argue against it.** - **"Always use Protobuf for performance" — for low-volume APIs the developer overhead isn't worth it.**

Common follow-up: **"What's the difference between Avro and Protobuf for schema evolution?"**

Scenario 25 · Java 21 virtual threads vs traditional Tomcat thread pool

Asked at: Razorpay · Cred · Atlassian · **Difficulty:** Hard · **Topic:** Java 21 features

The interviewer's prompt: **"You're upgrading to Java 21. Switch the Tomcat connector to virtual threads or keep the platform thread pool?"**

Thinking out loud: 1. **"What does each request do? If it spends most of its time waiting on I/O (DB, downstream HTTP), virtual threads are a massive win — one platform thread can handle thousands of virtual threads suspended on blocking calls."** 2. **"If requests are CPU-bound, virtual threads don't help — you're still limited by physical cores."**

The defensible answer: - **I/O-bound, blocking-style code: switch to virtual threads.** Spring Boot 3.2+ has `spring.threads.virtual.enabled=true`. Throughput goes up, p99 latency goes down under load. - **Verify your stack — JDBC drivers, JNI libraries, `synchronized` blocks on shared resources can pin virtual threads to carriers and erase the benefit.**

The tradeoff: - `synchronized` blocks pin the virtual thread to its carrier. Hibernate's `SessionImpl` and some connection pools use `synchronized` — pinning can starve your carrier pool. - **ThreadLocal usage spikes memory because**

each virtual thread carries its own — millions of virtual threads × heavy ThreadLocal = OOM.

When the OTHER option wins: - CPU-bound work → traditional thread pool sized to cores. - Heavy `synchronized` -using code paths → pinning negates the benefit; profile first.

What NOT to say: - "Virtual threads make everything faster" — only helps I/O-bound workloads. - "Just flip the flag and ship it" — pinning issues require profiling; ThreadLocal patterns may need rework.

Common follow-up: "What does 'pinning' mean for a virtual thread?"

Scenario 26 · JIT vs AOT (GraalVM native image)

Asked at: Cred · Atlassian · Difficulty: Hard · Topic: JVM

The interviewer's prompt: "For a Spring Boot microservice that runs in Kubernetes, JIT (HotSpot) or AOT (GraalVM native image)?"

Thinking out loud: 1. "Native image gives a 50ms startup and ~100MB memory footprint. HotSpot is more like 4s startup and 400MB." 2. "But native image loses JIT's adaptive optimization — peak throughput is typically 10-30% lower because no profile-guided inlining."

The defensible answer: - Short-lived workloads (serverless, scale-to-zero, batch jobs): AOT native image. Cold-start dominance changes the math. - Long-running services with stable load: HotSpot JIT. Peak throughput wins after warmup; the slow startup is a one-time cost.

The tradeoff: - Native image requires AOT-friendly code — reflection needs hints, dynamic proxy needs registration. Frameworks like Spring AOT generate these but it's not free. - Debugging a native image is harder; observability tools have limited support.

When the OTHER option wins: - Scale-to-zero Lambda-style → AOT pays for itself in cold starts. - High-throughput long-running service → JIT's adaptive optimization beats AOT after warmup.

What NOT to say: - "Native image is faster than JVM, period" — peak throughput is often LOWER than HotSpot after warmup. - "JIT is always better, GraalVM is hype" — for short-lived processes, JIT never reaches its peak.

Common follow-up: "What's profile-guided optimization in native image?"

Scenario 27 · Optional.orElse vs orElseGet

Asked at: Walmart · Cred · TCS Digital · **Difficulty:** Easy-Medium · **Topic:** Optional API

The interviewer's prompt: " `optional.orElse(buildExpensiveDefault())` vs `optional.orElseGet(() -> buildExpensiveDefault())` . Which one and why?"

Thinking out loud: 1. " `orElse(x)` evaluates `x` EAGERLY — `buildExpensiveDefault()` runs every time, even when the optional has a value." 2. " `orElseGet(supplier)` evaluates the supplier LAZILY — only when the optional is empty."

The defensible answer: - `orElseGet(...)` when the default is expensive to construct (DB lookup, network call, large allocation). The supplier only fires when actually needed. - `orElse(...)` when the default is a cheap constant or already-computed value.

The tradeoff: - `orElseGet` requires a supplier lambda — slightly more typing. - `orElse` accidentally executes side effects in the "default" argument even when not needed — a quiet bug source.

When the OTHER option wins: - Default is a literal constant (`orElse("UNKNOWN")`) → `orElse` is shorter and the eagerness is harmless. - Default is a precomputed local variable → `orElse` is fine.

What NOT to say: - "They're the same, use whichever" — they're not; the eagerness difference causes real bugs. - "Always use `orElseGet`" — overkill for cheap constants.

Common follow-up: "What's the difference between `Optional.map` and `Optional.flatMap`?"

Scenario 28 · `String.format` vs `StringBuilder` vs concatenation

Asked at: TCS · Infosys · Difficulty: Easy-Medium · Topic: Strings

The interviewer's prompt: "Building a log message with 5 dynamic values.

`String.format("%s %s", a, b)`, `StringBuilder`, or `"x = " + a + ", y = " + b`?"

Thinking out loud: 1. "Modern javac compiles string concatenation to `invokedynamic` with `StringConcatFactory` — very efficient, no manual `StringBuilder` needed." 2. "`String.format` is template-clean but uses a `Formatter` object internally — significantly slower in a hot path."

The defensible answer: - Concatenation with `+` for simple cases — javac is smart, the bytecode is efficient. - `StringBuilder` for loops where you're building a string across iterations. - `String.format` for human-facing formatted output (currency, dates, alignment) — when the format string is the documentation.

The tradeoff: - `String.format` is roughly 10x slower than concatenation due to parsing the format string each call. - `StringBuilder` in a single-statement concat is unnecessary; the compiler already generates equivalent code.

When the OTHER option wins: - Logging frameworks: use parameterized logging (`log.info("user={} action={}", user, action)`). The format string is parsed once and arguments are skipped entirely if the level is disabled. - Very large concatenated outputs in a loop → explicit `StringBuilder` gives the most control.

What NOT to say: - "Always use `StringBuilder` for concatenation" — micro-optimization that's harmful in readability and unnecessary post-Java 9. - "`String.format` is always fine" — it's measurably slow in tight loops.

Common follow-up: "What's parameterized logging and why is it faster than `log.info("x=" + x)`?"

Scenario 29 · ThreadLocal vs explicit context propagation

Asked at: Razorpay · Swiggy · **Difficulty:** Hard · **Topic:** Concurrency

The interviewer's prompt: "You need to carry a request ID through 5 layers of code without polluting method signatures. ThreadLocal or pass an explicit context object?"

Thinking out loud: 1. "ThreadLocal is invisible to callers — pro and con. The context just appears." 2. "With virtual threads or reactive code, ThreadLocal becomes tricky — propagation across thread boundaries needs MDC adapters or **ScopedValue** (Java 21+ preview)."

The defensible answer: - Logging context (MDC) and tracing IDs: ThreadLocal via Slf4j MDC — universal and well-supported. - Application-level state crossing async boundaries: explicit context parameter — survives executor handoffs and reactive subscriptions. - Java 21+: explore **ScopedValue** which is built for the structured-concurrency world.

The tradeoff: - ThreadLocal leaks if you don't clean up — pooled threads carry stale values into the next request. - Explicit context bloats every signature; refactoring to add a field touches every layer.

When the OTHER option wins: - A small codebase where one context object naturally fits → explicit beats hidden. - Reactive / virtual thread heavy → explicit context survives async hops without surprises.

What NOT to say: - "ThreadLocal is always bad" — it's the standard pattern for MDC and works fine when scoped correctly. - "Explicit context is too verbose" — sometimes explicit IS the point; hidden state is harder to debug.

Common follow-up: "What's a ThreadLocal memory leak in a Tomcat redeploy?"

Scenario 30 · Optimistic vs pessimistic locking on a high-contention row

Asked at: Razorpay · Cred · Difficulty: Hard · Topic: Transactions

The interviewer's prompt: "You're updating a shared `inventory` row with 200 concurrent decrements per second. Optimistic locking (`@Version`) or pessimistic `SELECT ... FOR UPDATE` ?"

Thinking out loud: 1. "Optimistic assumes conflicts are rare — version check at commit, retry on collision. With 200 concurrent updaters on ONE row, conflicts are NOT rare." 2. "Retry storms can amplify load — each conflict means another transaction attempt."

The defensible answer: - Pessimistic `SELECT ... FOR UPDATE` for a high-contention single row. Serializes access, no retry amplification. - Or redesign: use a counter table that updates atomically (`UPDATE inventory SET qty = qty - 1 WHERE id = ? AND qty > 0`) and check the rowcount — no lock needed, no retry.

The tradeoff: - Pessimistic locks block other transactions — long-running locks can cascade into application-wide stalls. - Atomic SQL update sacrifices the entity-based programming model — bypasses Hibernate dirty checking.

When the OTHER option wins: - Truly rare conflicts (user editing their profile) → optimistic with `@Version` is cheaper. - Distributed system without a single DB → optimistic with idempotency tokens is often the only option.

What NOT to say: - "Always use optimistic locking, it's modern" — wrong at high contention; retry storms make things worse. - "Just lock the row" — depends on the rest of the workload; locks block neighbors.

Common follow-up: "What's the retry storm pattern and how do you mitigate it?"

Scenario 31 · Spring @Transactional propagation: REQUIRED vs REQUIRES_NEW

Asked at: TCS Digital · Walmart · Difficulty: Medium · Topic: Spring transactions

The interviewer's prompt: "Inside `@Transactional` method A, you call method B which also writes to the DB. Should B use propagation `REQUIRED` or `REQUIRES_NEW`?"

Thinking out loud: 1. "REQUIRED (the default) joins the existing transaction — B's writes commit or roll back with A." 2. "REQUIRES_NEW suspends A's transaction and starts a fresh one for B — B commits independently of A."

The defensible answer: - REQUIRED for normal business logic — atomic with the caller. - REQUIRES_NEW for things that should persist regardless of the outer transaction outcome — audit logs, error event records, retry counters.

The tradeoff: - REQUIRES_NEW uses an extra DB connection from the pool — under load this can drain the pool. - A failure in the inner transaction doesn't roll back the outer — surprising if you assume nesting.

When the OTHER option wins: - Logging that must survive a rollback → REQUIRES_NEW. - Anything that's logically part of the same business operation → REQUIRED.

What NOT to say: - "Just use REQUIRES_NEW for safety" — drains connection pool, can deadlock. - "Nested transactions in Spring work like savepoints" — only with NESTED propagation, not REQUIRES_NEW.

Common follow-up: "What's the difference between NESTED and REQUIRES_NEW?"

Scenario 32 · Spring profile vs feature flag for environment toggles

Asked at: Walmart · Atlassian · Difficulty: Medium · Topic: Spring config

The interviewer's prompt: "You need to enable a feature in staging but not production. Spring profile or a feature flag?"

Thinking out loud: 1. "Profiles are build-time-ish — they select different bean wiring at startup. Switching means restart." 2. "Feature flags are runtime — flip a toggle in a config service and behavior changes without redeploy."

The defensible answer: - Spring profile when the difference is structural — different beans (mock email sender in staging, real one in prod), different data sources. - Feature flag when you want to gradually roll out, A/B test, or kill-switch a feature in production without a deploy.

The tradeoff: - Profiles require a restart to change — no gradual rollout. - Feature flags add runtime branches everywhere they're checked — code paths multiply.

When the OTHER option wins: - Hardcoded environment-specific bean wiring → profile is cleaner. - A new risky feature needing 1% → 100% rollout → feature flag with a tool like LaunchDarkly or a home-grown table.

What NOT to say: - "Profiles and feature flags are the same" — they solve different problems. - "Just use profiles for everything" — limits operational agility severely.

Common follow-up: "How would you safely roll out a feature flag to 10% of users?"

Scenario 33 · Eager vs lazy fetch in JPA

Asked at: TCS Digital · Infosys · **Difficulty:** Medium · **Topic:** JPA

The interviewer's prompt: "For a @OneToMany association, eager or lazy?"

Thinking out loud: 1. "Eager loads the children whenever the parent is loaded — convenient but causes massive joins or N+1 patterns when you didn't want the children." 2. "Lazy loads only when accessed — but accessing outside a session throws LazyInitializationException."

The defensible answer: - Lazy by default for @OneToMany and @ManyToMany . Fetch eagerly only when you genuinely always want the association. - Use entity graphs or JOIN FETCH to eagerly load on a per-query basis when needed.

The tradeoff: - Lazy fetching can cause LazyInitializationException when the entity is detached (DTO mapping outside the transaction). - Eager fetching always pays the cost, even when you only needed the parent's id.

When the OTHER option wins: - Always-needed @ManyToOne to a small reference table (currency, country) → eager is fine. - Reporting use case where every query needs both → eager simplifies and avoids forgotten fetches.

What NOT to say: - "Always eager for safety" — kills performance, leads to the famous Hibernate "select * from world" pattern. - "Just use FetchType.LAZY"

everywhere" — true default, but you must couple it with discipline about fetch plans.

Common follow-up: "What's the N+1 query problem and how do you fix it?"

Scenario 34 · DTO mapping by hand vs MapStruct vs ModelMapper

Asked at: TCS Digital · Walmart · Difficulty: Easy-Medium · Topic: Mapping

The interviewer's prompt: "You need to map between entities and DTOs in 30 places. Hand-written, MapStruct, or ModelMapper?"

Thinking out loud: 1. "Hand-written is verbose but transparent — what you see is what runs." 2. "MapStruct is compile-time code generation — no runtime reflection, IDE can step through the generated code." 3. "ModelMapper is runtime reflection — convenient but slower and silent on mismatches."

The defensible answer: - MapStruct. Compile-time safety, no reflection cost, generated code is debuggable, type mismatches caught at build time. - Hand-written for 1-2 conversions where the annotation processing overhead isn't worth it. - ModelMapper: avoid for new code — convenience trades against performance and quiet field-name-matching bugs.

The tradeoff: - MapStruct adds an annotation processor — build setup matters, IDE plugin needed for smooth experience. - Hand-written maps drift from entity changes — easy to forget a new field.

When the OTHER option wins: - One-off mapping → hand-written is faster than learning MapStruct conventions. - Codebase that already uses ModelMapper → consistency matters; switching gradually is fine.

What NOT to say: - "Reflection mappers are fine, performance doesn't matter" — silent typos cause production bugs. - "Always write by hand" — drift and missing-field bugs are real.

Common follow-up: "How does MapStruct handle nested object mapping?"

Scenario 35 · @Cacheable vs manual Caffeine cache

Asked at: Razorpay · Cred · **Difficulty:** Medium · **Topic:** Caching

The interviewer's prompt: "Spring's `@Cacheable` or instantiate a Caffeine cache yourself?"

Thinking out loud: 1. "`@Cacheable` is annotation-driven — declarative, works via Spring AOP proxies." 2. "Direct Caffeine gives you full control over loading behavior, weight, eviction, and metrics — none of which `@Cacheable` makes easy."

The defensible answer: - `@Cacheable` for simple, idempotent method-level caching with default semantics. - Direct Caffeine when you need: async loading, refresh-ahead, weight-based eviction, fine-grained stats, or to cache method-level intermediate results.

The tradeoff: - `@Cacheable` proxy semantics mean self-invocation skips the cache (same as `@Async`, `@Transactional` proxy issue). - Direct Caffeine adds explicit wiring — more code, but every caching decision is visible.

When the OTHER option wins: - Quick one-line caching on a service method → `@Cacheable`. - High-throughput cache with stats and tuning → direct Caffeine + `@Configuration` bean.

What NOT to say: - "Caches are caches, it doesn't matter" — misses both the proxy gotcha and the tuning ceiling. - "Always use `@Cacheable`, it's the Spring way" — limits you to declarative semantics; not always enough.

Common follow-up: "Why does calling a `@Cacheable` method from within the same bean skip the cache?"

Scenario 36 · Stateless service vs stateful actor for per-user state

Asked at: Cred · Razorpay · **Difficulty:** Hard · **Topic:** Architecture

The interviewer's prompt: "For real-time game state per user, stateless service backed by Redis or stateful actor-style holding state in memory?"

Thinking out loud: 1. "Stateless services scale horizontally trivially — any node can serve any request. The state hop adds latency on every interaction." 2. "Stateful actors keep state local — fast — but each user must consistently route to the same instance."

The defensible answer: - Stateless + Redis for most CRUD-style or moderate-throughput services. Operational simplicity wins. - Stateful (actor-per-user) when the latency budget can't afford the round-trip on every interaction — sub-millisecond response times, real-time gaming, trading.

The tradeoff: - Stateful needs partition-aware routing (consistent hashing or directory service), failover handling, state replication. - Stateless pays latency on every state read, can scale Redis as a separate concern.

When the OTHER option wins: - 99% read pattern with rare updates → stateless + cache. - 99% per-user-state mutation in real time → stateful actor model.

What NOT to say: - "Stateless is always better, it's 12-factor" — true for many things, not all; trading systems and games disagree. - "Just keep everything in memory" — without partition routing and failover, you'll lose state on the first restart.

Common follow-up: "How does consistent hashing work for actor placement?"

Scenario 37 · @RestController per resource vs facade controller

Asked at: Most Spring shops · **Difficulty:** Easy-Medium · **Topic:** Spring MVC

The interviewer's prompt: "Many small `@RestController` classes per resource or one large facade controller?"

Thinking out loud: 1. "Small controllers per resource read like the URL structure — clear, easy to find." 2. "Facade controllers grow into thousand-line files that nobody wants to touch."

The defensible answer: - One controller per resource (or aggregate). Keeps each file under 200 lines, mirrors REST URL structure, makes ownership clear. - Tests, validation, and exception handlers stay close to the endpoints they govern.

The tradeoff: - More files in the project — navigation matters. - Shared `@ControllerAdvice` and helpers need explicit wiring.

When the OTHER option wins: - Tiny app with 5 endpoints → one controller is fine.
- Heavily coupled endpoints sharing identical setup → facade reduces ceremony.

What NOT to say: - "Put everything in one controller for simplicity" — works until it doesn't, then refactoring is painful. - "One controller per HTTP method" — silly granularity.

Common follow-up: "How do you structure controllers when endpoints span multiple resources?"

Scenario 38 · Spring AOP vs decorator pattern for cross-cutting concerns

Asked at: Walmart · Atlassian · **Difficulty:** Medium-Hard · **Topic:** Spring AOP

The interviewer's prompt: "You need to add timing metrics around 50 service methods. Spring AOP `@Around` advice or manually wrap each call with a decorator?"

Thinking out loud: 1. "AOP gives you one place to put the cross-cutting logic — 50 methods covered with one aspect." 2. "Decorator is explicit — easier to debug, no proxy gotchas, no surprise about whether a self-call is intercepted."

The defensible answer: - AOP for truly cross-cutting concerns: logging, timing, security checks, transaction boundaries — patterns that span many classes. - Decorator when the concern is localized and clarity matters more than terseness — easier to reason about and trivially testable.

The tradeoff: - AOP proxies don't intercept self-invocation (calling `this.method()` from within the same bean). - Decorators multiply class count and add explicit wiring.

When the OTHER option wins: - One specific class needing a wrapper → decorator is straightforward. - Cross-cutting across many beans → AOP is purpose-built.

What NOT to say: - "AOP is always elegant" — proxy edge cases bite when you don't expect them. - "AOP is too magical, never use it" — misses the productivity gain for genuine cross-cutting concerns.

Common follow-up: "What are the proxy types Spring uses (JDK dynamic vs CGLIB)?"

Scenario 39 · Synchronous vs asynchronous notification dispatch

Asked at: Razorpay · Swiggy · **Difficulty:** Medium · **Topic:** Async patterns

The interviewer's prompt: "After a successful payment, you need to send an email and a push notification. Do these in the same request thread or hand off to async?"

Thinking out loud: 1. "What's the user waiting for? They want the payment confirmation page — they don't need the email to be sent before that loads." 2. "If the email service is down, do we want the payment to fail? No — these are independent."

The defensible answer: - Async dispatch via a message broker (RabbitMQ / SQS / Kafka). The request thread commits the payment and publishes an event; downstream consumers handle notifications. - For simpler setups, `@Async` with `@TransactionalEventListener(phase = AFTER_COMMIT)` works — fires after the payment commits successfully.

The tradeoff: - Async adds infrastructure (broker, consumer process, monitoring). - Failure modes split — you need a DLQ pattern and retry strategy.

When the OTHER option wins: - Internal admin tool where the user expects to see "email sent" → sync. - Tiny app with no broker → in-process `@Async` with retries.

What NOT to say: - "Just send the email inline, it's faster to ship" — couples user-facing latency to a third-party email service. - "Always async everything" — sometimes the user genuinely needs the result.

Common follow-up: "What's `@TransactionalEventListener` and how does it interact with rollback?"

Scenario 40 · ResponseEntity vs returning DTO directly

Asked at: TCS Digital · Wipro · **Difficulty:** Easy · **Topic:** Spring MVC

The interviewer's prompt: "In a Spring REST controller, return `ResponseEntity<UserDto>` or `UserDto` directly?"

Thinking out loud: 1. "Returning DTO directly relies on Spring's default 200 status — no headers, no status customization." 2. "ResponseEntity gives explicit control over status, headers, body."

The defensible answer: - ResponseEntity when the endpoint genuinely needs to vary status or set headers (Location header on 201, ETag, custom redirects). - Plain DTO when the endpoint always returns 200 OK with the body — simpler signature, ResponseEntity is needless ceremony.

The tradeoff: - Mixing both styles across a codebase is inconsistent. - ResponseEntity hides the body type behind a wrapper — slightly worse for OpenAPI generation.

When the OTHER option wins: - 201 Created with a Location header → ResponseEntity is the right tool. - 204 No Content → return `ResponseEntity<Void>` is clearest.

What NOT to say: - "Always wrap in ResponseEntity for flexibility" — adds ceremony for no benefit when status never varies. - "Always return DTO directly" — fails when you need status control.

Common follow-up: "What's the difference between @ResponseStatus and ResponseEntity for setting status codes?"

Scenario 41 · Polling vs WebSocket vs Server-Sent Events for real-time updates

Asked at: Razorpay · Swiggy · Zomato · **Difficulty:** Hard · **Topic:** Real-time

The interviewer's prompt: "Order status needs to update in real time on the user's phone. Long-polling, WebSocket, or SSE?"

Thinking out loud: 1. "What's the direction of communication? If only server-to-client, SSE is purpose-built and simpler than WebSocket." **2.** "WebSocket is bidirectional and binary-capable but adds protocol complexity and proxy-friendliness issues."

The defensible answer: - SSE for server-push-only flows (order status, notification feed, live scores). Plain HTTP, automatic reconnect, no protocol negotiation. - WebSocket when bidirectional or binary messages matter (chat, multiplayer games, collaborative editing). - Long-polling as a fallback if SSE/WebSocket can't traverse a hostile network.

The tradeoff: - SSE is unidirectional; client-to-server requires regular HTTP. - WebSocket has more failure modes — proxies, load balancers, version negotiations. - Long-polling holds connections open — more server resources per client.

When the OTHER option wins: - True chat or collaborative editing → WebSocket. - Hostile corporate proxy environment → long-polling is most likely to work.

What NOT to say: - "Always WebSocket for real time" — overkill for one-way push; SSE is often a better fit. - "Polling is fine, just every 2 seconds" — wastes bandwidth, doesn't feel real-time.

Common follow-up: "How does SSE handle reconnect after a network blip?"

Scenario 42 · Hibernate first-level vs second-level cache

Asked at: TCS Digital · Walmart · **Difficulty:** Hard · **Topic:** JPA caching

The interviewer's prompt: "You see repeated identical queries hitting the DB. Hibernate first-level cache, second-level cache, or query cache?"

Thinking out loud: 1. "First-level (session/persistence context) is per-Session — within one transaction, repeat `find()` calls don't hit the DB." **2.** "Second-level cache is shared across sessions — repeated queries across requests can hit the cache."

The defensible answer: - *First-level is automatic — make sure you're inside one transaction for repeated loads to benefit.* - *Second-level for entities that are read-mostly across many sessions (reference data, product catalog).* - *Query cache only when results of specific queries are highly repeatable — and only with second-level enabled.*

The tradeoff: - *Second-level cache adds complexity — invalidation across nodes needs a distributed cache (Infinispan, Redis).* - *Query cache is fragile — any matching entity update invalidates all related query keys.*

When the OTHER option wins: - *Tiny app with single DB and no scale → application-level Caffeine cache may be simpler than Hibernate second-level.* - *Mostly write workload → second-level cache invalidations dominate; turn it off.*

What NOT to say: - *"Just enable all the caches for speed" — invalidation correctness gets very hard.* - *"Second-level cache is dead, use application cache" — depends; second-level integrates with entity lifecycle elegantly.*

Common follow-up: *"What's the cache invalidation problem in a multi-node Hibernate setup?"*

Scenario 43 · Database-generated ID vs UUID

Asked at: *Razorpay · Walmart · Atlassian · Difficulty: Medium · Topic: ID strategy*

The interviewer's prompt: *"For a new entity's primary key, auto-increment integer, sequence, or UUID?"*

Thinking out loud: 1. *"Integer IDs are compact, sortable, easy to read in logs. UUIDs are 128 bits and unsortable, but can be generated client-side."* 2. *"With auto-increment, you can't know the ID until insert. UUIDs can be assigned before persistence — useful for client-generated entities and idempotency."*

The defensible answer: - *Sequence or auto-increment (integer/bigint) when IDs are mainly internal and only the DB creates them.* - *UUID (v7 if available) when IDs must be generated before insertion, must be unguessable, or must be unique across multiple DBs and merge-friendly.* - *Never expose auto-increment IDs in URLs to external users — enumeration attacks.*

The tradeoff: - *UUID indexes are larger and less cache-friendly than integer indexes.* - *UUID v4 is random — kills clustered-index insert performance on SQL Server / MySQL.* *UUID v7 is time-ordered and friendlier.*

When the OTHER option wins: - *Distributed systems with multiple write masters → UUID is essentially required.* - *Performance-critical OLTP with millions of rows per table → integer keys win on index size.*

What NOT to say: - *"Always UUID for safety" — performance can suffer noticeably at scale.* - *"Integer IDs are insecure, always UUID" — solve security at the URL layer (slug, opaque token), not the PK layer.*

Common follow-up: *"What's the difference between UUID v4 and v7?"*

Scenario 44 · Liquibase vs Flyway for DB migrations

Asked at: Walmart · **Cred:** · **Difficulty:** Medium · **Topic:** DB tooling

The interviewer's prompt: *"For database schema migrations, Liquibase or Flyway?"*

Thinking out loud: 1. *"Flyway is simpler — SQL files numbered `V1__create_users.sql` , `V2__add_email_index.sql` . Easy mental model."*
2. *"Liquibase supports YAML/XML/JSON change sets, rollback definitions, and conditional changes. Heavier but more flexible."*

The defensible answer: - *Flyway for most cases — SQL-first, easy to read, what-you-see-is-what-runs.* - *Liquibase when you need cross-DB-vendor abstractions, rollback definitions, or precondition-based migrations.*

The tradeoff: - *Flyway's rollback support is paid-tier — open-source version only does forward migrations.* - *Liquibase change set syntax is an extra layer between you and SQL.*

When the OTHER option wins: - *Multi-vendor DB support critical → Liquibase's abstractions help.* - *Pure single-vendor with simple needs → Flyway is faster to onboard.*

What NOT to say: - "Run schema changes manually" — recipe for production drift and undocumented schemas. - "ORM auto-DDL is fine" — Hibernate's `hbm2ddl.auto=update` is a footgun in production.

Common follow-up: "How do you handle a failed migration in production?"

Scenario 45 · Spring's @Scheduled vs Quartz for cron jobs

Asked at: TCS Digital · Walmart · **Difficulty:** Medium · **Topic:** Scheduling

The interviewer's prompt: "You need a cron job every 5 minutes. `@Scheduled` or Quartz?"

Thinking out loud: 1. "@Scheduled is dead simple — one annotation, runs in-process." 2. "With multiple instances, @Scheduled fires on EVERY instance — duplicates the work."

The defensible answer: - Single-instance app: @Scheduled, no need for more. - Multi-instance: Quartz with JDBC store (cluster-aware) or use ShedLock to dedupe @Scheduled across instances. - For heavy batch jobs, Spring Batch is purpose-built and integrates with the scheduler.

The tradeoff: - Quartz adds DB tables, configuration, and operational surface. - ShedLock adds a dependency and one shared table for coordination.

When the OTHER option wins: - Five-minute health ping job that's idempotent and doesn't matter if it duplicates → @Scheduled is fine. - Cross-job dependencies, retries with backoff → Quartz/Spring Batch is built for it.

What NOT to say: - "Just use @Scheduled, it always works" — fails silently in multi-instance deployments by running everything multiple times. - "Always use Quartz for safety" — overkill for trivial jobs.

Common follow-up: "How does ShedLock prevent duplicate @Scheduled executions across instances?"

Scenario 46 · Logging: SLF4J + Logback vs Log4j2 vs JUL

Asked at: Most shops · **Difficulty:** Easy-Medium · **Topic:** Logging

The interviewer's prompt: "For a new Spring Boot service, Logback or Log4j2?"

Thinking out loud: 1. "Logback is Spring Boot's default — works out of the box." 2. "Log4j2 has async appenders (LMAX Disruptor) that win on extreme throughput. Logback can do async too but the architecture is different."

The defensible answer: - Default Logback for most services. Stable, well-supported, Boot integration is seamless. - Log4j2 when async logging throughput matters (millions of log lines per minute) and the team can manage the migration.

The tradeoff: - Switching Boot's default requires excluding `spring-boot-starter-logging` — extra config friction. - Log4j had the high-profile Log4Shell vulnerability — historical baggage even though Log4j2 is well-patched now.

When the OTHER option wins: - Existing Log4j2 in the org's standard stack → consistency wins. - High-volume server with strict logging SLAs → Log4j2's async appender shines.

What NOT to say: - "Always use Log4j2, it's faster" — for normal apps, no measurable difference. - "Don't bother with SLF4J, use the framework directly" — SLF4J abstraction lets you swap engines; it's free flexibility.

Common follow-up: "What's parameterized logging and why does it matter for performance?"

Scenario 47 · Spring Boot `application.yml` vs externalized config server

Asked at: Walmart · **Cred:** · **Difficulty:** Medium · **Topic:** Config

The interviewer's prompt: "For a 20-service microservices system, embed config in each service's application.yml or use Spring Cloud Config Server?"

Thinking out loud: 1. "Embedded config means each service ships with its config — clean for single-service teams, but operationally you can't change config

without a redeploy." 2. "Config server centralizes — change once, services refresh. Operational power, central drift risk."

The defensible answer: - Embedded config + environment variable overrides for most cases. Simpler, no shared single point of failure. - Config server when ops teams need to change shared config across services without redeploying everything. - Kubernetes-native: ConfigMap + Secret often makes Spring Cloud Config Server redundant.

The tradeoff: - Config server is yet another service to deploy, secure, and monitor. - Embedded config means changes require redeploy — slower operational response.

When the OTHER option wins: - Ten teams, hundreds of services with shared config → config server. - One small service → embedded is fine.

What NOT to say: - "Always centralize for consistency" — adds operational risk that you may not need. - "Embed everything, simpler" — slow at scale; ops teams hate it.

Common follow-up: "How does Spring Cloud Bus + @RefreshScope refresh config without restart?"

Scenario 48 · Synchronous REST vs gRPC for inter-service calls

Asked at: Cred · Razorpay · Atlassian · Difficulty: Hard · Topic: Inter-service

The interviewer's prompt: "Two of your services need to call each other. REST/JSON or gRPC?"

Thinking out loud: 1. "REST/JSON is universal — anything can call anything, debugging with curl is trivial." 2. "gRPC is faster (Protobuf binary, HTTP/2 multiplexing), strongly typed, supports streaming. But it requires shared .proto files and code generation."

The defensible answer: - gRPC for high-throughput, latency-sensitive internal traffic where both sides are under your control. - REST/JSON for public APIs,

polyglot integrations where toolchain matters, and low-traffic services where simplicity wins.

The tradeoff: - gRPC has steeper operational tooling needs (envoy/protocol buffers, debugging without text payloads). - REST/JSON is verbose on the wire and slower to parse.

When the OTHER option wins: - External public API → REST always. - Real-time bidirectional streaming → gRPC streaming is purpose-built.

What NOT to say: - "gRPC is faster, always use it" — toolchain and operability matter for many teams. - "REST is universal, never use gRPC" — leaves performance on the table for internal traffic.

Common follow-up: "How does gRPC handle deadlines and cancellation across services?"

Scenario 49 · Mockito strict vs lenient stubbing

Asked at: Walmart · Cred · Difficulty: Medium · Topic: Testing

The interviewer's prompt: "In a unit test, strict stubbing (default in Mockito 3+) or lenient?"

Thinking out loud: 1. "Strict stubbing fails the test if a stub is set up but never called — catches dead stubs." 2. "Lenient is forgiving — useful when shared test setup creates more stubs than each test needs."

The defensible answer: - Strict by default — catches stale stubs and accidental over-mocking. Test maintenance is easier when stubs match what the code actually calls. - Lenient on individual stubs when the shared `@BeforeEach` setup is intentionally broader than what each test exercises.

The tradeoff: - Strict mode means small refactors can fail tests — slightly higher test maintenance. - Lenient hides accidental over-stubbing — bugs slip in.

When the OTHER option wins: - Heavy shared test fixture used by 20 tests → some lenient stubs are fine. - Brand-new fast-moving codebase → lenient temporarily, tighten as code stabilizes.

What NOT to say: - "Always lenient, easier" — defeats the value of strict stubbing.
- "Always strict, no exceptions" — pragmatism matters; shared fixtures are real.

Common follow-up: "What's the difference between Mockito's `verify(mock).method()` and an unverified stub?"

Scenario 50 · Integration test with `@SpringBootTest` vs slice tests

Asked at: Walmart · Atlassian · **Difficulty:** Medium · **Topic:** Spring testing

The interviewer's prompt: "For testing a controller, full `@SpringBootTest` or `@WebMvcTest` slice?"

Thinking out loud: 1. "`@SpringBootTest` boots the entire context — slow but realistic." 2. "`@WebMvcTest` loads only the web layer (controllers, advice, converters) — fast but doesn't exercise persistence or downstream beans."

The defensible answer: - `@WebMvcTest` for controller logic tests — mock the service layer, exercise validation and serialization quickly. - `@SpringBootTest` for end-to-end scenarios where multiple layers must cooperate. - `@DataJpaTest` for repository slices, `@JsonTest` for Jackson serialization tests.

The tradeoff: - Slice tests run in seconds; `@SpringBootTest` can take 30+ seconds per class. - Slice tests can miss configuration issues that only surface in the full context.

When the OTHER option wins: - Complex scenarios crossing controller → service → DB → external → only full `@SpringBootTest` gives confidence. - Pure logic tests with no Spring involvement → don't use Spring testing at all, plain JUnit.

What NOT to say: - "Always `@SpringBootTest` for safety" — test suite runtime explodes, CI becomes painful. - "Never `@SpringBootTest`, always slices" — end-to-end tests catch real config issues.

Common follow-up: "What's the difference between `@MockBean` and `@Mock` in Spring tests?"

Scenario 51 · Bean scope: singleton vs prototype vs request

Asked at: TCS Digital · Walmart · **Difficulty:** Medium · **Topic:** Spring scope

The interviewer's prompt: "For a bean that holds user-specific request state, singleton, prototype, or request scope?"

Thinking out loud: 1. "Singleton is the default — one instance per Spring context. Storing per-user state in a singleton is a classic thread-safety bug." 2. "Prototype creates a new instance each time it's injected — but injected once into a singleton means only ONE prototype gets created." 3. "Request scope creates one per HTTP request — but requires a web-aware context and proxy machinery."

The defensible answer: - Stateless services: singleton (default). Encapsulate per-request data as method parameters. - Per-request state: request scope (`@Scope(value = "request", proxyMode = ScopedProxyMode.TARGET_CLASS)`) with proxy mode set — needed when injected into a singleton. - Per-call state in prototype only when the caller actively asks for a fresh instance each time.

The tradeoff: - Request scope requires proxy machinery — small overhead, but works seamlessly. - Storing state in singletons is a common bug; the symptom is data bleeding across users.

When the OTHER option wins: - Stateful object that must be constructed fresh per call (rare) → prototype with explicit `ObjectProvider`. - Web context with per-request state needs → request scope is purpose-built.

What NOT to say: - "Just use singleton everywhere, it's faster" — corrupts data across users when state is held. - "Prototype solves all state issues" — common misconception; prototype injected into singleton creates ONE prototype, not many.

Common follow-up: "What's `ScopedProxyMode` and why is it required for request scope?"

Scenario 52 · @PreAuthorize vs filter-based authorization

Asked at: Walmart · Cred · Difficulty: Medium · Topic: Security

The interviewer's prompt: "To restrict endpoints to admin users, use `@PreAuthorize("hasRole('ADMIN')")` on the method or a security filter?"

Thinking out loud: 1. "Filters run before method dispatch — fast rejection, no business logic invoked." 2. "@PreAuthorize is method-level — close to the business logic, easier to express fine-grained rules involving method arguments."

The defensible answer: - URL-pattern coarse-grained rules: Spring Security HTTP filter chain (`http.authorizeHttpRequests`). Catches everything early. - Domain-aware fine-grained rules involving method args: `@PreAuthorize` with SpEL. - Both together is fine — defense in depth.

The tradeoff: - `@PreAuthorize` uses AOP proxies — same self-invocation gotcha as `@Transactional`. - Filter rules can drift from controller code as URLs evolve.

When the OTHER option wins: - Static role check on a URL pattern → filter is cheaper and clearer. - "User can only edit their own posts" — needs `@PreAuthorize("#post.owner == authentication.name")`.

What NOT to say: - "Just put `@Secured` on every method" — easy to forget; relying only on this is brittle. - "Filter is enough" — fine-grained authorization needs method-level checks.

Common follow-up: "What's the difference between `@PreAuthorize`, `@PostAuthorize`, and `@Secured`?"

Scenario 53 · JPA `save` vs `merge` vs `persist`

Asked at: TCS Digital · Walmart · Difficulty: Medium-Hard · Topic: JPA

The interviewer's prompt: "Use `repository.save(entity)`, `entityManager.persist(entity)`, or `entityManager.merge(entity)`?"

Thinking out loud: 1. "Spring Data's `save` is a convenience — internally it calls `persist` for new entities and `merge` for detached ones." 2. "`persist` makes a transient instance managed. `merge` copies a detached instance's state into a NEW managed instance and returns it."

The defensible answer: - Spring Data JPA: `save(entity)` works for both cases — convenience win. - Direct EntityManager: `persist` for new entities, `merge` only when you have detached entities being reattached. - Be aware: `merge` returns a NEW managed instance — your original detached reference stays detached.

The tradeoff: - `save` is convenient but hides a quirk: for entities with assigned IDs (UUID), Spring may treat them as detached and call `merge`, costing an extra SELECT. - `merge` re-runs a SELECT to find the existing row — silent extra DB hit.

When the OTHER option wins: - Assigned-ID entities where you know they're new → explicit `persist` avoids the merge SELECT. - Detached entity reattachment from a long conversation → `merge` is the right tool.

What NOT to say: - "save is always the same as persist" — wrong; behavior differs for detached entities and assigned IDs. - "Use merge for everything" — silent extra SELECTs add up.

Common follow-up: "Why does Spring Data's save() do an extra SELECT for entities with assigned IDs?"

Scenario 54 · Dockerizing JVM with `-Xmx` vs container-aware defaults

Asked at: Walmart · **Cred:** · **Difficulty:** Hard · **Topic:** JVM / containers

The interviewer's prompt: "Running JVM in Docker on Kubernetes. Set `-Xmx` explicitly or rely on container-aware JVM defaults?"

Thinking out loud: 1. "Modern JVMs (Java 10+) are container-aware — they read cgroup limits and size the heap to a percentage of container memory." 2. "But the default `MaxRAMPercentage` is 25% — often way too low for typical Spring services."

The defensible answer: - Java 11+: tune `MaxRAMPercentage` (e.g. 75) instead of hardcoding `-Xmx`. Then your config follows the container size automatically. - For predictable behavior, set `-Xmx` and `-Xms` to the same value — predictable heap, no resizing pauses.

The tradeoff: - Percentage-based config means changing container memory changes JVM heap — usually desirable, sometimes surprising. - Hardcoded `-Xmx` ignores container changes — drift between Pod spec and JVM behavior.

When the OTHER option wins: - Strict regulatory / performance environment needing exact memory budget → explicit `-Xmx`. - Many varied-size deployments (dev/staging/prod) → percentage scales cleanly.

What NOT to say: - "JVM doesn't know about containers" — outdated; Java 10+ is container-aware. - "Just don't set anything, defaults are fine" — 25% default leaves a lot of RAM unused.

Common follow-up: "What's the OOMKilled signal in Kubernetes and how do you tell if JVM hit its heap limit vs container limit?"

Scenario 55 · `CompletableFuture` vs `StructuredTaskScope` on Java 21

Asked at: Atlassian · Cred · Difficulty: Hard · Topic: Concurrency Java 21

The interviewer's prompt: "Fan-out three downstream calls and aggregate. `CompletableFuture.allOf` or Java 21 `StructuredTaskScope`?"

Thinking out loud: 1. "`CompletableFuture`'s `allOf` is the established pattern — but error handling is awkward, cancellation propagation is manual." 2.

"`StructuredTaskScope` (preview / incubator) treats the fan-out as a structured scope — if one task fails, siblings are cancelled automatically."

The defensible answer: - Java 21+ with virtual threads:

`StructuredTaskScope.ShutdownOnFailure`. Clearer semantics for cancellation and error propagation, code reads like a try-with-resources. - Pre-21 or stable production: `CompletableFuture.allOf` — battle-tested, well-understood.

The tradeoff: - `StructuredTaskScope` is still in preview/incubator on some Java versions — API changes possible. - `CompletableFuture` is verbose for complex flows — `thenCompose`, `thenCombine`, `exceptionally` chain hard to read.

When the OTHER option wins: - Need to express partial-success patterns where one failure doesn't cancel others → `CompletableFuture` with `allOf` and

individual exception handling. - Java 21 GA stable production with structured concurrency available → `StructuredTaskScope`.

What NOT to say: - "`CompletableFuture` is dead, use `StructuredTaskScope`" — it's still preview in many JDK versions; jump carefully. - "`StructuredTaskScope` is the same as `allOf`" — different semantics around cancellation and lifecycle.

Common follow-up: "What's structured concurrency and why does it matter?"

Scenario 56 · Open Session in View: keep it or kill it?

Asked at: TCS Digital · Walmart · Difficulty: Hard · Topic: JPA

The interviewer's prompt: "Spring Boot enables Open Session in View (OSIV) by default. Keep it on or turn it off?"

Thinking out loud: 1. "OSIV keeps the Hibernate session open during view rendering. Lets templates traverse lazy associations without `LazyInitializationException`." 2. "But it also opens DB connections for the entire request — connection pool exhaustion under load, hidden N+1 patterns in templates."

The defensible answer: - Turn it OFF (`spring.jpa.open-in-view=false`) for production microservices. Forces explicit fetch plans in the service layer. - Acceptable to keep ON for small server-rendered apps where the convenience outweighs the operational risk.

The tradeoff: - Turning off breaks code that lazily traverses associations in controllers or templates — needs DTO mapping or fetch joins. - Keeping it on hides performance problems and holds connections longer than necessary.

When the OTHER option wins: - Small server-rendered Thymeleaf app, single instance, low load → OSIV is fine. - High-load REST API → turn it off; explicit fetch plans.

What NOT to say: - "OSIV is fine, it's the default" — Spring Boot's default isn't always production-appropriate. - "Just turn it off, no exceptions" — small apps benefit from the convenience.

Common follow-up: "What is LazyInitializationException and what causes it?"

Scenario 57 · Connection pool sizing: small vs large

Asked at: Razorpay · Cred · Difficulty: Hard · Topic: Performance

The interviewer's prompt: "HikariCP pool — set max size to 100 or 20?"

Thinking out loud: 1. "More connections doesn't always mean faster — at some point you saturate the DB and connections queue at the DB level instead of the pool level." 2. "HikariCP's own docs cite a formula and recommend around 10-20 for many workloads. The famous PostgreSQL paper says small pools win."

The defensible answer: - Start at `cores * 2 + effective_spindle_count` (roughly 10-20 for a normal DB). - Measure. If wait queue grows, check whether the DB is saturated first — adding connections won't help.

The tradeoff: - Too small: request threads wait for connections, throughput caps low. - Too large: DB context switching and lock contention dominate; throughput actually drops past a certain point.

When the OTHER option wins: - Many short-lived connections (e.g. analytical queries) → larger pool can help. - Long-running transactions that hold connections → can't add your way out; rewrite the transactions.

What NOT to say: - "Bigger pool is always faster" — counter to measured reality; HikariCP docs warn against this. - "Just guess based on traffic" — measure with real load.

Common follow-up: "What's a connection leak and how would you detect it?"

Scenario 58 · @Transactional on service vs repository

Asked at: TCS Digital · Walmart · Difficulty: Medium · Topic: Spring transactions

The interviewer's prompt: "Put `@Transactional` on the service layer or the repository?"

Thinking out loud: 1. "Transactions encapsulate business operations — and business operations live in the service layer, not the repository." 2. "A repository method doing one **CRUD** action benefits from one short transaction; a service orchestrating multiple repo calls benefits from one longer transaction."

The defensible answer: - Service layer — that's where the unit-of-work boundary belongs. The service knows when work is complete. - Default `@Transactional` is read-write; mark read-only methods explicitly with `@Transactional(readOnly = true)` for the optimizer hint.

The tradeoff: - Service-level transactions mean longer-running transactions if the service calls many repo methods — connection held longer. - Repository-level granularity means each call commits independently — atomicity is lost across multi-step operations.

When the OTHER option wins: - A simple **CRUD** repository called directly (no service layer) → repository-level `@Transactional` is fine. - Read-only query service where every call is independent → can skip the wrapping transaction entirely.

What NOT to say: - "Put `@Transactional` on everything for safety" — extends transaction duration unnecessarily; can drain connection pool. - "Never put `@Transactional` on repositories" — sometimes that's where it belongs (no service layer above).

Common follow-up: "What's the difference between `@Transactional(readOnly = true)` and not using `@Transactional` at all for reads?"

Scenario 59 · Spring Data JPA derived queries vs `@Query`

Asked at: Walmart · TCS Digital · **Difficulty:** Easy-Medium · **Topic:** Spring Data

The interviewer's prompt: "For `findByEmailAndStatus`, use the derived query name or write `@Query(\"...\")`?"

Thinking out loud: 1. "Derived queries (`findByXAndY`) read naturally for simple lookups — Spring generates the JPQL." 2. "Method names get unreadable when conditions exceed 3 fields —

`findByFirstNameAndLastNameAndAgeGreaterThanAndStatusOrderBy...` "

The defensible answer: - Derived queries for 1-3 field lookups — clear, no JPQL boilerplate. - `@Query` for anything complex, especially with joins, projections, or aggregations. - Native SQL (`@Query(nativeQuery = true)`) for vendor-specific features (window functions, CTEs).

The tradeoff: - Derived query names break on entity rename refactors silently — runtime error, not compile-time. - `@Query` JPQL is a string — also not type-safe; jOOQ or Criteria are alternatives.

When the OTHER option wins: - Simple lookups → derived is shorter. - Complex queries → `@Query` is the only sane choice.

What NOT to say: - "Always use derived for everything" — fails on complex queries. - "Always use `@Query` for control" — overkill for trivial finders.

Common follow-up: "What's a projection interface in Spring Data and when is it useful?"

Scenario 60 · Idempotency keys vs database unique constraints

Asked at: Razorpay · Cred · PhonePe · **Difficulty:** Hard · **Topic:** Distributed systems

The interviewer's prompt: "To prevent duplicate payment submissions, use idempotency keys or rely on a unique constraint?"

Thinking out loud: 1. "Unique constraint catches duplicates after they hit the DB — works, but the caller doesn't know if the dupe was caught or there was another error." 2. "Idempotency key in a separate table lets the second call return the original result, not just reject the duplicate."

The defensible answer: - Idempotency key table: client sends a UUID, server stores (`key → result`) . Second call returns the original result. Standard pattern in payments. - Unique constraint as a backstop on the business key (e.g.

`(merchant_id, external_order_id)`). - Use both — the unique constraint catches what the idempotency table misses.

The tradeoff: - Idempotency table needs cleanup — keys eventually expire. - Unique constraints throw `DataIntegrityViolationException` — must be caught and translated.

When the OTHER option wins: - Tight DB-coupled flow where the constraint is the only writer → unique constraint may suffice. - Cross-region distributed write → idempotency keys are essentially required.

What NOT to say: - "Just use a unique constraint" — second caller gets an error, not the original result. - "Idempotency keys are overkill" — standard pattern in modern payments and APIs.

Common follow-up: "How long should an idempotency key be valid for?"

Scenario 61 · Synchronous Kafka producer vs async with callback

Asked at: Razorpay · Cred · Difficulty: Medium-Hard · Topic: Kafka

The interviewer's prompt: "Kafka producer — call `.get()` on the future (sync) or pass a callback (async)?"

Thinking out loud: 1. "Sync send waits for broker ack — slow, but you know it landed." 2. "Async send is fire-and-batch with a callback — much higher throughput but failures need explicit handling."

The defensible answer: - Async with callback for throughput — Kafka producer is designed around batching. - Use `acks=all` plus `enable.idempotence=true` to get reliability without sacrificing throughput. - Sync is fine for low-volume critical messages where the producer must verify delivery before continuing.

The tradeoff: - Async means messages may be in-flight at JVM crash — `linger.ms` and buffer behavior matter. - Sync drops throughput dramatically — typically 10-100x slower.

When the OTHER option wins: - One-off audit events at low volume → sync is fine and the simplicity wins. - Outbox pattern guarantees delivery via DB → can use either; outbox handles reliability.

What NOT to say: - "Always sync for reliability" — kills throughput; idempotence + acks gives reliable async. - "Async with no callback is fine" — silent failures, lost messages.

Common follow-up: "What does `enable.idempotence=true` actually guarantee?"

Scenario 62 · Domain events vs CDC for downstream syncs

Asked at: Walmart · Razorpay · **Difficulty:** Hard · **Topic:** Event-driven

The interviewer's prompt: "You need to notify downstream services when an Order changes. Application-level domain events or CDC (change data capture)?"

Thinking out loud: 1. "Domain events are explicit — the service publishes 'OrderShipped' with semantic meaning." 2. "CDC tails the DB write-ahead log — catches every row change but downstream gets schema-shaped events, not business-shaped ones."

The defensible answer: - Domain events when downstream cares about business meaning and the producing service controls the publish. - CDC (Debezium) when the producing service can't be modified, or when you need a strong guarantee that every DB change is captured. - Outbox pattern combines them: domain events written to an outbox table in the same DB transaction, CDC publishes them to Kafka.

The tradeoff: - Domain events couple the producer to publish logic — easy to miss a code path. - CDC exposes schema changes — downstream breaks when the table evolves.

When the OTHER option wins: - Legacy app you can't change → CDC is the only option. - New clean domain model → domain events are clearer.

What NOT to say: - "Always CDC, it's reliable" — exposes physical schema to consumers; coupling nightmare. - "Domain events catch everything" — only what you remember to publish.

Common follow-up: "What's the outbox pattern and how does it solve the dual-write problem?"

Scenario 63 · Spring Boot `@ConfigurationProperties` vs `@Value`

Asked at: Walmart · **Cred:** · **Difficulty:** Easy-Medium · **Topic:** Spring config

The interviewer's prompt: "For config values, `@ConfigurationProperties` class or `@Value("${...}...")` on fields?"

Thinking out loud: 1. "`@Value` injects one value per field — clean for one or two, ugly for ten." 2. "`@ConfigurationProperties` binds a whole namespace to a typed class — much better when there's structure."

The defensible answer: - `@ConfigurationProperties` with a record or POJO when you have a logical group of related config (e.g. `app.kafka.*`). - `@Value` for one-off scalars and SpEL expressions.

The tradeoff: - `@ConfigurationProperties` needs `@EnableConfigurationProperties` or `@ConfigurationPropertiesScan` to be active. - `@Value` scattered everywhere makes config drift hard to track.

When the OTHER option wins: - Single trivial value (`server.port`) → `@Value` is two lines, done. - Grouped, validated config → `@ConfigurationProperties` with `@Validated`.

What NOT to say: - "Always `@Value`, it's simpler" — at scale becomes a mess. - "Never `@Value`, always properties class" — overkill for one value.

Common follow-up: "How do you validate `@ConfigurationProperties` fields?"

Scenario 64 · Map.of vs new HashMap for an immutable lookup

Asked at: TCS Digital · Walmart · **Difficulty:** Easy · **Topic:** Java 9+ collections

The interviewer's prompt: "For a small constant lookup map, `Map.of("a", 1, "b", 2)` or `new HashMap<>()` with puts?"

Thinking out loud: 1. "`Map.of` returns a genuinely immutable map — modifications throw `UnsupportedOperationException`." 2. "A regular `HashMap` exposes mutation accidentally to anyone with a reference."

The defensible answer: - `Map.of(...)` for small constants — true immutability, compact syntax. - `Map.copyOf(existing)` to immutably snapshot an existing map. - `HashMap` when you need to build incrementally or mutate later.

The tradeoff: - `Map.of` accepts at most 10 entries; beyond that use `Map.ofEntries(Map.entry(k, v), ...)`. - `Map.of` doesn't allow null keys or values — throws `NPE`.

When the OTHER option wins: - Building up entries in a loop → `HashMap`, then maybe `Map.copyOf` at the end. - Need null values → `HashMap` is the only option.

What NOT to say: - "Use new `HashMap` for everything" — leaks mutation accidentally. - "`Map.of` is just syntactic sugar" — it's an actually-immutable type, not just a builder.

Common follow-up: "What's the difference between an unmodifiable view (`Collections.unmodifiableMap`) and a truly immutable map (`Map.copyOf`)?"

Scenario 65 · Scheduler thread pool vs ForkJoinPool

Asked at: Walmart · Atlassian · **Difficulty:** Hard · **Topic:** Concurrency

The interviewer's prompt: "For parallel processing of 1000 independent jobs, custom `ThreadPoolExecutor` or common `ForkJoinPool`?"

Thinking out loud: 1. "Common `ForkJoinPool` is shared by `parallelStream` and `CompletableFuture` default execution — overuse causes one heavy task to starve

everything else." 2. "Custom executor lets you size and tune for the specific workload — and isolate it from other parallel work."

The defensible answer: - Custom ThreadPoolExecutor (or ForkJoinPool) when the workload is meaningful — gives you naming, sizing, queue, and rejection control. - Common pool only for incidental parallelism in small utility code.

The tradeoff: - Custom executors need lifecycle management (shutdown, awaitTermination) — easy to leak threads. - Common pool is zero-config but shared globally.

When the OTHER option wins: - One-off `IntStream.range(...).parallel()` over a small array → common pool is fine. - Background workers that must not affect HTTP threads → custom pool.

What NOT to say: - "ForkJoinPool is always best for parallel" — fine for divide-and-conquer; less ideal for unrelated independent tasks. - "Just use `parallelStream`" — uses shared common pool; can starve other work.

Common follow-up: "Why is `parallelStream` considered risky in shared application code?"

Scenario 66 · Static factory methods vs constructors

Asked at: Most senior interviews · Difficulty: Medium · Topic: API design

The interviewer's prompt: "For creating instances of a domain class, public constructor or static factory method (`User.of(...)`)?"

Thinking out loud: 1. "Constructors have a fixed name — overloading by parameter list is the only way to vary behavior." 2. "Static factories can be named — `Money.fromCents(100)` vs `Money.fromDollars(1.0)` is much clearer than two two-arg constructors."

The defensible answer: - Static factory when the creation has meaning beyond 'new instance' — caching, polymorphism (return a subclass), descriptive naming. - Plain constructor when 'new X(a, b)' is unambiguous and there's no value-added behavior.

The tradeoff: - Static factories don't subclass via `new` — frameworks expecting public constructors may struggle. - Constructors are first-class in many APIs (`Class::getConstructor`).

When the OTHER option wins: - Simple immutable data → public constructor or record canonical constructor. - Multiple ways to construct that mean different things → static factories with descriptive names.

What NOT to say: - "Always use static factories, Effective Java says so" — Item 1, but not absolutely; constructors still have a place. - "Static factories hide intent" — usually the opposite; they make intent explicit.

Common follow-up: "What's the Builder pattern and when does it beat both?"

Scenario 67 · `BigDecimal` vs `double` for money

Asked at: Razorpay · PhonePe · Cred · Difficulty: Easy-Medium · Topic: Data types

The interviewer's prompt: "Storing a money amount. `double` , `BigDecimal` , or `long` cents?"

Thinking out loud: 1. "Floating-point can't represent 0.1 exactly. `0.1 + 0.2 != 0.3` — well-known. Disqualifying for money." 2. "`BigDecimal` is exact and supports configurable rounding modes." 3. "`Long` cents is exact and fast — but breaks when you need fractional cents (FX, micropricing)."

The defensible answer: - `BigDecimal` for any user-facing or accounting-grade money, with explicit `MathContext` and rounding rules. - `Long` cents (or smallest unit) for high-volume internal calculations where exactness AND speed both matter and fractions aren't needed. - Never `double` for money.

The tradeoff: - `BigDecimal` is slower than primitives — 10-100x for arithmetic. - `Long` cents loses generality the moment you need fractions or sub-cents.

When the OTHER option wins: - Performance-sensitive arithmetic with no fractions (e.g. cents in a high-frequency aggregation) → `long`. - Standard transactional business logic → `BigDecimal`.

What NOT to say: - "double is fine for small amounts" — rounding errors compound; auditors will find them. - "Always BigDecimal everywhere" — sometimes the performance cost is real and unnecessary.

Common follow-up: "What's the right way to compare two BigDecimal values for equality?"

Scenario 68 · Instant vs LocalDateTime vs ZonedDateTime

Asked at: Walmart · Atlassian · **Difficulty:** Medium · **Topic:** Date/time

The interviewer's prompt: "You need to store a 'created at' timestamp. `Instant`, `LocalDateTime`, or `ZonedDateTime`?"

Thinking out loud: 1. "Instant is a UTC point in time — no zone info — best for 'when did this happen'." 2. "LocalDateTime is a wall-clock time WITHOUT zone — ambiguous as a 'when did this happen' answer." 3. "ZonedDateTime is wall-clock + zone — useful for scheduling future events ('next Monday at 9am in Asia/Kolkata')." "

The defensible answer: - Instant for event timestamps — created_at, updated_at, log times. - ZonedDateTime for user-facing future scheduling ("9am next Tuesday in their timezone"). - LocalDateTime for things genuinely without zone (a recurring 'every day at 09:00' rule, applied per user).

The tradeoff: - Storing LocalDateTime as "when an event happened" loses zone info — ambiguous if your servers move or clocks shift. - ZonedDateTime in storage is bulky — usually you store Instant + a separate user-zone preference.

When the OTHER option wins: - Scheduling a recurring local-time job → LocalDateTime + zone applied at scheduling time. - Cross-time-zone calendar entries → ZonedDateTime preserves the original zone.

What NOT to say: - "Just use Date or Calendar, they're fine" — legacy, mutable, broken; java.time replaced them. - "Always store UTC strings" — possible, but the typed APIs do this for you.

Common follow-up: "What's the difference between `Instant.now()` and `System.currentTimeMillis()`?"

Scenario 69 · Spring Data Specification vs QueryDSL

Asked at: Walmart · Atlassian · Difficulty: Hard · Topic: Dynamic queries

The interviewer's prompt: "You need dynamic queries based on user-supplied filters. JPA Specification (Criteria API) or QueryDSL?"

Thinking out loud: 1. "Criteria API is built into JPA — no extra dependency, but verbose and string-typed for field names if you don't use the metamodel." 2. "QueryDSL generates a typed query model — `QUser.user.name.eq(...)` — refactor-safe and readable."

The defensible answer: - QueryDSL for any non-trivial dynamic query — type safety, readable predicates, refactor-safe field references. - JPA Specification when you want zero extra dependencies and the dynamic logic is modest.

The tradeoff: - QueryDSL adds an annotation processor and generated Q-classes — build complexity. - Specification with JPA metamodel is type-safe but extremely verbose.

When the OTHER option wins: - Small project, no annotation processor desired → Specifications. - Heavy dynamic-query system → QueryDSL.

What NOT to say: - "Just concatenate JPQL strings dynamically" — injection risk, unmaintainable. - "Criteria API is enough for everything" — verbose to the point of unmaintainable for complex queries.

Common follow-up: "What's the JPA metamodel and why does it matter for type safety?"

Scenario 70 · SQL window function vs Java post-processing

Asked at: Walmart · Atlassian · Difficulty: Hard · Topic: SQL vs code

The interviewer's prompt: "You need to compute a running total per customer. SQL window function (`SUM(...)` `OVER (PARTITION BY ...)`) or load rows and compute in Java?"

Thinking out loud: 1. "Window functions run in the DB — close to the data, no transfer of intermediate rows." 2. "Java post-processing means pulling all the rows and re-iterating — N rows over the wire, then memory in JVM."

The defensible answer: - SQL window function. Less data over the wire, DB's query planner is optimized for it. - Fall back to Java post-processing only when the logic involves business rules the DB can't easily express.

The tradeoff: - Window functions are vendor-specific in syntax details — portability suffers slightly. - Complex business logic in SQL is hard to test and version.

When the OTHER option wins: - The calculation needs external data not in the DB → Java with the joined info. - Heavily branching business rules → easier to write in Java with unit tests.

What NOT to say: - "Always pull and process in Java for testability" — wastes bandwidth and DB advantages. - "Always use SQL, DBs are fast" — misses cases where business logic doesn't fit SQL cleanly.

Common follow-up: "What's the difference between aggregation and window functions?"

Scenario 71 · Atomic file write vs database for small persistent state

Asked at: Cred · Atlassian · **Difficulty:** Medium · **Topic:** Persistence

The interviewer's prompt: "You need to persist a small piece of state (last-processed-offset, ~100 bytes). DB row or local atomic file write?"

Thinking out loud: 1. "DB is reliable, networked, but every read/write hops the network." 2. "Local file is in-process fast, but local-only and gets messy across restarts on a different node."

The defensible answer: - DB when state must outlive an instance — restarts, scaling, multiple instances. - Local atomic file (write-to-tmp, fsync, rename) when

state is per-instance and cheap to rebuild. - For Kubernetes-style ephemeral pods, default to DB or external KV — local files vanish with the pod.

The tradeoff: - DB adds latency and a runtime dependency. - Local files break on multi-instance and pod scheduling.

When the OTHER option wins: - Single-instance batch tool that runs on the same machine → file is simpler. - Anything that scales horizontally → DB or KV is required.

What NOT to say: - "Always file, it's faster" — fails on multi-instance. - "Always DB, even for tiny state" — sometimes the cost-benefit favors a file.

Common follow-up: "How do you do an atomic write on POSIX vs Windows?"

Scenario 72 · Spring Boot Actuator: enable everything or just what you need?

Asked at: Walmart · Cred · Difficulty: Easy-Medium · Topic: Operations

The interviewer's prompt: "In production, enable all Actuator endpoints or only the ones you use?"

Thinking out loud: 1. "Actuator endpoints can leak environment variables, config, heap dumps. Bad if exposed publicly." 2. "You only need a few in production (health, info, metrics, prometheus). Disable the rest."

The defensible answer: - Explicit allowlist

(`management.endpoints.web.exposure.include=health,info,prometheus`).

Default-deny everything else. - Bind Actuator port to internal network only

(`management.server.port=8081`), not the same port as the public API.

The tradeoff: - Adding a new endpoint requires explicit allowlist update — slight friction. - Wildcard exposure is convenient in dev but risky in prod.

When the OTHER option wins: - Dev/staging where everything is private → enable wildcard for convenience. - Locked-down internal cluster with auth on Actuator → wildcard is acceptable.

What NOT to say: - "Just enable everything, simpler" — security risk. - "Disable all Actuator in prod" — loses monitoring; you need health and metrics at minimum.

Common follow-up: "How do you secure Actuator endpoints in production?"

Scenario 73 · Synchronous validation vs async with `@TransactionalEventListener`

Asked at: Walmart · Razorpay · **Difficulty:** Medium · **Topic:** Async patterns

The interviewer's prompt: "After saving an order, you need to validate it against a remote system. Sync inline or async with `@TransactionalEventListener`?"

Thinking out loud: 1. "Sync inline blocks the request — user waits for the validation call." 2. "Async after commit means the user gets a fast response, but validation could fail later — needs a retry / compensation path."

The defensible answer: - Sync when the validation result must be reflected in the user's response (must accept/reject before responding). - Async with `@TransactionalEventListener(AFTER_COMMIT)` when validation is a downstream concern — user sees success, async correction flow if it fails.

The tradeoff: - Sync ties user latency to a third party. - Async needs compensation / retry — extra moving parts.

When the OTHER option wins: - Saga-style flow where each step can fail → async with explicit compensation. - Critical inline check (KYC verification) → sync.

What NOT to say: - "Always async for performance" — sometimes the user needs the answer now. - "Always sync for simplicity" — couples user latency to slow downstreams.

Common follow-up: "What's the difference between `AFTER_COMMIT` and `AFTER_COMPLETION` phases?"

Scenario 74 · `Iterator.remove` vs `collection.removeIf`

Asked at: Walmart · TCS Digital · Difficulty: Easy-Medium · Topic: Collections

The interviewer's prompt: "You need to remove all elements matching a predicate from a list. Iterator with `remove()` or `list.removeIf(...)`?"

Thinking out loud: 1. "For-each with `iterator.remove()` works but is verbose." 2. "removelf is one line and the implementation is optimized for the collection (e.g. ArrayList does a single-pass compaction)."

The defensible answer: - `removeIf(predicate)` . Concise, optimized per collection type, no `ConcurrentModificationException` risk. - Iterator with `remove()` when the loop body needs more than a predicate (e.g. side effects depending on iteration state).

The tradeoff: - `removeIf` doesn't expose the index — if you need it, fall back to iterator/for-loop. - Iterator approach is more flexible but verbose.

When the OTHER option wins: - Need to track index or do something during removal → explicit iterator. - Predicate is enough → removelf.

What NOT to say: - "Use a for-each loop with `.remove()` on the list" — `ConcurrentModificationException` waiting to happen. - "Always removelf, no exceptions" — misses cases where you need the iteration context.

Common follow-up: "What causes `ConcurrentModificationException` and how does `iterator.remove()` avoid it?"

Scenario 75 · Spring `@Async` vs platform thread pool

Asked at: Walmart · Cred · Difficulty: Medium · Topic: Async

The interviewer's prompt: "For background work in a Spring app, `@Async` annotation or manually submit to a `ThreadPoolTaskExecutor`?"

Thinking out loud: 1. "`@Async` is declarative — slap an annotation on a method, returns `CompletableFuture` or void." 2. "Manual submit gives full control over which executor, naming, error handling."

The defensible answer: - `@Async` for normal background work — clean, integrates with Spring's lifecycle. - Always configure a custom executor (don't use the default `SimpleAsyncTaskExecutor` which creates a new thread per call). - Manual submit when you need fine control over the executor at the call site.

The tradeoff: - `@Async` uses AOP proxy — same self-invocation gotcha. - Default `SimpleAsyncTaskExecutor` is a footgun — creates unbounded threads.

When the OTHER option wins: - Quick background work → `@Async` with a configured executor. - Need to vary the executor per call → manual submit.

What NOT to say: - "`@Async` works out of the box" — yes, with a thread-per-call default that's dangerous. - "Always use manual submit for control" — overkill when `@Async` with a configured executor works.

Common follow-up: "How do you propagate `SecurityContext` or `MDC` through `@Async` calls?"

Scenario 76 · Database read replica vs caching for read-heavy load

Asked at: Razorpay · Cred · Flipkart · Difficulty: Hard · Topic: Scaling

The interviewer's prompt: "Your read-write ratio is 10:1. Add a read replica or a Caffeine cache?"

Thinking out loud: 1. "Read replica scales horizontally — every read hits a replica, not the primary." 2. "Cache is even faster (no network) but you have the invalidation problem."

The defensible answer: - Cache for the hot subset that's a Pareto distribution (10% of keys are 90% of reads). Caffeine in-process for low-latency reads. - Read replica for the long tail of reads and reports — analytical queries that would otherwise hammer the primary. - Both together work well: cache absorbs the hottest reads, replica absorbs the rest.

The tradeoff: - Cache invalidation is hard; replicas have replication lag (read-your-writes pitfalls). - Both add operational overhead.

When the OTHER option wins: - Strict read-after-write consistency → can't use cache or replica naively; route those reads to primary. - Wide-uniform read distribution (no hot keys) → cache doesn't help much; replica is better.

What NOT to say: - "Just add a cache, replicas are expensive" — depends on the workload distribution. - "Always read from primary for consistency" — doesn't scale.

Common follow-up: "What's replication lag and how does it affect read-after-write semantics?"

Scenario 77 · JSON web token vs server-side session

Asked at: Cred · Razorpay · Walmart · **Difficulty:** Medium-Hard · **Topic:** Auth

The interviewer's prompt: "For your API auth, JWT or server-side session?"

Thinking out loud: 1. "JWT is stateless — server doesn't need to look up the session. Scales horizontally without sticky sessions." 2. "But JWT can't be revoked cheaply — once issued, it's valid until expiry. Server-side session can be invalidated instantly."

The defensible answer: - JWT (with short expiry, e.g. 15 min) + refresh token (in DB, revokable) for typical web/mobile apps. - Pure server-side session for internal admin tools where revocation is critical and scale is small.

The tradeoff: - JWT contains its claims — if a user's role changes, the old JWT is stale until expiry. - Server-side session adds a lookup hop per request.

When the OTHER option wins: - Need instant logout / role-change reflection → server-side session. - Mobile / SPA with long-lived sessions → JWT + refresh token pattern.

What NOT to say: - "JWT is more modern, always use it" — pitfalls around revocation are real. - "Always sessions, JWT is overhyped" — JWT solves real horizontal-scale problems.

Common follow-up: "How do you handle JWT revocation when a user logs out?"

Scenario 78 · Spring's `@Validated` at class level vs `@Valid` on parameter**Asked at: Walmart · TCS Digital · Difficulty: Easy-Medium · Topic: Validation****The interviewer's prompt: " `@Validated` on the class or `@Valid` on the method parameter?"****Thinking out loud: 1. "`@Valid` is the standard JSR-380 annotation — works on controller method parameters by default." 2. "`@Validated` is Spring's extension — supports validation groups, can be applied at class level for service-method-parameter validation."****The defensible answer: - `@Valid` on controller method parameters — that's the standard pattern; Spring MVC supports it natively. - `@Validated` on a service class when you want method-parameter validation (Spring's `MethodValidationInterceptor` takes over). - `@Validated` on a controller when using validation groups (e.g. `CreateGroup` vs `UpdateGroup`).****The tradeoff: - `@Validated` on a class involves a different machinery (AOP-based) — same proxy gotchas. - `@Valid` alone doesn't support validation groups.****When the OTHER option wins: - Need groups → `@Validated`. - Just plain validation on a DTO → `@Valid`.****What NOT to say: - "They're the same" — different machinery, different capabilities. - "Always use `@Validated`" — adds proxy machinery you may not need.****Common follow-up: "What are validation groups and when would you use them?"****Scenario 79 · Hibernate `StatelessSession` vs `Session` for batch****Asked at: Walmart · Atlassian · Difficulty: Hard · Topic: JPA****The interviewer's prompt: "Inserting 100k rows via Hibernate. Regular `Session` or `StatelessSession`?"**

Thinking out loud: 1. "Regular Session accumulates entities in the persistence context — at 100k entries, memory blows up." **2.** "StatelessSession bypasses persistence context — no first-level cache, no dirty checking, just send INSERTs."

The defensible answer: - StatelessSession (or plain JDBC batch) for bulk inserts. Skip the entity lifecycle overhead entirely. - Regular Session with `flush()` + `clear()` every 50 entities if you must keep features.

The tradeoff: - StatelessSession loses dirty checking, cascade, lifecycle callbacks. - Regular Session needs careful flush/clear to avoid memory blow-up.

When the OTHER option wins: - Small batches (under 1000) with cascades → regular Session is fine. - Very large pure-insert batches → StatelessSession or JDBC.

What NOT to say: - "Use the regular Session, it handles batches" — out-of-memory at scale. - "Always JDBC for inserts" — sometimes Hibernate batch with proper flushing is fine.

Common follow-up: "How does Hibernate's batch_size property work and what does ordered_inserts do?"

Scenario 80 · Spring's `@Bean` vs `@Component` discovery

Asked at: Walmart · TCS Digital · **Difficulty:** Easy-Medium · **Topic:** Spring DI

The interviewer's prompt: " `@Component` on the class or `@Bean` method in a `@Configuration` ?"

Thinking out loud: 1. "@Component lets the class declare its own beanness — convenient for your own classes." **2.** "@Bean is needed for third-party classes you don't own (you can't add @Component to them)."

The defensible answer: - @Component for your own beans — concise, scanned automatically. - @Bean for third-party classes, conditional registration, or when wiring needs explicit logic.

The tradeoff: - **@Component** scattered across the codebase — discovery via scan can be slower (negligible in practice). - **@Bean** methods in big **@Configuration** classes can grow unwieldy.

When the OTHER option wins: - Third-party library class needing wiring → **@Bean**.
- Conditional bean creation based on environment → **@Bean** with **@Conditional**.

What NOT to say: - "Always **@Bean**, more explicit" — overkill for your own classes.
- "Always **@Component**, simpler" — fails for third-party classes.

Common follow-up: "What's the difference between **@Configuration** with **@Bean** methods being CGLIB-proxied vs not?"

Scenario 81 · **try-with-resources** vs explicit close in finally

Asked at: TCS · Infosys · **Difficulty:** Easy · **Topic:** Resource management

The interviewer's prompt: "For closing a **Connection/InputStream**, **try-with-resources** or **finally block**?"

Thinking out loud: 1. "try-with-resources auto-closes in reverse order of declaration and handles exception suppression correctly." 2. "Manual finally has historical bugs — exception from **close()** masking the original exception."

The defensible answer: - **try-with-resources** always. Handles suppressed exceptions correctly, less code, harder to forget. - For resources not implementing **AutoCloseable**, wrap in a helper or write the boilerplate carefully.

The tradeoff: - Only works on **AutoCloseable** — legacy classes need wrappers.
- The pattern is Java 7+ — works everywhere modern.

When the OTHER option wins: - Legacy class without **AutoCloseable** that can't be wrapped → manual finally with care. - Otherwise, nothing.

What NOT to say: - "finally is fine, I know what I'm doing" — exception suppression bugs are subtle and common. - "try-with-resources can't handle multiple resources" — it can; declare them in one try.

Common follow-up: "What's exception suppression and how does try-with-resources handle it?"

Scenario 82 · Spring `@ControllerAdvice` vs per-controller exception handler

Asked at: Walmart · TCS Digital · Difficulty: Easy-Medium · Topic: Error handling

The interviewer's prompt: "For mapping exceptions to HTTP responses, `@ControllerAdvice` (global) or `@ExceptionHandler` inside each controller?"

Thinking out loud: 1. "Global `@ControllerAdvice` keeps exception mapping in one place — consistent across controllers." 2. "Per-controller is more localized but tends to repeat boilerplate."

The defensible answer: - `@ControllerAdvice` for shared exception mappings — single source of truth. - Per-controller `@ExceptionHandler` when the exception handling is genuinely controller-specific.

The tradeoff: - One `@ControllerAdvice` can grow into a god class — split by domain if needed. - Per-controller scatter makes consistency hard.

When the OTHER option wins: - A controller has truly unique error response shape → local handler. - Default case → global advice.

What NOT to say: - "Just catch everywhere" — anti-pattern; exceptions should bubble to a uniform handler. - "Always per-controller for isolation" — leads to inconsistent error shapes across the API.

Common follow-up: "What's the order of resolution between `@ExceptionHandler` in a controller and one in `@ControllerAdvice`?"

Scenario 83 · `String.intern` vs String pool growth

Asked at: Atlassian · Walmart · Difficulty: Hard · Topic: JVM internals

The interviewer's prompt: "You're processing millions of repeated strings (e.g. parsing log fields). Should you call `intern()` on them?"

Thinking out loud: 1. "intern() puts the string in the pool — duplicates collapse to one reference, saving memory." 2. "But the pool itself has bookkeeping cost — and unique strings still in the pool waste pool space."

The defensible answer: - Only intern strings with high duplication (e.g. enum-like field values, repeated keys). - For arbitrary unique strings, intern hurts more than helps. - Modern alternative: use a `ConcurrentHashMap` as a manual intern table with bounded size — more control than the global pool.

The tradeoff: - The string pool is global JVM state — fragmentation can affect class metadata behavior. - intern() lookups have overhead — not free.

When the OTHER option wins: - Bounded set of repeated values (e.g. log levels, country codes) → intern is fine. - Truly unique strings → don't intern.

What NOT to say: - "intern everything to save memory" — pool grows, GC suffers. - "Never intern, it's deprecated" — not deprecated, just often misused.

Common follow-up: "Where does the String pool live in modern JVMs (PermGen vs Metaspace vs heap)?"

Scenario 84 · Comparing `==` vs `equals` on Integer cache range

Asked at: TCS · Infosys · **Difficulty:** Medium · **Topic:** Autoboxing

The interviewer's prompt: " `Integer a = 127; Integer b = 127; a == b` returns true. With 200, returns false. Why?"

Thinking out loud: 1. "Integer caches boxed instances for -128 to 127. Inside that range, autoboxing returns the cached instance — references are equal." 2. "Outside the range, autoboxing creates new Integer instances — different objects, `==` is false."

The defensible answer: - Always use `.equals()` (or `Integer.compare(a, b) == 0`) for comparing boxed numerics. Never `==`. - Use primitive `int` whenever boxing isn't required.

The tradeoff: - `equals()` is one extra method call — irrelevant for correctness, negligible for performance.

When the OTHER option wins: - Comparing references intentionally (rare; mostly tests of identity, e.g. interning) → `==` . - Otherwise, never.

What NOT to say: - `"==" works for small integers` — true but a footgun; never rely on it. - `"Autoboxing is cheap"` — it is and it isn't; the comparison trap is the bigger issue.

Common follow-up: `"Can you change the Integer cache range?"`

Scenario 85 · Local memoization vs shared cache for expensive lookups

Asked at: Cred · Razorpay · **Difficulty:** Medium · **Topic:** Caching

The interviewer's prompt: `"An expensive function is called repeatedly with the same args. Memoize per-instance or use a shared cache?"`

Thinking out loud: 1. `"Per-instance memoization is fast and simple but doesn't share across instances or threads."` 2. `"Shared cache (Caffeine bean) is reusable but adds wiring."`

The defensible answer: - Per-method, low-stakes memoization:

`ConcurrentHashMap.computeIfAbsent` inside the class. - Shared, application-wide caching: Caffeine bean injected wherever needed.

The tradeoff: - Per-instance memo grows unbounded unless you add eviction. - Shared cache adds a dependency layer.

When the OTHER option wins: - One-off optimization in a single class → `ConcurrentHashMap` is enough. - Cross-cutting cache → shared bean.

What NOT to say: - `"Just use HashMap inside the method"` — race conditions across threads. - `"Always use Caffeine for memoization"` — overkill for trivial cases.

Common follow-up: `"What does computeIfAbsent guarantee about concurrent calls with the same key?"`

Scenario 86 · Spring `@RequestMapping` vs HTTP-method-specific shortcuts

Asked at: TCS Digital · Walmart · **Difficulty:** Easy · **Topic:** Spring MVC

The interviewer's prompt: "`@RequestMapping(method = POST)` or `@PostMapping` ?"

Thinking out loud: 1. "`@PostMapping` is a meta-annotation over `@RequestMapping(method=POST)` — equivalent but shorter." 2. "`@RequestMapping` is more flexible — supports multiple methods on one annotation, useful for an endpoint that handles GET and POST."

The defensible answer: - `@GetMapping` / `@PostMapping` / etc. for single-method endpoints — cleaner, intent obvious. - `@RequestMapping` for multi-method endpoints or class-level path mappings.

The tradeoff: - Mixing both styles across a codebase is inconsistent. - Method-specific shortcuts can't express the multi-method case.

When the OTHER option wins: - One endpoint truly handles both GET and POST → `@RequestMapping` with array. - Standard single-method endpoint → shortcut.

What NOT to say: - "Always `@RequestMapping`, more flexible" — verbose for the common case. - "Shortcuts are syntactic sugar, don't use them" — they're the recommended style.

Common follow-up: "What's the difference between `@PathVariable` and `@RequestParam`?"

Scenario 87 · Spring `@Transactional(readOnly = true)` — actually useful?

Asked at: Walmart · Cred · **Difficulty:** Hard · **Topic:** Transactions

The interviewer's prompt: "On a read-only service method, is `@Transactional(readOnly = true)` actually doing anything useful?"

Thinking out loud: 1. "With Hibernate, `readOnly = true` skips dirty-check at flush — measurable savings on large result sets." **2.** "With pure JDBC, `readOnly` is mostly a hint to the JDBC driver — minor."

The defensible answer: - With Hibernate: yes, `@Transactional(readOnly = true)` skips dirty checking and helps performance — also signals intent. - Even with pure JDBC: still useful as a hint to some drivers and routing logic (some setups route reads to replicas).

The tradeoff: - Slight ceremony — you must add it explicitly to read methods. - Tools like Spring's `LazyConnectionDataSourceProxy` can use `readOnly` hints to delay or route the connection.

When the OTHER option wins: - Service method doing zero JPA/DB work → don't put `@Transactional` at all. - Mixed read-then-write methods → don't lie with `readOnly`; the write will fail silently.

What NOT to say: - "It's just a hint, doesn't matter" — measurable Hibernate savings; routing logic can use it. - "Use it on every method" — only on methods that actually do reads.

Common follow-up: "How does Hibernate flush behavior change under `readOnly = true`?"

Scenario 88 · Spring `@Order` vs configuration class ordering

Asked at: Walmart · Atlassian · **Difficulty:** Medium · **Topic:** Spring config

The interviewer's prompt: "You need bean A initialized before bean B. `@Order`, `@DependsOn`, or constructor injection?"

Thinking out loud: 1. "Constructor injection naturally orders dependencies — Spring won't create B before A if B requires A." **2.** "`@DependsOn` forces an order without requiring injection — for side-effect ordering." **3.** "`@Order` affects list ordering when multiple beans of the same type are injected — different concern."

The defensible answer: - Constructor injection: the cleanest way to express 'B needs A' — Spring resolves the order automatically. - `@DependsOn` for side-effect

ordering (B must initialize after A even though it doesn't inject A). - @Order for ordering beans in an injected `List<MyInterface>` .

The tradeoff: - @DependsOn is brittle — string-based bean name reference, breaks on rename. - @Order on filters and aspects is a separate concern from bean creation order.

When the OTHER option wins: - Genuine side-effect ordering with no DI relationship → @DependsOn (rare). - Otherwise, redesign so DI expresses the dependency.

What NOT to say: - "Just use @DependsOn for everything" — string-based, brittle. - "Spring orders things magically" — explicit DI is how you tell it.

Common follow-up: "How does Spring detect and report a circular bean dependency?"

Scenario 89 · `@JsonInclude` vs filter for null handling

Asked at: Walmart · TCS Digital · Difficulty: Easy · Topic: Jackson

The interviewer's prompt: "Suppress null fields in JSON output.

`@JsonInclude(NON_NULL)` per class or global ObjectMapper config?"

Thinking out loud: 1. "Per-class annotation is local — easy to see the behavior for that DTO." 2. "Global setting affects every response — consistent, but surprising for new joiners reading one class in isolation."

The defensible answer: - Global config

(`ObjectMapper.setSerializationInclusion(NON_NULL)`) for app-wide consistency. Document the policy. - Per-class `@JsonInclude` when you genuinely need per-DTO behavior.

The tradeoff: - Global setting affects every response — including ones you may want nulls in for explicit-absent semantics. - Per-class annotation drifts when not enforced by code review.

When the OTHER option wins: - One DTO needs different behavior → per-class annotation. - Default uniform policy → global.

What NOT to say: - *"Always include nulls, more explicit" — bloats payloads.* - *"Never include nulls" — sometimes null vs absent is semantically meaningful.*

Common follow-up: *"What's the difference between NON_NULL, NON_EMPTY, and NON_ABSENT in @JsonInclude?"*

Scenario 90 · Spring's RestTemplate / WebClient timeout strategy

Asked at: *Razorpay · Cred · Difficulty: Medium-Hard · Topic: Resilience*

The interviewer's prompt: *"You're calling a downstream API. Default timeouts or set them explicitly?"*

Thinking out loud: 1. *"Default RestTemplate has NO timeout — calls can hang forever, exhausting your thread pool."* 2. *"Sensible timeouts (connect 3-5s, read 10-30s) prevent cascading failures."*

The defensible answer: - *Always set explicit connect and read timeouts. Never rely on defaults.* - *Layer with a circuit breaker (Resilience4j) for additional protection from sustained failures.* - *For downstream with known latency profile, tune timeouts to p99.5 + buffer.*

The tradeoff: - *Timeouts that are too aggressive cause spurious failures.* - *Timeouts that are too loose let slow downstreams take down your service.*

When the OTHER option wins: - *Long-running batch operation that genuinely takes minutes → longer timeouts (or use a different async pattern).* - *Critical low-latency call → tight timeout + retry.*

What NOT to say: - *"Default timeouts are fine" — RestTemplate's default of none is famously dangerous.* - *"Just retry forever" — amplifies upstream problems.*

Common follow-up: *"What's a circuit breaker and how does it relate to timeouts?"*

Scenario 91 · OneToMany owning side vs inverse

Asked at: TCS Digital · Walmart · Difficulty: Medium · Topic: JPA mapping

The interviewer's prompt: "For a OneToMany between Order and OrderItem, which side is the owning side?"

Thinking out loud: 1. "In JPA, the owning side is the one with the foreign key column — usually the Many side." 2. "Putting `mappedBy` on the One side means OrderItem owns the relationship via its `order_id` FK column."

The defensible answer: - Bidirectional: `@ManyToOne` on OrderItem (owning), `@OneToMany(mappedBy = "order")` on Order (inverse). - Unidirectional `@OneToMany` is possible but generates an extra join table by default — usually unwanted.

The tradeoff: - Bidirectional requires care to keep both sides in sync — convenience methods (`addItem`) that set both fields. - Unidirectional may force `JoinColumn` override to avoid the join table.

When the OTHER option wins: - Truly read-only relationship → unidirectional is fine and simpler. - Need cascade behavior controlled by the parent → bidirectional with proper cascade settings.

What NOT to say: - "OneToMany on the parent owns it" — wrong; the foreign key holder is the owner. - "Just use unidirectional, simpler" — extra join table is rarely what you want.

Common follow-up: "What's the `orphanRemoval` flag and how is it different from cascade `DELETE`?"

Scenario 92 · `CompletableFuture` exception handling: `handle` vs `exceptionally`

Asked at: Walmart · Cred · Difficulty: Medium · Topic: Async

The interviewer's prompt: "In a `CompletableFuture` pipeline, `.exceptionally(...)` or `.handle(...)`?"

Thinking out loud: 1. "exceptionally fires only on failure — recovery value is the new result." 2. "handle fires on both success and failure — `(value, error)` lambda. More flexible, more verbose."

The defensible answer: - *exceptionally for pure recovery — convert a failure to a fallback value.* - *handle when you want to do cleanup or transformation regardless of success/failure.* - *whenComplete for side-effect only (logging) without changing the result.*

The tradeoff: - *exceptionally is concise but only addresses failure.* - *handle is general but the lambda has to deal with nullable params.*

When the OTHER option wins: - *Pure success transformation + fallback → thenApply + exceptionally.* - *Need to log on both paths → handle or whenComplete.*

What NOT to say: - *"They're interchangeable" — different signatures, different intent.* - *"Just use handle for everything" — clutters simple recovery cases.*

Common follow-up: *"What's the difference between whenComplete and handle?"*

Scenario 93 · `synchronized` block vs `synchronized` method

Asked at: Most Java interviews · **Difficulty:** Medium · **Topic:** Concurrency

The interviewer's prompt: "`synchronized` on the method vs `synchronized (someLock) { ... }` block?"

Thinking out loud: 1. *"synchronized on a method locks `this` (instance methods) or the Class object (static methods). Coarse-grained, holds the monitor for the entire method body."* 2. *"synchronized block lets you choose the monitor object and narrow the scope to just the critical section."*

The defensible answer: - *Block on a dedicated `private final Object lock = new Object();` field. Fine-grained, no external code can acquire the same monitor accidentally.* - *synchronized method is OK for short methods where the entire body is the critical section AND you don't expose the lock object externally.*

The tradeoff: - *Block requires creating a lock field — slight ceremony.* - *Method-level locking on `this` lets outside code interfere by synchronizing on the same instance.*

When the OTHER option wins: - *Trivial setter with everything-is-critical* → *synchronized method is fine.* - *Anything more complex* → *dedicated lock field with block.*

What NOT to say: - *"synchronized method is the safe default"* — *exposes the lock object (this) accidentally.* - *"Always use a separate lock object"* — *sometimes synchronized method is genuinely fine.*

Common follow-up: "Why is `synchronized (someStringConstant)` a bad idea?"

Scenario 94 · Functional interface: built-in vs custom

Asked at: Walmart · TCS Digital · **Difficulty:** Easy · **Topic:** Java 8+

The interviewer's prompt: "For a lambda that takes a User and returns a Boolean, define a custom `UserChecker` interface or use `Predicate<User>`?"

Thinking out loud: 1. "Predicate is a built-in standard interface — composability with `.and()`, `.or()`, `.negate()` for free." 2. "Custom named interface communicates intent (UserChecker is more readable than Predicate in some contexts)."

The defensible answer: - *Built-in functional interfaces (Predicate, Function, Consumer, Supplier) by default — standard, composable, fits Stream API.* - *Custom named interface when the type carries domain meaning beyond what built-ins convey (e.g. `RetryPolicy` vs `Function<Attempt, Decision>`).*

The tradeoff: - *Custom interfaces don't compose with built-in `.and()` / `.or()`.* - *Built-ins can look generic in domain contexts.*

When the OTHER option wins: - *Domain abstraction with semantic name* → *custom.* - *Generic transformation* → *built-in.*

What NOT to say: - *"Always custom for clarity"* — *loses composability and standard tooling.* - *"Always built-in"* — *sometimes a named interface communicates better.*

Common follow-up: "What does `@FunctionalInterface` actually enforce?"

Scenario 95 · Java sealed classes vs final + visitor pattern

Asked at: Cred · Atlassian · **Difficulty:** Hard · **Topic:** Java 17 features

The interviewer's prompt: "You want a closed hierarchy of types. Sealed classes/interfaces (Java 17+) or the classic final + visitor pattern?"

Thinking out loud: 1. "Sealed classes let you say 'these are the only subclasses' — compiler enforces exhaustiveness in switch." 2. "Visitor pattern simulates this in pre-17 Java but requires lots of boilerplate."

The defensible answer: - Java 17+: sealed interfaces with record implementations + pattern matching switch. Compiler-enforced exhaustiveness, no boilerplate. - Pre-17 or where sealed isn't supported: visitor pattern still works.

The tradeoff: - Sealed classes require all permitted subclasses to be declared (or in the same module). - Visitor pattern is awkward — every new operation adds a visit method.

When the OTHER option wins: - Stuck on Java 8/11 → visitor. - Need to add operations without modifying the hierarchy → visitor still has its place for double-dispatch.

What NOT to say: - "Sealed classes replace visitor entirely" — they cover the case of bounded type sets; visitor still good for adding operations. - "Visitor is dead, never use it" — depends on the problem.

Common follow-up: "How does pattern matching with sealed types give exhaustiveness checking?"

Scenario 96 · Stream collect vs reduce

Asked at: Walmart · TCS Digital · **Difficulty:** Medium · **Topic:** Streams

The interviewer's prompt: "For aggregating a Stream into a List, `.collect(Collectors.toList())` or `.reduce(...)`?"

Thinking out loud: 1. "collect uses a mutable accumulator — perfect for building collections, strings, maps." 2. "reduce produces an immutable result — perfect for sums, products, finding extremes."

The defensible answer: - collect for building collections or strings — the API is purpose-built (Collectors.toList(), joining, groupingBy). - reduce for single-value aggregations — sum, product, max, fold.

The tradeoff: - reduce with mutable state is a footgun in parallel streams (results merged incorrectly). - collect's API has a learning curve.

When the OTHER option wins: - Single scalar aggregation → reduce. - Anything collection-shaped → collect.

What NOT to say: - "They're the same, use whichever" — reduce with mutable state breaks under parallelStream. - "Always collect" — overkill for a simple sum.

Common follow-up: "What's the difference between Collectors.toList() and Stream.toList() (Java 16+)?"

Scenario 97 · Apache HttpClient vs JDK HttpClient (Java 11+)

Asked at: Walmart · Cred · Difficulty: Medium · Topic: HTTP

The interviewer's prompt: "For HTTP calls in Java code, Apache HttpClient or the built-in `java.net.http.HttpClient` (Java 11+)?"

Thinking out loud: 1. "JDK HttpClient is built-in — no extra dependency, supports HTTP/2 and async out of the box." 2. "Apache HttpClient is mature with a huge feature set (connection pooling, classic interceptors, decades of fine-tuning)."

The defensible answer: - JDK HttpClient for new code — no extra dependency, modern API, HTTP/2 native. - Apache HttpClient when you need specific features (custom auth schemes, classic interceptor models, integration with libraries that expect it).

The tradeoff: - JDK HttpClient is newer and the ecosystem isn't as deep — fewer Stack Overflow answers. - Apache is heavyweight; one more JAR.

When the OTHER option wins: - Legacy integration expecting Apache HttpClient → keep it. - New code → JDK HttpClient.

What NOT to say: - "Apache is always better, it's mature" — JDK is mature enough now for most cases. - "Always use Spring's WebClient / RestClient" — for non-Spring code, JDK HttpClient is the natural choice.

Common follow-up: "How does JDK HttpClient handle async vs sync calls?"

Scenario 98 · Soft reference cache vs Caffeine

Asked at: Atlassian · Walmart · Difficulty: Hard · Topic: Caching

The interviewer's prompt: "For an in-memory cache, SoftReference values or Caffeine?"

Thinking out loud: 1. "SoftReference relies on GC to evict — opaque, JVM-dependent, hard to reason about." 2. "Caffeine has explicit eviction policies (size, time, weight) — predictable, observable."

The defensible answer: - Caffeine. Explicit policies, metrics, well-supported. - SoftReference is a legacy approach — GC behavior under different collectors (G1, ZGC) varies and surprises happen.

The tradeoff: - Caffeine adds a small dependency. - SoftReference is built-in but its behavior under GC is unpredictable.

When the OTHER option wins: - Need zero dependencies and don't care about precise control → SoftReference still works. - Otherwise, Caffeine.

What NOT to say: - "SoftReference is just as good" — modern GCs evict at unpredictable times. - "Always implement your own cache" — wastes time; Caffeine is excellent.

Common follow-up: "What's the difference between WeakReference and SoftReference for caching?"

Scenario 99 · Synchronous file I/O vs NIO with channels

Asked at: Walmart · Atlassian · **Difficulty:** Hard · **Topic:** I/O

The interviewer's prompt: "Reading a large file. java.io stream APIs or NIO with FileChannel + ByteBuffer?"

Thinking out loud: 1. "java.io stream APIs are simple — `Files.lines(...)`, `BufferedReader.readLine()`." 2. "NIO channels allow memory-mapped files for very large reads and finer buffer control."

The defensible answer: - `Files.lines` / `BufferedReader` for normal file reads — simple, fast, handles encoding. - NIO with `FileChannel` for memory-mapped scenarios (multi-GB log scanning), zero-copy transfers (`FileChannel.transferTo`).

The tradeoff: - NIO is more code, more concepts (buffers, channels, charsets). - java.io can be a bottleneck for huge files due to per-byte overhead.

When the OTHER option wins: - Normal text file → java.io. - Massive binary file or zero-copy needs → NIO.

What NOT to say: - "NIO is always faster" — for small files, the difference is negligible. - "Never use NIO, too complex" — it's the right tool for specific jobs.

Common follow-up: "What's the advantage of `FileChannel.transferTo` over manual copy?"

Scenario 100 · jOOQ vs JPA for query-heavy services

Asked at: Cred · Razorpay · **Difficulty:** Hard · **Topic:** Persistence

The interviewer's prompt: "For a service that's mostly complex queries (reports, search, aggregation), jOOQ or JPA?"

Thinking out loud: 1. "JPA is great for entity-centric CRUD; less great for ad-hoc queries with projections, joins, and window functions." 2. "jOOQ generates a type-safe Java DSL from your DB schema — SQL-first with compile-time safety."

The defensible answer: - jOOQ for query-heavy services — type-safe SQL, vendor-feature access, refactor-safe column references. - JPA for entity-centric

transactional services with CRUD and unit-of-work patterns. - Mix is fine: jOOQ for reads, JPA for writes.

The tradeoff: - jOOQ requires DB-driven code generation in the build pipeline. - JPA's abstraction hides SQL — both blessing and curse.

When the OTHER option wins: - Standard CRUD with simple finders → JPA + Spring Data is faster to build. - Complex query layer with vendor-specific SQL → jOOQ.

What NOT to say: - "jOOQ replaces JPA entirely" — different sweet spots. - "Always JPA, it's the standard" — for query-heavy workloads, jOOQ wins clearly.

Common follow-up: "How does jOOQ handle DB migrations alongside its generated code?"
